

FROM
SCRATCH

BUILD A

Frontend Web Framework

Angel Sola Orbaiceta

MEAP



MANNING

Build a Frontend Web Framework (From Scratch)

1. [Welcome](#)
2. [Part 1](#)
3. [1 Are frontend frameworks magic to you?](#)
4. [2 Vanilla JavaScript—like in the old days](#)
5. [Part 2](#)
6. [3 Rendering and the virtual DOM](#)
7. [4 Mounting and destroying the virtual DOM](#)
8. [5 State management and the application's lifecycle](#)
9. [6 Publishing and using your framework's first version](#)
10. [7 The reconciliation algorithm: diffing virtual trees](#)
11. [8 The reconciliation algorithm: patching the DOM](#)
12. [Part 3](#)
13. [9 Stateful components](#)
14. [10 Component methods](#)
15. [11 Subcomponents—Communication via props and events](#)
16. [12 Keyed lists](#)
17. [13 The component lifecycle hooks and the scheduler](#)
18. [14 Testing asynchronous components](#)
19. [Appendix A. Setting up the project](#)

Welcome

Thank you for purchasing the MEAP for *Build a Frontend Web Framework From Scratch*. I hope you enjoy reading this book as much as I've enjoyed writing it. I'm looking forward to you getting the same feeling of accomplishment as I did when you see your own frontend framework—written from the ground up—power web applications. There's something magical about building your own tools and seeing them working.

Throughout the book, you'll learn the most important concepts behind what makes frontend frameworks such useful tools, but not through lots of theory, but by writing all the code yourself. To make the most out of this book, I'll assume that you've got a decent understanding of JavaScript, Node JS and the Document API in the browser. You'll start by writing an application using just JavaScript, without a framework. From that exercise, you'll identify what are the pain points of writing applications without the help of a framework. It's crucial that you understand the need for them in the first place; after all, we take them for granted these days. You'll then create your own simple framework to address what you'll discover is the main problem with vanilla JavaScript applications: the mix of DOM manipulation and business logic code.

After just two chapters (chapters three and four) you'll have a working framework, a very simple one, but a framework that you can use in small applications nevertheless. From there onwards, you'll keep adding improvements to it, such as a Single Page Application (SPA) router that changes the page without reloading it, or a Webpack loader to transform HTML templates into render functions.

By the end of the book, you'll have written a pretty decent framework of your own. Not only can you use this framework for your own projects (something that's extraordinarily satisfactory), but you'll share it with the rest of the world via NPM. You heard me well! You will be publishing it the same way the big frameworks get released. Sure, the framework won't be even close to competing with the big frameworks out there, but you'll have a good

understanding of how they work that you can use to keep improving yours. But more important than creating a framework that can compete is the amount of fun, gratification and learning that you'll get from working on this project.

There's one last thing I want to mention. I'm writing this book for you, so it doesn't matter how much effort I put into it if my explanations aren't clear enough for you to follow. If you get lost, it's definitely not because you don't know enough, it's because I'm not expressing my thoughts as clearly as I could. So, if you do get lost or think I could be explaining things in more detail, I'd love to hear from you. Conversely, if you think I spend too much time explaining things you already know, also let me know. You, as a MEAP reader, have a very important role in making this book useful for the readers that'll come after you. Oh! and if you're enjoying the book as is, please do tell me as well. Your feedback is essential in developing the best book possible, so please visit the [liveBook's Discussion Forum](#) and leave your comments.

—Angel Sola Orbaiceta

In this book

[Welcome](#) [Part 1](#) [1 Are frontend frameworks magic to you?](#) [2 Vanilla JavaScript—like in the old days](#) [Part 2](#) [3 Rendering and the virtual DOM](#) [4 Mounting and destroying the virtual DOM](#) [5 State management and the application's lifecycle](#) [6 Publishing and using your framework's first version](#) [7 The reconciliation algorithm: diffing virtual trees](#) [8 The reconciliation algorithm: patching the DOM](#) [Part 3](#) [9 Stateful components](#) [10 Component methods](#) [11 Subcomponents—Communication via props and events](#) [12 Keyed lists](#) [13 The component lifecycle hooks and the scheduler](#) [14 Testing asynchronous components](#)
[Appendix A. Setting up the project](#)

Part 1

1.1 No Framework

This first part comprises two introductory chapters. The first chapter offers a high-level overview of frontend frameworks, providing a foundational understanding without delving into specifics.

In the second chapter, you'll embark on a hands-on journey to create a TODO application using vanilla JavaScript—no framework assistance. If you've never experienced building an interactive frontend application without a framework, this exercise will provide insight into the intricacies of connecting the DOM and JavaScript code manually. You'll also appreciate the challenge of programmatically modifying the DOM whenever the application's state changes.

The essence of a framework lies in automating these tasks, allowing you to concentrate on your application's logic. By mastering the manual process, you'll gain a deeper appreciation of the indispensable role a framework plays in your development workflow.

1 Are frontend frameworks magic to you?

This chapter covers

- Why you should build your own frontend framework
- The features of the framework we'll build together
- The big picture of how frontend frameworks work

How you ever wondered how the frontend frameworks you use work internally? Are you curious about how they decide when to re-render a component, and how they do to only update the parts of the DOM that change? Isn't it interesting how a single HTML page can change its content without reloading? That the URL in the browser's address bar changes without requesting the new page to a server? The inner workings of frontend frameworks are fascinating, and there is no better way to learn about them than by building one yourself, from scratch. But why would you want to learn how frontend frameworks work? Is't it enough to just know how to use them? — you might ask.

Good cooks know their tools; they can use knives skillfully. Great chefs go beyond that: they know the different types of knives, when to use each one, and how to keep their blade sharp. Carpenters know how to use a saw to cut wood, but great carpenters also understand how the saw works and can fix it in case it breaks. Electrical engineers not only understand that electricity is the flow of electrons through a conductor, but also have a deep understanding of the instruments they use to measure and manipulate it; for example, they could build a multimeter themselves. If their multimeter breaks, they can disassemble it, figure out what's wrong, and repair it.

As a developer, you have a frontend framework in your toolbox, but do you know how it works? Or, is it magic to you? If it broke—if you found a bug in it—would you be able to find its source and fix it? When a single-page application changes the route in the browser's URL bar and renders a new

page without requesting it from the server, do you understand how that happens?

1.1 Why build your own frontend framework?

The use of frontend frameworks is on the rise; it's uncommon to write pure-vanilla JavaScript applications nowadays, and rightly so. Modern frontend frameworks boost productivity and make building complex interactive applications a breeze. Frontend frameworks have become so popular that there are jokes about the unwieldy amount of them at our disposal. (In the time it took you to read this paragraph so far, a new frontend framework was created.) Some of the most popular frontend frameworks even have their own fan groups, with people arguing about why theirs is the best. Let's not forget that frontend frameworks are tools, a means to an end, and that the end is to build applications that solve real problems.

Framework vs. library

Frameworks and libraries are different. When you use a library, you import its code and call its functions. When you use a framework, you write code that the framework executes. The framework is in charge of running the application, and it executes your code when appropriate trigger happens. Conversely, you call the library's functions when you need them. Angular is an example of a frontend framework, whereas React claims to be a library that you can use to build UIs.

For convenience, I'll refer to both frameworks and libraries as *frameworks* in this book.

The most popular frameworks currently available, such as Vue, Svelte, Angular, and React, are all exceptional. You might be wondering, **why create your own frontend framework with so many great options already available?** Well, aside from the satisfaction and enjoyment of building a complex software piece from scratch, there are practical reasons to consider.

Let me tell you a little story about a personal experience to illustrate this.

1.1.1 "Do you understand how that works?"

When I was a little kid I went to one of my cousins' house to hang out. He was a few years older than me and a handyman. His cabinets were full of cables, screwdrivers, and other tools, and I'd spend hours just observing how he fixed all kinds of appliances. I remember once bringing a remote control car with me so we could play with it. He stared at it for some time, then asked me a question that got me by surprise: "Do you understand how this thing works?" I didn't; I was just a kid with zero electronics knowledge. He then said, "I like to know how the stuff I use works, so what do you say we take it apart and see what's inside?" I sometimes still think about that.

So now, let me ask you a question similar to that my cousin asked me: you use frontend frameworks every day, but do you really understand how they work? You write the code for your components, then hand it over to the framework for it to do its magic. When you load the application into the browser, it just works. It renders the views and handles user interactions, always keeping the page updated (in sync with the application's state). For most frontend developers—and this includes me from years ago—how this happens is a mystery. Is the frontend framework you use a mystery to you?

Sure, most of us have heard about that thing called the "virtual DOM," and that there needs to be a "reconciliation algorithm" that decides what is the smallest set of changes required to update the browser's DOM. We also know that single-page applications (SPAs for short) modify the URL in the browser's address bar without reloading the page, and if you're the curious kind of developer, you might have read about how the browser's [history API](#) is used to achieve this. But do you really understand how all of this works together? Have you disassembled and debugged the code of the framework you use? Don't feel bad if you have not; most developers haven't, including some very experienced ones. This reverse-engineering process isn't easy; it requires lots of effort (and motivation).

In this book, you and I—together as a team—will build a frontend framework from scratch. It'll be a simple one, but it'll be complete enough to understand how frontend frameworks work. From then on, what the framework does will no longer be a mystery to you. Oh! And it'll be lots of fun as well.

1.2 The framework we'll build

I like to set expectations early on, so here it goes: We won't build the next Vue or React as part of the book. You might be able to do that yourself after reading this book by filling in the missing details and optimizing a couple things here and there. The framework you'll build can't compete with the mainstream frameworks—that's not the objective anyway. The objective of this book is to teach you how these frameworks work in general, so what they do isn't magic to you anymore. You don't need to build the most advanced framework in the world to achieve this. In fact, that'd require a book four times as thick as this one, and the process of writing it wouldn't be as fun. (Did I mention that writing your own framework is fun?)

The framework we'll build borrows ideas from a few existing frameworks, most notably [Vue](#), [Mithril](#), [Svelte](#), [React](#), [Preact](#), [Angular](#) and [Hyperapp](#). Our goal is to build a framework that's simple enough to understand, but that at the same time includes the typical features you'd expect from a frontend framework. I also wanted it to be representative of some of the most relevant concepts that are behind the source code of the most popular frameworks.

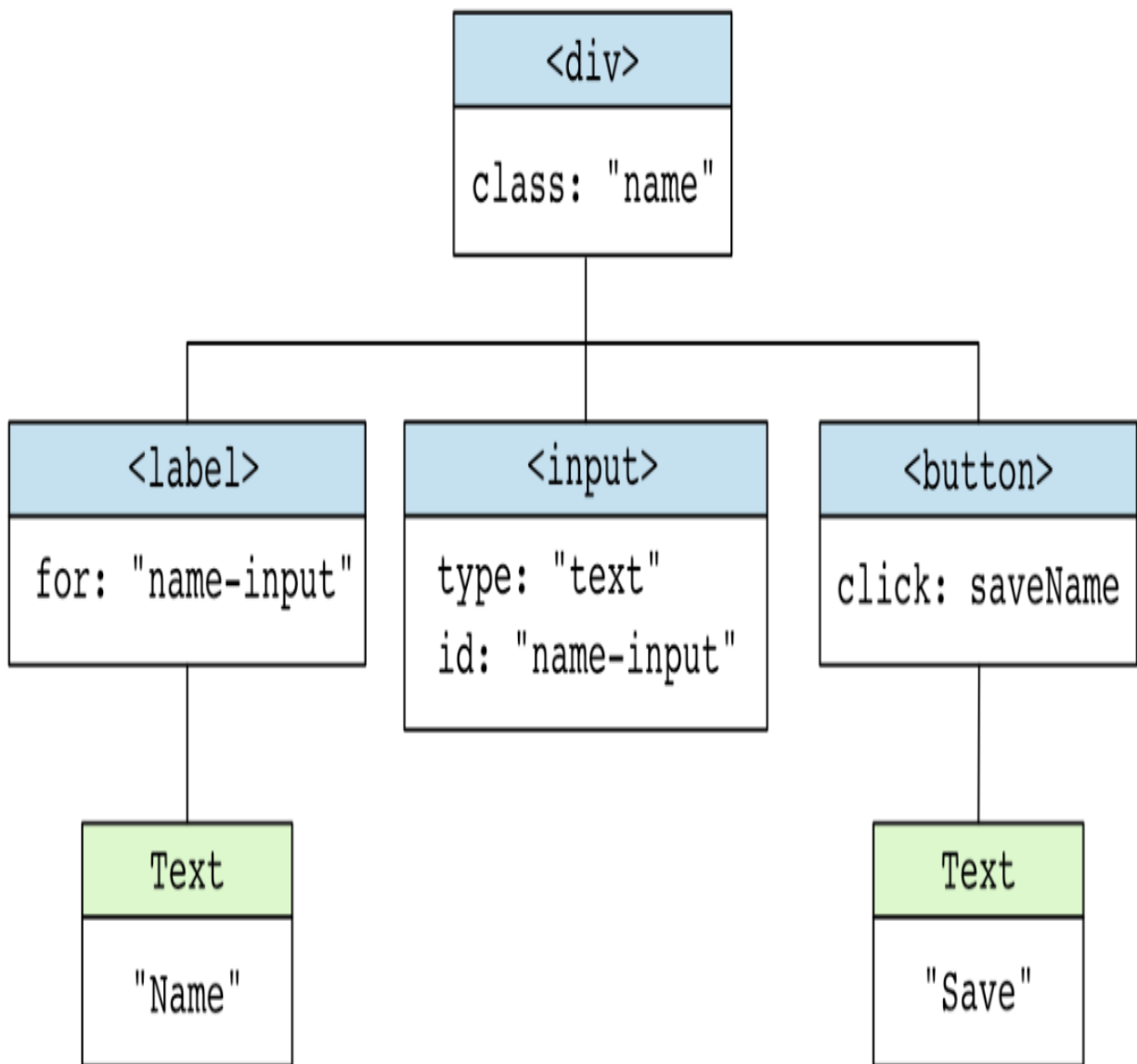
For example, not all frameworks use the virtual DOM abstraction (Svelte in particular [considers it to be "pure overhead,"](#) and the reasons are simply brilliant—I recommend you read their blog post), but a big portion of them do. I chose our framework to implement a virtual DOM so that's representative of the framework you're likely using today. In essence, I chose the approach that I thought would result in the most learning for you, the reader. I'll be covering the virtual DOM in detail in chapter [3](#), but in a nutshell, it's a lightweight representation of the DOM that's used to calculate the smallest set of changes required to update the browser's DOM. For example, the following HTML markup:

```
<div class="name">
  <label for="name-input">Name</label>
  <input type="text" id="name-input" />
  <button>Save</button>
</div>
```

would have a virtual DOM representation like that in figure [1.1](#). (Note that the `saveName()` event handler in the diagram doesn't appear in the HTML

markup: event handlers are typically not shown in the HTML markup, but added programmatically.) I'll be using these diagrams a lot throughout the book to illustrate how the virtual DOM and the *reconciliation algorithm* works. The reconciliation algorithm is the process that decides what changes need to be made to the browser's DOM to reflect the changes in the virtual DOM, which is the topic of chapters [7](#) and [8](#).

Figure 1.1 A virtual DOM representation of some HTML markup



Your framework will have some shortcomings that make it not an ideal choice for complex production applications, but definitely fit for your latest side project. For example, it'll only support the [standard HTML namespace](#), which means that SVG won't be supported. Most of the popular frameworks support this namespace, but we'll leave it out for simplicity.

1.2.1 Features

The framework you'll have after you finish the book will have the following features, which you'll build from scratch:

- A virtual DOM abstraction.
- A reconciliation algorithm that updates the browser's DOM.
- A component-based architecture where each component does the following:
 - holds its own state,
 - manages its own lifecycle,
 - re-renders itself and its children when its states changes.

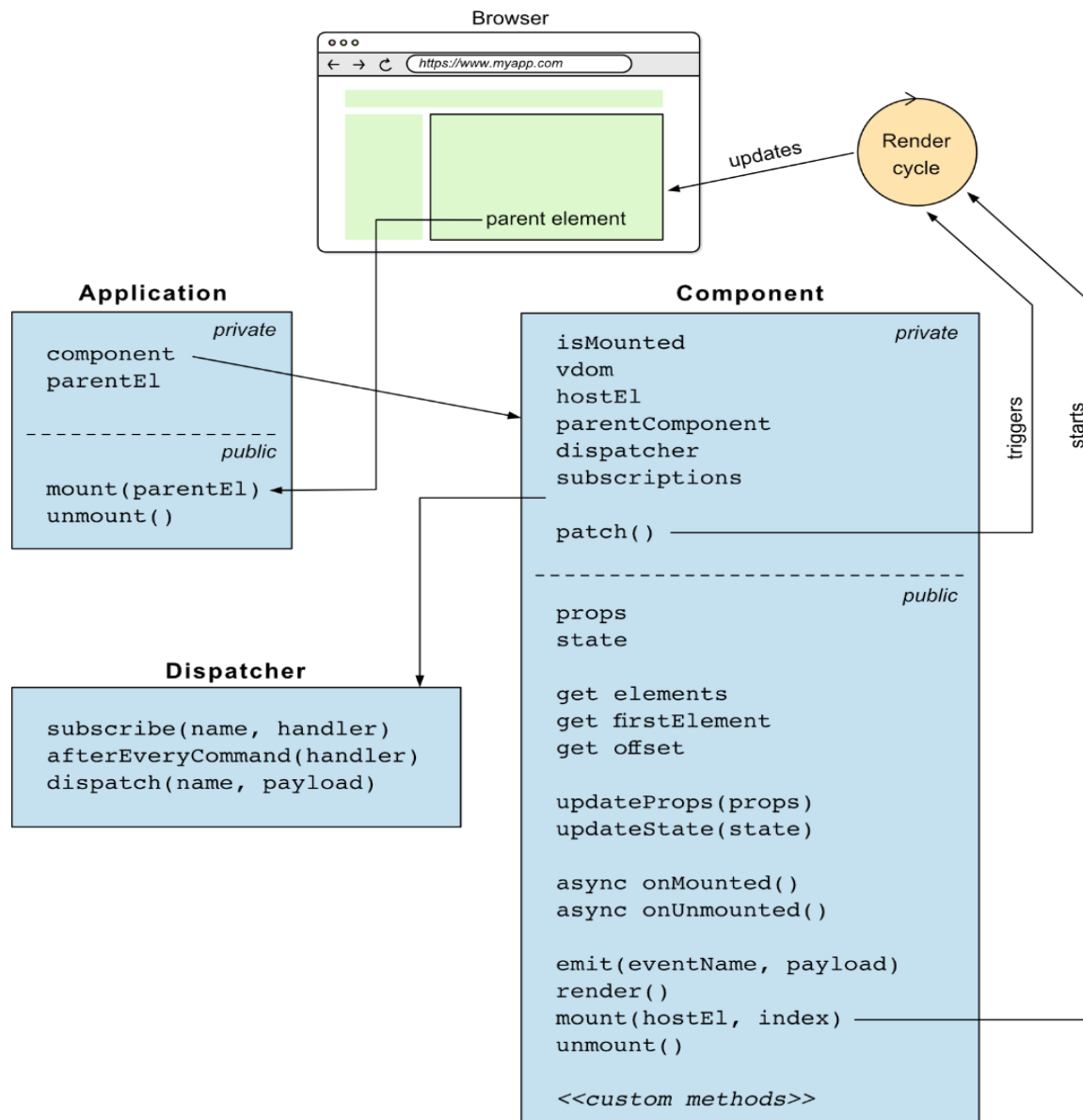
Then, if you decide you want to continue learning, I'll post some extra chapters on the book's GitHub (see Appendix A for more information) repository that'll cover the following more advanced features:

- An SPA router that updates the URL in the browser's address bar without reloading the page.
- Slots to render content inside a component.
- HTML templates that are compiled into JavaScript render functions.
- Server-side rendering.
- Building a browser extension to debug the framework.

As you can see, it's a pretty complete framework. It's not a full-blown framework like Vue or React, but it's enough to understand how they work. And the neat thing is that you'll build it line by line, so you'll understand how it all fits together. I'll use lots of diagrams and illustrations to help you understand the concepts that might be harder to grasp. I recommend you write the source code yourself as you read the book. Try to understand it line by line, take your time, debug it, and make sure you understand the decisions and trade-offs we make along the way.

Figure 1.2 shows the architecture of the framework we'll build. It includes all the parts of the framework you'll implement, and how they interact with each other.

Figure 1.2 Architecture of the framework we'll build



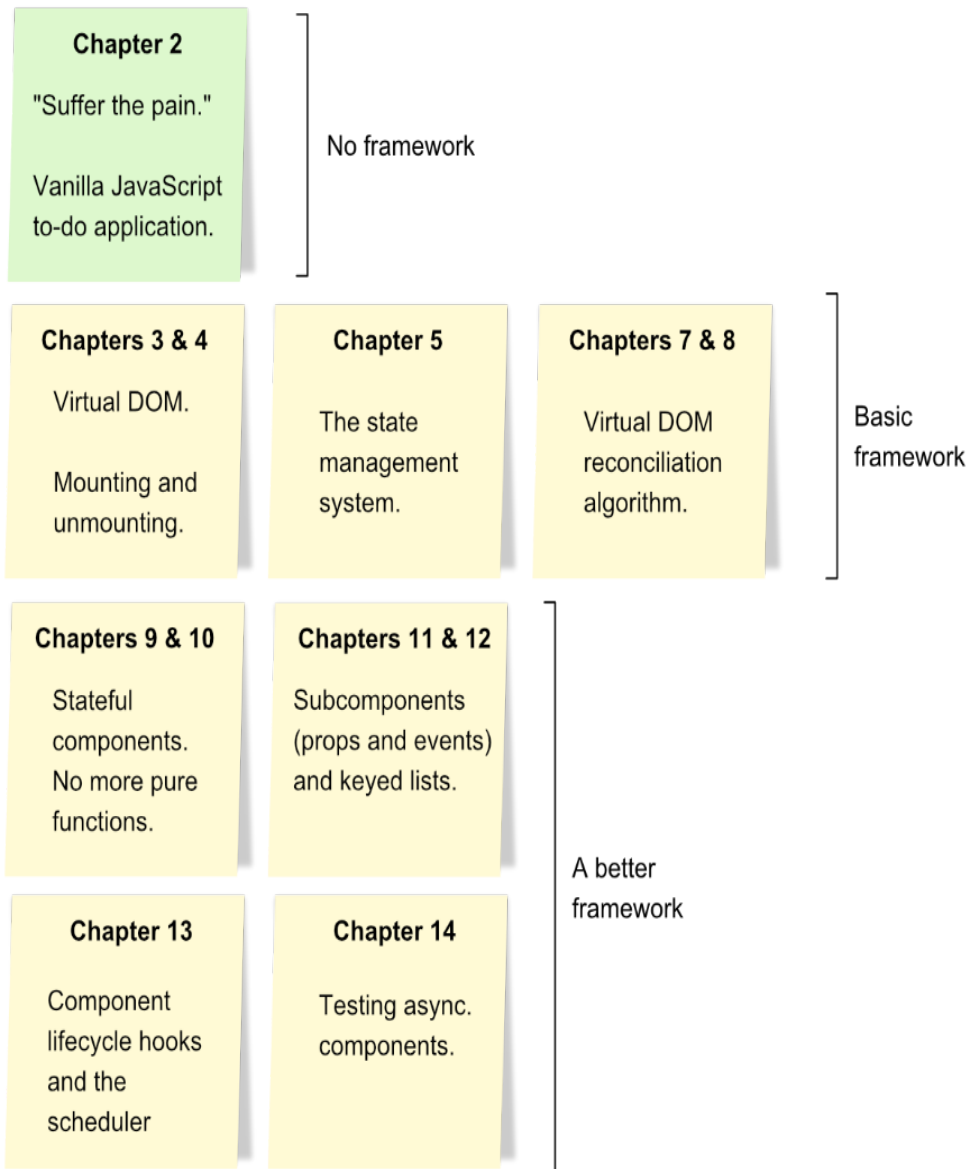
I will revisit this figure, making sure to highlight each part of the framework as we build it. You don't need to understand all the details of the architecture right now, but it's good to have a high-level understanding of it. At the end of

the book, this figure will make a lot of sense to you: you'll recognize each and every part, as you'll have built them yourself.

1.2.2 Implementation plan

As you can imagine, you can't build all of this in a single chapter. We want to break it down into smaller pieces so that you can focus on one thing at a time. Figure [1.3](#) shows the implementation plan for the framework. It resembles a kanban board, where each post-it represents the work of one or more chapters. You'll pick up a post-it in each chapter.

Figure 1.3 Implementation plan for the framework we'll build



You'll start by implementing a simple example application using just vanilla JavaScript, which will help you understand how frameworks can simplify the code of a frontend app. (Once you've suffered the "pain", you'll be in a better position to appreciate the benefits of a framework.)

Stateless components and global application state

Once you have a better understanding of the benefits of frameworks, you'll extract parts of the application's view to stateless components modeled by pure functions that return the virtual DOM representation of their view. The

application will hold the entire state of the application and pass it to the components. This allows you to focus on the DOM reconciliation algorithm—likely the most complex part of the framework.

Stateful components

Next, you'll allow components to have their own state, which makes state management much simpler. The application will no longer need to hold the entire state; it'll be split among components instead. Pure functions will turn into classes that implement a `render()` method, and each component will be its own little application with its own lifecycle.

Subcomponents

You'll then add support for subcomponents, which will allow you to split the application into smaller pieces. A component can pass information to its children via *props*, which are similar to the arguments of a function. Components can also communicate with their parents via *events* the parent can subscribe to.

Lifecycle hooks

You'll then add lifecycle *hooks* to the components, which make possible executing code at certain moments in time, like when the component is mounted into the DOM. An example of using a lifecycle hook is to fetch data from a remote server when the component is mounted.

Testing

Every application requires to be tested, so, how do you write unit tests for an application using our framework? You'll learn how to test the application's components and how to handle asynchronous events or lifecycle hooks.

Now that you know what you'll build and you have a plan, let's take a quick look at how frontend frameworks work.

1.3 Overview of how a frontend framework works

Let's quickly review how a frontend framework works when observed from the outside. (We'll learn about the internals in the next chapters.) Let's start from the side of the developer—someone using the framework to build an application. We'll then learn about the browser's perspective.

1.3.1 The developer's side

A developer starts by creating a new project with the framework's CLI (Command Line Interface) tool or by manually installing the dependencies and configuring the project. Reading the framework's documentation is important, as every framework works a little bit differently.

In a web application, developers create components that define a part of the application's view and how the user interacts with it. Components are written using HTML, CSS, and JavaScript code. Most frameworks use *single file components* (SFCs), where all the component's code lives in a single file: HTML, CSS, and JavaScript. A notable exception is Angular, which uses three files for each component: one for the HTML, one for the TypeScript code, and one for the CSS. This allows the developer to keep the languages separate and potentially get better syntax support from their IDE. However, it may be inconvenient to jump between files to see the entire component.

React and Preact use JSX—an extension of JavaScript—instead of writing HTML directly. Other frameworks, like Vue, Svelte, and Angular, use HTML templates with *directives* to add or modify the behavior of DOM elements, like iterating over and displaying an array of items or showing specific elements conditionally. For example, the following is how you'd conditionally show a paragraph in Vue:

```
<p v-if="hasDiscount">
  You get a discount!
</p>
```

The `v-if` directive is a custom directive that Vue provides to conditionally show an element. Other frameworks use slightly different syntaxes, but all of them provide the developer with a way to show or hide elements based on the

application's state. Just for the sake of comparison, here's how you'd do the same in Svelte:

```
{#if hasDiscount}  
<p>  
  You get a discount!  
</p>  
{/if}
```

And this would be how you'd write it in the case of React:

```
{hasDiscount && <p>You get a discount!</p>}
```

Once the developer is satisfied with the application, the code needs to be bundled into fewer files than were originally written, so the browser can load the application using fewer requests to the server. The files can also be *minified*, that is, made smaller by removing whitespace and comments, and renaming variables to shorter names. This process of turning the application's source code into the files that are shipped to the users is called *building*.

Building the application

To deploy a frontend application to production, we first need to build it. Most of the work of building an application using a specific framework is done by the framework itself. The framework—typically—provides a CLI tool that we can use to build the application by running a simple NPM script such as `npm run build`.

Note

There are many different ways an application can be built, resulting in a wide variety of bundle formats. Here, I'll explain a build process that encapsulates some of the most common practices.

Building the application includes a few steps:

1. The template for each component is transformed—by the template compiler—into JavaScript code that, executed in the browser, creates the component's view.

2. The components' code—split in multiple files—is transformed and bundled into a single JavaScript file, *app.bundle.js*. (For larger applications it's common to have more than one bundle and *lazy-load* them; that is, load them only when they'll become visible to the user.)
3. The third-party code used by the application is bundled into a single JavaScript file, *vendors.bundle.js*. This file includes the code for the framework itself, along with other third-party libraries.
4. The CSS code in the components is extracted and bundled into a single CSS file: *bundle.css*. (Same as before, larger applications may have more than one CSS bundle.)
5. The HTML file that will be served to the user (*index.html*) is generated or copied from the static assets directory.
6. The static assets (such as images, fonts or audio clips) are copied to the output directory. They can optionally be preprocessed, for example, to optimize images or convert audio files to a different format.

So, a typical build process results in four (or more, in the case of larger apps) files:

- *app.bundle.js* with the application's code,
- *vendors.bundle.js* with the third-party code,
- *bundle.css* with the application's CSS, and
- *index.html*, the HTML file that will be served to the user.

These files are uploaded to a server, and the application is ready to be served to the user. When a user requests the website, the HTML, JS, and CSS files are statically served.

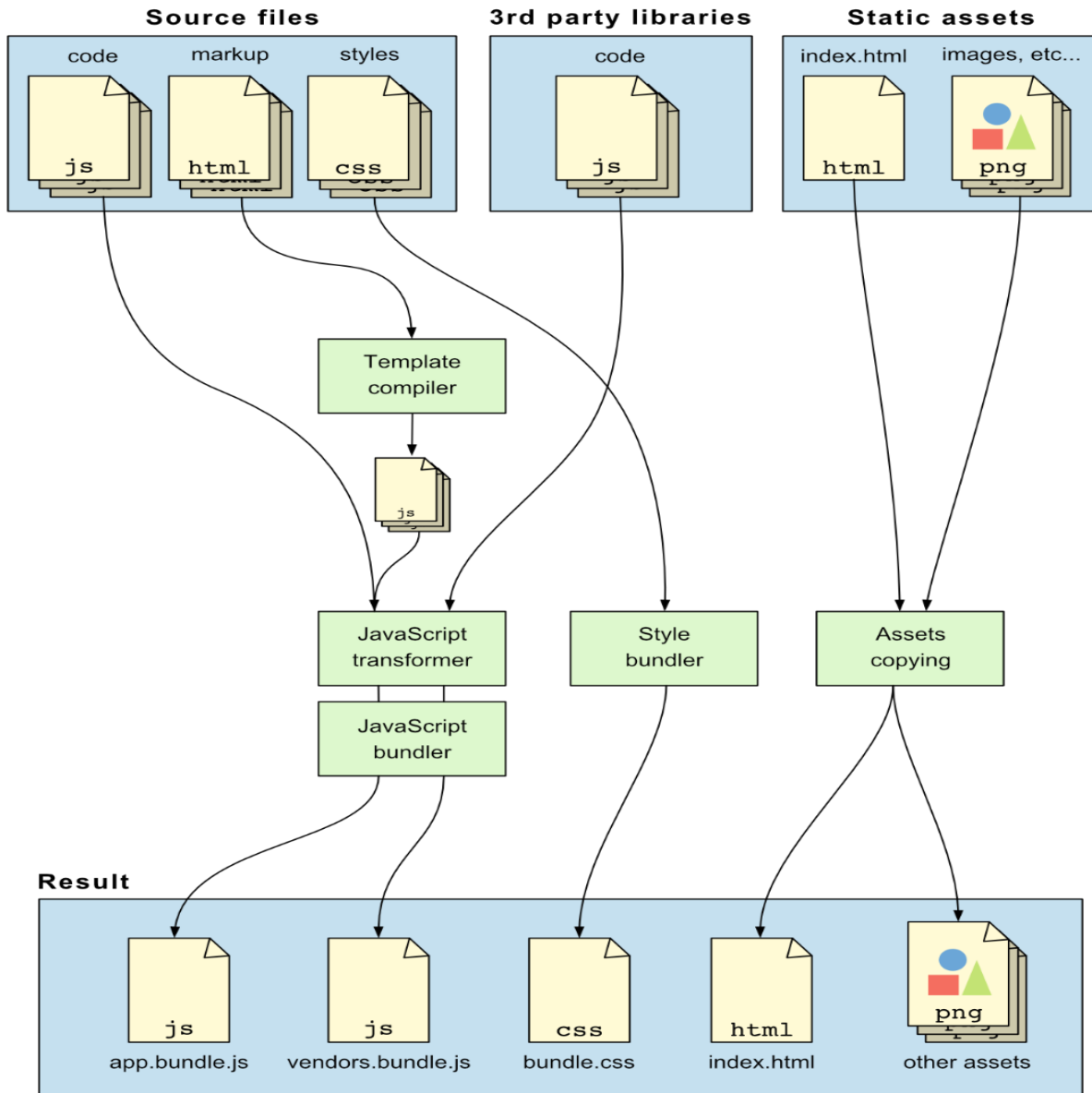
Note

When a file is *statically served*, the server doesn't need to do anything before sending it to the user. The server simply reads the file from disk and sends it. In contrast, when the application is rendered on the server, the server generates the HTML file before sending it to the user's browser.

Figure [1.4](#) shows a diagram of the build process I just described. Note that, a typical build process is more complex than the one shown in the figure, but this is enough to understand the concepts. I've included a step that transforms

the JavaScript code. This is a generic step that refers to any transformation that needs to be done to the code before it's bundled, like for example transpiling it using [Babel](#) or [TypeScript](#).

Figure 1.4 A simplified diagram of a frontend application's build process



Let's see what happens in the browser once these files are loaded. The flow is slightly different, depending on whether the application is rendered on the server (SSR) or statically served as a single page application (SPA). Let's start with the slightly simpler latter case.

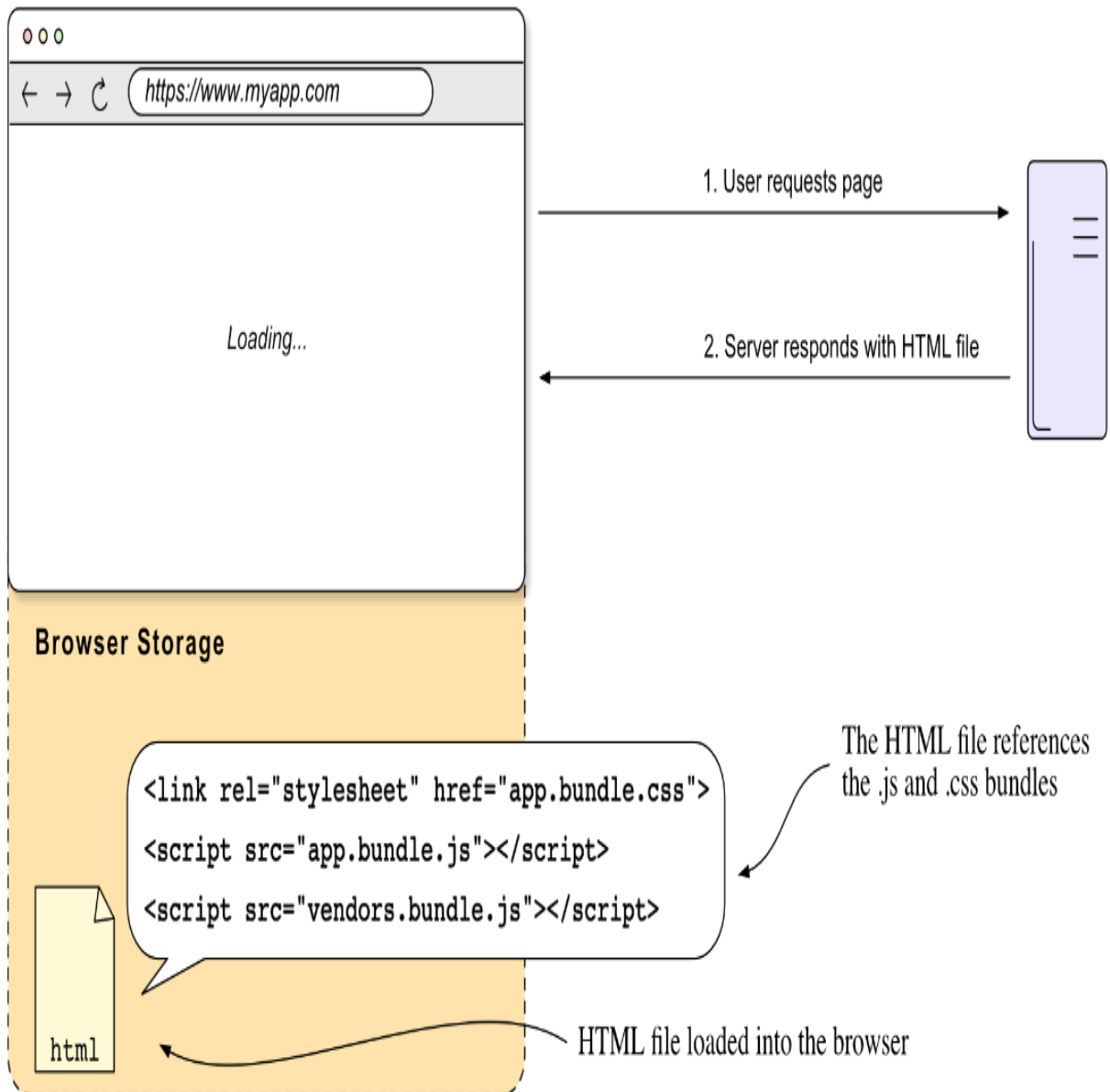
1.3.2 The browser side of an SPA

In an SPA, the server responds with a—mostly empty—HTML file that's used to load the application's JavaScript and CSS files. The framework then creates and updates the application's view using the Document API. A router makes sure not to reload the entire application when the user navigates to a different URL, but rather updates the view to show the new content. Oh, and it also updates the URL in the browser's address bar to give the user a nice experience. Let's see this process step by step.

Step 1: Loading the HTML file

When the user navigates to the application by writing its URL, the browser requests the page's HTML file (1), which is returned by the server (2), as illustrated in figure [1.5](#).

Figure 1.5 Single-page application requesting the—mostly empty—HTML page

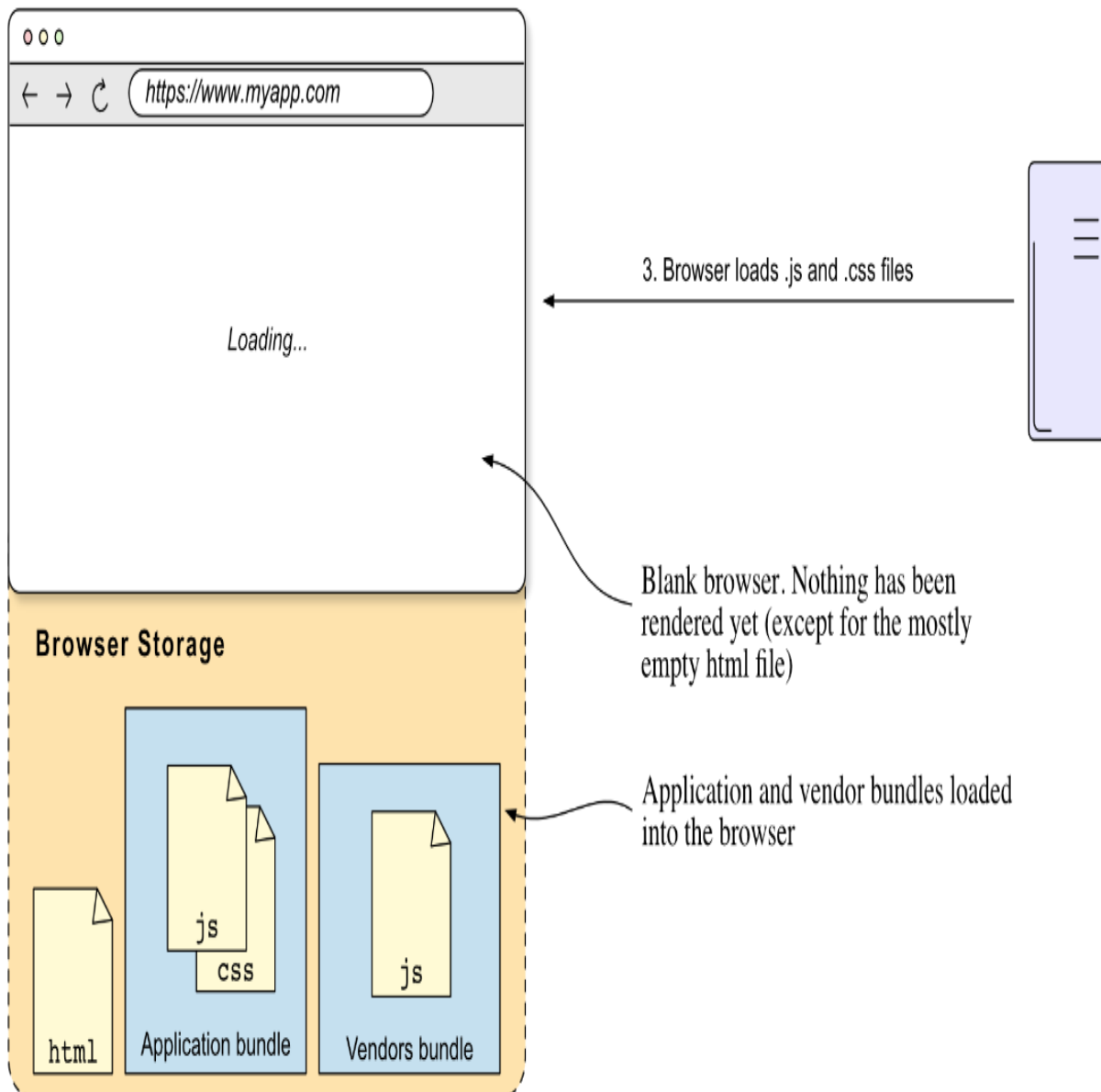


The browser loads the HTML file and parses it. This HTML is mostly empty and is used to load the JavaScript and CSS bundles declared in the `<script>` and `<link>` tags. These are the application and vendor bundles we talked about in the previous section.

Step 2: Loading the JavaScript and CSS files

The browser loads the JavaScript and CSS files referenced in the HTML file (3) and parses the JavaScript code. This is depicted in figure [1.6](#).

Figure 1.6 The browser loads the JavaScript and CSS files referenced in the HTML

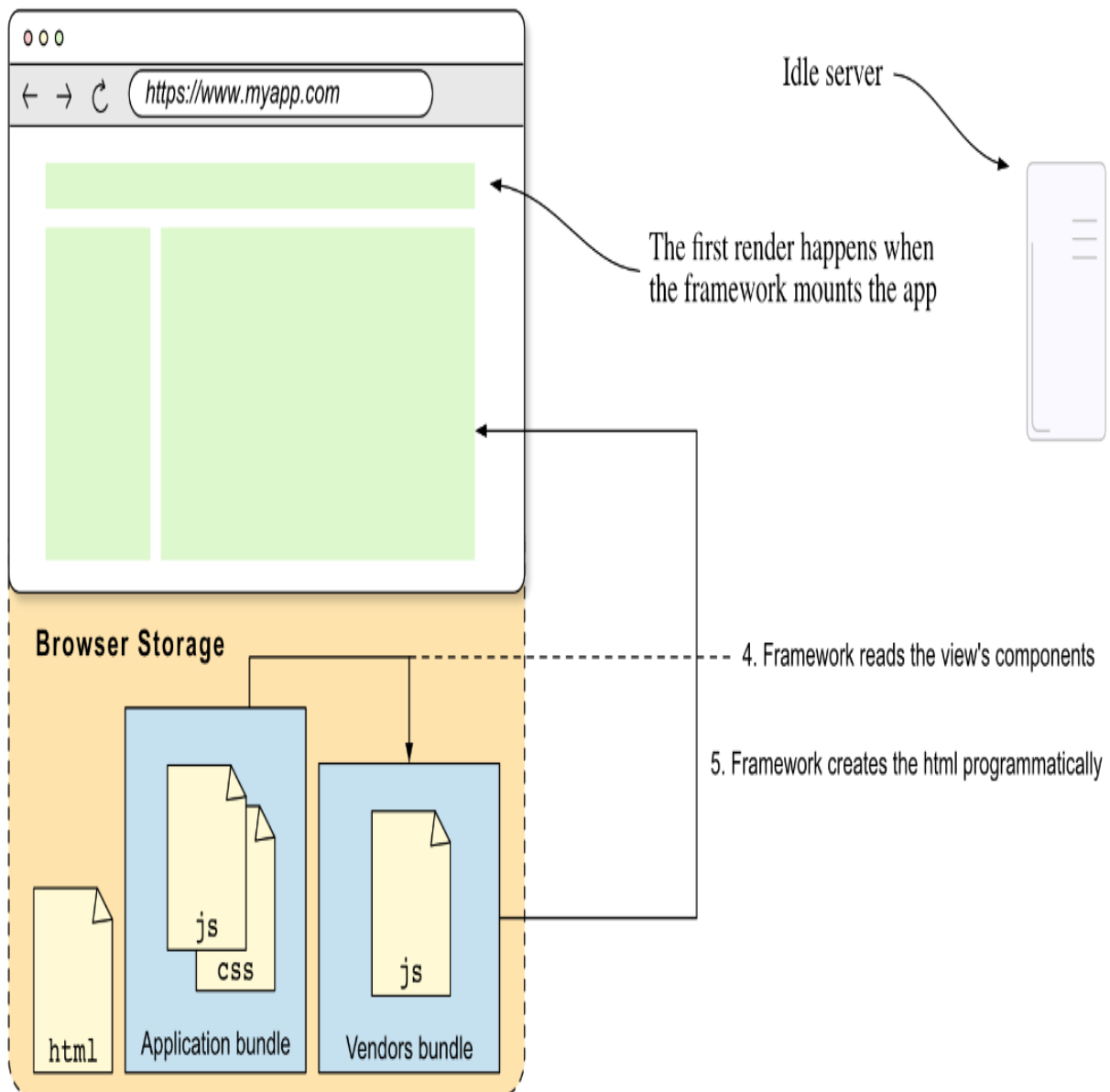


The browser is still blank at this point. It has rendered the HTML file, but this file's `<body>` element is mostly empty (except maybe for a `<div id="app">` tag that some frameworks use to render the app). The view of an SPA is created dynamically by the framework's JavaScript code, which is what happens next.

Step 3: Creating the application's view (mounting the app)

The framework JavaScript code (living in the vendors bundle) finds the components defined in the application's code that need to be rendered (4) and creates the application's view (5). This is depicted in figure 1.7. This initial rendering is called *mounting* the application.

Figure 1.7 The framework creates the application's view using the Document API



The framework uses the Document API to create the application's HTML programmatically. The Document API allows the creation of HTML elements

programmatically, using JavaScript. Let's take a quick detour to see how this works.

For example, given an empty HTML `<body>` element like the following:

```
<body></body>
```

A paragraph can be programmatically created and appended to the document like so:

```
const paragraph = document.createElement('p')
paragraph.textContent = 'Hello, World!'
document.append(paragraph)
```

This would result in the following HTML:

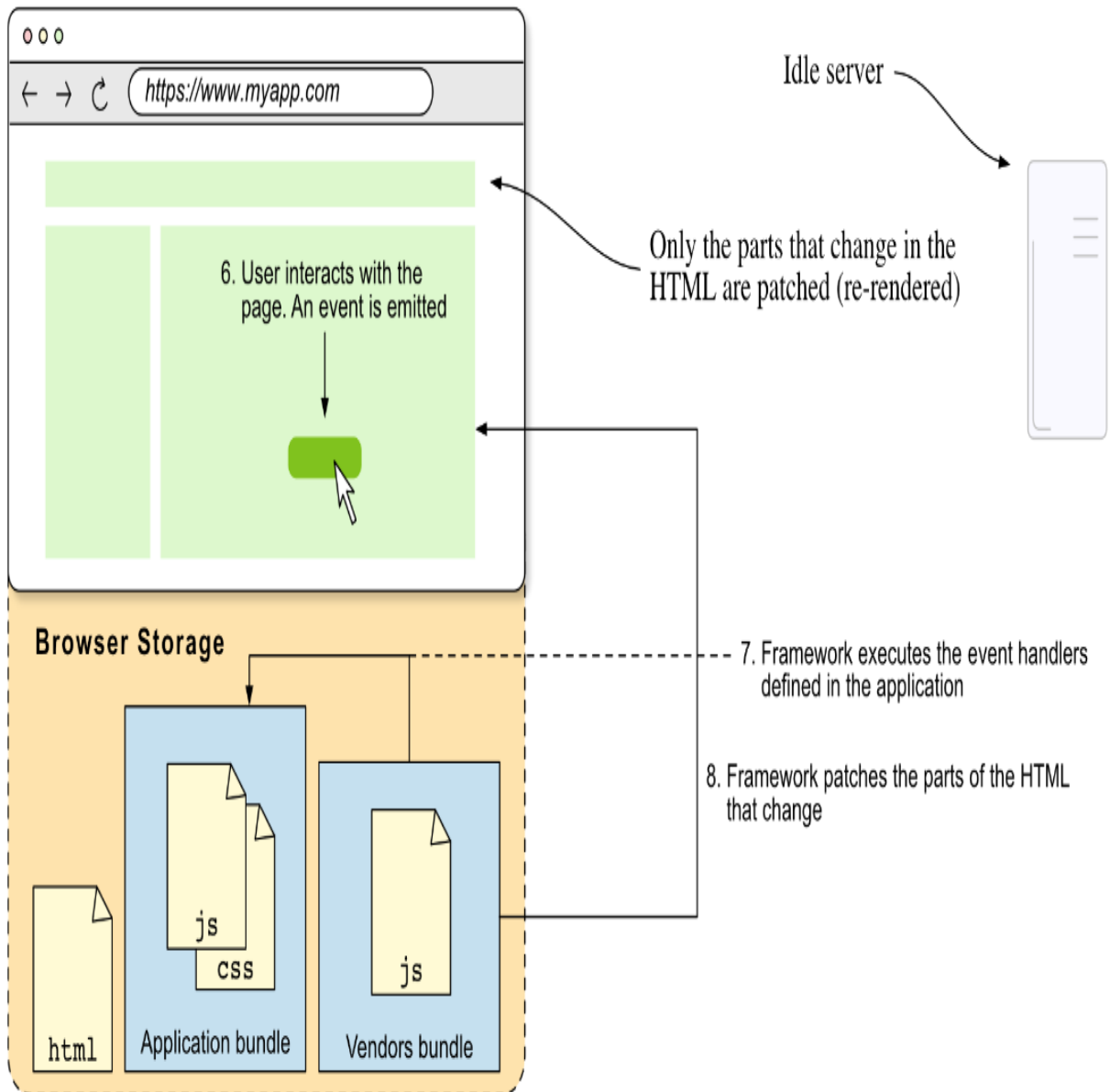
```
<body>
  <p>Hello, World!</p>
</body>
```

Going back to how SPAs work: what happens when the user interacts with the application?

Step 4: Handling user interactions

When the user interacts with the application (6), the framework handles the event and updates the view accordingly. This is depicted in figure [1.8](#). The framework handles the event from the browser by executing the event handling code defined in the application's code (7) and then updating the view to reflect the changes in the application's state (8).

Figure 1.8 SPA handling user interactions



The framework is responsible for updating only the parts of the HTML that need to be updated, a process that we call *patching* the DOM.

Patching the DOM

A single change made to the DOM by the framework is called a *patch*. The process of updating the view to reflect the changes in the application's state is called *patching the DOM*.

Making changes to the document is expensive, so a well-implemented framework minimizes the number of changes to the document for it to reflect the required updates. By expensive, I mean that the browser needs to repaint and reflow the document to reflect the changes, and that consumes resources.

To better understand why the changes to the DOM are expensive in terms of computation, I recommend you read web.dev/critical-rendering-path-render-tree-construction. This article explains how the browser renders the document, and gives you an overview of everything that happens under the hood.

How this is achieved varies heavily across frameworks. Some frameworks use a virtual DOM—like yours will—to compare the current state of the document with the desired state and apply only the necessary changes, and some use a completely different approach. To learn more about how some frameworks update the view, check the sidebar note.

How some frameworks update the view

Svelte understands the ways the view can be updated at compilation time, and produces JavaScript code to update the exact parts of the view that need to be patched for each possible state change. Svelte is remarkably performant because it does only the least amount of work in the browser to update the view.

Angular runs a change detection routine—comparing the last state it used to render the view with the current state—every time it detects the state might have changed. Changes to the state of a component typically happen when an event listener runs, when data is requested to a server via an HTTP request, or when MacroTasks (such as `setTimeout()`) or MicroTasks (such as `Promise.then()`) are executed. Angular makes this possible thanks to [zone.js](https://zone.js.org/), an execution context that's aware of the asynchronous tasks running at any given time. Thanks to `zone.js`, Angular can detect when a MacroTask or MicroTask is executed and run the change detection routine. (javascript.info/event-loop is a wonderful resource to learn more about the JavaScript event loop and the difference between the micro- and macro-task queues.)

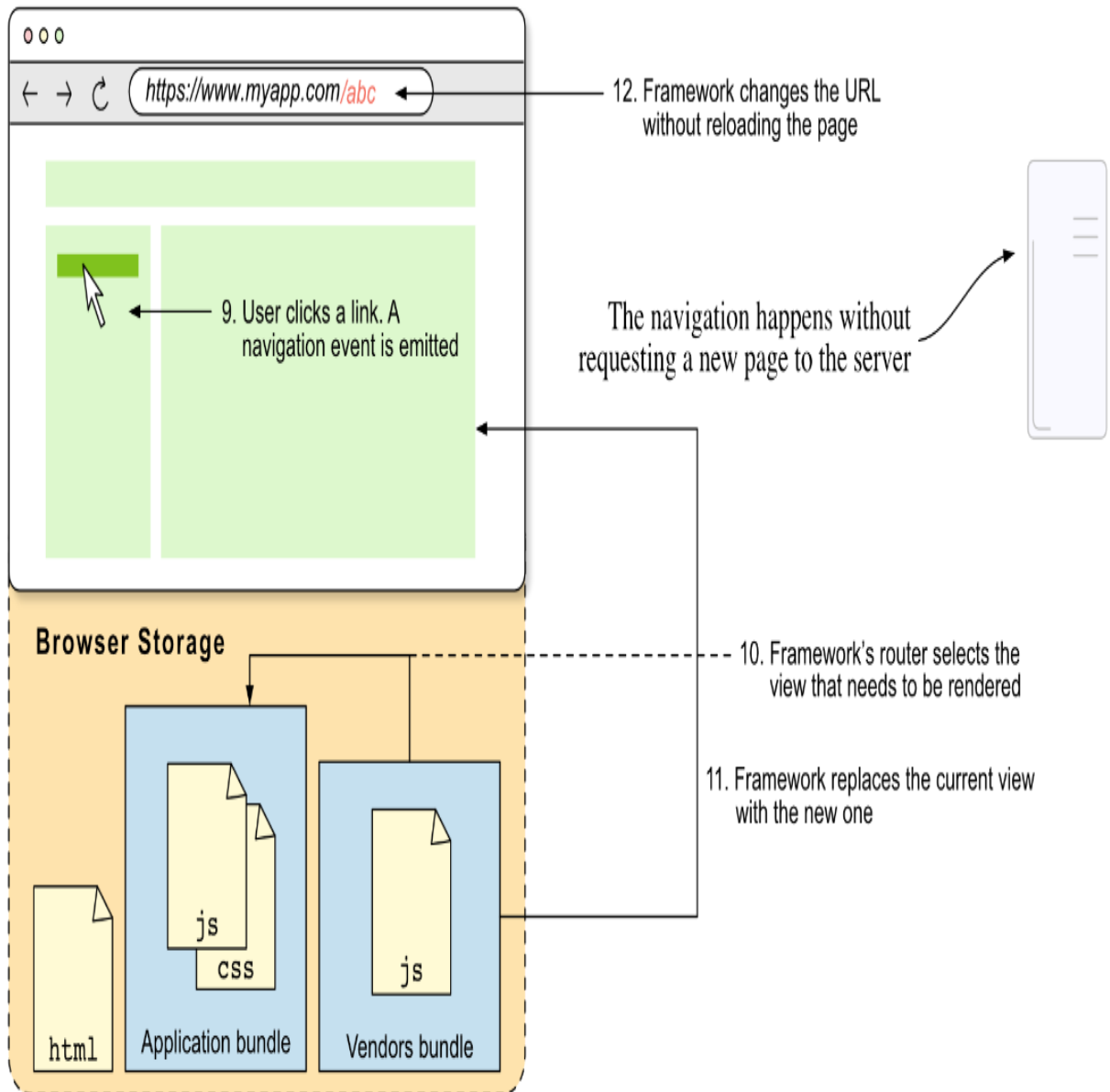
Most other widely used frameworks—including Vue, React, Preact or Inferno—use a virtual DOM representation of the view. So by comparing the last known virtual DOM with the virtual DOM after the state has changed, they compute the minimum changes required to update the HTML. React does this virtual DOM comparison every time the state is changed by the component, using either `setState()` or the `useState()` hook’s mechanism. Vue uses a remarkably smart approach and includes a reactivity layer the developer can use to define the application’s state. These reactivity primitives wrap regular JavaScript objects (this includes arrays or sets) and primitives (such as strings, numbers or booleans) and automatically detect when values change and notify the components that use them they need to re-render.

The last step we need to cover is how the framework handles navigation between routes.

Step 5: Navigating between routes

When the user clicks a link (9), the framework’s router prevents the default behavior of reloading the page, and instead renders the component that’s configured for the new route (10 and 11). The router is also in charge of changing the URL (12) to reflect the new route. This is depicted in figure [1.9](#).

Figure 1.9 SPA navigation between routes

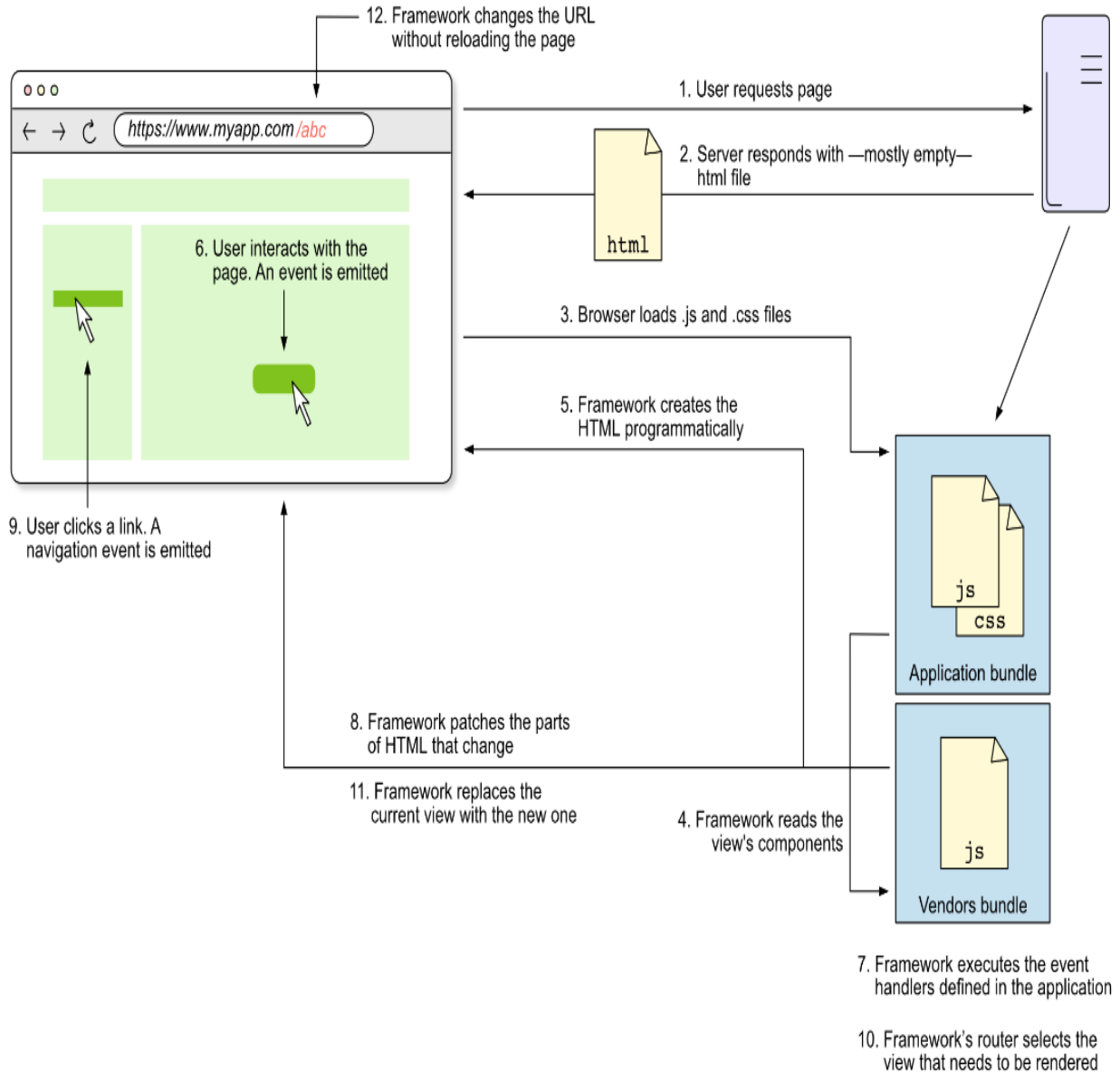


An SPA works with a single HTML file where the HTML markup code is programmatically updated by the framework, so new HTML pages aren't requested to the server when the user navigates to a different route. This is why they are called single page applications, because there is a single HTML file. The illusion of multiple pages is created by the framework, which renders the components that are configured for each route.

The complete flow of an SPA

Figure 1.10 shows the complete flow of a single-page application, including all the steps described earlier, but in a more schematic way.

Figure 1.10 The complete flow of a single-page application rendered in the browser



Now that you know how single-page applications work, let's compare them with an application that's server-side rendered.

1.3.3 The browser and server side of a SSR application

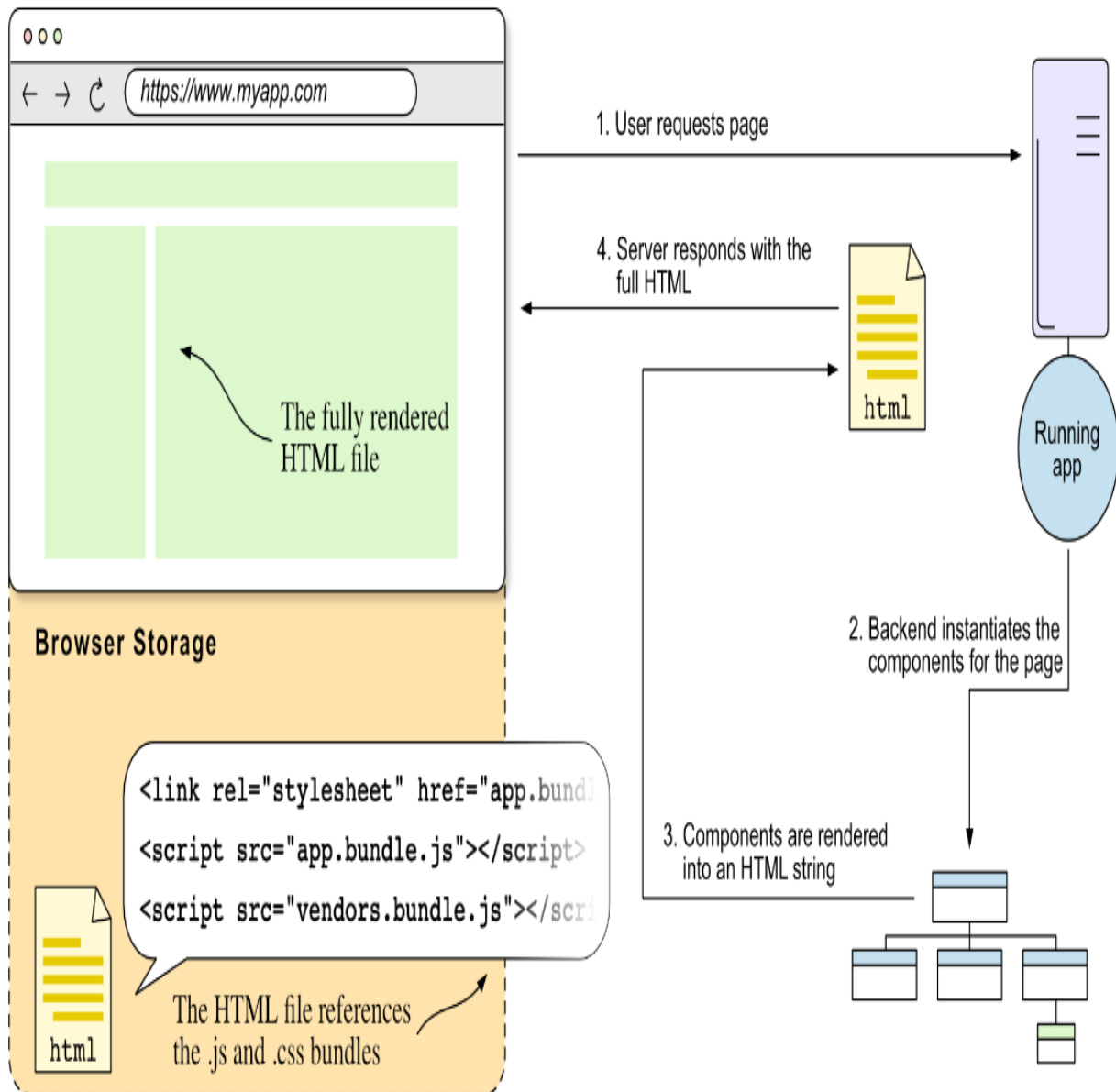
A server-side rendered application is a web application that renders the HTML markup on the server and sends it to the browser. Therefore, there needs to be a backend that handles requests and renders the HTML pages. In the browser, the frontend code is responsible for handling user interactions and updating the view to reflect the changes in the application's state—just like in a SPA. But when the user navigates to a different route, the browser requests a new HTML page from the server instead of programmatically updating the HTML markup.

Let's see how a SSR application works step by step.

Step 1: Loading an HTML page

When the user types the application's URL into their browser, the browser asks the server for the HTML file (1). The server sends back a complete page that is created each time someone requests it. To create the page, the server uses the application's router to figure out which components to show based on the requested route and instantiates them (2). Then, each component loads data from other servers or databases and executes its mounting code before being rendered. Finally, the components are turned into HTML (3) and sent to the user (4). This is illustrated in figure [1.11](#).

Figure 1.11 SSR application requesting an HTML page to the server



Even though it looks like a static HTML file to the user, the server generates the page each time it's requested. The HTML file served to the user displays already rendered HTML markup, so the framework doesn't need to use the Document API to programmatically generate it. But, the HTML coming from the server lacks the event handlers defined in the application code, so the application doesn't respond to user interactions. Here's where the hydration process comes into play.

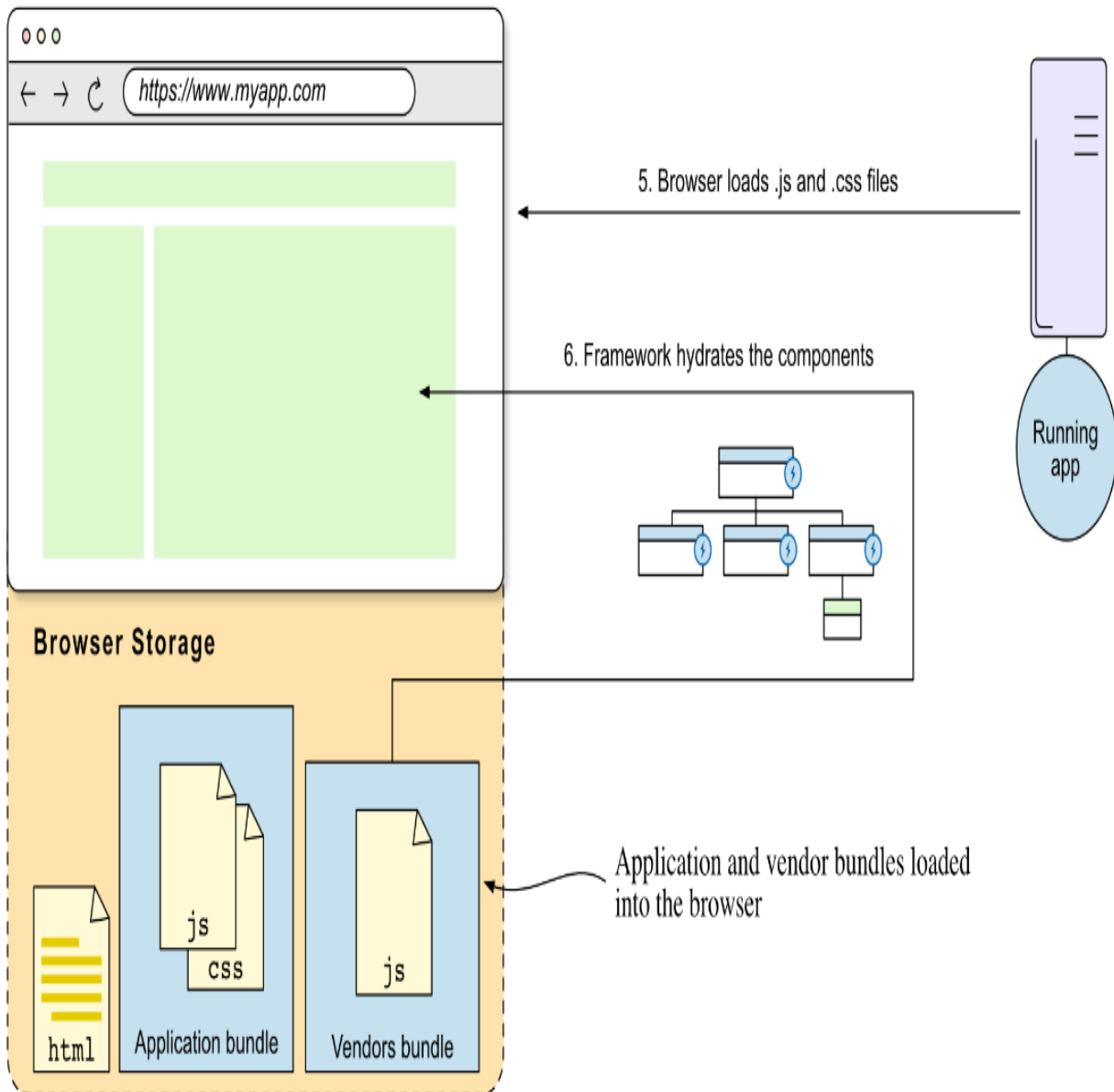
SSR Hydration

Hydration is the process by which the framework matches HTML elements with their corresponding virtual DOM nodes and attaches event handlers to make the HTML markup interactive in the browser. The hydration algorithm binds the browser's HTML to each component's virtual DOM, allowing for dynamic updates.

Step 2: Hydrating the HTML page

The HTML document instructs the browser to load the application JavaScript files and CSS style sheets (5) — the same as in the case of an SPA. Once the JavaScript code is parsed, the framework code needs to connect the existing HTML—produced in the server—to the component's virtual DOM, as well as attach event handlers (6). This is called the hydration process, and it's depicted in figure [1.12](#).

Figure 1.12 The framework hydrates the HTML page

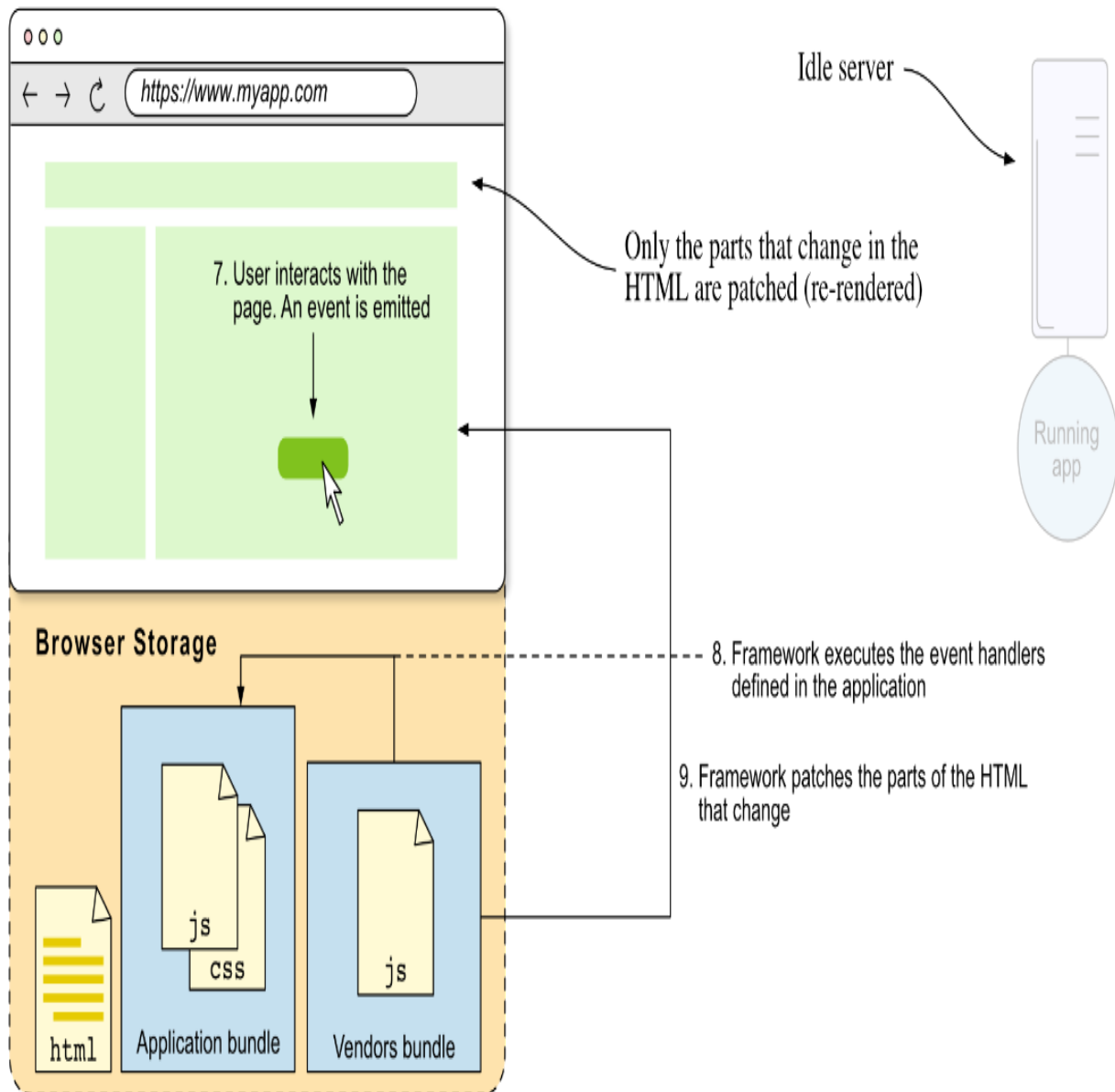


Once the page is made responsive by the framework's hydration process, the user can interact with it.

Step 3: Handling user interactions

When the user interacts with the page (7), the framework's event handlers are triggered (8), and—same as in the SPA case—the framework patches the parts of the HTML that need to be updated (9). All of this happens in the browser, so the server isn't involved in this process, as you can appreciate in [figure 1.13](#).

Figure 1.13 SSR application handling user interactions

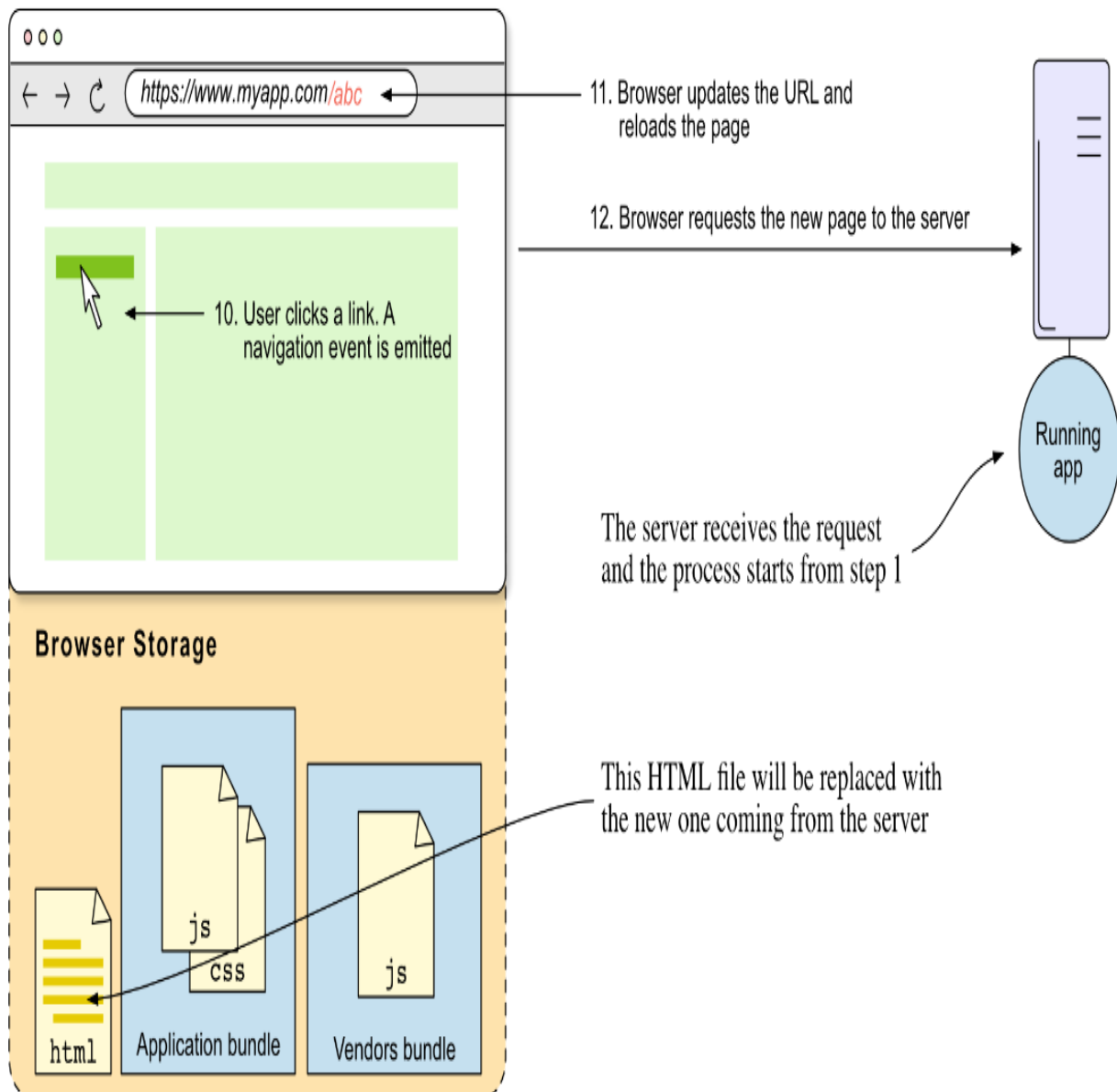


What happens when the user navigates to a different route?

Step 4: Navigating between routes

When the user clicks a link (10), the URL is changed by the browser (the framework doesn't do anything in the browser side this time), and the page is reloaded. A new HTML page is requested from the server, and the process starts again from step 1.

Figure 1.14 SSR application navigating between routes

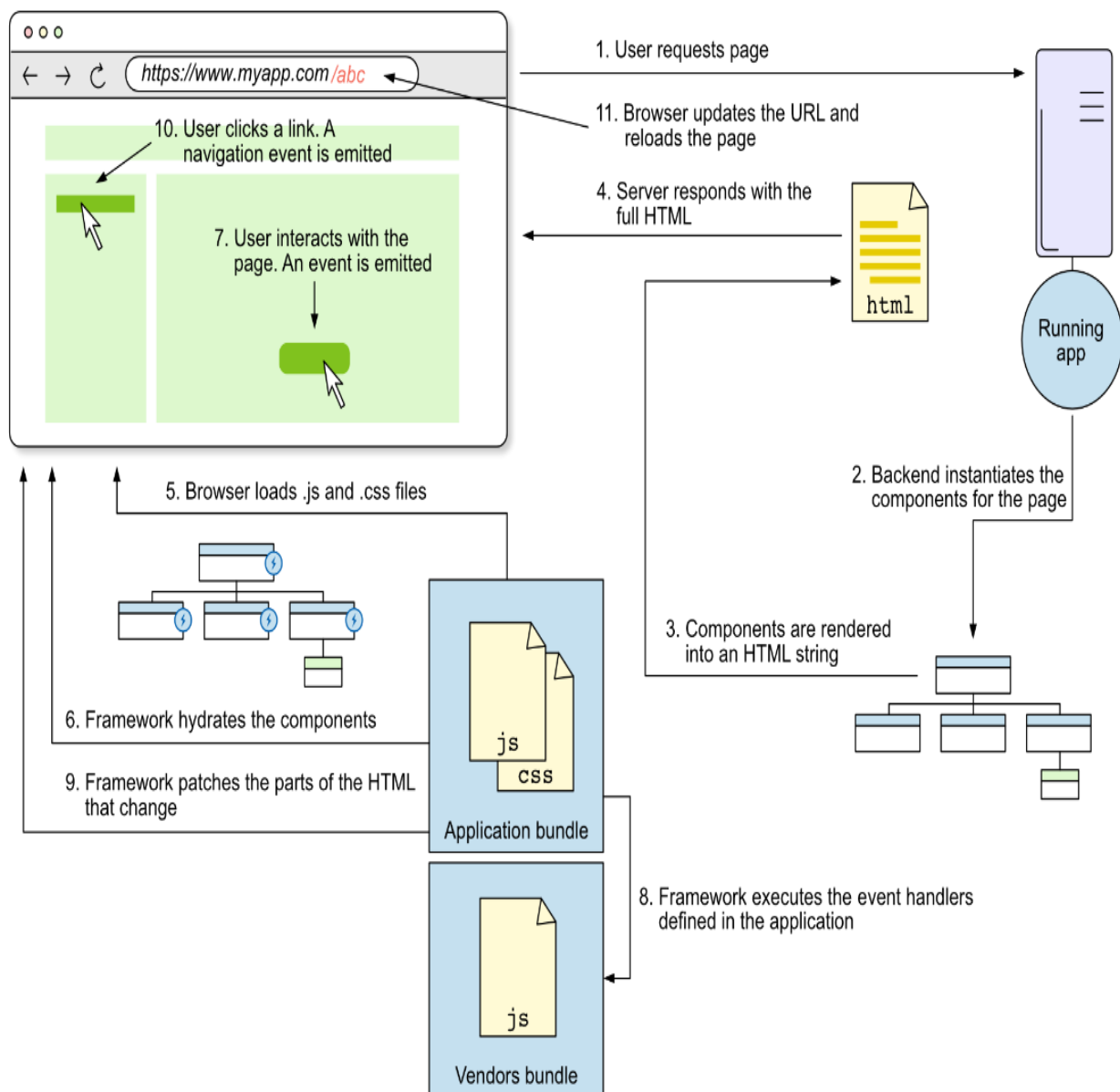


As you can see, navigating between pages is quite different in a SSR application than in an SPA. In the case of an SPA, the server isn't involved in the process. With SSR applications, pages are generated in the server.

The complete flow of a SSR application

Figure [1.15](#) shows the complete flow of a server-side rendered application, including all the steps described earlier, but in a more schematic way.

Figure 1.15 The complete flow of a SSR application—rendered in the server and hydrated in the browser



And this is how both single-page applications and server-side rendered applications work. What about building a simple application yourself, without using a framework?

1.4 Summary

- Building a frontend framework from scratch is a great way to learn how they work.
- Frontend frameworks bundle the application's code into a single JavaScript file, the third party dependencies into another file, and the CSS styles into yet another file. If the application is large, the framework might split the application's code into multiple bundles that are loaded "lazily," that is, just as they're needed.
- The Document API allows the creation of HTML elements programmatically, using JavaScript. The Document API is used by frontend frameworks to create the application's view.
- Single-page applications consist on a single HTML file that's loaded by the browser and updated by the framework to reflect the application's state. When the route changes, it's the framework changing the view. The browser doesn't reload the page.
- The hydration process is the process of connecting the existing HTML markup, rendered in the server, to the component's state and event listeners.

2 Vanilla JavaScript—like in the old days

This chapter covers

- Building an application using vanilla JavaScript and HTML
- Programmatically creating DOM elements
- Using the Document API to manipulate the DOM

To understand the benefits of using a frontend framework, you first need to understand the problems that it solves, and there is no better way to do this than to write an application without a framework—do the framework’s job yourself. The objective of this chapter is making you "suffer the pain" of writing applications without a framework, so that you can build some appreciation for the job that frameworks do for you.

In the old days (I’m not that old, it’s just that technology evolves fast), we’d write applications using only vanilla JavaScript and HTML. JQuery was the best we had: it provided a nice API to interact with the DOM, hiding away the browser differences. But we’d still have to write code down to the level of working with the DOM, and to be fair, it wasn’t that bad. That is, until we used our first modern frontend framework (it was Angular, in my case). Now there’s no going back; we’ve been there; we know how much simpler it’s become to write JavaScript applications.

You’re probably accustomed to writing applications leveraging the power of a framework—something you should keep on doing if you get paid to ship applications quickly—so it might be hard for you to realize the problems the framework solves. Or maybe you’ve been, like me, in the pre-framework era but haven’t written an application without a framework in a long time, in which case this chapter will be good for refreshing your memory. If your case is the former, I’m positive that by taking away the framework from you, you’ll quickly realize why you were using it in the first place. It’s like when you always find your working space clean and tidy, but you rarely appreciate it

because you're used to seeing it that way. That is until the cleaning personnel get sick and you have to clean things up yourself. You suddenly realize that vacuuming every corner of the office is arduous, removing the dust from the shelves and behind your computer is a pain, let alone cleaning the bathroom. Only then you realize how much you value working when things are clean and tidy around you, and start to really appreciate the cleaning personnel's job.

In this chapter, you'll do the cleaning yourself; that is, you'll build a simple application from scratch using only vanilla JavaScript and HTML. The cleaning personnel—the existing frontend frameworks—will be on strike. Despite the simplicity of the app, you'll notice how the code operates at a low abstraction level by directly manipulating the DOM, and it's very imperative in nature. You'll need to write code explicitly to update the HTML document with every change in the application state. It's evident that not using a framework will become a challenge as the complexity and size of an application increase. However, the purpose of this chapter is for you to realize this on your own by experiencing the process of creating an application without framework support.

Important

Before you go any further, go to appendix A and follow the instructions to set up the project where you'll be writing the code. Bear in mind that appendix A will be a little detour from the main topic of this chapter, but it's necessary to set up the project. When you finish and without further ado, let's begin the application you'll be building in this chapter.

You can find the code you'll be writing in this chapter in the GitHub repository, inside the *examples/ch02* directory:

<https://github.com/angelsolaorbaiceta/fe-fwk-book/tree/main/examples/ch02>

2.1 The assignment: a TODOs app

So you're a developer in a consulting company, and your manager has just assigned you a new project. There is this new client with an innovative idea for a new application, and they want you to build it. They say it has the

potential to disrupt the market, so it might be interesting. Your manager sets up a meeting with the client to discuss the project. In the meeting you make sure you understand the requirements, which you summarize as the following:

- Main idea: keep a list of the things you need to do (to-dos) in a day.
- A to-do can be marked as done, so it's removed from the list.
- A to-do can be modified, for the cases where the user makes a typo or wants to change the description.

The idea is so simple that you're a bit wary of it being "super-mega revolutionary" (you noted that down; that's what the client said), but your job is to build it, not to question its disruptiveness.

TODOs applications and frontend frameworks

The TODOs application is a [classic in the frontend framework world](#). They're the equivalent of "Hello World" when learning a new programming language. Framework authors like to use it as an example when they're working on their framework, both to test it and to show other developers how it's used. We don't want to break the tradition, so we'll be using it as well.

Vue.js implemented one, which can be found from the earliest versions, inside the *examples* directory of the framework's old repository:

github.com/vuejs/vue/tree/v0.7.0/examples/todomvc. React did as well, as early as in its initial public release, v0.3.0, which can be found inside the *examples* directory:

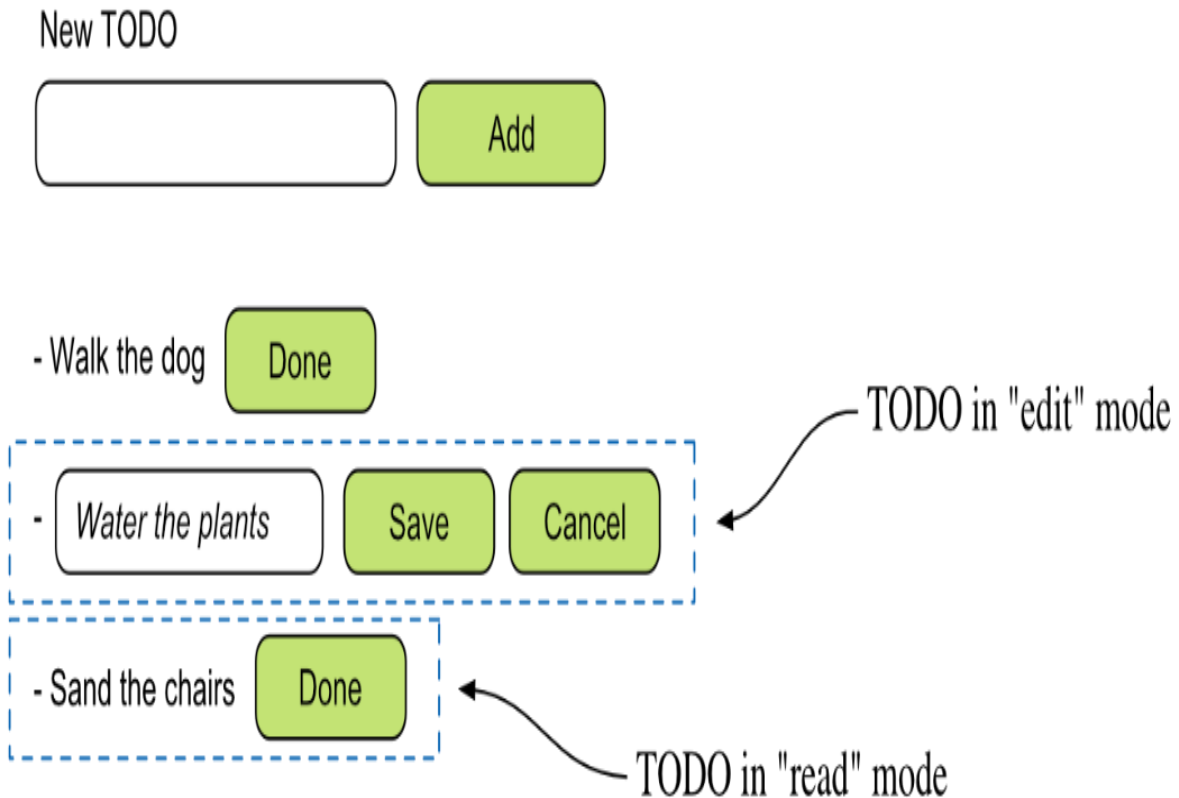
github.com/facebook/react/tree/v0.3.0/examples/todomvc. And one more example is Mithril's, which can be found in the *examples* directory of the framework's repository as well:

github.com/MithrilJS/mithril.js/tree/v1.0.0/examples/todomvc.

What you do next is talk with the design team to get the mockups for the application. They love the idea—although they swear they've seen something similar before—and they come up with a quick wireframe design that looks like figure [2.1](#).

Figure 2.1 Wireframe design for the TODOs app

My TODOs



You show it to the client, and they love it. Time to get down to business.

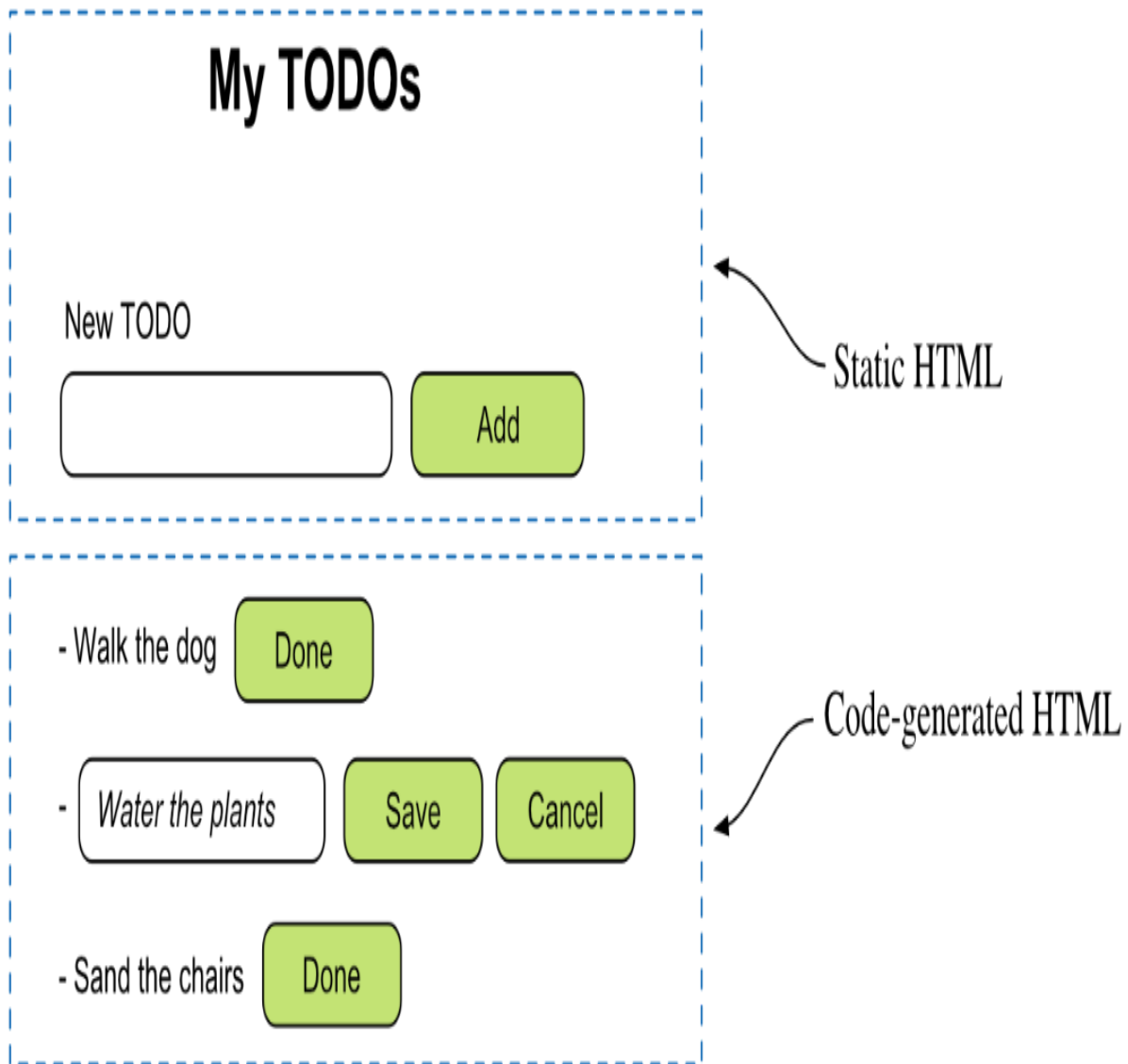
2.2 Writing the application

Now that you have the requirements and the design for the TODOs app, it's time to start writing the code. Before you start, you decide to take some time to plan ahead how you're going to tackle the task—as every good developer does. You realize the application is a simple one: it doesn't have fancy features or complex requirements. So, the first decision you make is to use

plain JavaScript and HTML, without any framework. You might use one later, if the application grows in complexity; but for now you want to keep things simple.

You figure out that part of the HTML markup is static—it won't change as the user interacts with the application—and part of needs to be dynamically generated, because it depends on the application's current state. For example, the list of to-dos will be programmatically generated from JavaScript because we can't know in advance what TODOs the user will write. In contrast, the title "My TODOs," the input box where the user writes a new to-do, its label, and the Add button will always be the same. Figure [2.2](#) shows the static and dynamic parts of the application.

Figure 2.2 Static and dynamic HTML of the TODOs app



Then, you think about what makes up the application's *state*.

State

The state is the information that the application keeps track of that makes it look and behave the way it does at a particular moment in time.

The application will look different when there are no to-dos than when there are some, for example. This means that the list of to-dos is part of the

application's state, so you'll need an array of strings to keep track of the existing to-dos. The strings represent the to-dos descriptions.

Last, before you start coding, you think about the application's behavior. Based on the requirements, the design, and a short conversation you had with the user-experience specialist, you decide that the application will behave as follows:

- When the user writes a new to-do and clicks Add, the to-do is added to the list of to-dos.
- If the user presses the Enter key while the input field is focused, the to-do is appended to the list of to-dos as well.
- Don't allow the user to add to-dos that are shorter than three characters.
- To edit a to-do, the user has to double-click it.
- If the user discards the changes by clicking the Cancel button, the to-do is restored to its previous state; the changes are lost.
- When a to-do is marked as done, it's removed from the list of to-dos.

With this in mind, it's time to start writing the code. Let's start by setting up the project.

2.2.1 Setting up the project

You'll write the vanilla JavaScript version of the TODOs application inside the *examples* directory in your project. (Make sure you've completed the set up from appendix A.) First, create a new folder inside the *examples* directory for this chapter: *ch02*.

```
$ cd examples
$ mkdir ch02
```

Then, create two new files inside the *ch02* directory:

- *todos.html* — where you'll write the HTML markup for the application
- *todos.js* — where you'll write the JavaScript code for the application

Your *examples* directory should look like this:


```
examples/  
└─ ch02/  
   └─ todos.html  
   └─ todos.js
```

You want to move your terminal's working directory back to the project's root directory, so you can run the `serve:examples` script from there:

```
$ cd ..  
$ npm run serve:examples
```

Your browser should open the *examples* directory and show the *ch02* directory it. Then click the *todos.html* file to open it. It should show an empty page because you haven't written any HTML markup yet. Let's write the static part of the HTML markup now.

2.2.2 The HTML markup

The static part of the HTML markup is pretty simple: it consists of a title (`<h1>`), an input field (`<input>`) with its label (`<label>`), and a button (`<button>`). There should also be a list—empty to start with—where the todos will be rendered programmatically (`<ul>`). And, very importantly, the HTML document should load the JavaScript file that will contain the application's code: *todos.js*.

The first thing you need to do is load the *todos.js* file as an ES module inside the `<head>` of the document. This is done by adding the `type="module"` attribute to the `<script>` element. ES modules are supported by all modern browsers, and a neat feature of them is that they are [deferred by default](#), which means that it won't start executing the JavaScript code until the HTML document has been parsed. That's why we can load the JavaScript file at the top of the document, the `<head>`, and still be sure the HTML markup is already available when the JavaScript code starts executing.

Tip

Read more about how ES modules are different from classic scripts in the browser in V8's blog at v8.dev/features/modules. It's a good read that

clarifies a lot of concepts around ES modules and their behavior in the browser, written by the people who work on V8 itself.

Then, you'll add the input field, its label, and the Add button inside a `<div>` element. Note that we need to add `id` attributes to the input field and the button so we can reference them from the JavaScript code. The same goes for the `<ul>` element that will contain the to-dos, which is below the `<div>` element and is empty to start with. The to-dos will be rendered programmatically by the JavaScript code, as `<li>` elements inside the `<ul>` element.

Open the *todos.html* file and add the markup in listing [2.1](#).

Listing 2.1 The static HTML markup for the TODOs app (todos.html)

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>My TODOs</title>

    <script type="module" src="todos.js"></script> #1
  </head>

  <body>
    <h1>My TODOs</h1>

    <div> #2
      <label for="todo-input">New TODO</label>
      <input type="text" id="todo-input" />
      <button id="add-todo-btn" disabled>Add</button>
    </div>

    <ul id="todos-list"></ul> #3
  </body>
</html>
```

If you now refresh the browser window, you should see something like Figure [2.3](#).

Figure 2.3 The HTML markup for the TODOs app

← → ↻ ⓘ 127.0.0.1:8080/examples/ch02/todos.html

My TODOs

New TODO

The remaining part of the HTML markup will be dynamically generated by the JavaScript code. Here's where the fun begins.

Exercise 2.1

Add some CSS styles to the title, the input field, and the button. Here're some suggestions:

- Use a nicer font, instead of the default one. You can choose one from Google Fonts (they're free to use), like [Roboto](#) and apply it to the

document.

- Center the application in the middle of the page, horizontally.
 - Make the title bigger.
 - Place the input field's label above the input field and make it italic.
 - Give the input and button some padding.
-

Find the solution at <http://mng.bz/qj9x>.

2.2.3 The JavaScript code

You will write the JavaScript code in the *todos.js* file, so make sure you have it open in your editor.

First of all, you want to define the application's state, which is a list of to-dos—an array of strings. You'll add some to-dos already populated in the array, so that when you open the page in the browser, you can see some to-dos already rendered. Then you want to grab references to the DOM elements that you need to interact with, using the `document.getElementById()` function from the Document API.

Open the *todos.js* file and write the code in listing [2.2](#).

Listing 2.2 The state and HTML element references (todos.js)

```
// State of the app
const todos = ['Walk the dog', 'Water the plants', 'Sand the
chairs']

// HTML element references
const addTodoInput = document.getElementById('todo-input')
const addTodoButton = document.getElementById('add-todo-btn')
const todosList = document.getElementById('todos-list')
```

So far, your application doesn't do anything when you type a new to-do description in the input field and click the Add button. The to-do items in the state aren't rendered either. This is because you have neither added event listeners nor written the code that renders the to-dos. Let's fix that.

Initializing the view

Now, you want to initialize the view of the application—that is, dynamically generate the HTML markup that depends on the application's state—and attach event listeners to the DOM elements that need them. To initialize the view, you iterate over the to-dos in the application's state and *render* each one using a function you'll call `renderTodoInReadMode()`. Each element is then append it to the `<ul>` element using the `todosList` element `append()` method.

Rendering

To render means to transform some data into a visual representation; something we can see.

In this context, when we *render a to-do*, what we're doing is creating the HTML elements that represent the to-do in our application.

Rendering a to-do—a JavaScript string—into an HTML representation will be done by a function that you'll call `renderTodoInReadMode()`. The naming is important here: we're saying that the to-do is rendered in read mode. If you remember from your discussion with the client, the to-do can be edited, so we need to render it in "edit mode" as well. In short: a to-do can be rendered in two different ways (it has two different visual representations). You'll also write a `renderTodoInEditMode()` function later on.

After rendering the to-dos in read mode you need to add a few event listeners to the DOM elements. First, you'll add a listener on the `<input>` field's `input` event—fired every time the user types something in the input field. This handler function should check if the input field has less than three characters, in which case the button is kept disabled to prevent the user from adding empty (or very short) to-do items. The button is enabled—by removing the `disabled` attribute—when the to-do has at least three characters (figure [2.4](#)). If you remember from the HTML markup (see listing [2.1](#)), the `<button>` element is disabled by default.

Figure 2.4 The field is disabled when the input field has fewer than three characters

New TODO

With fewer than three characters
the button remains disabled

New TODO

With three characters or more
the button is enabled

Then, you'll add a listener on the `<input>` field's `keydown` event, which fires every time the user presses a key—any key. But you're not interested in responding to every key the user presses, only the "Enter" key is relevant. For this, you want to check if the key pressed is "Enter", and if so, call a function you'll name `addTodo()`, which you'll implement in a minute and will be used to add a new to-do to the application's state, and render it in the HTML.

Finally, you need a listener on the Add `<button>` element's `click` event. The event handler is the same as the one for the `keydown` event: it calls the `addTodo()` function, clears the input field and disables the Add button

In the *todos.js* file write the code in listing [2.3](#).

Note

When you see a `// TODO` comment in the code, it means that you'll implement that part of the code later on in the chapter. I sometimes add these comments for you to know that there's something missing in the code and that you'll get to it soon.

Listing 2.3 The initialization of the application (todos.js)

```
// Initialize the view
for (const todo of todos) { #1
  todosList.append(renderTodoInReadMode(todo))
}

addTodoInput.addEventListener('input', () => { #2
  addTodoButton.disabled = addTodoInput.value.length < 3
})

addTodoInput.addEventListener('keydown', ({ key }) => { #3
  if (key === 'Enter' && addTodoInput.value.length >= 3) {
    addTodo()
  }
})

addTodoButton.addEventListener('click', () => { #4
  addTodo()
})

// Functions
function renderTodoInReadMode(todo) {
  // TODO: implement me!
}

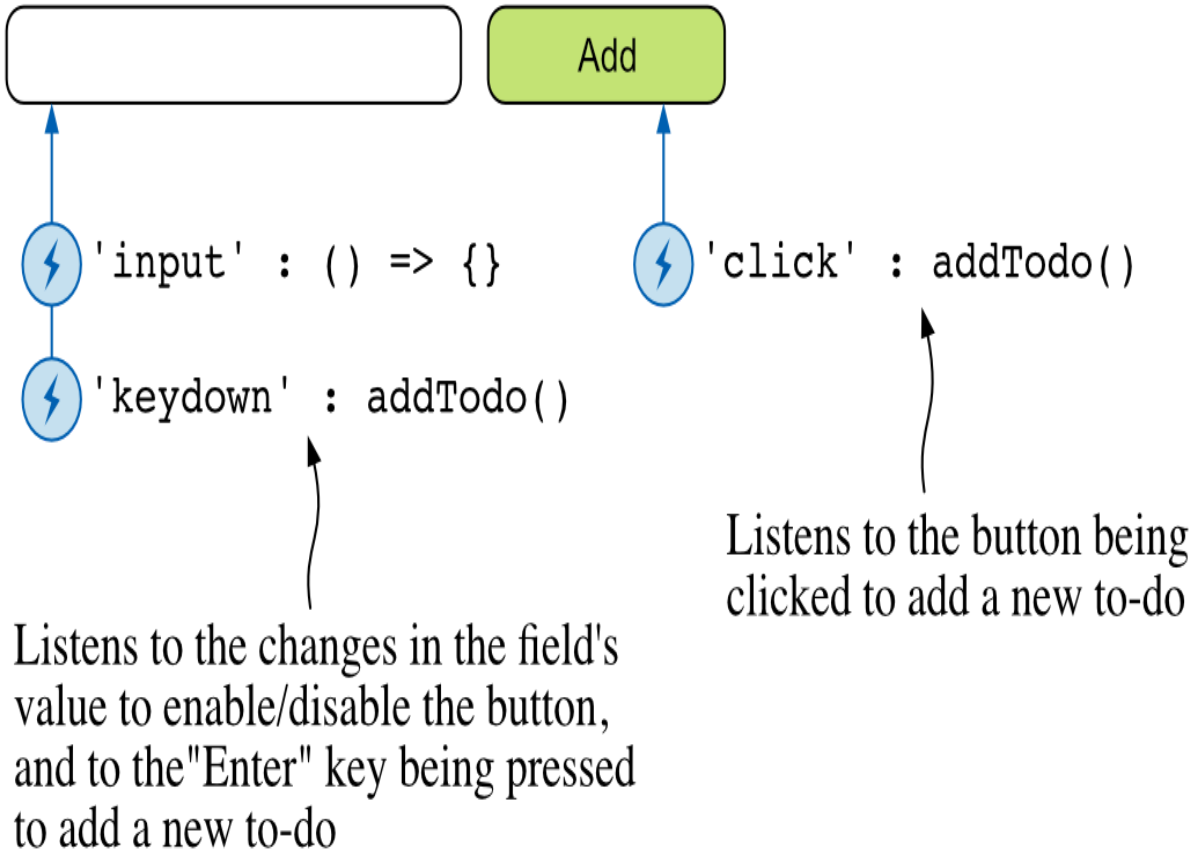
function addTodo() {
  // TODO: implement me!
}
```

Figure [2.5](#) shows a visual representation of the events you've added to the static part of the HTML markup.

Figure 2.5 The event listeners added to the HTML elements

My TODOs

New TODO



Now we're getting to the meat of the application: rendering to-dos! This is the part that a framework would do for you, but you're going to do it yourself in this chapter.

Exercise 2.2

Add a CSS transition to the `color` property of the button when it's enabled or disabled. This will make the button's color change smoothly when it's enabled or disabled.

Here're some tips in case you get stuck:

- You can use the `transition` CSS property. Go ahead and read about it in the MDN documentation if you need a refresher.
 - You can use the `:disabled` pseudo-class to style the button when it's disabled.
 - You can use the `:enabled` pseudo-class to style the button when it's enabled.
-

Find the solution at <http://mng.bz/7vVQ>.

Rendering to-dos in "read mode"

To render a to-do in read mode, you need to use the `document.createElement()` method of the Document API to create some HTML elements:

- The to-do items are inside an unordered list element (`<ul>`), so each to-do should go inside a list item element (`<li>`).
- The to-do itself is a simple text that you can render inside a `<span>` element.
- Then, the user should be able to mark a to-do as done, so you need a button to do that.

Figure [2.6](#) depicts the HTML markup for a to-do in read mode.

Figure 2.6 A TODO in "read mode" is rendered as a `<li>` element containing a `<span>` element with the to-do text and a button to mark it as done



`<li>`

`<span>Walk the dog</span>`



`<span>` `<button>`

`<button>Done</button>`



`</li>`

`<li>`

The to-do description can be added as the `textContent` property of the `<span>` element. We could have created a text node and appended it to the `<span>` element (for example doing: `span.appendChild(todo)`), but setting the `textContent` is a bit more concise.

The `<span>` element needs to have a listener attached to its `dblclick` event that's going to replace the to-do in read mode with the "edit mode" version. To accomplish this, you'll call the `replaceChild()` method on the `<li>` DOM node. This method removes the entire `<li>` element and its children from the list of to-dos and renders the "edit mode" version of the to-do in its

place. The rendering is done by calling the `renderTodoInEditMode()` function, which you'll implement in the next section.

The `replaceChild()` method

The `replaceChild()` method from the DOM API is used to replace a child node of a DOM node (the one upon which the method is called) with another node. It accepts two arguments:

- `newNode`: the node that will replace the old node.
- `oldNode`: the node that will be replaced.

Lastly, you want to attach an event listener to the `<button>` element's `click` event that's going to remove the to-do from the list of to-dos. For that, you'll write a `removeTodo()` function that you'll also need to fill in later on.

Now that you know what the plan is, fill in the `renderTodoInReadMode()` function as in listing [2.4](#).

Listing 2.4 Rendering the to-dos in "read mode" (`todos.js`)

```
function renderTodoInReadMode(todo) {
  const li = document.createElement('li') #1

  const span = document.createElement('span') #2
  span.textContent = todo
  span.addEventListener('dblclick', () => { #3
    const idx = todos.indexOf(todo)

    todosList.replaceChild( #4
      renderTodoInEditMode(todo),
      todosList.childNodes[idx]
    )
  })
  li.append(span)

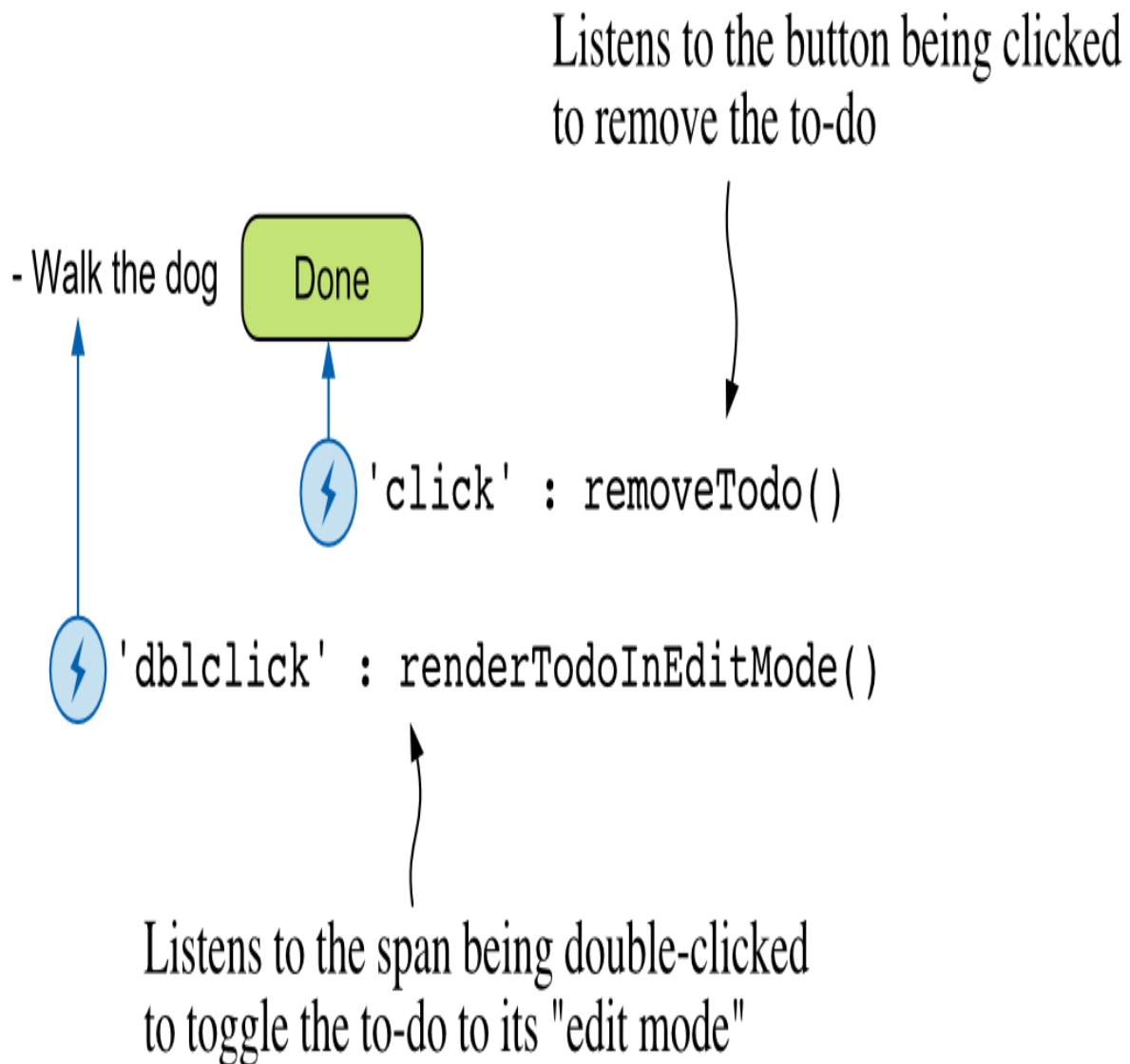
  const button = document.createElement('button') #5
  button.textContent = 'Done'
  button.addEventListener('click', () => { #6
    const idx = todos.indexOf(todo)
    removeTodo(idx)
  })
  li.append(button)
```

```
    return li
}

function removeTodo(index) {
    // TODO: implement me!
}
```

Figure [2.7](#) shows a visual representation of the events you've added to the to-dos in read mode.

Figure 2.7 The event listeners added to the to-dos in "read mode"

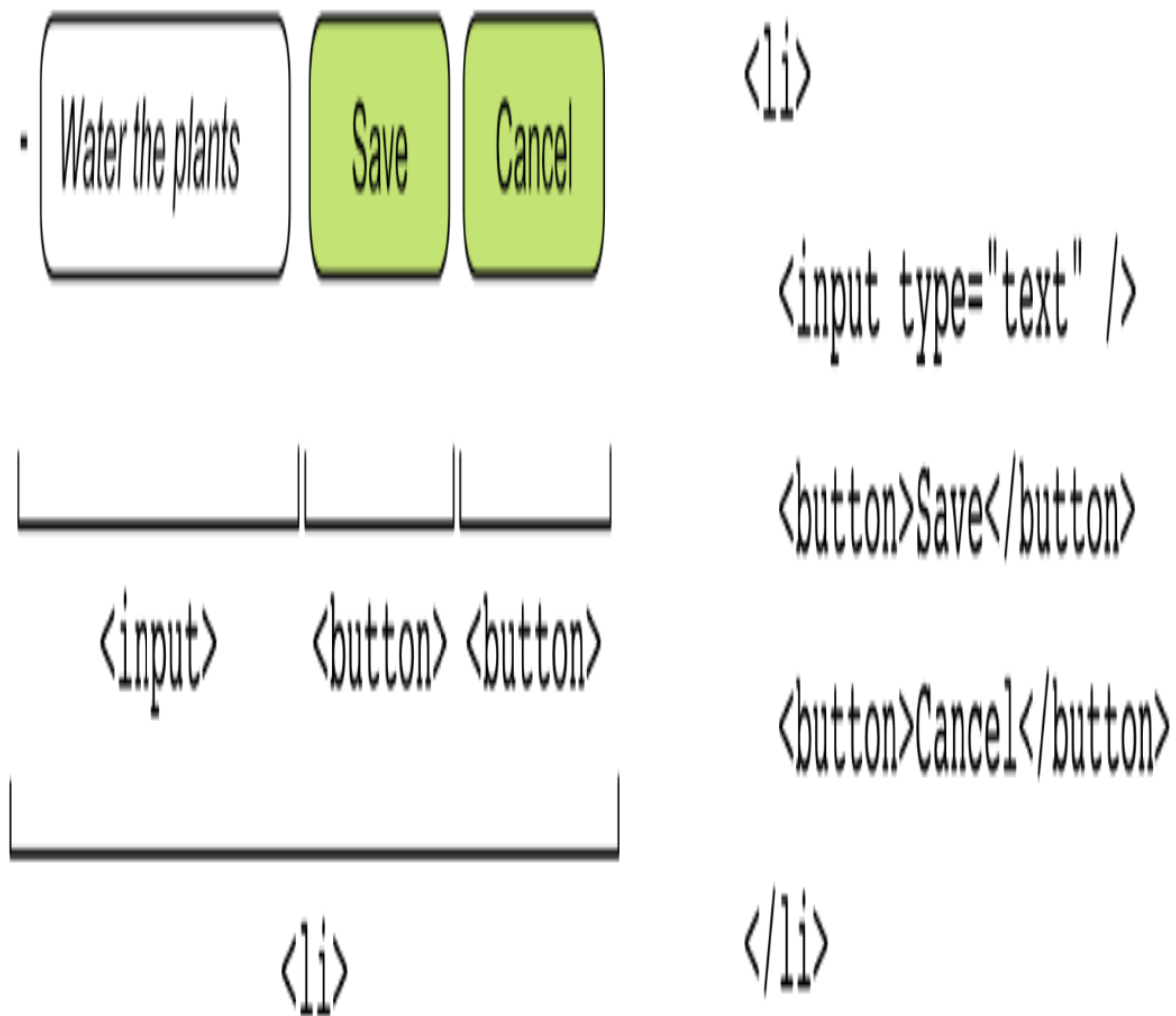


Let's now implement the `renderInEditMode()` function.

Rendering to-dos in "edit mode"

The to-do in edit mode is also part of the unordered list of to-dos, thus it should also appear inside a `<li>` element. But this time, the `<li>` element should contain an `<input>` element instead of a `<span>` element, so that the user can modify the to-do description. And instead of having one button, we need two: one to save the changes and another to cancel them. Figure [2.8](#) shows the HTML markup for a to-do in "edit mode".

Figure 2.8 A TODO in "edit mode" is rendered as a `<li>` element containing an `<input>` element with the to-do text and two buttons to save or cancel the changes



When the user clicks on the save button, a function that you'll write later named `updateTodo()` will modify the to-do description in the state, and replace the to-do in edit mode with the read mode version (once the user is done editing the to-do, we want them to see the updated version back in read mode). When the user clicks on the Cancel button instead, you just need to call the `renderTodoInReadMode()` function.

Write the code for the `renderTodoInEditMode()` function as in listing [2.5](#).

Listing 2.5 Rendering the to-dos in "edit mode" (todos.js)

```
function renderTodoInEditMode(todo) {
  const li = document.createElement('li') #1

  const input = document.createElement('input') #2
  input.type = 'text'
  input.value = todo
  li.append(input)

  const saveBtn = document.createElement('button') #3
  saveBtn.textContent = 'Save'
  saveBtn.addEventListener('click', () => { #4
    const idx = todos.indexOf(todo)
    updateTodo(idx, input.value)
  })
  li.append(saveBtn)

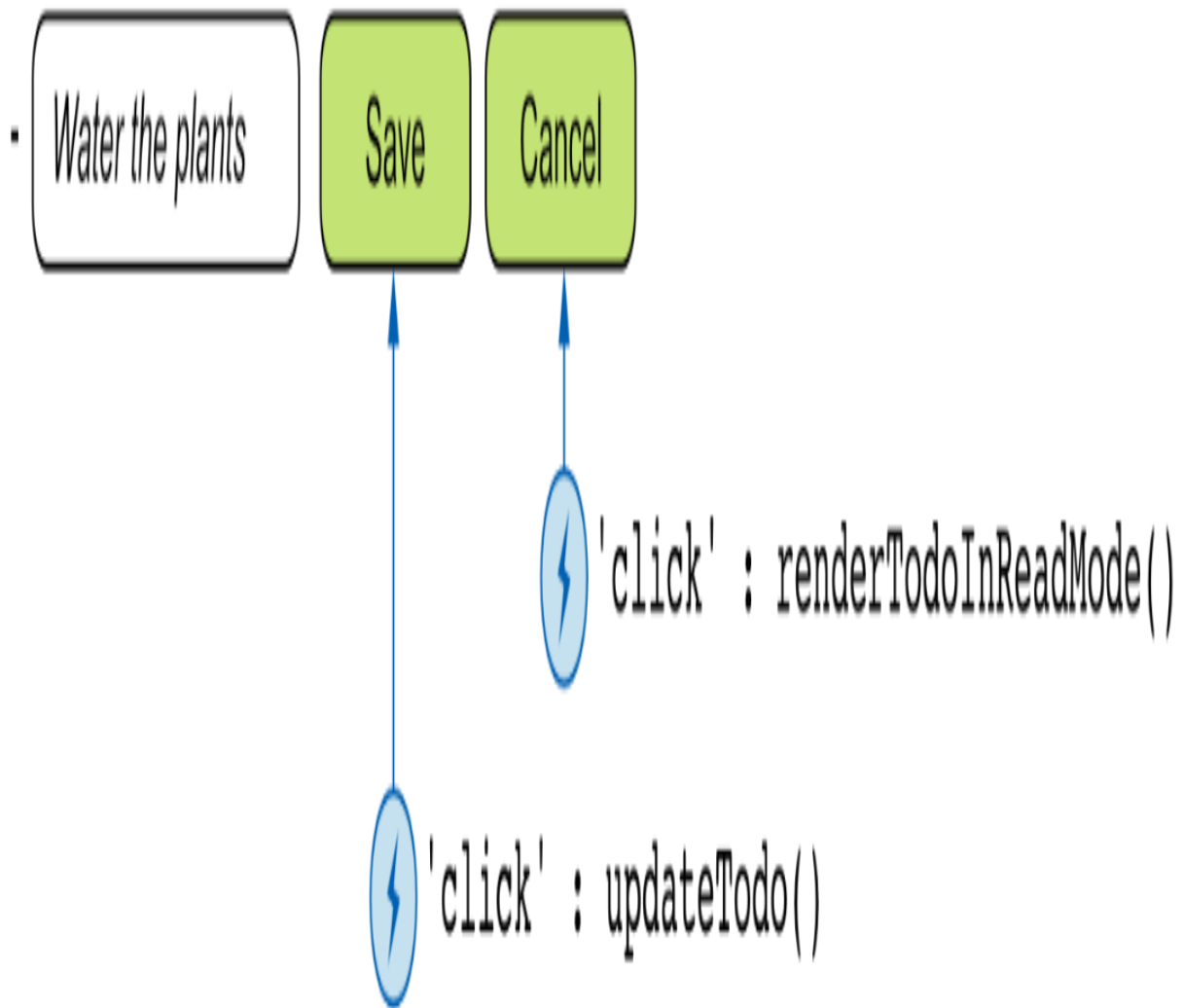
  const cancelBtn = document.createElement('button') #5
  cancelBtn.textContent = 'Cancel'
  cancelBtn.addEventListener('click', () => { #6
    const idx = todos.indexOf(todo)
    todosList.replaceChild( #7
      renderTodoInReadMode(todo),
      todosList.childNodes[idx]
    )
  })
  li.append(cancelBtn)

  return li
}

function updateTodo(index, description) {
  // TODO: implement me!
}
```

The code is very similar to the one for the read mode version of the to-do. Figure [2.9](#) shows the events you've added to the to-dos in "edit mode".

Figure 2.9 The events added to the to-dos in "edit mode"



So, all that's missing to implement are the `addTodo()`, `removeTodo()`, `updateTodo()` functions. Let's do that now.

Adding, removing and updating to-dos

The functions to add, remove and update to-dos are defined in our `todos.js` file, but they're not implemented yet. We left "TODO" comments (oh, the irony!) in the code to remind us of that. The implementation of these functions is straightforward, so let's see what each of them does.

The `addTodo()` function reads the description for the new to-do from the `<input>` element's `value` property, and pushes it into the array of to-dos. Then it calls the `renderTodoInReadMode()` function to render the HTML for the new to-do and appends it to the `todosList` element. Lastly, it clears the `<input>` element's `value` property so that the user can enter a new to-do description, and disables the add button.

The `removeTodo()` function removes the to-do from the array of to-dos and the `<li>` element from the document. To remove the `<li>` element from its parent `<ul>`, it calls the `remove()` method on the target node, which you can locate by index inside the `childNodes` array of the `<ul>` element.

The `updateTodo()` function needs two parameters passed to it: the index of the to-do to update, and the new description for the to-do. The passed description overwrites whatever is in the array of to-dos at the given index. Then, using the `renderTodoInReadMode()` function, you can render the HTML for the updated to-do, and finally replace the to-do at the given index inside the `todosList` element's `childNodes` array with the new HTML.

Listing [2.6](#) shows the code for the `addTodo()`, `removeTodo()` and `updateTodo()` functions.

Listing 2.6 Functions to add, remove and edit to-dos (todos.js)

```
function addTodo() { #1
  const description = addTodoInput.value

  todos.push(description)
  const todo = renderTodoInReadMode(description)
  todosList.append(todo)

  addTodoInput.value = ''
  addTodoButton.disabled = true
}

function removeTodo(index) { #2
  todos.splice(index, 1)
  todosList.childNodes[index].remove()
}

function updateTodo(index, description) { #3
  todos[index] = description
```

```
const todo = renderTodoInReadMode(description)
todosList.replaceChild(todo, todosList.childNodes[index])
}
```

If you refresh the page now, you should be able to add, remove and update todos. Your application should look similar to Figure 2.10. You've written a web application without a framework. It wasn't that painful, was it?

Figure 2.10 The finished TODOs app



Try to add a new to-do by writing something like "Sing a song" in the input field and pressing the Add button. Then, press the "Done" button to remove the to-do from the list. Give a try clicking one of the to-dos to see how it gets replaced by its "edit mode" version, like in Figure [2.11](#).

Figure 2.11 The TODOs app in "edit mode"



The screenshot shows a web browser window with the address bar displaying "127.0.0.1:8080/examples/ch02/todos.html". The main heading is "My TODOs". Below the heading, there is a "New TODO" section with a text input field and an "Add" button. Below this, there is a list of three TODO items:

- Walk the dog Done
- Water the plants Save Cancel
- Sand the chairs Done

I hope you feel excited about what you've just built: an interactive web application without a framework! But I also hope that you've realized how inefficient it'd be to write a large application like this; that's why using a

framework is a great idea. Operating at such a low-level to programmatically generate the HTML markup for the DOM is tedious and error-prone.

Exercise 2.3

Imagine that the customer tells you that they don't want "done" to-dos to be removed from the list, but instead crossed out. Can you implement this feature for them?

Find the solution at <http://mng.bz/mjg8>.

Exercise 2.4

The customer tried the application and loved it! But they realized they can add the same to-do multiple times—unacceptable! Can you fix this bug, so that if the user tries to add a to-do that already exists, the application doesn't add it again and warns the user about it?

Find the solution at <http://mng.bz/5oj7>.

Exercise 2.5

The customer came with a challenging request for you. To aid their users with a hearing impairment, they'd like the application to read out loud a to-do when it's added to the list. Can you implement this feature for them?

Psss... here's a little tip from a senior developer at your company. You might want to read about the Web Speech API.

Find the solution at <http://mng.bz/6ngR>.

The first thing we want our framework to take care for us is the usage of the Document API to create and manipulate the DOM; that's the part that's the most burdensome to write. If we can abstract away the manipulation of the DOM, we can focus on the application logic, which is what makes our

applications useful. Think about it: the time spent working on manipulating the DOM doesn't really add value to the final user. We need to do it for the application to be interactive, but it's not what the user cares about. A good framework should allow us to forget about dealing with the DOM and focus on the application logic. That's exactly what we'll do in the next chapter.

2.3 Summary

- Nothing prevents us from writing complete frontend applications without a framework, but doing so can easily result in code that's a mix of application logic and DOM manipulation, that is, using the Document API to modify the browser's document.
- Using vanilla JavaScript to write a fronted application, every event that changes the state of the application forces us to write the code that updates the DOM to reflect the new state. This code tends to be very imperative, and verbose.

Part 2

2.1 A basic framework

Get ready, because the real excitement begins now!

You’ve already dabbled in the creation of a TODO application using nothing but HTML and vanilla JavaScript. It provided you with a glimpse of the challenges involved in developing large, complex web applications that require manual DOM manipulation and real-time synchronization between the view and the model as users interact with the app.

But don’t worry; frontend frameworks were designed to make your life easier! We’ll start by abstracting away the nitty-gritty DOM manipulation code required for view creation. Enter the concept of the virtual DOM. With a virtual DOM tree, you can define how a view should appear, and let the framework handle the heavy lifting using the DOM API to create its representation in the browser.

In Chapter 5, you’ll dive into crafting a state management system that communicates changes to your framework whenever the application state is altered. By bringing together the function responsible for rendering the virtual DOM and the state management system, you’ll have the initial version of your very own framework.

Chapter 6 will take you one step further as you publish your framework on npm and use it to enhance your TODO application—now that’s exciting!

Before we conclude this section, we’ll tackle a critical algorithm that underpins virtual DOM-based frameworks: the reconciliation algorithm. While it’s a complex concept, I’ll guide you through it step by step. This algorithm ensures that only the portions of the DOM requiring updates are modified, minimizing unnecessary changes.

An incredible adventure lies ahead, one that I embarked on with great enthusiasm. I hope you'll find this journey as exhilarating as I did!

3 Rendering and the virtual DOM

This chapter covers

- What the virtual DOM is
- What problem the virtual DOM solves
- Implementing functions to create virtual DOM nodes
- Defining the concept of a stateless component

As you've seen in the previous chapter, mixing application and DOM manipulation code gets unwieldy quickly. If for every event resulting from the user interacting with the application, we have to implement not only the business logic—the one that gives value to the application—but also the code to update the DOM, the codebase becomes a hard-to-maintain mess. This is because we mix two different levels of abstraction together: the application logic and the DOM manipulation. What a maintenance nightmare!

Manipulating the DOM results in very *imperative code*, that is, code that describes how to do something, step by step. This is in contrast with *declarative code*, which describes what to do, without specifying how to do it—those details are implemented somewhere else. Also, manipulating the DOM is a very *low level* operation, that is, it requires a lot of knowledge of the Document API, and sits below the application logic. Contrast this with *higher level* application code, which is framed in a language that is close to the business, and anyone working in the project can—should—understand.

We would have a much cleaner codebase if we could describe in a more declarative manner what we want the view of our application to look like and let the framework take care of manipulating the DOM to create the view. What we need is similar to the blueprints of a house: a description of what needs to be built without specifying how to build it. The architect designs the blueprints for the house and lets the construction company take care of building it. Imagine if the architect had to, not only design the house, but also go to the construction site and build it; or at least tell the construction workers

how they need to do their job, step by step, without missing any single detail. That would be a very inefficient way of building.

For the sake of productivity, the architect focuses on the "what" needs to be built, and let the construction company take care of the "how" it's built. Similarly, we want the application developer to focus on the "what" (what the view should look like), and let the framework take care of the "how" (how to assemble the view using the Document API).

Important

You can find all the listings in this chapter in the [listings/ch03 directory](#) of the book's repository.

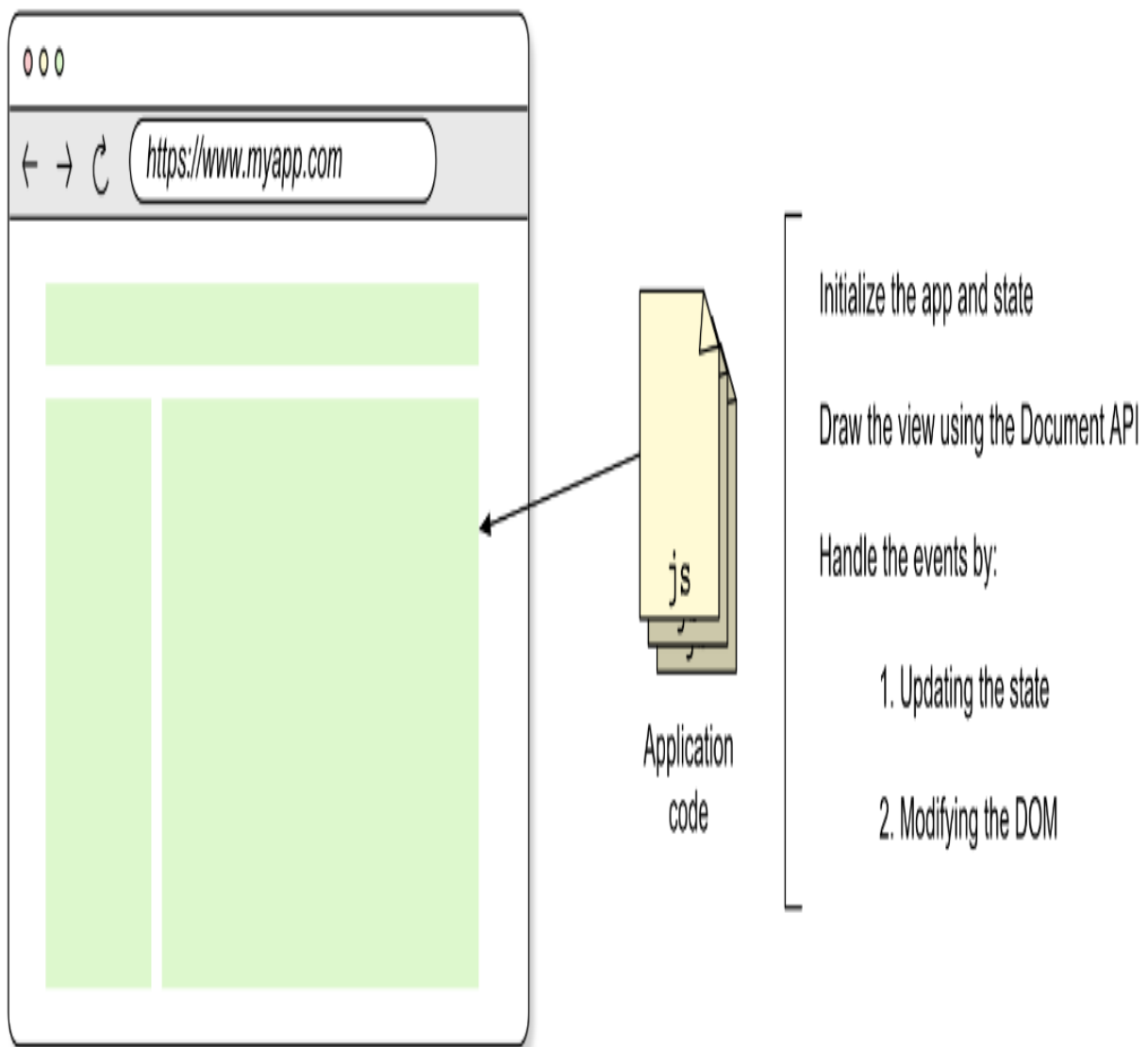
The code you write in this chapter is for the framework's first version, which you'll publish in chapter [6](#). Therefore, the code in this chapter can be checked out from [the ch6 label](#):

```
$ git switch --detach ch6
```

3.1 Separating concerns—DOM manipulation vs. application logic

In the previous chapter, you wrote all the code together as part of the application, like you can see in figure [3.1](#). That code was in charge of initializing the application and its state, programmatically build the view using the Document API, and handling the events that result from the user interacting with the application by modifying the DOM accordingly.

Figure 3.1 So far, all the code is written together as part of the application



What we want to accomplish in this chapter is separating the code that describes the view—the application’s code—from the code that uses the Document API to manipulate the DOM and create the view—the framework’s code. There’s a term widely used in the software industry: *separation of concerns*.

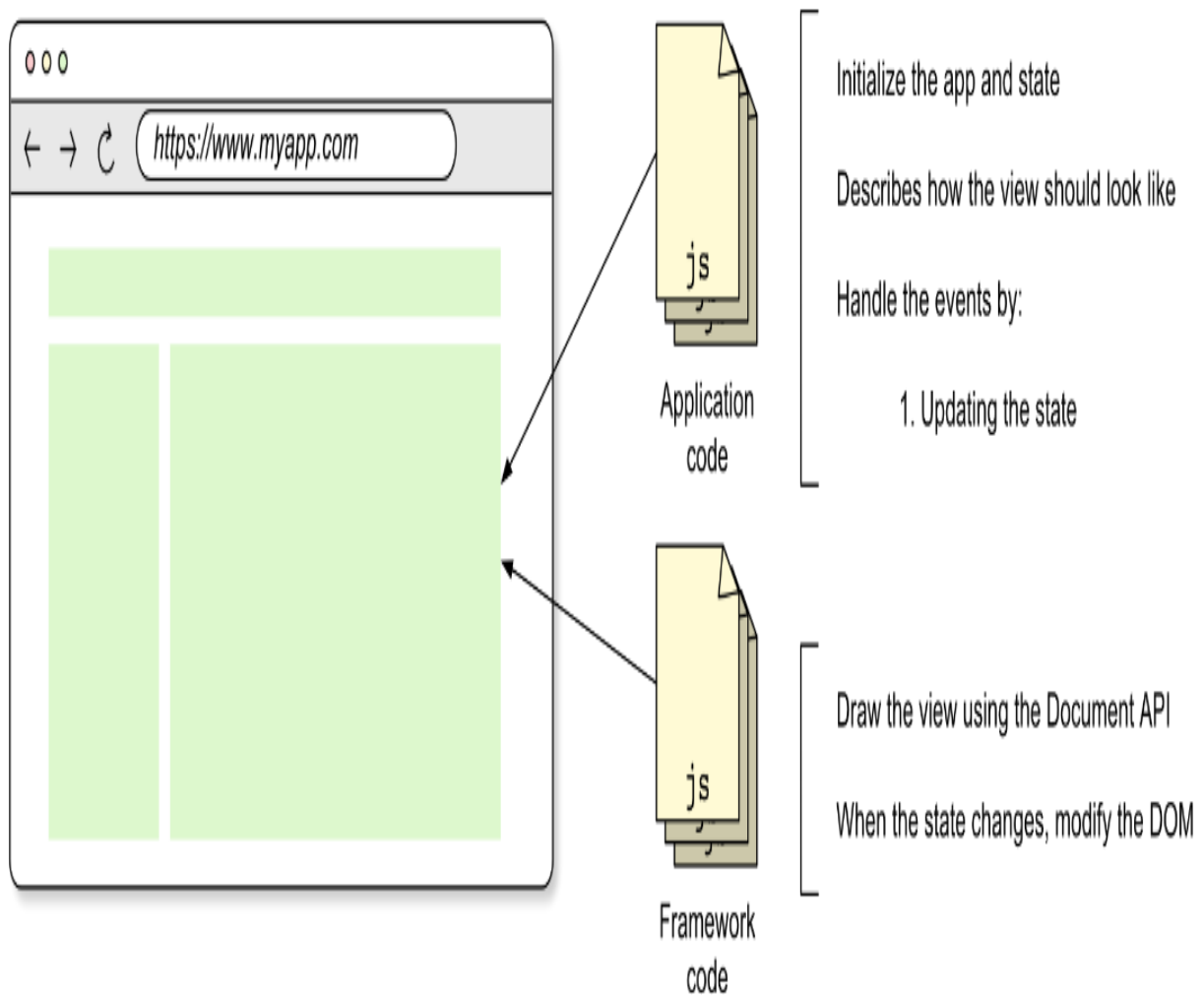
Separation of concerns

Separating concerns means splitting the code so that the parts of it that carry out different responsibilities can be found separated from each other, which

helps the developer understand the code and maintain it.

Figure 3.2 shows the separation of concerns we want to achieve: splitting the application code from the framework code that deals with the DOM manipulation and keeps track of the state. We will be focusing on rendering the view in this and the next chapter, and leave the state management for chapter 5.

Figure 3.2 By the end of next chapter, you'll have separated the code that describes the view from the code that manipulates the DOM



The main objective of this separation of concerns is to simplify the application's developer job: they only need to focus on the application logic, and let the framework take care of the DOM manipulation. This results in three clear benefits:

- *Developer productivity* — the application developer doesn't need to write DOM manipulation code, they can instead focus on the application logic. They have to write less code, and that makes them ship value faster.
- *Code maintainability* — the DOM manipulation and application logic aren't mixed together, and that makes the code more succinct and easier to understand.
- *Framework performance* — the framework author, who's likely to understand how to produce efficient DOM manipulation code better than the application developer, can optimize how the DOM is manipulated to make the framework more performant.

Going back to the blueprint analogy: how do you define how the view should look like, the same way the architect does with the blueprints? The answer is the *virtual DOM*.

3.2 The virtual DOM

The word "virtual" is used to describe something that isn't real, but mimics something that is. A *virtual machine* is, for example, software written to mimic the behavior of a real machine—hardware. It gives you the impression that you're running a real machine, but it's actually software running on top of your computer's hardware.

Virtual DOM

The virtual DOM is a representation of the actual DOM (the [Document Object Model](#) in the browser). The DOM is an in-memory tree structure managed by the browser engine, representing the HTML structure of the web page. In contrast, the virtual DOM is a JavaScript-based in-memory tree of virtual nodes that mirrors the structure of the actual DOM. Each node in this virtual

tree is referred to as a *virtual node*, and the entire construct is known as the *virtual DOM*.

The nodes in the actual DOM are heavy objects that have hundreds of properties, whereas the virtual nodes are lightweight objects that only contain the information needed to render the view. Virtual nodes are cheap to create and manipulate.

Let's imagine that we want to produce the following HTML:

```
<form action="/login" class="login-form">
  <input type="text" name="user" />
  <input type="password" name="pass" />
  <button>Log in</button>
</form>
```

The HTML consists of a `<form>` with three child nodes: two `<input>` and a `<button>`. A virtual DOM representation for this HTML needs to contain the same information as the DOM, namely:

- What nodes are in the tree and their attributes.
- The hierarchy of the nodes in the tree.
- The relative position of the nodes in the tree.

For example, it's important that the virtual DOM includes the `<form>` as the root node, and that the two `<input>` and `<button>` are its children. The form has an `action` and a `class` attribute, and the button—although not visible in the HTML—has an `onclick` event handler. The `type` and `name` attributes of the `<input>` elements are also crucial: it's not the same an `<input>` of type `text` and an `<input>` of type `password`. Also, the relative position of the form's children is important: the button should go below the inputs. The framework needs all this information for the view to be rendered correctly.

A possible virtual DOM representation—made of pure JavaScript objects—for this HTML could be the following:

```
{
  type: 'element',
  tag: 'form',
  props: { action: '/login', class: 'login-form' },
  children: [
```

```

    {
      type: 'element',
      tag: 'input',
      props: { type: 'text', name: 'user' }
    },
    {
      type: 'element',
      tag: 'input',
      props: { type: 'password', name: 'pass' }
    },
    {
      type: 'element',
      tag: 'button',
      props: { on: { click: () => login() } },
      children: [
        {
          type: 'text',
          value: 'Log in'
        }
      ]
    }
  ]
}

```

Each node in the virtual DOM is an object with a `type` property that identifies what kind of node it is. In this example, there are two types of nodes:

- *element* — represents a regular HTML element, such as `<form>`, `<input>`, or `<button>`.
- *text* — represents a text node, such as the "Log in" text of the `<button>` element in the example above.

We'll see one more type of node later in the chapter, the *fragment* node: a node used to group other nodes together, but has no semantic meaning of its own.

Each type of node has its own set of properties that describe it. Text nodes, for example, have one property apart from the `type`:

- `value`: the string of text.

Element virtual nodes have three properties:

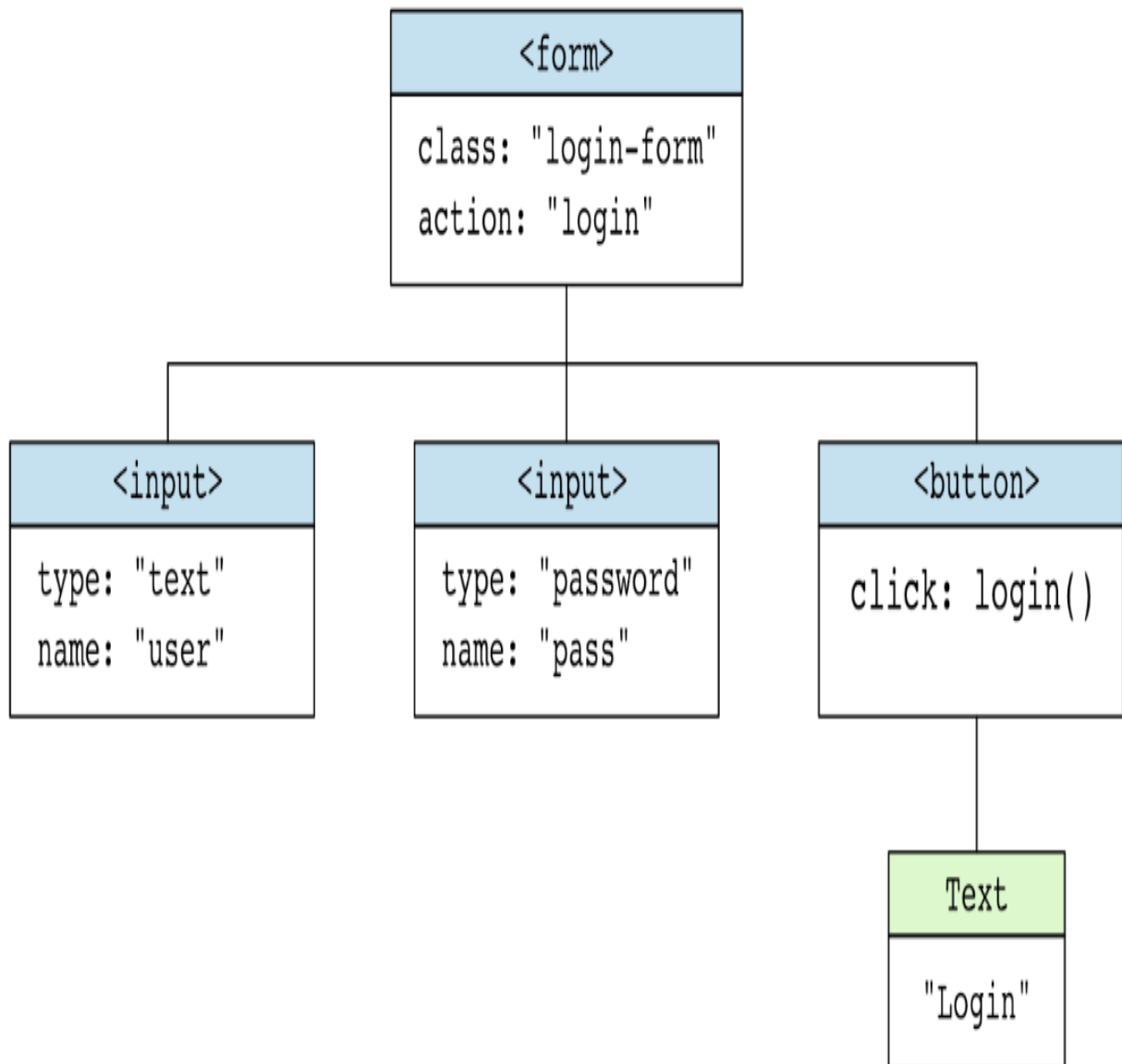
- `tag`: the tag name of the HTML element.
- `props`: the attributes of the HTML element, including the event handlers inside an `on` property.
- `children`: the ordered children of the HTML element. If absent, the element is a leaf node.

And as we'll see, fragment nodes have a `children` array of nodes, similar to the `children` array of element nodes.

Using this virtual DOM representation allows the developer to describe how the view of their application—the rendered HTML—should look like. You—the framework author—implement the code that takes that virtual DOM representation and builds the real one in the browser. This way, you effectively separate the code that describes the view from the code that manipulates the DOM.

We can represent the virtual DOM in the previous example graphically as a tree, as shown in figure [3.3](#). The `<form>` element is the root node of the tree, and the two `<input>` and `<button>` elements are its children. The properties of each node are inside the node's box, such as the `action` and `class` attributes of the `<form>` element. The `<button>` element has a child node, a text node with the text "Login."

Figure 3.3 The virtual DOM is a representation of the DOM made of JavaScript objects



Note

As you can see in the diagram [3.3](#), the nodes of the tree have a title indicating their type. The HTML elements are written in all lowercase letters and between angle brackets, such as "<form>" or "<input>." Text nodes are inside a box whose title is simply "Text."

This representation of an application's view holds all the information that we need to know in order to unequivocally build the DOM. It maintains the hierarchy of the elements, the attributes, event handlers and position of the

child elements. If you are given such a virtual DOM, you can derive the corresponding HTML markup without any ambiguity.

Important

The virtual DOM is the blueprint of the view of an application. It describes how the final rendered HTML should look like, but it doesn't tell the framework how it should handle the details of building it.

Creating virtual trees manually is a tedious task that can result in errors, such as misspelling property names. To simplify the process of defining an application's view, you will write functions that generate each type of virtual node instead of having the developer do it manually. Although using these functions makes the virtual DOM definition process less painful, it's still not as convenient as writing HTML templates or JSX. However, it's a starting point.

Exercise 3.1

Given the HTML markup below, can you draw the corresponding virtual DOM tree diagram (similar to figure [3.3](#))?

```
<div id="app">
  <h1>TODOs</h1>
  <input type="text" placeholder="What needs to be done?">

  <ul>
    <li>
      <input type="checkbox">
      <label>Buy milk</label>
      <button>Remove</button>
    </li>
    <li>
      <input type="checkbox">
      <label>Buy eggs</label>
      <button>Remove</button>
    </li>
  </ul>
</div>
```

Find the solution at <http://mng.bz/orpD>.

3.3 Getting ready

You want to make sure you've read appendix A and set up the project structure. All the code you'll write in this book will be part of the *runtime* package, which is the part of the framework that runs in the browser. So, when I refer to the *src/* directory, I mean that of the *runtime* package until further notice.

You want to create a file called *h.js* inside the *src/* directory. This file is where you'll write most of the code in this chapter. Also create a *utils/* directory inside *src/*, and add a file called *arrays.js* inside it. Here you'll write an utility function to filter `null` and `undefined` values from an array.

Your *runtime* package should look like this (the configuration files have been omitted, and in bold font are the files you just created):

```
runtime/
├── src/
│   ├── utils/
│   │   └── arrays.js
│   ├── h.js
│   └── index.js
```

Let's get started writing some code.

3.4 Types of nodes

As far as we've seen, you can have three different types of DOM nodes that need to be represented as virtual nodes:

- *Text nodes* — They represent text content.
- *Element nodes* — The most common type of node; they represent HTML elements that have a tag name, such as a `'div'` or a `'p'`.
- *Fragment nodes* — They represent a collection of nodes that don't have a parent node until they are attached to the DOM. (We haven't covered them yet, but we will shortly.)

These have different properties, so we need to represent them differently. Later on, you'll have to write code that operates on the virtual nodes and do

something different depending on the type of node. It's therefore a good idea to define a constant for each of these types, so you avoid typos.

Inside the *h.js* file, write the following code:

```
export const DOM_TYPES = {  
  TEXT: 'text',  
  ELEMENT: 'element',  
  FRAGMENT: 'fragment',  
}
```

You have defined three constants, one for each type of node:

- `DOM_TYPES.TEXT`: the type for a text node, which is `'text'`.
- `DOM_TYPES.ELEMENT`: the type for an element node, which is `'element'`.
- `DOM_TYPES.FRAGMENT`: the type for a fragment node, which is `'fragment'`.

Let's now implement the functions that create the virtual nodes, starting with element nodes.

3.5 Element nodes

Element nodes are the most common type of virtual node: they represent the regular HTML elements that you use to define the structure of your web pages. To name a few, you have `<h1>` through `<h6>` for headings, `<p>` for paragraphs, `<ul>` and `<ol>` for lists, `<a>` for links, and `<div>` for generic containers. These nodes have a tag name (such as `'p'`), attributes (such as a class name or the `type` attribute of an `<input>` element), and children nodes (the nodes that are inside of them, between the opening and closing tags).

You'll now implement a function `h()` to create element nodes taking three arguments:

- `tag` — the element's tag name.
- `props` — an object with its attributes (that we'll call *props*, for properties).
- `children` — an array of its children nodes.

The name `h()` is short for *hyperscript*, or a script that creates hypertext. (Recall that HTML is acronym for *HyperText Markup Language*.) The name `h()` for the function is a common one used in some frontend frameworks, probably because it's short and easy to type, which is important because you'll be using it a lot.

`h()`, `hyperscript()`, or `createElement()`

React uses the `React.createElement()` function to create virtual nodes. It's a long name, but you typically never call that function directly, as you use JSX instead. Each HTML element you write in JSX is transpiled to a `React.createElement()` call.

Other frameworks, such as Vue, do name the virtual node producing function `h()`. Mithril, for example, gives the user a function called `m()` to create the virtual DOM, but it's the same idea. Internally, Mithril implements the virtual node creating function named as `hyperscript()`. The user facing function has a nice and short name (`m()`), but the internal function has this more descriptive name.

The `h()` function should return a virtual node object with the passed in tag name, props, and children, plus a `type` property set to `DOM_TYPES.ELEMENT`. You want to give default values to the `props` and `children` parameters, so that you can call the function with only the tag name, as in `h('div')`. This should be equivalent to calling `h('div', {}, [])`.

Some child nodes might come as `null` (I'll explain why in a minute), so you want to filter them out. To filter `null` values from an array, you'll write a function called `withoutNulls()` in the next section (you'll see that function imported in the next listing, but it's not implemented yet).

Some child nodes inside the passed in `children` array might happen to be strings, not objects representing virtual nodes. In this case, you want to transform them into virtual nodes of type `DOM_TYPES.TEXT`. You'll do this using a function called `mapTextNodes()` that you'll write later.

Let's now implement the `h()` function. In the `h.js` file, write the code in bold in listing [3.1](#).

Listing 3.1 The `h()` function to create element virtual nodes (*h.js*)

```
import { withoutNulls } from './utils/arrays'

export const DOM_TYPES = {
  TEXT: 'text',
  ELEMENT: 'element',
  FRAGMENT: 'fragment',
}

export function h(tag, props = {}, children = []) {
  return {
    tag,
    props,
    children: mapTextNodes(withoutNulls(children)),
    type: DOM_TYPES.ELEMENT,
  }
}
```

Let's implement the `withoutNulls()` and `mapTextNodes()` functions now.

3.5.1 Conditional rendering—removing null values

When using *conditional rendering*, that is, rendering nodes only when a condition is met, some children might be `null` in the array, and this means that they shouldn't be rendered at all. We want these `null` values to be removed from the array of children.

Let's use our TODO app as an example. Recall that the add new to-do button is disabled when there's no text in the input, or the text is too short. If instead of disabling the button you decided to remove it from the page, you'd have a conditional like the following:

```
{
  tag: 'div',
  children: [
    { tag: 'input', props: { type: 'text' } },
    addTodoInput.value.length > 2
      ? { tag: 'button', children: ['Add'] }
      : null
  ]
}
```

When the condition `addTodoInput.value.length > 2` is false, a `null` node is added to the `div` node's children array:

```
{
  tag: 'div',
  children: [
    { tag: 'input', props: { type: 'text' } },
    null
  ]
}
```

This `null` value means that the button shouldn't be added to the DOM. The simplest way to make this work is to filter out `null` values from the children array when a new virtual node is created, so that a `null` node isn't passed around the framework:

```
{
  tag: 'div',
  children: [
    { tag: 'input', props: { type: 'text' } }
  ]
}
```

Inside the *utils/arrays.js* file, write a function called `withoutNulls()` that takes an array and returns a new array with all the `null` values removed.

```
export function withoutNulls(arr) {
  return arr.filter((item) => item !== null)
}
```

Note the usage of the `!==` operator, as opposed to using `!=`: this is so you remove both `null` and `undefined` values. You aren't expecting `undefined` values, but this way you'll remove them if they appeared—just in case. (Your linter might complain about this if you have the [`eqeqeq` rule enabled](#), but you can disable it for this line; tell the linter you know what you're doing.)

3.5.2 Mapping strings to text nodes

After filtering out the `null` values from the children array, you pass the result to a `mapTextNodes()` function. We said that this function transforms strings into text virtual nodes. Why do we want to do this? Well, just as a convenience for creating text nodes, so instead of doing:

```
h('div', {}, [hString('Hello '), hString('world!')])
```

we can do:

```
h('div', {}, ['Hello ', 'world!'])
```

As you can anticipate, you'll use text children a lot, so this will make your life easier—if only a little bit. Let's now write that missing `mapTextNodes()` function. In the *h.js* file, below the `h()` function, write the code for the function as follows:

```
function mapTextNodes(children) {  
  return children.map((child) =>  
    typeof child === 'string' ? hString(child) : child  
  )  
}
```

You've used the `hString()` function to create text virtual nodes out of strings, but that function doesn't exist yet. That's the next thing you'll do: implement the function that creates text virtual nodes.

3.6 Text nodes

Text nodes are the nodes in the DOM that contain text. They have no tag name, no attributes, and no children; just text.

Creating text nodes is the simplest of the three types of virtual nodes. A text virtual node is simply an object with the `type` property set to `DOM_TYPES.TEXT`, and the `value` property set to the text content. In the *h.js* file, write the `hString()` function like so:

```
export function hString(str) {  
  return { type: DOM_TYPES.TEXT, value: str }  
}
```

That was easy! You're just missing the `hFragment()` function to create fragment virtual nodes.

3.7 Fragment nodes

A fragment is a type of virtual node used to group multiple nodes that need to be attached to the DOM together, but don't have a parent node in the DOM. You can think of them as simply being a container for an array of virtual nodes.

Note

Fragments exist in the Document API, they're used to create subtrees of the DOM that can be appended to the document at once. They're represented by the [DocumentFragment class](#), and can be created using the `document.createDocumentFragment()` method. We won't be using the `DocumentFragment` to insert the virtual fragment nodes into the DOM, but it's good to know that they exist.

Let's implement the `hFragment()` function to create fragment virtual nodes.

3.7.1 Implementing fragment nodes

Fragments are just an array of child nodes, so its implementation is very simple. In the `h.js` file, write the `hFragment()` function as follows:

```
export function hFragment(vNodes) {
  return {
    type: DOM_TYPES.FRAGMENT,
    children: mapTextNodes(withoutNulls(vNodes)),
  }
}
```

Same as before, you've filtered out the `null` values from the array of children, and then you've mapped the strings in the `children` array to text virtual nodes. That's all there is to it!

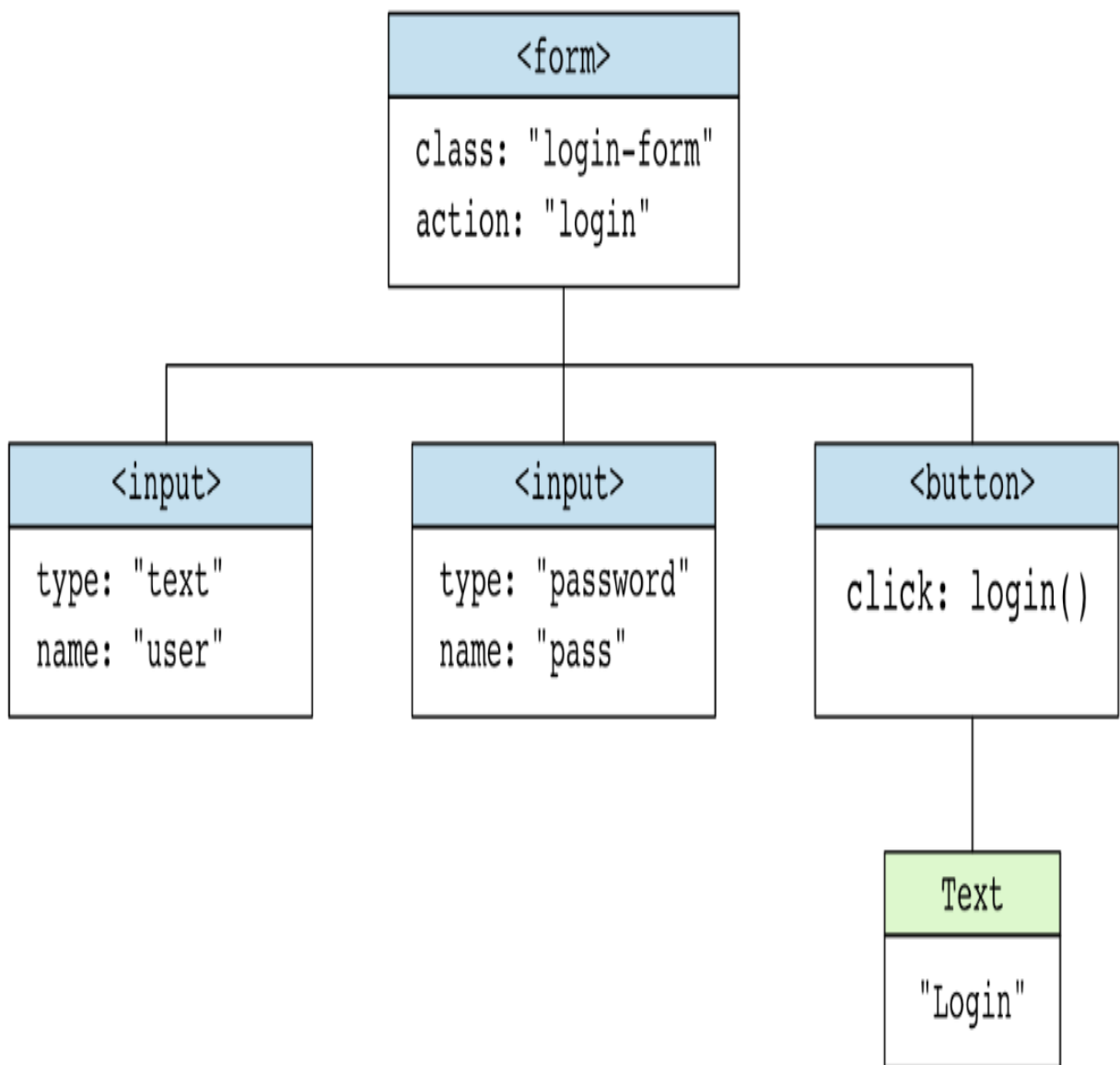
3.7.2 Testing the virtual DOM functions

You can now use the `h()`, `hString()`, and `hFragment()` functions to create virtual DOM representations of the view of your application. What we'll implement next is the code that takes in a virtual DOM and creates the real DOM for it, but first, let's put the virtual DOM functions to the test. Let's use the `h()` function to define the view of a login form:


```
h('form', { class: 'login-form', action: 'login' }, [
  h('input', { type: 'text', name: 'user' }),
  h('input', { type: 'password', name: 'pass' }),
  h('button', { on: { click: login } }, ['Login'])
])
```

This will create a virtual DOM depicted in figure [3.4](#). Arguably, using the `h()` functions is more concise than defining the virtual DOM manually, as a tree of JavaScript objects.

Figure 3.4 Example of creating a virtual DOM tree



This virtual DOM, passed to the framework, will be used to create the real DOM, the HTML code that will be rendered in the browser. In this case, the HTML markup would be:

```
<form class="login-form" action="login">
  <input type="text" name="user">
  <input type="password" name="pass">
  <button>Login</button>
</form>
```

Note

The `<button>` element doesn't have a `click` event handler rendered in the HTML markup. This is because the framework will add the event handler to the button programmatically, when it is attached to the DOM. The event handlers added from JavaScript aren't shown in the HTML.

As you can imagine, you typically don't define the virtual DOM for your entire application in one place; that can get unwieldy as the application grows in size. What you do instead is split the view into subparts, each of which we call a *component*. Components are the cornerstone of frontend frameworks: they allow us to break down a large application into smaller, more manageable pieces, each of which in charge of a specific part of the view.

Exercise 3.2

Using the `h()` function, define the virtual DOM equivalent to the following HTML markup:

```
<h1 class="title">My counter</h1>
<div class="container">
  <button>decrement</button>
  <span>0</span>
  <button>increment</button>
</div>
```

Find the solution at <http://mng.bz/n1Md>.

Exercise 3.3

Create a function, called `lipsum()`, that takes in a number, and returns a virtual DOM consisting of a fragment with as many paragraphs as the number passed to the function. Every paragraph should contain the text: *"Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat."*

This function might come in handy when you're building a UI and need some [placeholder text](#) to fill in the space, so you can see how the UI looks like with real content.

Find the solution at <http://mng.bz/vPza>.

Let's see what makes a component in your early version of the framework.

3.8 Components—the cornerstone of frontend frameworks

The component has emerged as the revolutionary concept that has made frontend frameworks so popular. The ability to break down a large application into smaller parts, each of which defines a specific part of the view, and manages its interaction with the user, has been a game changer (well, arguably; because a good use of the MVC or MVVM patterns could already get us as far). Every frontend framework uses the concept of components, and yours will be no different.

Let's take a small detour from the implementation of the virtual DOM to understand how you'll decompose the view of your application into components using your first version of the framework.

3.8.1 What is a component?

A component in your framework will be a mini-application of its own: it'll have its own internal state and lifecycle, and it'll be in charge of rendering a part of the view. It'll communicate with the rest of the application emitting

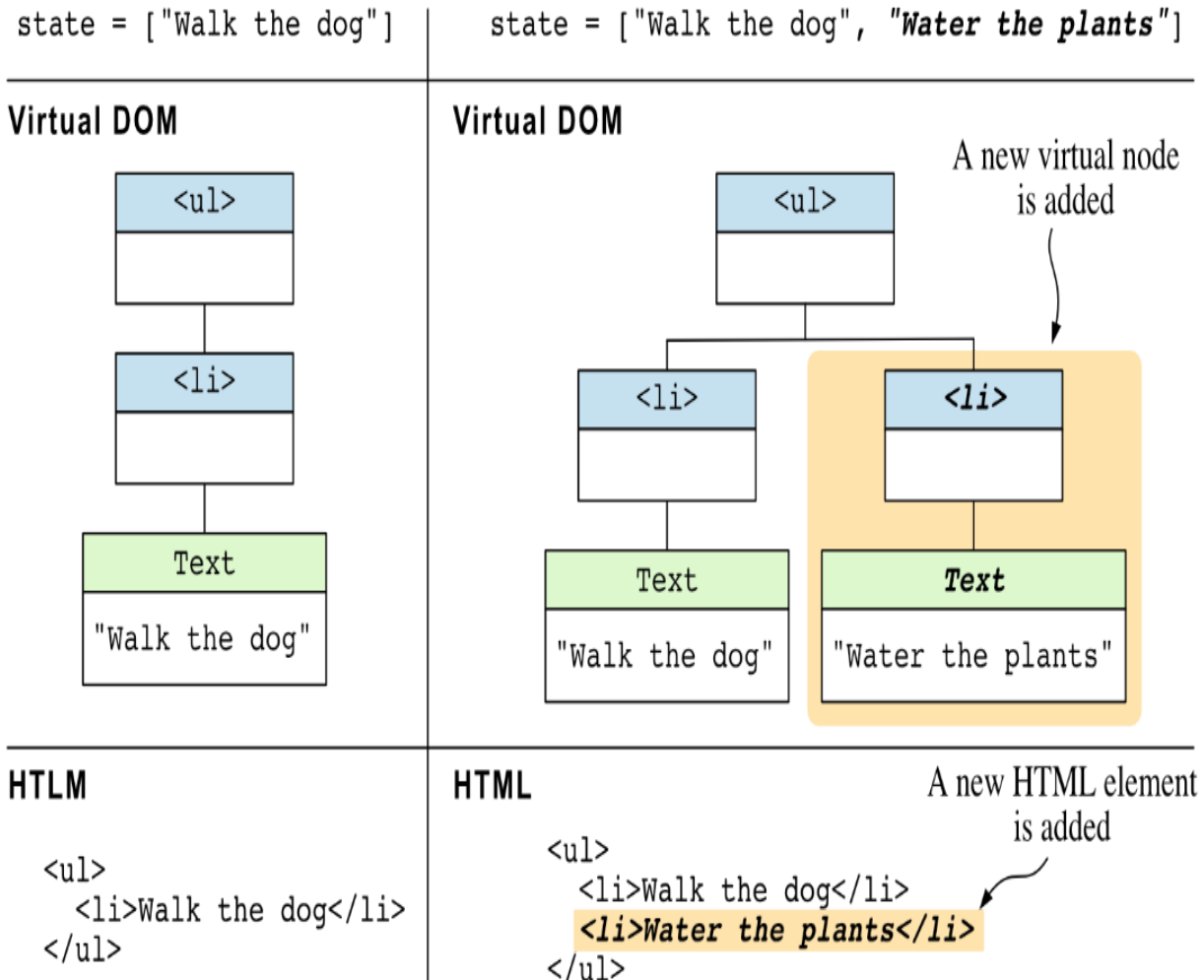
events, and receive *props* (data passed to the component from the outside), re-rendering its view when a new set of props is passed to it. But it'll take us a few chapters to get there. Your first version of a component will be much simpler: a pure function that takes in the state of the whole application and returns the virtual DOM representing the view of the component. In a later chapter, you'll make components have their own internal state and lifecycle, but for now, let's start by breaking down the view of the application into pure functions that, given the state, return the virtual DOM representing a part of it.

3.8.2 The virtual DOM as a function of the state

The view of an application depends on the state of the application, thus we can say that the virtual DOM is a function of the state. Each time the state changes, the virtual DOM should be re-evaluated, and the framework needs to update the real DOM accordingly. This dependency between the state and the view is depicted in figure [3.5](#). In the left column, the state consists in a list of to-dos with just one to-do, "Walk the dog." In the right column, the state changes to also include a second to-do, "Water the plants." Notice how the virtual DOM changes accordingly, and how the HTML markup changes as well.

Figure 3.5 The view of an application is a function of the state. When a new to-do is added to the state, the virtual DOM is re-evaluated and the DOM is updated with the new to-do

A new to-do is added to the array
of to-dos: the state changes.



This means that, to produce the virtual DOM representing the view, the current state of the application must be taken into account, and that, when the state changes, the virtual DOM must be re-evaluated. If we generate the virtual DOM that represents the view of the application by calling a function that receives the state as parameter, we can easily re-evaluate it when the state changes. For example, in the case of our TODOs application, the virtual DOM for the list of to-dos (consisting of only the to-do description, for the sake of simplicity) could be generated by a function like this:

```
function TodosList(todos) {
  return h('ul', {}, todos.map((todo) => h('li', {},
```

```
[todo])))  
}
```

If we call the `TodosList()` function with the following list of to-dos as argument:

```
TodosList(['Walk the dog', 'Water the plants'])
```

the function would return the following virtual DOM (the empty `props` objects have been omitted for brevity):

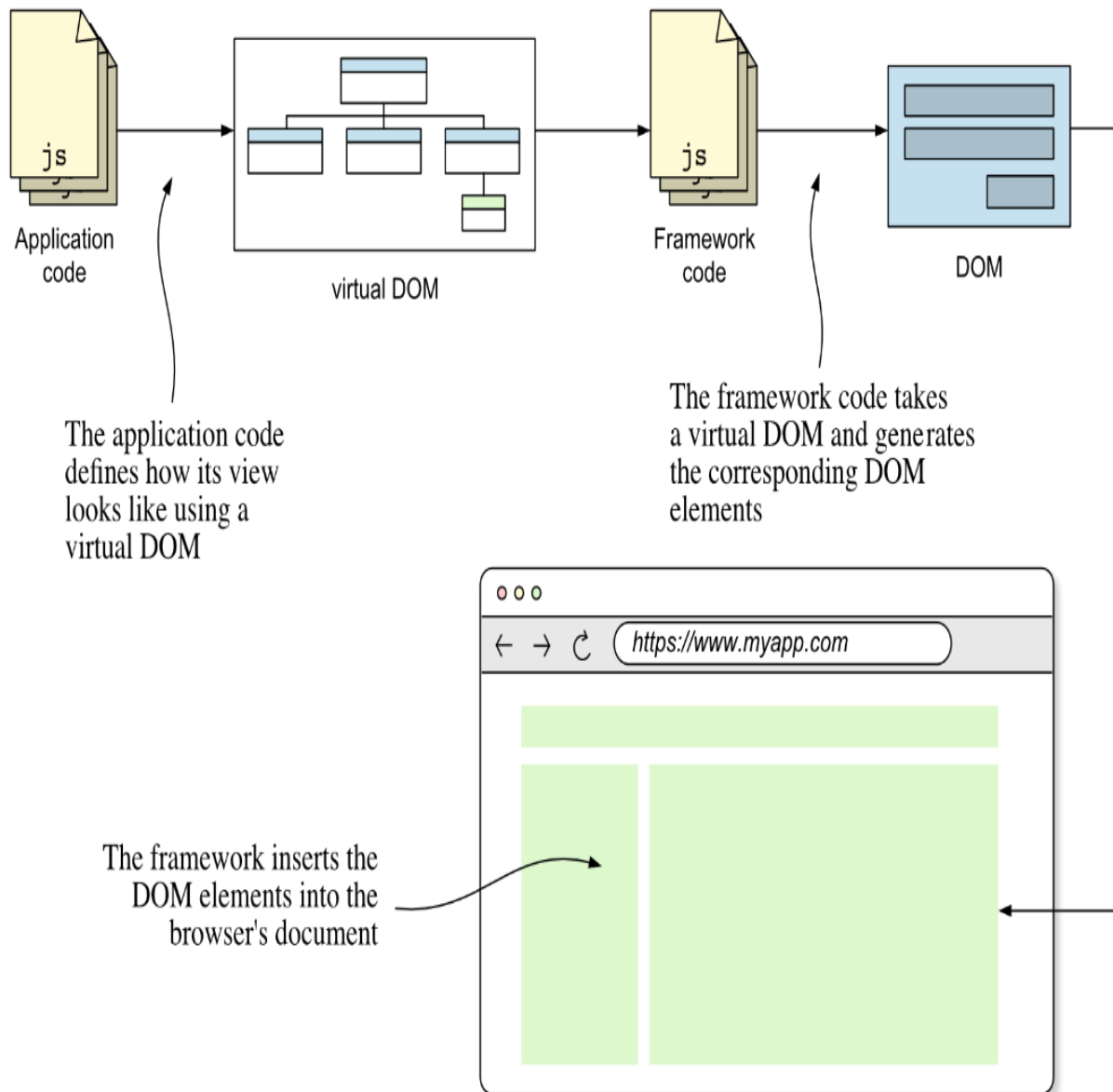
```
{  
  tag: 'ul',  
  type: 'element',  
  children: [  
    { tag: 'li', children: [{ type: 'text', value: 'Walk the  
dog' }] },  
    { tag: 'li', children: [{ type: 'text', value: 'Water the  
plants' }] }  
  ]  
}
```

This virtual DOM representing the list of to-dos would be rendered by the framework into the following HTML:

```
<ul>  
  <li>Walk the dog</li>  
  <li>Water the plants</li>  
</ul>
```

You can see this process summarized in figure [3.6](#): the application code—written by the developer—generates the virtual DOM describing the view, then the framework—written by yourself—creates the real DOM from the virtual DOM and inserts it into the browser’s document.

Figure 3.6 The application creates the virtual DOM that the framework renders into the DOM



One nice property of using functions to produce the virtual DOM, receiving the application's state as argument, is that we can break the view into smaller parts. Pure functions—functions which don't produce any side effects and always return the same result for the same input arguments—can be easily composed to build more complex views.

Exercise 3.4

Write a function called `MessageComponent()` that takes an object with two properties:

- `level` — a string that can be either `'info'`, `'warning'`, or `'error'`
- `message` — a string with the message to display.

The function should return a virtual DOM that represents a message box with the message and the corresponding CSS class depending on the level. The CSS classes are `message--info`, `message--warning`, and `message--error` for the different levels, and the markup should look like this:

```
<div class="message message--info">
  <p>This is an info message</p>
</div>
```

Find the solution at <http://mng.bz/46ZQ>.

3.8.3 Composing views—components as children

Pure functions have the nice property that they can be composed nicely to build more complex functions. If we generate the virtual DOM to represent the view of the application by composing smaller functions, we can easily break the view into smaller parts. These sub-functions represent a part of the view, the *components*. The arguments passed to a component are known as *props*, as we’ve already mentioned.

Components v1.0

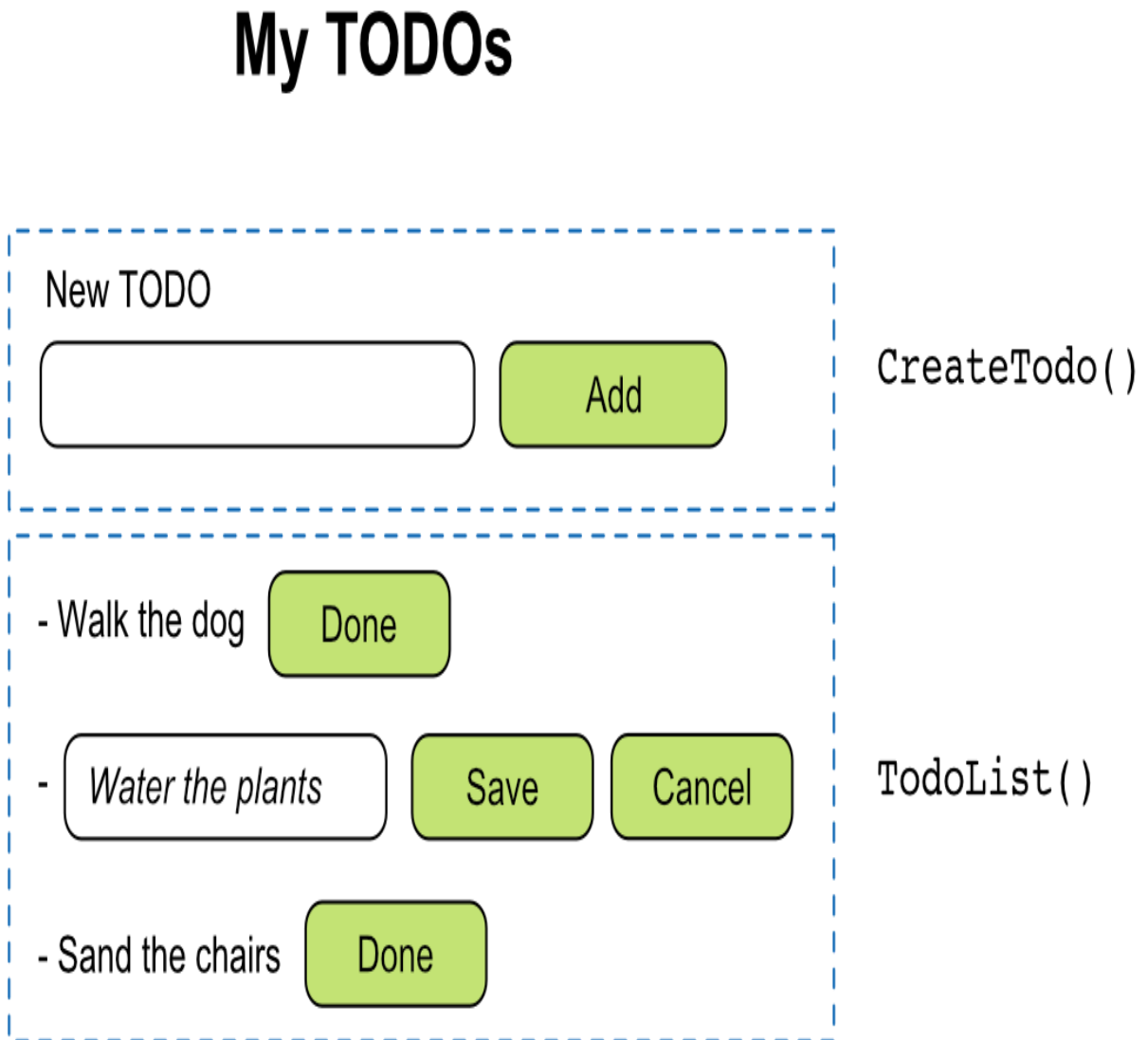
Components are functions that generate the virtual DOM for a part of the application’s view. They take the state of the application, or part of it, as their argument. The arguments passed to a component, the data coming from outside the component, are known as *props*.

This definition of a component will change later in chapter 9, as components will be more than pure functions and will handle their own state and lifecycle.

Let’s work an example with the TODOs application view. If we don’t break the view into smaller parts, we’ll have a single function that generates the virtual DOM for the entire application. Such a function would be long and hard to understand by other developers—and probably by ourselves as well. But we can clearly distinguish two parts in the view: the form to add a new

to-do and the list of to-dos (figure [3.7](#)); those can be generated by two different functions. That is, we can break the view into two sub-components.

Figure 3.7 The TODOs application can be broken into two sub-components: `CreateTodo()` and `TodoList()`



So the virtual DOM for the whole application could be created by a component like the following:

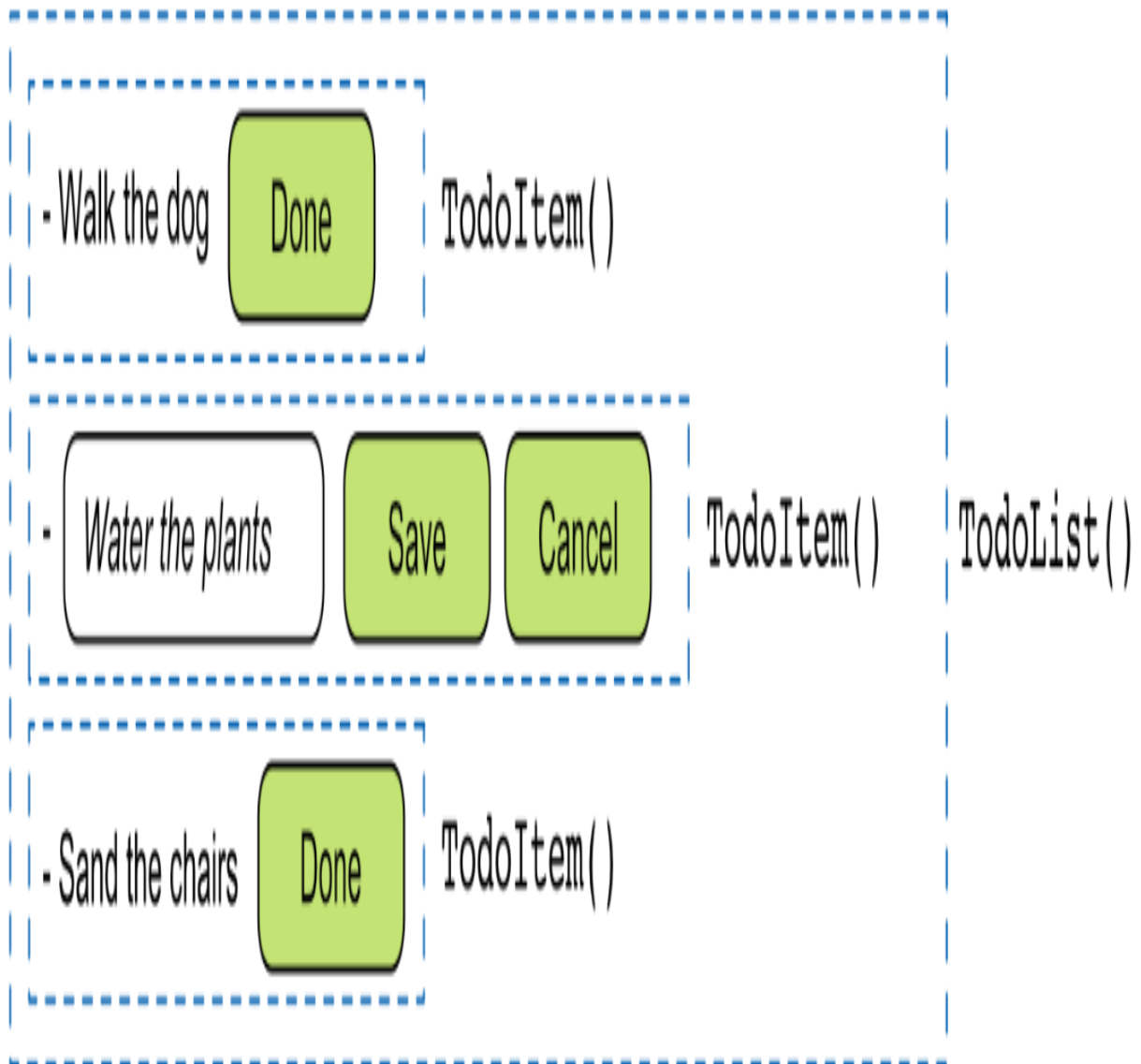
```
function App(state) {  
  return hFragment([
```

```
    h('h1', {}, ['My TODOs']),  
    CreateTodo(state),  
    TodoList(state)  
  ])  
}
```

Note that in this case, there isn't a parent node in the virtual DOM that contains the header of the application and the two sub-components: we use a fragment to group the elements together. Also note the naming convention: the functions that generate the virtual DOM are written in *PascalCase*. This is to signal that they are components that create a virtual DOM tree, and not regular functions.

Similarly, the `TodoList()` component—as you've probably guessed—can be further broken down into another sub-component, the `TodoItem()`. You can see this illustrated in figure [3.8](#).

Figure 3.8 The `TodoList()` component can be broken down into a `TodoItem()` sub-component



and thus, the `TodoList()` component would look similar to the following:

```
function TodoList(state) {  
  return h('ul', {},  
    children: state.todos.map(  
      (todo, i) => TodoItem(todo, i, state.editingIdxs)  
    )  
  )  
}
```

The `TodoItem()` component would render a different thing depending on whether the to-do is in "read" or "edit" mode. Those could be further

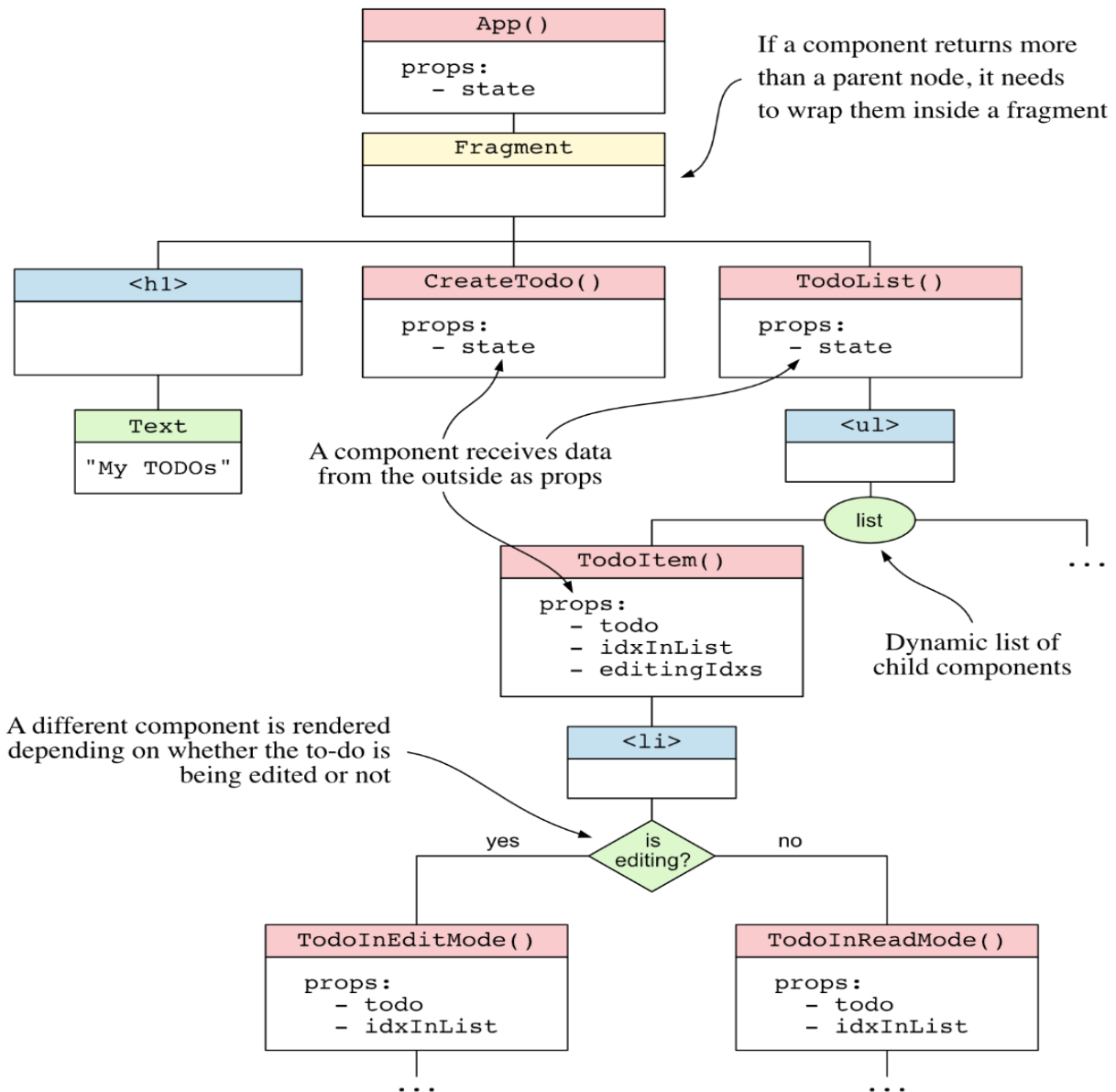
decomposed into two different sub-components: `TodoInReadMode()` and `TodoInEditMode()`. It'd be something like the following:

```
// idxInList is the index of this todo item in the list of todos.  
// editingIdxs is a Set of indexes of todos that are being edited.  
function TodoItem(todo, idxInList, editingIdxs) {  
  const isEditing = editingIdxs.has(idxInList)  
  
  return h('li', {}, [  
    isEditing  
      ? TodoInEditMode(todo, idxInList)  
      : TodoInReadMode(todo, idxInList)  
  ]  
)  
}
```

Defining the views of our application using pure functions—the components—allows you to easily compose them to build more complex views. This is probably not new to you; you've been decomposing your applications into a hierarchy of components when using frontend frameworks like React, Vue, Svelte or Angular.

We can visualize the hierarchy of components for the example above in a tree, as shown in figure [3.9](#).

Figure 3.9 The hierarchy of components of the TODOs application, where each component has below the virtual DOM nodes it generates



In this diagram we see the view of the application as a tree of components with the virtual DOM nodes they generate below them. The `App()` component is the root of the tree, and it has three children: an `<h1>` element, and the `CreateTodo()` and `TodoList()` components. A component can only return a single virtual DOM node as its root, so the three children of `App()` are grouped together in a fragment.

Then, following the hierarchy down, we see the `TodoList()` component has a single child, the `<ul>` element, which in turn has a list of `TodoItem()` components. The ellipsis in the tree indicates that the `<ul>` element might

have more children than the ones shown in the diagram, a number that depends on how many to-dos are in the list. Finally, the `TodoItem()` component has two children: the `TodoInEditMode()` and `TodoInReadMode()` components. These components would render more virtual DOM nodes, but we don't show them in the diagram for simplicity.

Note

As you can see in the diagram [3.9](#), the component nodes have their title written in *PascalCase* and include the parenthesis to indicate they are functions. Fragments are titled "Fragment."

Now that you understand how to break down the view of your application into components—pure functions that generate the virtual DOM given the state of the application—you're ready to implement the code in the framework which is in charge of mounting the virtual DOM returned by the `h()` functions to the browser's DOM. You'll do exactly that in the next chapter.

3.9 Summary

- The virtual DOM is the blueprint for the view of the application: it allows the developer describe how the view should look like in a declarative way (similar to what an architect does with the blueprints of a house) and moves the responsibility of manipulating the DOM to the framework.
- Thanks to using a virtual DOM to declare how the view should look like, the application developer is freed from having to know how to manipulate the DOM using the Document API, and they don't need to mix business logic with DOM manipulation code.
- A component is a pure function—a function with no side effects—that takes the state of the application as input and returns a virtual DOM tree representing a chunk of the view of the application. In later chapters, the definition of a component will be extended to include the ability to have internal state and a lifecycle that's independent from that of the application.
- There are three types of virtual nodes: text nodes, element nodes, and fragment nodes. The most interesting one is the element node, as it

represents the regular HTML elements that can have attributes, children, and event listeners.

- The `hString()`, `h()`, and `hFragment()` functions are used to create text, element, and fragment virtual nodes, respectively. The virtual DOM can be directly declared as a tree of JavaScript objects, but calling these functions makes the process simpler.
- Fragment virtual nodes consist of an array of children virtual nodes.
- Fragment virtual nodes are useful when a component would return a list of virtual nodes without a parent node. The DOM—and by extension the virtual DOM—is a tree data structure, every level in the tree (except the root) must have a parent node, so a fragment node can be used to group a list of virtual nodes.

4 Mounting and destroying the virtual DOM

This chapter covers

- Creating HTML nodes from virtual DOM nodes
- Inserting the HTML nodes into the browser's document
- Removing HTML nodes from the browser's document

In the previous chapter, you learnt what the virtual DOM is and how to create it. You implemented the `h()`, `hString()`, and `hFragment()` functions to create virtual nodes of type `element`, `text`, and `fragment`, respectively. Now it's time to learn how to create the real DOM nodes from the virtual DOM nodes, and insert them into the browser's document. This is achieved using the Document API, as you'll see in this chapter.

When the view of your application is no longer needed, you want to remove the HTML nodes from the browser's document. You'll learn how to do this in this chapter as well.

Important

You can find all the listings in this chapter in the [listings/ch04 directory](#) of the book's repository.

The code you write in this chapter is for the framework's first version, which you'll publish in chapter [6](#). Therefore, the code in this chapter can be checked out from [the *ch6* label](#):

```
$ git switch --detach ch6
```

Code catch-up

In the last chapter you implemented the `h()`, `hString()`, and `hFragment()` functions. These functions are used to create virtual nodes of type `'element'`,

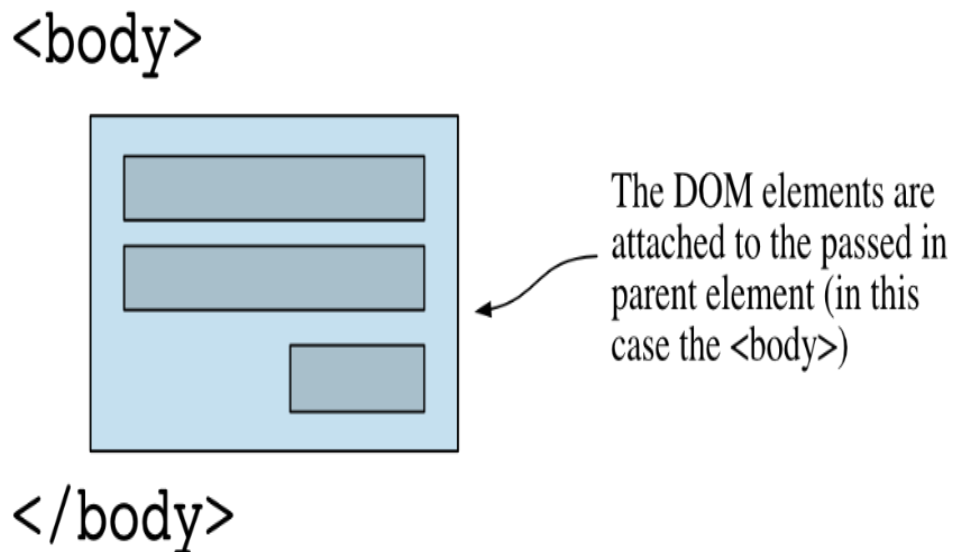
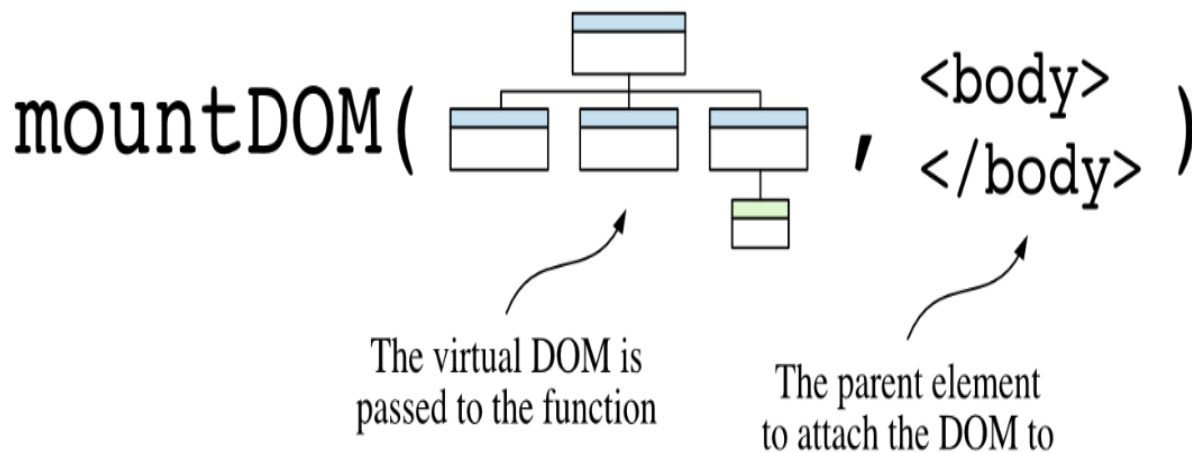
'text', and 'fragment', respectively. You also defined the `DOM_TYPES` object containing the different types of virtual nodes.

4.1 Mounting the virtual DOM

Given a virtual DOM tree, you want your framework to create the real DOM tree out of it and attach it to the browser's document. We call this process *mounting* the virtual DOM. You implement this code in the framework, so that the developers using it don't need to use the Document API themselves. You'll implement this process in the `mountDOM()` function.

Figure [4.1](#) is a visual representation of how the `mountDOM()` function works. You can see the first argument is a virtual DOM—represented in the style of our diagrams—and the second argument is the parent element where we want the view inserted, in this case the document's `<body>` element. The result is a DOM tree attached to the parent element—the `<body>` of the document.

Figure 4.1 Mounting a virtual DOM to the browser's document `<body>` element



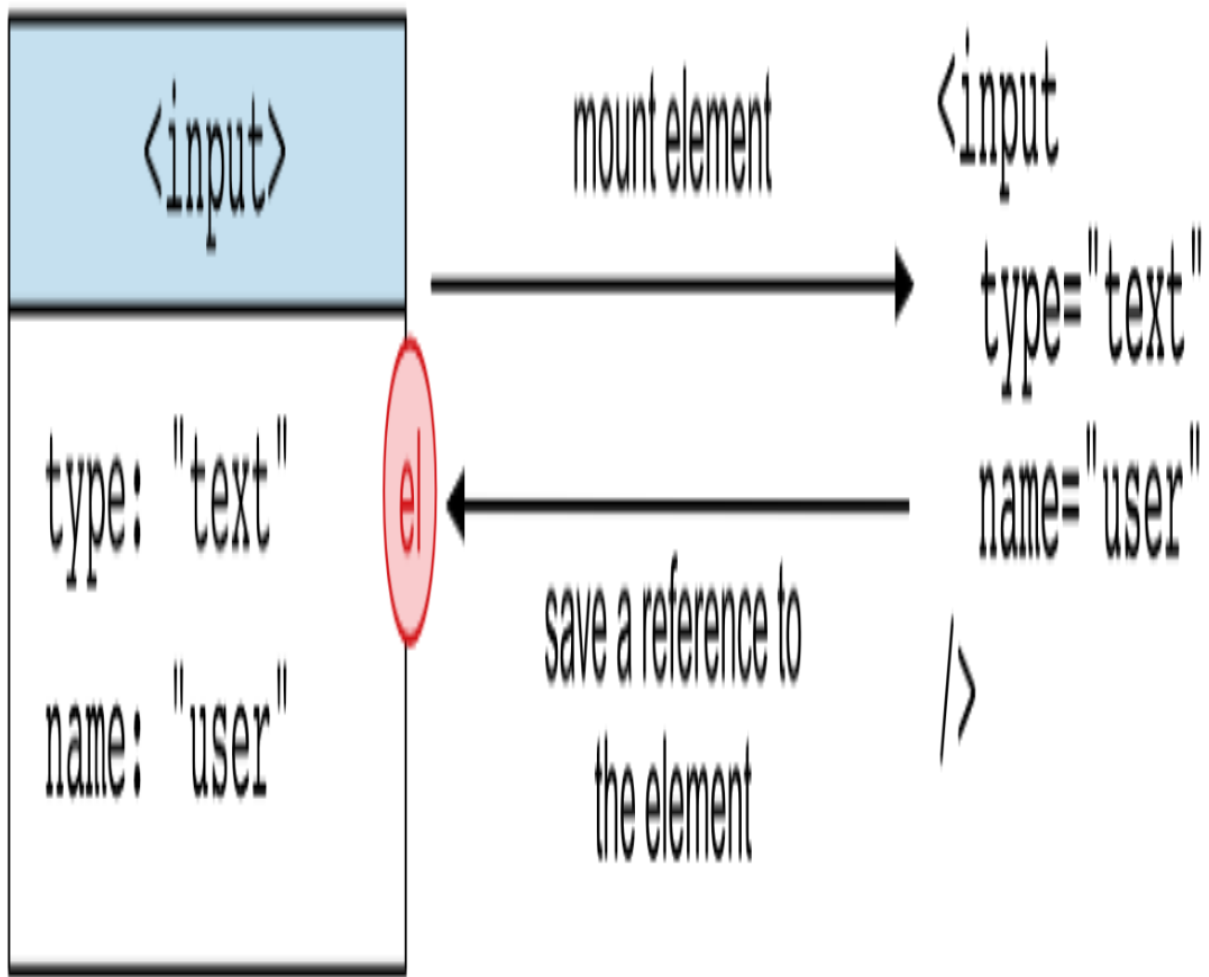
Warning

In many examples in this book and the tests I wrote in the repository I mount the application inside the `<body>` element of the document. This is okay for the examples and tests, but you should avoid doing this in your applications. Some third-party libraries might attach elements to the `<body>` element, and this might cause your reconciliation algorithm to work incorrectly.

When the `mountDOM()` function creates each of the DOM nodes for the virtual DOM, it needs to save a reference to the real DOM node in the virtual node,

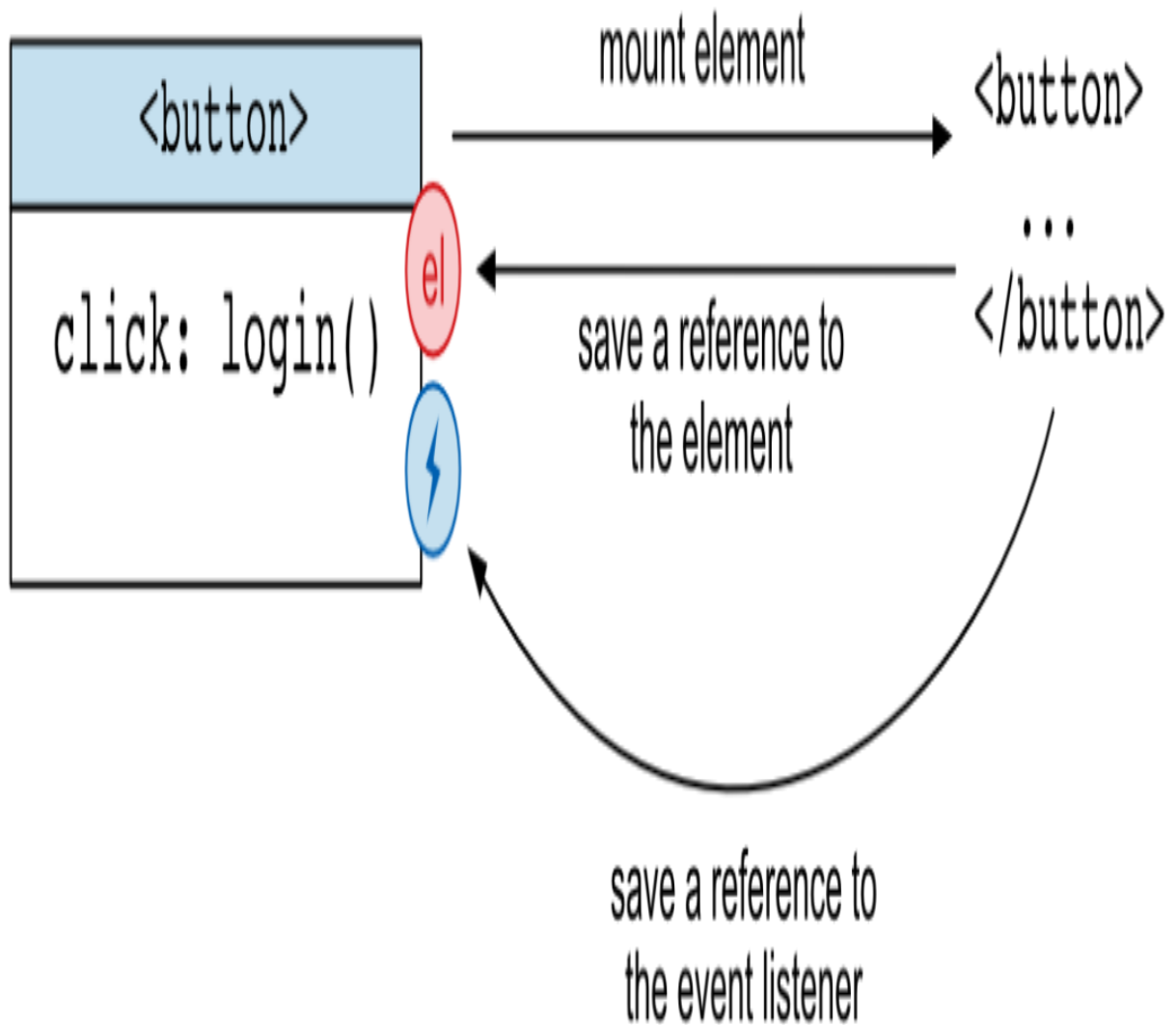
under the `el` property (`el` for *element*). You can see this in figure [4.2](#). This reference is used by the reconciliation algorithm you'll write in chapters [7](#) and [8](#), to know what DOM nodes to update.

Figure 4.2 The virtual node's `el` property keeps a reference to the real DOM node



Similarly, if the node included event listeners, the `mountDOM()` function saves a reference to the event listener in the virtual node, under the `listeners` property.

Figure 4.3 The virtual node's `listeners` property keeps a reference to the event listener



Saving these references has a double purpose. First, it allows the framework to remove the event listeners and detach the element from the DOM when the virtual node is unmounted. Second, it's required by the reconciliation algorithm to know what element in the DOM needs to be updated. This will become clear in chapter [7](#); for now, bear with me.

Using the example from earlier, the virtual DOM we defined as:

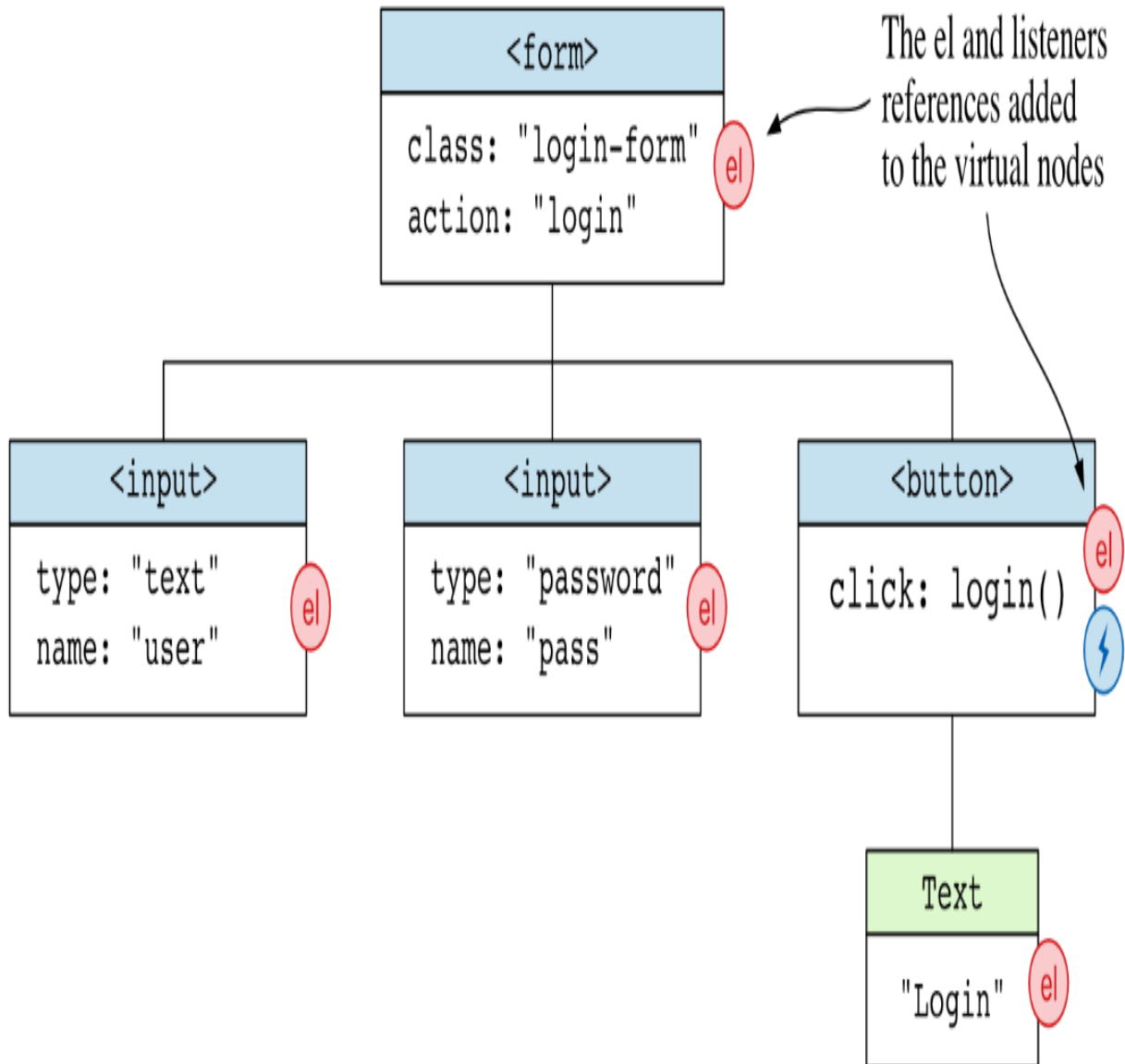
```
const vdom = h('form', { class: 'login-form', action: 'login'
}, [
  h('input', { type: 'text', name: 'user' }),
  h('input', { type: 'password', name: 'pass' }),
  h('button', { on: { click: login } }, ['Login'])
])
```

Passed to the `mountDOM()` function as follows:

```
mountDOM(vdom, document.body)
```

would result in the virtual DOM tree depicted in figure [4.4](#), where you can see the `el` and `listeners` references in the virtual nodes.

Figure 4.4 The login form virtual DOM example mounted to the browser's document `<body>` element. The virtual nodes keep a reference to the real DOM nodes in the `el` property, and to the event listeners in the `listeners` property (shown as a lightning icon).



This HTML tree would be attached to the `<body>` element, and the resulting HTML markup would be:

```
<body>
  <form class="login-form" action="login">
    <input type="text" name="user">
    <input type="password" name="pass">
    <button>Login</button>
  </form>
</body>
```

Different types of virtual nodes require different DOM nodes to be created, namely:

- A virtual node of type `text` requires a `Text` node to be created (via the `document.createTextNode()` method).
- A virtual node of type `element` requires an `Element` node to be created (via the `document.createElement()` method).

The `mountDOM()` function needs to differentiate between the different values of the `type` property of the virtual node, and create the appropriate DOM node.

With this in mind, let's implement the `mountDOM()` function.

4.1.1 Mounting virtual nodes into the DOM

Create a new file called *mount-dom.js* in the *src/* directory. Then, write the `mountDOM()` function as is in listing [4.1](#). The listing also includes some *TODO* comments towards the end of the file. You don't need to write those comments in your code; there are there as placeholders to show you were you will be implementing the missing functions later.

Listing 4.1 The `mountDOM()` function used to mount the virtual DOM to the browser's document (*mount-dom.js*)

```
import { DOM_TYPES } from './h'

export function mountDOM(vdom, parentEl) {
  switch (vdom.type) {
    case DOM_TYPES.TEXT: {
      createTextNode(vdom, parentEl) #1
      break
    }

    case DOM_TYPES.ELEMENT: {
      createElementNode(vdom, parentEl) #2
      break
    }

    case DOM_TYPES.FRAGMENT: {
      createFragmentNodes(vdom, parentEl) #3
      break
    }
  }
}
```

```

    }

    default: {
        throw new Error(`Can't mount DOM of type:
${vdom.type}`)
    }
}

// TODO: implement createTextNode()

// TODO: implement createElementNode()

// TODO: implement createFragmentNodes()

```

The function uses a `switch` statement that checks the type of the virtual node. Depending on the node's type, the appropriate function to create the real DOM node gets called. If the node type isn't one of the three supported types, the function throws an error. (If you made a mistake, like misspelling the type of a virtual node, this error will help you find it.)

4.1.2 Mounting text nodes

Text nodes are the simplest type of node to create because they don't have any attributes or event listeners. To create a text node, the Document API provides you with the [createTextNode\(\) method](#). It expects a string as an argument, which is the text that the text node will contain.

If you recall, the virtual nodes created by the `hString()` function you implemented earlier have the following structure:

```

{
  type: DOM_TYPES.TEXT,
  value: 'I need more coffee'
}

```

These virtual nodes have a `type` property, identifying them as a text node, and a `value` property, which is set to the string that the `hString()` function receives as an argument. This is the text that you need to pass to the `createTextNode()` method. After creating the text DOM node, you need to do two things:

1. Save a reference to the real DOM node in the virtual node, under the `el` property.
2. Attach the text node to the parent element.

Inside the *mount-dom.js* file, write the `createTextNode()` function as follows:

```
function createTextNode(vdom, parentEl) {  
  const { value } = vdom  
  
  const textNode = document.createTextNode(value) #1  
  vdom.el = textNode #2  
  
  parentEl.append(textNode) #3  
}
```

Let's now implement the `createFragmentNodes()` function.

Exercise 4.1

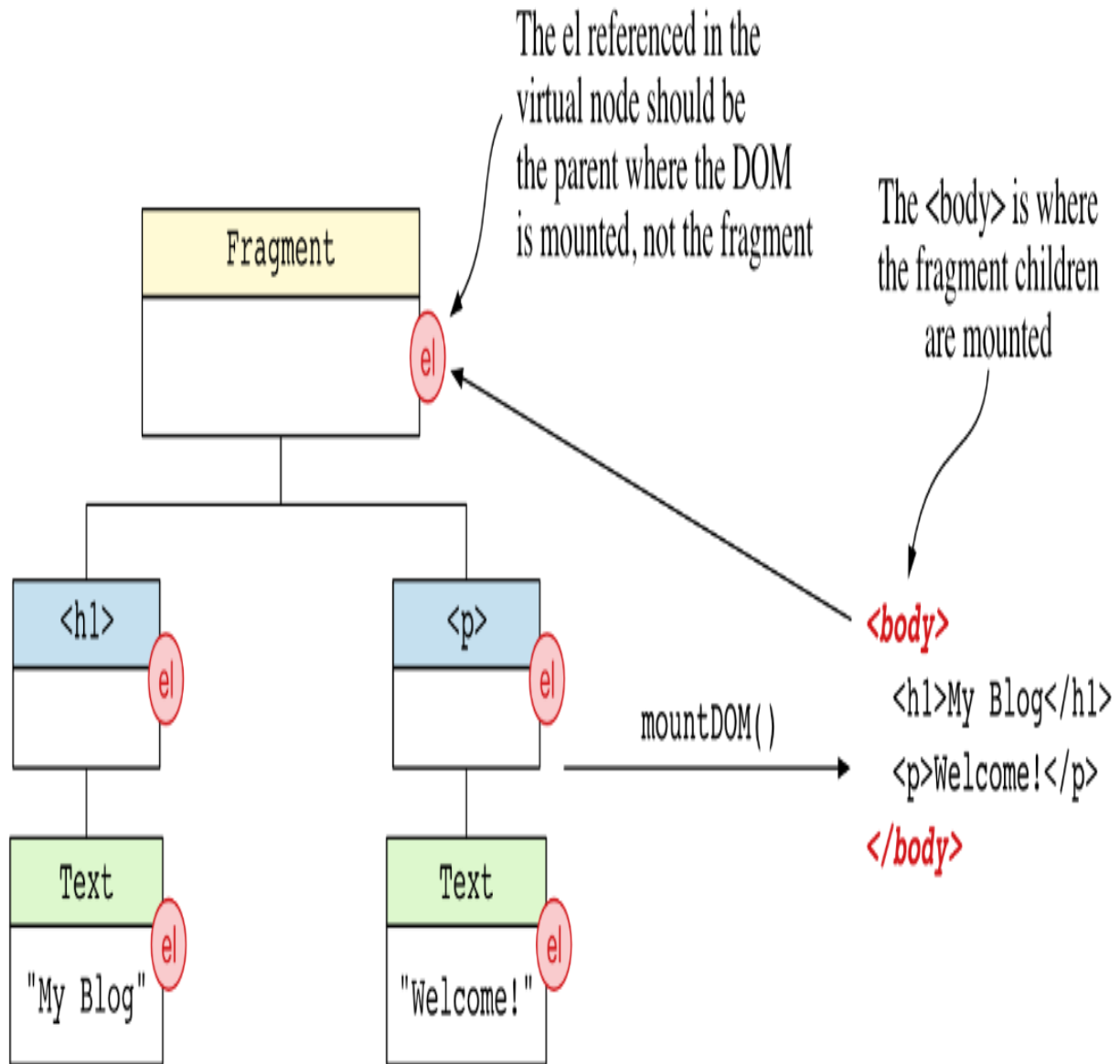
Using the `hString()` and `mountDOM()` functions, insert the text "OMG, so interesting!" below the headline of your local's newspaper website. (You'll need to copy/paste some of your code in the browser's console to do this.)

Find the solution at <http://mng.bz/g7oG>.

4.1.3 Mounting fragment nodes

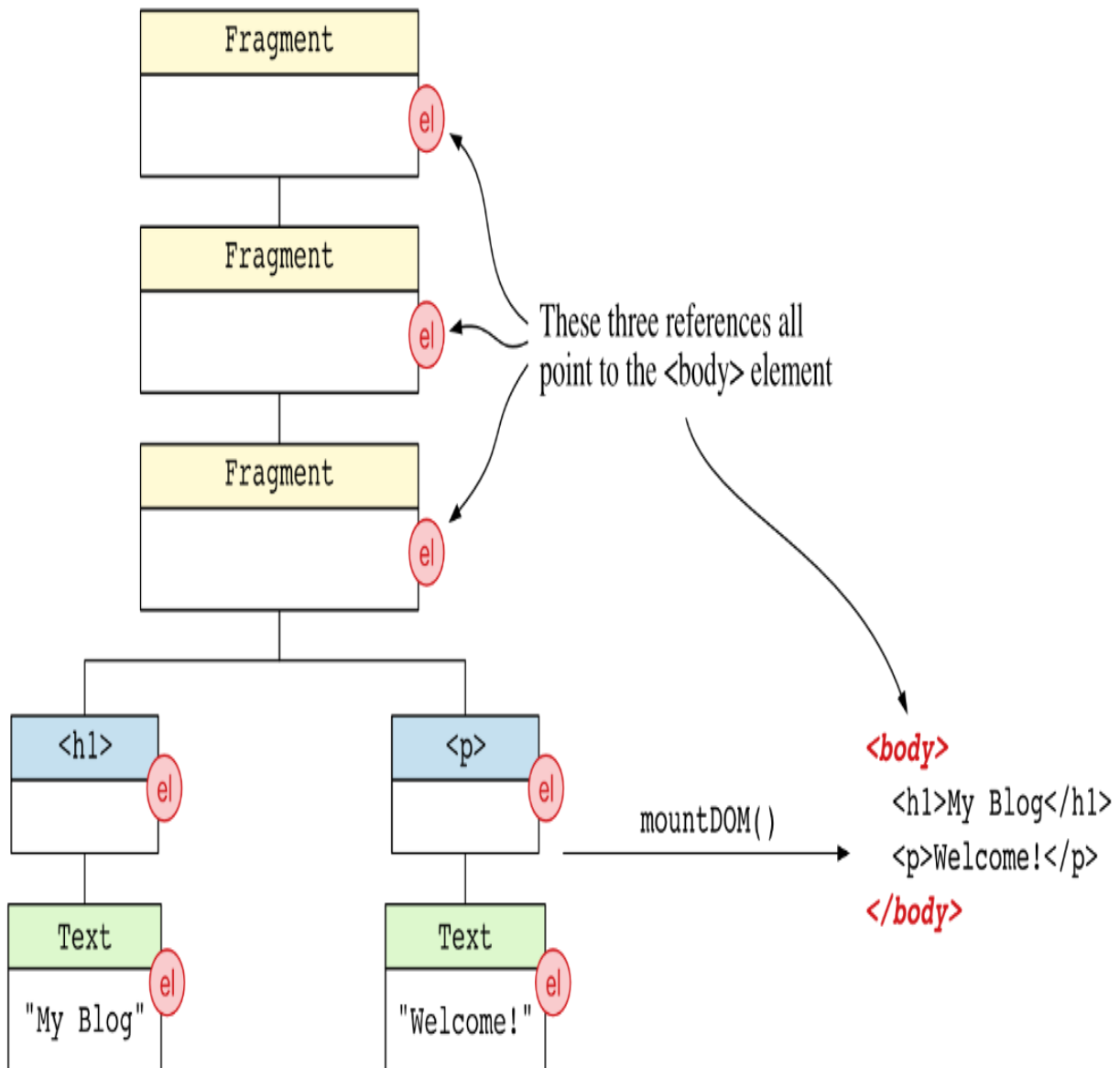
Fragment nodes are very simple to mount: you just need to mount the children of the fragment. What's important to remember about fragments is that they aren't a node that gets attached to the DOM; they're just an array of children. It's for this reason that the `el` property of a fragment virtual node should point to the parent element where the fragment's children are attached to, as depicted in figure [4.5](#).

Figure 4.5 Fragment's `el` should reference the parent element where its children are attached to



Note that, if you have nested fragment nodes, all the fragment nodes' children will be appended to the same parent element. This means that all the `el` references of those fragment virtual nodes should point to the same parent element, as you can see depicted in figure [4.6](#).

Figure 4.6 Nested fragments will all point to the same parent element



Now that you have a good understanding of how fragments work, it's time to write some code. Write the `createFragmentNode()` function inside the *mount-dom.js* file as follows:

```
function createFragmentNodes(vdom, parentEl) {
  const { children } = vdom
  vdom.el = parentEl #1

  children.forEach((child) => mountDOM(child, parentEl)) #2
}
```

Great—Now you’re ready to implement the `createElementNode()` function. It’s the most important one, because it’s the one that creates the element nodes; the visual bits of the DOM tree.

4.1.4 Mounting element nodes

To create element nodes (those regular HTML elements with tags like `<div>` and `<span>`), you use the [createElement\(\) method](#) from the Document API. You have to pass the tag name to the `createElement()` function, and the Document API will return an element node that matches that tag, or [HTMLUnknownElement](#) if the tag is unrecognized.

You can try this yourself in the browser console:

```
Object.getPrototypeOf(document.createElement('foobar'))
//HTMLUnknownElement
```

An obviously inexistent tag like `foobar` will return an `HTMLUnknownElement` node. So, if a virtual node has a tag that isn’t recognized by the Document API, the `document.createElement()` function returns an `HTMLUnknownElement` node. We’re not going to worry about this case: if an `HTMLUnknownElement` results from the `createElement()` function: we’ll assume it’s an error on the developer’s part.

If you recall, calling the `h()` function to create element virtual nodes returns an object with the `type` property set to `DOM_TYPES.ELEMENT`, a `tag` property with the tag name, and a `props` property with the attributes and event listeners. If the virtual node has children, they appear inside the `children` property.

For example, a `<button>` virtual node with a class of `btn` and an `onclick` event listener would look like this:

```
{
  type: DOM_TYPES.ELEMENT,
  tag: 'button',
  props: {
    class: 'btn',
    on: { click: () => console.log('yay!') }
  },
}
```

```

children: [
  {
    type: DOM_TYPES.TEXT,
    value: 'Click me!'
  }
]
}

```

To create the corresponding DOM element from the virtual node, you need to:

1. Create the element node using the `document.createElement()` function.
2. Add the attributes and event listeners to the element node, saving the added event listeners in a new property of the virtual node, called `listeners`.
3. Save a reference to the element node in the virtual node, under the `el` property.
4. Mount the children, recursively, into the element node.
5. Append the element node to the parent element.

As you can see, the `props` property of the virtual node is the one that contains the attributes and event listeners, both together. But attributes and event listeners are handled differently, so you'll need to separate them.

Then, from the attributes, there are two special cases that need special handling as well, and are those related to styling: `style` and `class`. You'll extract those from the `props` object as well, and handle them separately.

Go ahead and write the `createElementNode()` function inside the *mount-dom.js* file (listing [4.2](#)).

Listing 4.2 Mounting an element node into a parent element (mount-dom.js)

```

import { setAttributes } from './attributes'
import { addEventListeners } from './events'

// --snip-- //

function createElementNode(vdom, parentEl) {
  const { tag, props, children } = vdom

  const element = document.createElement(tag) #1

```

```

    addProps(element, props, vdom) #2
    vdom.el = element

    children.forEach((child) => mountDOM(child, element))
    parentEl.append(element)
  }

function addProps(el, props, vdom) {
  const { on: events, ...attrs } = props #3

  vdom.listeners = addEventListeners(events, el) #4
  setAttributes(el, attrs) #5
}

```

Note that you're using two functions you haven't implemented yet: `setAttributes()` and `addEventListeners()`, imported from the *attributes.js* and *events.js* files, respectively. You will write them in a minute.

Setting the attributes and adding event listeners is the part where the code differs from text nodes. With text nodes, you want to attach event listeners and set attributes to its parent element node, not to the text node itself.

Note

The `Text` type defined in the Document API inherits from the [EventTarget interface](#), which declares the `addEventListener()` method. So, in principle, you can add event listeners to a text node. But if you go ahead and try to do that in the browser, you'll see that the event listeners are never called—it just doesn't work.

Let's now implement the `addEventListeners()` function, in charge of adding event listeners to an element node. After it, we shall look at the `setAttributes()` function, and we'll be done with creating element nodes.

4.1.5 Adding event listeners

To add an event listener to an element node, you call its [`addEventListener\(\)` method](#). This method is available because an element node is an instance of the [EventTarget interface](#). This interface—which declares the `addEventListener()` method—is implemented by all the DOM nodes that can receive events. All instances returned by calling

`document.createElement()` implement the `EventTarget` interface, so you can safely call the `addEventListener()` method on them.

Our implementation of the `addEventListener()` function in this chapter of the book is going to be very simple: it'll just call the `addEventListener()` method on the element and return the event handler function it registered. You want to return the function registered as event handler, because later on, when you implement the `destroyDOM()` method—which as you can figure out it does the opposite of `mountDOM()` — you'll need to remove the event listeners to avoid memory leaks. You need the handler function that was registered in the event listener to be able to remove it, by passing it as an argument to the `removeEventListener()` method.

Later, in chapter [2](#), when you make the components of your framework stateful (recall that they'll be pure functions with no state for the moment), you'll also have references to the event listeners added to the components, which will behave in a different way than the event handlers of the DOM nodes. At this point in the book, you'll need to come back and modify the implementation of the `addEventListener()` function to account for this new case. So if you wonder why we're implementing a function that does so little of its own, this is the reason; it's just a placeholder for the more complex implementation that you'll write later.

Create a new file under the `src/` directory called `events.js` and add the following code:

```
export function addEventListener(eventName, handler, el) {
  el.addEventListener(eventName, handler)
  return handler
}
```

As promised, the `addEventListener()` function is very simple. But, if you recall, the event listeners defined in a virtual node come packed in an object, where the keys are the event names and the values are the event handler functions, like so:

```
{
  type: DOM_TYPES.ELEMENT,
  tag: 'button',
  props: {
```

```

    on: {
      mouseover: () => console.log('almost yay!'),
      click: () => console.log('yay!') ,
      dblclick: () => console.log('double yay!'),
    }
  }
}

```

So it makes sense to have another function—if only for convenience—that allows you to add multiple event listeners in the form of an object to an element node. So, inside the *events.js* file, add another function called `addEventListener()` (in plural):

```

export function addEventListeners(listeners = {}, el) {
  const addedListeners = {}

  Object.entries(listeners).forEach(([eventName, handler]) => {
    const listener = addEventListener(eventName, handler, el)
    addedListeners[eventName] = listener
  })

  return addedListeners
}

```

You might feel tempted to simplify that function by removing the `addedListeners` variable, and simply return the same `listeners` object the function got as input:

```

export function addEventListeners(listeners = {}, el) {
  Object.entries(listeners).forEach( ... )

  return listeners
}

```

After all, the same event handler functions that we got as input we're returning as output, so the refactor seems legit. But, that is now; it won't be the case later when you make the components have their own state. You won't be adding the same functions as event handlers, but new functions that will be created by the framework with some extra logic around them. This might sound confusing now, but stick with me, and you'll see how this makes sense later.

Now that you've implemented the event listeners, let's move on and implement the `setAttributes()` function. We're getting closer to having a working `mountDOM()` function.

4.1.6 Setting the attributes

To set an attribute in an `HTMLElement` instance in code, you set the value in the corresponding property of the element. Setting the property of the element will reflect the value in the corresponding attribute of the rendered HTML. It's important to understand that, when you're manipulating the DOM through code, you're working with DOM nodes, instances of the `HTMLElement` class. These instances have properties that you can set in code, as any other JavaScript object do. When these properties are set, the corresponding attribute is automatically reflected in the rendered HTML.

For instance, if you have a paragraph in HTML, such as the following:

```
<p id="foo">Hello, world!</p>
```

Assuming you have a reference to the `<p>` element in a variable called `p`, you can set the `id` property of the `p` element to a different value, like so:

```
p.id = 'bar'
```

And the rendered HTML reflects the change:

```
<p id="bar">Hello, world!</p>
```

In a nutshell: `HTMLElement` instances (such the `<p>` element, which is an instance of the `HTMLParagraphElement` class) have properties that correspond to the attributes that are rendered in the HTML markup. When you set the value of these properties, the corresponding attributes in the rendered HTML are automatically updated. Even though things are a bit more nuanced than that, this is the gist of it.

Caution

There are some attributes that work a bit differently. For instance, the `value` attribute of an `<input>` element is not reflected in the rendered HTML. If you

you have the following HTML:

```
<input type="text" />
```

And you programmatically set its value like so:

```
input.value = 'yolo'
```

You'll see the string "yolo" in the input, but the rendered HTML will still be the same; no `value` attribute will be rendered. But even more interesting is the fact that, you can add the attribute in the HTML markup:

```
<input type="text" value="yolo" />
```

And the "yolo" string will appear in the input when it first renders, but if you type in something different, the same value for the `value` attribute will remain in the rendered HTML. But you can read whatever was typed in the input by reading the `value` property of the input element.

Read more about this behavior in the HTML specification at <http://mng.bz/yQ17>.

There are, nevertheless, two special attributes that we'll handle differently: the `style` attribute and the `class` attribute. You'll see why in a moment.

Let's start by writing the `setAttributes()` function: the one you used in listing [4.2](#) to set the attributes on the element node. The role of this function is to extract the attributes that require special handling (the `style` and `class` attributes) from the rest of the attributes, and then call the `setStyle()` and `setClass()` functions to set those attributes. The rest of the attributes are passed to the `setAttribute()` function. You'll write these `setStyle()`, `setClass()`, and `setAttribute()` functions later.

Start by creating the *attributes.js* file under the *src/* directory and write the code in listing [4.3](#).

Listing 4.3 Setting the attributes of an element node (attributes.js)

```
export function setAttributes(el, attrs) {  
  const { class: className, style, ...otherAttrs } = attrs #1
```

```

    if (className) {
      setClass(el, className) #2
    }

    if (style) {
      Object.entries(style).forEach(([prop, value]) => {
        setStyle(el, prop, value) #3
      })
    }

    for (const [name, value] of Object.entries(otherAttrs)) {
      setAttribute(el, name, value) #4
    }
  }

  // TODO: implement setClass

  // TODO: implement setStyle

  // TODO: implement setAttribute

```

Now that you have the function that splits the attributes into the ones that require special handling and the rest, and calls the appropriate functions to set them, let's go ahead and look at each of those in turn. In order of appearance, the first one is the `setClass()` function, which is in charge of setting the `class` attribute. Note that you've destructured the `attrs` property and aliased the `class` attribute to the `className` variable, as `class` is a reserved word in JavaScript.

Setting the "class" attribute

When you write HTML, you set the `class` attribute of an element node like this:

```
<div class="foo bar baz"></div>
```

In this case, the `<div>` element has three classes: `foo`, `bar`, and `baz`. Easy! But now comes the tricky part: a DOM element (an instance of the [Element](#) class) doesn't have a `class` property, but instead has two properties, namely [className](#) and [classList](#), that are related to the `class` attribute. Let's look at the `classList` first.

The [classList](#) [property](#) returns an object, a [DOMTokenList](#) to be more specific, that comes in pretty handy when you want to add, remove, or toggle classes on an element. A `DOMTokenList` object has an [add\(\)](#) [method](#), which takes multiple class names, and adds them to the element. For example, if you had a `<div>` element like the following:

```
<div></div>
```

and wanted to add the `foo`, `bar`, and `baz` classes to it, you could do it like this:

```
div.classList.add('foo', 'bar', 'baz')
```

This would result in the following HTML:

```
<div class="foo bar baz"></div>
```

Then is the [className](#) [property](#), which is a `string` that contains the value of the `class` attribute. Following the example above, if you wanted to add the same three classes to the `<div>` element, you could do it like this:

```
div.className = 'foo bar baz'
```

And this would yield the same HTML as the previous example.

You actually want to use both ways of setting the `class` attribute, depending on the situation. We should allow the developers using your framework to set the `class` attribute in two ways: either as a `string` or as an array of `string` items. So, to add multiple classes to an element, the developer could define the following virtual node:

```
{
  type: DOM_TYPES.ELEMENT,
  tag: 'div',
  props: {
    class: ['foo', 'bar', 'baz']
  }
}
```

Or, alternatively, they can use a single `string` like this:

```
{
  type: DOM_TYPES.ELEMENT,
  tag: 'div',
  props: {
    class: 'foo bar baz'
  }
}
```

Both of these options should work. Thus, the `setClass()` function needs to distinguish between the two cases and handle them accordingly. With this in mind, write the following code in the *attributes.js* file:

```
function setClass(el, className) {
  el.className = '' #1

  if (typeof className === 'string') {
    el.className = className #2
  }

  if (Array.isArray(className)) {
    el.classList.add(...className) #3
  }
}
```

With the `setClass()` function out of the way, let's move on to the `setStyle()` function, which is in charge of setting the `style` attribute of an element.

Setting the "style" attribute

The [style property](#) of an `HTMLElement` instance is a [CSSStyleDeclaration object](#). What you need to know about this object is that you can set the value of a CSS property using conventional object notation, like this:

```
element.style.color = 'red'
element.style.fontFamily = 'Georgia'
```

Changing the `style` property key-value pairs of an `HTMLElement` instance is reflected in the value of the `style` attribute of the element. If the element in the previous snippet was a paragraph (`<p>`), the resulting HTML would be:

```
<p style="color: red; font-family: Georgia;"></p>
```

The `CSSStyleDeclaration` is converted into a string with the set of semi-colon-separated key-value pairs. You can inspect this string representation of the `style` attribute by using the [cssText property](#) in code:

```
element.style.cssText // 'color: red; font-family: Georgia;'
```

For example, using the `element` from the previous snippet, you could remove the `color` style by doing this:

```
element.style.color = null
element.style.cssText // 'font-family: Georgia;'
```

Now that you know how to work with the `style` property of an `HTMLElement` instance, go ahead and write the `setStyle()` and `removeStyle()` functions. The first function takes an `HTMLElement` instance, the name of the style to set, and the value of the style, and sets that style on the element. The second function takes an `HTMLElement` instance and the name of the style to remove, and removes that style from the element.

Inside the *attributes.js* file, write the following code:

```
export function setStyle(el, name, value) {
  el.style[name] = value
}

export function removeStyle(el, name) {
  el.style[name] = null
}
```

Note that you haven't used the `removeStyle()` function yet. The code you wrote before only used the `setStyle()` function, but as you'll need to remove styles later on, this was a good time to write it.

You're almost done; you're only missing the `setAttributes()` function, which is in charge of setting the rest of the attributes other than the `class` and `style`.

Setting the rest of the attributes

The `setAttribute()` function takes three arguments: an `HTMLElement` instance, the name of the attribute to set, and the value of the attribute. If the value of the attribute is `null`, the attribute is removed from the element. (Conventionally, setting a DOM element's property to `null` is the same as removing the attribute.) If the attribute is of the form `data-*`, the attribute is set using the `setAttribute()` function. Otherwise, the attribute is set to the given value using object notation (`object.key = value`). (Or we shall say the property of the DOM element is set to the given value, and the attribute in the HTML will reflect that value.) To remove an attribute, you want to both set it to `null` and remove it from the `attributes` object, using the `removeAttribute()` method.

Inside the *attributes.js* file, write the following code:

```
export function setAttribute(el, name, value) {
  if (value == null) {
    removeAttribute(el, name)
  } else if (name.startsWith('data-')) {
    el.setAttribute(name, value)
  } else {
    el[name] = value
  }
}

export function removeAttribute(el, name) {
  el[name] = null
  el.removeAttribute(name)
}
```

And with this last function, you're done implementing the `mountDOM()` function, that takes a virtual DOM and mounts it to the real DOM inside the passed in parent element. Congratulations!

4.1.7 A `mountDOM()` example

Thanks to this function, you can now define a view using the virtual DOM representation of it, and mount it to the real DOM. For example, you can create a view like this:

```
const vdom = h('section', {} [
  h('h1', {}, ['My Blog']),
  h('p', {}, ['Welcome to my blog!'])
])
```

```
])
```

```
mountDOM(vdom, document.body)
```

And the resulting HTML would be:

```
<body>
  <section>
    <h1>My Blog</h1>
    <p>Welcome to my blog!</p>
  </section>
</body>
```

You now want to have a function that, given a mounted virtual DOM, destroys it and removes it from the document: `destroyDOM()`. With this function you'll be able to clear the document's body from the previous example, like so:

```
destroyDOM(vdom, document.body)
```

The work done by `mountDOM()` is undone by `destroyDOM()`. This would result in the document's body being empty again:

```
<body></body>
```

Let's close this chapter implementing the `destroyDOM()` function.

Exercise 4.2

Using the `hFragment()`, `h()`, and `mountDOM()` functions, insert a new section below the headline of your local's newspaper website. The section should have a title, a paragraph, and a link to an article in Wikipedia. Be creative!

Find the solution at <http://mng.bz/5oj8>.

Exercise 4.3

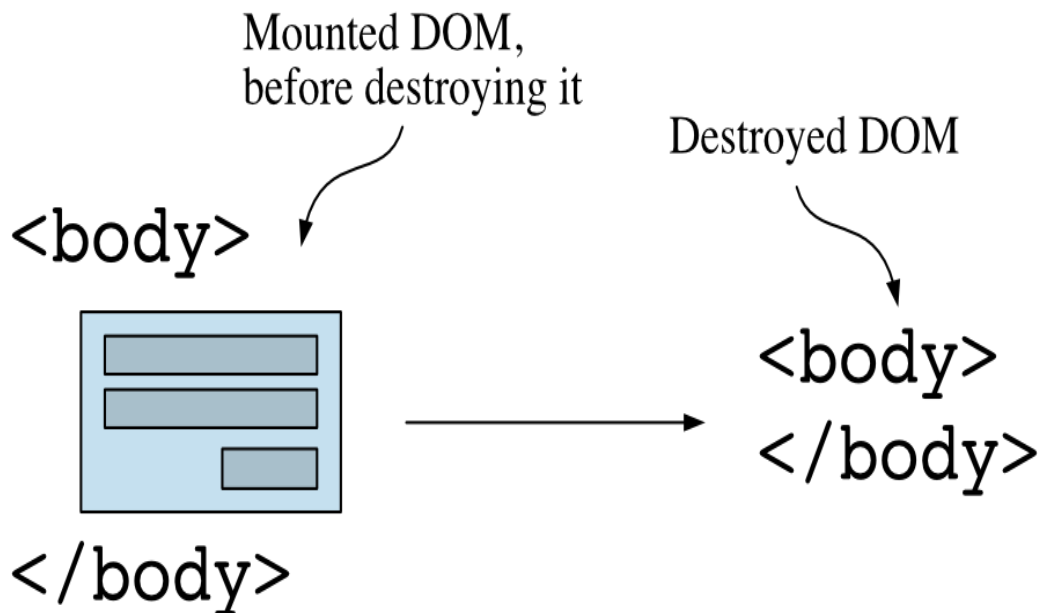
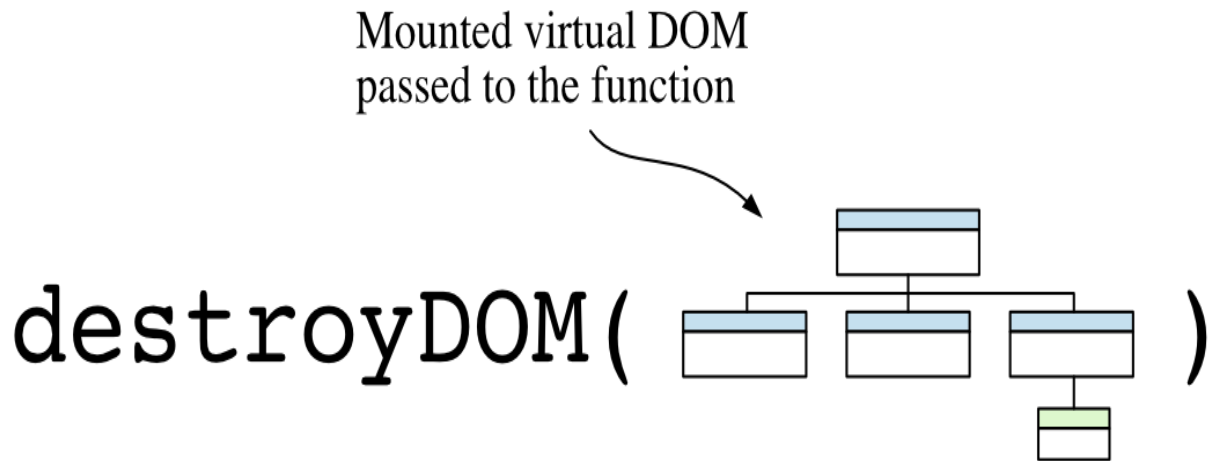
Following up on the previous exercise, inspect the virtual DOM tree in the browser's console. Check the `el` property of the fragment virtual node. What does it point to? What about the `el` property of the paragraph and link virtual nodes?

Find the solution at <http://mng.bz/6ngp>.

4.2 Destroying the DOM

Destroying the DOM is simpler than mounting it. Well, destroying anything is always simpler than creating it in the first place. Destroying the DOM is the process by which the HTML elements that were created by the `mountDOM()` function are removed from the document. This process is depicted in [4.7](#).

Figure 4.7 Destroying the DOM



To destroy the DOM associated to a virtual node, you have to take into account what type of node it is:

- *text node* — remove the text node from its parent element using the `remove()` method.
- *fragment node* — remove each of its children from the parent element, which if you recall, is referenced in the `el` property of the fragment virtual node.
- *element node* — do the two previous things, plus remove the event listeners from the element.

In all of the cases, you want to remove the `el` property from the virtual node, and in the case of an element node, also remove the `listeners` property. This is so that you can tell that the virtual node has been destroyed and allow the garbage collector to free the memory of the HTML element. When a virtual node doesn't have an `el` property, you can safely assume that it's not mounted to the real DOM, and therefore can't be destroyed.

To handle these three cases, you need a `switch` statement that, depending on the `type` property of the virtual node calls a different function.

You are ready to implement the `destroyDOM()` function. Create a new file inside the `src` folder called *destroy-dom.js* and write the code in Listing [4.4](#).

Listing 4.4 Destroying the virtual DOM (destroy-dom.js)

```
import { removeEventListeners } from './events'
import { DOM_TYPES } from './h'

export function destroyDOM(vdom) {
  const { type } = vdom

  switch (type) {
    case DOM_TYPES.TEXT: {
      removeTextNode(vdom)
      break
    }

    case DOM_TYPES.ELEMENT: {
      removeElementNode(vdom)
      break
    }

    case DOM_TYPES.FRAGMENT: {
      removeFragmentNodes(vdom)
      break
    }

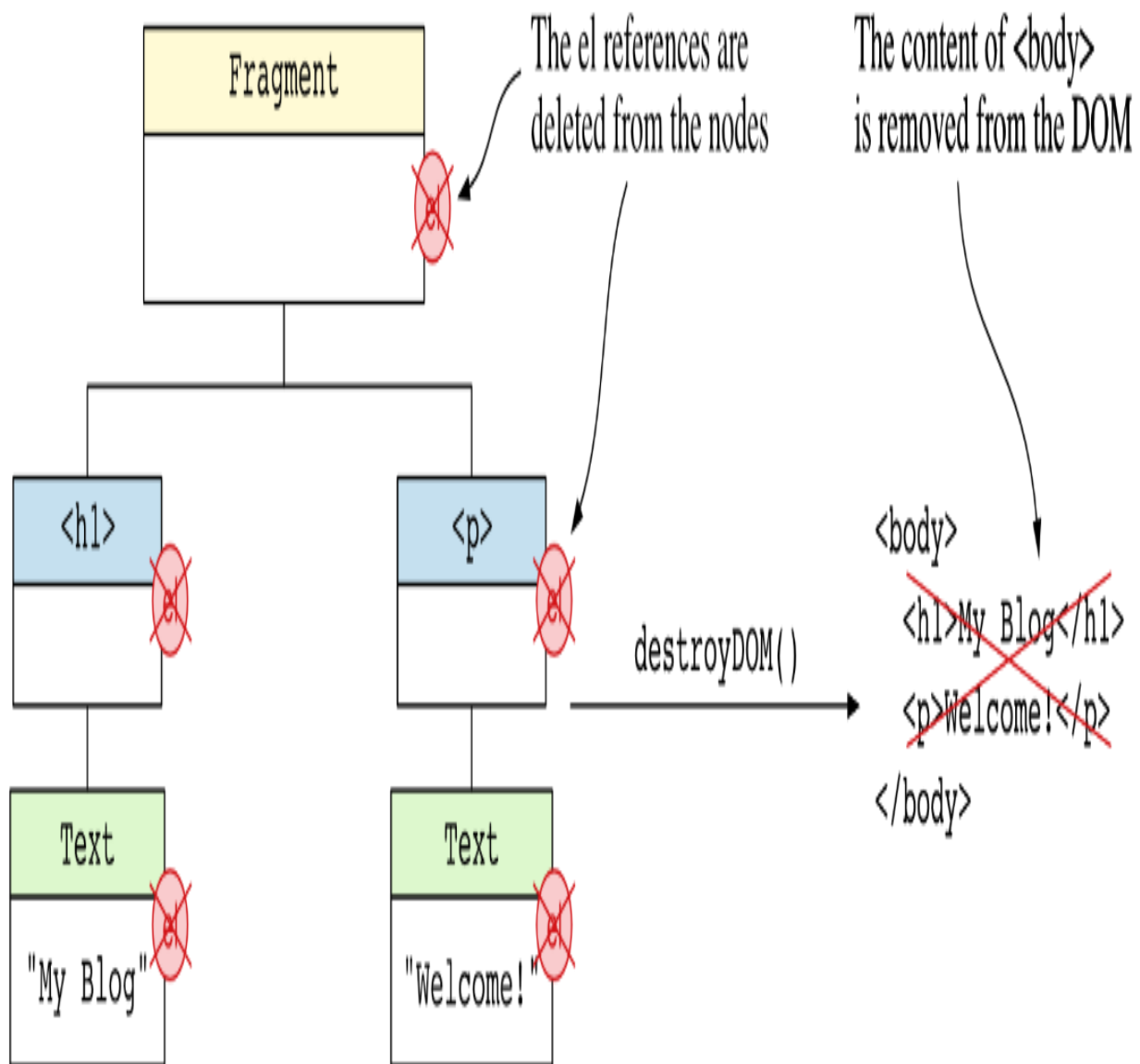
    default: {
      throw new Error(`Can't destroy DOM of type: ${type}`)
    }
  }

  delete vdom.el
}
```

```
// TODO: implement removeTextNode()  
  
// TODO: implement removeElementNode()  
  
// TODO: implement removeFragmentNodes()
```

You've written the algorithm for destroying the DOM associated with a passed in virtual node: `vdom`. You've handled each type of virtual node separately—you'll need to write the missing functions in a minute—and you've lastly deleted the `el` property from the virtual node. This process is depicted in [4.8](#).

Figure 4.8 Removing the `el` references from the virtual nodes



Note that you've imported the `removeEventListeners()` function from the *events.js* file, but you haven't implemented that one yet. You will in a minute.

Let's start with the code for destroying a text node.

4.2.1 Destroying a text node

Destroying a text node is the simplest case:

```
function removeTextNode(vdom) {  
  const { el } = vdom
```

```
    el.remove()
  }
```

Let's move on to the code for destroying an element, which is a bit more interesting.

4.2.2 Destroying an element

To destroy an element you start by removing it from the DOM, in a similar fashion to how you just did with a text node. Then, you want to recursively destroy the children of the element, by calling the `destroyDOM()` function for each of them. Finally, you want to remove the event listeners from the element and delete the `listeners` property from the virtual node.

Go ahead and implement the `removeElementNode()` function in the *destroy-dom.js* file:

```
function removeElementNode(vdom) {
  const { el, children, listeners } = vdom

  el.remove()
  children.forEach(destroyDOM)

  if (listeners) {
    removeEventListeners(listeners, el)
    delete vdom.listeners
  }
}
```

Here's where you've used the missing `removeEventListeners()` function to remove the event listeners from the element. Then, you've also deleted the `listeners` property from the virtual node.

Let's write the `removeEventListeners()` function now. It's a function that given an object of event names and event handlers, it removes the event listeners from the element. Recall that the `listeners` property of the virtual node is an object that maps event names to event handlers. the following could be an example of the `listeners` object for a virtual node representing a button:

```
{
  mouseover: () => { ... },
  click: () => { ... },
  dblclick: () => { ... }
}
```

There would be three event handlers to remove in the case above. So, for each of them, you want to call the `Element` object's [`removeEventListener\(\)` method](#) (that it inherits from the `EventTarget` interface):

```
el.removeEventListener('mouseover', listeners['mouseover'])
el.removeEventListener('click', listeners['click'])
el.removeEventListener('dblclick', listeners['dblclick'])
```

Open the *events.js* file and fill in the missing code:

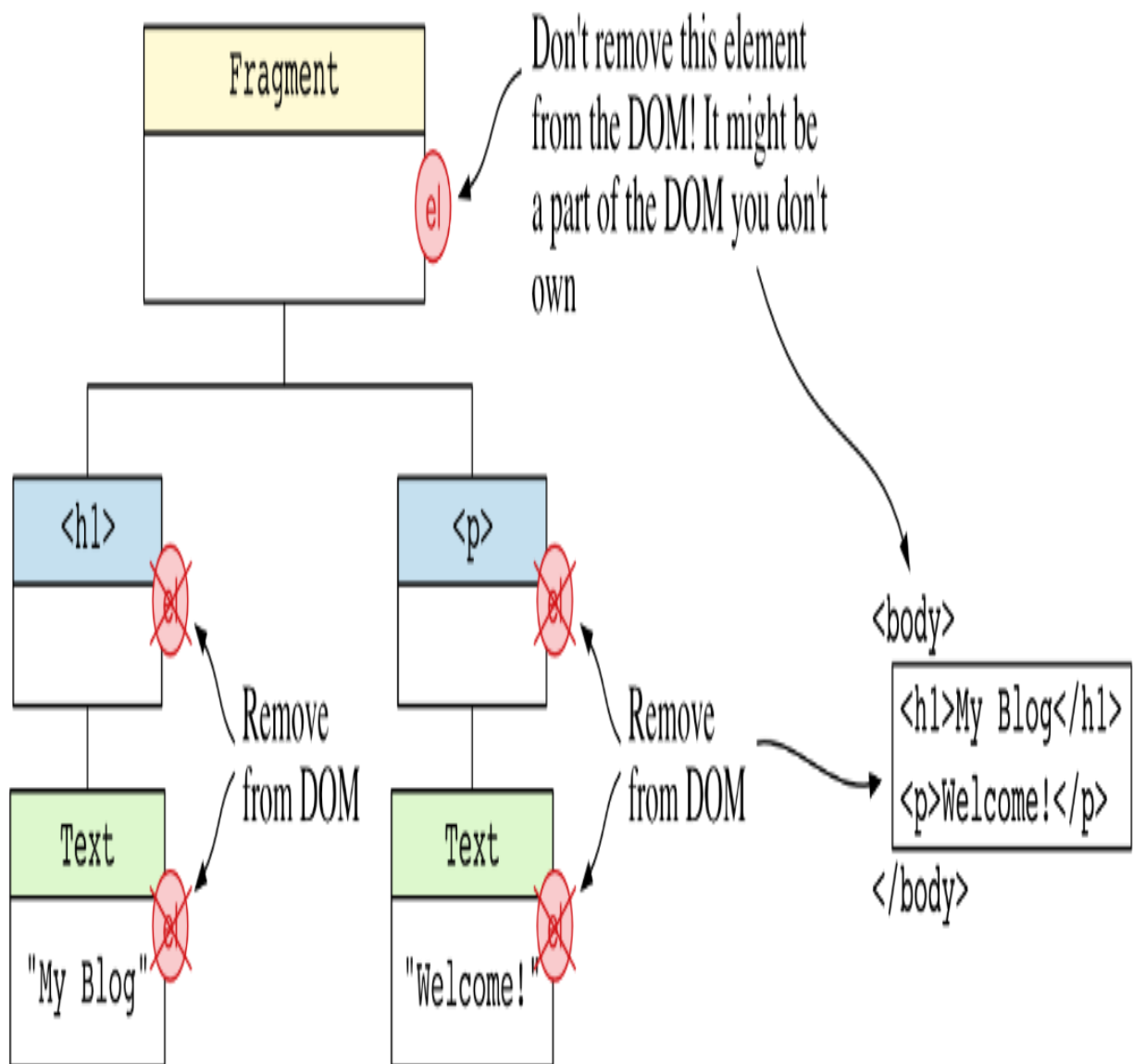
```
export function removeEventListeners(listeners = {}, el) {
  Object.entries(listeners).forEach(([eventName, handler]) => {
    el.removeEventListener(eventName, handler)
  })
}
```

Great! You're only missing the code for destroying a fragment.

4.2.3 Destroying a fragment

Destroying a fragment is simple: you simply need to call the `destroyDOM()` function for each of its children. But you have to be careful not to remove the `el` referenced in the fragment's virtual node from the DOM, because that references the element where the fragment children are mounted, not the fragment itself. If the fragment children were mounted inside the `<body>`, and you called the `remove()` method on the element, you would remove the whole document from the DOM. That's not what you want—or do you? Figure [4.9](#) shows the problem in a more graphical way.

Figure 4.9 When destroying a fragment, don't remove it's referenced element from the DOM; it might be the `<body>` or some other element that you didn't create, and therefore don't own



The implementation of the `removeFragmentNodes()` is therefore very simple:

```
function removeFragmentNodes(vdom) {
  const { children } = vdom
  children.forEach(destroyDOM)
}
```

That's it! You've implemented the `mountDOM()` and `destroyDOM()` functions. These functions, together with the state management system that you'll

implement in the next chapter, will be the core of the first version of your framework. You will use the framework to refactor the TODOs application.

Exercise 4.4

Using the `destroyDOM()` function, remove the section that you added to your local newspaper website in the previous exercise from the DOM. Make sure that the fragment's referenced element is not removed from the DOM, as it was originally created by the newspaper website, not by you.

Then, check the vdom tree you used to create the section, and make sure that the `el` property has been removed from all the virtual nodes.

Find the solution at <http://mng.bz/orpy>.

The first version won't be very sophisticated—it'll destroy the entire DOM using `destroyDOM()` and mount it from scratch using `mountDOM()` every time the state changes—but it will be a great starting point. After finishing the next chapter you'll have a working framework, so I hope you're excited about it!

See you in the next chapter!

4.3 Summary

- Mounting a virtual DOM means to create the DOM nodes that represent each virtual node in the tree, and to insert them into the DOM, inside a parent element.
- Destroying a virtual DOM means to remove the DOM nodes created out of it, also making sure to remove the event listeners from the DOM nodes.

5 State management and the application's lifecycle

This chapter covers

- What state management is
- Implementing a state management solution
- Mapping JavaScript events to commands that change the state
- Updating the state using reducer functions
- Re-rendering the view when the state changes

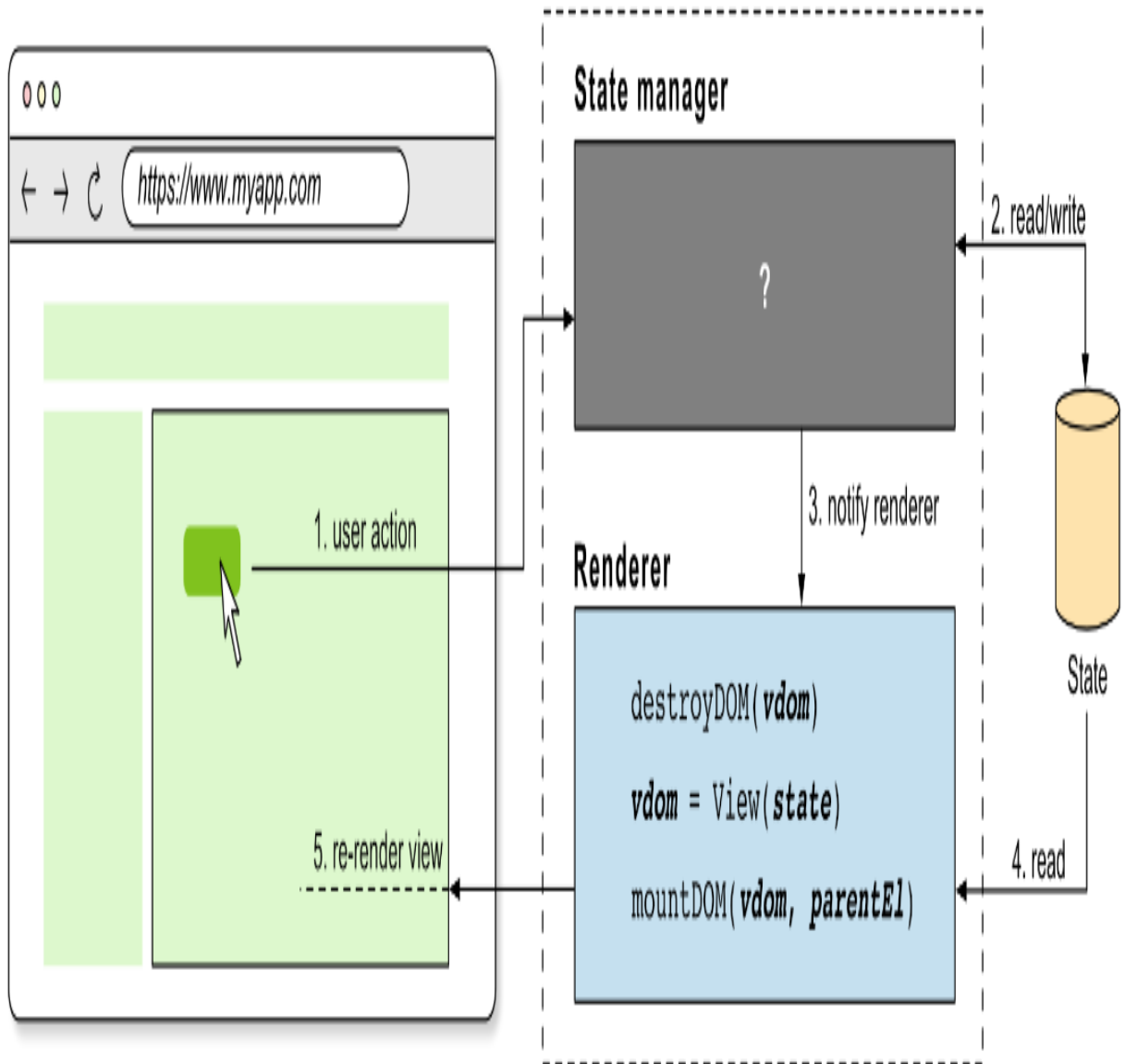
I remember some time ago I went to the coast in the south of Spain, to a small village in Cadiz. There was this restaurant, so popular that they'd run out of food quickly; they had a limited number of each of the dishes they served, and they would update a chalkboard with the number of servings they had left. When there was no more of a particular dish, they'd cross it out. The waiter took orders from the customers and updated the chalkboard to reflect the new state of the restaurant: having one less of that dish. Us, the customers, could easily know what we could order by looking at the chalkboard. But from time to time, and due to the work load, the waiter forgot to update the chalkboard, and customers would find out the dish they've been waiting so long in line to order was sold out. You can picture the drama. It's clearly important for the restaurant to have an updated chalkboard that matched the remaining servings of each dish.

To have a working framework, you're missing a key piece that does a similar job as to the waiter in the restaurant, a *state manager*. In the restaurant anecdote, the waiter is in charge of keeping the chalkboard in sync with the state of the restaurant: the number of servings of each dish they had left. In a frontend application, the state changes as the user interacts with it, and the view needs to be updated to reflect that changes. The state manager keeps the application's state in sync with the view, responding to user input by modifying the state accordingly, and notifying the *renderer* when the state has

changed. The renderer is the entity in your framework that takes the virtual DOM and mounts it into the DOM.

In this chapter, you'll implement both the renderer (using the `mountDOM()` and `destroyDOM()` functions from the previous chapter) and the state manager entities, and you'll learn how they communicate. By the end of the chapter, you'll have your first version of the framework, whose architecture (shown in figure [5.1](#)) is basically the state manager and renderer glued together. (The state manager is displayed with a question mark because we'll discover how it works in this chapter.)

Figure 5.1 Your first framework is just a state manager and a renderer glued together.



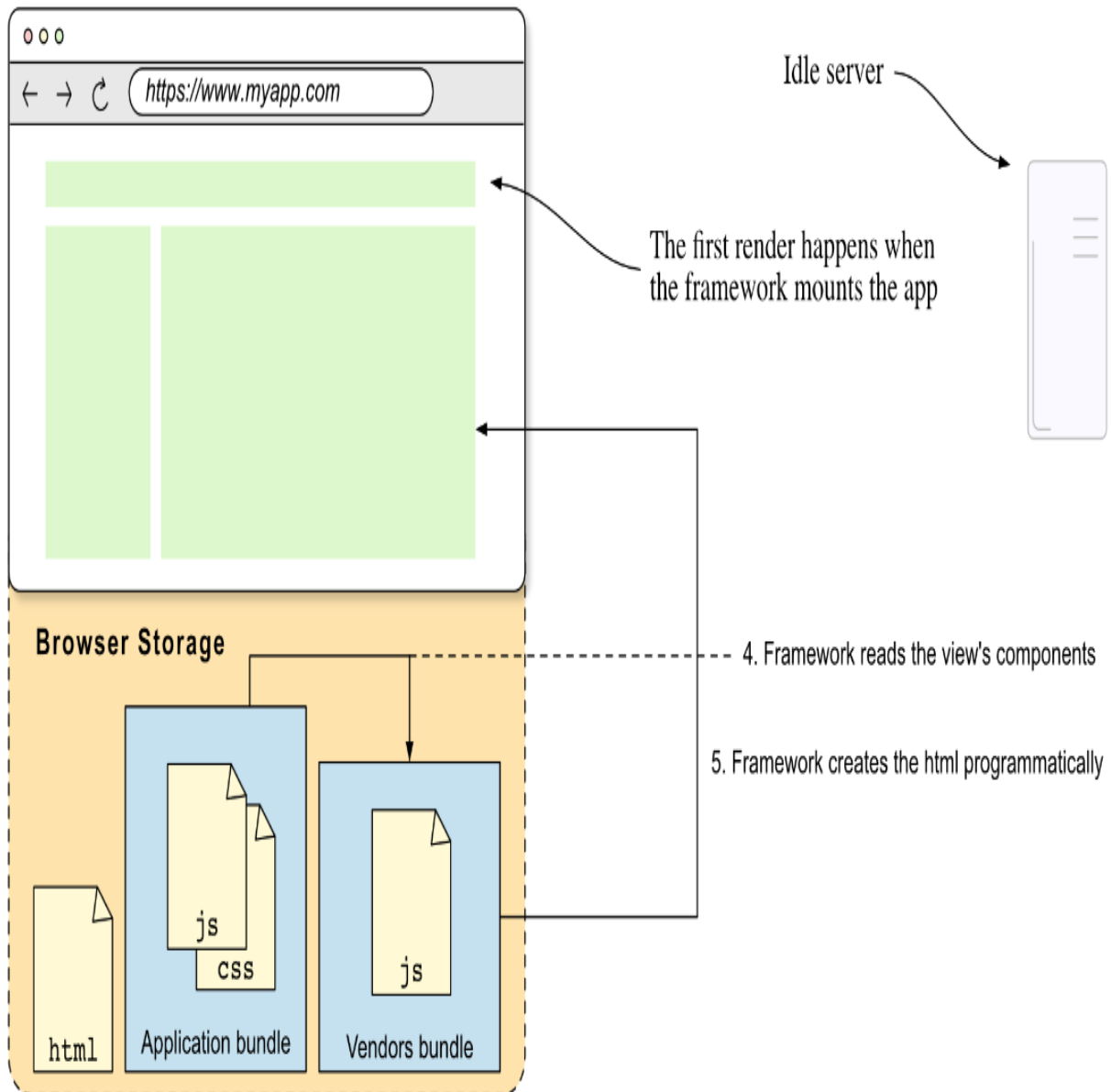
The framework you'll have at the end of the chapter is very rudimentary: to ensure the view reflects the current state, its renderer completely destroys and mounts the DOM every time the state changes. As you can see in the diagram in figure [5.1](#), the renderer draws the view as a three-step process:

1. Completely destroy the current DOM (calling `destroyDOM()`).
2. Produce the virtual DOM representing the view given the current state (calling the `View()` function, the top level component).
3. Mount the virtual DOM into the real DOM (calling `mountDOM()`).

Destroying and mounting the DOM from scratch is far from ideal, and we'll discuss why later on. In any case, it's a good starting point, and you'll improve the rendering mechanism in chapters [7](#) and [8](#), where thanks to the reconciliation algorithm, your framework can update the DOM in a more efficient way. You have to walk before you can run, they say.

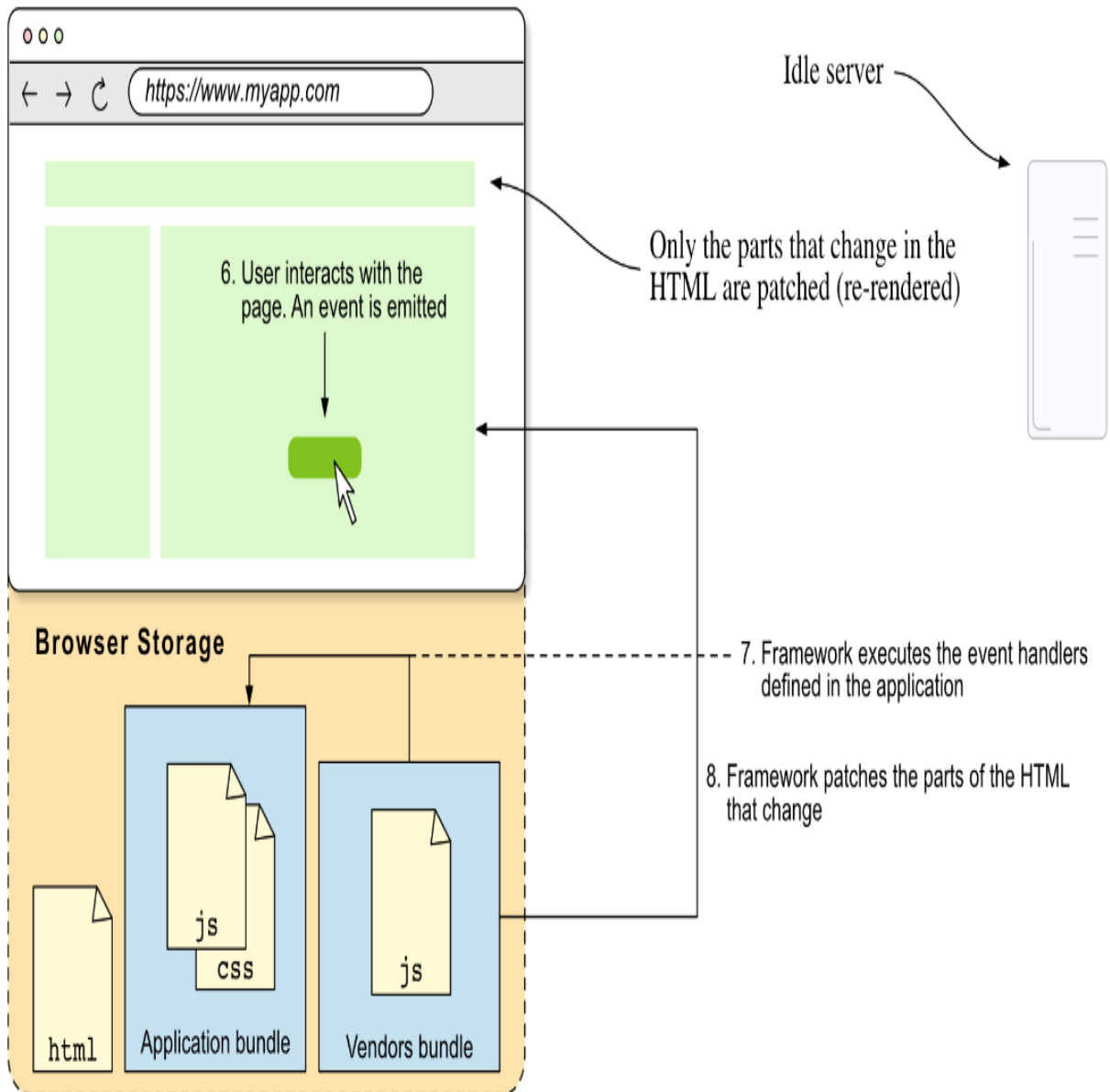
How does what you'll do in this chapter—implementing a renderer and state manager—fit into the bigger picture of your framework? Let's briefly take a step back. If you recall from our discussion in chapter [1](#), when the browser loads an application, the framework code renders the application's view (step 5 in figure [5.2](#), reproduced from figure [1.7](#) for convenience). This can be done by the renderer alone—the state manager doesn't intervene here. (Well—it might. But let's leave that case for now.)

Figure 5.2 Single-page application first render in the browser



Then, when the user interacts with the application (step 6 in figure [5.3](#), reproduced from figure [1.8](#)), the state manager updates the state (step 7) and notifies the renderer to re-render the view (step 8). This dynamic is a bit more complex and requires the state manager and renderer to work together.

Figure 5.3 Single-page application responding to events



The state manager somehow needs to be aware of the events that the user interactions can trigger, such as clicking a button or typing in an input field, and know what to do with the state when those events take place. In a way, it needs to know beforehand everything the user might do with the application, and how that affects the state.

In this chapter, you'll learn exactly how to do that. So, with the big picture in mind, let's start working on the state manager.

Important

You can find all the listings in this chapter in the [listings/ch05 directory](#) of the book's repository.

The code you write in this chapter is for the framework's first version, which you'll publish in chapter [6](#). Therefore, the code in this chapter can be checked out from [the *ch6* label](#):

```
$ git switch --detach ch6
```

Code catch-up

In the previous chapter you implemented the `mountDOM()` and `destroyDOM()` functions.

`mountDOM()` takes a virtual DOM tree and a parent DOM element, and mounts the virtual DOM into the parent element. Its implementation is broken down into a few sub-functions:

- `createTextNode()` — to create HTML text nodes.
- `createElementNode()` — to create HTML element nodes. This function uses two sub-functions: `addEventListener()` and `setAttributes()`.
- `createFragmentNodes()` — to create lists of nodes that have a common parent.

`destroyDOM()` takes a DOM element and removes it from the DOM. Its implementation is also broken down into a few sub-functions:

- `removeTextNode()` — to remove HTML text nodes.
- `removeElementNode()` — to remove HTML element nodes. This function uses another function to remove the element's event listeners: `removeEventListeners()`.
- `removeFragmentNodes()` — to remove lists of nodes that have a common parent.

5.1 The state manager

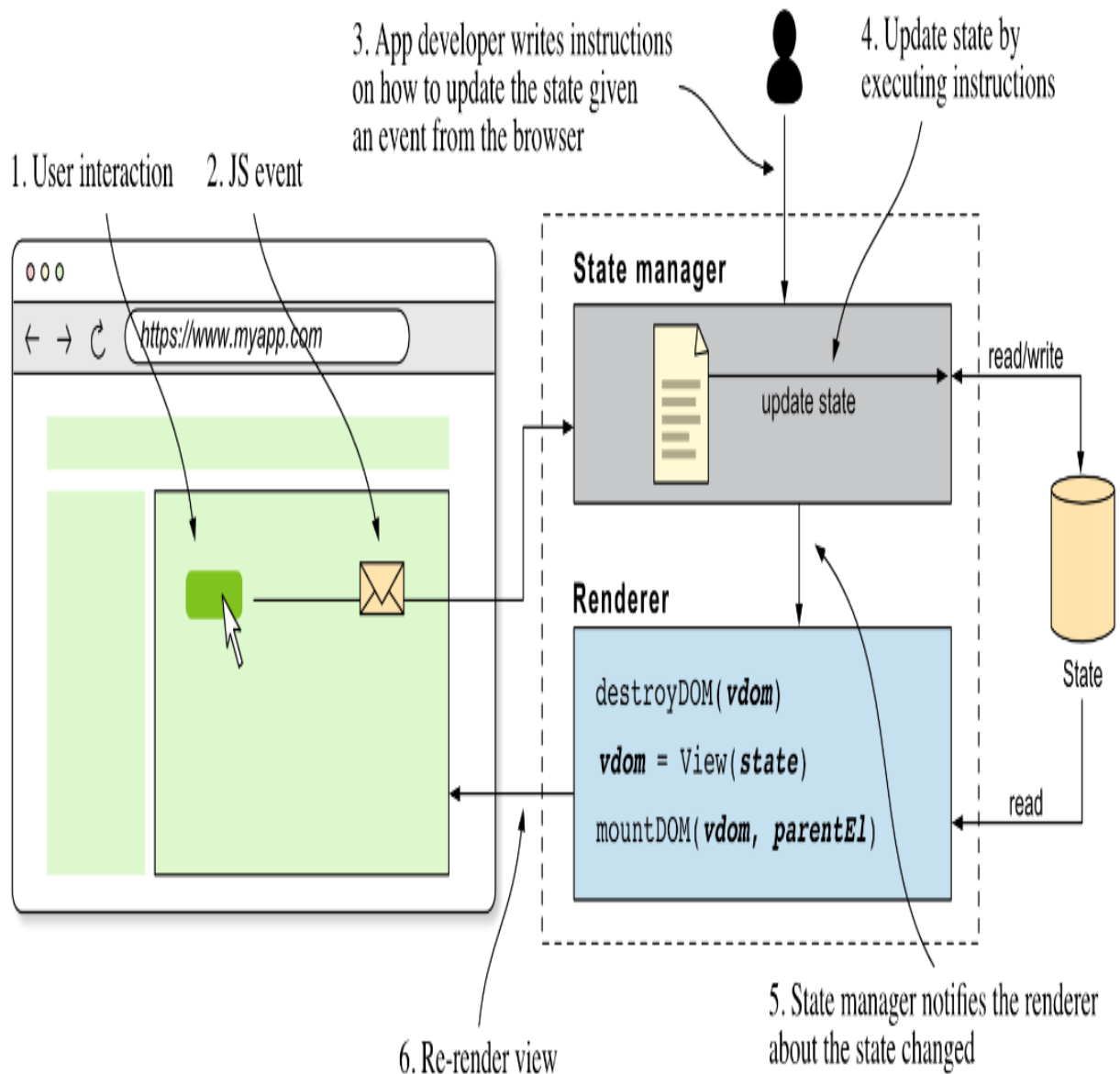
Let's study the chronology of everything that happens between the user interacting with the application and the view being updated. In the list that

follows, the actor goes first, then the action it performs.

1. *The user* — interacts with the application's view (for example, clicks a button).
2. *The browser* — dispatches a native JavaScript [event](#) (for example, [MouseEvent](#), [KeyboardEvent](#)).
3. *The application developer* — programmed the framework on how to update the state for each event.
4. *The framework's state manager* — updates the state according to the instructions given by the application developer.
5. *The framework's state manager* — notifies the renderer that the state has changed.
6. *The framework's renderer* — re-renders the view with the new state.

Well, this isn't exactly a chronology, because the application developer doesn't intervene between the user interacting with the application and the state manager updating the state, but rather gives those instructions on how to update the state during the application development (not at runtime). But still, it's a good way to understand the flow of events. Figure [5.4](#) illustrates this pseudo-chronology in the architectural diagram of the framework I presented in figure [5.1](#).

Figure 5.4 The pseudo-chronology of events in a frontend application



From this pseudo-chronology, there're two key questions that we should ask ourselves: *how does the application developer instruct the framework how to update the state when a particular event is dispatched?* And, *how does the state manager execute those instructions?* In the answer to these questions lays the key to understanding the state manager.

5.1.1 From JavaScript events to application domain commands

The first thing that you need to notice is that the JavaScript events dispatched by the browser don't have a concrete meaning in the application domain on their own. The user clicked this button, so what? It's the application developer who translates user actions into something meaningful for the application. Think about the TODOs application:

- When the user clicked the *Add* button or pressed the *Enter* key, that meant they wanted to add a new TODO item to the list.
- "*Adding a to-do*" is framed in the language of the application, whereas "*clicking a button*" is a very generic thing to do. (You click buttons in all sorts of applications, but that only translates to "*adding a to-do*" in the TODOs application.)

If the application developer wants to update the state when a particular event is dispatched, they first need to determine what that event means in terms of the application domain. They then map that event to a *command* that the framework can understand. A command is a request to do something, as opposed to an event, which is a notification about something that has happened. These commands are a request to the framework to update the state, and are expressed using the domain language of the application.

Events vs. commands

An *event* is a notification about something that has happened. "A button was clicked," "a key was pressed," "a network request was completed" are all examples of events. Events aren't requesting the framework or application to do anything, they're just notifications with some additional information. Event names are usually framed in the past tense: 'button-clicked', 'key-pressed', 'network-request-completed', and so on.

A *command* is a request to do something in a particular context. "Add todo," "edit todo," "remove todo" are three examples of commands. Commands are written in imperative tense, because they're requests to do something: 'add-todo', 'edit-todo', 'remove-todo', and more.

Following with the TODOs application example, here's a table of a few events that the user can trigger and the commands that the application developer would dispatch to the framework in response to those events.

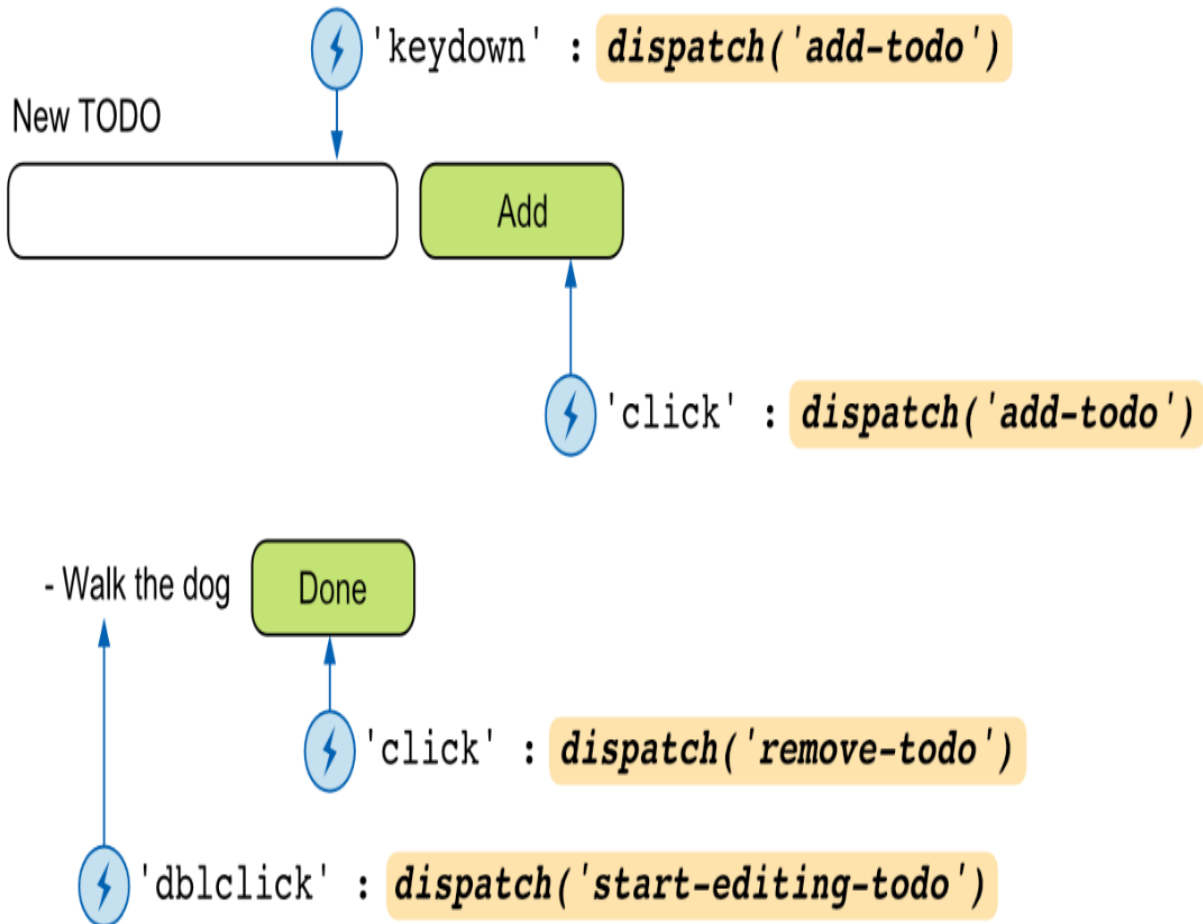
Table 5.1 Mapping between browser events and application commands in the TODOs application

Browser Event	Command	Explanation
Click the <i>Add</i> button	<code>add-todo</code>	Clicking the <i>Add</i> button adds a new to-do item to the list.
Press the <i>Enter</i> key (while the input field is focused)	<code>add-todo</code>	Pressing the <i>Enter</i> key adds a new to-do item to the list.
Click the <i>Done</i> button	<code>remove-todo</code>	Clicking the <i>Done</i> button marks the to-do item as done, removing it from the list.
Double-click a to-do item	<code>start-editing-todo</code>	Double-clicking a on a to-do item sets the to-do item in edit mode.

And in figure [5.5](#) you can see the same mapping in the form of a diagram. In it, you can establish a link between the browser events and the application commands, which are dispatched by the application developer to the framework, using the `dispatch()` function. You'll learn more about this function in a minute.

Figure 5.5 The mapping between browser events and application commands

My TODOs



Why is this relevant to answer the previous questions? you might be asking. Because once the application domain commands are identified, the application developer can supply functions that update the state as a response to those commands. The state manager executes those functions to update the state. Let's look into what these functions are and how they're executed.

Exercise 5.1

Imagine an application consisting in a counter and two buttons: one to increment the counter and another one to decrement it. It's HTML could look

like this:

```
<button>-</button>  
<span>0</span>  
<button>+</button>
```

When each of the buttons is clicked, what commands would you dispatch to the framework? Can you draw a diagram similar to the one in figure [5.5](#) that maps browser events to application commands?

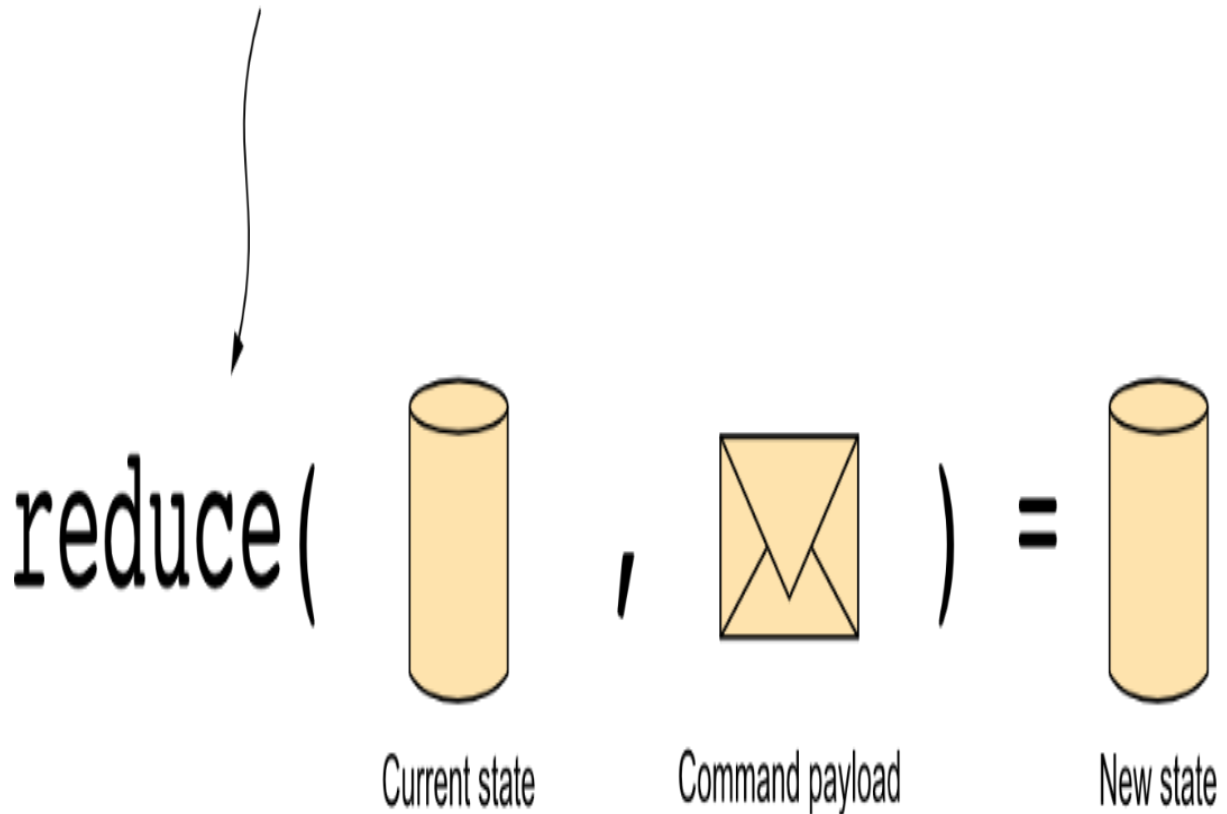
Find the solution at <http://mng.bz/n1MV>.

5.1.2 The reducer functions

Reducer functions can be implemented in a few different ways, but if we decide to stick to the functional programming principles of using *pure functions* and making data *immutable*, instead of updating the state by mutating it, these functions should create a new one. This is illustrated in figure [5.6](#).

Figure 5.6 A reducer function takes the current state and a payload and returns a new state

A function that given the current state
and the payload of a command, returns
a new, updated state



Note

This might sound familiar to you if you've used [Redux](#) before: these functions are the *reducers*. (The term *reducer* comes from the *reduce* function in functional programming.) A reducer, in our context, is a function that takes the current state and a payload (the command's data) and returns a new updated state. These functions never mutate the state that's passed to them (that'd be considered a *side effect*, and therefore the function wouldn't be pure), but instead create a new state.

Let's see an example with the TODOs application. To create a new version of the state when the user removes a to-do item from the list (recall that the state was just the list of to-dos), the reducer function associated to this `'remove-todo'` command would look like this:

```
function removeTodo(state, todoIndex) {  
  return state.toSpliced(todoIndex, 1)  
}
```

If we had the following state:

```
let todos = ['Walk the dog', 'Water the plants', 'Sand the  
chairs']
```

And we wanted to remove the to-do item at index 1, the new state would be computed using the `removeTodo()` reducer, as follows:

```
todos = removeTodo(todos, 1)  
// todos = ['Walk the dog', 'Sand the chairs']
```

In this case, the payload associated with the `'remove-todo'` command is the index of the to-do item to remove. Note how the original array is not mutated, but a new one is created instead (the `toSpliced()` method of an array returns a new array).

Exercise 5.2 — challenge

Imagine you're building a [Tic-Tac-Toe](#) game as a web application. You have a 3x3 grid of squares, and when one of the two players clicks on one of them, you want to mark it with an x (cross) or an o (nought). The player who places three of their marks in a horizontal, vertical, or diagonal row wins the game.

How would you design the state of the application? What commands would you dispatch to the framework in response to the user clicking a square? What reducer functions would you implement to update the state?

Here are some tips to help you get started:

- The state needs to have, at least, four pieces of information: the grid of squares, the player who's turn it is, the winner of the game (if any), and

- whether the game ended in a draw.
- To design what commands are needed, think about what actions the user can perform in the application.
 - The reducers can be deduced from the commands, and how the state needs to be updated in response to them.
-

Find the solution at <http://mng.bz/vPzm>.

Let's do a quick recap of what we've seen so far. We've seen that the application developer translates browser events into application domain commands, and that the state manager executes the reducer functions associated to those commands to update the state. But, how does the state manager know which reducer function to execute when a command is dispatched? There needs to be something that maps the commands to the reducer functions. We'll call this mechanism a *dispatcher*, and it's the state manager's central piece.

5.1.3 The dispatcher

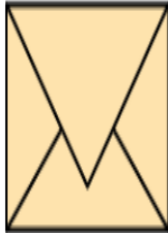
The association between commands and reducer functions is done by an entity we'll call the *dispatcher*. The name *dispatcher* reflects the fact that this entity is responsible for dispatching the commands to the functions that handle the command, that is, for executing the corresponding handler functions in response to commands. To do this, it needs to be told—by the application developer—which handler function (or functions, as there might be more than one) to execute in response to each command.

These command handler functions are *consumers*. Consumer is the technical term to refer to a function that accepts a single parameter—the command's payload in our case—and returns no value, as figure [5.7](#) shows.

Figure 5.7 A consumer function takes a single parameter and returns no value

A function that accepts a single parameter
and returns no value



consume() = void

Payload

A consumer that handles a command can easily wrap a reducer function, as the following example shows:

```
function removeTodoHandler(todoIndex) {  
  // Calls the removeTodo() reducer function to update the  
  state.  
  state = removeTodo(state, todoIndex)  
}
```

As you can see, the command handler function that removes a to-do from the list receives the to-do index as its single parameter, and then calls the

`removeTodo()` reducer function to update the state. The handler simply wraps the reducer function, and it's the dispatcher's responsibility to execute the handler function in response to the `'remove-todo'` command.

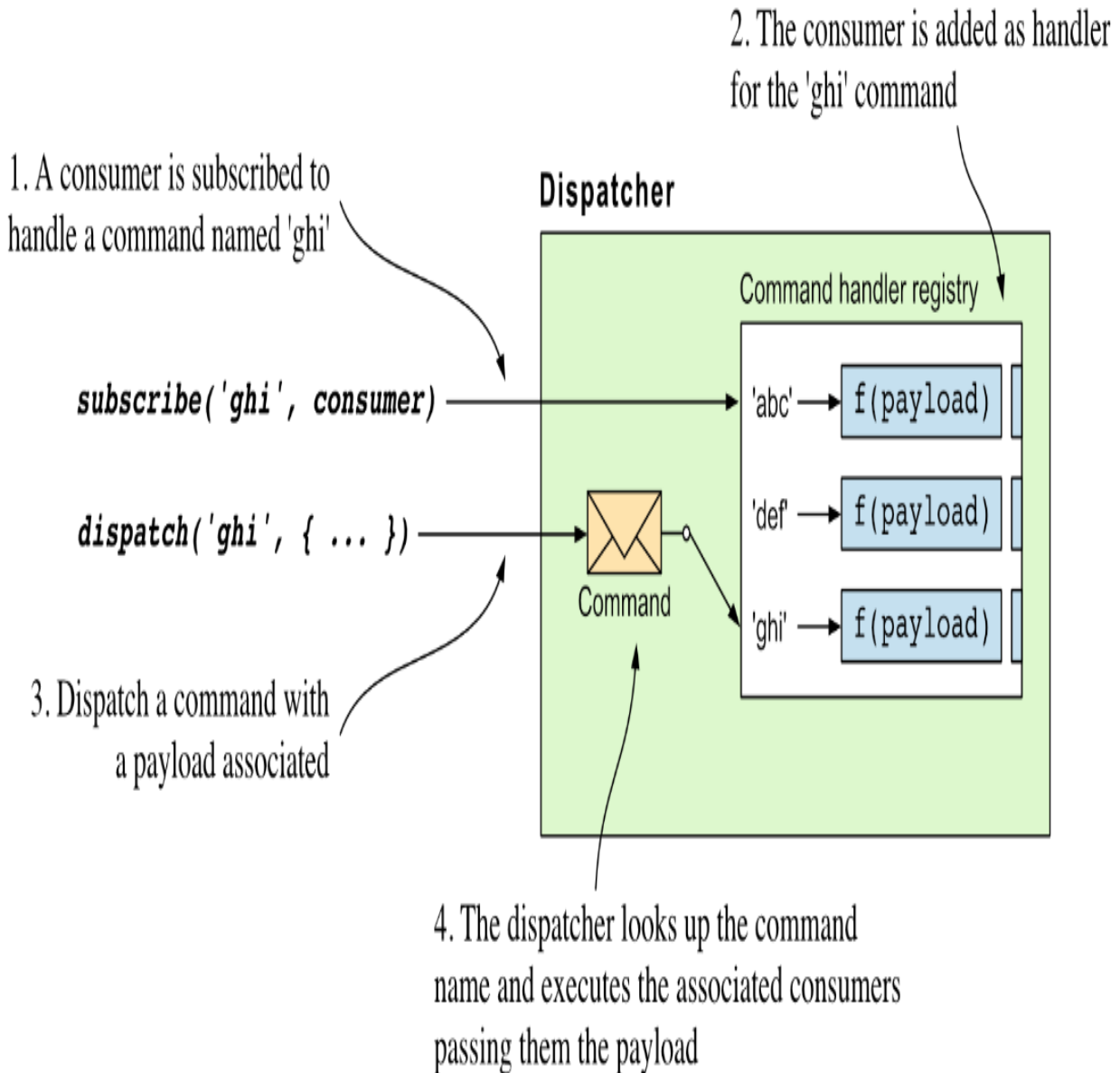
But how do you tell the dispatcher which handler function to execute in response to a command?

Assigning handlers to commands

Your dispatcher needs to have a `subscribe()` method that registers a consumer function—the handler—to respond to commands with a given name. And the same way you can register a handler for a command, you should be able to un-register it when it doesn't need to be executed anymore (because the relevant view has been removed from the DOM, for example). To accomplish this, the `subscribe()` method should return a function that can be called to un-register the handler.

Your dispatcher also needs to have a `dispatch()` method that executes the handler functions associated to a command. You can see in figure [5.8](#) a sequence diagram of how the dispatcher works.

Figure 5.8 The dispatcher's `subscribe()` method registers handlers to respond to commands with a given name, and `dispatch()` executes the handlers associated to a command



It's time to implement the dispatcher. Create a new file called *dispatcher.js* in the *src/* directory. Your *runtime/src/* directory should look like this (in bold font the new files):

```
src/
├── utils/
│   └── arrays.js
├── attributes.js
├── destroy-dom.js
├── dispatcher.js
├── events.js
└── h.js
```

```
└─ index.js
└─ mount-dom.js
```

Then, write the following code in listing [5.1](#).

Note

The hash # in front of the variable name is the ES2020 way to make the variable private inside a class. Starting from ES2020, any variable or method that starts with a hash is private and can only be accessed from within the class.

Listing 5.1 Registering handlers to respond to commands (dispatcher.js)

```
export class Dispatcher {
  #subs = new Map()

  subscribe(commandName, handler) {
    if (!this.#subs.has(commandName)) { #1
      this.#subs.set(commandName, [])
    }

    const handlers = this.#subs.get(commandName)
    if (handlers.includes(handler)) { #2
      return () => {}
    }

    handlers.push(handler) #3

    return () => { #4
      const idx = handlers.indexOf(handler)
      handlers.splice(idx, 1)
    }
  }
}
```

The `Dispatcher` is a class with a private variable called `subs` (short for *subscriptions*): a [JavaScript map](#) to store the registered handlers by event name. Note how more than one handler can be registered for the same command name.

The `subscribe()` method takes a command name and a handler function as parameters, checks if there's an entry in `subs` for that command name, and

creates one with an empty array if there isn't. Then appends the handler to the array in case it wasn't already registered. If the handler was already registered, you simply return a function that does nothing, because there's nothing to unregister.

If the handler function was registered, the `subscribe()` method returns a function that removes the handler from the corresponding array of handlers, so it's never notified again. To do this, you first look for the index of the handler in the array, and then call its `splice()` method to remove the element at that index. Note that the index lookup only happens when the returned function—the un-registering function—is called. This is a very important detail, because if you did the lookup outside that function—inside the `subscribe()` method body—the index that you'd get might not be valid by the time you want to unregister the handler, because the array might have changed in the meantime.

Warning

In the case where the handler is already registered, instead of returning an empty function, you might be tempted to return a function that un-registers the existing handler. But returning an empty function is a better idea, because it avoids the side effects of a developer inadvertently calling the returned function twice (one for each time they called `subscribe()` using the same handler). In this case, when the same handler is unregistered for the second time, `indexOf()` returns `-1`, because the handler isn't in the array anymore. It'd then call `splice()` with an index of `-1`, which would remove the last handler in the array, and this is not what you want.

This *silent failure* — something going wrong without throwing an exception — is something we want to avoid at all costs, as debugging these kinds of issues can be a nightmare.

Now that you've implemented the dispatcher first method, `subscribe()`, there's something we haven't addressed yet: how does the dispatcher tell the renderer about state changes? Let's try to figure this out next.

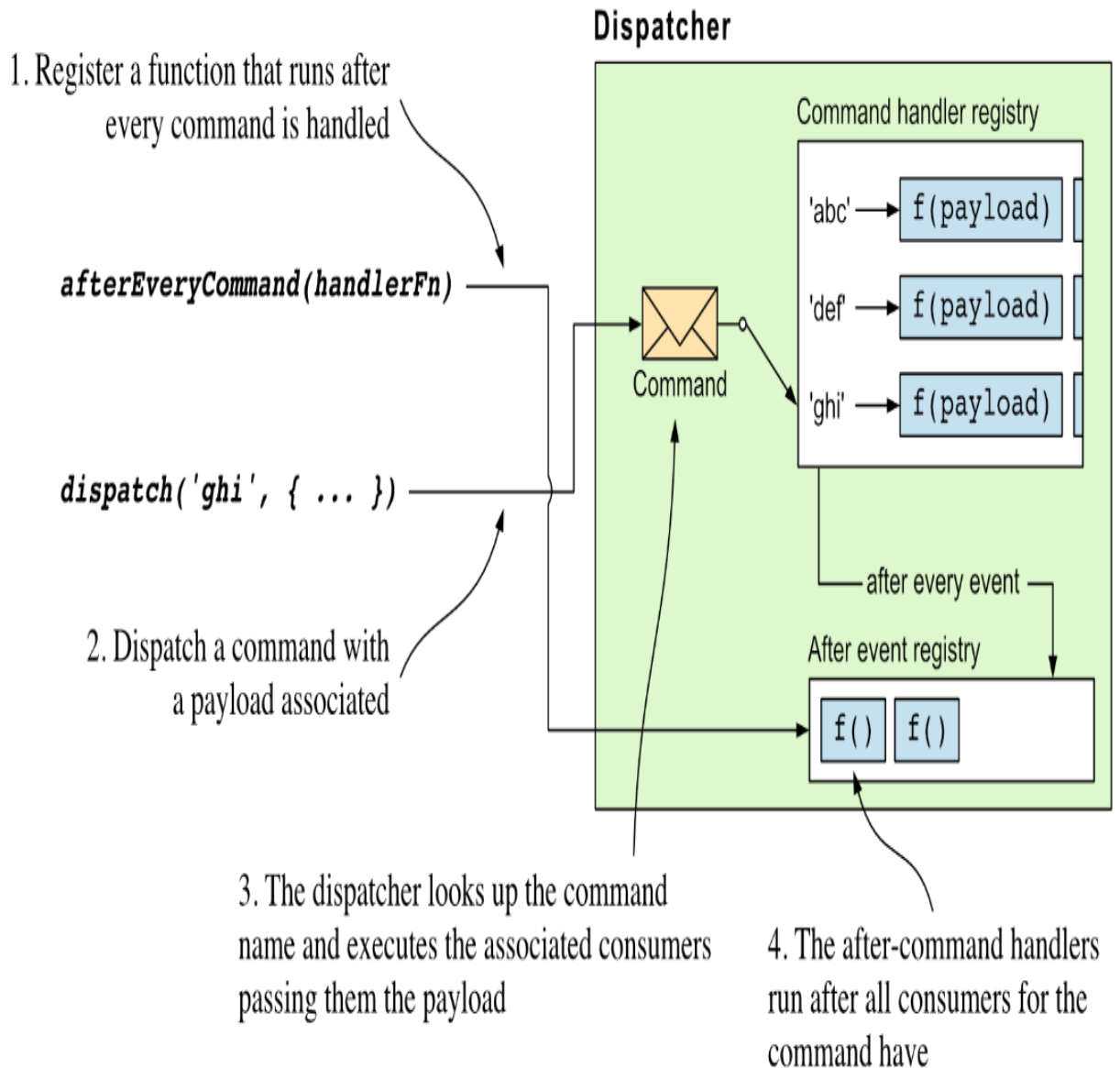
Notifying the renderer about state changes

In the beginning of this chapter we said that the state manager is in charge of keeping the state in sync with the views. It does so by notifying the renderer about state changes, so that the renderer can update the views accordingly. The question is then: how does the dispatcher notify the renderer?

You know that the state can change only in response to commands. A command triggers the execution of one or more handler functions, which execute reducers, which in turn update the state. Therefore, the best moment to notify the renderer about state changes is after the handlers for a given command have been executed. You should allow the dispatcher to register special handler functions, which we'll call *after-command handlers*, and are executed after the handlers for any dispatched command have been executed. The framework uses these handlers to notify the renderer about potential state changes, so it can update the view.

The diagram in figure [5.9](#) shows how the `afterEveryCommand()` as part of the dispatcher's architecture. The functions registered with this method are called after every command is handled, which you can use to notify the renderer about state changes.

Figure 5.9 The dispatcher's `afterEveryCommand()` method registers handlers to run after every command is handled



Write the code in bold font that's shown inside the `Dispatcher` class, to add the `afterEveryCommand()` method:

Listing 5.2 Registering functions to run after commands (dispatcher.js)

```
export class Dispatcher {
  #subs = new Map()
  #afterHandlers = []

  // --snip--
}
```



```

afterEveryCommand(handler) {
  this.#afterHandlers.push(handler) #1

  return () => { #2
    const idx = this.#afterHandlers.indexOf(handler)
    this.#afterHandlers.splice(idx, 1)
  }
}
}

```

This method is very similar to the `subscribe()` method, except that it doesn't take a command name as a parameter: these handlers are called for all dispatched commands. This time we're not checking for duplicates; we're allowing for the same handler to be registered multiple times. After-command handlers don't modify the state, they are a notification mechanism, and so notifying about the same event multiple times might be a valid use-case.

The last part that's missing is the `dispatch()` method, to dispatch a command and call all the registered handlers.

Dispatching commands

A dispatcher wouldn't be much of a dispatcher if it didn't have a `dispatch()` method, would it? This method takes two parameters: the name of the command to dispatch and its payload. It looks up the handlers registered for the given command name, and calls them one by one, in order, passing them the command's payload as parameter. Last, it runs the after-command handlers.

Listing 5.3 Dispatching a command given its name and payload (dispatcher.js)

```

export class Dispatcher {
  // --snip--

  dispatch(commandName, payload) {
    if (this.#subs.has(commandName)) { #1
      this.#subs.get(commandName).forEach((handler) =>
handler(payload))
    } else {
      console.warn(`No handlers for command: ${commandName}`)
    }
  }
}

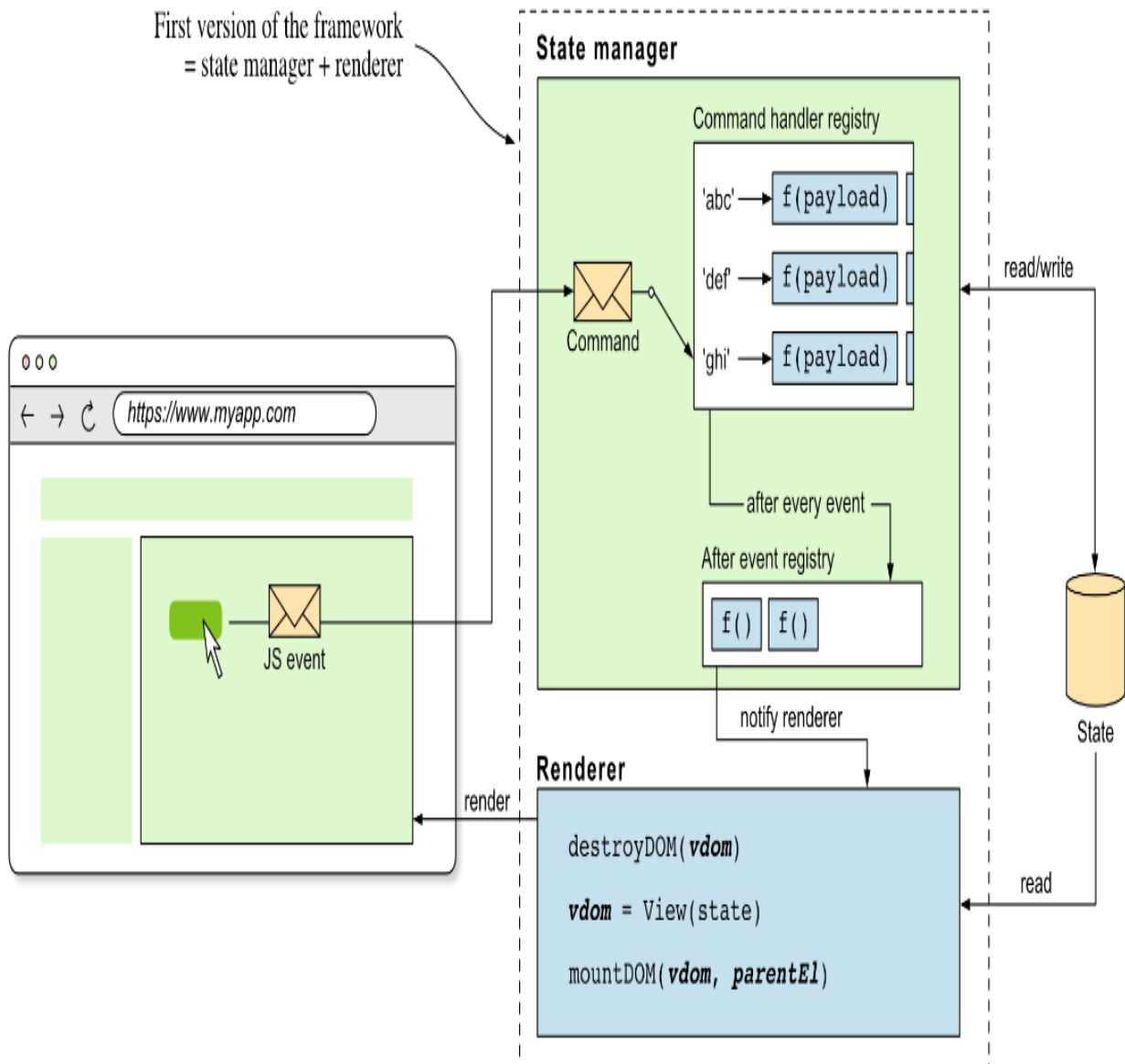
```

```
    this.#afterHandlers.forEach((handler) => handler()) #2  
  }  
}
```

Note that, if a command with no handlers associated is dispatched, we warn the developer about it in the console. This will be handy later on when we write our example applications, because it's easy to misspell a command name and not notice it, but then bang your head against the wall because you don't understand why your code isn't working.

That's it! Figure [5.10](#) shows the framework's first version architecture. You've implemented the dispatcher—the state manager—and now it's time to integrate it with the renderer to form a working framework.

Figure 5.10 The framework's first version architecture



Before integrating the dispatcher with the rendering system, make sure the code you wrote matches the one in [listing 5.4](#) in the next section. If you need to copy and paste to compare, you can find the complete listing at the book's GitHub repository (see appendix A for more information).

Result

Just for your reference, here's the complete `Dispatcher` class implementation:

Listing 5.4 Complete Dispatcher implementation (dispatcher.js)

```
export class Dispatcher {
  #subs = new Map()
  #afterHandlers = []

  subscribe(commandName, handler) {
    if (!this.#subs.has(commandName)) {
      this.#subs.set(commandName, [])
    }

    const handlers = this.#subs.get(commandName)
    if (handlers.includes(handler)) {
      return () => {}
    }

    handlers.push(handler)

    return () => {
      const idx = handlers.indexOf(handler)
      handlers.splice(idx, 1)
    }
  }

  afterEveryCommand(handler) {
    this.#afterHandlers.push(handler)

    return () => {
      const idx = this.#afterHandlers.indexOf(handler)
      this.#afterHandlers.splice(idx, 1)
    }
  }

  dispatch(commandName, payload) {
    if (this.#subs.has(commandName)) {
      this.#subs.get(commandName).forEach((handler) =>
handler(payload))
    } else {
      console.warn(`No handlers for command: ${commandName}`)
    }

    this.#afterHandlers.forEach((handler) => handler())
  }
}
```

Make sure you wrote the code correctly and your implementation matches the one in listing [5.4](#). If so, let's move on to the next section.

Exercise 5.3

To put your newly implemented `Dispatcher` class to use, paste the code in listing [5.4](#) into the browser's console (remember to leave the `export` statement out). Then, create a new instance of the `Dispatcher` class and register a handler for a command called `'greet'`. This command payload should be a string with the name of the person to greet. When the command is dispatched, the handler should log a greeting to the console: `'Hello, <name>!'` (where `<name>` is the name of the person in the payload). You also want to have an after-command handler that logs a message to the console saying "Done greeting!".

For example, when the command `'greet'` is dispatched with the payload `'John'`:

```
dispatcher.dispatch('greet', 'John')
```

the console should log the following:

```
Hello, John!  
Done greeting!
```

Find the solution at <http://mng.bz/46Z5>.

5.2 Assembling the state manager into the framework

Assemble is defined by Merriam-Webster as *to bring together (as in a particular place or for a particular purpose)*. To assemble the state manager and renderer, you need a "particular place" where to bring them together. This would be an object that contains and connects them so they can communicate. If you think about it, this object represents the running application that uses your framework, so we can refer to it as the *application instance*. Let's think about how you want developers to create an application instance in your framework.

5.2.1 The application instance

The application instance is the object that manages the lifecycle of the application: it manages the state, renders the views, and updates the state in response to user input. There are three things that developers need to pass to the application instance:

- The initial state of the application.
- The reducers that update the state in response to commands.
- The top-level component of the application.

Your framework can take care of the rest: instantiating a renderer and a state manager, and wiring them together. (Remember, your initial version of the framework won't be much more than these two things glued together.) The application instance can expose a `mount()` method that takes a DOM element as parameter, mounts the application in it, and kicks off the application's lifecycle. From this point on, the application instance is in charge of keeping the state in sync with the views, like the waiter in the restaurant from the beginning of the chapter.

This might sound a bit abstract at the moment, but it'll become clear later on when you rewrite the TODO application using your framework. For now, bear with me and let's move on to implementing the application instance. Let's start with the renderer.

5.2.2 The application instance's renderer

To start, you implement a function called `createApp()` that returns an object with a single method, `mount()`, which takes a DOM element as parameter and mounts the application in it. This object is the application instance inside which you implement the renderer and the state manager.

The `createApp()` function takes an object with two properties: `state` and `view`. The `state` property is the initial state of the application, and the `view` property is the top-level component of the application. You'll add the reducers later.

You need two variables in the closure of the `createApp()` function: `parentEl` and `vdom`. They keep track of the DOM element where the application is mounted and the virtual DOM tree of the previous view,

respectively. They should both be initialized to `null` because the application hasn't been mounted yet.

Then goes the renderer, which is implemented as a function: `renderApp()`. This function, as previously discussed, renders the view by first destroying the current DOM tree—if there is one—and mounting the new one. At this point, this function is called only once: when the application is mounted by the `mount()` method, the only method exposed to the developer. It takes a DOM element as parameter and mounts the application in it. Note how you save the DOM element in the `parentEl` variable so that you can use it later to unmount the application.

Create a new file inside the `src/` directory called `app.js` and write the code in listing [5.5](#).

Listing 5.5 The application instance with its renderer (app.js)

```
import { destroyDOM } from './destroy-dom'
import { mountDOM } from './mount-dom'

export function createApp({ state, view }) { #1
  let parentEl = null
  let vdom = null

  function renderApp() {
    if (vdom) {
      destroyDOM(vdom) #2
    }

    vdom = view(state)
    mountDOM(vdom, parentEl) #3
  }

  return {
    mount(_parentEl) { #4
      parentEl = _parentEl
      renderApp()
    },
  }
}
```

Okay, you've got the renderer, you could already render an application in the browser, but it wouldn't respond to user input. For that, you need the state

manager, that tells the renderer to re-render the application when the state changes. Let's move on to that.

5.2.3 The application instance's state manager

The state manager is a bit more complex than just a function, like the renderer was. The `Dispatcher` class you implemented in the previous section is the central piece of the state manager, but you have to hook some things up. Most notably, you need to wrap the state reducers—given by the developer—in a consumer function that will be called by the dispatcher every time a command is dispatched. Let's see how this is done.

Write the code in bold in listing [5.6](#). Note that part of the code you wrote earlier is elided for clarity.

Listing 5.6 Adding the state manager to the application instance (app.js)

```
import { destroyDOM } from './destroy-dom'
import { Dispatcher } from './dispatcher'
import { mountDOM } from './mount-dom'

export function createApp({ state, view, reducers = {} }) {
  let parentEl = null
  let vdom = null

  const dispatcher = new Dispatcher()
  const subscriptions =
[dispatcher.afterEveryCommand(renderApp)] #1

  for (const actionName in reducers) {
    const reducer = reducers[actionName]

    const subs = dispatcher.subscribe(actionName, (payload)
=> {
      state = reducer(state, payload) #2
    })
    subscriptions.push(subs) #3
  }

  // --snip-- //
}
```


Let's unpack what you've done here. First, you've added a `reducers` property into the `createApp()` function parameter. This property is an object which maps command names to reducer functions, functions that take the current state and the command's payload and return a new state.

Next, you've created an instance of the `Dispatcher` class and saved it in the `dispatcher` variable. The line below that is crucial: you've subscribed the `renderApp()` function to be an after-command handler, so that the application is re-rendered after every command is handled. Not every command necessarily changes the state, but you don't know that in advance, so you have to re-render the application after every command.

Note

To avoid re-rendering the application when the state didn't change, you could compare the state before and after the command was handled. This comparison can nevertheless become expensive if the state is a heavy and deeply nested object and the commands are frequent. In chapters [7](#) and [8](#) you'll improve the performance of the renderer, by only patching the DOM where it's necessary, so re-rendering the application will be a reasonably fast operation. Not checking if the state changed is a trade-off we're making to keep the code simple.

Can you see why the concept of after-command handlers were a good idea now? Note that, the `afterEveryCommand()` function returns a function that unsubscribes the handler, so you've saved it in the `subscriptions` array, an array that you've initialized to have this function as its first element.

You then iterate the `reducers` object, wrap each of the reducers inside a handler function that calls the reducer and updates the state, and subscribe that handler to the dispatcher. You've been careful to save the subscription functions in the `subscriptions` array, so that you can unsubscribe them later when the application is unmounted.

Great—you've got the state manager hooked up to the renderer. But there's something we haven't talked about yet: how do the components dispatch commands? Let's look at that next.

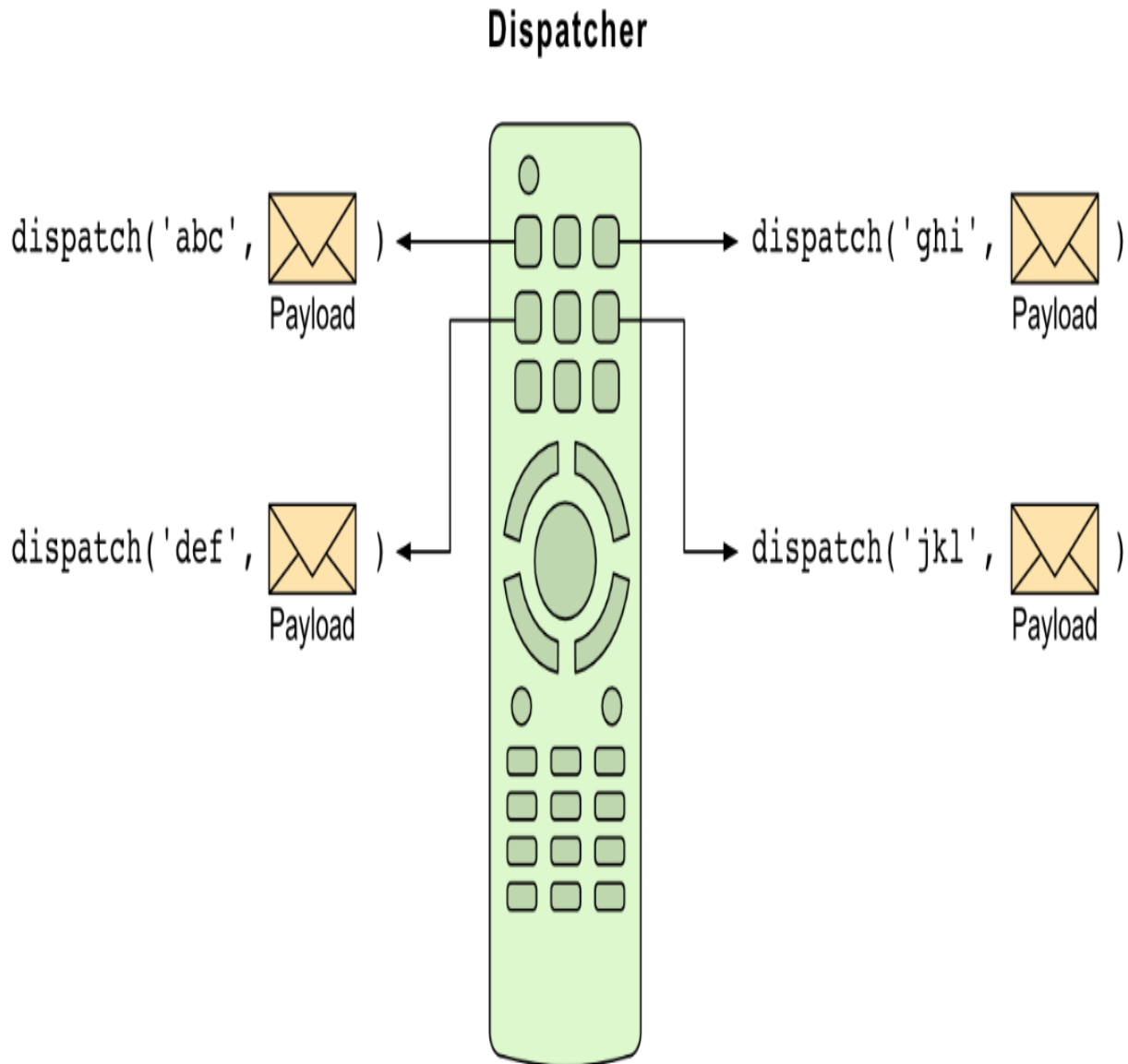
5.2.4 Components dispatching commands

If you recall, your virtual DOM implementation allows to attach event listeners to DOM elements:

```
h(  
  'button',  
  { on: { click: () => { ... } } },  
  ['Click me']  
)
```

If you want to be able to dispatch commands from within those event listeners, you need to pass the dispatcher to the components. In a way, you can imagine the dispatcher as a remote control, where each button dispatches a command whose handler function can modify the state of the application (figure [5.11](#)). By passing the dispatcher to the components, you give them the ability to dispatch commands in response to user input.

Figure 5.11 The dispatcher can be thought of as a remote control, where each button dispatches a command whose handler function can modify the state of the application



The dispatcher in the application instance has the command handlers that the developer has provided. The component can dispatch those commands using the `dispatch()` method of the dispatcher. For example, to remove a to-do item from the list, the component can dispatch a `remove-todo` command like this:

```
h(  
  'button',  
  {  
    on: {  
      click: () => dispatcher.dispatch('remove-todo',
```

```

    todoIdx)
      }
    },
    ['Done']
  )
)

```

To allow the components to dispatch commands, change your code in *app.js*, adding the code in bold in listing [5.7](#).

Listing 5.7 Allowing components to dispatch commands (app.js)

```

export function createApp({ state, view, reducers = {} }) {
  let parentEl = null
  let vdom = null

  const dispatcher = new Dispatcher()
  const subscriptions =
[dispatcher.afterEveryCommand(renderApp)]

  function emit(eventName, payload) {
    dispatcher.dispatch(eventName, payload)
  }

  // --snip-- //

  function renderApp() {
    if (vdom) {
      destroyDOM(vdom)
    }

    vdom = view(state, emit)
    mountDOM(vdom, parentEl)
  }

  // --snip-- //
}

```

To allow components to dispatch commands in a more convenient way, you've implemented an `emit()` function. So instead of writing `dispatcher.dispatch()`, you can write `emit()` inside the components, which is a bit more concise. Then, you've passed the `emit()` function to the components as a second argument.

Bear in mind that, from now onward, components will receive two arguments: the state and the `emit()` function. If a component doesn't need to dispatch commands, it can ignore the second argument.

You're almost done! There's only one thing missing: unmounting the application.

5.2.5 Unmounting the application

When an application instance is created, the state reducers are subscribed to the dispatcher, and the `renderApp()` function is subscribed to the dispatcher as an after-command handler. When the application is unmounted, apart from destroying the view, you need to unsubscribe the reducers and the `renderApp()` function from the dispatcher.

To clean up the subscriptions and destroy the view, you need to add an `unmount()` method to the application instance, as shown in bold in listing [5.8](#).

Listing 5.8 Unmounting the application (app.js)

```
export function createApp({ state, view, reducers = {} }) {
  let parentEl = null
  let vdom = null

  // --snip-- //

  return {
    mount(_parentEl) {
      parentEl = _parentEl
      renderApp()
    },

    unmount() {
      destroyDOM(vdom)
      vdom = null
      subscriptions.forEach((unsubscribe) => unsubscribe())
    },
  }
}
```

The `unmount()` method uses the `destroyDOM()` from last chapter to destroy the view, sets the `vdom` property to `null`, and unsubscribes the reducers and

the `renderApp()` function from the dispatcher.

That's it! It's a good time to review the code you wrote in *app.js* to make sure you got it right.

5.2.6 Result

Your *app.js* should look like the code in listing [5.9](#).

Listing 5.9 The application instance (app.js)

```
import { destroyDOM } from './destroy-dom'
import { Dispatcher } from './dispatcher'
import { mountDOM } from './mount-dom'

export function createApp({ state, view, reducers = {} }) {
  let parentEl = null
  let vdom = null

  const dispatcher = new Dispatcher()
  const subscriptions =
    [dispatcher.afterEveryCommand(renderApp)]

  function emit(eventName, payload) {
    dispatcher.dispatch(eventName, payload)
  }

  for (const actionName in reducers) {
    const reducer = reducers[actionName]

    const subs = dispatcher.subscribe(actionName, (payload)
=> {
      state = reducer(state, payload)
    })
    subscriptions.push(subs)
  }

  function renderApp() {
    if (vdom) {
      destroyDOM(vdom)
    }

    vdom = view(state, emit)
    mountDOM(vdom, parentEl)
  }
}
```

```

return {
  mount(_parentEl) {
    parentEl = _parentEl
    renderApp()
  },

  unmount() {
    destroyDOM(vdom)
    vdom = null
    subscriptions.forEach((unsubscribe) => unsubscribe())
  },
}
}

```

That's the first version of your framework! Put together in less than 50 lines of code, it's a pretty simple framework, but it's enough to build simple applications.

In the next chapter, you build and publish your framework to NPM and refactor the TODOs application to use it.

5.3 Summary

- Your framework's first version is made of a renderer and a state manager wired together.
- The renderer first destroys the DOM (if it exists) and then creates it from scratch. This isn't very efficient and creates problems with the focus of input fields, among other things.
- The state manager is in charge of keeping the state and view of the application in sync.
- The developer of an application maps the user interactions to commands, framed in the business language, that are dispatched to the state manager.
- The commands are processed by the state manager, updating the state and notifying the renderer that the DOM needs to be updated.
- The state manager uses a reducer function to derive the new state from the old state and the command's payload.

6 Publishing and using your framework's first version

This chapter covers

- Publishing the first version of your framework to NPM
- Refactoring the TODOs application to use your framework

In the previous chapter you implemented a state manager and assembled it, together with a renderer, to build the first version of your frontend framework. In this chapter, you'll publish the first version of your framework, and then refactor the TODOs application you built using vanilla JavaScript to use it.

Important

You can find all the listings in this chapter in the [listings/ch06 directory](#) of the book's repository. The code in this chapter can be checked out from [the ch6 label](#):

```
$ git switch --detach ch6
```

Code catch-up

In the last chapter you implemented the `Dispatcher` class, used to dispatch commands from the application and subscribe handler functions to those commands. You also implemented the `createApp()` function, which takes in the state of the application, reducer functions to update the state, and a view function to render the application, and wires everything together.

6.1 Building and publishing the framework

The first thing you want to do to build your framework is export the functions that you want to expose to the developer using it from the `src/index.js` barrel file. Whatever you export from this file is what makes the public API of the

framework. The *src/index.js* is the entry point of the build process, so everything that's exported from this file will be made available in the final bundle.

You want developers to use the `h()`, `hString()`, and `hFragment()` functions to create virtual DOM nodes, and the `createApp()` function to create an application. You don't need to export the `mountDOM()` and `destroyDOM()` functions, because they're only used internally by the framework.

Open the *src/index.js* file, delete whatever is in there, and add the following lines:

```
export { createApp } from './app'
export { h, hFragment, hString } from './h'
```

Now, run the `build` script inside the *runtime* workspace to build the framework:

```
$ npm run build
```

As you've seen in appendix A, this script bundles the JavaScript code into a single ESM file: *dist/<fwk-name>.js*. (Recall that *<fwk-name>* is the name of your framework, as named in the *package.json* file's *name* property.)

Important

Before you proceed, make sure you followed the instructions in appendix A to set up your NPM account to publish your package.

You can import this file directly from *packages/runtime/dist/<fwk-name>.js* in your examples to use the framework, but you can also publish it to NPM so that you and other developers can install it in their projects using NPM like so:

```
$ npm install <fwk-name>
```

You want the `version` field in your *package.json* file to be `1.0.0`; this is the first version of your framework, but you'll be publishing more versions later.

```
"version": "1.0.0",
```

To publish the package to <https://www.npmjs.com/>, make sure your terminal's working directory is the *runtime* workspace, and run the `publish` NPM script, as follows:

```
$ npm publish
```

And that's it: You've published the first version of your frontend framework to NPM.

You can find it at <https://www.npmjs.com/package/> followed by the name of your framework (the `name` field in your `package.json` file), and can install it in your projects through NPM by running the command, `npm install <fwk-name>`. You can also find it at <https://unpkg.com/> followed by the name of your package, where you can import it directly in your HTML files. We'll use the latter method in most of the examples in the book.

6.2 A short example

You're probably excited to start using your framework to see how it works; we haven't seen a complete example yet. You'll rewrite your `TODOs` app to use the framework in a minute, but I want to help you make sense of everything you've done so far; to that end, you'd probably appreciate a short example that shows how the framework works.

You don't need to write the code in this section (unless you want to try it for yourself); you can just read it and see how the framework works. You'll get your hands dirty in the next section.

I can't think of anything simpler than a button that counts how many times you click it. This application's view is just a button, but it's interactive: it renders something different based on the state, which is a number representing a count. You can implement this simple application with the following few lines of code:

```
createApp({
  state: 0,

  reducers: {
    add: (state, amount) => state + amount,
```

```

    },
    view: (state, emit) =>
      h(
        'button',
        { on: { click: () => emit('add', 1) } },
        [hString(state)]
      ),
  }) .mount(document.body)

```

This is how little code you need to make a simple application with the framework. No DOM manipulation drama; that's handled by the framework now so you can focus on the important stuff: the application's logic.

This application renders as a single button with the number 0 on it (you haven't clicked it yet):

```

<body>
  <button>0</button>
</body>

```

When you click the button, the number it displays increments by one, and the button renders the new number:

```

<body>
  <button>1</button>
</body>

```

The framework removes the `<button>` element from the DOM and creates a new one every time the state changes—that is, when you click the button. But this happens in a fraction of a millisecond, so your eye can't even notice it. To you, it looks like the button is updating its number, but in reality, you never click the same button twice. (How philosophical is that?)

Exercise 6.1

Using your framework, implement a counter application that allows the user to increment and decrement a counter. The counter should start at 0 and the user should be able to increment it by clicking a button with a "+" (plus sign) label. The user should also be able to decrement it by clicking another button, this time with the "-" (minus sign) label. The counter should be displayed between the two buttons. You can create the application inside the *examples/* directory.

Find the solution at <http://mng.bz/QRml>.

Let's put the framework to use in the TODOs application, shall we?

6.3 Refactoring the TODOs app

Let's now take the TODOs application you built without a framework and refactor it to use the framework you just built. You'll see how much simpler the code becomes (except for the nuances of writing virtual DOMs instead of HTML), and be in a good position to assess the benefits of using a frontend framework versus writing your own DOM-manipulation code.

The first thing you want to do to refactor the TODOs application is to clean up the `<body>` tag in the *todos.html* file. All the HTML markup will be created by your framework this time, so the `<body>` tag should be empty. Your *todos.html* file should look like listing [6.1](#).

Listing 6.1 Remove all the markup from the HTML file (todos.html)

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <script type="module" src="todos.js"></script>
    <title>My TODOs</title>
  </head>

  <body></body> #1
</html>
```

Do the same with the *todos.js* file; you'll be writing all the code from scratch. Then, import the functions exported by the framework from either the *dist/* directory or, preferably, from unpkg.com. (I will be using the unpkg.com in this book, but you can use whichever method you prefer.) Here's what your *todos.js* file should look like:

```
import { createApp, h, hFragment } from
'https://unpkg.com/<fwk-name>@1'
```

Note

Please recall (and this will be the last time I mention it to avoid sounding repetitive) that *<fwk-name>* is the name of your framework, as named in the *package.json* file's *name* property.

Next, you want to define the application's state. This time, it'll be a bit more nuanced than a simple array of to-do items.

6.3.1 Defining the state

The state of the TODOs application, when you wrote it using vanilla JavaScript, was simply an array of strings, where each string was a to-do item. But there were other pieces of information that you didn't need to keep as part of the state, like the text of the new to-do item that the user was typing into the input field, because we could simply grab it from the DOM, like so:

```
const addTodoInput = document.getElementById('todo-input')

// The text of the new to-do item
addTodoInput.value
```

The idea of using a framework is to abstract away the manipulation of the DOM, so we want to avoid accessing the DOM at all. Any piece of information that's relevant to the application should be part of the state. The value of the input field where the user writes the new to-do item's text has to be part of the state, and it needs to be up to date with what the user is typing.

There are three pieces of information that you'll need to keep in the state this time:

- `todos`: the array of to-do items (the same as before).
- `currentTodo`: the text of the new to-do item that the user is typing into the input field.
- `edit`: an object containing information about the to-do item being edited by the user:
 - `idx`: the index of the to-do item in the `todos` array that's being edited.

- `original`: the original text of the to-do item before the user started editing it (in case the edition is cancelled and we need to bring the original value back).
- `edited`: the text of the to-do item as the user is editing it.

With this in mind, write the code in listing [6.2](#) to define the new state.

Listing 6.2 The state for the TODOs application (todos.js)

```
import { createApp, h, hFragment } from
'https://unpkg.com/<fwk-name>@1'

const state = {
  currentTodo: '',
  edit: {
    idx: null,
    original: null,
    edited: null,
  },
  todos: ['Walk the dog', 'Water the plants'],
}
```

Let's now think about the actions that the user can perform on the application and how they affect the state.

6.3.2 Defining the reducers

This seemingly simple application has a few actions that the user can perform on it. We need to write a reducer function to update the state for each of these actions. If you think about the different ways a user can interact with the application, you'll come up with a list similar to the following:

- *Update the current to-do* — The user types a new character in the input field, so the current to-do needs to be updated.
- *Add a new to-do* — The user clicks the add button to add a new to-do to the list.
- *Start editing a to-do* — The user double clicks a to-do item to start editing it.
- *Edit a to-do* — The user types a new character in the input field while editing a to-do item.

- *Save an edited to-do* — The user finishes editing a to-do and saves the changes.
- *Cancel editing a to-do* — The user cancels editing a to-do and discards the changes.
- *Remove a to-do* — The user marks a to-do as completed so it can be removed from the list.

Write a reducer function for each of these actions below the state definition, as shown in listing [6.3](#). Recall that a reducer must be a pure function, so it can't have side effects or mutate its arguments; it has to return a new state object, not a modified version of the current state.

Listing 6.3 The reducer functions of the state (todos.js)

```
const reducers = {
  'update-current-todo': (state, currentTodo) => ({ #1
    ...state,
    currentTodo, #2
  }),

  'add-todo': (state) => ({
    ...state,
    currentTodo: '', #3
    todos: [...state.todos, state.currentTodo], #4
  }),

  'start-editing-todo': (state, idx) => ({ #5
    ...state,
    edit: {
      idx,
      original: state.todos[idx], #6
      edited: state.todos[idx], #7
    },
  }),

  'edit-todo': (state, edited) => ({ #8
    ...state,
    edit: { ...state.edit, edited }, #9
  }),

  'save-edited-todo': (state) => {
    const todos = [...state.todos] #10
    todos[state.edit.idx] = state.edit.edited #11
  }
}
```

```

    return {
      ...state,
      edit: { idx: null, original: null, edited: null }, #12
      todos,
    }
  },

  'cancel-editing-todo': (state) => ({
    ...state,
    edit: { idx: null, original: null, edited: null }, #13
  }),

  'remove-todo': (state, idx) => ({ #14
    ...state,
    todos: state.todos.filter((_, i) => i !== idx), #15
  }),
}

```

One interesting thing to note is that some events, like 'add-todo' don't have a payload associated with them. It isn't necessary because the new to-do description is now part of the state, so the reducer can access it directly:

```

'add-todo': (state) => ({
  ...state,
  currentTodo: '',
  todos: [...state.todos, state.currentTodo],
})

```

Now that you have the state and the reducers, you can define the view.

6.3.3 Defining the view

Let's break the application down into small components. We'll start with the top-level component, which you'll call `App()`. This component consists on a fragment containing the title (a `<h1>` element), a `CreateTodo()` and `TodoList()` components. Write the code in listing [6.4](#).

Listing 6.4 The `App()` component, the top-level view (todos.js)

```

function App(state, emit) {
  return hFragment([
    h('h1', {}, ['My TODOs']),
    CreateTodo(state, emit),
    TodoList(state, emit),
  ])
}

```



```
  ])  
}
```

As you recall from earlier, the components are now functions that take in not only the state, but also the `emit()` function to dispatch events to the application's dispatcher. Implement the `CreateTodo()` component next, as shown in listing 6.5. This component is equivalent to the static HTML markup you had in the *todos.html* file: the `<label>`, `<input>` and `<button>` elements.

Listing 6.5 The `CreateTodo()` component (todos.js)

```
function CreateTodo({ currentTodo }, emit) { #1  
  return h('div', {}, [  
    h('label', { for: 'todo-input' }, ['New TODO']), #2  
    h('input', {  
      type: 'text',  
      id: 'todo-input',  
      value: currentTodo, #3  
      on: {  
        input: ({ target }) => emit('update-current-todo',  
target.value), #4  
        keydown: ({ key }) => {  
          if (key === 'Enter' && currentTodo.length >= 3) {  
#5  
            emit('add-todo') #6  
          }  
        },  
      },  
    ]),  
  ]),  
  h(  
    'button',  
    {  
      disabled: currentTodo.length < 3, #7  
      on: { click: () => emit('add-todo') }, #8  
    },  
    ['Add']  
  ),  
])  
}
```

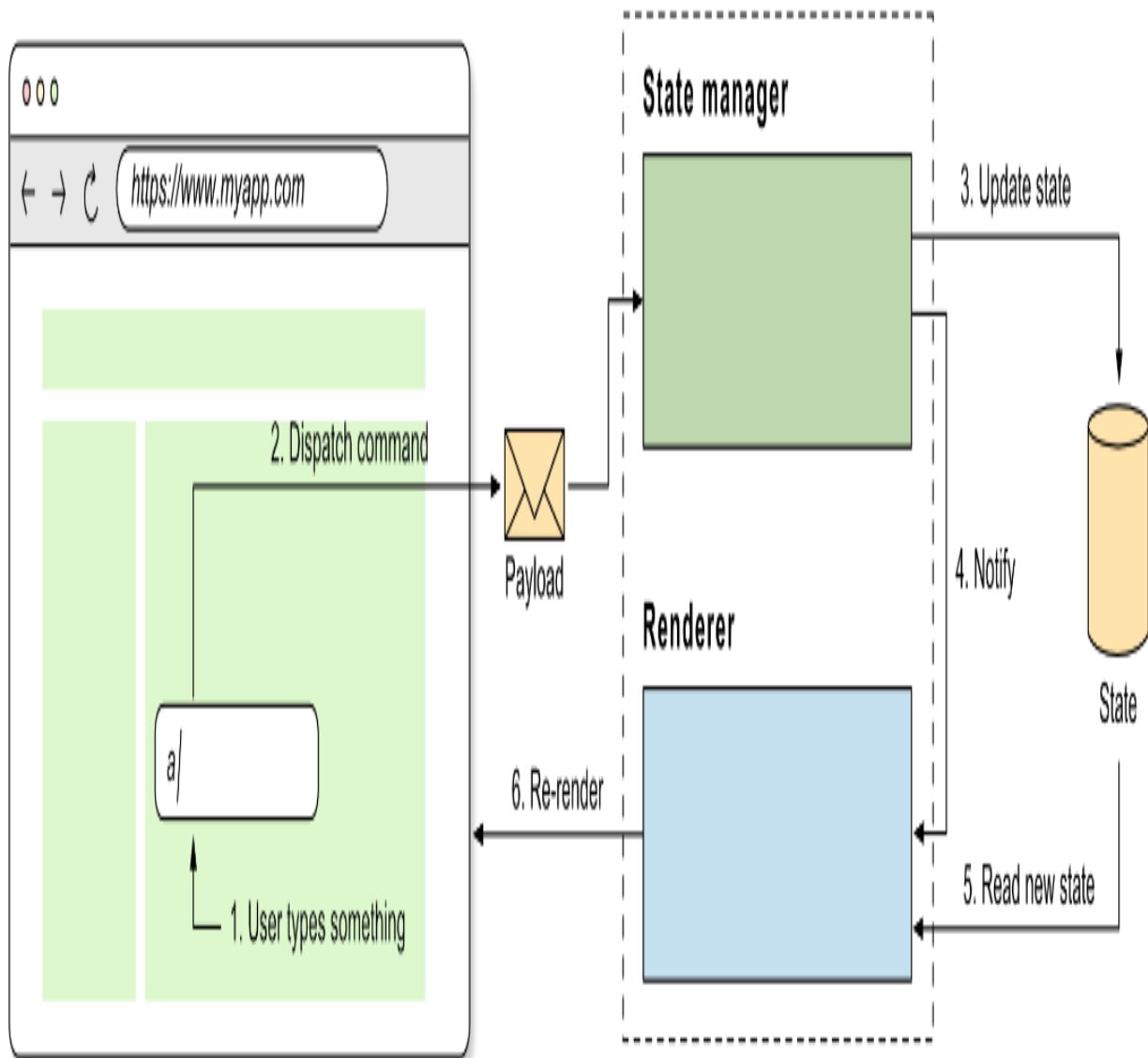
It's the first time you see the `emit()` function being used inside a component, so a few words of explanation are in order. The case of the `<input>` is particularly interesting, because you've set up a *two-way binding* between the input field and the state. A two-way binding reflects the changes in the state in

the input field and anything typed in the input field is set in the state. Whatever side changes (the state or the DOM), the other is updated. You've accomplished this two-way binding by using the `value` attribute of the `<input>` element, and by setting an event listener on the `oninput` event:

```
h('input', {
  type: 'text',
  value: state.currentTodo,
  on: {
    input: ({ target }) => emit('update-current-todo',
target.value)
  }
})
```

This way, when the `currentTodo` in the state changes its value, this is reflected in the input field, and when the user types a new character in the input field, the `oninput` event is triggered, and the `update-current-todo` event is dispatched, so the reducer updates the state. This is a beautiful put-to-use of your renderer and state manager working together, as you can see in figure [6.1](#).

Figure 6.1 A two-way binding between the state and the DOM



Let's now implement the `TodoList()` component, which is the list of to-do items. Write the following code below the `CreateTodo()` component (listing [6.6](#)).

Listing 6.6 The `TodoList()` component (todos.js)

```
function TodoList({ todos, edit }, emit) {
  return h(
    'ul',
    {},
    todos.map((todo, i) => TodoItem({ todo, i, edit }, emit))
  )
}
```

```
)  
}
```

The `TodoList()` component is a simple one: it's just a `<ul>` element with a list of `TodoItem()` components. Implement the missing `TodoItem()` component to finish the application (listing [6.7](#)).

Listing 6.7 The `TodoItem()` component (`todos.js`)

```
function TodoItem({ todo, i, edit }, emit) {  
  const isEditing = edit.idx === i  
  
  return isEditing  
    ? h('li', {}, [ #1  
      h('input', {  
        value: edit.edited, #2  
        on: {  
          input: ({ target }) => emit('edit-todo',  
target.value) #3  
        }  
      }),  
      h(  
        'button',  
        {  
          on: {  
            click: () => emit('save-edited-todo') #4  
          }  
        },  
        ['Save']  
      ),  
      h(  
        'button',  
        {  
          on: {  
            click: () => emit('cancel-editing-todo') #5  
          }  
        },  
        ['Cancel']  
      ),  
    ]) : h('li', {}, [ #6  
    h(  
      'span',  
      {  
        on: {  
          dblclick: () => emit('start-editing-todo', i)
```

```

#7
    },
    [todo] #8
  ),
  h(
    'button',
    {
      on: {
        click: () => emit('remove-todo', i) #9
      }
    },
    ['Done']
  ),
])
}

```

You're passing the index the to-do item occupies in the list, because some of the dispatched events need to know the index of the to-do item, like when you want to edit or remove a to-do. That index is known by the parent component, `TodoList()`, so it needs to pass it to the child component as a prop. It isn't part of the application's state—what we typically pass as the first argument to the component—but it's relevant to the component.

The only piece that you're missing is putting everything together. Write the last line of the *todos.js* file as follows:

```

createApp({ state, reducers, view: App
}).mount(document.body)

```

If you now run the `serve:examples` script, you'll be able to see the application running in your browser:

```

$ npm run serve:examples

```

You can now add new to-do items, edit them, and remove them. Everything works the same as before, but this time you wrote no DOM manipulation code, so you should be proud of yourself for having built your first frontend framework. Congratulations!

There's one thing though that you've probably noticed when you attempted to add a new to-do item: every keystroke you type in the input field removes the focus from the input field, so you have to click it again to type the next

character. Can you guess why this is happening? It's because every time the state changes, your framework is destroying the DOM and re-creating it from scratch. It means that the field where you wrote the last character is no longer in the DOM; you're writing in a new input field. (You never write two characters in the same input field—oh philosophy!) This is far from ideal, but worry not: you'll fix this in the next chapter.

Exercise 6.2 — Challenge

Write the tic-tac-toe game using your framework. If you need a refresher on how the game works, you can find the rules in the Wikipedia page: <https://en.wikipedia.org/wiki/Tic-tac-toe>.

The game should have the following features:

- The game should start with an empty board (you can use a `<table>` element to represent the board).
- The first player is always the X player. There should be a title that says whose turn it is: "It's X's turn" or "It's O's turn".
- When a player clicks on a cell, the cell should be filled with the player's symbol (X or O) and the turn should be passed to the other player.
- When a player wins the game, a message should be displayed saying who won the game: "Player X wins!" or "Player O wins!". The remaining cells should be disabled.
- When all the cells are filled and there's no winner, a message should be displayed saying that the game ended in a draw: "It's a draw!".

You can use the same state and reducer you used in exercise 5.2, but this time you'll need to write the logic to determine if a player has won the game (the `checkWinner()` function).

Find the solution at <http://mng.bz/Xqdl>.

The next two chapters are two of the most challenging in the book: you'll write the reconciliation algorithm, thanks to which your framework can update the DOM when the state changes, without needing to completely destroy and recreate it. The reconciliation algorithm is complex, but it's also lots of fun to

write, so I hope you'll enjoy it. Be sure to grab a cup of coffee and maybe go out for a relaxing walk before you move on to the next chapter. If you're into Yoga, you want to do a couple Sun Salutations. Whatever you do, come back refreshed and ready to write some code!

6.4 Summary

- To publish your framework on NPM, you first bundle it using `npm run build`, then you publish it using the `npm publish` command.
- Whatever you export from the *src/index.js* file is what's going to be available to the users of your framework. (You configured it this way in appendix A.)
- In a two-way binding, the state and the DOM are synchronized: whatever changes in one side is reflected in the other.
- Using the first version of your framework, you've rewritten the to-do application you wrote in chapter 2. This time, you didn't have to worry about DOM manipulation, but only about the application's logic.

7 The reconciliation algorithm: diffing virtual trees

This chapter covers

- Comparing two virtual DOM trees
- Finding the differences between two objects
- Finding the differences between two arrays
- Finding a sequence of operations that transform one array into another

Picture this: you're in the supermarket doing the groceries using the shopping list your partner gave you. You walk around the aisles, picking up the items one by one, and putting them in the cart. When you're done, you head to the checkout counter, but at that very moment something vibrates in your pocket—it must be a message from your partner. She realized you already had a dozen eggs in the fridge, and what's actually missing is a bottle of wine for that fancy dinner you're going to have tonight with some friends. She texts you the updated shopping list.

You now have two lists: the original one—whose items are already in your cart—and the updated one, as you can see in figure [7.1](#).

Figure 7.1 The two shopping lists: the original one and the updated one

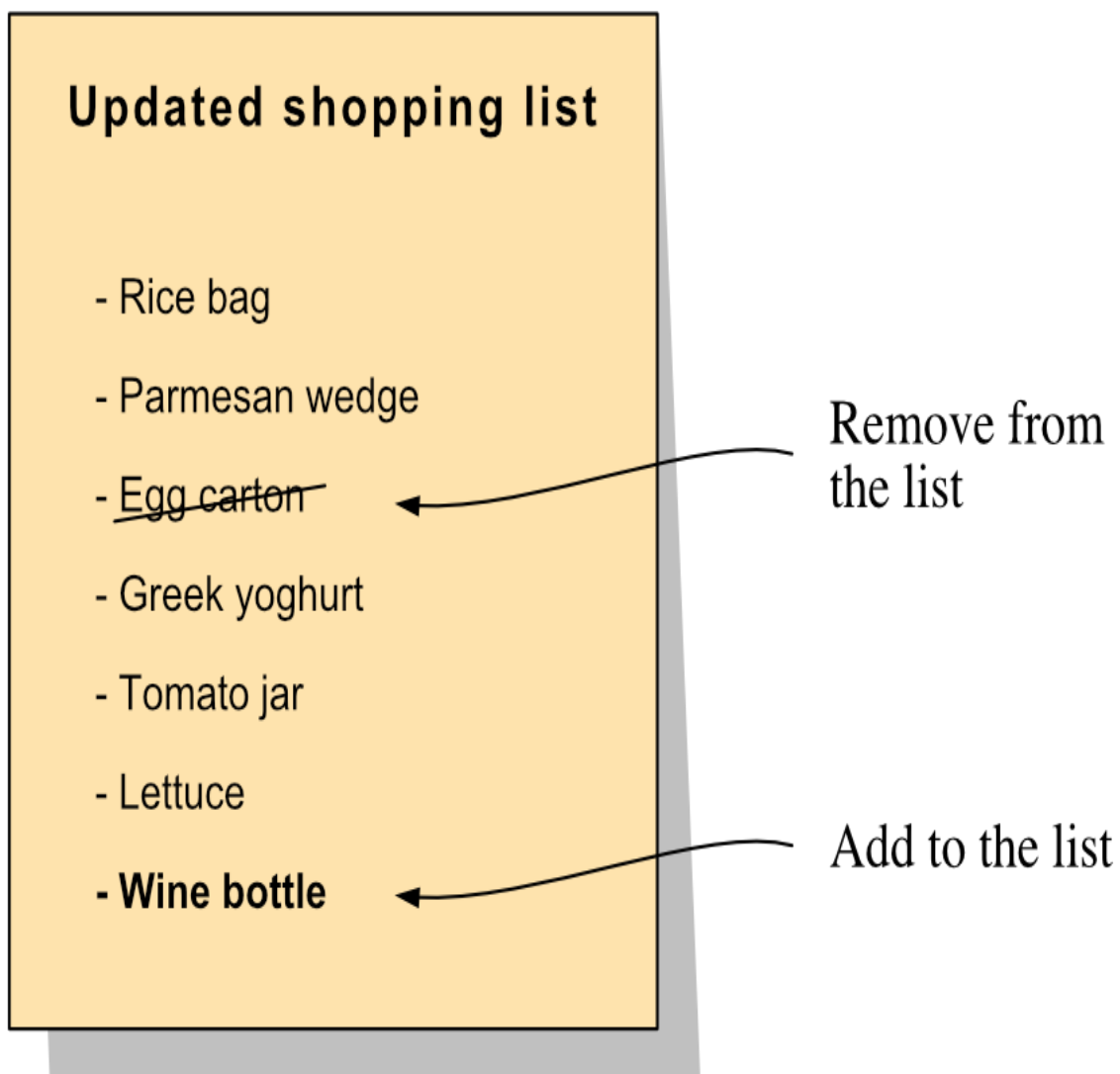


You have two options:

1. You can start over, emptying the cart and putting back the items to the shelves, then picking up the items in the updated list. This way you make sure you have the right items—the ones in the updated list—in your cart.
2. You can pause for a minute, compare the two lists and figure out what items were removed and what items were added. Then you can get rid of the items that were removed from the original list, and pick the items that were added to the updated list.

You're a smart person (after all, you're reading this book, aren't you?), so you choose the second option. It requires some extra brain power, but it's less labor-intensive. It only takes you a few seconds to realize that, if you take the original list, strike out the egg carton, and add the bottle of wine, you'll have the updated list (figure [7.2](#)). You figured out the minimum number of operations you need to have the items in the new list in your cart.

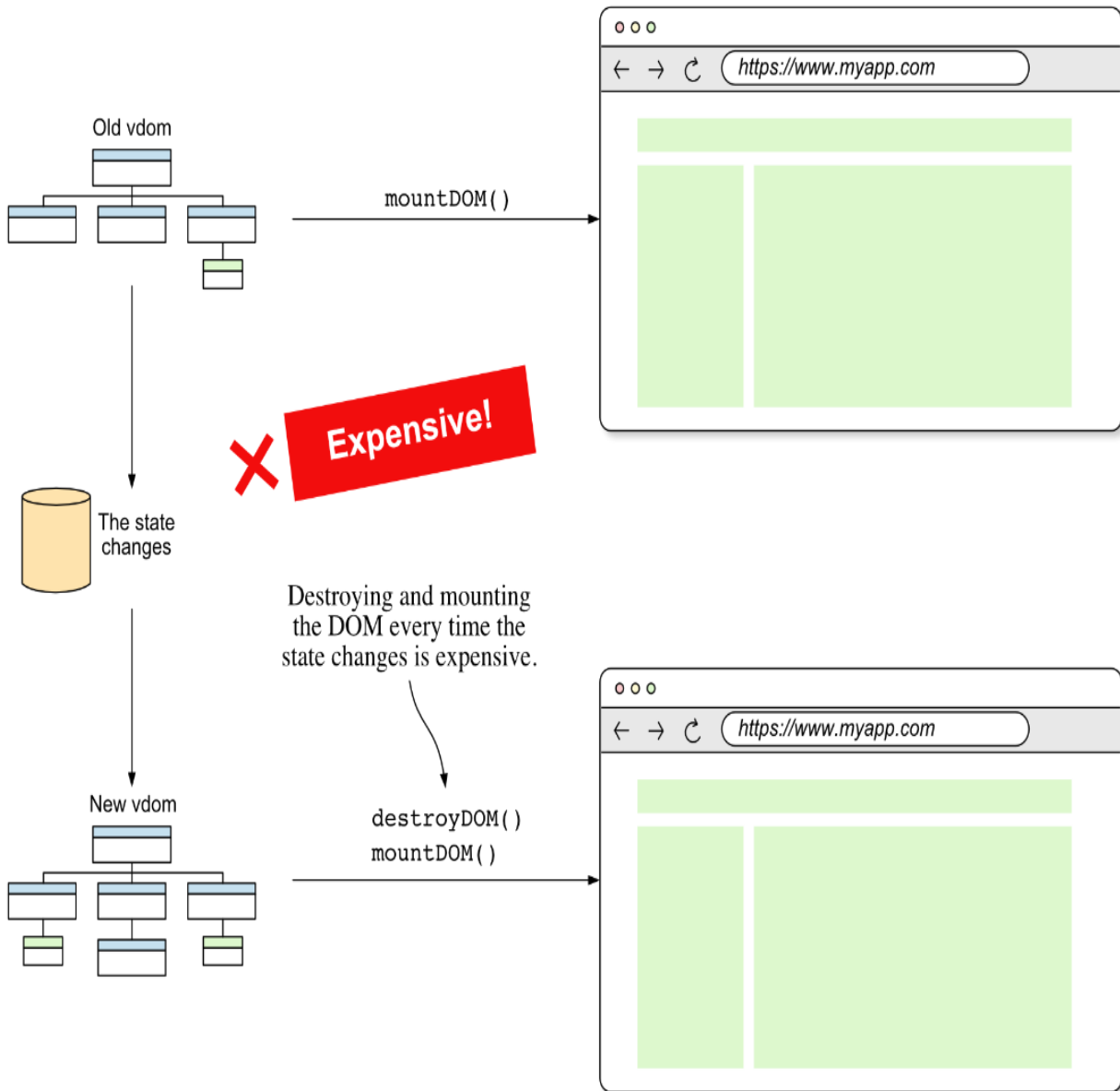
Figure 7.2 The updated shopping list: the original list with the egg carton struck out and the bottle of wine added



This analogy perfectly illustrates what the reconciliation algorithm is about: comparing two virtual DOM trees, figuring out what changes, and applying those changes to the real DOM. Destroying the entire DOM and recreating it from scratch every time the state changes is like emptying the cart and picking the new items from scratch every time there's a new shopping list: you don't need to think, but it's laborious. You want your framework to be smart to figure out the minimum number—or at least a reasonably small number—of operations to apply to the real DOM to have it reflect the new state.

The diagram in figure [7.3](#) shows how your framework is currently rendering the view when the state changes. Destroying and recreating the DOM every time the state changes is an expensive operation. In a frontend application, the state can change a couple times every second, so imagine if your shopping list changed that often and you had to empty your cart and start over every time your partner sent you an updated shopping list.

Figure 7.3 The current framework destroys and recreates the DOM every time the state changes. This is an expensive operation.



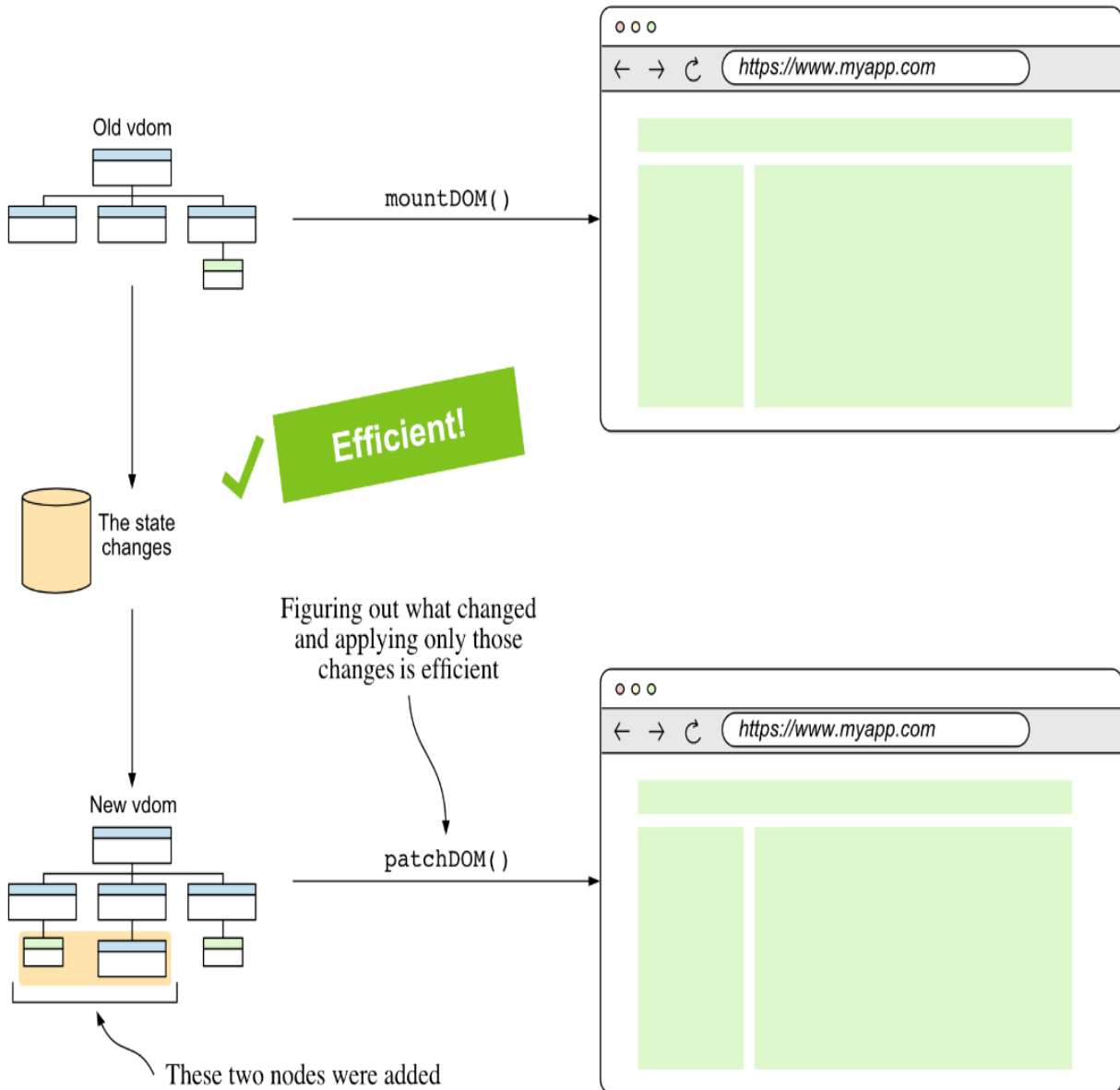
The objective of this chapter is to implement the first half of the *reconciliation algorithm* (you'll implement the rest in the next chapter). This algorithm does two things:

1. It figures out the differences between two virtual DOM trees (this chapter), and
2. It applies those differences to the real DOM, so that the framework can update the DOM in a more efficient way (next chapter).

The same way that when you get a new shopping list, you have to first compare it with the original one, find the differences, and then make the changes to the items in your cart. You have to compare the old virtual DOM with the new virtual DOM, find the differences, and then apply those differences to the real DOM. You'll focus on the first part in this chapter.

In the next chapter, you'll write a `patchDOM()` function implementing the algorithm, putting all the pieces together, and your framework will update the view in a much more efficient way, as you can see in figure [7.4](#).

Figure 7.4 The framework will update the DOM in a more efficient way, using the reconciliation algorithm written in the `patchDOM()` function.



So, what are the mystery ingredients of the reconciliation algorithm?

Important

You can find all the listings in this chapter in the [listings/ch07 directory](#) of the book's repository.

The code you write in this chapter is for the framework's second version, which you'll publish in chapter [8](#). Therefore, the code in this chapter can be checked out from [the ch8 label](#):

```
$ git switch --detach ch8
```

Code catch-up

In the last chapter you rewrote the TODOs application using the framework you built in the previous chapters.

7.1 The three key functions of the reconciliation algorithm

Believe it or not, finding the differences between two virtual DOMs—which is the most complex part of the reconciliation algorithm—can be solved with three functions, as follows:

- A function to find the differences between two objects, returning the keys that were added, the keys that were removed, and the keys whose associated value changed (used to compare the attributes and CSS style objects).
- A function to find the differences between two arrays, returning the items that were added, and the items that were removed (used to compare two CSS classes arrays).
- A function that given two arrays figures out a sequence of operations that, if applied to the first array, transform it into the second array (used to turn the array of a node's children into its new shape).

The two first functions are somehow straightforward, but the third is a bit more complex. In fact, I'd say that it's where 90% of the complexity of the reconciliation algorithm lies. In this chapter, you'll write these functions, which you'll use in the next chapter to implement the `patchDOM()` function.

Let's start by exploring how to figure out what changed from one virtual DOM tree to another, and how to apply those changes to the real DOM.

7.2 Comparing two virtual DOM trees

As we've seen, the view is a function of the state: when the state changes, the virtual DOM representing the view also changes. The reconciliation algorithm

compares the old virtual DOM—the one used to render the current view—with the new virtual DOM after the state changes. Its job is to *reconcile* the two virtual DOM trees, that is, figure out what changed and apply those changes to the real DOM, so that the view reflects the new state.

To compare two virtual DOM trees, you start comparing the two root nodes, checking if they're equal (we'll see what it takes for two nodes to be equal) and if their attributes or event listeners have changed. If you find that the node is not the same, you look no further, destroy the subtree rooted at the old node, and replace it with the new node and everything under it (we'll explore this more in detail later on). You then compare the children of the two nodes, traversing the trees recursively in a depth-first manner. When you compare two arrays of child nodes, you need to figure out if a node was added, removed, or moved to a different position.

Note

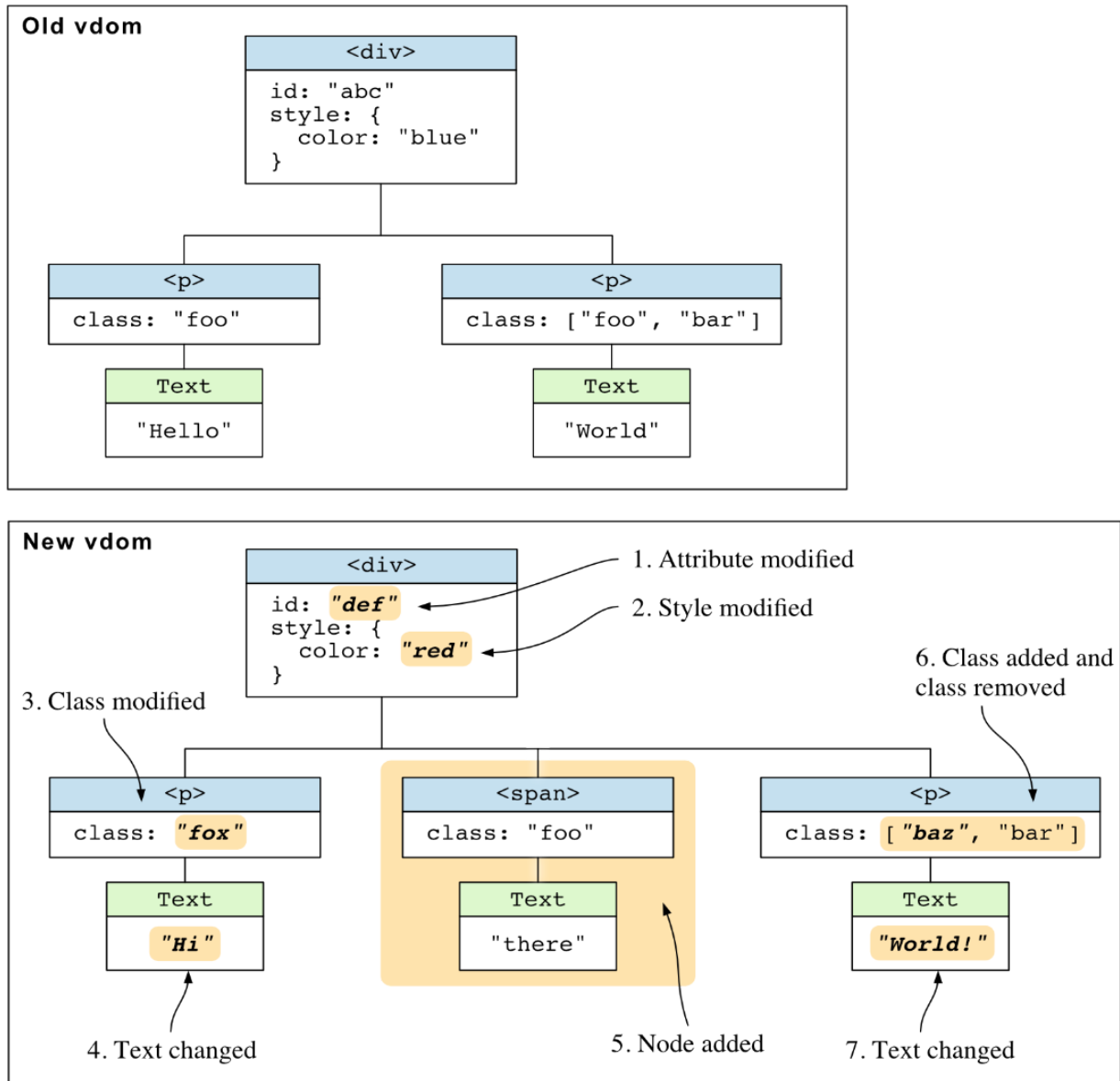
By traversing the tree in a depth-first manner, you ensure that the changes are applied to a complete branch of the tree before moving to the next branch. This is important in the case of fragments, because the children of a fragment are added to the fragment's parent, and if the number of children changes, that could potentially change the indices where the siblings of the fragment's parent are inserted.

In a sense, depth-first traversal is the natural order in which the DOM is modified.

7.2.1 Finding the differences

Let's look at an example. Imagine that you're comparing the two virtual DOM trees in figure 7.5: the old virtual DOM tree at top and the new virtual DOM tree at the bottom. In this illustration the differences have already been highlighted, but we'll explore how to find them step by step.

Figure 7.5 Comparing two virtual DOM trees to find what changed



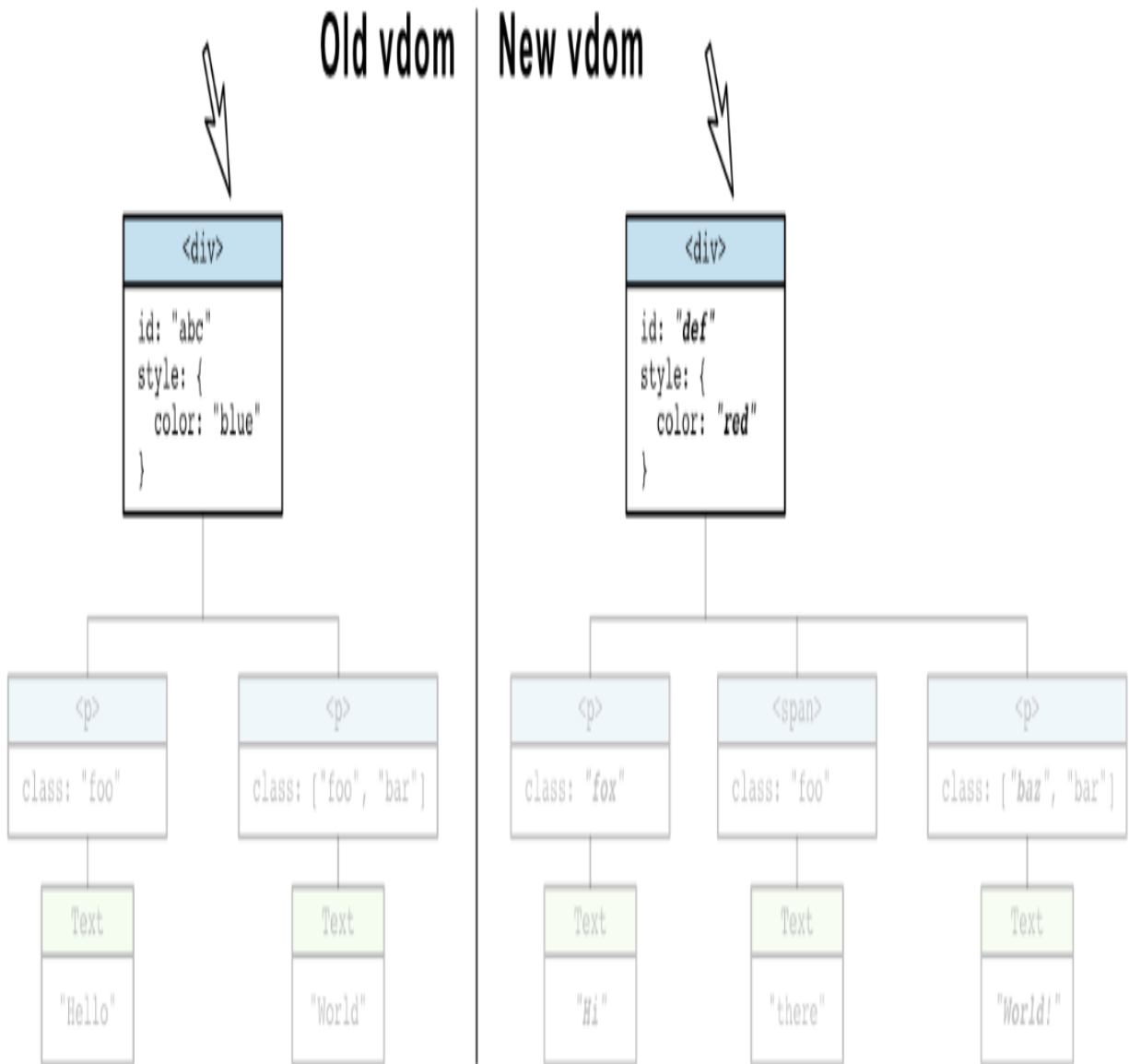
Exercise 7.1

Being able to translate from a virtual DOM diagram to HTML (and viceversa) is useful. Can you write the HTML that both the old and new virtual trees in the example in figure 7.5 would produce?

Find the solution at <http://mng.bz/yZY7>.

Step one. You compare the two root `<div>` nodes (figure [7.6](#)) and find the `id` attribute changed from `"abc"` to `"def"`, and the `style` attribute changed from `"color: blue"` to `"color: red"`. These changes, labelled as "1. Attribute modified" and "2. Style modified" in figure [7.5](#), are the first changes you need to apply to the real DOM, as we'll see below.

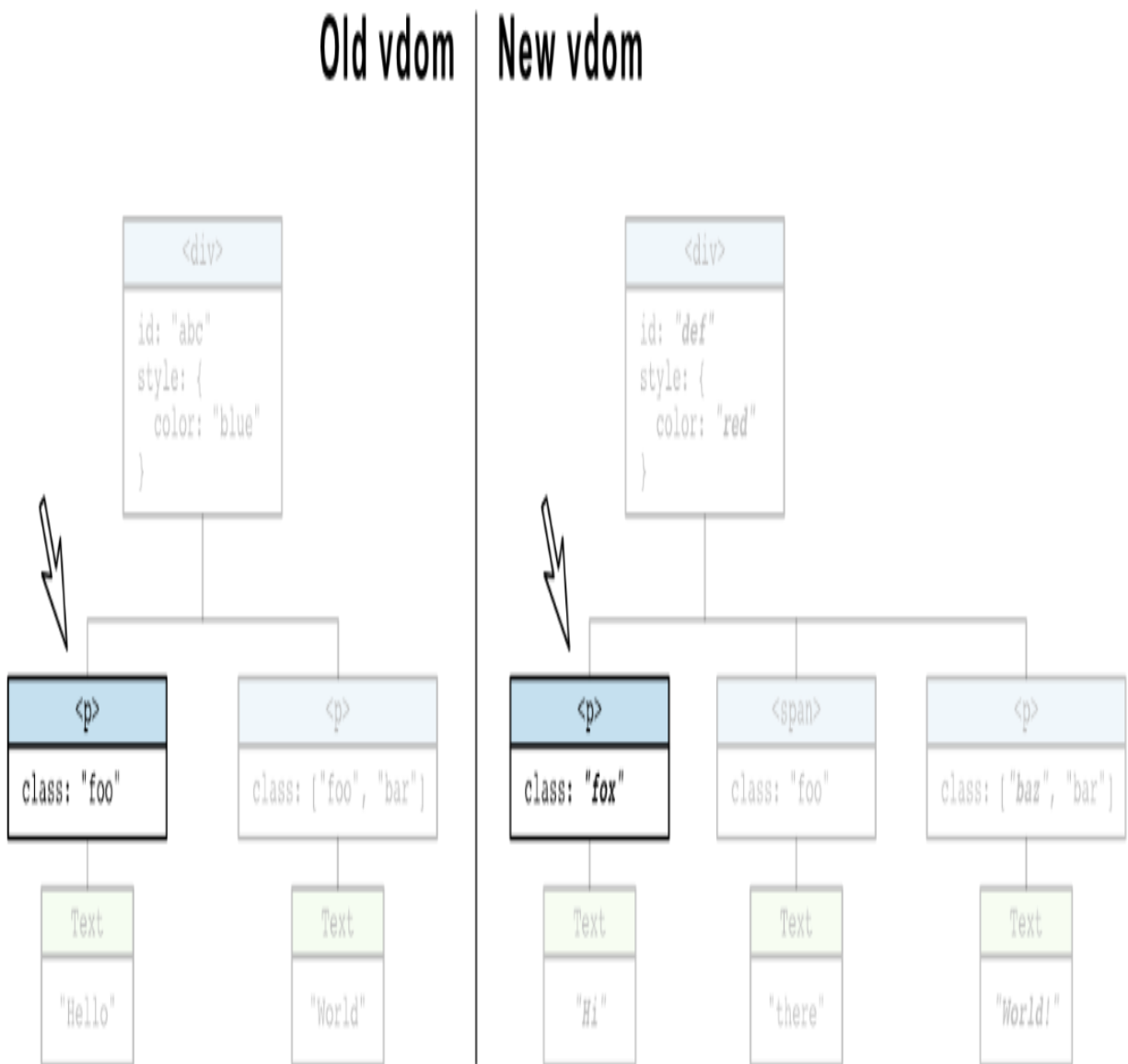
Figure 7.6 Comparing the root `<div>` nodes



Step two. You then compare the two children of the `<div>` node one by one. The first child is a `<p>` element (figure [7.7](#)), and it seems to be in the same

position in both trees—it didn't move. It's `class` attribute has changed from `"foo"` to `"fox"`. This change is labelled as "3. Class modified" in figure [7.5](#), and is the next change to apply to the real DOM.

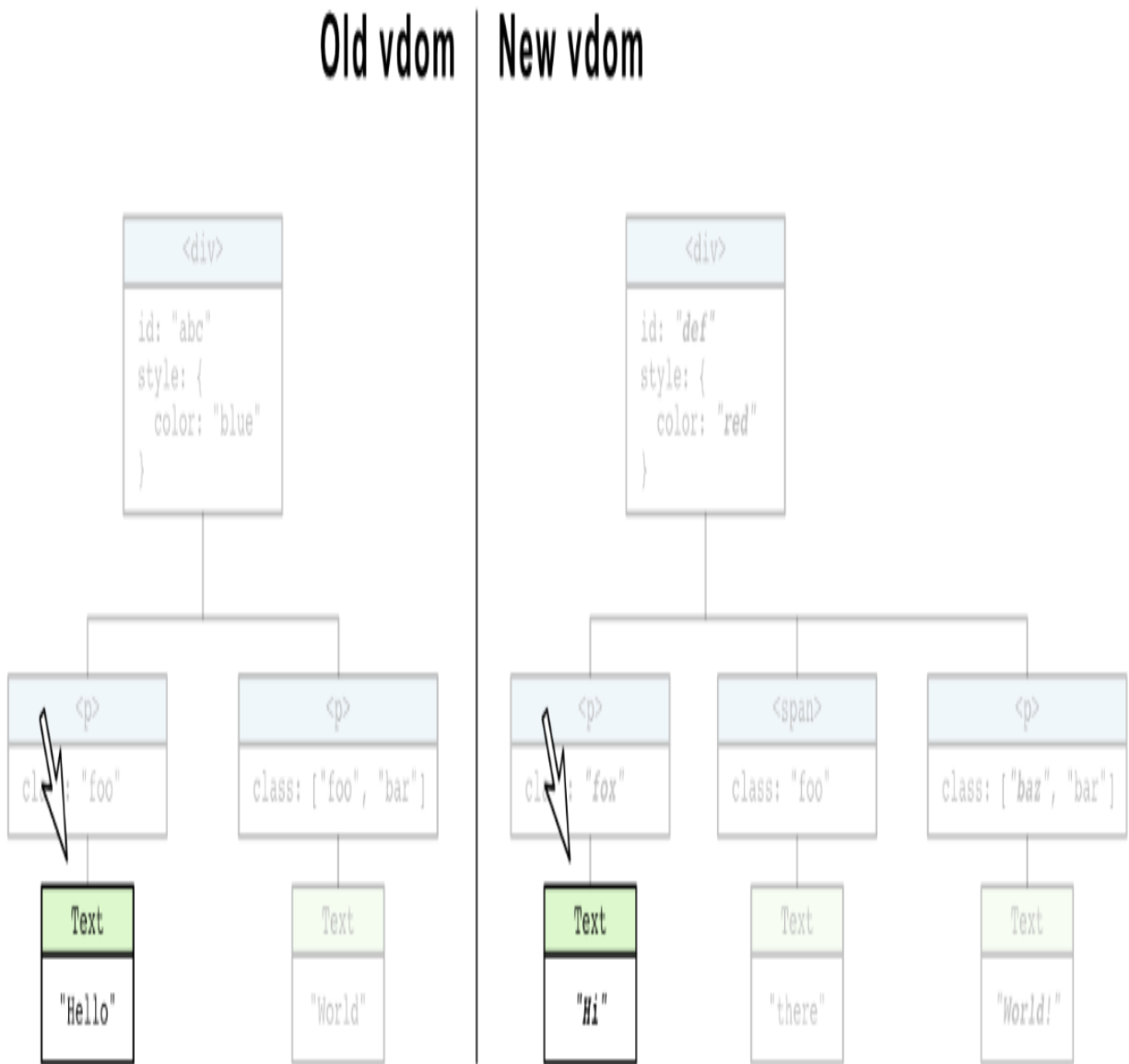
Figure 7.7 Comparing the first child of the `<div>` nodes: a `<p>` element



Step three. Remember that we want to iterate the trees in a depth-first manner, so you'd compare the children of the `<p>` element next: their text content. The text content has changed from `"Hello"` to `"Hi"` (figure [7.8](#)). This

change, labelled as "4. Text changed" in figure [7.5](#), is applied to the real DOM next.

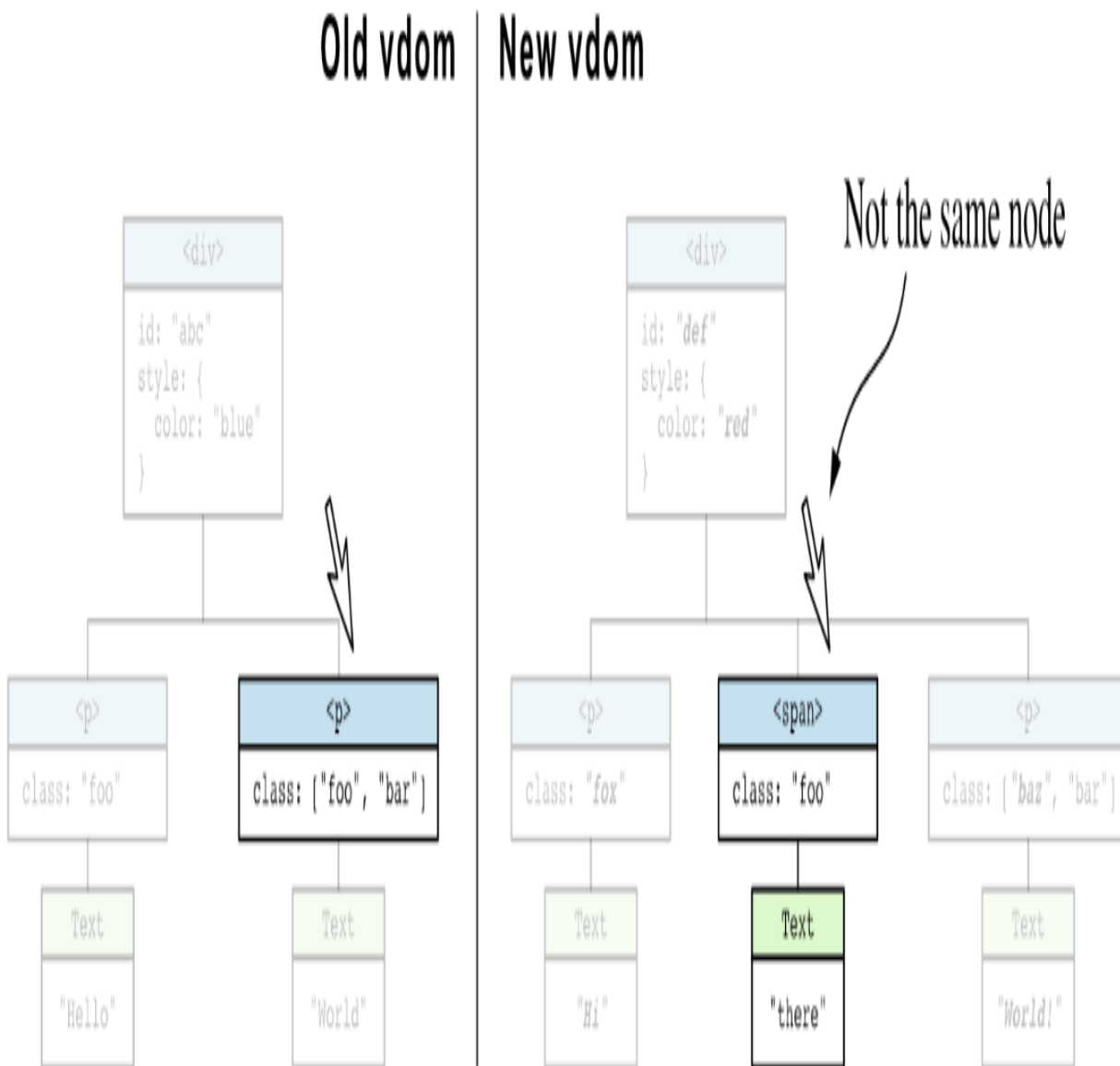
Figure 7.8 Comparing the text nodes of the `<p>` element



Step four. Then you find that the second child is a `<p>` in the old tree, but it's a `<span>` in the new tree (figure [7.9](#)). By looking at the children of the new `<div>` node, you quickly realize that the `<p>` that used to be the second child has moved one position to the right. You conclude that the `<span>` was added, and it naturally moved the `<p>` one position to the right. Adding the new

`<span>` node, including its text node, is the next change to apply to the DOM. This change is labelled as "5. Node added" in figure [7.5](#).

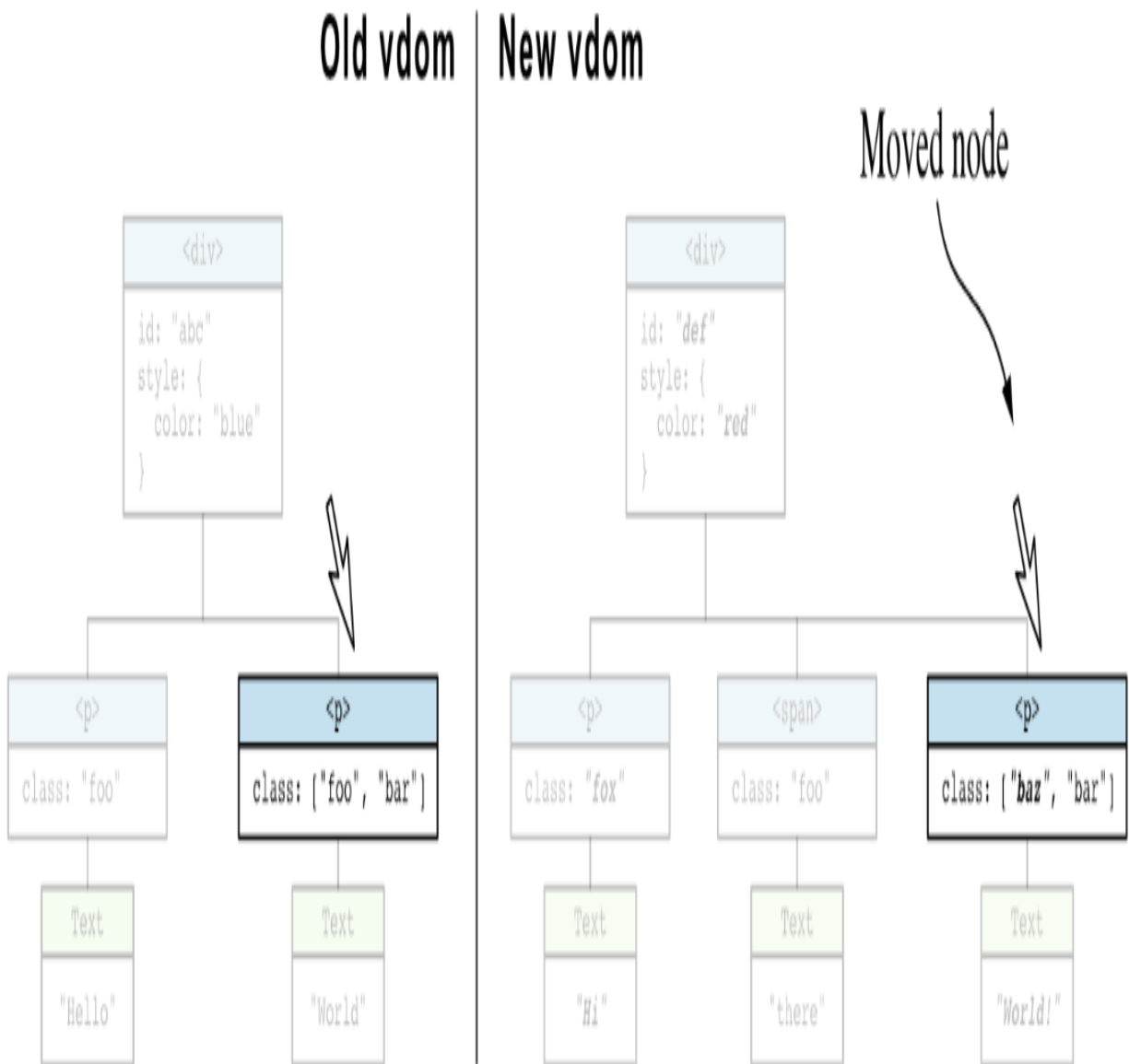
Figure 7.9 Comparing the second child of the `<div>` nodes: a `<p>` element in the old tree, and a `<span>` element in the new tree



Step five. Then you look at the `<p>` node that you know moved one position (figure [7.10](#)), and see that its `class` attribute changed from `["foo", "bar"]` to `["baz", "bar"]`, which means that the class `"foo"` was removed and the

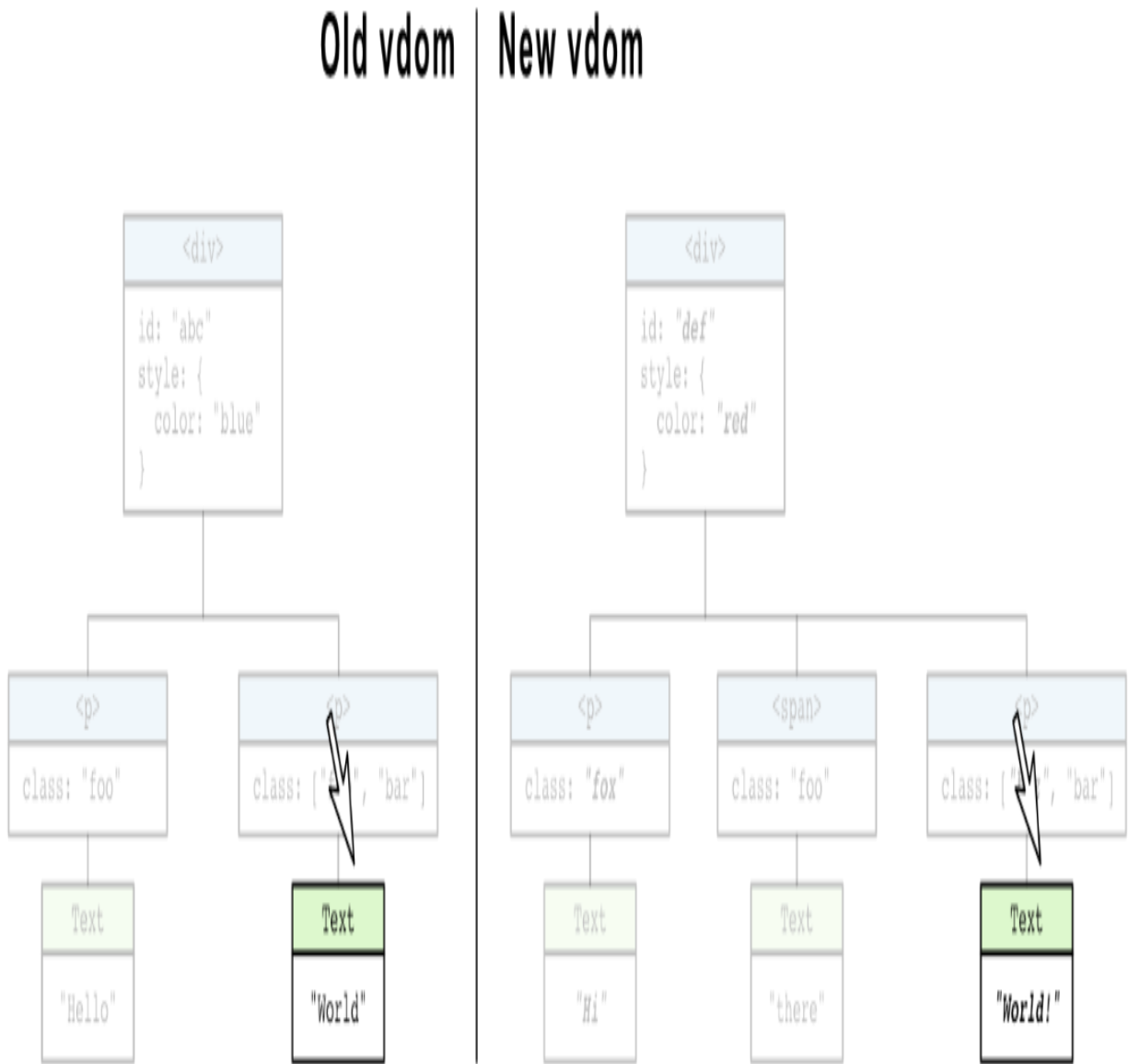
class "baz" was added. This change is labelled as "6. Class added and class removed" in figure 7.5.

Figure 7.10 Comparing the <p> element that moved one position to the right



Step six. Last, you check the children of the <p> element (figure 7.11), and find that the text content changed from "World" to "World!". This change is labelled as "7. Text changed" in figure 7.5.

Figure 7.11 Comparing the text nodes of the <p> element that moved one position to the right



After comparing the two trees, you found a total of seven changes that need to be applied to the real DOM. The next step is to apply them.

Note

Remember that you'll implement this part—modifying the DOM—in the next chapter. But we still want to see how it works for you to have a good high-level understanding of the complete reconciliation process.

Let's see how you do that.

7.2.2 Applying the changes

Let's see how you'd apply these changes in order. Figure [7.12](#) shows the HTML markup of the old virtual DOM tree.

Figure 7.12 The HTML markup of the old virtual DOM tree

```
<div id="abc" style="color: blue;">
  <p class="foo">
    Hello
  </p>
  <p class="foo bar">
    World
  </p>
</div>
```

You'll now apply the changes you identified by comparing the two trees in the previous section. Applying a change means modifying the real DOM to match

the new virtual DOM tree. Every change we make will be shown in the figure, so you can see how the DOM changes with each operation.

Modifying the `id` and `style` attributes of the parent `<div>` node

The first two changes were to the root `<div>` node's attributes, as shown in figure [7.13](#). To apply these changes, you'd need to update the `id` and `style` attributes of the `<div>` node:

```
div.id = 'def'  
div.style.color = 'red'
```

Figure 7.13 Applying the attribute and style changes to the DOM's `<div>` node

1. Change id value

2. Change color value



```
<div id="def" style="color: red;">
  <p class="foo">
    Hello
  </p>
  <p class="foo bar">
    World
  </p>
</div>
```

Modifying the `class` attribute and text content of the first `<p>` node

Then go the changes to the first `<p>` node, as shown in figure [7.14](#). In this case, you'd need to update the `class` attribute and the text content of the `<p>` node:

```
p1.className = 'fox'
p1Text.nodeValue = 'Hi'
```

Figure 7.14 Applying the changes to the first `<p>` node

```
<div id="def" style="color: red;">
```

```
<p class="fox">
```

Hi

```
</p>
```

```
<p class="foo bar">
```

World

```
</p>
```

```
</div>
```

3. Change class value

4. Change text content

Adding the new `<span>` node

The next change is to add the `<span>` node, as shown in figure [7.15](#). To add a new node (and its children) to the DOM, you can pass the virtual node to the `mountDOM()` function you wrote earlier, passing the `<div>` node as the parent node:

```
mountDOM(spanVdom, div)
```

Note that, as is, the `mountDOM()` function would append the `<span>` node to the children array of the `<div>` node, but that's not what we want. We want to specify that the `<span>` node should be inserted at the second position (index 1) of the children array. We'll need to modify the `mountDOM()` function to accept a third argument, the index where the new node should be inserted:

```
mountDOM(spanVdom, div, 1)
```

You will modify the `mountDOM()` function in the next chapter, but for now, you can just assume that it takes an index as the third argument and inserts the new node at that position.

Figure 7.15 Adding the new `<span>` node and its text node

```
<div id="def" style="color: red;">
```

```
  <p class="fox">
```

```
    Hi
```

```
  </p>
```

```
    <span class="foo">
```

```
      there
```

```
    </span>
```

```
  <p class="foo bar">
```

```
    World
```

```
  </p>
```

```
</div>
```

5. Add element node



Modifying the `class` attribute and text content of the second `<p>` node

Last, you reach the second `<p>` node. You don't need to move it because when the `<span>` node was added, it naturally moved the second `<p>` node one position to the right. When we say that it was *naturally* moved, we mean that the movement happened as the result of another operation, but it's not a move operation you need to explicitly do yourself. (Keep this in mind; it'll be important later on.) The `class` attribute changes from `["foo", "bar"]` to

`["baz", "bar"]`, so you need to change the `classList` property of the `<p>` node:

```
p2.classList.remove('foo')  
p2.classList.add('baz')
```

The text content changed from `"World"` to `"World!"`, so you need to update the text node, as follows:

```
p2Text.nodeValue = 'World!'
```

You can see these changes in figure [7.16](#).

Figure 7.16 Applying the changes to the second `<p>` node

```
<div id="def" style="color: red;">
  <p class="fox">
    Hi
  </p>
  <span class="foo">
    there
  </span>
  <p class="baz bar">
    World!
  </p>
</div>
```

6. Change class value

7. Change text content

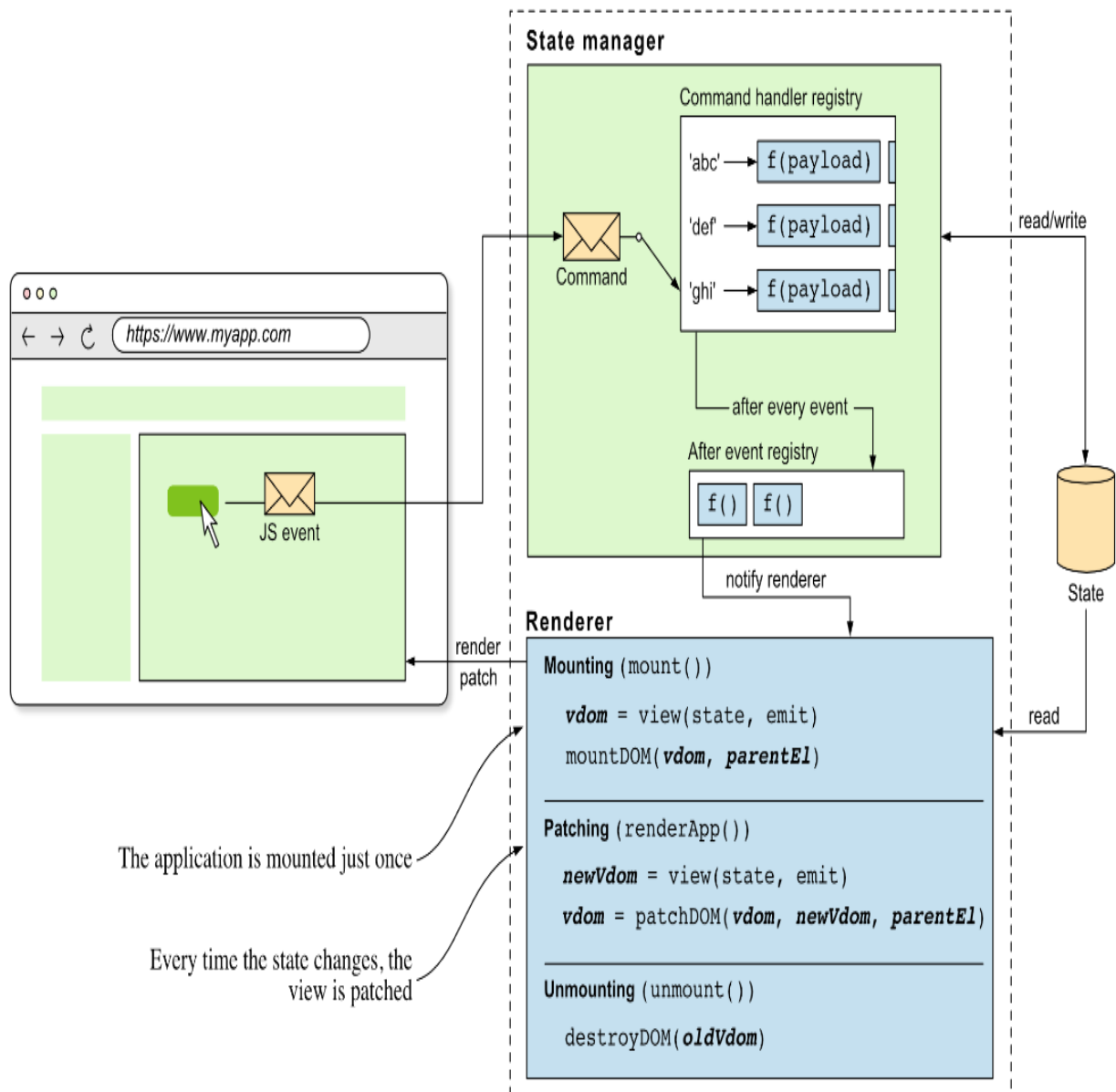
This is what the `patchDOM()` function you'll write in the next chapter does, in a nutshell. To have a better high-level understanding of how the rendering works when the `patchDOM()` function enters the scene, let's start by modifying the `renderApp()` function to call the `patchDOM()` function instead of destroying and recreating the DOM every time.

7.3 Changes in the rendering

To understand the role of the `patchDOM()` function—the implementation of the reconciliation algorithm—it’s helpful to see the big picture of how the rendering works with it. Figure [7.17](#) shows the changes in the rendering mechanism, using the `patchDOM()` function. As you can see, the renderer is now split in three sections: mounting, patching, and unmounting. Whereas in the previous chapter, the same `renderApp()` function was responsible for mounting and updating the DOM, you’ll now split that into the two following functions:

- `mount()`, which internally uses `mountDOM()`
- `renderApp()`, which internally uses `patchDOM()`

Figure 7.17 Changes in the rendering mechanism, using the `patchDOM()` function



You need to modify the code so the `renderApp()` function doesn't completely destroy and recreate the DOM anymore; and this is thanks to the `patchDOM()` function. `patchDOM()` takes the last saved virtual DOM (stored in the `vdom` variable inside the application's instance), the new virtual DOM resulting from calling the `view()` function, and the parent element (stored in the `parentEl` instance) of the DOM, where the view was mounted, and figures out the changes that need to be applied to the DOM.

The application's instance `mount()` method doesn't need to use the `renderApp()` function anymore; `renderApp()` is only called when the state

`changes.mount()` calls the `view()` function to get the virtual DOM, and then calls the `mountDOM()` function to mount the DOM. The user is supposed to call the `mount()` method only once. If they call it more than once, the same application will be mounted multiple times, which is not what we want. You can add a check to prevent this from happening by throwing an error if the `vdom` variable is not `null` (you'll do this as an exercise).

Let's reflect this important change in the code. Open the *app.js* file, and make the changes shown in listing [7.1](#).

Listing 7.1 Changes in the rendering

```
import { destroyDOM } from './destroy-dom'
import { Dispatcher } from './dispatcher'
import { mountDOM } from './mount-dom'
import { patchDOM } from './patch-dom'

export function createApp({ state, view, reducers = {} }) {
  let parentEl = null
  let vdom = null

  // --snip-- //

  function renderApp() {
    if (vdom) {
      destroyDOM(vdom)
    }
    const newVdom = view(state, emit) #1
    mountDOM(vdom, parentEl)
    vdom = patchDOM(vdom, newVdom, parentEl) #2
  }

  return {
    mount(_parentEl) {
      parentEl = _parentEl
      renderApp()
      vdom = view(state, emit)
      mountDOM(vdom, parentEl) #3
    },

    // --snip-- //
  }
}
```

Warning

You're importing the `patchDOM()` from the *patch-dom.js* file, which you'll write in the next chapter (your IDE might complain that such file doesn't exist, but that's okay for now).

Now that you've changed the rendering flow, let's start thinking about the three functions that you'll use to compare two virtual DOM trees:

`objectsDiff()`, `arraysDiff()`, and `arraysDiffSequence()`.

Exercise 7.2

Implement a check that doesn't allow the user to call the `mount()` method more than once to prevent the same application from being mounted multiple times. If you detect that the application was already mounted, throw an error.

Find the solution at <http://mng.bz/M9XE>.

7.4 Diffing objects

When comparing two virtual nodes, you need to find the differences between the attributes of the two nodes (the `props` object) to patch the DOM accordingly. You want to know what attributes were added, what attributes were removed, and what attributes changed.

To illustrate this let's look at the example in figure [7.18](#), the attributes of an `<input>` element. In the example, there's an old object (the one on top) and a new object (the one on the bottom). By observing the two objects, you can see that the `min` key was added in the new object, the `max` key was removed, and the `disabled` key changed from `false` to `true`.

Figure 7.18 Finding the difference between two objects: what keys were added, removed, or changed?

Old object

```
{  
  type: 'number',  
  disabled: false,  
  max: 999,  
}
```

New object

```
{  
  type: 'number',  
  disabled: true,  
  min: 0,  
}
```

Added
min

Removed
max

Modified
disabled

This exercise that you've done by mere observation is what the `objectsDiff()` function that you'll implement now does. The code you write works very similar to your observation, described in the following list:

1. Take a key in the old object. If you don't see it in the new object, you know that the key was removed. Repeat with all keys.
2. Take a key in the new object. If you don't see it in the old object, you know that the key was added. Repeat with all keys.
3. Take a key in the new object. If you see it in the old object, and the value associated with the key is different, you know that the value associated

with the key changed.

Create a new file inside the *utils* folder called *objects.js*, and write the `objectsDiff()` function as shown in listing [7.2](#).

Listing 7.2 Finding the difference between two objects (*utils/objects.js*)

```
export function objectsDiff(oldObj, newObj) {
  const oldKeys = Object.keys(oldObj)
  const newKeys = Object.keys(newObj)

  return {
    added: newKeys.filter((key) => !(key in oldObj)), #1
    removed: oldKeys.filter((key) => !(key in newObj)), #2
    updated: newKeys.filter(
      (key) => key in oldObj && oldObj[key] !== newObj[key]
#3
    ),
  }
}
```

Note

In the code for the `objectsDiff()` function, I’m iterating the `newKeys` set twice. I could’ve avoided it and just perform a single iteration in a `for` loop, where I save the added and updated keys. This would’ve been more efficient, but I chose to use two separate iterations—one for the added keys and another for the updated keys—to make the code more readable.

As I mentioned in Appendix A, I’m favoring readability and code simplicity over performance. The goal of the code is for you—the reader—to learn how to build a frontend framework; it isn’t being super performant. But you should feel free to optimize your code if you want.

Fantastic—you’ve implemented the `objectsDiff()` function. Can you think of a set of unit tests that would help you verify that the function works as expected?

Exercise 7.3

Implement a set of test cases to cover all the possible scenarios for the `objectsDiff()` function. If you're not familiar with unit testing, try to think of the different cases your function should handle, and execute the function in the Browser's console with different inputs to see if it works as expected.

Find the solution at <http://mng.bz/amox>.

Let's now look at the second function that you'll implement, one that does a similar job for arrays.

7.5 Diffing arrays

Remember that the classes in an element virtual node can be given in the form of an array:

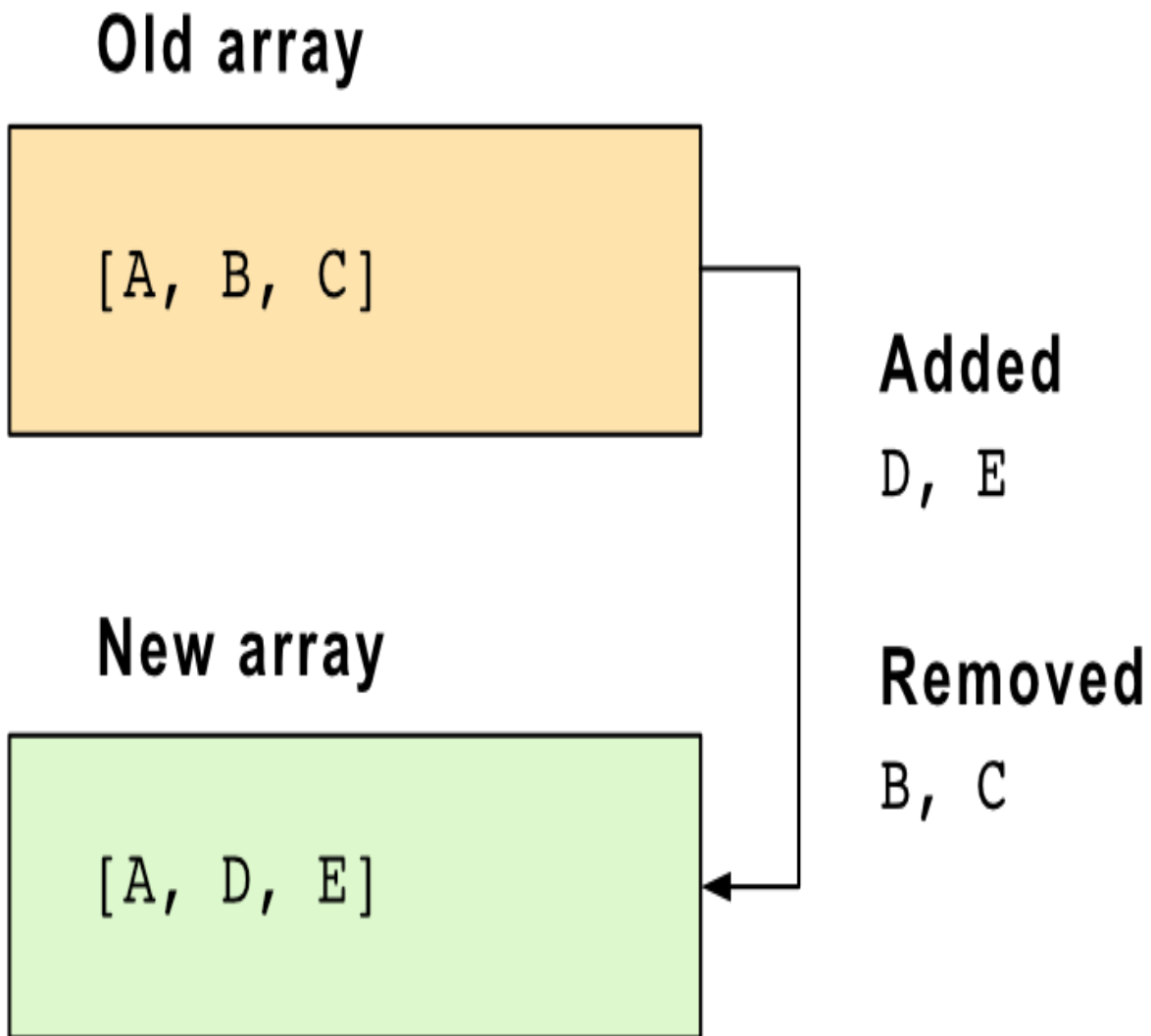
```
h('p', { class: ['foo', 'bar'] }, ['Hello world'])
```

When you compare two virtual nodes, you need to find the differences between the classes of the two nodes (the `class` array) to patch the DOM accordingly (as you did in the example of figure 7.5, in step 5). In this case, it doesn't matter if the items in the array are in a different order, we care about only the items that were added or removed. For this, you'll implement the `arraysDiff()` function next. But first, let's look at the example in figure 7.19.

In the example, there's an old array (the one on top) and a new array (the one on the bottom). When we compare the two arrays, we see the following:

- A appears in both arrays
- B and C were removed (only appear in the old array)
- D and E were added (only appear in the new array)

Figure 7.19 Finding the difference between two arrays: what items were added or removed?



Note that in this comparison, items are either added or removed, but they are never changed. If an item is changed, we detect it as a removal of the old item and an addition of the new item.

With this in mind, open the *utils/arrays.js* file (where you previously implemented the `withoutNulls()` function) and write the `arraysDiff()` function as shown in listing [7.3](#).

Listing 7.3 Finding the differences between two arrays (*utils/arrays.js*)

```
export function arraysDiff(oldArray, newArray) {
  return {
    added: newArray.filter(
      (newItem) => !oldArray.includes(newItem) #1
    ),
    removed: oldArray.filter(
      (oldItem) => !newArray.includes(oldItem) #2
    ),
  }
}
```

Warning

In the `arraysDiff()` function, you're not specifying at what index an item was added or removed, because to modify the `classList` of an element, you'll simply add or remove the classes. This might be problematic because, as you know, the order of classes in the `classList` matters. A class that comes later in the list can override the styles of a class that comes earlier in the list. This is a trade-off that you're making to keep the code simple, but bear in mind that a more robust solution is to maintain the order of classes in the `classList`.

With these two out of the way, you're ready to tackle the beast: the `arrayDiffSequence()` function. That's the function where we might sweat a bit. But once you've written that function, the rest of the book will feel like a breeze (or something like that). Take a deep breath, and let's move on.

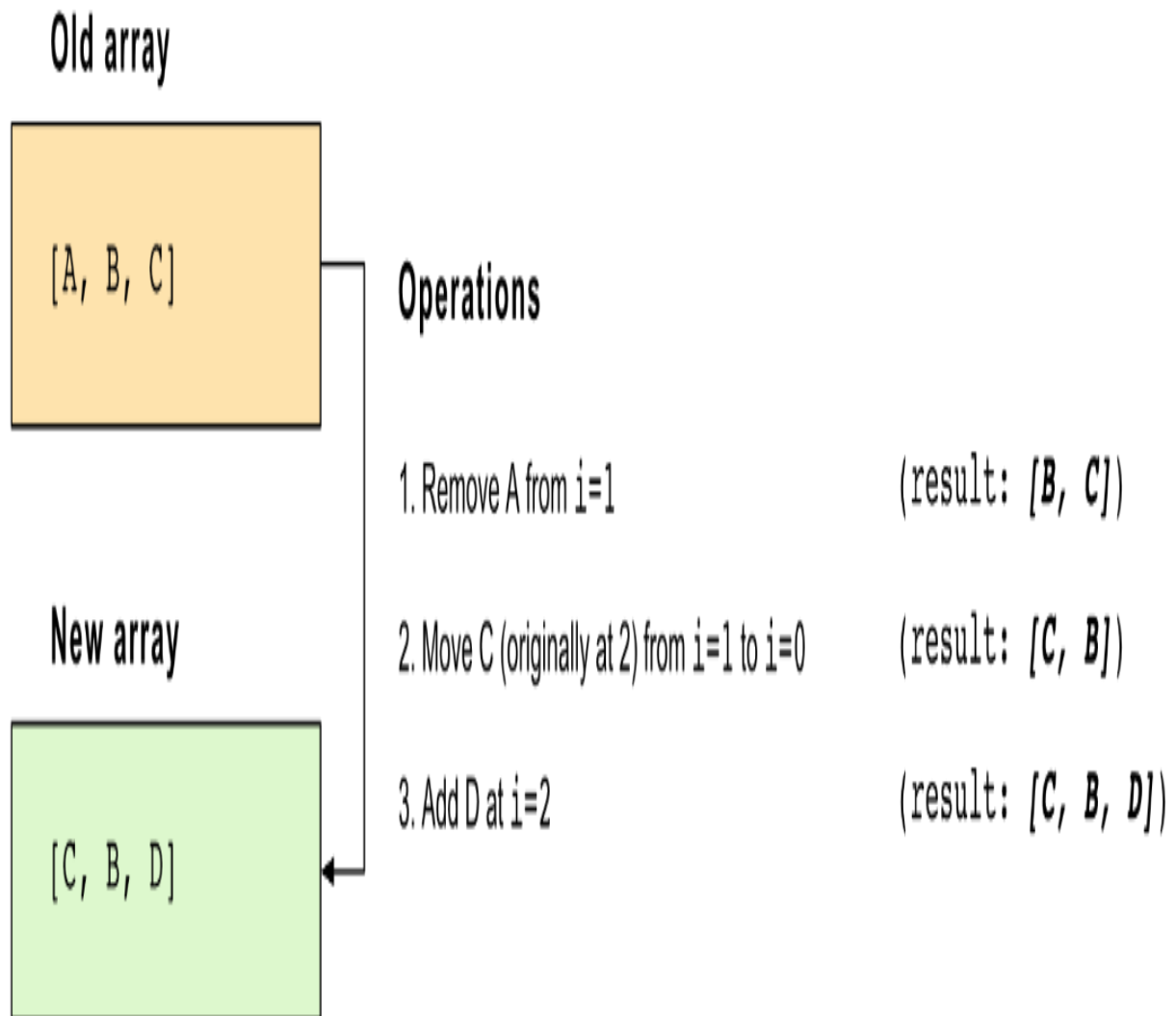
7.6 Diffing arrays as a sequence of operations

A virtual node has children, and those children can move around from a render to the next one. Child nodes are added and removed all the time as well. When you compare two virtual nodes, you need to find the differences between the children of the two nodes (the `children` array) to patch the DOM accordingly. You want to find a sequence of operations that, executed on the DOM, transform the old children array—rendered as HTML elements—into the new children array.

If you're given two arrays and I ask you to come up with a list of *add*, *remove*, and *move* operations that transform the first array into the second one, how

would you go about it? You can see an example in figure [7.20](#). In this example, the old array is [A, B, C] and the new array is [C, B, D], and I was able to find a sequence of three operations that, if applied in order, transform the old array into the new array.

Figure 7.20 Finding a sequence of operations that transform the original array into the new one



Important

Notice how the move operation's `from` index is different from the original index that `c` had in the old array? `c` appeared at `i=2`, but after removing `A`, it moved to `i=1`. The indices in the operations are always relative to how they find things in the array at the moment of the operation. But you still want to keep track of the original index of the item, and I'll explain why in a moment.

There are many other sequences of operations (infinite, in fact) that would yield the same result. In other words, the sequence of operations that transforms the old array into the new array is not unique.

Exercise 7.4

Can you find another sequence of similar operations that would also transform the old array into the new array?

Find the solution at <http://mng.bz/g7oZ>.

One constraint that we can impose on the sequence is that we want to minimize the number of operations. (You could come up with a sequence that starts with moving an item from its position to another one, and then moving it back to its original position, and repeat that a lot of times, but that's not what we want.) DOM modifications are relatively expensive, so the fewer operations we perform, the better.

7.6.1 Defining the operations you can use

Let's start by defining the operations that you'll use to transform the old array into the new array. In the example before we used three operations: *add*, *remove*, and *move*. You actually need to add a fourth operation: *noop* (no operation).

You'll use a *noop* operation when you find an item that's in both arrays, which requires no action to stay where it *ends up*. Notice how I said *ends up* and not *starts at*; this distinction is important.

There are items that *move naturally* to their final position, because items are added, removed or moved around them. For example, if we want to go from

[A, B] to [B], we just need to remove A. B falls into place naturally: without any explicit move operation it goes from `i=1` to `i=0`.

When an item moves naturally, you need to include a noop operation in the sequence. This helps you keep track of the items that moved naturally, from where they started to where they ended up. You need this operation because patching the DOM is a recursive process: each of the items in the children array that moved around also need to be compared to look for differences. For this to be possible, you need to know where each item started and where it ended up, in other words, their initial and final indexes in the array. This will become much clearer in the next chapter.

The following list is the operations and the data they need for them to be completely characterized:

- *add*: { op: 'add', item: 'D', index: 2 }
- *remove*: { op: 'remove', item: 'B', index: 0 }
- *move*: { op: 'move', item: 'C', originalIndex: 2, from: 2, index: 0 }
- *noop*: { op: 'noop', item: 'A', originalIndex: 0, index: 1 }

As you can see, in all cases, the object that describes the operation has an `op` property that indicates the type of operation. The `item` property is the item that's being added, removed, or moved (naturally or forcefully). The `index` property is the index where the item ends up, except in the case of a remove operation, where it's the index where the item was removed from. For the move and noop operations, the `originalIndex` property indicates the index where the item started. Last, for the move operation, we also keep track of the `from` property, which is the index where the item was moved from. Note that in the case of a move operation, the `from` and `originalIndices` need not be the same.

Following the example in figure [7.20](#), the sequence of operations that transforms the old array into the new array is:

```
[
  {op: 'remove', index: 0, item: 'A'}
  {op: 'move', originalIndex: 2, from: 1, index: 0, item:
```

```
'C'}  
  {op: 'noop', index: 1, originalIndex: 1, item: 'B'}  
  {op: 'add', index: 2, item: 'D'}  
]
```

Exercise 7.5

Apply the operations in the previous sequence to the old array (`[A, B, C]`), and check that you end up with the new array (`[C, B, D]`). You'll likely be doing this by hand a couple times this chapter, to debug the code you'll write.

Find the solution at <http://mng.bz/eEMw>.

Let's now talk about how the algorithm to find the sequence of operations works.

7.6.2 Finding the sequence of operations—the algorithm

The idea is to iterate over the indices of the new array, and for each step, find the way of transforming the old array so that at the current index, the items in both arrays are the same. You focus on one item at a time, so after each step, that item and all the previous ones are in their final position. The algorithm modifies the old array (a copy of it, to avoid mutating the original one) as it goes, and it keeps track of the operations that transform the old array into the new one.

Going one item at a time and modifying a copy of the old array ensures that every operation is performed over the updated indices of the old array. When the new array is completely iterated over, any excess items in the old array are removed; the new array doesn't have them, so they're not needed.

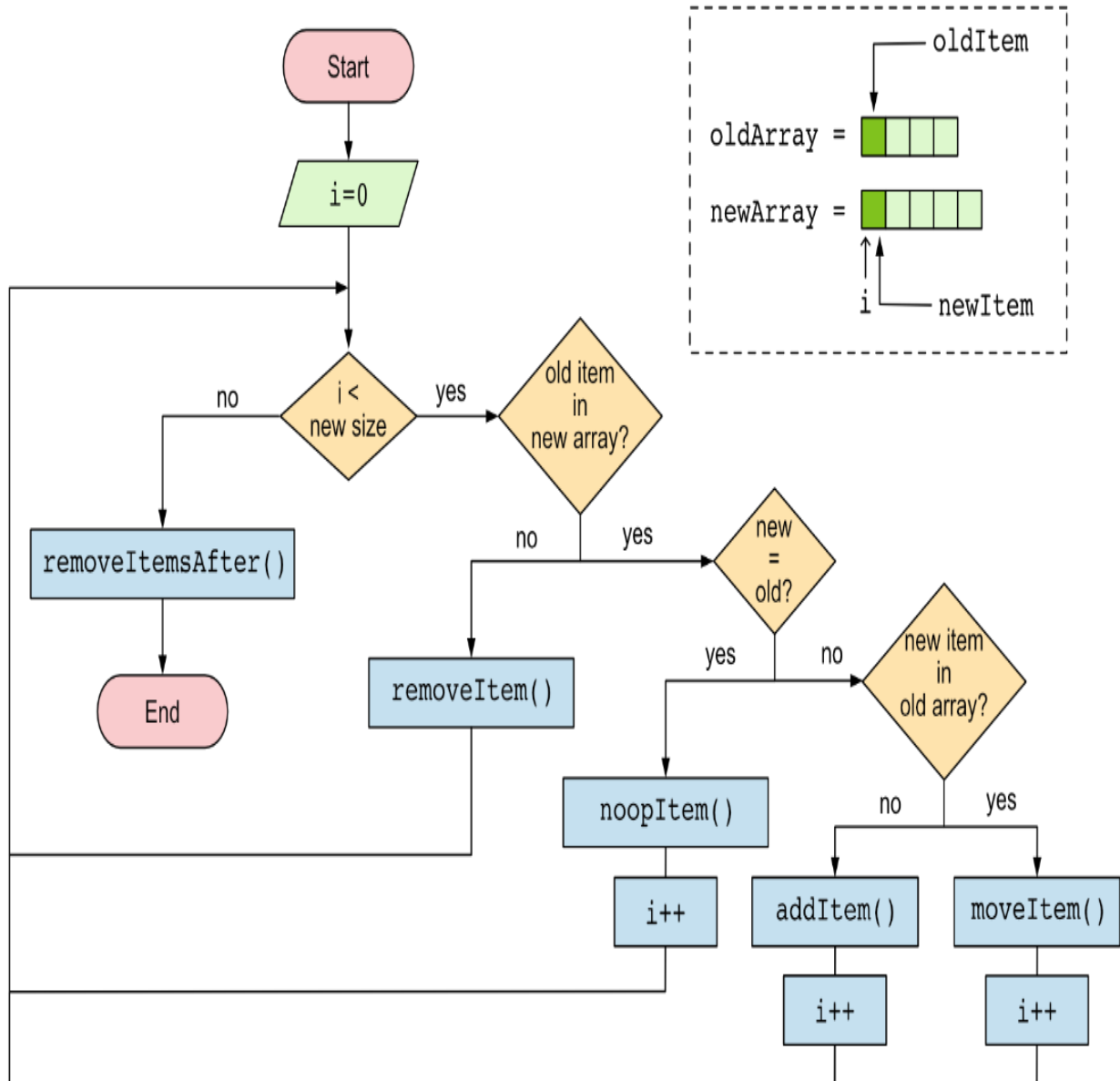
The algorithm is as follows:

1. Iterate over the indices of the new array
 - Let `i` be the index ($0 \leq i < \text{newArray.length}$)
 - Let `newItem` be the item at `i` in the new array
 - Let `oldItem` be the item at `i` in the old array (provided there's one)
2. If `oldItem` doesn't appear in the new array:

- add a remove operation to the sequence,
 - remove the `oldItem` from the array, and
 - start again from step 1 without incrementing `i` (stay at the same index)
3. If `newItem == oldItem`:
 - add a noop operation to the sequence, using the `oldItem` original index (its index at the beginning of the process)
 - start again from step 1 incrementing `i`
 4. If `newItem != oldItem` and `newItem` can't be found in the old array starting at `i`:
 - add an add operation to the sequence
 - add the `newItem` to the old array at `i`
 - start again from step 1 incrementing `i`
 5. If `newItem != oldItem` and `newItem` can be found in the old array starting at `i`:
 - add a move operation to the sequence, using the `oldItem` current index and the original index
 - move the `oldItem` to `i`
 - start again from step 1 incrementing `i`
 6. If `i` is greater than the length of `newArray`:
 - add a remove operation for each of the remaining items in `oldArray`
 - remove all the remaining items in `oldArray`
 - stop the algorithm

This algorithm isn't straightforward, but it's simpler than it appears at first sight. Figure [7.21](#) shows a flow chart of the algorithm with the same steps as the list above. I'll use it later when you have to implement the algorithm in code, to show you exactly where you are in the process. It's a complex algorithm, so I want to make sure you don't get lost.

Figure 7.21 The algorithm's flow chart



Let's work through an example to see how the algorithm works.

7.6.3 An example by hand

Let's say you have the following arrays:

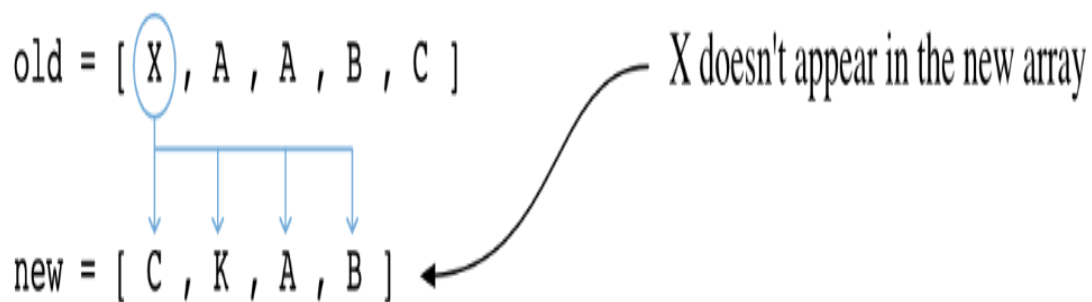
- `oldArray = [X, A, A, B, C]`
- `newArray = [C, K, A, B]`

Let's apply the algorithm step by step.

Step one ($i=0$). The x in the old array doesn't appear in the new array, so you want to remove it (figure [7.22](#)).

Figure 7.22 At $i=0$ in the old array is x , which doesn't appear in the new array: it's a remove operation.

$i=0$



OPERATION: Remove X at $i=0$

old = [~~X~~ , A , A , B , C]

new = [C , K , A , B]

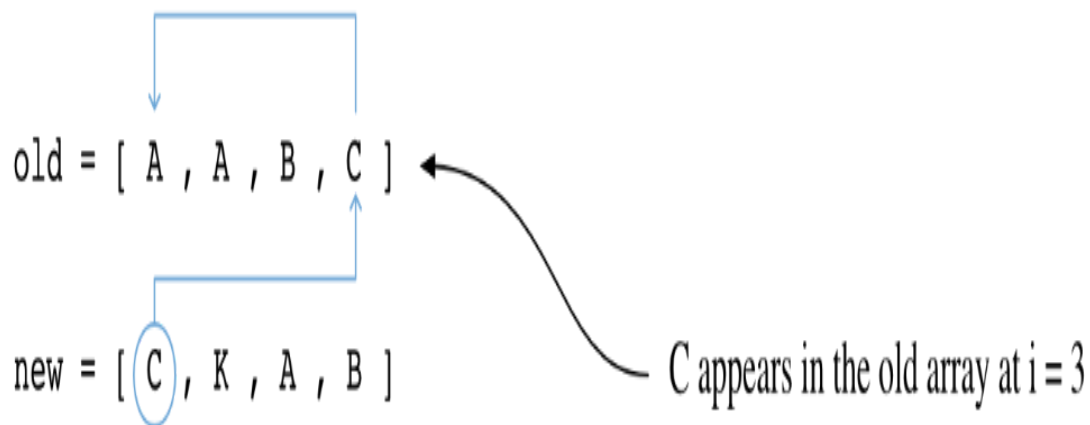
After removing the item and adding the remove operation to the list of operations, you keep the index at $i=0$ because you haven't fulfilled the condition of having the same item in both arrays at that index.

Let's move on to the next step.

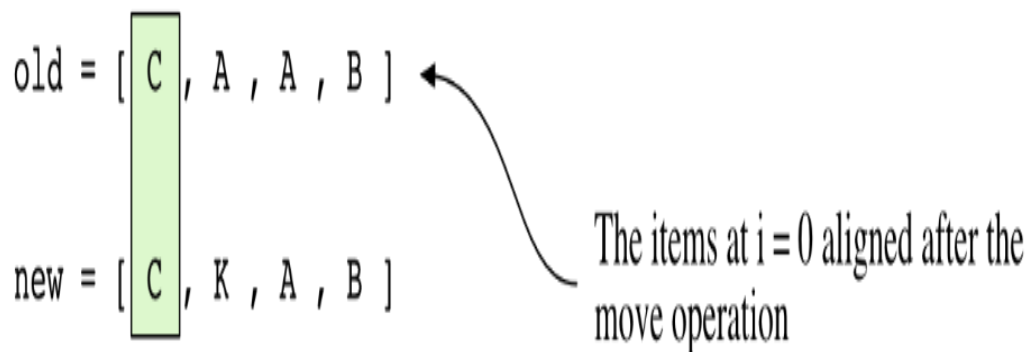
Step two ($i=0$). You find a `c` in the new array, but there's an `A` in the old array at that same index. Let's look for the `c` in the old array, starting at $i=0$. Oh! There's one at $i=3$ (figure 7.23). Note that, if there were more than one `c` in the old array, you'd choose the first one you find.

Figure 7.23 At $i=0$ there's a `c` in the new array. We can move the `c` in the old array from $i=4$.

$i=0$



OPERATION: Move `C` (originally at 4) from $i=3$ to $i=0$

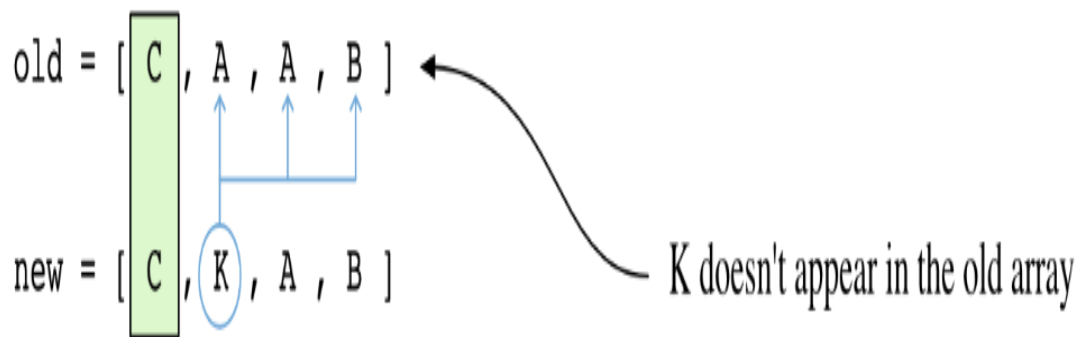


You add the move operation to the sequence of operations, and move the c from $i=3$ to $i=0$ in the old array. On to the next step!

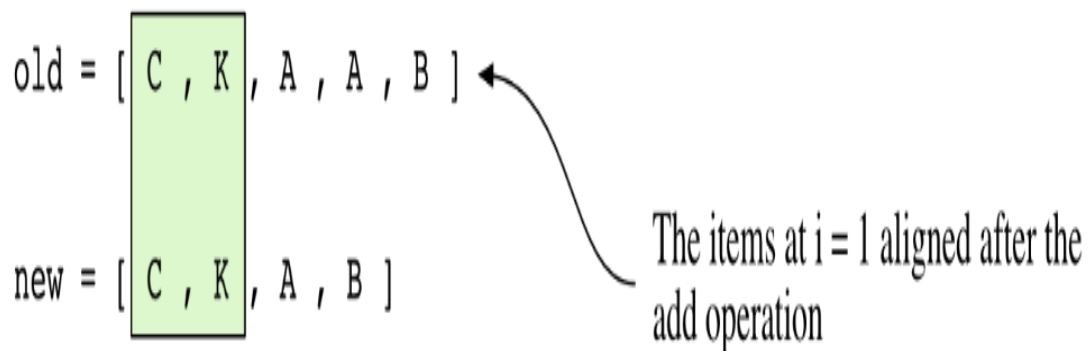
Step three ($i=1$). There's a κ in the new array, but an A in the old array. You look for a κ in the old array, starting at $i=1$. In this case there isn't one, so you need an add it (figure [7.24](#)).

Figure 7.24 At $i=0$ is a κ in the new array, but there's no κ in the old array. We need an add operation.

$i=1$



OPERATION: Add K at $i=1$



You add the κ to the old array at $i=1$, and append the operation to the sequence of operations. Let's move on.

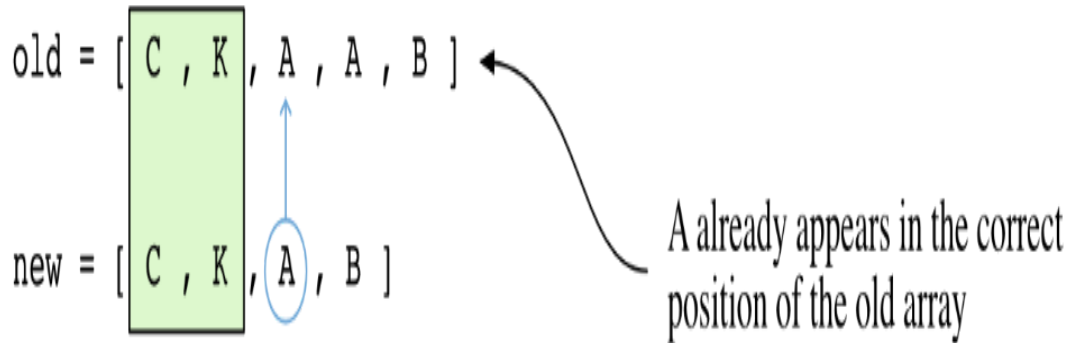
Step four ($i=2$). At $i=2$ both arrays have an A — hooray! (figure [7.25](#)). You add a noop operation in these cases. For that, you need the original index of A in the old array. But that A moved naturally due to the previous operations:

- It moved one position to the left when the x was removed in step one.
- It moved one position to the right when the c was moved in step two, which cancelled the previous move.
- It moved one position to the right when the κ was added in step three.

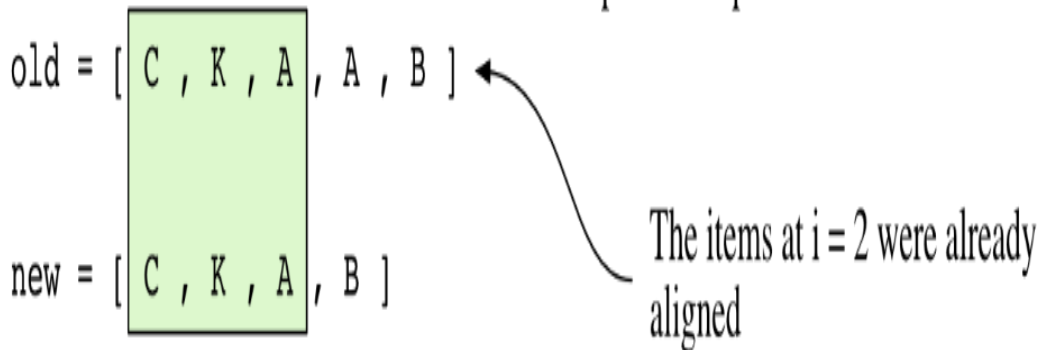
In total, the A moved one position to the right, so you need to subtract one from the current index of the A in the old array. So, the original index of the A is $i=1$.

Figure 7.25 At $i=2$ both arrays have an A . We add a noop operation, using the original index of the A in the old array.

$i=2$



OPERATION: Noop A (originally at 1) to $i=2$ ← A was originally at $i = 1$, but it moved due to the previous operations

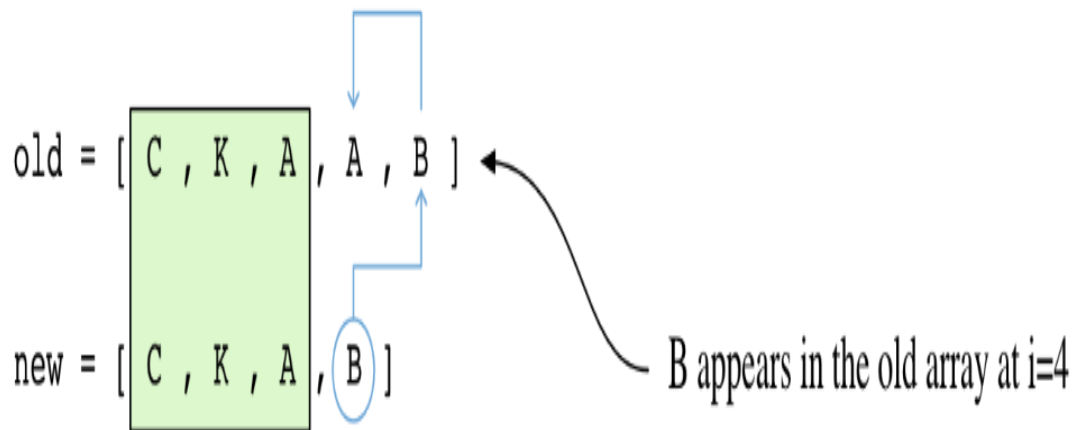


You want to keep track of the old array items original indexes, so you have them available when you add noop and move operations. As you can imagine, calculating the original index of an item based on the previous operations isn't complicated, but it's much better if you just save it at the beginning of the process—keep this in mind. Let's move on to the next step.

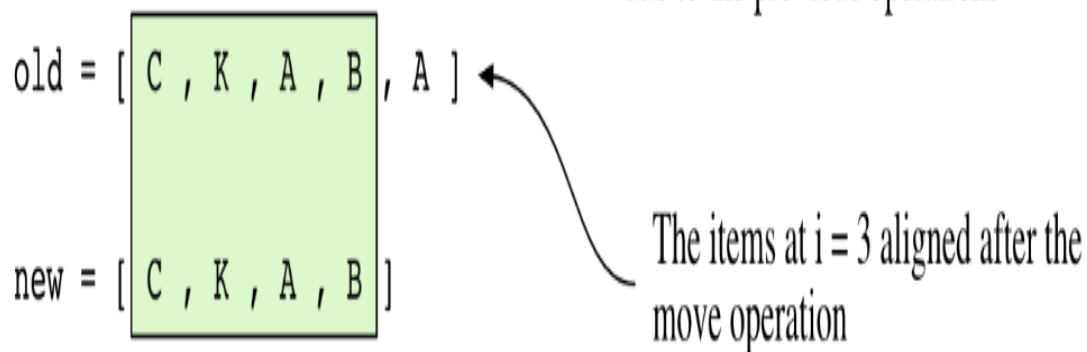
Step five ($i=3$). At $i=3$ the new array has a B, but the old array has an A. Look for a B in the old array, starting at $i=3$. There's one at $i=4$, so you can move it ([figure 7.26](#)).

Figure 7.26 At $i=3$ there's a B in the new array. We can move the B in the old array from $i=4$.

$i=3$



OPERATION: Move B (originally at 3) from $i=4$ to $i=3$ ← B was originally at $i=3$, but it moved due to the previous operations

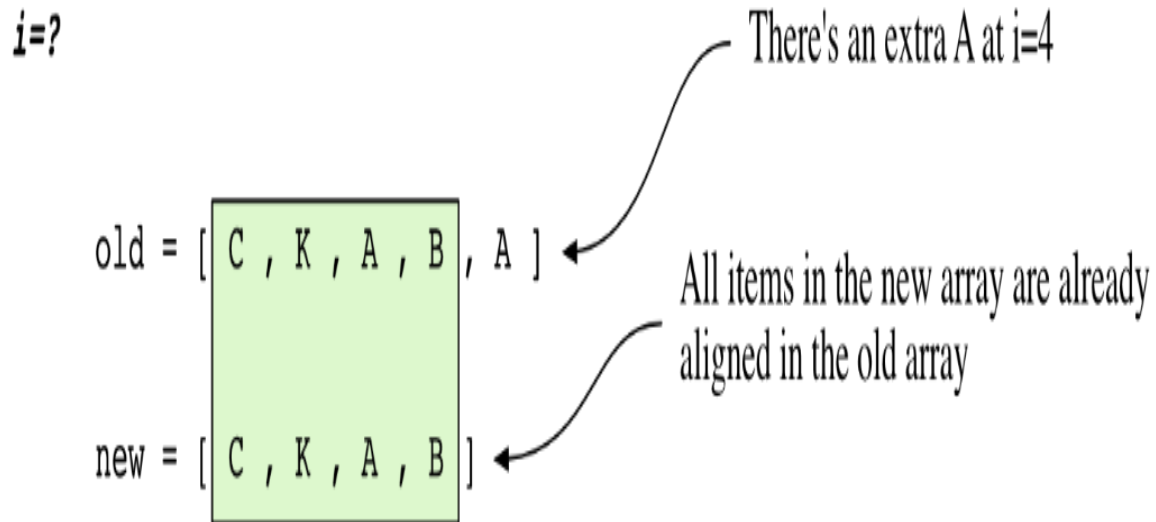


You append the move operation to the sequence of operations, and move the B to $i=3$ in the old array.

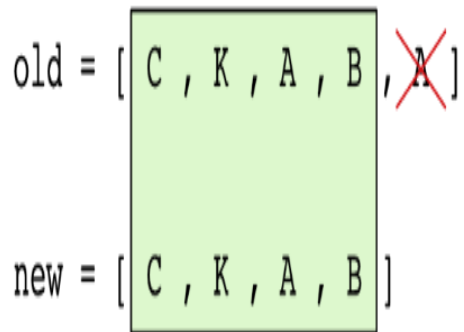
At this point, you're done iterating the elements in the new array, and all items from $i=0$ to $i=3$ in the old array are aligned with the new array. You're only missing to remove what's left in the old array: the excess items that don't appear in the new array.

Step six. At $i=4$ in the old array is an extra **A** that you want to remove (figure [7.27](#)).

Figure 7.27 At $i=4$ in the old array is an extra **A** that we need to remove.



OPERATION: Remove A at $i=4$



In fact, if there were more items in the old array, you'd want to remove them as well. All of those remove operations would be at index $i=4$, because as an item is removed, the next one occupies its place at that index.

I hope the algorithm is clear now after working out an example by hand. It's time to implement it in code.

7.6.4 Implementing the algorithm

Let's start by defining a constant for the operation names. Inside the *utils/arrays.js* file, add the following constant:

```
export const ARRAY_DIFF_OP = {  
  ADD: 'add',  
  REMOVE: 'remove',  
  MOVE: 'move',  
  NOOP: 'noop',  
}
```

You want a way of keeping track of the old array original indices, so when you modify a copy of the old array as you apply each operation, you can still keep the original indices. A good solution is to create a class, let's call it *ArrayWithOriginalIndices*, that wraps a copy of the old array and keeps track of the original indices. Whatever item is moved inside that array wrapper, the original index is also moved.

This wrapper class is going to search for a given item in the wrapped array (like in step two of the previous example, where you looked if there was a *c* somewhere in the array) for which you want a custom comparison function. The items inside the array will be virtual nodes in the next chapter, and you want a function that tells you if two virtual nodes are equal. (You'll implement this function that checks if two virtual nodes are equal in the next chapter.)

Warning

Recall that, for JavaScript, two objects are equal if they're the same reference — that is, the same object in memory. Because our virtual nodes are objects, you need to define a custom function to compare them. You can't rely on the default `===` comparison, because if a virtual node is the same as another one but it's a different object, the comparison will return `false`, and that's not what you want.

You can test that yourself like so:

```
const a = { foo: 'bar' }
const b = { foo: 'bar' }
console.log(a === b) // false
```

In the *utils/arrays.js* file, add the class in listing [7.4](#). You'll add functionality to this class in a minute.

Listing 7.4 The `ArrayWithOriginalIndices` class wraps an array and keeps track of the original indices (*utils/arrays.js*)

```
class ArrayWithOriginalIndices { #1
  #array = []
  #originalIndices = []
  #equalsFn

  constructor(array, equalsFn) {
    this.#array = [...array] #2
    this.#originalIndices = array.map((_, i) => i) #3
    this.#equalsFn = equalsFn #4
  }

  get length() { #5
    return this.#array.length
  }
}
```

Let's define the `arraysDiffSequence()` function with some TO-DO comments where each case of the algorithm will be implemented. Inside the *utils/arrays.js* file, add the function in listing [7.5](#).

Listing 7.5 The `arraysDiffSequence()` function (*utils/arrays.js*)

```
export function arraysDiffSequence(
  oldArray,
  newArray,
  equalsFn = (a, b) => a === b #1
) {
  const sequence = []
  const array = new ArrayWithOriginalIndices(oldArray,
  equalsFn) #2

  for (let index = 0; index < newArray.length; index++) { #3
    // TODO: removal case

    // TODO: noop case
```

```

    // TODO: addition case

    // TODO: move case
}

// TODO: remove extra items

return sequence #4
}

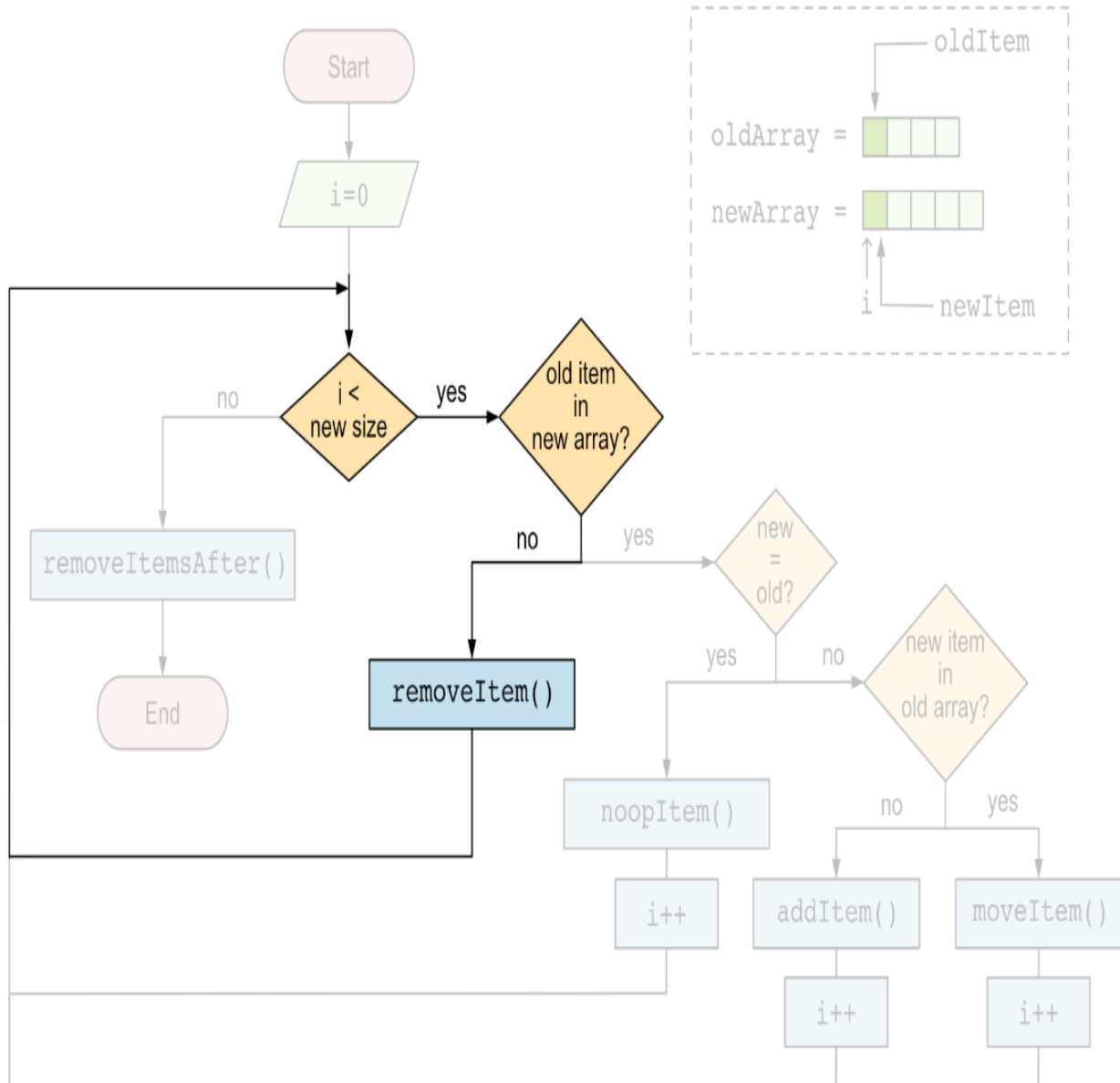
```

Giving the `equalsFn` parameter a default value using the default equality operator (`===`) will be handy, as you'll test this function passing it arrays of strings in this chapter, mostly. Let's start with the remove case.

The remove case

To find if an item was removed, you check if the item in the old array at the current index doesn't exist in the new array. This branch in the flow chart is shown in figure [7.28](#).

Figure 7.28 At `i=4` in the old array is an extra `a` that we need to remove.



Implement this logic inside a method called `isRemoval()` in the `ArrayWithOriginalIndices` class, as shown in bold font in listing [7.6](#).

Listing 7.6 Detecting if an operation is a removal (*utils/arrays.js*)

```

class ArrayWithOriginalIndices {
  #array = []
  #originalIndices = []
  #equalsFn

  constructor(array, equalsFn) {

```

```

    this.#array = [...array]
    this.#originalIndices = array.map((_, i) => i)
    this.#equalsFn = equalsFn
  }

  get length() {
    return this.#array.length
  }

  isRemoval(index, newArray) {
    if (index >= this.length) { #1
      return false
    }

    const item = this.#array[index] #2
    const indexInNewArray = newArray.findIndex((newItem) =>
#3      this.#equalsFn(item, newItem) #4
    )

    return indexInNewArray === -1 #5
  }
}

```

And now, let's implement a method to handle the removal of an item and return the operation. You want to reflect the removal operation both in the `#array` and `#originalIndices` properties of the `ArrayWithOriginalIndices` instance. Add the `removeItem()` method to the `ArrayWithOriginalIndices` class, as shown in listing [7.7](#).

Listing 7.7 Removing an item from the old array and returning the operation (*utils/arrays.js*)

```

class ArrayWithOriginalIndices {
  // --snip-- //

  isRemoval(index, newArray) {
    // --snip-- //
  }

  removeItem(index) {
    const operation = { #1
      op: ARRAY_DIFF_OP.REMOVE,
      index,
      item: this.#array[index], #2
    }
  }
}

```

```

    this.#array.splice(index, 1) #3
    this.#originalIndices.splice(index, 1) #4

    return operation #5
  }
}

```

And with these two methods, you can implement the remove case in the `arraysDiffSequence()` function. Add the code in bold font in listing [7.8](#) to the `arraysDiffSequence()` function.

Listing 7.8 Implementing the removal case (*utils/arrays.js*)

```

export function arraysDiffSequence(
  oldArray,
  newArray,
  equalsFn = (a, b) => a === b
) {
  const sequence = []
  const array = new ArrayWithOriginalIndices(oldArray,
    equalsFn)

  for (let index = 0; index < newArray.length; index++) {
    if (array.isRemoval(index, newArray)) { #1
      sequence.push(array.removeItem(index)) #2
      index-- #3
      continue #4
    }

    // TODO: noop case

    // TODO: addition case

    // TODO: move case
  }

  // TODO: remove extra items

  return sequence
}

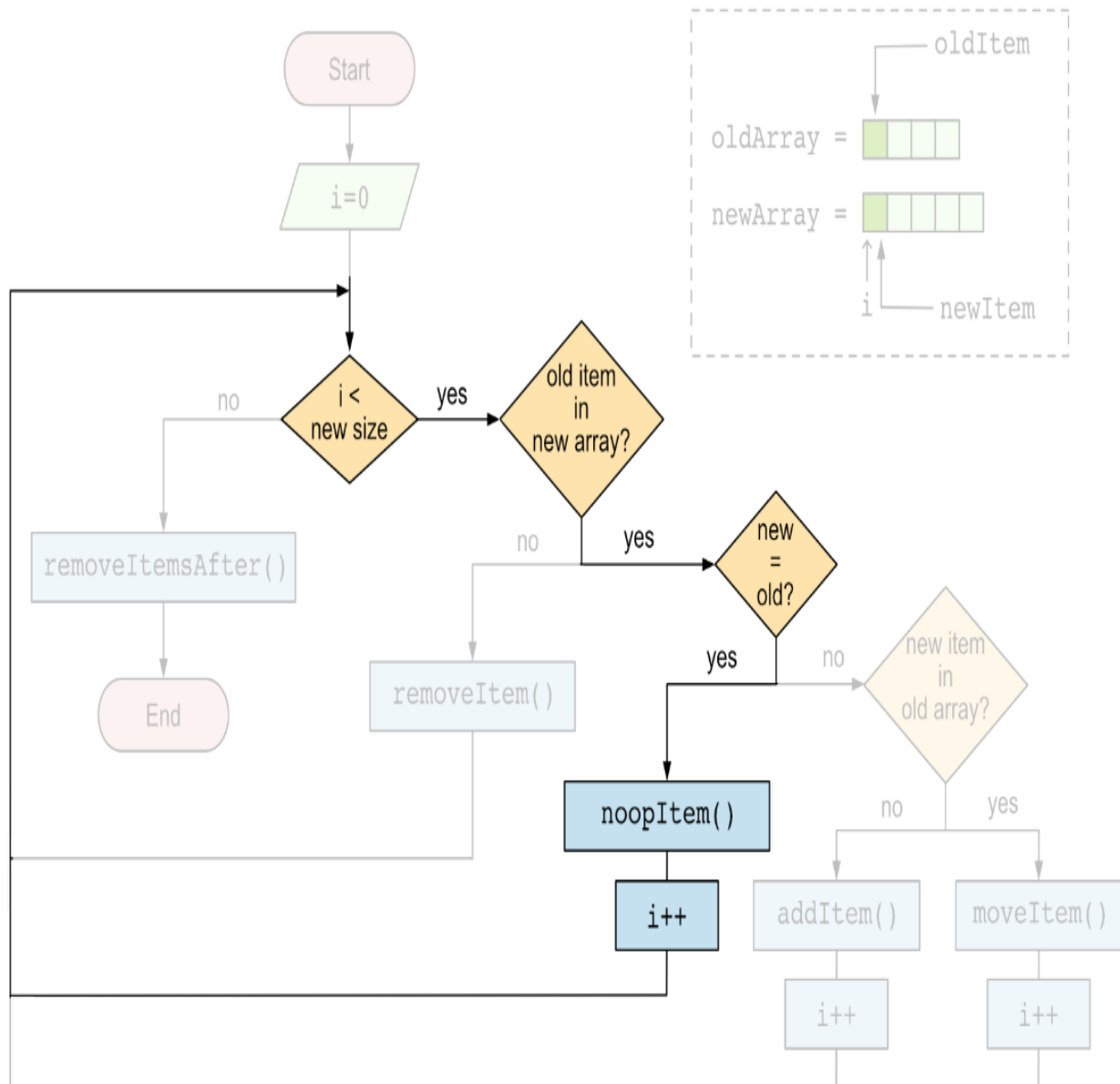
```

Great! Let's take care of the noop case next.

The noop case

The noop case happens when at the current index, both the old and new arrays have the same item. Thus, detecting this case is straightforward. You can see the noop case highlighted in the flow chart in figure [7.29](#).

Figure 7.29 The noop case in the flow chart



Implement a method called `isNoop()` in the `ArrayWithOriginalIndices` class, as shown in bold font in listing [7.9](#).

Listing 7.9 Detecting if an operation is a noop (*utils/arrays.js*)

```
class ArrayWithOriginalIndices {
  // --snip-- //

  isNoop(index, newArray) {
    if (index >= this.length) { #1
      return false
    }

    const item = this.#array[index] #2
    const newItem = newArray[index] #3

    return this.#equalsFn(item, newItem) #4
  }
}
```

Once you detect there's a noop at the current index, you need to return the noop operation. And here's where the original indices come in handy.

Implement a method called `noopItem()` in the `ArrayWithOriginalIndices` class, as shown in bold font in listing [7.10](#). Note that you also implement an `originalIndexAt()` method to get the original index of an item in the old array.

Listing 7.10 Returning a noop operation (*utils/arrays.js*)

```
class ArrayWithOriginalIndices {
  // --snip-- //

  isNoop(index, newArray) {
    // --snip-- //
  }

  originalIndexAt(index) {
    return this.#originalIndices[index] #1
  }

  noopItem(index) {
    return { #2
      op: ARRAY_DIFF_OP.NOOP,
      originalIndex: this.originalIndexAt(index), #3
      index,
      item: this.#array[index], #4
    }
  }
}
```

```

    }
  }
}

```

As you can see, you don't need to do anything to the old array to reflect the noop operation. Things stay the same. With these methods in place you can go ahead and implement the noop case in the `arraysDiffSequence()` function. Write the code in bold font in listing [7.11](#) inside the `arraysDiffSequence()` function.

Listing 7.11 Implementing the noop case (*utils/arrays.js*)

```

export function arraysDiffSequence(
  oldArray,
  newArray,
  equalsFn = (a, b) => a === b
) {
  const sequence = []
  const array = new ArrayWithOriginalIndices(oldArray,
    equalsFn)

  for (let index = 0; index < newArray.length; index++) {
    if (array.isRemoval(index, newArray)) {
      sequence.push(array.removeItem(index))
      index--
      continue
    }

    if (array.isNoop(index, newArray)) { #1
      sequence.push(array.noopItem(index)) #2
      continue #3
    }

    // TODO: addition case

    // TODO: move case
  }

  // TODO: remove extra items

  return sequence
}

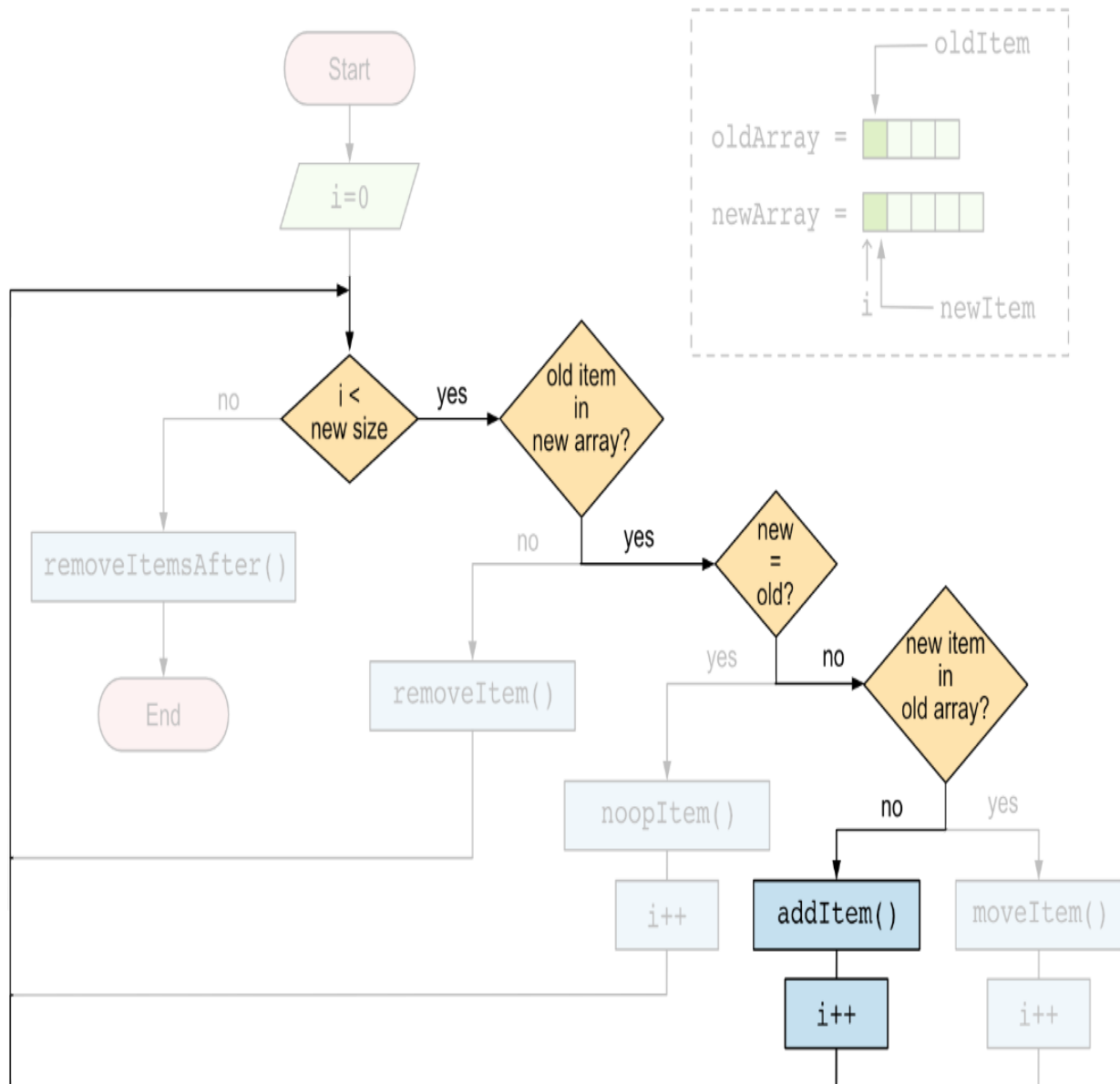
```

Time to move on to the addition case.

The addition case

To check if an item was added in the new array, you need to check if that item doesn't exist in the old array, starting from the current index. Figure [7.30](#) shows the addition case in the flow chart.

Figure 7.30 The addition case in the flow chart



To find the item in the old array starting from a given index, you implement a method called `findIndexFrom()`. Using that method, the `isAddition()`

method is straightforward to implement.

Add the code in bold font in listing [7.12](#) to the `ArrayWithOriginalIndices` class.

Listing 7.12 Detecting if an operation is an addition (*utils/arrays.js*)

```
class ArrayWithOriginalIndices {  
  // --snip--  
  
  isAddition(item, fromIdx) {  
    return this.findIndexFrom(item, fromIdx) === -1 #1  
  }  
  
  findIndexFrom(item, fromIndex) {  
    for (let i = fromIndex; i < this.length; i++) { #2  
      if (this.#equalsFn(item, this.#array[i])) { #3  
        return i  
      }  
    }  
  
    return -1 #4  
  }  
}
```

Dealing with an addition is as simple as adding the item to the old array and returning the add operation. You have to add an entry to the `#originalIndices` property, but the added item wasn't present in the original old array, so you can use a `-1` in this case.

Add the `addItem()` method in bold font in listing [7.13](#) to the `ArrayWithOriginalIndices` class.

Listing 7.13 Adding an item to the old array and returning an add operation (*utils/arrays.js*)

```
class ArrayWithOriginalIndices {  
  // --snip--  
  
  isAddition(item, fromIdx) {  
    return this.findIndexFrom(item, fromIdx) === -1  
  }  
  
  findIndexFrom(item, fromIndex) {
```



```

    for (let i = fromIndex; i < this.length; i++) {
      if (this.#equalsFn(item, this.#array[i])) {
        return i
      }
    }

    return -1
  }

  addItem(item, index) {
    const operation = { #1
      op: ARRAY_DIFF_OP.ADD,
      index,
      item,
    }

    this.#array.splice(index, 0, item) #2
    this.#originalIndices.splice(index, 0, -1) #3

    return operation #4
  }
}

```

These two methods make it easy to implement the addition case in the `arraysDiffSequence()` function. Add the code in bold font in listing [7.14](#) to the `arraysDiffSequence()` function.

Listing 7.14 Implementing the addition case (*utils/arrays.js*)

```

export function arraysDiffSequence(
  oldArray,
  newArray,
  equalsFn = (a, b) => a === b
) {
  const sequence = []
  const array = new ArrayWithOriginalIndices(oldArray,
    equalsFn)

  for (let index = 0; index < newArray.length; index++) {
    if (array.isRemoval(index, newArray)) {
      sequence.push(array.removeItem(index))
      index--
      continue
    }

    if (array.isNoop(index, newArray)) {

```

```

        sequence.push(array.noopItem(index))
        continue
    }

    const item = newArray[index] #1

    if (array.isAddition(item, index)) { #2
        sequence.push(array.addItem(item, index)) #3
        continue #4
    }

    // TODO: move case
}

// TODO: remove extra items

return sequence
}

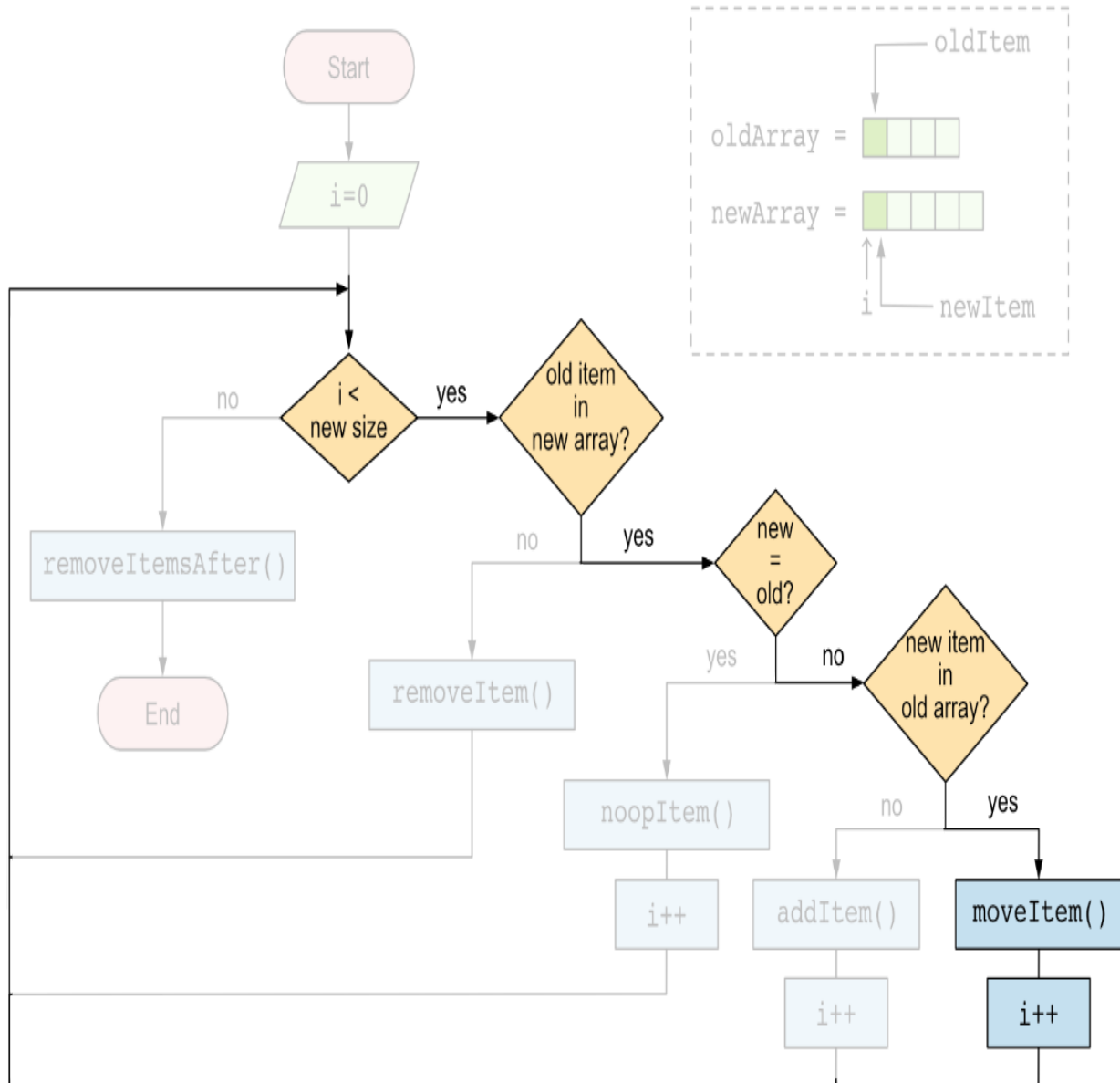
```

Let's move on with the move case. This one is interesting.

The move case

The neat thing about the move case is that you don't need to explicitly test for it; if the operation isn't a removal, an addition, or a noop, it must be a move. This branch is highlighted in the flow chart in figure [7.31](#).

Figure 7.31 The move case in the flow chart



To move an item, you want to extract it from the array (you can use the `splice()` method for that) and insert it in the new position (you can use the `splice()` method again). There are two things you want to keep in mind: one is that you need to remember to also move the original index to its new position, and the other is that you have to include the original index in the move operation. The importance of this will become clear in the next chapter.

Inside the `ArrayWithOriginalIndices` class, add the code in bold font in listing [7.15](#) to implement the `moveItem()` method.

Listing 7.15 Detecting if an operation is an addition (*utils/arrays.js*)

```
class ArrayWithOriginalIndices {
  // --snip-- //

  moveItem(item, toIndex) {
    const fromIndex = this.findIndexFrom(item, toIndex) #1

    const operation = { #2
      op: ARRAY_DIFF_OP.MOVE,
      originalIndex: this.originalIndexAt(fromIndex), #3
      from: fromIndex,
      index: toIndex,
      item: this.#array[fromIndex],
    }

    const [_item] = this.#array.splice(fromIndex, 1) #4
    this.#array.splice(toIndex, 0, _item) #5

    const [originalIndex] =
this.#originalIndices.splice(fromIndex, 1) #6
    this.#originalIndices.splice(toIndex, 0, originalIndex)
#7

    return operation #8
  }
}
```

Adding the move case to the `arraysDiffSequence()` function is straightforward. Include the line in bold font in listing [7.16](#) in the `arraysDiffSequence()` function.

Listing 7.16 Adding an item to the old array and returning an add operation (*utils/arrays.js*)

```
export function arraysDiffSequence(
  oldArray,
  newArray,
  equalsFn = (a, b) => a === b
) {
  const sequence = []
  const array = new ArrayWithOriginalIndices(oldArray,
equalsFn)

  for (let index = 0; index < newArray.length; index++) {
    if (array.isRemoval(index, newArray)) {
```

```

        sequence.push(array.removeItem(index))
        index--
        continue
    }

    if (array.isNoop(index, newArray)) {
        sequence.push(array.noopItem(index))
        continue
    }

    const item = newArray[index]

    if (array.isAddition(item, index)) {
        sequence.push(array.addItem(item, index))
        continue
    }

    sequence.push(array.moveItem(item, index))
}

// TODO: remove extra items

return sequence
}

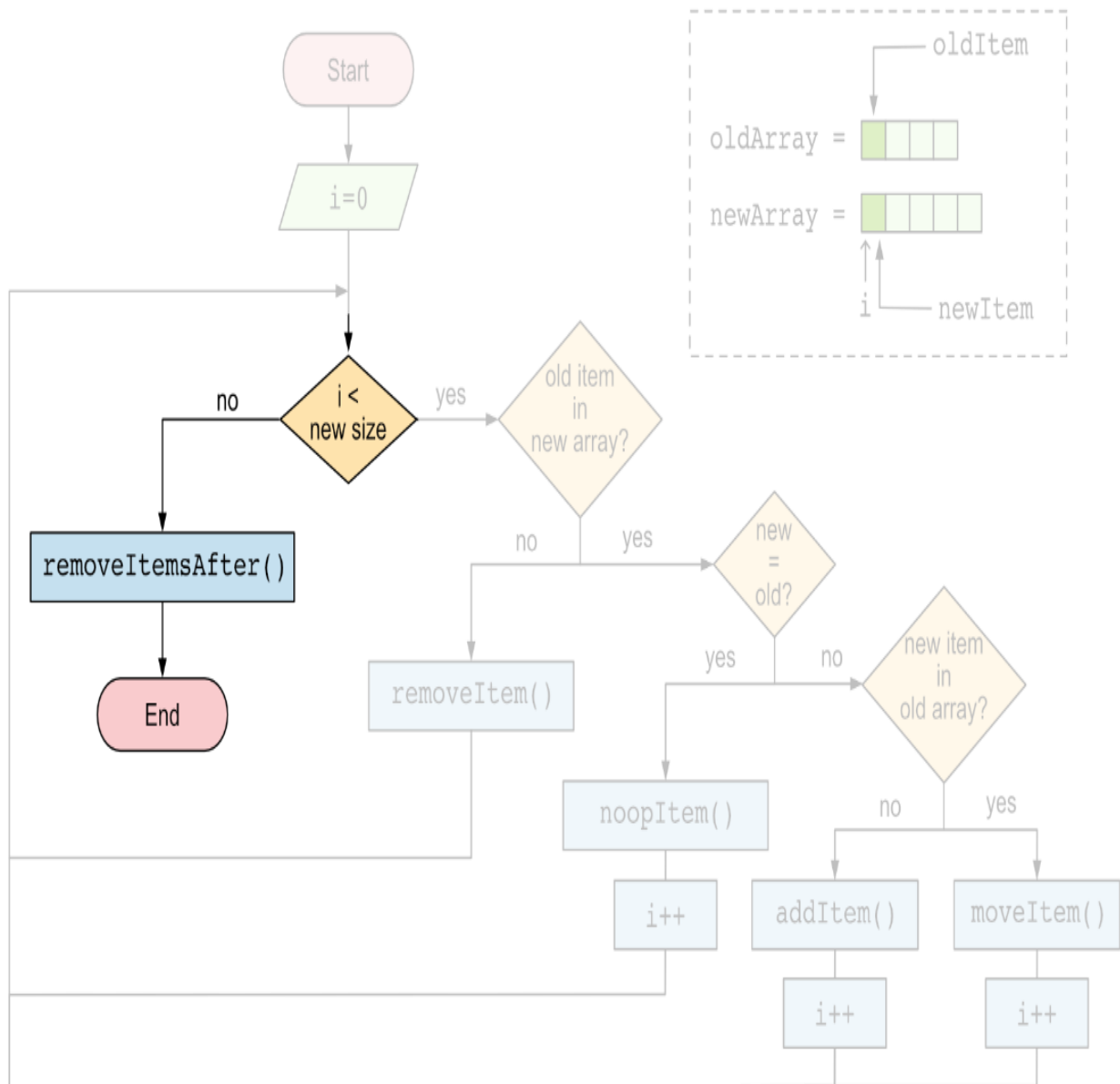
```

You're almost there! You're only missing to remove the outstanding items from the old array.

Removing the outstanding items

What happens when you reach the end of the new array but there are still items in the old array? You remove them all, of course. Figure [7.32](#) shows the removal case in the flow chart.

Figure 7.32 The removal case in the flow chart



To remove the outstanding items, you want to remove all items past the current index (which is the length of the new array) from the old array. You can use a `while` loop that keeps removing items while the old array is longer than the index. Write the code for the `removeItemsAfter()` method in bold font in listing [7.17](#).

Listing 7.17 Detecting if an operation is an addition (*utils/arrays.js*)

```
class ArrayWithOriginalIndices {
  // --snip-- //
```

```

removeItemsAfter(index) {
  const operations = []

  while (this.length > index) { #1
    operations.push(this.removeItem(index)) #2
  }

  return operations #3
}
}

```

Finally, add the code in bold font in listing [7.18](#) to the `arraysDiffSequence()` function to remove the outstanding items. Note that `removeItemsFrom()` doesn't return a single operation, but an array of operations.

Listing 7.18 Adding an item to the old array and returning an add operation (*utils/arrays.js*)

```

export function arraysDiffSequence(
  oldArray,
  newArray,
  equalsFn = (a, b) => a === b
) {
  const sequence = []
  const array = new ArrayWithOriginalIndices(oldArray,
    equalsFn)

  for (let index = 0; index < newArray.length; index++) {
    if (array.isRemoval(index, newArray)) {
      sequence.push(array.removeItem(index))
      index--
      continue
    }

    if (array.isNoop(index, newArray)) {
      sequence.push(array.noopItem(index))
      continue
    }

    const item = newArray[index]

    if (array.isAddition(item, index)) {
      sequence.push(array.addItem(item, index))
      continue
    }
  }
}

```

```

    sequence.push(array.moveItem(item, index))
}

sequence.push(...array.removeItemAfter(newArray.length))

return sequence
}

```

And just like that, you have a working implementation of the `arraysDiffSequence()` function! Believe it or not, this was the hardest algorithm to implement in the whole book. If you're still with me, the rest of the book will be easy to follow.

Exercise 7.6

Write a function called `applyArraysDiffSequence()` that given an array and a sequence of operations, applies the operations to the array and returns the resulting array.

To test it, pass the following arrays to the `arraysDiffSequence()` function:

- old array: ['A', 'A', 'B', 'C']
- new array: ['C', 'K', 'A', 'B']

Save the resulting operations in a variable called `sequence`. Then pass the old array and the sequence to the `applyArraysDiffSequence()` function and check that the resulting array is the same as the new array.

Find the solution at <http://mng.bz/p1JE>.

7.7 Summary

- The reconciliation algorithm has two main steps: *diffing* and *patching*.
- Diffing is the process of finding the differences between two virtual trees.
- Patching is the process of applying the differences to the real DOM.
- Diffing two virtual trees to find their differences boils down to solving three problems: finding the differences between two objects, finding the

differences between two arrays, and finding a sequence of operations that applied to an array will transform it into another array.

8 The reconciliation algorithm: patching the DOM

This chapter covers

- Implementing the `patchDOM()` function
- Using the `objectsDiff()` function to find the differences in attributes and styles
- Using the `arraysDiff()` function to find the differences between CSS classes
- Using the `arraysDiffSequence()` function to find the differences between virtual DOM children
- Using the Document API to patch the changes in the DOM

In the previous chapter, you saw how the reconciliation algorithm works in two phases: finding the differences between two virtual DOM trees, and patching those differences in the real DOM.

You implemented the three key functions to find differences between two objects or two arrays: `objectsDiff()`, `arraysDiff()` and `arraysDiffSequence()`. In this chapter, you'll use these functions to implement the reconciliation algorithm inside the `patchDOM()` function. `patchDOM()` finds the differences between two virtual DOM trees (the one that's currently rendered and the one after the state has changed) and patches the real DOM accordingly.

With the `patchDOM()` function, your framework will be able to update the DOM using a small set of operations, instead of replacing the whole DOM tree every time the state changes. Thanks to this function, the new version of your framework—which you'll publish at the end of this chapter—will be much more efficient than the previous version. The best part is that, the TODOs application rewrite you did in chapter 6 will still work with the new version of your framework, as its public API doesn't change. You'll just need to update the `import` statement to import the new version of the framework

instead of the old one. This time though, you won't lose the focus from the `<input>` fields every time you type a character, because the fields will be updated without being replaced.

We'll end the chapter looking at a few ways you can use browser developer tools to inspect the DOM and observe how the reconciliation algorithm patches small portions of the DOM.

Important

You can find all the listings in this chapter in the [listings/ch08 directory](#) of the book's repository. The code in this chapter can be checked out from [the ch8 label](#):

```
$ git switch --detach ch8
```

Code catch-up

In the last chapter you modified the `createApp()` so that the returned application's object `renderApp()` method doesn't destroy and recreate the whole DOM, but rather call a `patchDOM()` function to patch the changes in the DOM. You'll implement the `patchDOM()` function in this chapter.

Then you wrote the `objectsDiff()` and `arraysDiff()` functions to find the differences between two objects or two arrays, respectively. Last, you implemented the `arraysDiffSequence()` function to find the sequence of operations that transform one array into another. To help this function keep track of the indices of the elements in the original array and how they change, you wrote the `ArrayWithOriginalIndices` class.

8.1 Mounting the DOM at an index

To patch the DOM, you compare the two virtual DOM trees by traversing them individually in a depth-first manner and apply the differences you find along the way to the real DOM (as you already saw in the opening example from last chapter). Very often, what happens is that a new node is added somewhere in the middle of another's node children array. Let's look at an example of this.

Think, for example, of a case of an error message that appears between an `<input>` field and a `<button>` element when the user enters an invalid value in the `<input>` field. Before the error appears, the DOM looks like this:

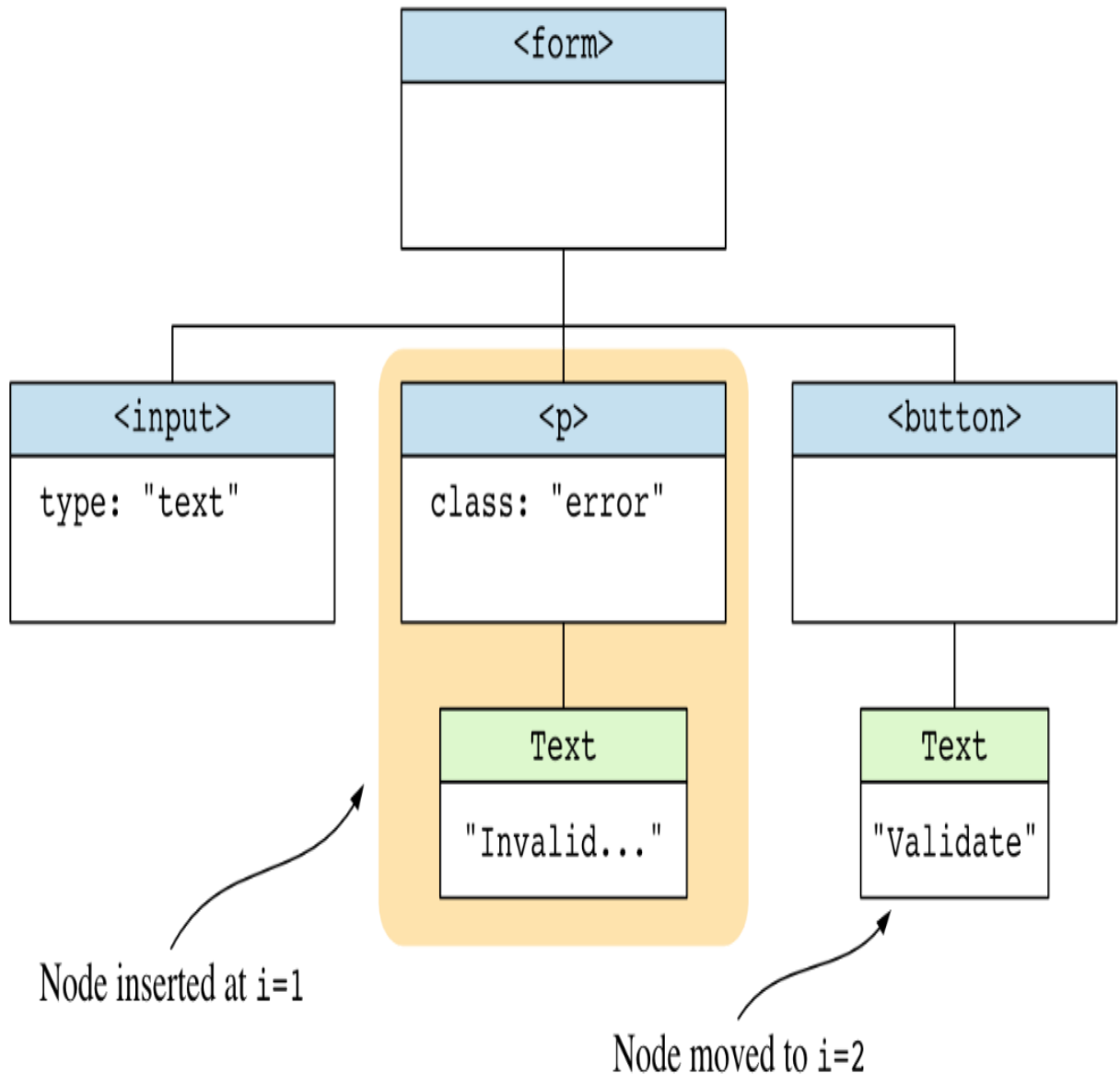
```
<form>
  <input type="text"/>
  <button>Validate</button>
</form>
```

Then, the user writes some invalid text in the `<input>` field (whatever "invalid" means here), clicks the validate `<button>` and the error message appears:

```
<form>
  <input type="text"/>
  <p class="error">Invalid text! Try something else</p>
  <button>Validate</button>
</form>
```

As you can see, the error message is inserted between the `<input>` field and the `<button>` element (at position `i=1`) inside the `<form>` element's children. This is illustrated in figure [8.1](#). Your current implementation of the `mountDOM()` function doesn't allow you to insert a node at a given index inside a parent node.

Figure 8.1 Inserting the node at index `i=1`



To insert new nodes at arbitrary positions, you need to modify the `mountDOM()` function so that it accepts an index where to put a node, instead of always appending it at the end (like the function does at the moment).

8.1.1 The `insert()` function

Let's write an `insert()` function inside the `mount-dom.js` file to insert a node at a given index inside a parent node. You can then use this function inside the `mountDOM()` function. You want the function to also append nodes at the end without having to figure out what the index would be, because that's still a

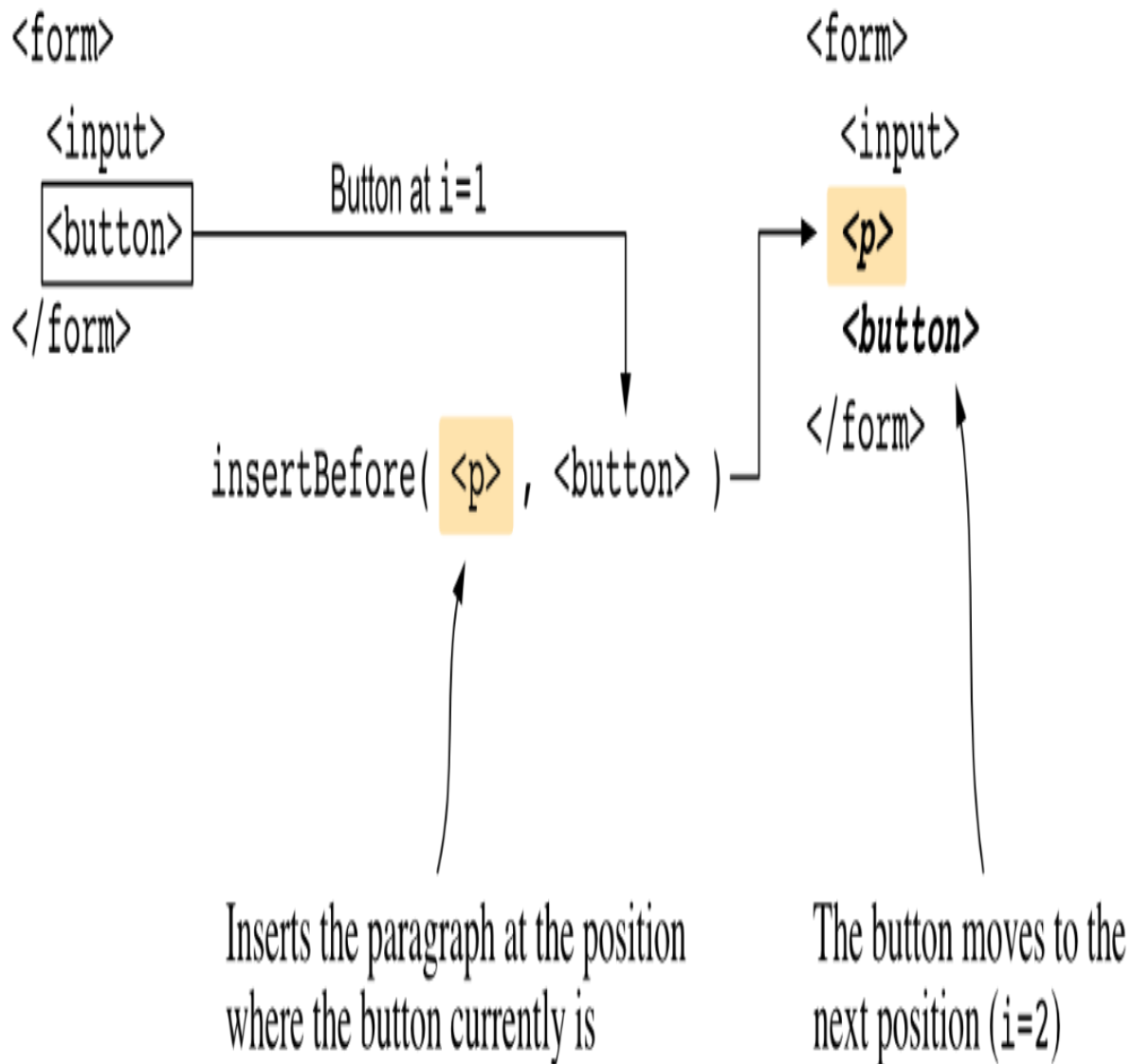
common use case. Here's what you can do: if the passed in `index` is `null` or `undefined`, append the node to the parent node.

Then, in the cases where the index is defined (neither `null` nor `undefined`), you need to account for the following cases:

- If the index is negative, throw an error, because negative indices don't make sense.
- If the index is greater or equal to the number of children, append the node to the parent node.
- Otherwise, you know the index lies somewhere in the children array, so you insert the node at the given index.

To insert a node at a given index, you can use the [`insertBefore\(\)` method](#) of the `Node` interface. This method requires two arguments: the new element to insert and the reference element, which is the element before which the new element will be inserted. Figure [8.2](#) illustrates how the `insertBefore()` method would work for the example in figure [8.1](#).

Figure 8.2 Using the `insertBefore()` method



Exercise 8.1

In a blank HTML file, create a `<div>` element with two `<p>` children, as follows:

```
<div>
  <p>One</p>
  <p>Three</p>
</div>
```

Then, use the `insertBefore()` method to insert a new `<p>` element between the two existing ones. The new `<p>` element should contain the text `Two`.

Find the solution at <http://mng.bz/OP12>.

Open the *mount-dom.js* file and write the `insert()` function inside it, below the `mountDOM()` function (listing [8.1](#)).

Listing 8.1 The `insert()` function (*mount-dom.js*)

```
function insert(el, parentEl, index) {
  // If index is null or undefined, simply append.
  // Note the usage of == instead of ===.
  if (index == null) {
    parentEl.append(el) #1
    return
  }

  if (index < 0) {
    throw new Error(`Index must be a positive integer, got
    ${index}`) #2
  }

  const children = parentEl.childNodes

  if (index >= children.length) {
    parentEl.append(el) #3
  } else {
    parentEl.insertBefore(el, children[index]) #4
  }
}
```

Note

Recall that, depending on how you configured your linter, you might get a complaint about using `==` instead of `===` (this comes from the ["equeqeq" rule](#)). You can safely ignore this warning; the `==` operator is fine here, because you want to check for both `null` and `undefined` values.

With the `insert()` function in place, you can now modify the `mountDOM()` function to accept an index as a third parameter (listing [8.2](#)), so that you can

insert the node at that index. The index should be passed down to the text and element node creation functions, which will then use the `insert()` function to insert the node at the given index.

Listing 8.2 Adding the `index` as parameter to the functions (*mount-dom.js*)

```
export function mountDOM(vdom, parentEl, index) {
  switch (vdom.type) {
    case DOM_TYPES.TEXT: {
      createTextNode(vdom, parentEl, index)
      break
    }

    case DOM_TYPES.ELEMENT: {
      createElementNode(vdom, parentEl, index)
      break
    }

    case DOM_TYPES.FRAGMENT: {
      createFragmentNodes(vdom, parentEl, index)
      break
    }

    default: {
      throw new Error(`Can't mount DOM of type:
${vdom.type}`)
    }
  }
}
```

Let's now modify the `createTextNode()`, `createElementNode()`, and `createFragmentNodes()` functions to use the `insert()` function to insert the node at the given index.

8.1.2 Text nodes

To insert a text node at a given index, you modify the `createTextNode()` function to use the `insert()` function, as in listing [8.3](#).

Listing 8.3 Using `insert()` inside `createTextNode()` (*mount-dom.js*)

```
function createTextNode(vdom, parentEl, index) {
  const { value } = vdom
```

```

const textNode = document.createTextNode(value)
vdom.el = textNode

parentEl.append(textNode)
insert(textNode, parentEl, index)
}

```

Let's now look at element nodes.

8.1.3 Element nodes

In the case of element nodes, the change is exactly the same as for text nodes, as you can see in listing [8.4](#).

Listing 8.4 Using `insert()` inside `createElementNode()` (*mount-dom.js*)

```

function createElementNode(vdom, parentEl, index) {
  const { tag, props, children } = vdom

  const element = document.createElement(tag)
  addProps(element, props, vdom)
  vdom.el = element

  children.forEach((child) => mountDOM(child, element))
  parentEl.append(element)
  insert(element, parentEl, index)
}

```

8.1.4 Fragment nodes

The case of fragment nodes is slightly different, because you need to insert all the children of the fragment starting at the given index. Each child is inserted at an index that's computed by adding the given index and its own index inside the fragment:

```

children.forEach((child, i) => {
  mountDOM(child, parentEl, index + i)
})

```

Don't forget to account for the case when the passed `index` is `null`, in which case you want to pass `null` to the `insert()` function, so that the children are

appended at the end of the parent node. Modify the `createFragmentNodes()` function to use the `insert()` function, as in listing [8.5](#).

Listing 8.5 Using `insert()` inside `createElementNode()` (*mount-dom.js*)

```
function createFragmentNodes(vdom, parentEl, index) {
  const { children } = vdom
  vdom.el = parentEl

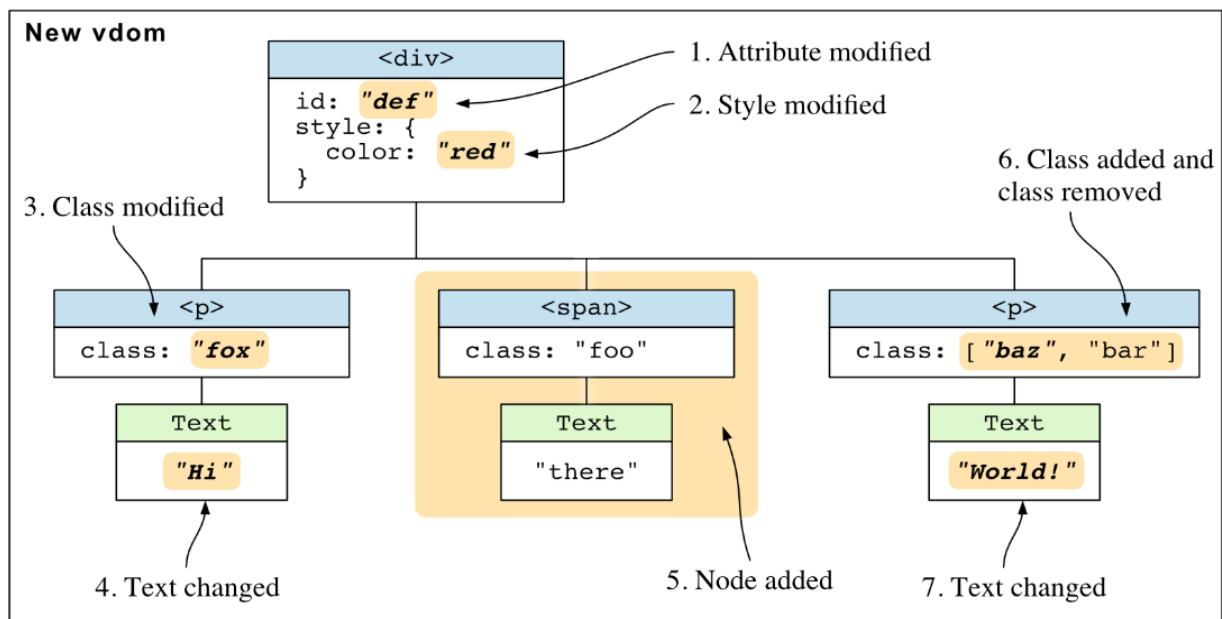
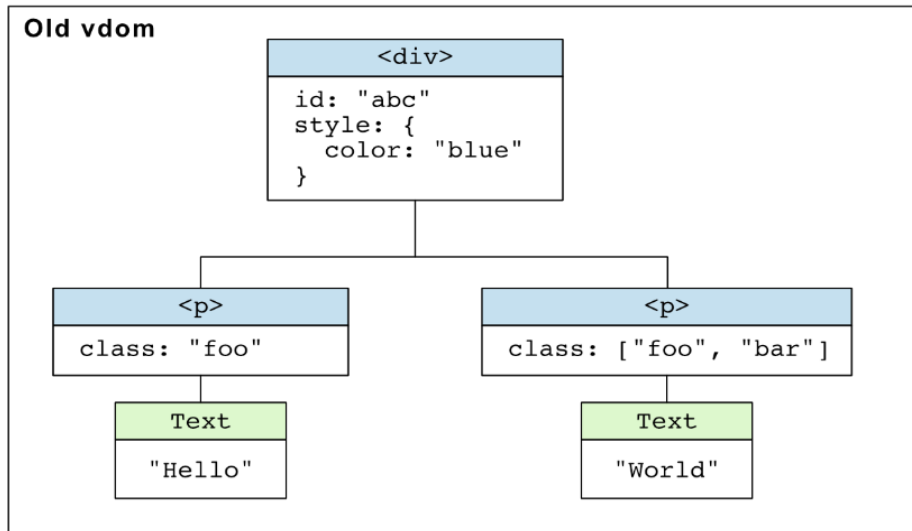
  children.forEach((child) => mountDOM(child, parentEl))
  children.forEach((child, i) =>
    mountDOM(child, parentEl, index ? index + i : null)
  )
}
```

With these changes in place, let's now turn our attention to the `patchDOM()` function.

8.2 Patching the DOM

In the previous chapter we saw an example of how two virtual DOM trees are compared and the changes are applied to the real DOM. Figure [8.3](#) (reproduced from figure [7.5](#)) shows the two virtual DOM trees in the example and the changes that need to be applied to the real DOM.

Figure 8.3 Comparing two virtual DOM trees to find what changed



We did the exercise of comparing the two virtual DOM trees by hand, starting by the top-level `<div>` element, then moving on to its children in a depth-first manner. Let's turn these steps from the exercise into an algorithm that you can implement, *the reconciliation algorithm*.

8.2.1 The reconciliation algorithm

Let's first define the algorithm in plain English, and then we'll translate it into code. I've already touched on what this algorithm is about, but it's time to give a more formal definition.

The reconciliation algorithm

The reconciliation algorithm compares two virtual DOM trees, finds the sequence of operations that transform one into the other, and patches the real DOM by applying those operations to it. The algorithm is recursive, and it starts at the top-level nodes of both virtual DOM trees. After comparing these nodes, it moves on to their children, and so on until it reaches the leaves of the trees.

Thanks to the exercise we worked out by hand in the previous chapter, you have a fair understanding of what the algorithm does. Let's try to put that knowledge into words. Here's the algorithm, step by step:

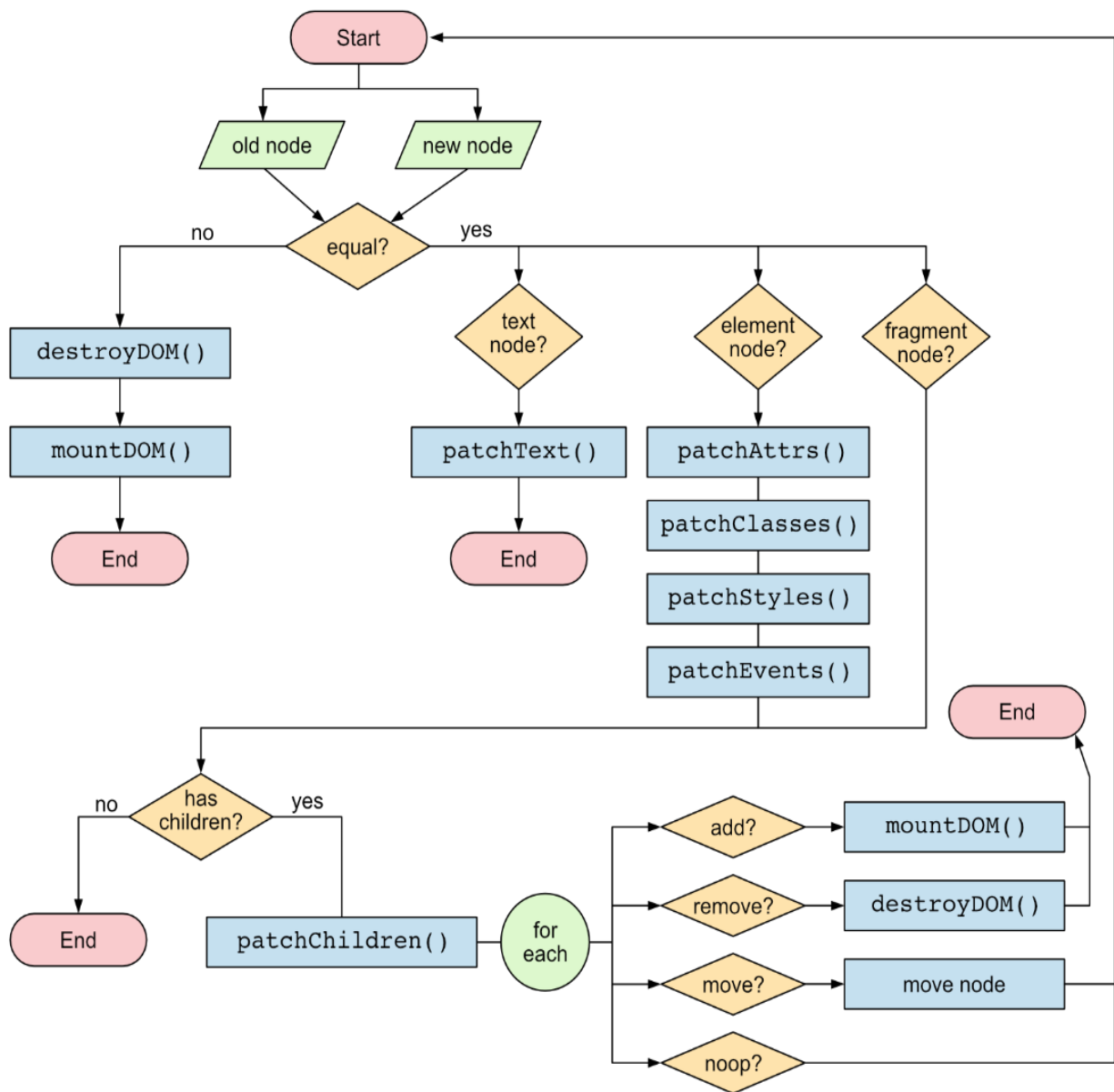
1. Start at the top-level nodes of both virtual DOM trees.
2. If the nodes are different, destroy the DOM node (and everything that's below it), and replace it with the new node and its subtree.
3. If the nodes equal:
 - a. *text nodes* — compare and patch their `nodeValue` (the property containing the text).
 - b. *element nodes* — compare and patch their props (attributes, CSS classes and styles, and event listeners).
4. Find the sequence of operations that transforms the first node's children array into the second node's.
5. For each operation, patch the DOM accordingly:
 - a. *adding a node* — mount the new node (and its subtree) at the given index.
 - b. *removing a node* — destroy the node (and its subtree) at the given index.
 - c. *moving a node* — move the node to the new index. Start from step 1 using these as the new top-level nodes.
 - d. *noop* — start from step 1 using these as the new top-level nodes.

Note

The idea of destroying a DOM node and its subtree when it's found to have changed comes from React. You can read more about React's reconciliation algorithm at reactjs.org/docs/reconciliation.html. I'll explain in a minute why this is a good idea in practice.

The algorithm doesn't look very complicated, but you'll see the implementation has a lot of details. The good news is that you already implemented the most complex part of it in the previous chapter: the `arraysDiffSequence()` function. You'll use this function in a minute. Figure [8.4](#) shows the algorithm as a flowchart. Have this handy as you read the next section; it'll help you during the implementation.

Figure 8.4 The reconciliation algorithm flowchart

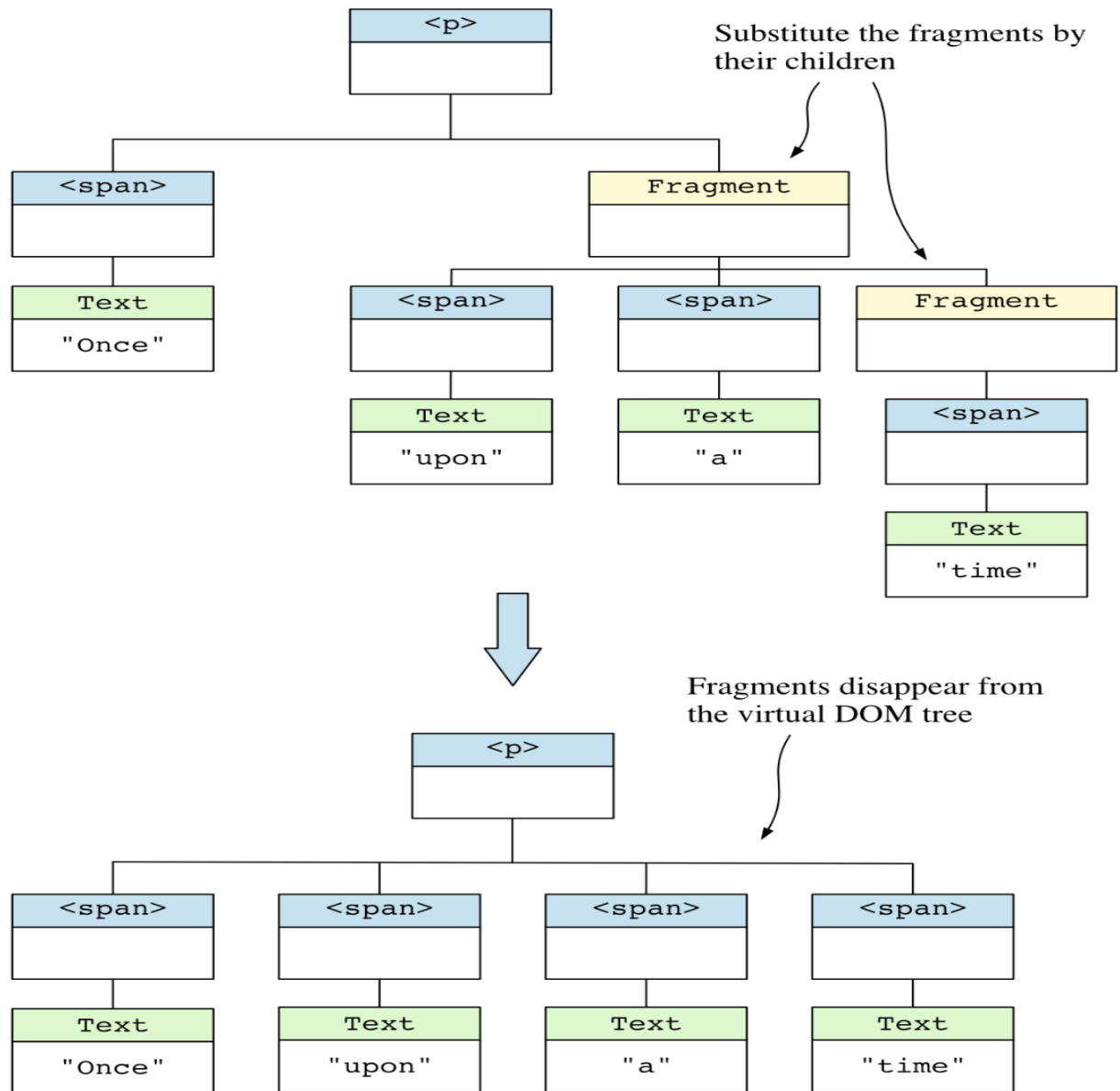


The first thing you should notice is that the algorithm is recursive: once you find two nodes in the children arrays that are the same, you start the algorithm again, this time using those nodes as the top-level nodes. This happens in previous steps 5.c and 5.d, the cases where nodes moved—either forcefully or naturally—so you want to also inspect what happened to their subtrees. You can see this in figure [8.4](#), where the *move* and *noop* branches after the `patchChildren()` call all go back to the top of the algorithm.

The cases where a child node was added or removed are simple enough: you simply call the `mountDOM()` or `destroyDOM()` functions, respectively. If you recall, these functions already take care of mounting or destroying the whole subtree. This is why the algorithm isn't recursive in these cases. As you can see, in figure [8.4](#), the *add* and *remove* branches after the `patchChildren()` call don't go back to the top of the algorithm, but rather end the algorithm.

To patch the children, you need to first extract the children arrays from the two nodes. When you find a fragment node in the children array, instead of keeping it, you want to extract its children. This is illustrated in figure [8.5](#), where you can see that the fragment nodes at every level are replaced by its children.

Figure 8.5 The child nodes of a fragment are extracted and added to the parent's children array



You want to extract the children of a fragment node so that the trees that the reconciliation algorithm works with are as similar as possible to the real DOM, and if you recall, fragments aren't part of the DOM; they're just a way to group nodes together. In the example illustrated in figure [8.5](#), the DOM would look like this:

```
<p>
  <span>Once</span>
  <span>upon</span>
  <span>a</span>
```



```
<span>time</span>
</p>
```

As you can see, there is no trace of the fragments in the real DOM. So, in short, fragments never make it to the reconciliation algorithm, because there aren't any fragments in the real DOM, just a useful construct to group nodes together in the virtual DOM.

The last thing that I want you to notice is step 2, where if you find two top-level nodes that are different, you destroy the whole subtree to mount the new one. There are two things that require some explanation here. First of all, what do we mean when we say that two virtual nodes are different? Second, why do you have to destroy the whole subtree? Doesn't that sound a bit wasteful?

Let's discuss node equality first.

8.2.2 Virtual Node equality

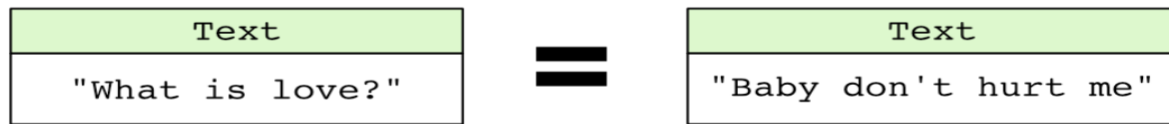
You want to know when two virtual nodes are equal so that you can reuse the existing DOM node (as saved in the virtual node's `el` property); if not, it needs to be destroyed and replaced with a new one. Reusing an existing DOM node and just patching its properties is much more efficient than destroying it and mounting a new one, so you want to reuse nodes as much as possible. For two virtual nodes to be equal, first they need to be of the same type. A text node and an element node can't never be equal, for example, because you wouldn't be able to reuse the existing DOM node. When you know the two nodes you're comparing are of the same type, the rules are the following:

- *text nodes* — two text nodes are always equal (even if their `nodeValue` is different).
- *fragment nodes* — two fragment nodes are always equal (even if they contain different children).
- *element nodes* — two element nodes are equal if their `tagName` properties are equal.

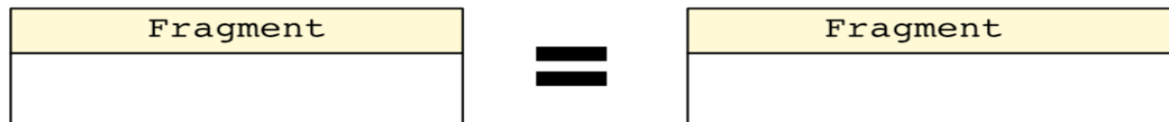
These rules are illustrated in Figure [8.6](#).

Figure 8.6 The rules for virtual node equality

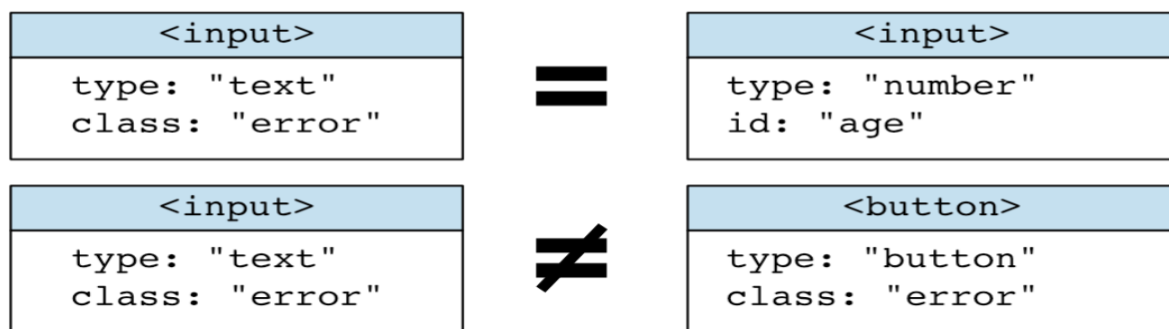
Text nodes are always equal



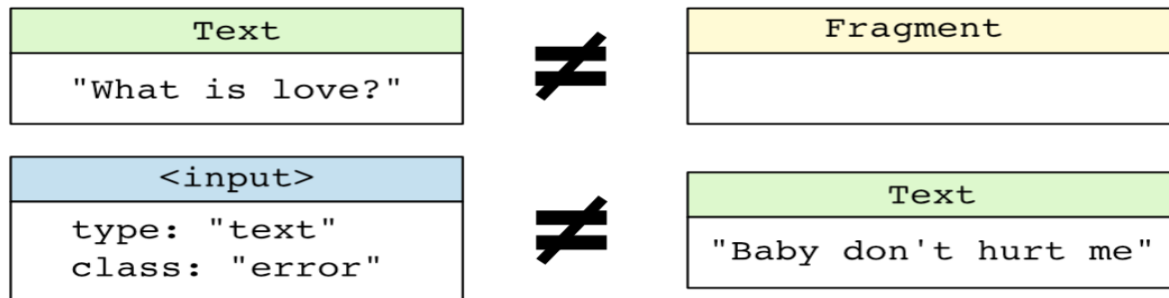
Fragment nodes are always equal



Element nodes are equal if they have the same tag



Nodes of different type are never equal



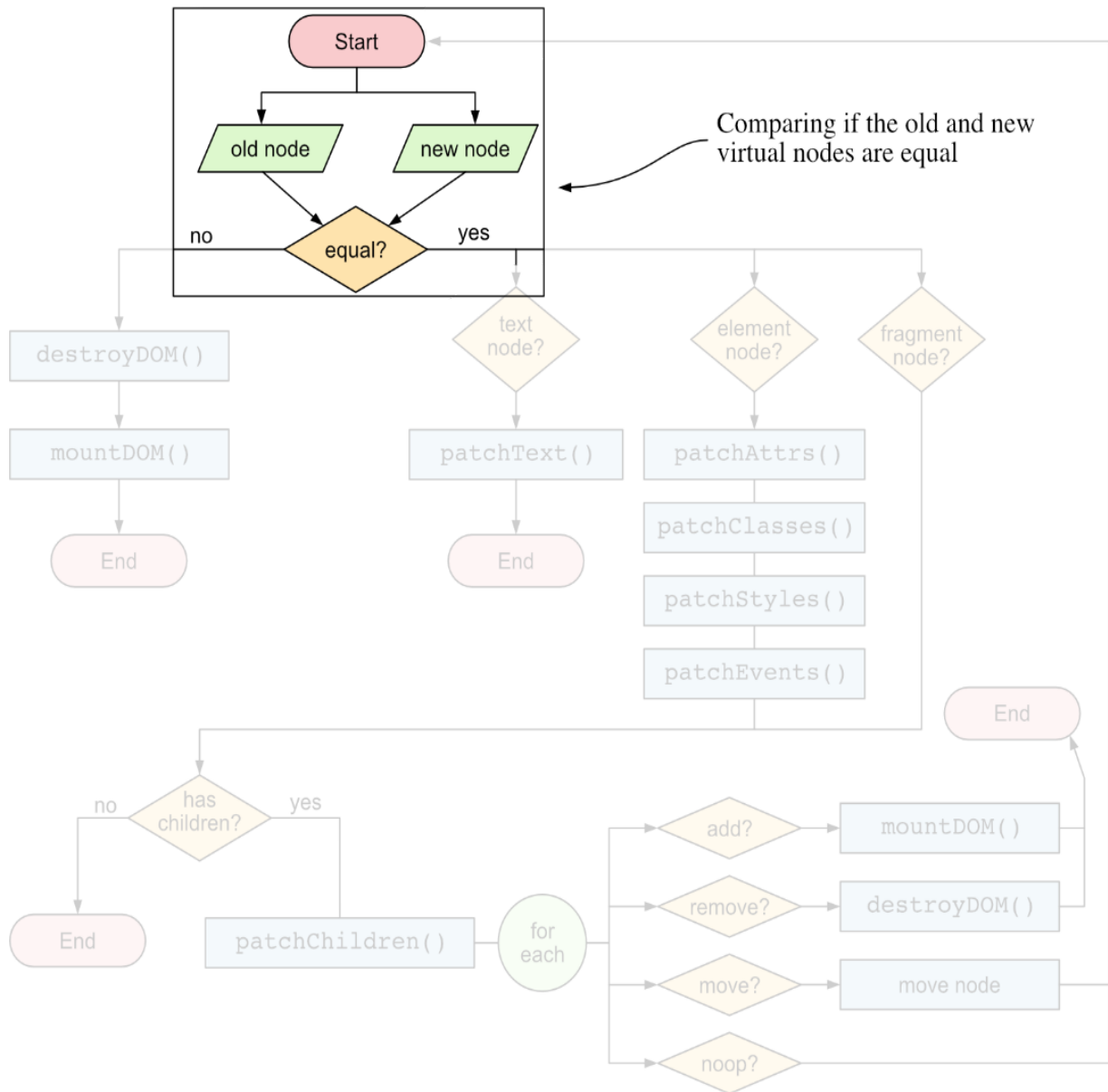
- Two text nodes are always equal because their text is something that can be patched, so there's no need to destroy it and mount a new node with a different text.
- Fragment nodes always equal because they're just containers for other nodes and have no properties of their own.
- The element node is the most interesting case: for two element nodes to be equal, their tag must be the same. You can't programmatically change the tag of a `<div>` to a `<span>` and expect the browser to render it correctly; these are two different objects (`HTMLDivElement` and `HTMLSpanElement`, to be more precise) that have different properties.

Note

You could definitely change the tag of a `<div>` to a `<span>` in the HTML source using the browser's developer tools, and that would work perfectly fine. But what's happening under the hood is that the browser is destroying the old `HTMLDivElement` and creating a new `HTMLSpanElement`. That's exactly what you need to do in your code using the Document API if you compare two element nodes and find the tag changed: destroy the node with the old tag and mount a new one with the new tag.

With this in mind, let's implement a function to check if two nodes are equal: `nodesEqual()`. If you look at the algorithm's flowchart [8.7](#), the equality check happens at the top, when the old and new nodes enter the algorithm (highlighted in the figure).

Figure 8.7 Checking if the old and new nodes are equal in the algorithm's flowchart



To create a function to check if two nodes are equal, create a new file called *nodes-equal.js* and write the code in listing 8.6. The `areNodesEqual()` function applies the aforementioned rules to check if two nodes are equal.

Listing 8.6 Comparing two nodes for equality (*nodes-equal.js*)

```

import { DOM_TYPES } from './h'

export function areNodesEqual(nodeOne, nodeTwo) {
  if (nodeOne.type !== nodeTwo.type) { #1
    return false
  }

```

```

    }

    if (nodeOne.type === DOM_TYPES.ELEMENT) {
        const { tag: tagOne } = nodeOne
        const { tag: tagTwo } = nodeTwo

        return tagOne === tagTwo #2
    }

    return true
}

```

Exercise 8.2

Paste the `areNodesEqual()` function into the browser's console (don't forget to also include the `DOM_TYPES` constant) and test it with the following cases:

- Two text nodes with the same text.
- Two text nodes with different text.
- An element node and a text node.
- A `<p>` element node and a `<div>` element node.
- Two `<p>` element nodes with different text content.

What are the results?

Find the solution at <http://mng.bz/YRAe>.

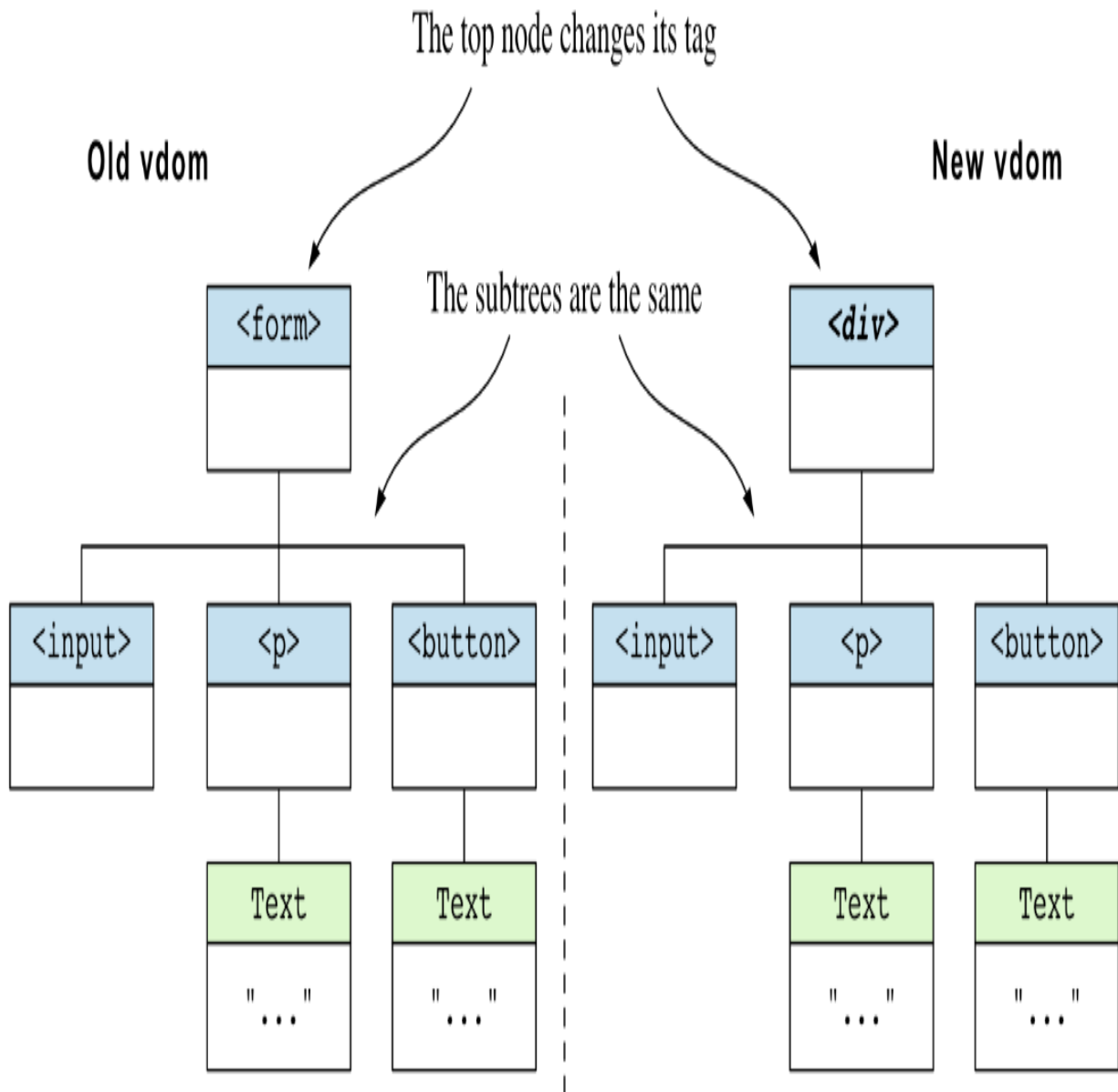
Now that you have a function to check if two nodes are equal, you can use it in the algorithm to decide whether to destroy the old node and mount the new one. This was our second question: why do you have to destroy the whole subtree? Let's look into that.

8.2.3 Subtree change

Destroying the whole subtree and recreating it when you detect the node changed sounds like going back to when your framework removed the whole DOM tree and mounted a new one. Isn't that what we're trying to avoid with the reconciliation algorithm? Couldn't you just compare their children to

check if at least the subtrees are the same so that you need to recreate only the nodes that are different? This is illustrated in figure [8.8](#).

Figure 8.8 The top node's tag changed, but the subtrees are the same

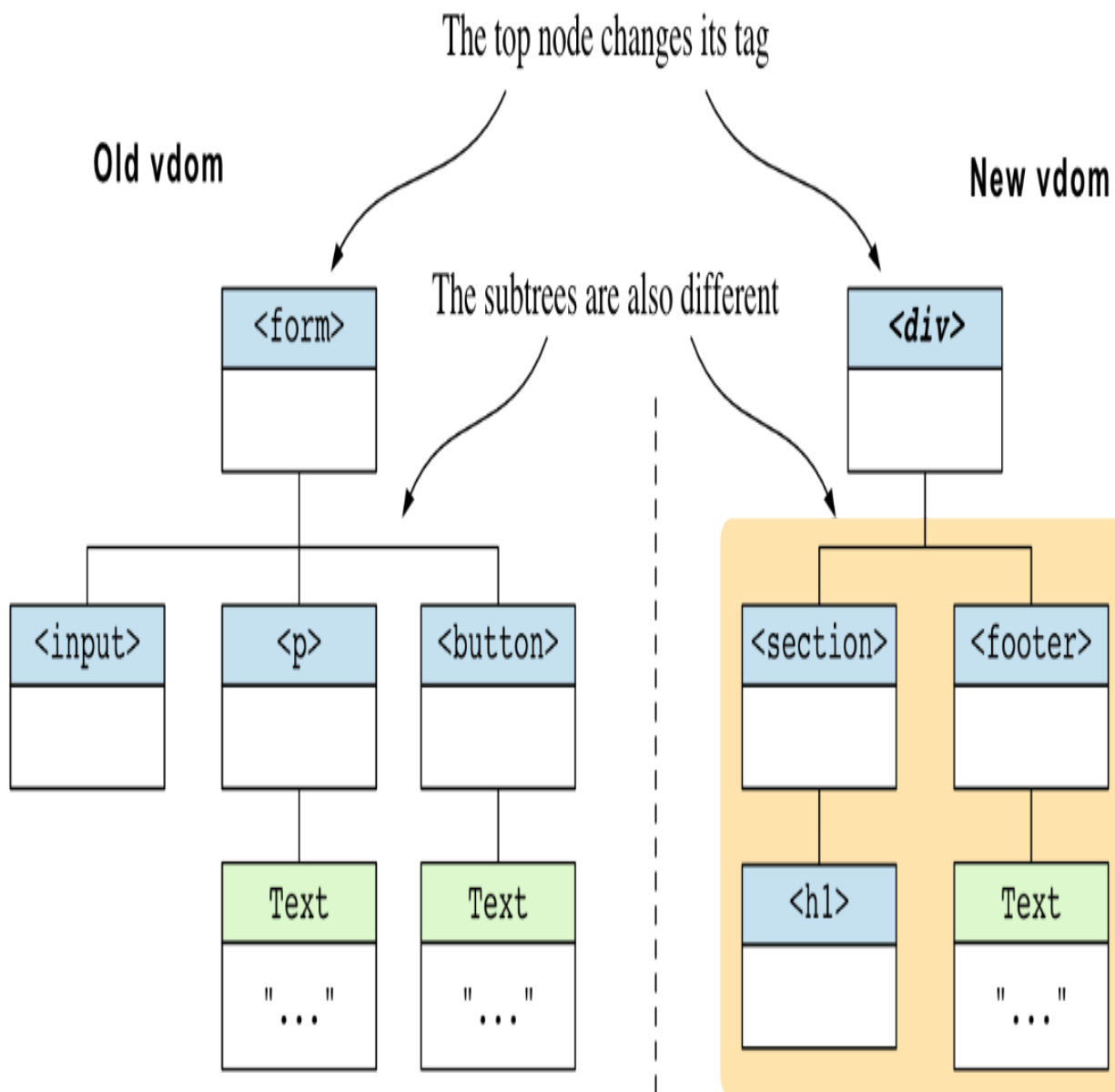


In figure [8.8](#), the top nodes—the ones you're comparing—are different, but their subtrees are the same. You could patch just the top node and leave the subtrees alone. If the children are different, you can patch only them with the help of the `arraysDiffSequence()` function.

What happens in general is that, when you detect that a node has changed, it usually means that a new part of the view enters the scene while the old one leaves. This new part of the view typically has a different subtree below it, completely unrelated to the old subtree. If you decided to patch the top node and then start comparing the two subtrees, you'd find a lot of differences, because the new subtree is completely different from the old one. You'd end up doing a lot of comparisons and DOM operations that are equivalent to destroying the whole subtree and recreating it from scratch, only using many more operations.

So, a more realistic situation than the one in figure [8.8](#) is the one in figure [8.9](#), where you can see that when a parent node changes, their subtrees are likely to be completely different. When you go from having a `<form>` to having a `<div>`, you can probably anticipate that there will be something different below it. And if it's not the case, if the parent node changes but the subtree is the same, destroying and recreating the whole subtree is a price you're willing to pay for the simplicity of the algorithm.

Figure 8.9 A more realistic case where the top nodes are different and so are their subtrees



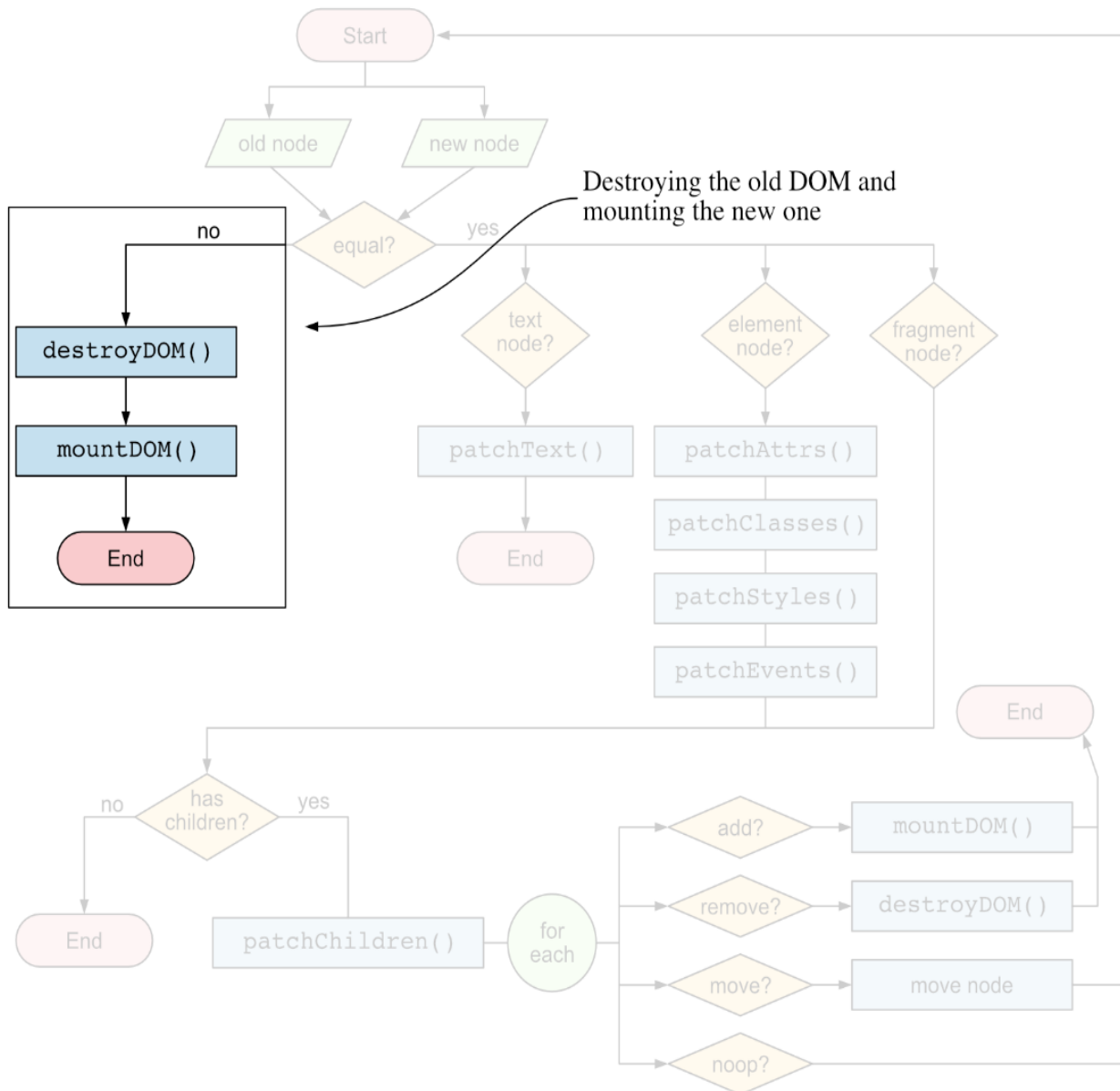
Having illustrated why it's better to simply destroy the whole subtree and re-create it when the top virtual nodes being compared are different, let's implement the algorithm. You'll start writing the `patchDOM()` function, which is the main entry point of the algorithm. You'll add more cases to the function as we go, but for now, you'll just implement the case where the top nodes are different—the easiest one to implement.

You first find the index of the old node's element (referenced by the `e1` property of the virtual node) in its DOM's parent node (referenced by the `parentEl` property), destroy it, and then mount the new node in the same

place. To find this index, you can use the `indexOf()` method of the parent element's list of child nodes (`parentEl.childNodes`), passing it the old node's DOM element. One case to bear in mind, though, is when the index returned by `indexOf()` is `-1`, which means that the old node's DOM element is not among the parent children. This case when the old node is a fragment, where the `el` property references the parent element; you can't find an element inside itself. For this case, we want to just use a `null` index, which means the new vdom will be appended at the end of the parent node's list of child nodes.

Figure [8.10](#) highlights this step in the algorithm's flowchart.

Figure 8.10 Replacing the old DOM with the new DOM



Create a new file called *patch-dom.js* and write the code in listing [8.7](#).

Listing 8.7 Patching the DOM when the top nodes are different (*patch-dom.js*)

```

import { destroyDOM } from './destroy-dom'
import { mountDOM } from './mount-dom'
import { areNodesEqual } from './nodes-equal'

export function patchDOM(oldVdom, newVdom, parentEl) {
  if (!areNodesEqual(oldVdom, newVdom)) {
    const index = findIndexInParent(parentEl, oldVdom.el) #1
    destroyDOM(oldVdom) #2
  }
}

```

```

    mountDOM(newVdom, parentEl, index) #3

    return newVdom
  }
}

function findIndexInParent(parentEl, el) {
  const index = Array.from(parentEl.childNodes).indexOf(el)
  if (index < 0) {
    return null
  }

  return index
}

```

Great—You’ve implemented the first case of the algorithm. Let’s now continue with the case where the top nodes are text nodes.

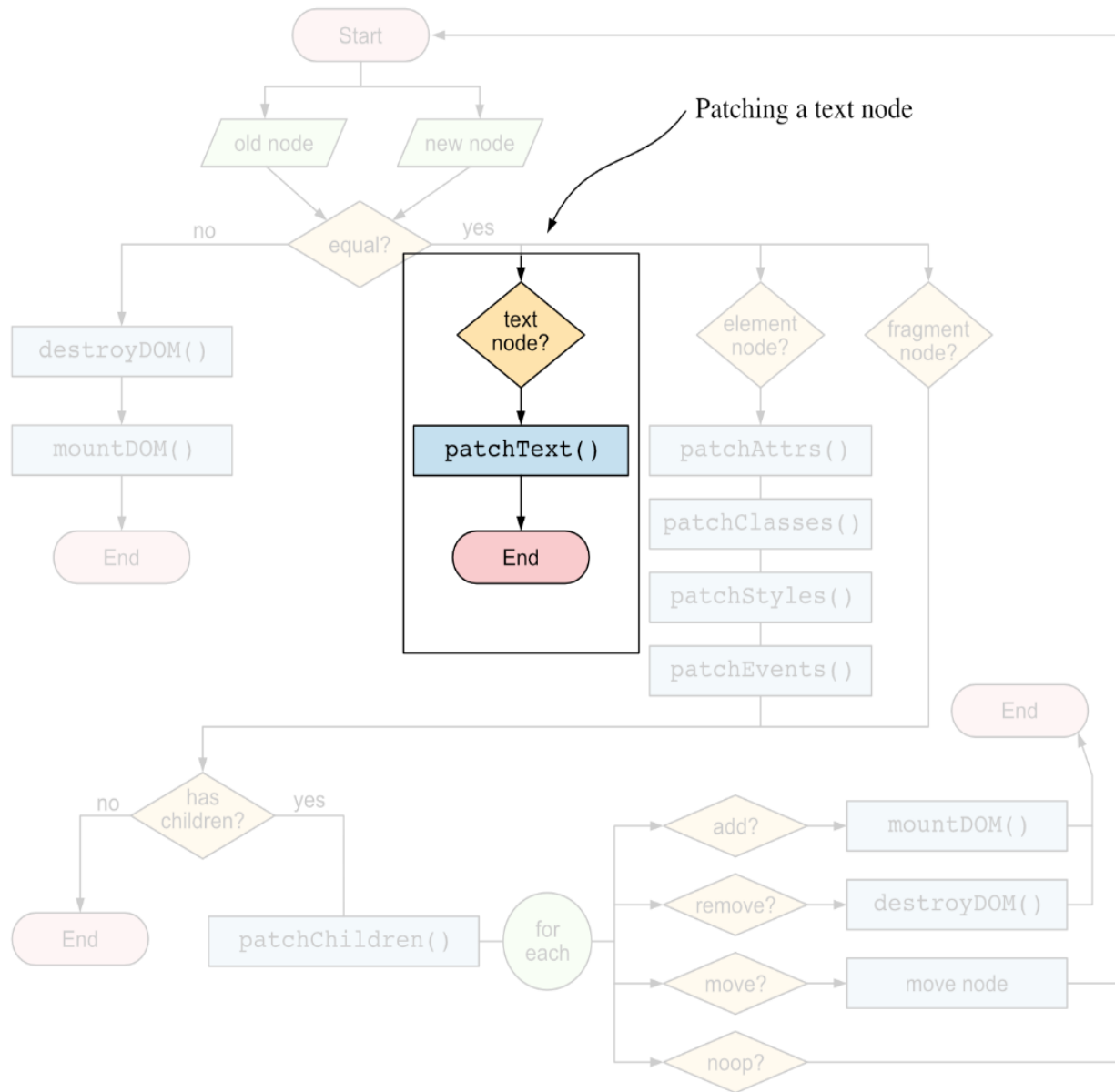
8.2.4 Patching text nodes

Patching text nodes is easy: you just need to set the `nodeValue` property of the DOM node to the new text, in the case where this is different than the old one. If you find that the text content in both virtual nodes is the same, you don’t need to do anything.

As you can imagine, the first thing you need to do in the `patchDOM()` function—just after the `areNodesEqual()` comparison from the previous section—is check whether the nodes are text nodes. If so, you call a function to patch the text node, `patchText()`, which you need to implement. You can use a `switch` statement to check the type of the nodes and act accordingly.

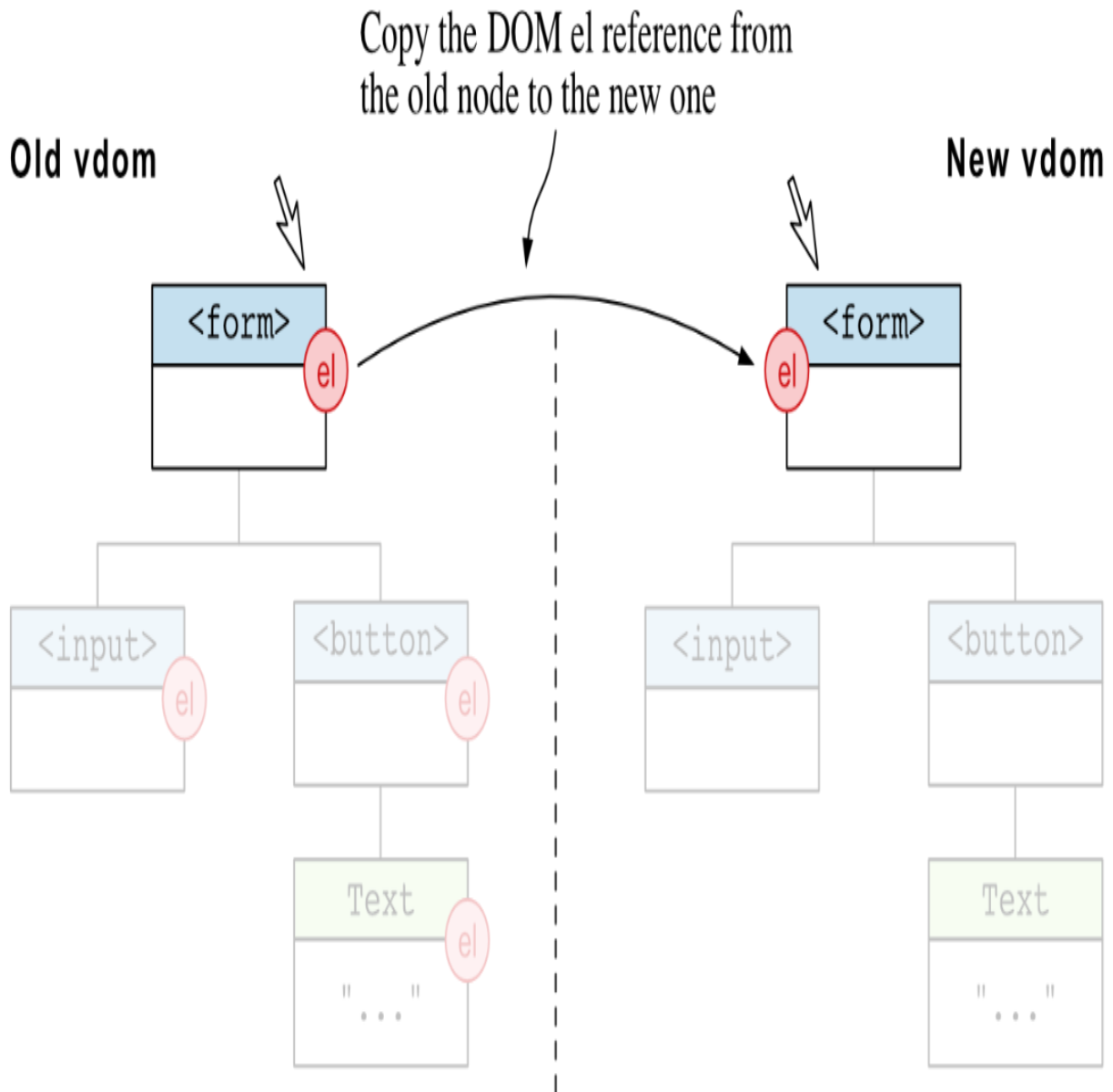
Figure [8.11](#) illustrates where the patching of a text node is in the algorithm’s flowchart.

Figure 8.11 Patching a text node



If you remember, when you implemented the `mountDOM()` function, you saved the reference to the DOM element representing the virtual node in the `el` property of the virtual node. Now, in the `patchDOM()` function, in the cases where the nodes aren't created but simply patched, you need to save this reference from `oldVdom` to `newVdom` so that you don't lose it. This is illustrated in figure [8.12](#).

Figure 8.12 Copying the DOM element reference from the old virtual node to the new one



Modify the `patchDOM()` function, adding the code in bold font in listing [8.8](#).

Listing 8.8 The case of a text node (*patch-dom.js*)

```
import { destroyDOM } from './destroy-dom'
import { DOM_TYPES } from './h'
import { mountDOM } from './mount-dom'
import { areNodesEqual } from './nodes-equal'

export function patchDOM(oldVdom, newVdom, parentEl) {
  if (!areNodesEqual(oldVdom, newVdom)) {
    const index = findIndexInParent(parentEl, oldVdom.el)
```

```

    destroyDOM(oldVdom)
    mountDOM(newVdom, parentEl, index)

    return newVdom
}

newVdom.el = oldVdom.el #1

switch (newVdom.type) {
  case DOM_TYPES.TEXT: {
    patchText(oldVdom, newVdom) #2
    return newVdom #3
  }
}

return newVdom
}

```

Let's now write the `patchText()` function. This function compares the texts in the `nodeValue` property of the old and new virtual nodes. If they are different (read: the text has changed), set the `nodeValue` property of the DOM element to the new text.

Inside the *patch-dom.js* file, write the code in listing [8.9](#).

Listing 8.9 Patching a text node (*patch-dom.js*)

```

function patchText(oldVdom, newVdom) {
  const el = oldVdom.el
  const { value: oldText } = oldVdom
  const { value: newText } = newVdom

  if (oldText !== newText) {
    el.nodeValue = newText
  }
}

```

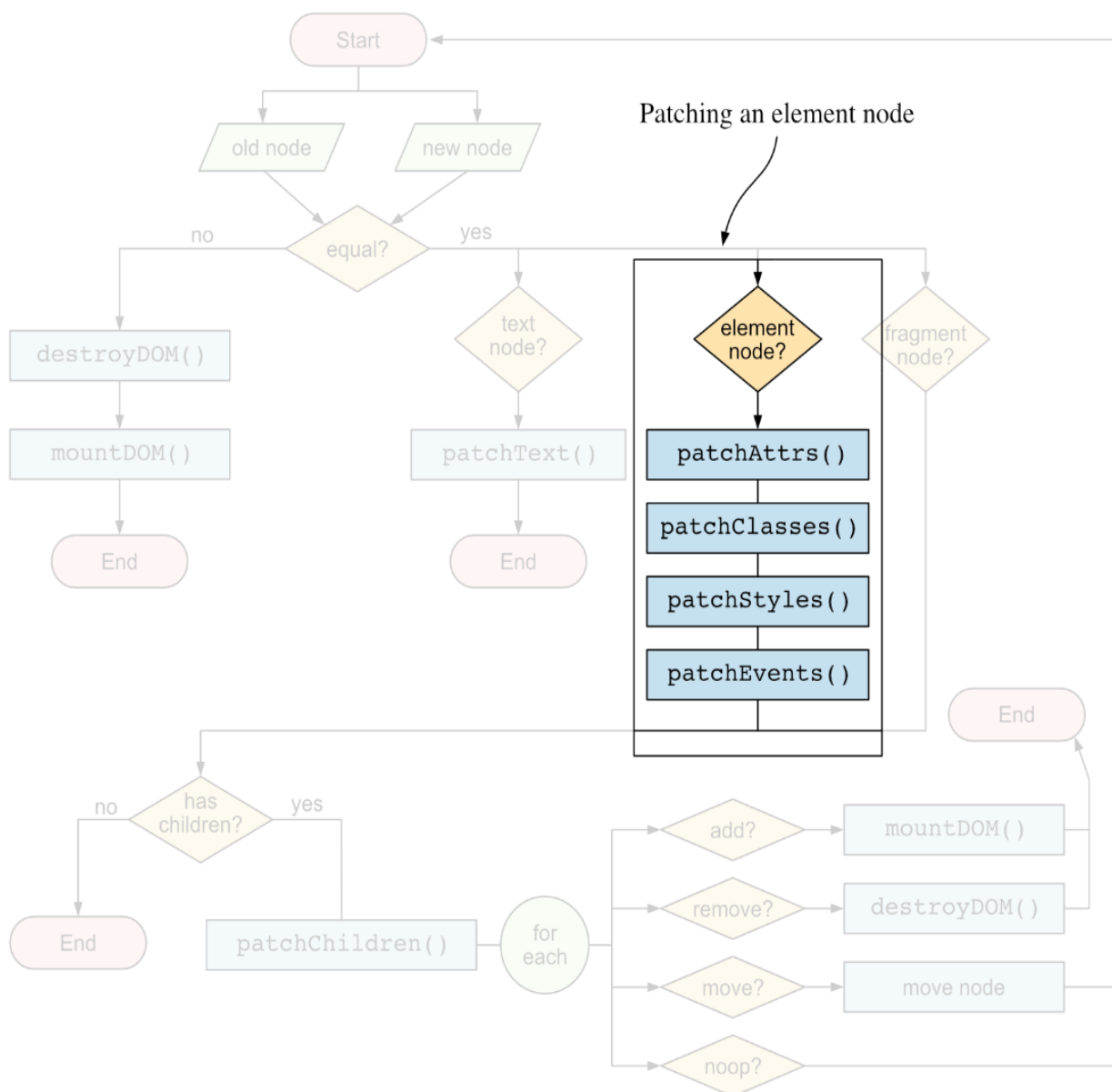
Note how, in the first line of the function, you're extracting the DOM element from the `oldVdom` virtual node's `el` property, but you could have also used the `newVdom` virtual node's reference, because before calling this function, `patchDOM()` has already saved it there. (At this point `oldVdom.el === newVdom.el` must be true.)

Let's now move on to the case where the top nodes are element nodes. This is the most interesting—as well as complex—case.

8.2.5 Patching element nodes

You can see where the patching of an element node is in the algorithm's flowchart diagram in figure 8.13. As you can observe, patching an element node requires a couple of different steps—four, to be precise.

Figure 8.13 Patching an element node



Let's start by including the appropriate `switch` case in the `patchDOM()` function. Then, I'll explain how to write the `patchElement()` function; the one that does all the work. Add the code in bold font to *patch-dom.js* in listing [8.10](#). (As you can see, the work of patching element nodes is delegated to the `patchElement()` function.)

Listing 8.10 The case of an element node (*patch-dom.js*)

```
import { destroyDOM } from './destroy-dom'
import { DOM_TYPES } from './h'
import { mountDOM } from './mount-dom'
import { areNodesEqual } from './nodes-equal'

export function patchDOM(oldVdom, newVdom, parentEl) {
  if (!areNodesEqual(oldVdom, newVdom)) {
    const index = findIndexInParent(parentEl, oldVdom.el)
    destroyDOM(oldVdom)
    mountDOM(newVdom, parentEl, index)

    return newVdom
  }

  newVdom.el = oldVdom.el

  switch (newVdom.type) {
    case DOM_TYPES.TEXT: {
      patchText(oldVdom, newVdom)
      return newVdom
    }

    case DOM_TYPES.ELEMENT: {
      patchElement(oldVdom, newVdom)
      break
    }
  }

  return newVdom
}
```

Element nodes are more complex than text nodes, because they have attributes, styles, event handlers, and can have children. You'll take care of patching the children of a node in the next subsection, which incidentally covers the case of fragment nodes (these nodes are no more than an array of

children nodes). But you'll focus on the attributes, styles (including CSS classes), and event handlers in this section.

The `patchElement()` function is in charge of extracting the attributes, CSS classes and styles, and event handlers from the old and new virtual nodes, and then passing them to the appropriate functions to patch them. You'll write functions to patch each of these separately, because they follow different rules:

- `patchAttrs()`: patches the attributes (such as `id`, `name`, `value`, and so on)
- `patchClasses()`: patches the CSS class names
- `patchStyles()`: patches the CSS styles
- `patchEvents()`: patches the event handlers, and returns an object with the current event handlers

The event handlers returned by the `patchEvents()` function should be saved in the `listeners` property of the new virtual node, so that you can remove them later. Recall that, when you implemented the `mountDOM()` function, you saved the event handlers in the `listeners` property of the virtual node.

Write the code for the `patchElement()` function in listing [8.11](#), inside the *patch-dom.js* file. (You don't need to include the `// TODO` comments in the code listing; they're just there to remind you of what you of the functions you need to write next.)

Listing 8.11 Patching an element node (*patch-dom.js*)

```
function patchElement(oldVdom, newVdom) {
  const el = oldVdom.el
  const {
    class: oldClass,
    style: oldStyle,
    on: oldEvents,
    ...oldAttrs
  } = oldVdom.props
  const {
    class: newClass,
    style: newStyle,
    on: newEvents,
    ...newAttrs
  }
```

```

    } = newVdom.props
    const { listeners: oldListeners } = oldVdom

    patchAttrs(el, oldAttrs, newAttrs)
    patchClasses(el, oldClass, newClass)
    patchStyles(el, oldStyle, newStyle)
    newVdom.listeners = patchEvents(el, oldListeners,
oldEvents, newEvents)
  }

  // TODO: implement patchAttrs()

  // TODO: implement patchClasses()

  // TODO: implement patchStyles()

  // TODO: implement patchEvents()

```

Now that you’ve broken down the work of patching element nodes into smaller tasks, let’s start with the first one: patching attributes.

Patching attributes

The attributes of a virtual node are all of the key-value pairs that come inside its `props` object—except for the `class`, `style`, and `on` properties, which have a special meaning. Now, given the two objects containing the attributes of the old and new virtual nodes (`oldAttrs` and `newAttrs`, respectively), you need to find out which attributes have been added, removed, or changed. You wrote a function in the last chapter that does exactly that: `objectsDiff()`.

The `objectsDiff()` function tells you which attributes have been added, removed, or changed. You can get rid of the attributes that have been removed using the `removeAttribute()` function that you wrote earlier in the book (inside the *attributes.js* file). The attributes that have been added or changed can be set using the `setAttribute()` function, that you also wrote earlier.

Write the code for the `patchAttrs()` function in listing [8.12](#) at the bottom of the *patch-dom.js* file. (Don’t forget to include the new `import` statements at the top of the file.)

Listing 8.12 Patching the attributes (*patch-dom.js*)

```

import {
  removeAttribute,
  setAttribute,
} from './attributes'
import { destroyDOM } from './destroy-dom'
import { DOM_TYPES } from './h'
import { mountDOM } from './mount-dom'
import { areNodesEqual } from './nodes-equal'
import { objectsDiff } from './utils/objects'

// --snip-- //

function patchAttrs(el, oldAttrs, newAttrs) {
  const { added, removed, updated } = objectsDiff(oldAttrs,
newAttrs) #1

  for (const attr of removed) {
    removeAttribute(el, attr) #2
  }

  for (const attr of added.concat(updated)) {
    setAttribute(el, attr, newAttrs[attr]) #3
  }
}

```

Let's now move on to patching CSS classes.

Patching CSS classes

The tricky part—but's not that tricky, really—about patching the CSS classes is that they can come as a string (for example: 'foo bar'), or as an array of strings (for example: ['foo', 'bar']). Here's what you'll do:

- If the CSS classes come in an array, filter the blank or empty strings out of it, and keep them as an array.
- If the CSS classes come as a string, split it on whitespace, and filter the blank or empty strings out of it.

This way, you work with two arrays of strings representing the CSS classes of the old and new virtual DOMs. You can then use the `arraysDiff()` function you implemented in the previous chapter to find out which CSS classes have been added and removed. The DOM element has a `classList` property (an

instance of the `DOMTokenList` interface) that you can use to add and remove CSS classes from the element. Add the new classes using the `classList.add()` method, and remove the old ones using the `classList.remove()` method. The classes that were neither added nor removed don't need to be touched—they can stay as they are.

Write the code for the `patchClasses()` function in listing [8.13](#), at the bottom of the *patch-dom.js* file. Again, don't forget to include the new `import` statements at the top of the file, and this time, pay attention to a function that you need to import, `isNotBlankOrEmptyString()`, that you'll write in a second.

Listing 8.13 Patching the CSS classes (*patch-dom.js*)

```
import {
  removeAttribute,
  setAttribute,
} from './attributes'
import { destroyDOM } from './destroy-dom'
import { DOM_TYPES } from './h'
import { mountDOM } from './mount-dom'
import { areNodesEqual } from './nodes-equal'
import {
  arraysDiff,
} from './utils/arrays'
import { objectsDiff } from './utils/objects'
import { isNotBlankOrEmptyString } from './utils/strings'

// --snip--

function patchClasses(el, oldClass, newClass) {
  const oldClasses = toClassList(oldClass) #1
  const newClasses = toClassList(newClass) #2

  const { added, removed } = arraysDiff(oldClasses,
newClasses) #3

  if (removed.length > 0) {
    el.classList.remove(...removed) #4
  }
  if (added.length > 0) {
    el.classList.add(...added) #5
  }
}
```

```

}

function toClassList(classes = '') {
  return Array.isArray(classes)
    ? classes.filter(isNotBlankOrEmptyString) #6
    : classes.split(/\s+/).filter(isNotBlankOrEmptyString)
#7
}

```

You're missing the `isNotBlankOrEmptyString()` function, which you need to write in the *utils/strings.js* file. This function takes a string as an argument, and returns `true` if the string is neither blank nor empty, and `false` otherwise. It's helpful to split that function: `isNotEmptyString()` to test if a string is empty, and `isNotBlankOrEmptyString()` that uses the former, passing it a trimmed version of the string.

Create a new file under the *utils* directory, called *strings.js*, and write the code for the `isNotBlankOrEmptyString()` function in listing [8.14](#).

Listing 8.14 Filtering empty and blank strings (*utils/strings.js*)

```

export function isNotEmptyString(str) {
  return str !== ''
}

export function isNotBlankOrEmptyString(str) {
  return isNotEmptyString(str.trim())
}

```

Let's move on to patching the style.

Patching the style

Patching the style is similar to patching the attributes: you compare the old and new style objects (using the `objectsDiff()` function), and then you add the new or modified styles, and remove the old ones. To set or remove styles, you can use the `setStyle()` and `removeStyle()` functions that you wrote earlier in the book.

Inside the *patch-dom.js* file, write the code for the `patchStyle()` function in listing [8.15](#). (Don't forget to import the `removeStyle()` and `setStyle()`

functions at the top of the file.)

Listing 8.15 Patching the styles (*patch-dom.js*)

```
import {
  removeAttribute,
  setAttribute,
  removeStyle,
  setStyle,
} from './attributes'
import { destroyDOM } from './destroy-dom'
import { DOM_TYPES } from './h'
import { mountDOM } from './mount-dom'
import { areNodesEqual } from './nodes-equal'
import {
  arraysDiff,
} from './utils/arrays'
import { objectsDiff } from './utils/objects'
import { isNotBlankOrEmptyString } from './utils/strings'

// --snip-- //

function patchStyles(el, oldStyle = {}, newStyle = {}) {
  const { added, removed, updated } = objectsDiff(oldStyle,
newStyle)

  for (const style of removed) {
    removeStyle(el, style)
  }

  for (const style of added.concat(updated)) {
    setStyle(el, style, newStyle[style])
  }
}
```

And last, let's implement the event listeners patching.

Patching event listeners

The `patchEvents()` function is a bit different in that it has an extra parameter, `oldListeners` (the second one), which is an object containing the event listeners that are currently attached to the DOM. Let's see why this is necessary.

If you recall, when you implemented the `mountDOM()` function, you wrote a function called `addEventListener()` to attach event listeners to the DOM. It currently uses the function you pass it as the event listener, but remember that we said this would change. Indeed, later in the book, this function will wrap the event listener you pass it into a new function, and that is the function it'll use as event listener, very similar to the following:

```
function addEventListener(eventName, handler, el) {  
  // Function that wraps the original handler  
  function boundHandler(event) { // -- snip --  
    handler(event) }  
  
  el.addEventListener(eventName, boundHandler)  
  
  return boundHandler  
}
```

You will understand why you want to do this soon, but for now, just know that the functions defined in the virtual DOM to handle events are not the same as the functions that are attached to the DOM. This is the reason why the `patchEvents()` function needs the `oldListeners` object—the function's actually attached to the DOM. As you know, to remove an event listener from a DOM element, you call its `removeEventListener()` method, passing it the name of the event and the function that you want to remove, so we need the functions that were used to attach the event listeners to the DOM.

The second difference of the `patchEvents()` function with respect to the previous ones is that it returns an object containing the event names and handler functions that have been added to the DOM. (The other functions didn't return anything.) You save this object in the `listeners` key of the new virtual DOM node. You use them later, to remove the event listeners from the DOM when the view is destroyed.

For the rest, it's just a matter of using the `objectsDiff()` function to find out which event listeners have been added, modified, or removed, and then calling the `addEventListener()` function and `el.removeEventListener()` method correspondingly. In this case though, when an event listener has been modified (that is, the event name is the same but the handler function is different), you need to remove the old event listener and then add the new one. Keep this detail in mind.

There are two very important things you should pay attention to in the code. The first one is that, you use the function in `oldListeners[eventName]` to remove the event listener, and not the function in `oldEvents[eventName]`. I already explained why this is, but's important that you write this line of code correctly for your framework to work properly:

```
el.removeEventListener(eventName, oldListeners[eventName])
```

The second thing is that you use your own implementation of the `addEventListener()` function to add the event listeners to the DOM, and not the `el.addEventListener()` method of the DOM element. To remove the event listeners, you use the `el.removeEventListener()` method.

Write the code for the `patchEvents()` function in listing [8.16](#). (Don't forget importing the `addEventListener` function at the top of the file.)

Listing 8.16 Patching the event listeners (*patch-dom.js*)

```
import {
  removeAttribute,
  setAttribute,
  removeStyle,
  setStyle,
} from './attributes'
import { destroyDOM } from './destroy-dom'
import { addEventListener } from './events'
import { DOM_TYPES } from './h'
import { mountDOM } from './mount-dom'
import { areNodesEqual } from './nodes-equal'
import {
  arraysDiff,
} from './utils/arrays'
import { objectsDiff } from './utils/objects'
import { isNotBlankOrEmptyString } from './utils/strings'

// --snip--

function patchEvents(
  el,
  oldListeners = {},
  oldEvents = {},
  newEvents = {}
) {
```



```

    const { removed, added, updated } = objectsDiff(oldEvents,
newEvents) #1

    for (const eventName of removed.concat(updated)) {
        el.removeEventListener(eventName,
oldListeners[eventName]) #2
    }

    const addedListeners = {} #3

    for (const eventName of added.concat(updated)) {
        const listener = addEventListener(eventName,
newEvents[eventName], el) #4
        addedListeners[eventName] = listener #5
    }

    return addedListeners #6
}

```

You've implemented the `patchElement()` function, which is the function that patches the DOM element and required four other functions to do so:

`patchAttrs()`, `patchClasses()`, `patchStyles()`, and `patchEvents()`.

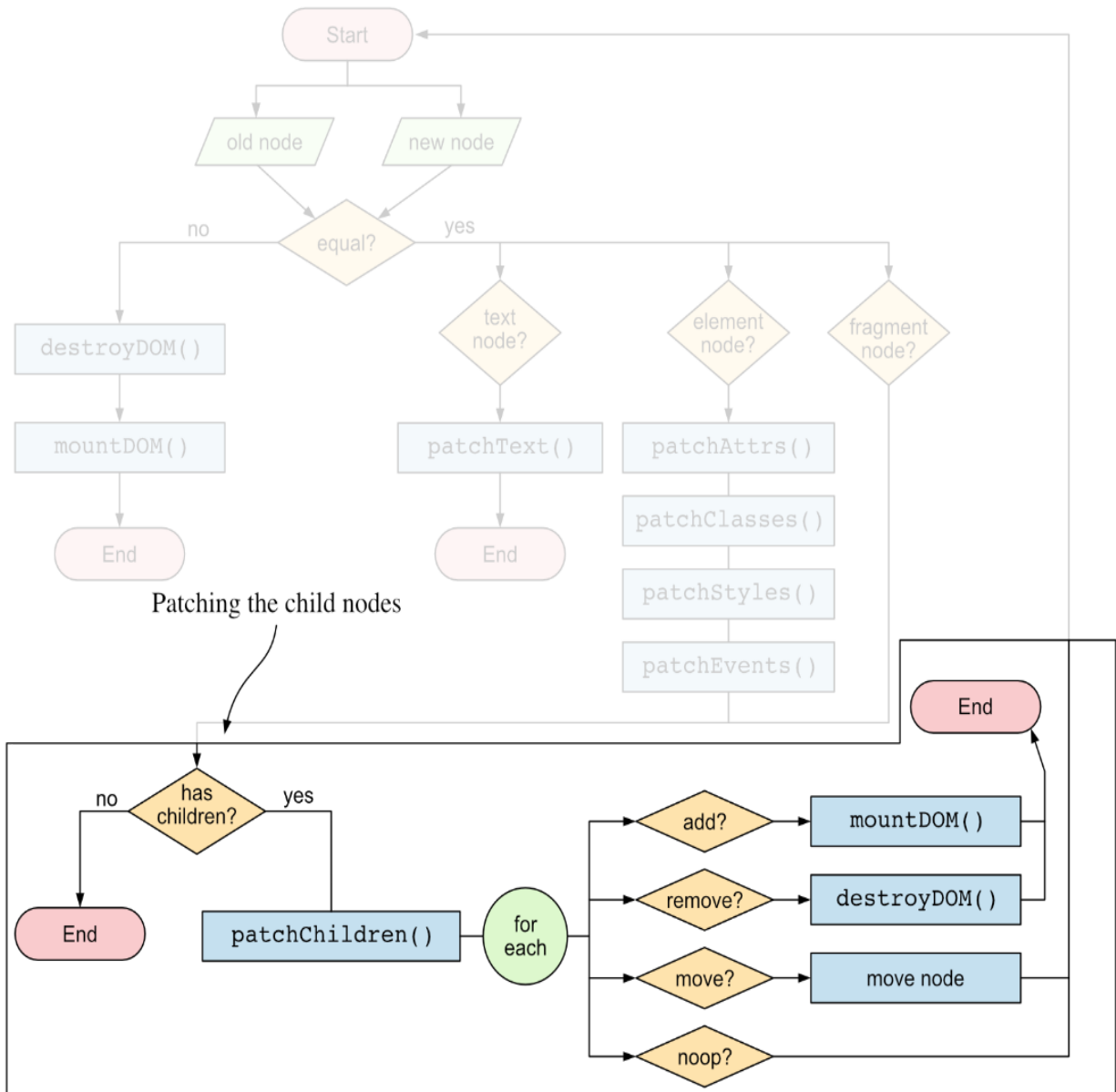
Believe it or not, you're missing only patching the children of a node to have a complete implementation of the `patchDOM()` function. Let's do that now.

8.2.6 Patching child nodes

Both element and fragment nodes can have children, so the `patchDOM()` function needs to patch the children arrays of both types of nodes. Patching the children means figuring out which children have been added, which ones have been removed, and which ones have been shuffled around. For this, you implemented the `arraysDiffSequence()` in the previous chapter.

You can see in figure [8.14](#) where the patching of children is in the algorithm's flowchart. Remember that, in the case of *move* and *noop* operations between children, the algorithm recursively calls the `patchDOM()` function, passing it the old and new children nodes. The *add* and *remove* operation cases terminate the algorithm.

Figure 8.14 Patching the children of a node



Start by modifying the `patchDOM()` function to call the `patchChildren()` function after the `switch` statement. This will execute the `patchChildren()` function for all types of elements except the text nodes, which return early from the `patchDOM()` function.

Modify the `patchDOM()` function by adding the code in bold font in listing [8.17](#). This includes importing `arraysDiffSequence` and `ARRAY_DIFF_OP` from the `utils/arrays.js` file, and calling the `patchChildren()` function.

Listing 8.17 Patching the children of a node (*patch-dom.js*)

```

import {
  removeAttribute,
  setAttribute,
  removeStyle,
  setStyle,
} from './attributes'
import { destroyDOM } from './destroy-dom'
import { addEventListener } from './events'
import { DOM_TYPES } from './h'
import { mountDOM } from './mount-dom'
import { areNodesEqual } from './nodes-equal'
import {
  arraysDiff,
  arraysDiffSequence,
  ARRAY_DIFF_OP,
} from './utils/arrays'
import { objectsDiff } from './utils/objects'
import { isNotBlankOrEmptyString } from './utils/strings'

export function patchDOM(oldVdom, newVdom, parentEl) {
  if (!areNodesEqual(oldVdom, newVdom)) {
    const index = findIndexInParent(parentEl, oldVdom.el)
    destroyDOM(oldVdom)
    mountDOM(newVdom, parentEl, index)

    return newVdom
  }

  newVdom.el = oldVdom.el

  switch (newVdom.type) {
    case DOM_TYPES.TEXT: {
      patchText(oldVdom, newVdom)
      return newVdom
    }

    case DOM_TYPES.ELEMENT: {
      patchElement(oldVdom, newVdom)
      break
    }
  }

  patchChildren(oldVdom, newVdom)

  return newVdom
}

// TODO: implement patchChildren()

```

The `patchChildren()` function extracts the children arrays from the old and new nodes (or use an empty array in their absence), and then calls the `arraysDiffSequence()` function to find the operations that transform the old array into the new one. (Let's not forget that this function requires the `areNodesEqual()` function to compare the nodes in the arrays.) Then, for each operation, it performs the appropriate patching, as we'll see in a minute.

You first need a function, which you can call `extractChildren()`, that extracts the children array from a node, such that, if it encounters a fragment node, it extracts the children of the fragment node and adds them to the array. This function needs to be recursive, so that if a fragment node contains another fragment node, it also extracts the children of the inner fragment node.

Inside the *h.js* file, where the virtual node creation functions are defined, write the code for the `extractChildren()` function in listing [8.18](#).

Listing 8.18 Extracting the children of a node (*h.js*)

```
export function extractChildren(vdom) {
  if (vdom.children == null) { #1
    return []
  }

  const children = []

  for (const child of vdom.children) { #2
    if (child.type === DOM_TYPES.FRAGMENT) {
      children.push(...extractChildren(child, children)) #3
    } else {
      children.push(child) #4
    }
  }

  return children
}
```

With this function ready, you can now write the code for the `patchChildren()` function in listing [8.19](#). (Don't forget importing the `extractChildren()` function at the top of the file.)

Listing 8.19 Implementing the `patchChildren()` function (*patch-dom.js*)

```

import {
  removeAttribute,
  setAttribute,
  removeStyle,
  setStyle,
} from './attributes'
import { destroyDOM } from './destroy-dom'
import { addEventListener } from './events'
import { DOM_TYPES } from './h'
import { mountDOM, extractChildren } from './mount-dom'
import { areNodesEqual } from './nodes-equal'
import {
  arraysDiff,
  arraysDiffSequence,
  ARRAY_DIFF_OP,
} from './utils/arrays'
import { objectsDiff } from './utils/objects'
import { isNotBlankOrEmptyString } from './utils/strings'

// --snip-- //

```

```

function patchChildren(oldVdom, newVdom) {
  const oldChildren = extractChildren(oldVdom) #1
  const newChildren = extractChildren(newVdom) #2
  const parentEl = oldVdom.el

  const diffSeq = arraysDiffSequence( #3
    oldChildren,
    newChildren,
    areNodesEqual
  )

  for (const operation of diffSeq) { #4
    const { originalIndex, index, item } = operation

    switch (operation.op) { #5
      case ARRAY_DIFF_OP.ADD: {
        // TODO: implement
      }

      case ARRAY_DIFF_OP.REMOVE: {
        // TODO: implement
      }

      case ARRAY_DIFF_OP.MOVE: {
        // TODO: implement
      }
    }
  }
}

```

```
        case ARRAY_DIFF_OP.NOOP: {
            // TODO: implement
        }
    }
}
```

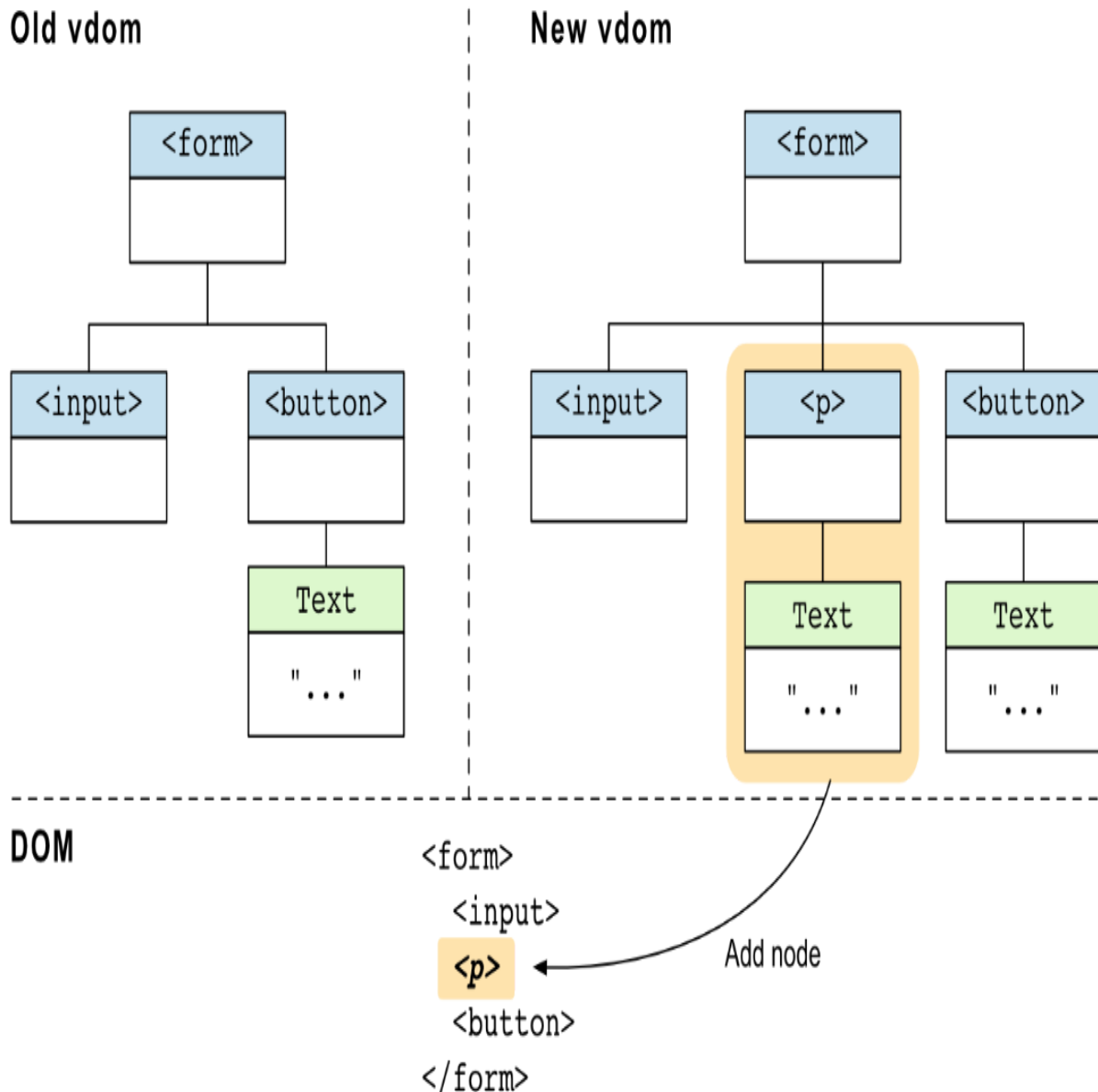
All that's left to do is fill in the `switch` statement with the code for each operation (where the `// TODO` comments are). This will be simpler than you think. Let's start with the `ARRAY_DIFF_OP.ADD` operation.

Add operation

When a new node is added to the children array, it's as if you were mounting a subtree of the DOM at a specific place. Thus, you can simply use the `mountDOM()` function to do this, passing it the `index` at which the new node should be inserted.

You can see in figure [8.15](#) how when a node addition is detected (an operation of type `ARRAY_DIFF_OP.ADD`), it's inserted into the DOM.

Figure 8.15 Adding a node to the children array



Write the code in bold font in listing [8.20](#) to implement the first case of the `switch` statement.

Listing 8.20 Patching the children—adding a node (*patch-dom.js*)

```
function patchChildren(oldVdom, newVdom) {
  // --snip-- //

  for (const operation of diffSeq) {
    const { from, index, item } = operation
```

```

switch (operation.op) {
  case ARRAY_DIFF_OP.ADD: {
    mountDOM(item, parentEl, index)
    break
  }

  case ARRAY_DIFF_OP.REMOVE: {
    // TODO: implement
  }

  case ARRAY_DIFF_OP.MOVE: {
    // TODO: implement
  }

  case ARRAY_DIFF_OP.NOOP: {
    // TODO: implement
  }
}
}
}

```

Next comes the `ARRAY_DIFF_OP.REMOVE` operation.

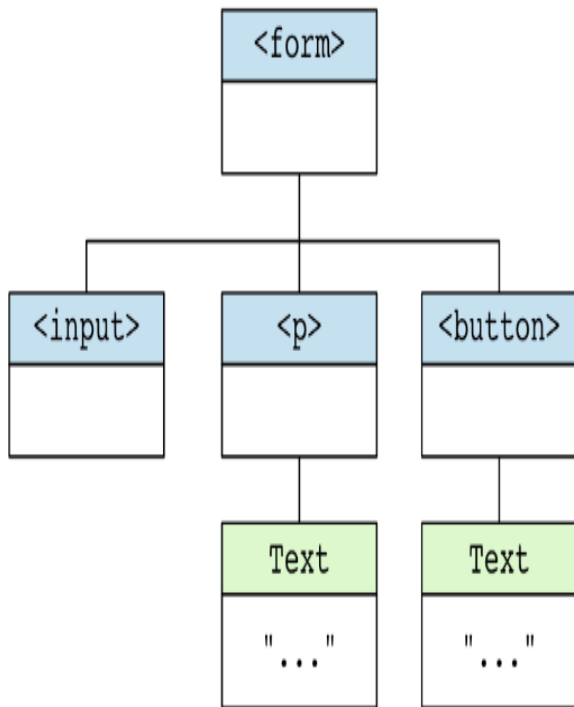
Remove operation

When a node is removed from the children array, you want to unmount it from the DOM. Thanks to the `destroyDOM()` function you wrote earlier in the book, this is easy.

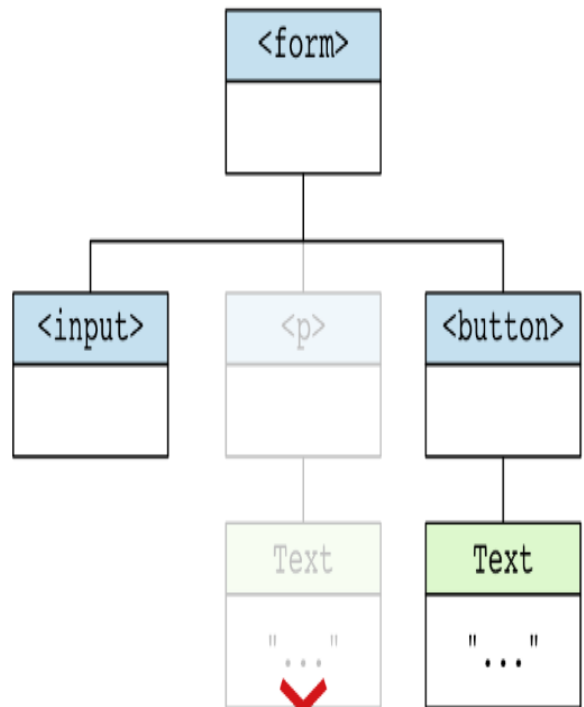
Figure [8.16](#) illustrates the case where a node is removed from the children array (an operation of type `ARRAY_DIFF_OP.REMOVE`), and how it's removed from the DOM.

Figure 8.16 Removing a node from the children array

Old vdom



New vdom



DOM

<form>
<input>
~~<input>~~
<button>
</form>

Remove node

Write the code in bold face in listing [8.21](#).

Listing 8.21 Patching the children—removing a node (*patch-dom.js*)

```
function patchChildren(oldVdom, newVdom) {  
  // --snip--  
  
  for (const operation of diffSeq) {  
    const { from, index, item } = operation  
  
    switch (operation.op) {  
      case ARRAY_DIFF_OP.ADD: {
```

```

        mountDOM(item, parentEl, index)
        break
    }

    case ARRAY_DIFF_OP.REMOVE: {
        destroyDOM(item)
        break
    }

    case ARRAY_DIFF_OP.MOVE: {
        // TODO: implement
    }

    case ARRAY_DIFF_OP.NOOP: {
        // TODO: implement
    }
}
}
}

```

Next goes the `ARRAY_DIFF_OP.MOVE` operation, which is a bit more nuanced.

Move operation

When you detect that a node has moved its position in the children array, you have to move it in the DOM as well. To do this, you need to grab the reference to the DOM node, and use its `insertBefore()` method to move it to the new position. The `insertBefore()` method requires a reference to a DOM node that will be the next sibling of the node you're moving; you need to find the node that's currently at the desired index.

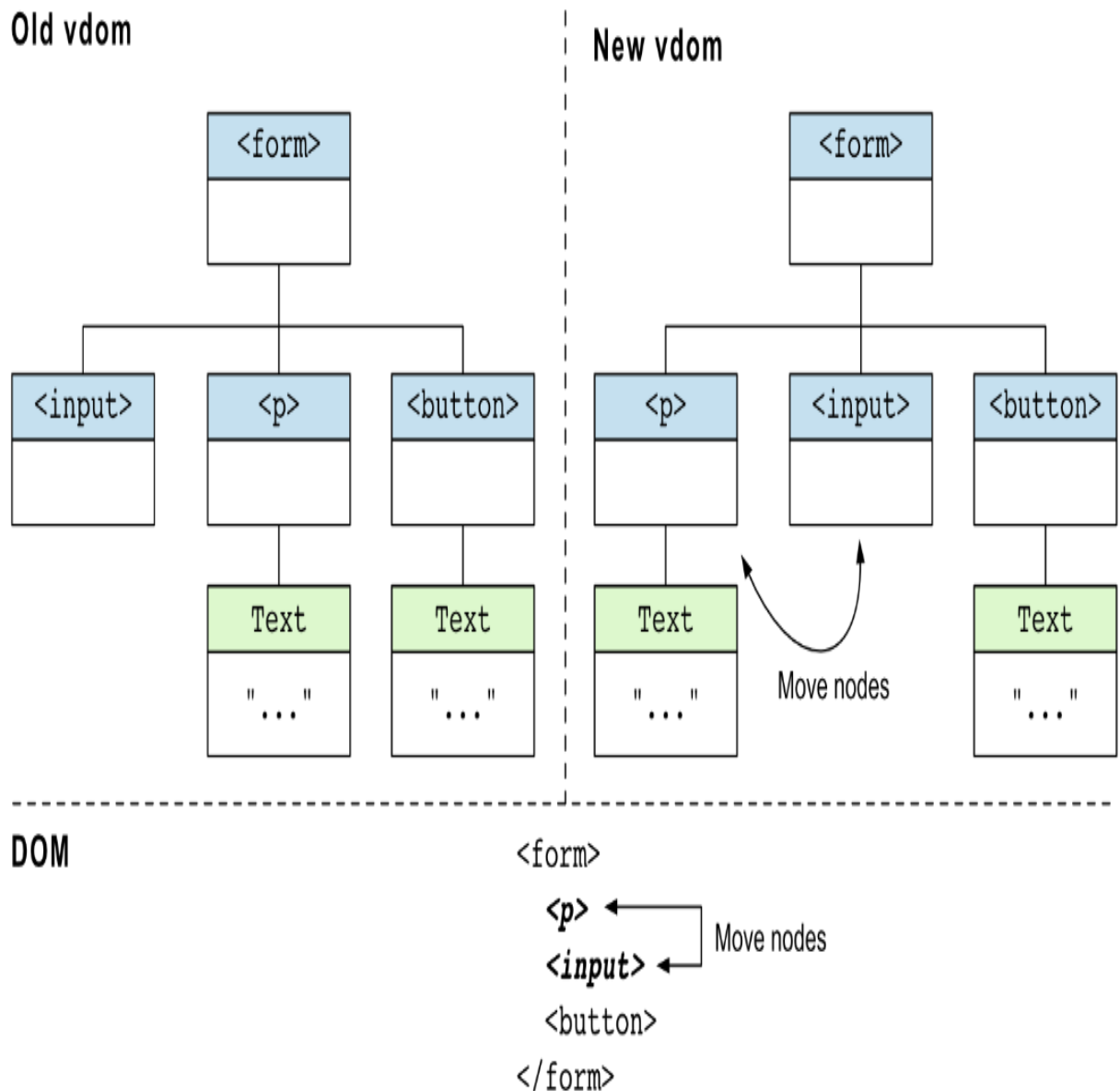
Note

The browser automatically removes the node from its original position when you move it; you won't end up with the same node in two different places.

After moving the node, you want to pass it to the `patchDOM()` function to patch it. These nodes stay in the DOM from one render to the next, so they might have not only moved around, but also changed in other ways (for example, a CSS class was added to them).

You can see the case of a node moving its position illustrated in figure [8.17](#). Here, when a node moves inside its parent node's children array (an operation of type `ARRAY_DIFF_OP.MOVE`), the same movement is replicated in the DOM.

Figure 8.17 Moving a node inside the children array



Write the code in bold face in listing [8.22](#).

Listing 8.22 Patching the children—moving a node (*patch-dom.js*)

```

function patchChildren(oldVdom, newVdom) {
  // --snip-- //

  for (const operation of diffSeq) {
    const { from, index, item } = operation

    switch (operation.op) {
      case ARRAY_DIFF_OP.ADD: {
        mountDOM(item, parentEl, index)
        break
      }

      case ARRAY_DIFF_OP.REMOVE: {
        destroyDOM(item)
        break
      }

      case ARRAY_DIFF_OP.MOVE: {
        const oldChild = oldChildren[originalIndex] #1
        const newChild = newChildren[index] #2
        const el = oldChild.el #3
        const elAtTargetIndex = parentEl.childNodes[index] #4

        parentEl.insertBefore(el, elAtTargetIndex) #5
        patchDOM(oldChild, newChild, parentEl) #6

        break
      }

      case ARRAY_DIFF_OP.NOOP: {
        // TODO: implement
      }
    }
  }
}

```

The last operation is the `ARRAY_DIFF_OP.NOOP` operation.

Noop operation

If you recall, some of the child nodes might have not moved, or moved due to other nodes being added or removed around them (what I call natural movements). You don't need to explicitly move them, because they fall into

their new positions naturally. But, what you need to do is patch them, because they might have changed in other ways, as noted before.

Write the code in bold font in listing [8.23](#).

Listing 8.23 Patching the children—noop operation (*patch-dom.js*)

```
function patchChildren(oldVdom, newVdom) {
  // --snip-- //

  for (const operation of diffSeq) {
    const { from, index, item } = operation

    switch (operation.op) {
      case ARRAY_DIFF_OP.ADD: {
        mountDOM(item, parentEl, index)
        break
      }

      case ARRAY_DIFF_OP.REMOVE: {
        destroyDOM(item)
        break
      }

      case ARRAY_DIFF_OP.MOVE: {
        const el = oldChildren[from].el
        const elAtTargetIndex = parentEl.childNodes[index]

        parentEl.insertBefore(el, elAtTargetIndex)
        patchDOM(oldChildren[from], newChildren[index],
parentEl)

        break
      }

      case ARRAY_DIFF_OP.NOOP: {
        patchDOM(oldChildren[originalIndex],
newChildren[index], parentEl)
        break
      }
    }
  }
}
```

That's it: you've implemented the reconciliation algorithm. And most remarkably, you've done it from scratch.

Let's now publish the new and improved version of your framework to see it in action in the TODOs application.

8.3 Publishing the framework's new version

In the last two chapters, you've implemented the `patchDOM()` function—the reconciliation algorithm. It took quite some code, but you made it. Thanks to it, your framework can figure out what changed between two virtual DOM trees, and patch the real DOM tree accordingly. It's a good time to publish a new version of your framework that you can use in the TODOs application.

First, bump the version of the *runtime* package by incrementing the `version` field in the *package.json* file to `2.0.0`:

```
{  
  "version": "1.0.0",  
  "version": "2.0.0",  
}
```

Then, run the `npm publish` command to publish the new version of the package. That's it! Your new and improved version of the framework is now available on NPM and unpkg.com.

8.4 The TODOs application

It's time to put your improved framework to use! The nice thing is, that the public API of your framework hasn't changed, so you can use the TODOs application code from chapter [6](#).

Clone the *examples/ch06/todos* directory containing the code of the TODOs app into *examples/ch08/todos*. You can do it in your preferred way, but if you're using a Unix-like operating system, the simplest way is to run the following commands:

```
$ mkdir examples/ch08  
$ cp -r examples/ch06/todos examples/ch08/todos
```

Now, the only line that you need to change is the first line in the *index.js* file, where you import your framework. Change it to import the new version of the framework from unpkg.com:

```
import { createApp, h, hFragment } from
'https://unpkg.com/<fwk-name>@1'
import { createApp, h, hFragment } from
'https://unpkg.com/<fwk-name>@2'
```

And with that simple change, your TODOs application is now using the new version of your framework. You can serve the application by running the following command at the root of the project:

```
$ npm run serve:examples
```

Open the application in your browser at <http://localhost:8080/ch08/todos/todos.html>. Everything should work exactly as it did in chapter 6, but this time, when you write in the input field, it's focus is preserved, as the DOM isn't recreated every time you type a character.

How can you make sure your algorithm isn't doing extra work and it's only patching what changed?

8.4.1 Inspecting the DOM tree changes

An interesting way of checking that the reconciliation algorithm is working as expected is to open the browser's developer tools, and inspect the DOM tree (use the Inspector tab in Firefox, or the Elements tab in Chrome). This panel in the developer tools shows you the DOM tree of the page you're currently viewing, and when a node changes, it highlights it with a flashing animation.

For example, if you open the TODOs application from chapter 6 — when the DOM was recreated every time—and type a character in the input field, you'll see that the entire DOM tree is highlighted, as you can see in figure 8.18. (If you followed the folder naming conventions explained in appendix A, your application from chapter 6 should be running at <http://localhost:8080/examples/ch06/todos/todos.html>.)

Figure 8.18 With your previous version of the framework, the entire DOM tree is highlighted when you type a character in the input field.

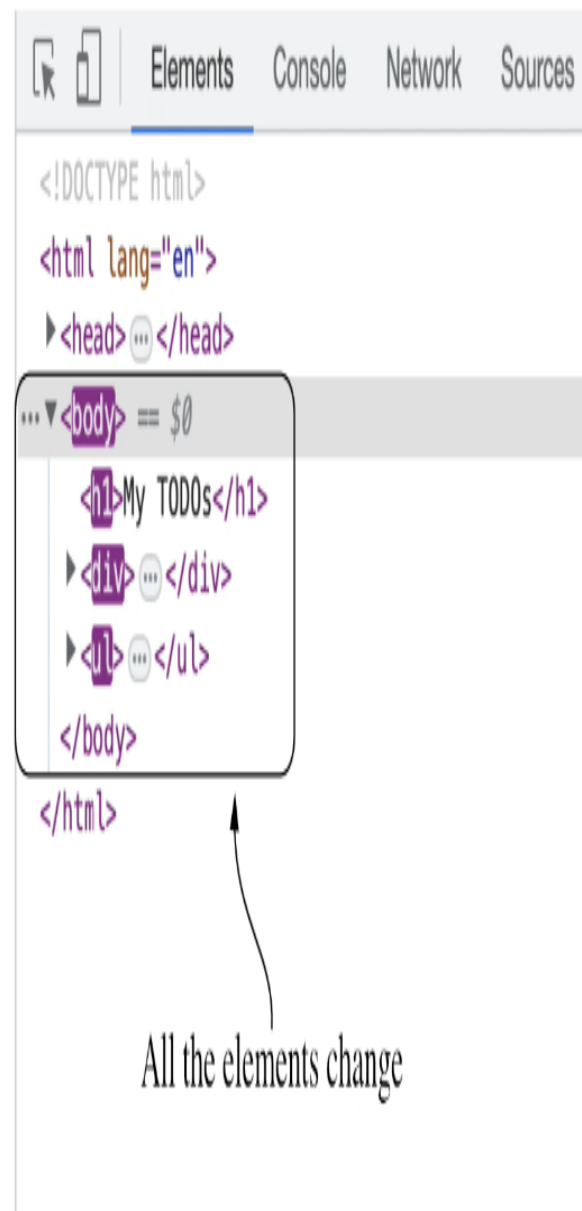
My TODOs

New TODO

• Walk the dog

• Water the plants

• Sand the chairs



If you open the TODOs application from this chapter, and type a character in the input field... nothing is highlighted. That's because the DOM tree didn't change at all (just the `value` property of the input field). But once you've written three characters, the `disabled` attribute is removed from the Add `<button>`, and you see this node flashing in the DOM tree (see figure [8.19](#)).

Figure 8.19 With the new version of your framework, nothing is highlighted when you type a character in the input field, until you've written three characters and the `disabled` attribute is

removed from the Add `<button>`.

My TODOs

New TODO

- Walk the dog
- Water the plants
- Sand the chairs

Only the button changes (by removing the disabled attribute) when the text in the input reaches a length of three



```
<!DOCTYPE html>
<html lang="en">
  <head>...</head>
  <body> == $0
    <h1>My TODOs</h1>
    <div>
      <label>New TODO</label>
      <input type="text" id="todo-input">
      <button>Add</button>
    </div>
    <ul>
      <li>
        ::marker
        <span>Walk the dog</span>
        <button>Done</button>
      </li>
      <li>
        ::marker
        <span>Water the plants</span>
        <button>Done</button>
      </li>
    </ul>
  </body>
</html>
```

Chrome has a neat feature that allows you to see the areas in the page that are repainted. You can find it in the developer tools, under the Rendering tab, and it's called ["Paint flashing"](#). If you select it, you can see highlighted in green rectangles the parts of the page that the browser repaints.

Repeating the experiment by first looking at the TODOs application from chapter [6](#), you can see that the entire page is repainted every time you type a character (figure [8.20](#)).

Figure 8.20 With your previous version of the framework, the entire page is repainted every time you type a character in the input field.



But if you look at the TODOs application from this chapter, you can see that only the input field's text and the field's label are repainted (figure [8.21](#)).

Figure 8.21 With the new version of your framework, only the input field's text and the field's label are repainted when you type a character in the input field.

My TODOs

New TODO

read

Add

- Walk the dog

Done
- Water the plants

Done
- Sand the chairs

Done

The fact that the "New TODO" label is repainted is a bit surprising, but it doesn't mean that the framework patched something it shouldn't have. As you saw in the DOM tree, nothing flashes there as you write in the input field. These "paint flashes" are a sign of the rendering engine doing its job of figuring out the layout of the page and repainting the pixels that might have moved or changed. Nothing you should worry about.

Experiment a bit with the developer tools to understand how the DOM is patched by your framework. The exercises that follow will give you some ideas of the experiments you can do with them.

Exercise 8.3

Use the "Elements" tab in Chrome (or the "Inspector" tab in Firefox) to inspect the DOM modifications (shown as flashes in the nodes) that happen when you add, modify or mark a to-do item as completed. Compare what happens in both the TODOs application from chapter 6 and that from this chapter.

Find the solution at <http://mng.bz/G96V>.

Exercise 8.4

Use the "Sources" tab in Chrome (or the "Debugger" tab in Firefox) to debug the `patchDOM()` function (you can set a breakpoint in the first line of the function) as you:

- Type a character in the input field.
 - Add a new to-do item.
 - Mark a to-do item as completed.
 - Edit the text of a to-do item
-

Find the solution at <http://mng.bz/z0Gr>.

8.5 Summary

- The reconciliation algorithm compares two virtual DOM trees, finds the sequence of operations that transform one into the other, and patches the real DOM by applying those operations to it. The algorithm is recursive, and it starts at the top-level nodes of both virtual DOM trees. After comparing these nodes, it moves on to their children, and so on, until it reaches the leaves of the trees.
- To know if a DOM node can be reused, you need to compare the corresponding virtual DOM nodes. If the virtual nodes are of a different type, the DOM node can't be reused. If the virtual nodes are either text or

fragment nodes, they can be reused. Element nodes can be reused if they have the same tag name.

- When the nodes being compared aren't equal—that is, they can't be reused—the DOM is completely destroyed and recreated from scratch. This makes sense in most of the cases, as if the parent node of a subtree changes, it's likely that the children are different as well.
- Text nodes are patched by setting the DOM node's `nodeValue` property to the new node's text.
- Element nodes are patched by separately patching their attributes, CSS classes and styles, and event listeners. You use the `objectsDiff()` and `arraysDiff()` functions to find the differences between the old and new values of these properties, and apply the changes to the DOM.
- Both fragment and element nodes need their children patched.
- To patch the children of a node, you use the `arraysDiffSequence()` to find the sequence of operations that transform one array of children into the other, and apply those operations to the DOM.

Part 3

3.1 Improving the framework

The groundwork has been laid, and your framework can successfully render a website based on a virtual DOM representation while keeping it in sync as users interact with it. However, let's be honest, having the entire application's state confined to a single object managed by the application isn't the most practical approach. Wouldn't it be more efficient if each component could take charge of its own piece of the state, focusing solely on the view it oversees?

In this final part of the book, you'll delve into the world of stateful components—these components autonomously manage their own state and lifecycles. They are responsible exclusively for the view they oversee and can be combined to create more intricate views. This enhancement empowers the application to bypass the reconciliation algorithm for every change, updating only the components affected by the modification.

You'll explore how stateful components can incorporate other components within their view and establish communication between them. You'll also implement a scheduler that enables the asynchronous execution of component lifecycle hooks, a potent feature that a robust framework should offer.

And last but not least, you'll master the art of performing thorough unit testing on components defined within your framework, even when they involve asynchronous behavior.

9 Stateful components

This chapter covers

- The anatomy of a stateful component
- Implementing a factory function to define components
- Implementing the first version of a component that manages its own state

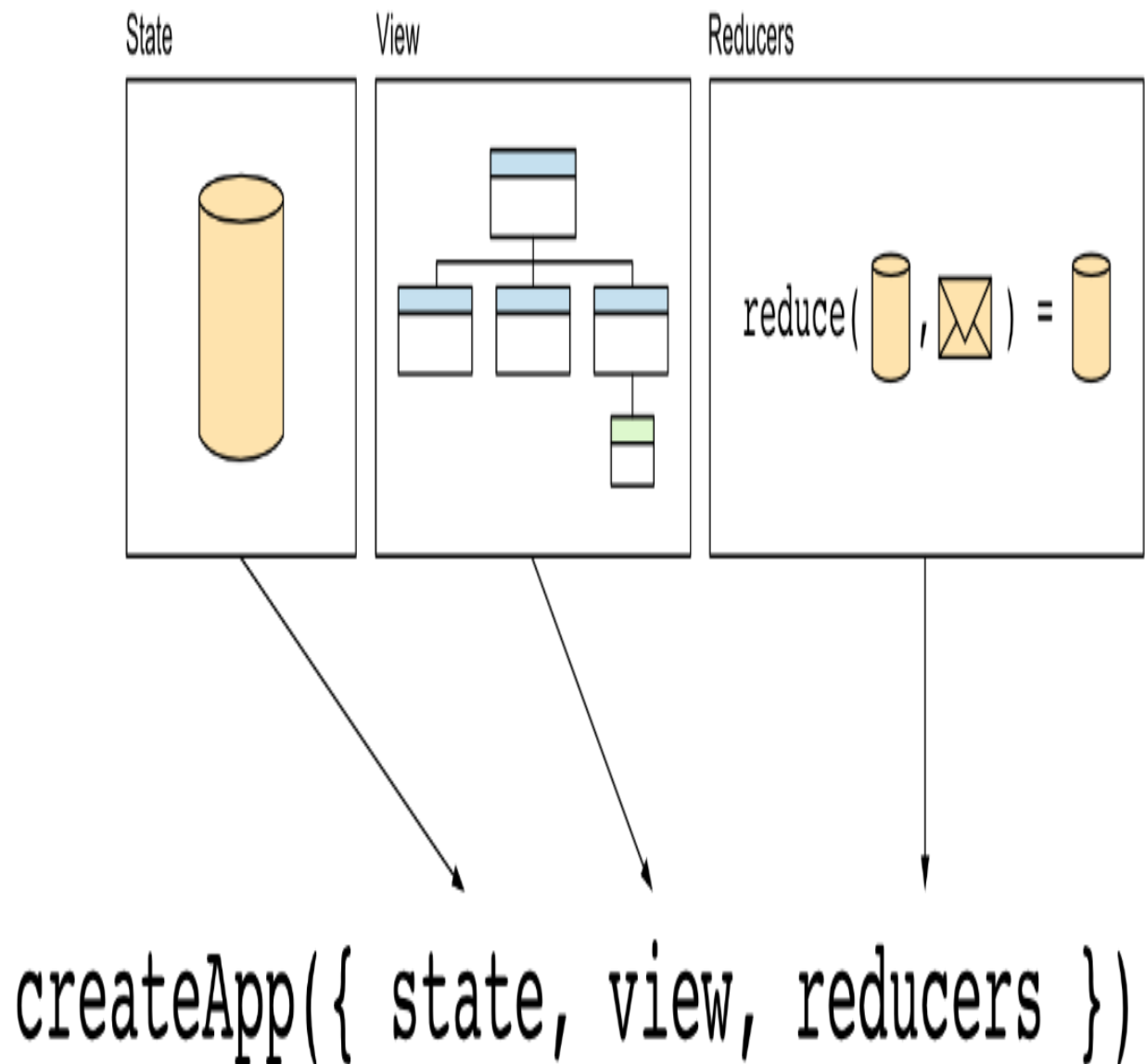
Your framework works great—it manipulates the DOM so the developer doesn't need to, and patches the DOM in a very efficient way thanks to the reconciliation algorithm. But using stateless components (pure functions) forced us to move all the state to the top of the application along with the reducers. There are a few reasons why this isn't ideal:

- A component deep down the hierarchy needs all the components above it to pass the state down to it, even when they don't need it. As the number of levels increases, the number of components that need to pass the state down to their children gets unwieldy.
- As the application grows, the state gets bigger and bigger, and the amount of reducers increases as well. Having all of them together in a single place can become a bit messy.
- When a component updates the state, all the application is re-rendered, even if the component that updated the state is deep down the hierarchy. Your reconciliation algorithm traverses the entire application's virtual DOM every time the state changes.

To address these issues, you can have your components handle their own state. This means they will have internal state that they can update and adjust their view accordingly when the state changes. By breaking down the state into smaller parts at the component level, you avoid having a massive state object with numerous reducers at the application level. Smaller state objects within components are more focused and simpler to understand. Consequently, when a component's state changes, only that specific component needs to be re-rendered, instead of the entire application.

To create an application with your current framework, you have to pass the `createApp()` function a state object, the reducers object, and a root component (figure [9.1](#)).

Figure 9.1 The current framework's application instance requires the global state, reducers, and the root component

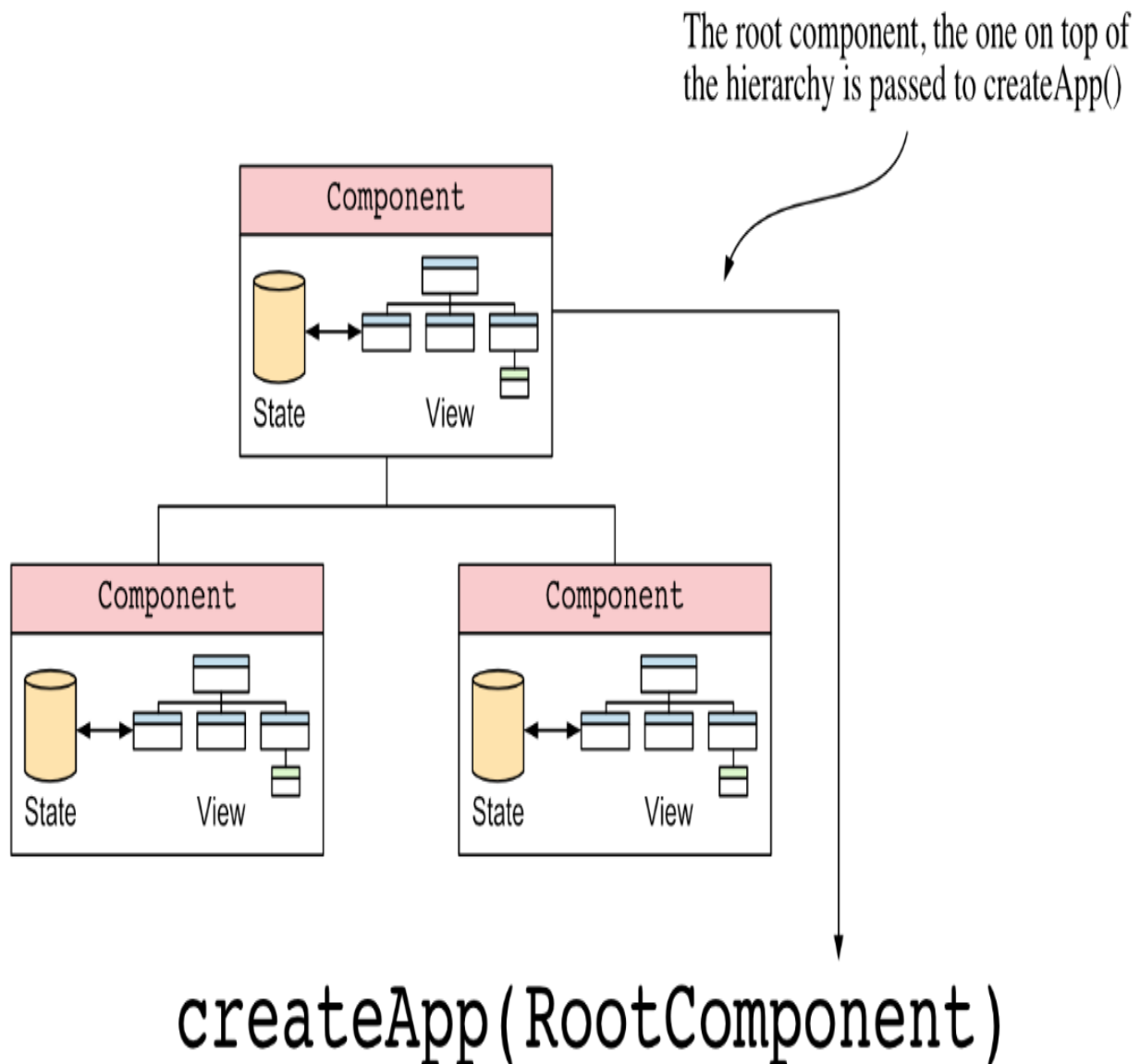


The plan for the new version of the framework (version 3.0) is to have components that manage their own state, and patch only the part of the DOM they are responsible for. The application will be represented by a hierarchy of

components, and the root component—that on top of the hierarchy—is the one that’s passed to the `createApp()` function (figure 9.2).

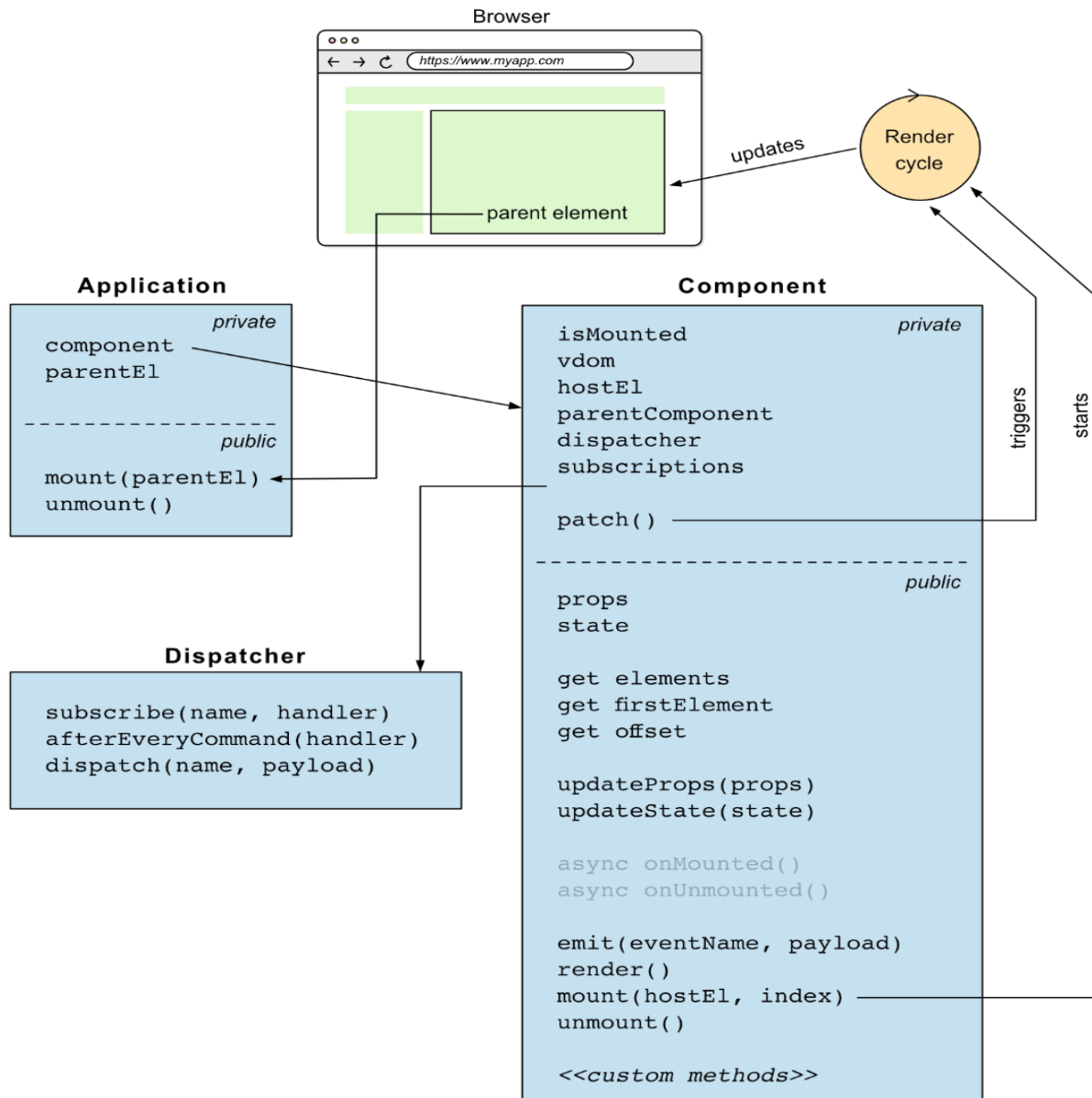
As you’ll see in this chapter, components won’t use the reducers system of handling the state; we’ll go for a simpler approach.

Figure 9.2 The new version of the framework’s application instance requires the root component only



With these changes (that'll take three—more like four—chapters to complete), you're getting closer to the framework's final architecture, which you saw in chapter 1 and is reproduced here in figure 9.3.

Figure 9.3 The component in the framework's architecture



Before you start writing the code, let's take a look at the anatomy of the stateful component you'll implement; but just the parts that are relevant to this chapter. There are many different ways you could implement a stateful

component, and we'll go for a simple, yet powerful, one. Explaining the parts of a component will help you understand the code you'll write.

Important

You can find all the listings in this chapter in the [listings/ch09 directory](#) of the book's repository.

The code you write in this chapter is for the framework's version v3.0, which you'll publish in chapter 12. Therefore, the code in this chapter can be checked out from [the ch12 label](#):

```
$ git switch --detach ch12
```

Code catch-up

In the last chapter you modified the `mountDOM()` function to insert nodes at a specific index in the child list of a parent node. You implemented a function called `insert()` to do the insertion. You modified the `createTextNode()` and `createElementNode()` functions to receive an index as their third parameter and use the `insert()` function to append nodes to the DOM.

You then implemented a function to compare if two virtual DOM nodes are equal, called `areNodesEqual()`. This function is used by the `patchDOM()` function, which you implemented next.

The `patchDOM()` function—depending of the type of the virtual DOM node—calls the `patchText()` or `patchElement()` functions to patch the DOM. The `patchElement()` function was especially involved, and required you to break the logic into sub-functions: `patchAttrs()`, `patchClasses()`, `patchStyles()`, and `patchEvents()`.

The `patchDOM()` function is recursive, and calls itself to patch the children of the node it's patching. You implemented the work of patching the children in the `patchChildren()` function.

9.1 Anatomy of a stateful component

Let's extract the component from figure [9.3](#) and analyze its anatomy: the properties and methods that a component has. Our discussion is based in figure [9.4](#).

A stateful component can't be a pure function anymore, because the output of pure functions depend exclusively on their arguments, and hence, they can't maintain state. (For a discussion on how React's stateful functional components work, see the sidebar discussion.)

Components v2.0

A stateful component is a component that maintains its own state. Stateful components can be instantiated, having each instance its own separate state and lifecycle. A stateful component updates its own view when its state changes.

React's functional stateful components

React once used classes to define components:

```
import { Component } from 'react'

class Greeting extends Component {
  render() {
    return <h1>Hello, {this.props.name}!</h1>
  }
}
```

Those classes can include a `state` property that can be updated using the `setState()` method. In fact, we'll follow a very similar approach. But React now encourages to use functions (functional components) instead. They're quite convenient:

```
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>
}
```

As you can see, the component is the equivalent of implementing the `render()` method of the class. But, if I just said that in order to have stateful components we need to move away from pure functions, how can React make

functional components stateful? The answer is, that functional components in React aren't pure functions (which is totally okay!), because to declare state you need to use the `useState()` hook:

```
function Counter() {
  const [count, setCount] = useState(0)

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Click me!
    </button>
    </div>
  )
}
```

The `useState()` hook, used to maintain state inside a functional component, has what we call "side effects." In this context, a side effect is when a function modifies something outside its scope. In this case, it's clear that `useState()` is storing the component's state somewhere, and thus the component function where it's used can't be a pure function.

A pure function is a function that's deterministic, meaning that given the same input, it'll always return the same output and doesn't have any side effects. But it's clear that you don't get the same result every time you call the component's function, because some state is maintained somewhere, and based on that state the component returns a different virtual DOM.

When you use the `useState()` hook, React is storing the state of that component inside an array at application level. By keeping track of the position of the state of each component, React can update the state of the component when you call the `setState()` method, and get you the correct value when you ask for it. The source code for hooks can be found at <http://mng.bz/4Ddg>, in case you want to take a look at it. The [ReactFiberHooks.js](#) file inside the *react-reconciler* package is also worth taking a look at. The `mountState<S>()` function implements the functionality behind the `useState()` hook.

Let me finish this discussion by saying that, what React has achieved with functional components and hooks is simply brilliant, fruit of very talented

engineers working on designing a better way to build user interfaces. Kudos to them!

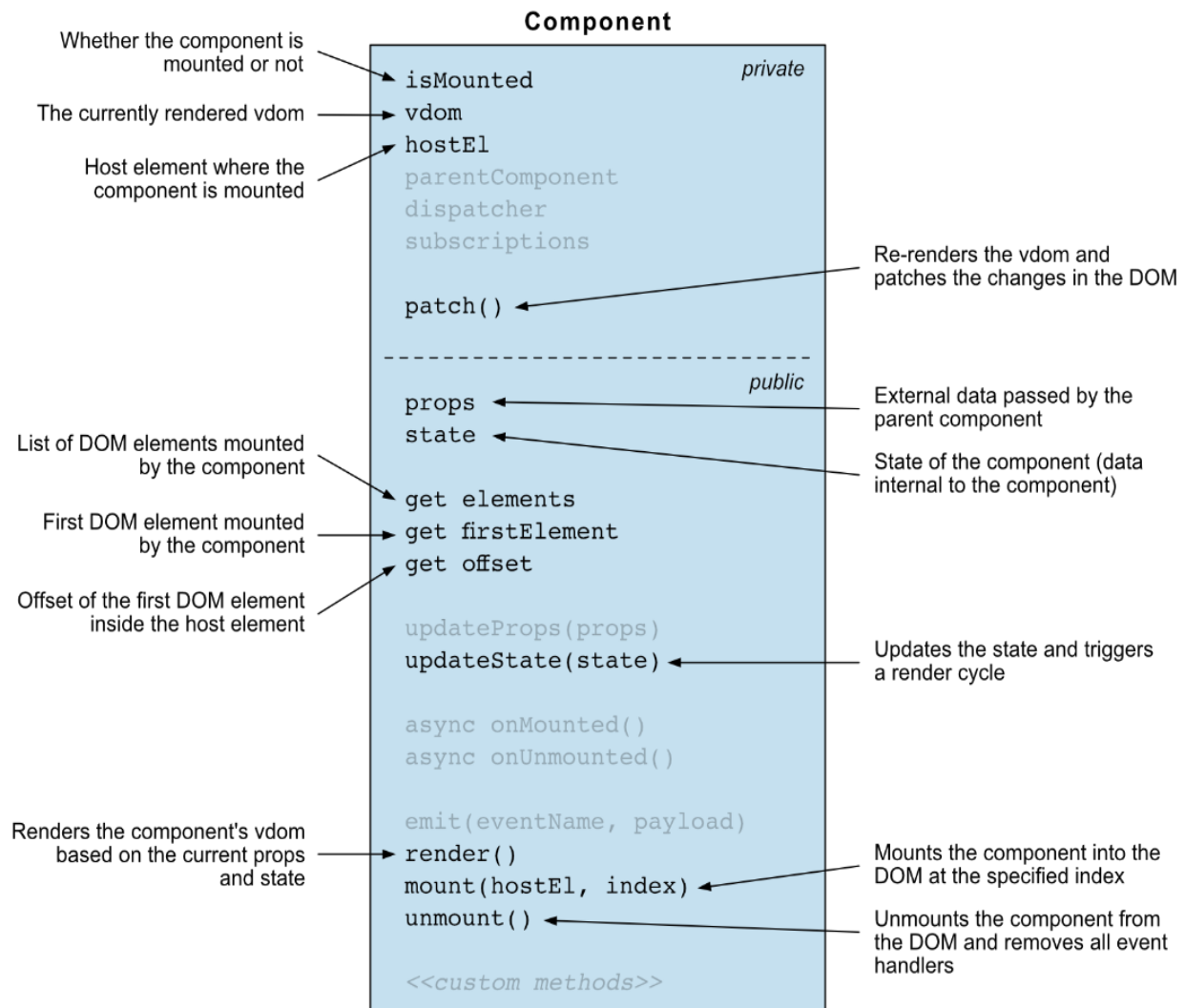
These are the four main characteristics of the component system that you'll implement:

- The view of a component depends on its props and state: every time one of these changes, the view needs to be patched.
- Components can have other components as children, as part of their view's virtual DOM.
- A component can pass data to its children, and the child components receive this data as props.
- A component can communicate with its parent component by emitting events that the parent component can listen to.

We'll focus on the first point in this chapter, and leave the communication between components for the next one. Let's take a look at figure [9.4](#) and briefly discuss the properties and methods of a stateful component that you'll implement in this chapter.

Figure 9.4 The anatomy of a stateful component

Anatomy of a stateful component



The diagram isn't exhaustive, but it covers the most important properties and methods of a component. Those grayed out, you'll implement them in the following chapters.

9.1.1 The properties of a stateful component

The public properties of a stateful component (simply called *component* from now on) are:

- `props`: the data that the component receives from its parent component.

- `state`: the data that the component maintains internally.

Even though these properties are public, they shouldn't be modified directly. Why shouldn't they? Because if you do so, the component won't know that they have changed, and hence, it won't update its view. To update the state of a component, you'll use the `updateState()` method, and `updateProps()` for the props, which we'll discuss in the next section and next chapter, respectively. These methods, apart from updating the state and props, also trigger the re-rendering of the component.

There are also three interesting get-only properties in a component:

- `elements`: a list of the DOM elements mounted by the component. When the top node of the view of a component is a fragment, this is the list of the DOM elements mounted by the component's children. If the component has a single root node, this is a list with a single element.
- `firstElement`: the first element of the `elements` list.
- `offset`: the offset of the `firstElement` inside the component's host element in the DOM. You'll need this offset to correctly patch the component's DOM, as you'll see later in this chapter.

There are also three private properties:

- `isMounted`: a boolean that indicates if the component is mounted or not.
- `vdom`: the virtual DOM of the component that's currently rendered in the DOM.
- `hostEl`: The element in the DOM where the component is mounted.

Let's now take a look at the methods of a component.

9.1.2 The methods of a stateful component

A component has two methods that you'll use to mount and unmount it:

- `mount()`: mounts the component in the DOM.
- `unmount()`: unmounts the component from the DOM and unsubscribes all the subscriptions.

To update the state the component maintains internally, you'll use the following method:

- `updateState()`: updates the state of the component, and triggers the re-rendering of the component. This method should be called inside the component, whenever the state of the component changes.

As we'll see in this and the following chapters, the state of a component can be the props of its children components; a component can pass its internal state to its children components as props. When new props are passed to a component, this should re-render its view. To that end, the component uses two methods:

- `render()`: returns the virtual DOM of the component, based on the current props and state of the component.
- `patch()`: patches the DOM of the component with the new virtual DOM. This is a private method that's called as part of updating the state or props of the component, but isn't meant to be called from outside the component.

The best way to understand how this component system works is by implementing it, so let's get started. We'll start simple and improve it, adding more features, as we go. Warm up your code editor and stretch your fingers, because you'll write a lot of code in this chapter!

9.2 Components as classes

Start by creating a new file inside the *src/* folder, called *component.js*. We'll write all the code in this section in this file.

Let's think about how you can define a component. You want to be able to instantiate components, and each component instance should have its own state. You can therefore use classes to define the component as so:

```
class Component {}
```

You can now instantiate the component (`new Component()`), but it doesn't do anything... yet! You want components to be able to render their view, and for

that they need a `render()` method:

```
class Component {
  render() {
    // return the virtual DOM of the component
  }
}
```

But here comes the first complication. Each component the user defines is different; their `render()` method will return a different virtual DOM. For example, in the TO-DOs app you wrote in chapter 8, the `CreateTodo()` and the `TodoList()` components return very different virtual DOMs (which makes sense, right?). So, how do you define the `render()` method of the `Component` class when you don't know what the virtual DOM of the component will be beforehand?

The solution is to define a function, `defineComponent()`, which takes an object containing a `render()` function that creates the virtual DOM of that specific component, and returns the component class:

```
export function defineComponent({ render }) {
  class Component {
    render() {
      return render()
    }
  }

  return Component
}
```

The `defineComponent()` function returns a class, so you can have as many different component classes as you want by simply defining how each component renders its view:

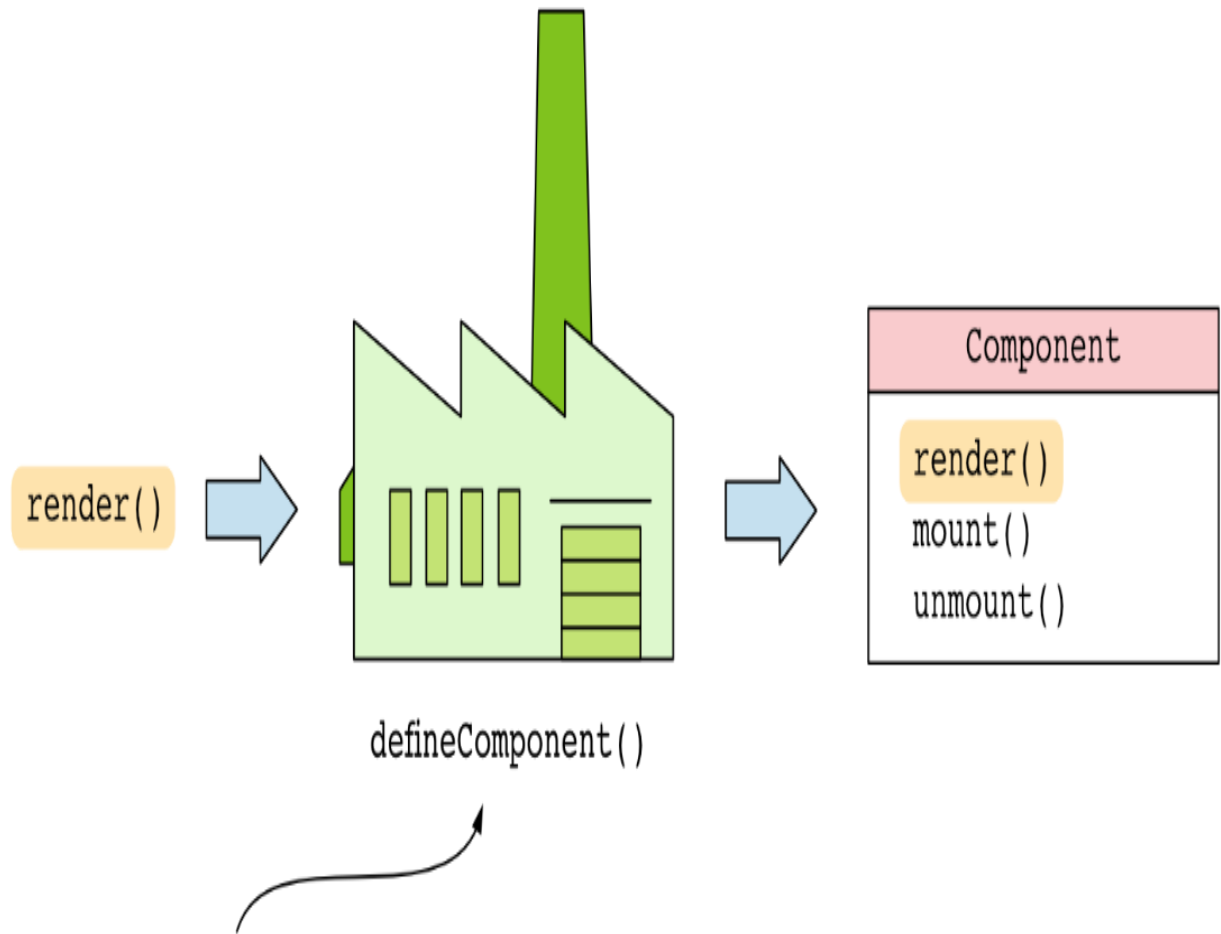
```
const CreateTodo = defineComponent({
  render() {
    // return the virtual DOM of the component
  }
})

const TodoList = defineComponent({
  render() {
    // return the virtual DOM of the component
  }
})
```

```
    }  
  })
```

In a sense, `defineComponent()` is a factory function that, given a `render()` function, creates a component class that uses that function to render its view. The returned component class can be instantiated as many times as you want, and all of its instances render the same view. This is illustrated in figure [9.5](#). Once we add state to the component instance, each component might render a different—although similar—view, because as you know, the view of a component depends on its state.

Figure 9.5 The `defineComponent()` factory function creates component classes



The function is a factory that, given a render function, creates a component class that can be instantiated and renders the same view

The component also needs `mount()` and `unmount()` methods so that their view can be added to the DOM and removed from it, so let's add those to the `Component` class. Complete the `Component` class with the code in listing [9.1](#).

Listing 9.1 Basic component with `render()`, `mount()`, and `unmount()` methods (component.js)

```
import { destroyDOM } from '../destroy-dom'
import { mountDOM } from '../mount-dom'

export function defineComponent({ render }) { #1
  class Component { #2
```

```

    #vdom = null
    #hostEl = null

    render() { #3
        return render()
    }

    mount(hostEl, index = null) {
        this.#vdom = this.render() #4
        mountDOM(this.#vdom, hostEl, index) #5

        this.#hostEl = hostEl
    }

    unmount() {
        destroyDOM(this.#vdom) #6

        this.#vdom = null
        this.#hostEl = null
    }
}

return Component #7
}

```

Great! — But, you could mount the same instance of a component multiple times, like so:

```

const component = new MyComponent()
component.mount(document.body)
component.mount(document.body) // Same component mounted
twice!

```

or unmount it before it was mounted:

```

const component = new MyComponent()
component.unmount() // Unmounting a component that's not
mounted!

```

To prevent these situations, you need to keep track of the "mounting state" of the component. Add an `#isMounted` private property to the `Component` class and throw an error if you detect the aforementioned situations. Write the code in bold font in listing [9.2](#).

Listing 9.2 Check if the component is already mounted or not yet mounted (component.js)

```

export function defineComponent({ render }) {
  class Component {
    #isMounted = false #1
    #vdom = null
    #hostEl = null

    render() {
      return render()
    }

    mount(hostEl, index = null) {
      if (this.#isMounted) { #2
        throw new Error('Component is already mounted')
      }

      this.#vdom = this.render()
      mountDOM(this.#vdom, hostEl, index)

      this.#hostEl = hostEl
      this.#isMounted = true #3
    }

    unmount() {
      if (!this.#isMounted) { #4
        throw new Error('Component is not mounted')
      }

      destroyDOM(this.#vdom)

      this.#vdom = null
      this.#hostEl = null
      this.#isMounted = false #5
    }
  }

  return Component
}

```

This is a great start, but where's the state of the component? So far, the `Component` class doesn't have any state or external props, so the `render()` method always returns the same virtual DOM (unless it depends on global data define outside the component, which it shouldn't). Let's add the state and props to the component.

Exercise 9.1

Using the `defineComponent()` function and the code it depends on (the `mountDOM()` and `destroyDOM()` functions), create a component that renders the following HTML:

```
<h1>Important news!</h1>
<p>I made myself coffee.</p>
<button>Say congrats</button>
```

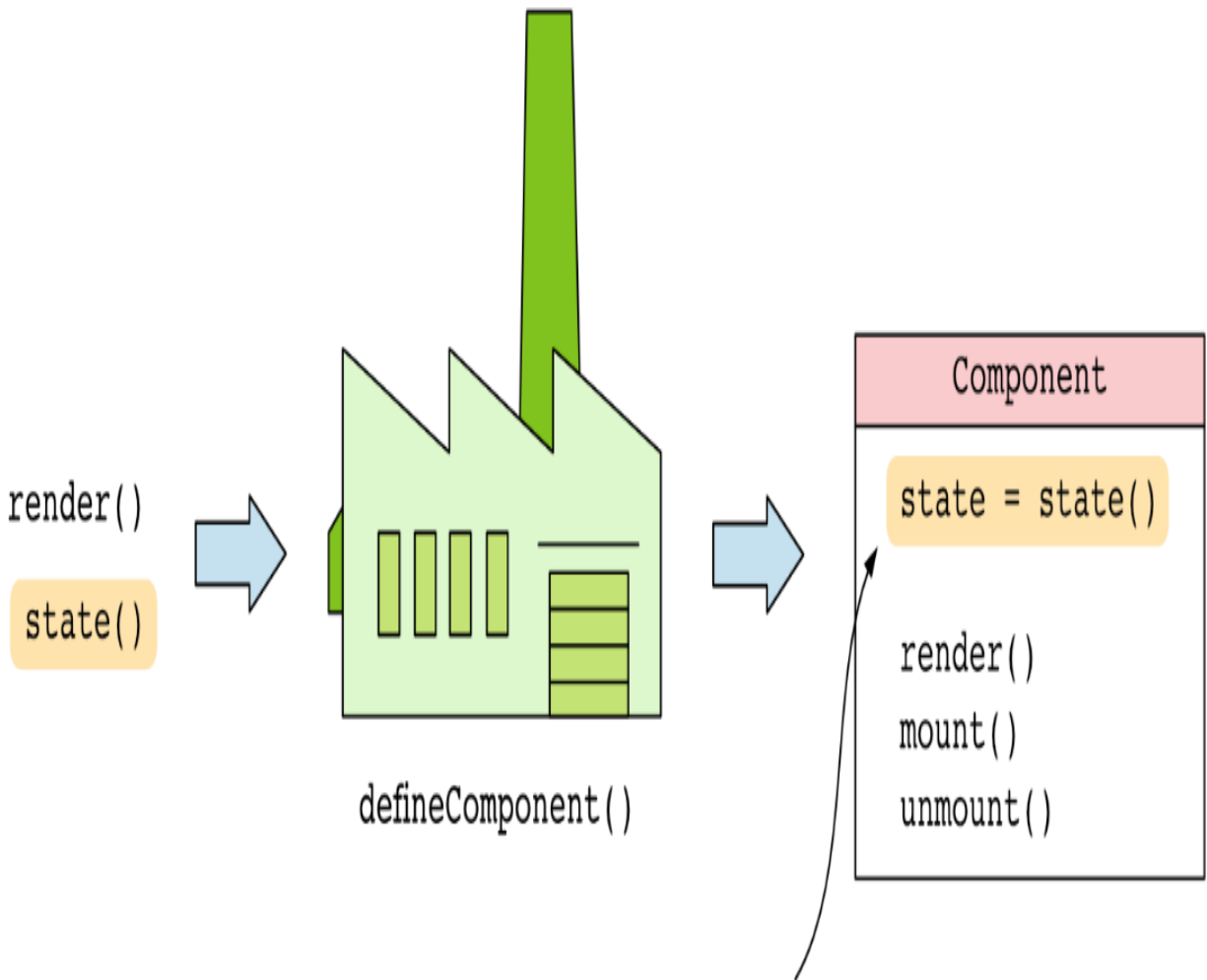
The button should print the following message in the console when clicked: "Good for you!" Mount the component inside a web page of your choice (for example in your local newspaper's website). Click the button and check that the message is printed in the console. Last, unmount the component from the DOM and verify that the component's HTML is removed from the page.

Find the solution at <http://mng.bz/0lrm>.

9.3 Components with state

The components that the `defineComponent()` function creates are static; they always render the same virtual DOM. This is because you haven't implemented the state of the component yet, so the `render()` method inside your component can't return different virtual DOMs. What you want is to pass the `defineComponent()` a function that returns the initial state of the component, as shown in figure [9.6](#).

Figure 9.6 The `defineComponent()` factory function uses the `state()` function to create the initial state of the component



Inside the component's constructor, you can call the `state()` function to create the initial state of the component and save it in the `state` property:

```
class Component {  
  constructor() {  
    this.state = state()  
  }  
}
```

Here's how you'd define the `state()` function that returns the initial state for a component that keeps track of the number of times a button is clicked:


```
const Counter = defineComponent({
  state() {
    return { count: 0 }
  },
  render() { ... }
})
```

But you might want to be able to set the initial state of a component's instance when you instantiate it. Maybe you don't want all instances of a component to have the same initial state.

Imagine for example the `Counter` component in the previous example. By default, the count starts at 0, but you might want to start the count of a specific instance at 10. To satisfy this need, you can pass the `state()` function the props passed to the component's constructor, so that the initial state can be based on the props:

```
class Component {
  constructor(props = {}) {
    this.state = state(props)
  }
}
```

In the counter example, you could do something like this:

```
const Counter = defineComponent({
  state(props) {
    return { count: props.initialCount ?? 0 }
  },
  render() { ... }
})
```

```
const props = { initialCount: 10 }
const counter = new Counter(props)
```

Add the code in bold font in listing [9.3](#) to implement the component's initial state and props.

Listing 9.3 Add the `state()` method to create the initial state based on the external props (component.js)

```
export function defineComponent({ render, state }) { #1
  class Component {
```

```

#isMounted = false
#vdom = null
#hostEl = null

constructor(props = {}) {
  this.props = props #2
  this.state = state ? state(props) : {} #3
}

render() {
  return render()
}

mount(hostEl, index = null) {
  // --snip-- //
}

unmount() {
  // --snip-- //
}
}

return Component
}

```

The component now has both state and props. You've used the props to initialize the state of the component, but now we'll leave the props aside and focus on the state. We'll come back to props in the next chapter.

Let's implement the `updateState()` function to update the state of the component patch the DOM with the new view.

9.3.1 Updating the state and patching the DOM

The state of the component evolves as the user interacts with it. Thus, you want to implement a function that updates the state of the component by merging the passed state with the current state. Something like this:

```
this.state = { ...this.state, ...newState }
```

This way, those pieces of the state that are not updated are preserved.

After the state has been updated, the DOM needs to be patched with the new virtual DOM. Patching the DOM should be handled by a different method, which you'll call `#patch()`, in the component, and it's a two-step process:

1. Call the component's `render()` method to get the new virtual DOM based on the new state
2. Call the `patchDOM()` function to patch the DOM with the old and new virtual DOM trees. Save the result of the `patchDOM()` function in the `#vdom` property of the component.

A caveat is that the component can't be patched if it's not mounted. Keep this in mind when implementing the `#patch()` method. Write the code in bold font in listing [9.4](#) to implement the `updateState()` and `#patch()` methods.

Listing 9.4 Update the state and the component's DOM (component.js)

```
import { destroyDOM } from './destroy-dom'
import { mountDOM } from './mount-dom'
import { patchDOM } from './patch-dom'

export function defineComponent({ render, state }) {
  class Component {
    #isMounted = false
    #vdom = null
    #hostEl = null

    constructor(props = {}) {
      this.props = props
      this.state = state ? state(props) : {}
    }

    updateState(state) {
      this.state = { ...this.state, ...state } #1
      this.#patch() #2
    }

    render() {
      return render()
    }

    mount(hostEl, index = null) {
      // --snip--
    }
  }
}
```

```

    unmount() {
      // --snip-- //
    }

    #patch() {
      if (!this.#isMounted) { #3
        throw new Error('Component is not mounted')
      }

      const vdom = this.render() #4
      this.#vdom = patchDOM(this.#vdom, vdom, this.#hostEl)
#5
    }
  }

  return Component
}

```

Great! Now you have a component whose state can be updated and the DOM patched with the new virtual DOM. You can access the `state` property inside the `render()` method, to render the virtual DOM based on the state of the component. The `updateState()` method can be called from within the component's `render()` method as well, as part of the event handlers.

For example, going back to the counter example, if next to the counter label you have a button to increment the counter, you could do something like this:

```

const Counter = defineComponent({
  state() {
    return { count: 0 }
  },

  render() {
    return hFragment([
      h('p', `Count: ${this.state.count}`), #1
      h(
        'button',
        {
          on: {
            click() {
              this.updateState({ count: this.state.count + 1
#2
            }
          }
        },
      ),
    ],
  ),
}

```

```

        'Increment',
      ),
    ]),
  },
})

```

But, if you tried this right now (and I encourage you to do so), you'd get the following error (can you guess why?):

```

Uncaught TypeError: Cannot read properties of undefined
(reading 'count')

```

The problem is that the `this` keyword inside the `render()` function that's called inside the `render()` method is not bound to the component, but the `Window` object (in strict mode it'd be `undefined`). This happens because the `render()` function passed to the `defineComponent()` function isn't a method of the class—in which case the `this` keyword would be bound to the component.

Note

You can read more about the `this` keyword in the MDN web docs: <http://mng.bz/QPGc>. It's important that you understand the rules of how `this` is bound in JavaScript, so you can follow the code in this book.

I highly recommend that you also read the "This works" chapter of the "Objects and classes" book, from the "You don't know JavaScript" series, by Kyle Simpson. You can in fact read this chapter in his GitHub account, at <http://mng.bz/XN8M>. This chapter contains everything you might need to know about the `this` keyword, which you'll use in this and the following chapters of the book. Trust me when I say that it's worth your time.

To fix this issue, you need to use explicit binding to bind the `this` keyword inside the `render()` function to the component. The fix is as simple as shown in listing 9.5. Modify your code so that it matches the code in listing 9.5.

Listing 9.5 Binding the `call()` function to the component (`component.js`)

```

export function defineComponent({ render, state }) {
  class Component {

```

```

    // --snip-- //

    render() {
      return render()
      return render.call(this)
    }

    // --snip-- //
  }

  return Component
}

```

Let's quickly review the code you've written so far.

9.3.2 Result

You've created a `defineComponent()` function that takes an object with a `render()` function to create a component. The `render()` function returns a virtual DOM based on the state of the component. The component's state can be updated with the `updateState()` function, which merges the new state with the current state and patches the DOM with the new virtual DOM.

If you followed along, your *component.js* file should look like listing [9.6](#).

Listing 9.6 The `defineComponent()` function so far (component.js)

```

import { destroyDOM } from './destroy-dom'
import { mountDOM } from './mount-dom'
import { patchDOM } from './patch-dom'

export function defineComponent({ render, state }) {
  class Component {
    #isMounted = false
    #vdom = null
    #hostEl = null

    constructor(props = {}) {
      this.props = props
      this.state = state ? state(props) : {}
    }

    updateState(state) {
      this.state = { ...this.state, ...state }
    }
  }
}

```

```

    this.#patch()
  }

  render() {
    return render.call(this)
  }

  mount(hostEl, index = null) {
    if (this.#isMounted) {
      throw new Error('Component is already mounted')
    }

    this.#vdom = this.render()
    mountDOM(this.#vdom, hostEl, index)

    this.#hostEl = hostEl
    this.#isMounted = true
  }

  unmount() {
    if (!this.#isMounted) {
      throw new Error('Component is not mounted')
    }

    destroyDOM(this.#vdom)

    this.#vdom = null
    this.#hostEl = null
    this.#isMounted = false
  }

  #patch() {
    if (!this.#isMounted) {
      throw new Error('Component is not mounted')
    }

    const vdom = this.render()
    this.#vdom = patchDOM(this.#vdom, vdom, this.#hostEl)
  }
}

return Component
}

```

Let's now discuss an interesting implication of patching the DOM owned by a component, when those DOM elements are inserted into an element where

other elements are already present. It's subtle, but without taking into account this detail, the component's DOM would be patched incorrectly.

Oh! But before that, why don't you go ahead and try to solve the following challenge?

Exercise 9.2 — Challenge

Using the `defineComponent()` function and the code it depends on (the `mountDOM()` and `destroyDOM()` functions), create a component called `FlyingButton`, which renders a `<button>` element, absolutely positioned at a random position inside the browser's viewport. The button's label should be "Move". When the button is clicked, the button should move to a random position inside the viewport.

Bear in mind that, the position of the button should be random, but it should never be outside the viewport; at least not entirely.

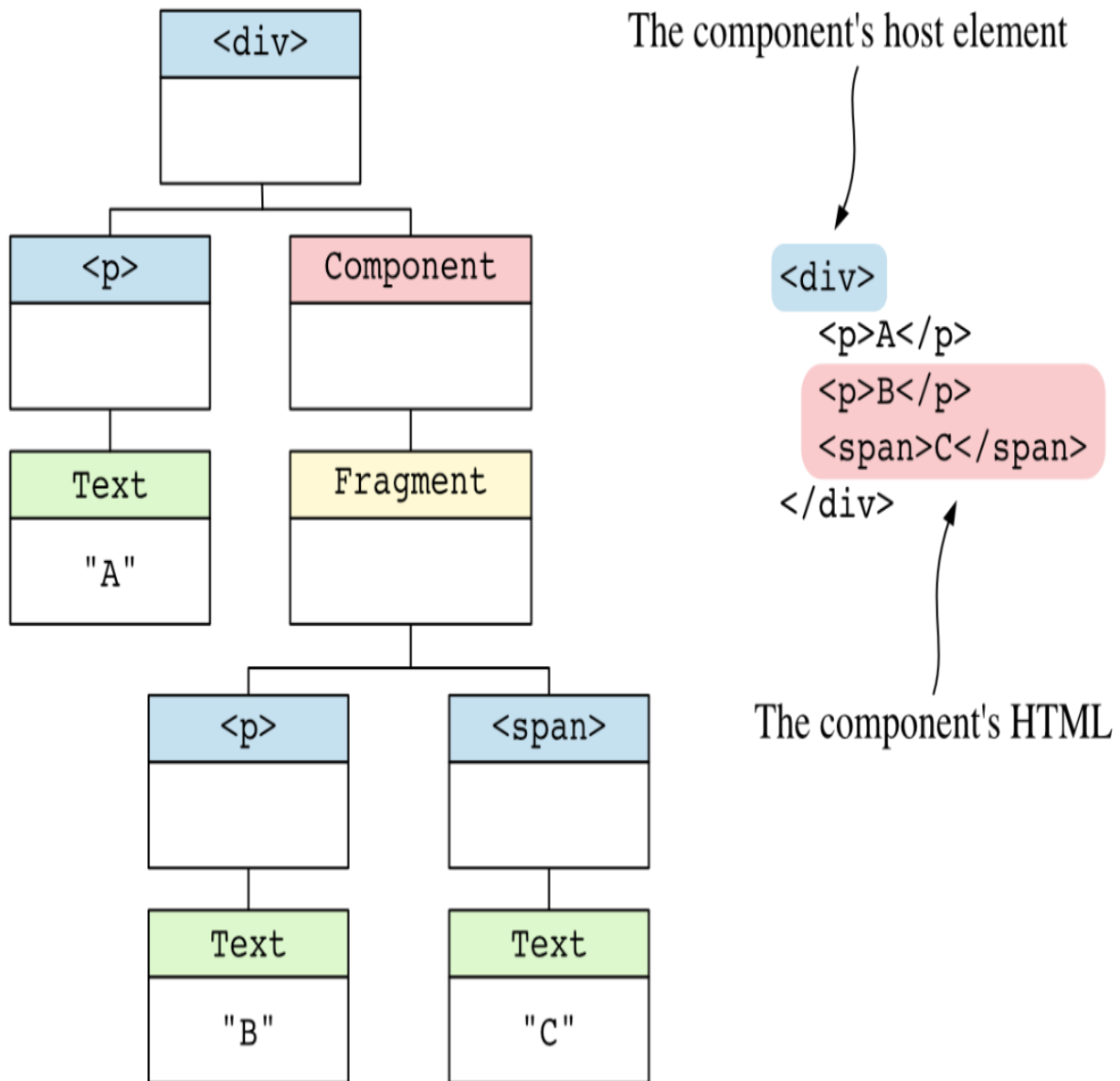
You can mount the button in the `<body>` element, or in any other element you want (it's absolutely positioned, so it shouldn't affect the layout of the page) inside your favorite website.

Find the solution at <http://mng.bz/K944>.

9.3.3 The component's offset

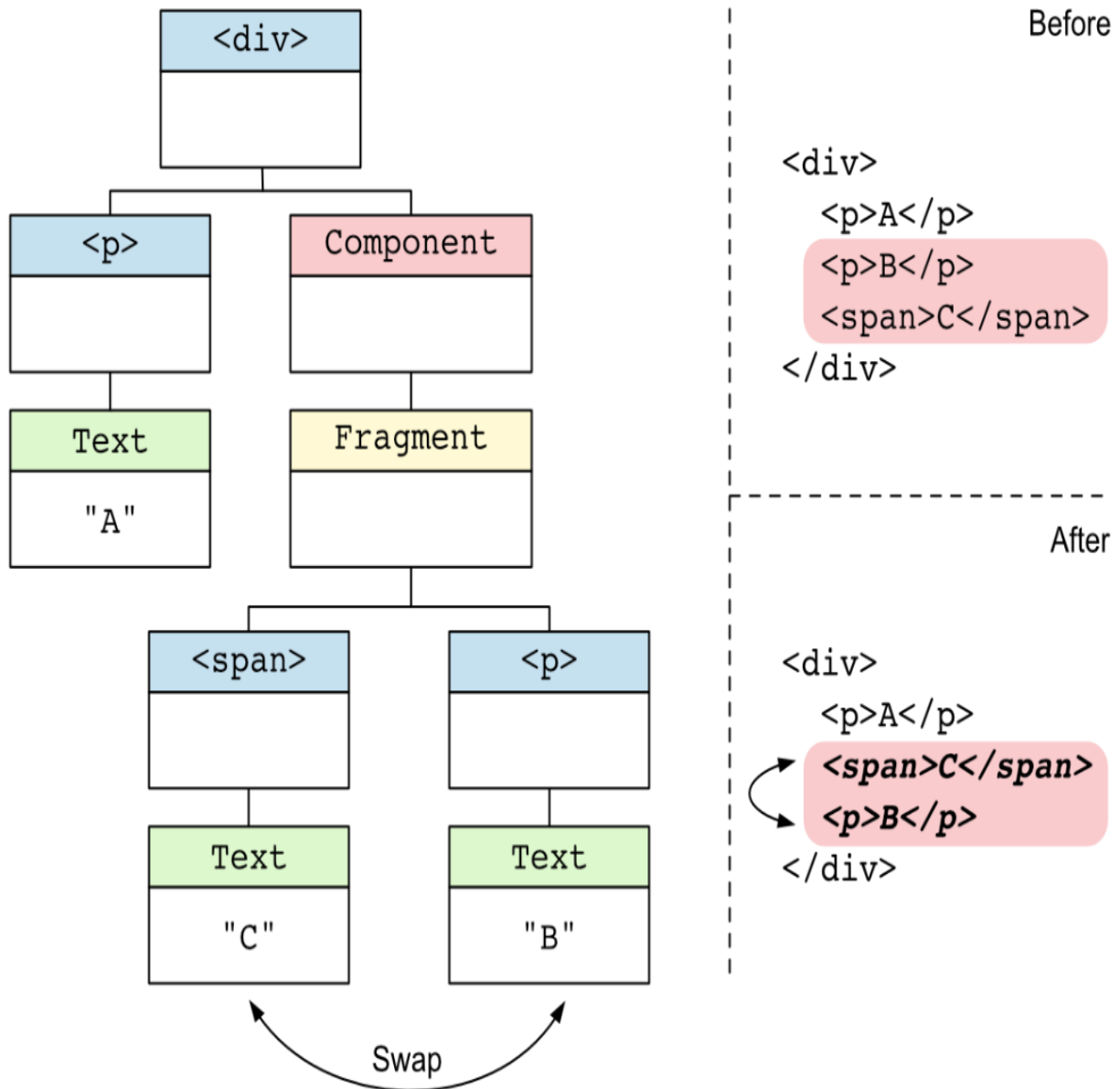
Let's say that you have a component whose HTML is mounted in the same `<div>` as a `<p>` element. In this configuration, we say the `<div>` is the component's host element, the one that contains the component's HTML. The view of this component consists of a fragment containing a `<p>` element and a `<span>` element (figure [9.7](#)).

Figure 9.7 A `<div>` element containing a `<p>` element and a component



Here's the thing that's important to notice: since the component's view is made of a fragment, all of its elements are siblings of the `<p>` element that's outside the component's view. Now, let's say that the component, after a change in its state, decides that it wants to swap the `<p>` element with the `<span>` element. The result is shown in figure [9.8](#).

Figure 9.8 Swap the `<p>` element with the `<span>` element inside the component



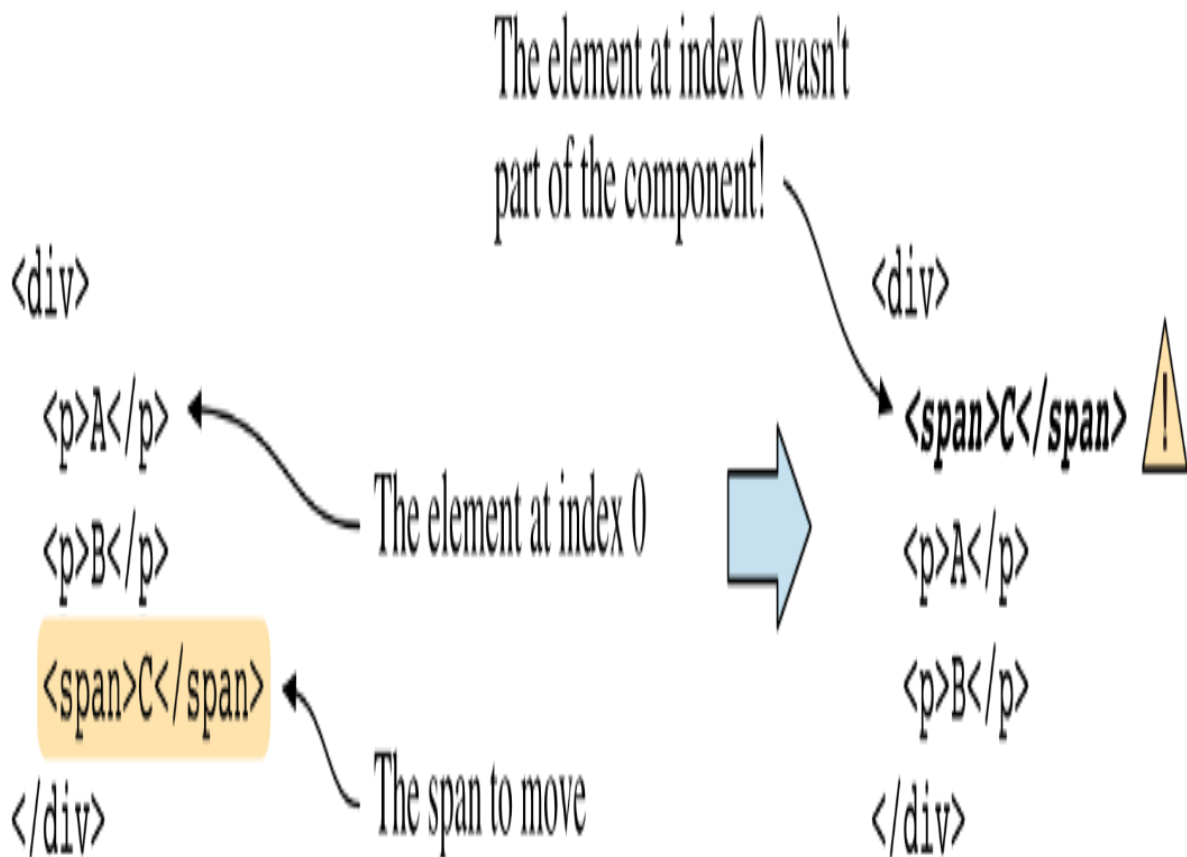
After the state is changed, the `updateState()` method calls the `#patch()` method, which in turn calls the `patchDOM()` function, passing it its virtual DOM. Recall that this virtual DOM is a fragment containing a `<p>` element and a `<span>` element, but not the first `<p>` element with the text "A;" that element is outside the component's view. `patchDOM()` in turn asks `arraysDiffSequence()` to find the sequence of operations that transforms the current DOM into the new virtual DOM. Your robust and well implemented `arraysDiffSequence()` function returns the following sequence of operations:

1. Move the `<span>` element from `i=1` to `i=0`
2. Noop the `<p>` element from `i=0` to `i=1`

And then goes on to apply the sequence of operations to the DOM. Figure [9.9](#) shows the result of applying the first operation to the DOM. Before you look at the figure, can you guess what the result will be? Is it what you expected? Is it correct?

Figure 9.9 Move the `<span>` element from `i=1` to `i=0`

1. Move the `<span>` element to `i=0`



That's wrong! The problem is that the `arraysDiffSequence()` calculates the indices of the operations relative to the lists of nodes it receives as arguments, which are the nodes inside the component's fragment. This list of nodes doesn't include all the elements under the parent `<div>` element; in fact, it's missing the first `<p>` element that's outside the component's view. This arises from having components whose vdom is a fragment, because its elements are inserted into a node that's outside the component, a host element that might contain other elements external to the component. When the component's view has a single root element, this isn't a problem (why this is I'll leave it as an exercise for you—the answer is in the end of the chapter).

The indices of the move operations are relative to the fragment—they don't account for the elements outside the component's fragment. Something similar happens when a node is added in the component's view, if the component's view is a fragment. The index where the node is to be added is relative to the fragment items, not accounting for the elements outside the fragment.

How do we go about fixing this? Let's introduce the concept of a component's *offset*.

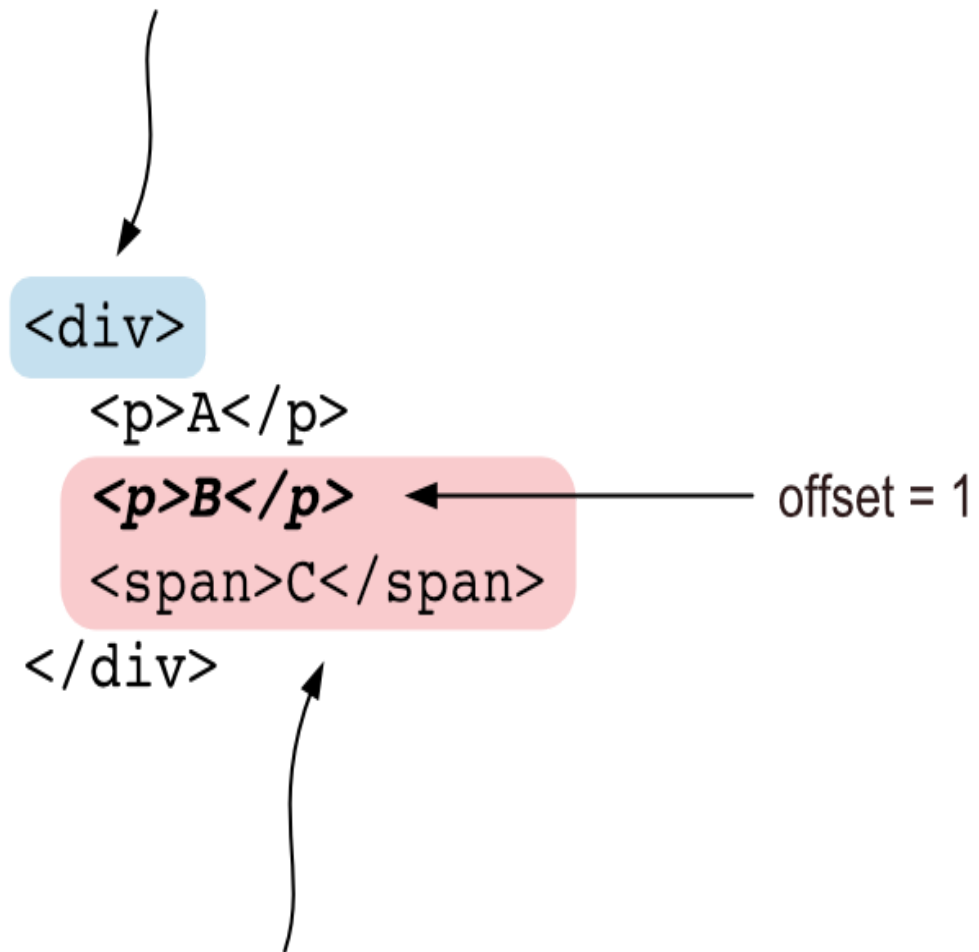
Offset of a component

The offset of a component is the number of elements that precede the component's first element in the parent element when the former's view top-node is a fragment. In other words, it's the index of the component's first element in the parent element.

If you can compute the offset of a component, then you can correct the indices of the move and add operations returned by `arraysDiffSequence()`. The offset for the example component is 1, because the first element of the component is the second element in the parent element (figure [9.10](#)).

Figure 9.10 The offset of the component is 1

The component's host element

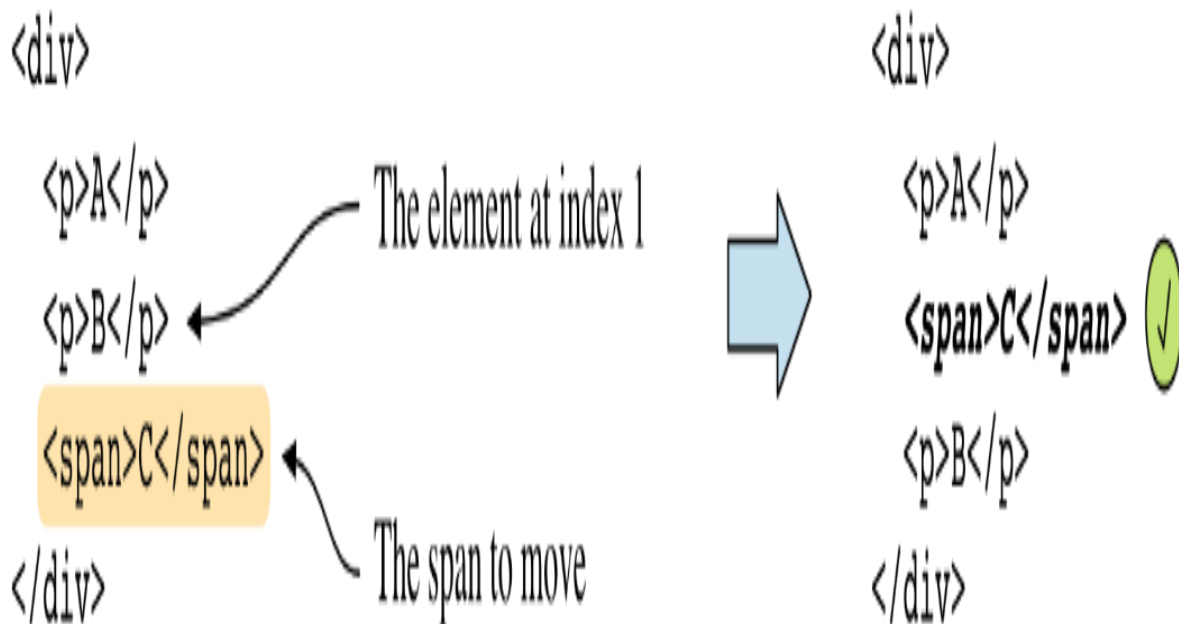


The component's HTML

If we correct the index of the operations by adding the offset, then the operations will be relative to the parent element, and not to the fragment. You can see the correct application of the move operation in figure [9.11](#).

Figure 9.11 Move the `<span>` element from `i=2` to `i=1` (corrected)

1. Move the `<span>` element to $i=0+\text{offset}$



Let's implement a getter for the component's offset. Write the code in bold font in listing [9.7](#), just below the component's constructor.

Listing 9.7 Get the component's first element offset inside the parent element (component.js)

```
import { destroyDOM } from './destroy-dom'
import { DOM_TYPES, extractChildren } from './h'
import { mountDOM } from './mount-dom'
import { patchDOM } from './patch-dom'

export function defineComponent({ render, state }) {
```

```

class Component {
  #isMounted = false
  #vdom = null
  #hostEl = null

  constructor(props = {}) {
    this.props = props
    this.state = state ? state(props) : {}
  }

  get elements() {
    if (this.#vdom == null) { #1
      return []
    }

    if (this.#vdom.type === DOM_TYPES.FRAGMENT) { #2
      return extractChildren(this.#vdom).map((child) =>
child.el)
    }

    return [this.#vdom.el] #3
  }

  get firstElement() {
    return this.elements[0] #4
  }

  get offset() {
    if (this.#vdom.type === DOM_TYPES.FRAGMENT) { #5
      return
Array.from(this.#hostEl.children).indexOf(this.firstElement)
    }

    return 0 #6
  }

  updateState(state) {
    this.state = { ...this.state, ...state }
    this.#patch()
  }

  // --snip-- //
}

return Component
}

```

Now that you can ask a component about its offset, you can correct the indices of the operations returned by `arraysDiffSequence()`. Let's go ahead and refactor the `patchDOM()` function to include the offset.

9.3.4 Patching the DOM using the component's offset

To use the component's offset in the `patchDOM()` function, you need to pass the component instance to the function. Go ahead and do that in listing [9.8](#).

Listing 9.8 Pass the component instance to the `patchDOM()` function (`component.js`)

```
export function defineComponent({ render, state }) {
  class Component {
    // --snip-- //

    #patch() {
      if (!this.#isMounted) {
        throw new Error('Component is not mounted')
      }

      const vdom = this.render()
      this.#vdom = patchDOM(this.#vdom, vdom, this.#hostEl,
this) #1
    }
  }

  return Component
}
```

You now want to include the host component as argument to the `patchDOM()` function and pass it to the `patchChildren()` function. It'll have a default value of `null`. Include the code in bold font in listing [9.9](#), in the `patch-dom.js` file.

Listing 9.9 Pass a host component to the `patchDOM()` function (`patch-dom.js`)

```
export function patchDOM(oldVdom, newVdom, parentEl,
hostComponent = null) { #1
  if (!areNodesEqual(oldVdom, newVdom)) {
    const index = findIndexInParent(parentEl, oldVdom.el)
    destroyDOM(oldVdom)
    mountDOM(newVdom, parentEl, index)
  }
}
```



```

    return newVdom
}

newVdom.el = oldVdom.el

switch (newVdom.type) {
  case DOM_TYPES.TEXT: {
    patchText(oldVdom, newVdom)
    return newVdom
  }

  case DOM_TYPES.ELEMENT: {
    patchElement(oldVdom, newVdom)
    break
  }
}

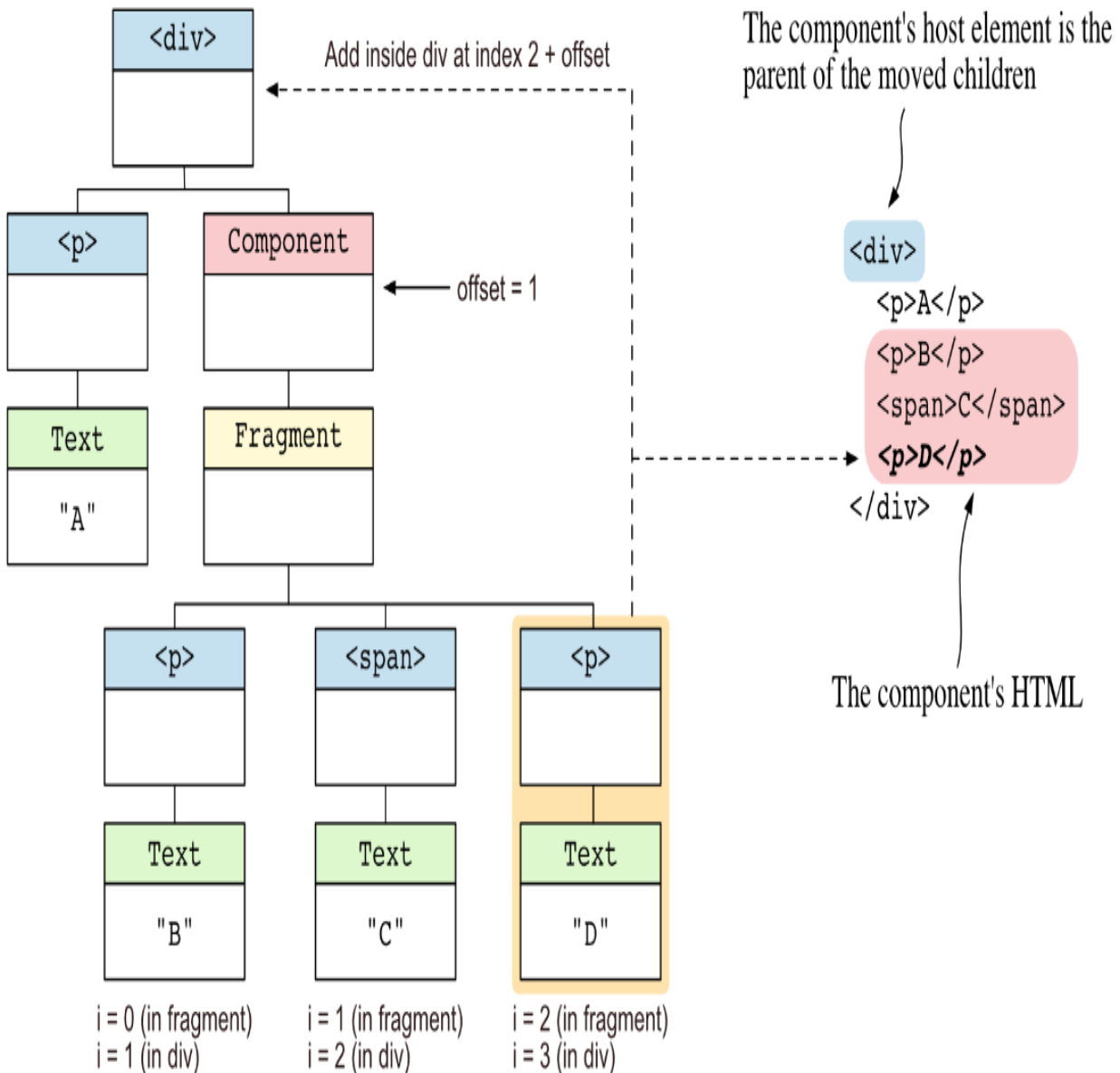
patchChildren(oldVdom, newVdom, hostComponent) #2

return newVdom
}

```

And now with the important part: you need to use the component's offset to correct the indices of the operations returned by `arraysDiffSequence()`, but only if a host component is passed. Otherwise you can assume the component's offset is 0, as this means that you're dealing with a root component—one that doesn't have a parent component—and thus it can't have an offset. Only the operations that reference elements in the DOM by their index need to be corrected. If you recall, only the move and add operations do this. Thus, neither the remove nor the noop operations need to be corrected, but you still have to modify them to pass the `hostComponent` argument back to the `patchDOM()` function.

Figure 9.12 Adding a node to the component's view, with an offset



We saw a graphical example of the move operation in figure 9.8. Figure 9.12 shows a graphical example of the add operation. As you can see, the index of the add operation is relative to the fragment, not to the parent element, so we need to correct it by adding the component's offset.

Include the code in bold font in listing 9.10.

Listing 9.10 Patching the children takes into account the component's offset (patch-dom.js)

```
function patchChildren(oldVdom, newVdom, hostComponent) { #1
  const oldChildren = extractChildren(oldVdom)
```

```

const newChildren = extractChildren(newVdom)
const parentEl = oldVdom.el

const diffSeq = arraysDiffSequence(
  oldChildren,
  newChildren,
  areNodesEqual
)

for (const operation of diffSeq) {
  const { from, index, item } = operation
  const offset = hostComponent?.offset ?? 0 #2

  switch (operation.op) {
    case ARRAY_DIFF_OP.ADD: {
      mountDOM(item, parentEl, index + offset,
hostComponent) #3
      break
    }

    case ARRAY_DIFF_OP.REMOVE: {
      destroyDOM(item)
      break
    }

    case ARRAY_DIFF_OP.MOVE: {
      const el = oldChildren[from].el
      const elAtTargetIndex = parentEl.childNodes[index +
offset] #4

      parentEl.insertBefore(el, elAtTargetIndex)
      patchDOM(
        oldChildren[from],
        newChildren[index],
        parentEl,
        hostComponent #5
      )

      break
    }

    case ARRAY_DIFF_OP.NOOP: {
      patchDOM(
        oldChildren[from],
        newChildren[index],
        parentEl,
        hostComponent #6
      )
    }
  }
}

```

```
        break
    }
}
}
```

With this nuance out of the way, we can now focus on the component's methods.

Exercise 9.3

Can you explain why the problem of the component's offset doesn't arise when the component's view is a single root element?

Find the solution at <http://mng.bz/9QNa>.

9.4 Summary

- A stateful component can be modeled as a class that can be instantiated.
- Each instance of a component has its own state and lifecycle, and manages its own DOM subtree.
- To allow users to create component classes with custom states, render functions, and other methods, a factory function can be used.
- The state of a component is created at instantiation time, and it can be updated by calling the `updateState()` method.
- When the `updateState()` method is called, the component patches its view.
- The `render()` function and event handlers of a component need to be explicitly bound to the component instance, so that `this` refers to the component instance inside them.

10 Component methods

This chapter covers

- Implementing component methods to handle events
- Binding the methods context to the component
- Passing the host component reference to mount and patch functions

What happens when handling an event in a component requires a couple of lines of code? One-liners, like the one below, look fine inside the virtual DOM definition.

```
render() {
  const { count } = this.state

  return hFragment([
    h(
      'button',
      { on: { click: () => this.updateState({ count: count -
1 }) }} #1
      ['Decrement']
    ),
    h('span', {}, [count]),
    h(
      'button',
      { on: { click: () => this.updateState({ count: count +
1 }) }} #2
      ['Increment']
    )
  ])
}
```

But what happens when you want to do more than fits in a single line? Well, you can fit it inside the virtual DOM definition, but it'll make the code harder to read. Look at the following example of a list, where clicking a button that loads more items from the server:

```
render() {
  const { items, offset, isLoading } = this.state
```

```

return hFragment([
  h(
    'ul',
    {},
    items.map((item) => h('li', {}, [item])),
  ),
  isLoading
    ? h('p', {}, ['Loading...'])
    : h(
        'button',
        {
          on: {
            click: async () => { #1
              const { items, offset } = this.state
              this.updateState({ isLoading: true })

              const newItems = await fetchItems(offset)

              this.updateState({
                items: [...items, ...newItems],
                isLoading: false,
                offset: offset + newItems.length,
              })
            },
          },
        },
        ['Load more'],
      ),
  ])
}

```

This doesn't look as nice as the previous example. What we want to do instead is to move the event handler code to a method of the component, and reference the method as the handler for the event. This results in a much cleaner code:

```

render() {
  const { items, offset, isLoading } = this.state

  return hFragment([
    h(
      'ul',
      {},
      items.map((item) => h('li', {}, [item])),
    ),
    isLoading

```

```

    ? h('p', {}, ['Loading...'])
    : h(
      'button',
      {
        on: { click: this.loadMore }, #1
      },
      ['Load more'],
    ),
  ])
}

```

And so, a `loadMore()` method inside the component would handle the event. We need a way to add custom methods to the component. How do you go about doing that? We can pass the `defineComponent()` function, together with the `render()` and `state()` functions, other arbitrary methods. For example, in the previous example, we'd define the component as follows:

```

const InfiniteList = defineComponent(
  state() { ... },
  render() { ... },
  async loadMore() { #1
    const { items, offset } = this.state
    this.updateState({ isLoading: true })

    const newItems = await fetchItems(offset)

    this.updateState({
      items: [...items, ...newItems],
      isLoading: false,
      offset: offset + newItems.length,
    })
  },
)

```

This is the objective of this chapter: to modify the `defineComponent()` function so that it can receive custom methods that can be used to handle events. The changes are straightforward, but the methods won't be correctly bound to the component's context when used as event handlers. As you'll see in a minute, you need to explicitly bind the event handlers to the component's context, and this is what'll take most of the chapter.

Important

You can find all the listings in this chapter in the [listings/ch10 directory](#) of the book's repository.

The code you write in this chapter is for the framework's version v3.0, which you'll publish in chapter 12. Therefore, the code in this chapter can be checked out from [the ch12 label](#):

```
$ git switch --detach ch12
```

Code catch-up

In the previous chapter you wrote the `defineComponent()` function, which receives the `render()` and `state()` functions as arguments, and returns a `Component` class. You implemented the following methods in the `Component` class:

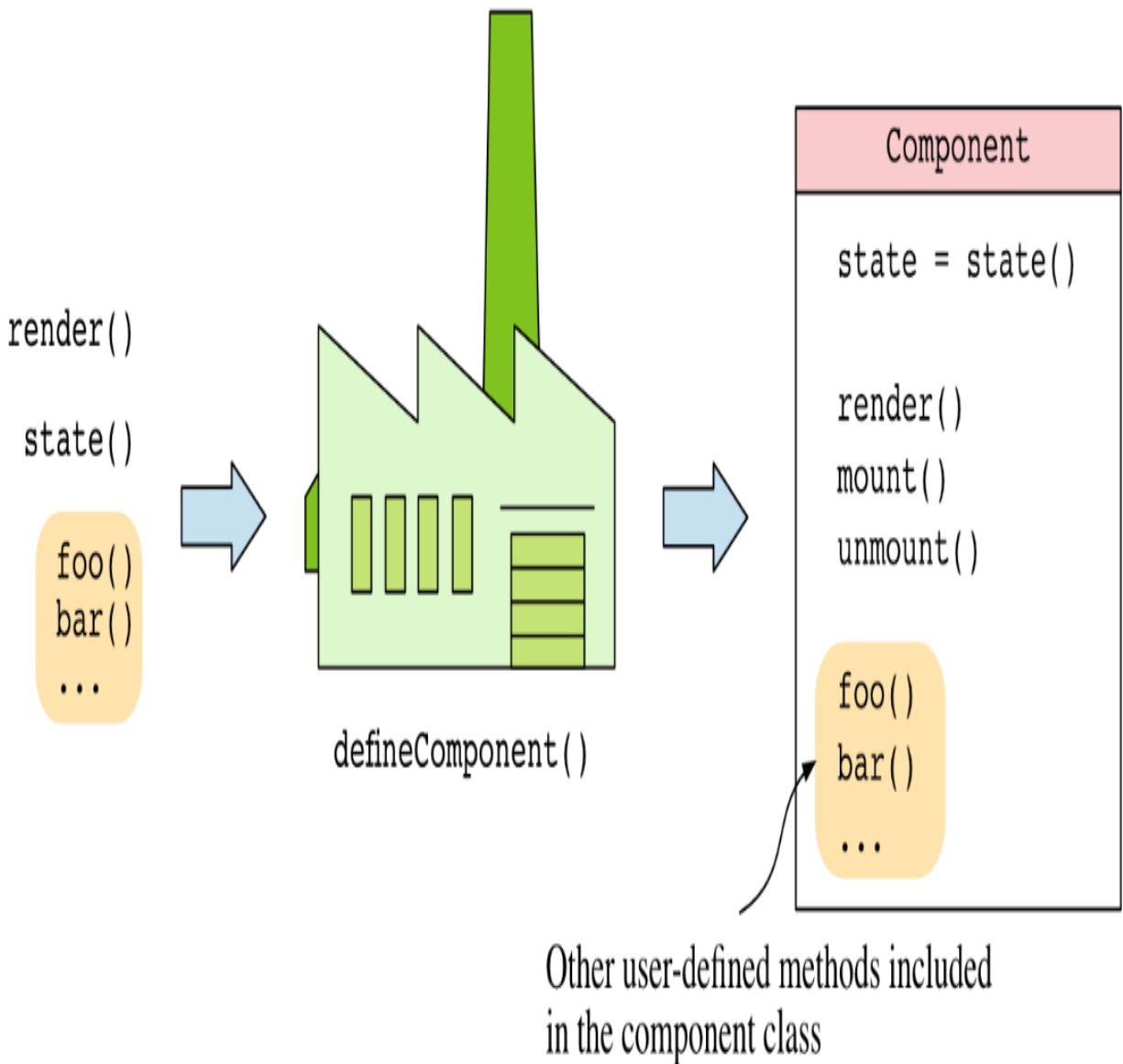
- `render()` — renders the component's virtual DOM
- `mount()` — mounts the component's view into the DOM
- `unmount()` — unmounts the component from the DOM
- `#patch()` — patches the component's view in the DOM
- `updateState()` — updates the component's state and patches the view in the DOM

You also modified the `patchChildren()` function to receive one more argument: the host component (the component that owns the children). You needed this component to calculate the offset of the children, so that the operations that require moving nodes in the DOM can be performed correctly. For the `patchChildren()` function to receive the host component, you also had to modify the `patchDOM()` function, adding the `hostComponent` argument to it.

10.1 Component methods

We need to update our analogy from the previous chapter of `defineComponent()` as being a component factory. This factory, in addition to the `render()` and `state()` functions, can also receive other methods that will be added to the component's prototype (figure [10.1](#)).

Figure 10.1 Using `defineComponent()` to define a component with custom methods



To add custom methods to the component you can leverage JavaScript's [prototypal inheritance](#). Classes in JavaScript are just syntactic sugar for prototypes, and extending a prototype to include a new method is as simple as:

```
class Component {}  
  
Component.prototype.loadMore = function () { ... }
```

But, you want to prevent the user from redefining a method that already exists in the component, such as `updateState()` or `mount()`. Overriding these methods would break the component. For that you want to check if the method already exists in the component's prototype using the `hasOwnProperty()` method. Unfortunately, this method is not safe to use, and ESLint will complain about it. Fortunately for us, the same ESLint rule that complains about `hasOwnProperty()` also provides a suggestion for a safe implementation of it.

Note

If you want to understand better why using an object's `hasOwnProperty()` method is unsafe, refer to <http://mng.bz/yQOE>. You might also want to learn more about JavaScript's prototypal inheritance. I recommend you take some time to read <http://mng.bz/MBem>.

Inside the *utils/objects.js* file, add the `hasOwnProperty()` function as in listing [10.1](#).

Listing 10.1 A safe implementation of `hasOwnProperty()` (*utils/objects.js*)

```
export function hasOwnProperty(obj, prop) {  
  return Object.prototype.hasOwnProperty.call(obj, prop)  
}
```

And now, go back to the *component.js* file and write the code in bold font in listing [10.2](#) to add custom methods to the component.

Listing 10.2 Adding custom methods to the component (*component.js*)

```
import { destroyDOM } from './destroy-dom'  
import { DOM_TYPES, extractChildren } from './h'  
import { mountDOM } from './mount-dom'  
import { patchDOM } from './patch-dom'  
import { hasOwnProperty } from './utils/objects'  
  
export function defineComponent({ render, state, ...methods  
}) { #1  
  class Component {  
    // --snip--  
  }  
}
```

```

    for (const methodName in methods) { #2
      if (hasOwnProperty(Component, methodName)) { #3
        throw new Error(
          `Method "${methodName}()" already exists in the
component.`
        )
      }

      Component.prototype[methodName] = methods[methodName] #4
    }

    return Component
  }

```

This is great, but... The `InfiniteList` component from the example above (reproduced below) wouldn't work now.

```

const InfiniteList = defineComponent(
  state() { ... },

  render() {
    const { items, offset, isLoading } = this.state

    return hFragment([
      h(
        'ul',
        {},
        items.map((item) => h('li', {}, [item])),
      ),
      isLoading
        ? h('p', {}, ['Loading...'])
        : h(
            'button',
            {
              on: { click: this.loadMore }
            },
            ['Load more'],
          ),
    ])
  },

  async loadMore() { ... },
)

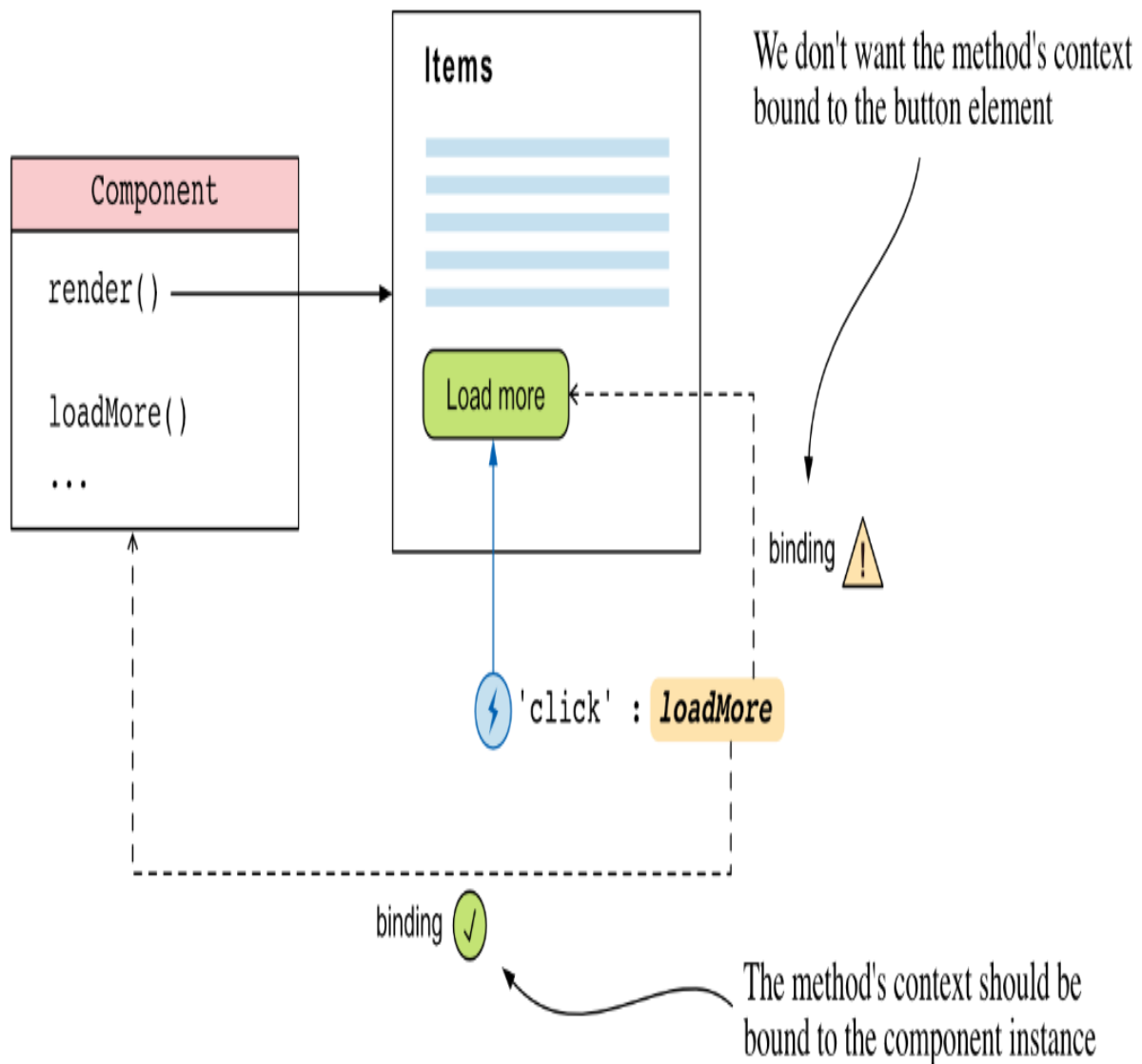
```

We get an error because `this` is bound to the `<button>` element that's emitting the click event, and not to the component. Hence, `this.loadMore()` is undefined, because buttons in the DOM don't include a `loadMore()` method—if they did, you wouldn't see any error, but your `loadMore()` method would never be called. The culprit is the line in bold:

```
h(
  'button',
  {
    // Fails, because 'this' is bound to the <button> element
    on: { click: this.loadMore },
  },
  ['Load more'],
)
```

When you add an event handler using the `addEventListener()` method, the `this` keyword of the function is bound to the element that emitted the event, in the example above, the `<button>` element. You don't want the `this` keyword to be bound to the element, you want it to be bound to the component, so that the methods inside it can be correctly resolved, as shown in figure [10.2](#).

Figure 10.2 Binding of the `loadMore()` method's `this` keyword



There's a quick way we can get around this, and that's by wrapping the event handler in an arrow function:

```
h(  
  'button',  
  {  
    // Works by wrapping the event handler in an arrow  
    function  
    on: { click: () => this.loadMore() },  
  },  
  ['Load more'],  
)
```

Important

Arrow functions can't have their own `this` keyword, so the `this` keyword inside the arrow function is inherited from the containing function. Arrow functions are lexically scoped, which means that the `this` keyword is bound to the value of `this` in the enclosing (function or global) scope.

In our case this is the `render()` function, and the `this` keyword is bound to the component (and that's why you had to do `render.call(this)` inside the component's render method). Arrow functions use [lexical scoping](#), so the `addEventListener()` method can't bind the `this` keyword to the element that emitted the event.

But we don't want to force the user to wrap the event handler in an arrow function. We want them to be able to just reference a method from the component, and have it work. For that, we need to explicitly bind the event handler to the component, so that the `this` keyword is bound to the component.

10.2 Binding event handlers to the component

Go ahead and open the *events.js* file; there are some modifications we need to make to the `addEventListener()` function. You want to pass the host component reference to the function, which can be `null` in case the event handler is not bound to a component. If the `hostComponent` argument is not `null`, you want to bind the event handler to the component using the `.apply()` method (which is similar to `.call()`, but you can pass it an array of arguments instead of passing arguments one by one). When the `hostComponent` is `null` you just want to call the event handler as is:

```
hostComponent
  ? handler.apply(hostComponent)
  : handler()
```

You need to keep in mind that an event handler can receive arguments, so you need to pass those arguments to the event handler when you call it. Here's what you can do: you can wrap the code in the previous listing in a new handler function that passes its arguments to the original handler function:

```
function boundHandler() {
  hostComponent
    ? handler.apply(hostComponent, arguments)
    : handler(...arguments)
}
```

Note

If you're not familiar with the `arguments` object, you can read more about it at <http://mng.bz/a1xY>. In a nutshell, it's an array-like object that contains the arguments passed to a function. Bear in mind that, inside an arrow function, the `arguments` object is not available.

Now that the plan is clear, modify your code inside the *events.js* file as in listing [10.3](#).

Listing 10.3 Event handlers are bound to the component (events.js)

```
export function addEventListeners(
  listeners = {},
  el,
  hostComponent = null #1
) {
  const addedListeners = {}

  Object.entries(listeners).forEach(([eventName, handler]) => {
    const listener = addEventListener(
      eventName,
      handler,
      el, hostComponent
    ) #2
    addedListeners[eventName] = listener
  })

  return addedListeners
}

export function addEventListener(
  eventName,
  handler,
  el,
  hostComponent = null #3
) {
```

```

    el.addEventListener(eventName, handler)
    return handler
    function boundHandler() {
        hostComponent
        ? handler.apply(hostComponent, arguments) #4
        : handler(...arguments) #5
    }

    el.addEventListener(eventName, boundHandler) #6

    return boundHandler
}

```

Now that your event handlers are properly bound to the component—in case there is one—you need to modify the `mountDOM()` and `patchDOM()` functions so they pass the host component to the `addEventListener()` function.

10.3 Mounting the DOM with a host component

Open the *mount-dom.js* file and modify the `mountDOM()` function, adding the code in bold font in listing [10.4](#), so that the host component can be passed as argument.

Listing 10.4 Passing the host component to the `mountDOM()` function (*mount-dom.js*)

```

export function mountDOM(
    vdom,
    parentEl,
    index,
    hostComponent = null #1
) {
    switch (vdom.type) {
        case DOM_TYPES.TEXT: {
            createTextNode(vdom, parentEl, index)
            break
        }

        case DOM_TYPES.ELEMENT: {
            createElementNode(vdom, parentEl, index, hostComponent)
#2
            break
        }
    }
}

```



```

        case DOM_TYPES.FRAGMENT: {
            createFragmentNodes(vdom, parentEl, index,
hostComponent) #3
            break
        }

        default: {
            throw new Error(`Can't mount DOM of type:
${vdom.type}`)
        }
    }
}

```

You passed the `hostElement` reference down to the `createElementNode()` and `createFragmentNodes()` functions. Let's now modify them to accept that argument. First, add the `hostComponent` argument to the `createElementNode()` function, as in listing [10.5](#). From there, you can pass the `hostComponent` argument to the `addProps()` and back to `mountDOM()` functions.

Listing 10.5 Passing the host component to the `createElementNode()` function (mount-dom.js)

```

function createElementNode(vdom, parentEl, index,
hostComponent) { #1
    const { tag, props, children } = vdom

    const element = document.createElement(tag)
    addProps(element, props, vdom, hostComponent) #2
    vdom.el = element

    children.forEach((child) =>
        mountDOM(child, element, null, hostComponent) #3
    )
    insert(element, parentEl, index)
}

```

Now, modify the `addProps()` function so that the passed in `hostComponent` argument is passed to the `addEventListener()` function, as in listing [10.6](#). This is in fact, the most important change in this section: it's where we pass the host component to the `addEventListener()` function you modified earlier.

Listing 10.6 Passing the host component to the `addProps()` function (mount-dom.js)

```
function addProps(el, props, vdom, hostComponent) { #1
  const { on: events, ...attrs } = props

  vdom.listeners = addEventListeners(events, el,
hostComponent) #2
  setAttributes(el, attrs)
}
```

And last, modify the `createFragmentNodes()` function so that the `hostComponent` argument is passed back to the `mountDOM()` function, as in listing [10.7](#).

Listing 10.7 Passing the host component to the `createFragmentNodes()` function (`mount-dom.js`)

```
function createFragmentNodes(
  vdom,
  parentEl,
  index,
  hostComponent #1
) {
  const { children } = vdom
  vdom.el = parentEl

  children.forEach((child) =>
    mountDOM(
      child,
      parentEl,
      index ? index + i : null,
      hostComponent #2
    )
  )
}
```

Great! Let's end the chapter by modifying the `patchDOM()` function so that the host component is passed and can be used to patch event handlers.

10.4 Patching the DOM with a host component

If you recall, you modified the `patchDOM()` function in the previous chapter so that it passes the host component to the `patchElement()` function. You did this so that you could pass it to the `patchChildren()` function, so that it take into account the component's offset. Open the *patch-dom.js* file again, and

pass the `hostComponent` reference this time to the `mountDOM()` function (the case when the DOM is recreated), and to the `patchElement()` function, as in listing [10.8](#).

Listing 10.8 Passing the host component to `mountDOM()` and `patchElement()` inside the `patchDOM()` function (`patch-dom.js`)

```
export function patchDOM(
  oldVdom,
  newVdom,
  parentEl,
  hostComponent = null
) {
  if (!areNodesEqual(oldVdom, newVdom)) {
    const index = findIndexInParent(parentEl, oldVdom.el)
    destroyDOM(oldVdom)
    mountDOM(newVdom, parentEl, index, hostComponent) #1

    return newVdom
  }

  newVdom.el = oldVdom.el

  switch (newVdom.type) {
    case DOM_TYPES.TEXT: {
      patchText(oldVdom, newVdom)
      return newVdom
    }

    case DOM_TYPES.ELEMENT: {
      patchElement(oldVdom, newVdom, hostComponent) #2
      break
    }
  }

  patchChildren(oldVdom, newVdom, hostComponent)

  return newVdom
}
```

Let's now modify the `patchElement()` function so that it receives the `hostComponent` argument, as in listing [10.9](#). You want to also pass it down to the `patchEvents()` function, which you'll modify next.

Listing 10.9 Passing the host component to the `patchElement()` function (`patch-dom.js`)

```

function patchElement(oldVdom, newVdom, hostComponent) { #1
  const el = oldVdom.el
  const {
    class: oldClass,
    style: oldStyle,
    on: oldEvents,
    ...oldAttrs
  } = oldVdom.props
  const {
    class: newClass,
    style: newStyle,
    on: newEvents,
    ...newAttrs
  } = newVdom.props
  const { listeners: oldListeners } = oldVdom

  patchAttrs(el, oldAttrs, newAttrs)
  patchClasses(el, oldClass, newClass)
  patchStyles(el, oldStyle, newStyle)
  newVdom.listeners = patchEvents(
    el,
    oldListeners,
    oldEvents,
    newEvents,
    hostComponent #2
  )
}

```

Now into the `patchEvents()` function, add the `hostComponent` argument, as in listing [10.10](#). In the cases where an event handler is added or updated, the `addEventListener()` is called, so remember to pass the `hostComponent` argument to it.

Listing 10.10 Passing the host component to the `patchEvents()` function (`patch-dom.js`)

```

function patchEvents(
  el,
  oldListeners = {},
  oldEvents = {},
  newEvents = {},
  hostComponent #1
) {
  const { removed, added, updated } = objectsDiff(oldEvents,
    newEvents)

  for (const eventName of removed.concat(updated)) {

```

```

    el.removeEventListener(eventName,
oldListeners[eventName])
  }
  const addedListeners = {}

  for (const eventName of added.concat(updated)) {
    const listener = addEventListener(
      eventName,
      newEvents[eventName],
      el,
      hostComponent #2
    )
    addedListeners[eventName] = listener
  }

  return addedListeners
}

```

And last, you need to go back to the *component.js* file and modify the `mount()` function so that it passes the `hostComponent` argument to the `mountDOM()` function, as in listing [10.11](#).

Listing 10.11 Component's `mount()` passes this reference to `mountDOM()` (component.js)

```

export function defineComponent({ render, state, ...methods
}) {
  class Component {
    // --snip-- //

    mount(hostEl, index = null) {
      if (this.#isMounted) {
        throw new Error('Component is already mounted')
      }

      this.#vdom = this.render()
      mountDOM(this.#vdom, hostEl, index, this) #1

      this.#hostEl = hostEl
      this.#isMounted = true
    }

    // --snip-- //
  }

  // --snip-- //
}

```

```
    return Component
  }
```

You're done! Before we close the chapter, let's review what should be the result of the `defineComponent()` function returning a `Component` class. Then I'll give you a challenge exercise to test your knowledge.

10.4.1 Result

Your *component.js* file should look like listing [10.12](#). Make sure you got all the details right before moving on to the next chapter. Can you explain what each property and method of the `Component` class does?

Listing 10.12 The result of the `defineComponent()` function returning a `Component` class (*component.js*)

```
import { destroyDOM } from './destroy-dom'
import { DOM_TYPES, extractChildren } from './h'
import { mountDOM } from './mount-dom'
import { patchDOM } from './patch-dom'
import { hasOwnProperty } from './utils/objects'

export function defineComponent({ render, state, ...methods
}) {
  class Component {
    #isMounted = false
    #vdom = null
    #hostEl = null

    constructor(props = {}) {
      this.props = props
      this.state = state ? state(props) : {}
    }

    get elements() {
      if (this.#vdom == null) {
        return []
      }

      if (this.#vdom.type === DOM_TYPES.FRAGMENT) {
        return extractChildren(this.#vdom).map((child) =>
child.el)
      }
    }
  }
}
```

```

    return [this.#vdom.el]
  }

  get firstElement() {
    return this.elements[0]
  }

  get offset() {
    if (this.#vdom.type === DOM_TYPES.FRAGMENT) {
      return
      Array.from(this.#hostEl.children).indexOf(this.firstElement)
    }

    return 0
  }

  updateState(state) {
    this.state = { ...this.state, ...state }
    this.#patch()
  }

  render() {
    return render.call(this)
  }

  mount(hostEl, index = null) {
    if (this.#isMounted) {
      throw new Error('Component is already mounted')
    }

    this.#vdom = this.render()
    mountDOM(this.#vdom, hostEl, index, this)

    this.#hostEl = hostEl
    this.#isMounted = true
  }

  unmount() {
    if (!this.#isMounted) {
      throw new Error('Component is not mounted')
    }

    destroyDOM(this.#vdom)

    this.#vdom = null
    this.#hostEl = null
    this.#isMounted = false
  }

```

```

    #patch() {
      if (!this.#isMounted) {
        throw new Error('Component is not mounted')
      }

      const vdom = this.render()
      this.#vdom = patchDOM(this.#vdom, vdom, this.#hostEl,
this)
    }
  }

  for (const methodName in methods) {
    if (hasOwnProperty(Component, methodName)) {
      throw new Error(
        `Method "${methodName}()" already exists in the
component.`
      )
    }
  }

  Component.prototype[methodName] = methods[methodName]
}

return Component
}

```

And not it's time for the challenge exercise!

Exercise 10.1 — Challenge

There is a free API that returns random cocktails:

GET <https://www.thecocktaildb.com/api/json/v1/1/random.php>

Go ahead and try it out in your browser to see what it returns.

For this challenge exercise I want you to write the code for a component that fetches a random cocktail and displays it. When the component is mounted, it should show an HTML similar to the following:

```

<h1>Random cocktail</h1>
<button>Get a cocktail</button>

```


When the button is clicked, the component should fetch a random cocktail and display it. You should display a "Loading..." message while the cocktail is being fetched, and maybe add an artificial timeout of 1 or 2 seconds for the message to have time to be displayed. When the cocktail is fetched, display its name (`strDrink` field), image (`strDrinkThumb` field), and preparation instructions (`strInstructions` field). Include a button to load another cocktail:

```
<h1>Cocktail name</h1>
<p>Preparation instructions</p>

<button>Get another cocktail</button>
```

Copy and paste the code for your component (and the framework code that's needed for it to work) into the console of your browser and mount it inside a website of your choice.

Find the solution at <http://mng.bz/j1By>.

10.5 Summary

- Methods are a convenient way of handling events in components. When the handling logic has more than a couple of lines, it's cleaner to define a method for it.
- Methods can be used as event handlers in the `on` object of the `h()` function. They can be simply referenced as `this.methodName` in the `on` object.
- Functions added as event handlers have to be explicitly bound to the component instance so that `this` inside the function refers to the component.

11 Subcomponents— Communication via props and events

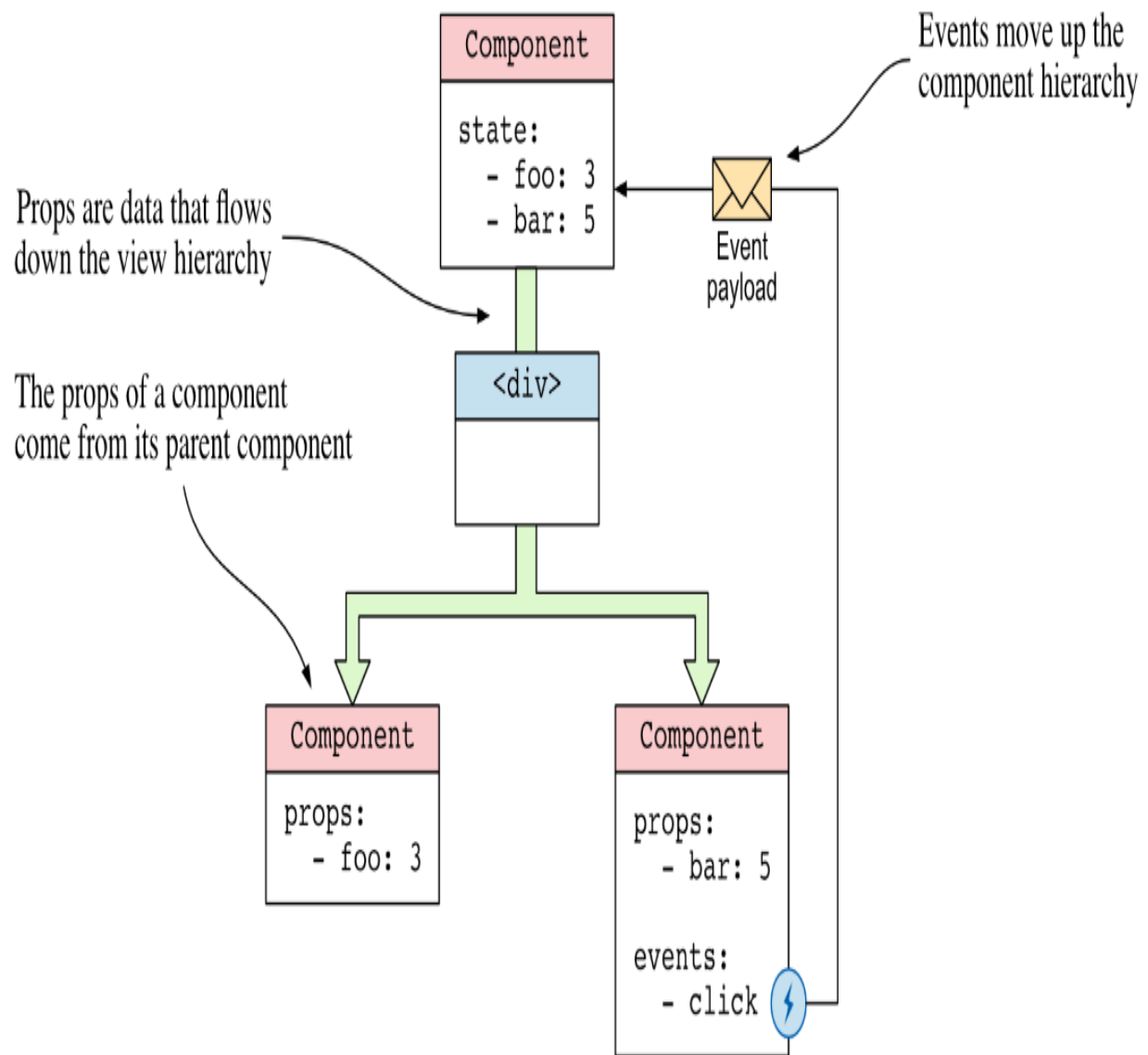
This chapter covers

- Adding a new virtual DOM type to represent components
- Implementing subcomponents: components inside other components
- Passing data from a parent component to its children using props
- Communicating between components using events

What's the value of a component if it can't include other components inside it? The `Component` class you've implemented in the previous two chapters can't return other components from its `render()` method; it can only return fragments, elements, or text nodes. But as you can imagine, building a complex application using just a single component that renders the entire view is... well, not very practical.

In this chapter, you learn how to add subcomponents (components inside other components) to your framework, and how to communicate between them using props and events. A parent component can pass data to its children using props, and children can communicate with their parents emitting events, as illustrated in figure [11.1](#).

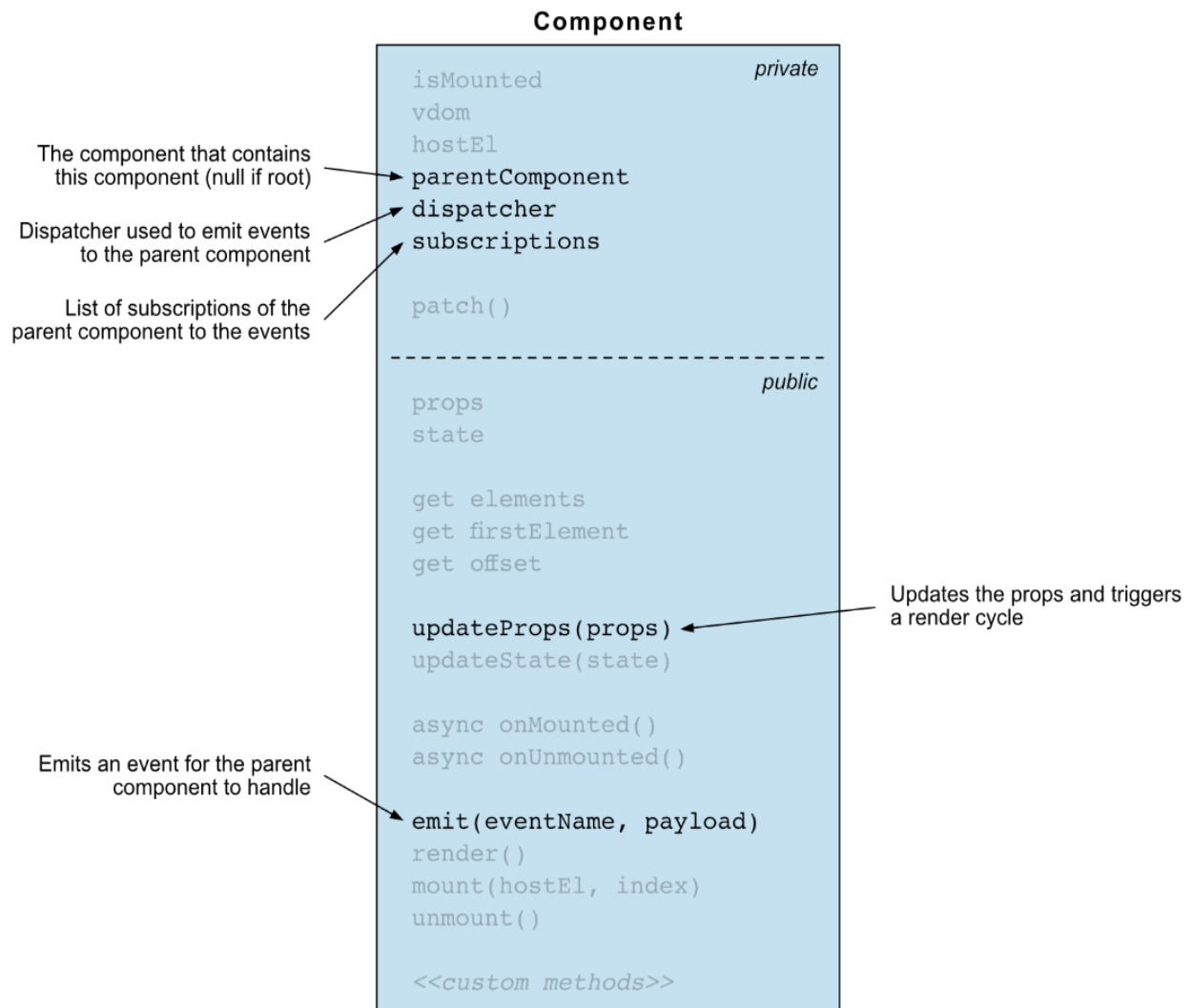
Figure 11.1 Props flow down, events flow up



Let's briefly review the "anatomy of a stateful component" diagram from chapter 9 and the annotated properties and methods you'll implement in this chapter (figure [11.2](#)).

Figure 11.2 Anatomy of a stateful component

Anatomy of a stateful component



In this chapter you'll add the following private properties to the `Component` class:

- `parentComponent`: A reference to the component that contains the current component. This property is `null` for top-level components.
- `dispatcher`: An instance of the `Dispatcher` class used to emit events and subscribe handlers to them.
- `subscriptions`: An array of subscriptions of event handlers to events emitted by the component. These need to be unsubscribed when the component is unmounted.

You'll also implement two important methods, namely:

- `updateProps()`: This method is called when the component receives new props. It updates the component's `props` property and triggers a re-render. I'll show you an optional optimization you can implement to avoid re-rendering the component if the new props are the same as the current ones.
- `emit()`: This method is used to emit events from the component.

When you finish this chapter, you'll have a fully functional component prototype that handles its own state and lifecycle, and that can include other components inside it. That's an enormous step forward in the development of your framework, so let's get started!

Important

You can find all the listings in this chapter in the [listings/ch11 directory](#) of the book's repository.

The code you write in this chapter is for the framework's version v3.0, which you'll publish in chapter 12. Therefore, the code in this chapter can be checked out from [the ch12 label](#):

```
$ git switch --detach ch12
```

Code catch-up

In the previous chapter you modified the `defineComponent()` function to include custom methods in the component's prototype. You had to modify the `addEventListener()` function so that if the element to which the listener is to be attached has a host component, the listener is bound to it. This change required you to modify the `mountDOM()`, `createElementNode()`, and `createFragmentNodes()`. You also modified the `patchDOM()`, `patchElement()`, and `patchEvents()` functions, in all cases to add the host component argument that needs to be passed around, so the `addEventListener()` function can access it.

11.1 Adding components as a new virtual DOM type

The reason that your component—as implemented in the previous two chapters—can’t return other components is that you currently have no way to include components in the virtual DOM tree. Ideally, you’d want to use the `h()` function to create component nodes, like this:

```
const FooComponent = defineComponent({ ... })
const BarComponent = defineComponent({ ... })

const ParentComponent = defineComponent({
  render() {
    return hFragment([
      h(FooComponent),
      h(BarComponent),
    ])
  }
})
```

The `render()` method of the `ParentComponent` returns a fragment with two component nodes: `FooComponent` and `BarComponent`. The virtual DOM tree could look something like the following:

```
{
  type: 'fragment',
  children: [
    { type: 'component', tag: FooComponent, props: {},
children: [] },
    { type: 'component', tag: BarComponent, props: {},
children: [] },
  ]
}
```

Notice that the `type` property of the component nodes is `'component'`, and that the `tag` property contains the component prototype instead of a string with the HTML tag name.

But the `h()` function doesn’t know how to mount component nodes. At least not yet, so let’s teach it how to do it.

To add components inside a virtual DOM tree, you need to add a new virtual DOM type to the `DOM_TYPES` constant to represent them. Then, you need to teach the `h()` function how to differentiate between element nodes and component nodes. A way to do this is to check whether the `tag` argument is a string, in which case you know the node must be an element node.

Note

In JavaScript, the `typeof` operator returns `'function'` for classes, as you can check yourself in the console:

```
class Component {}  
typeof Component // 'function'
```

You could use this fact to check whether the `tag` argument is a component, but I chose to check whether the `tag` argument is a string. Both approaches work the same.

Open the *h.js* file, add the `COMPONENT` constant to the `DOM_TYPES` object and the condition to check if the `tag` argument is a string, as the code in bold font in listing [11.1](#) shows.

Listing 11.1 Adding components as a new virtual DOM type (h.js)

```
export const DOM_TYPES = {  
  TEXT: 'text',  
  ELEMENT: 'element',  
  FRAGMENT: 'fragment',  
  COMPONENT: 'component',  
}  
  
export function h(tag, props = {}, children = []) {  
  const type =  
    typeof tag === 'string' ? DOM_TYPES.ELEMENT :  
    DOM_TYPES.COMPONENT  
  
  return {  
    tag,  
    props,  
    type,  
    children: mapTextNodes(withoutNulls(children)),  
    type: DOM_TYPES.ELEMENT,  
  }
```

```
}  
}
```

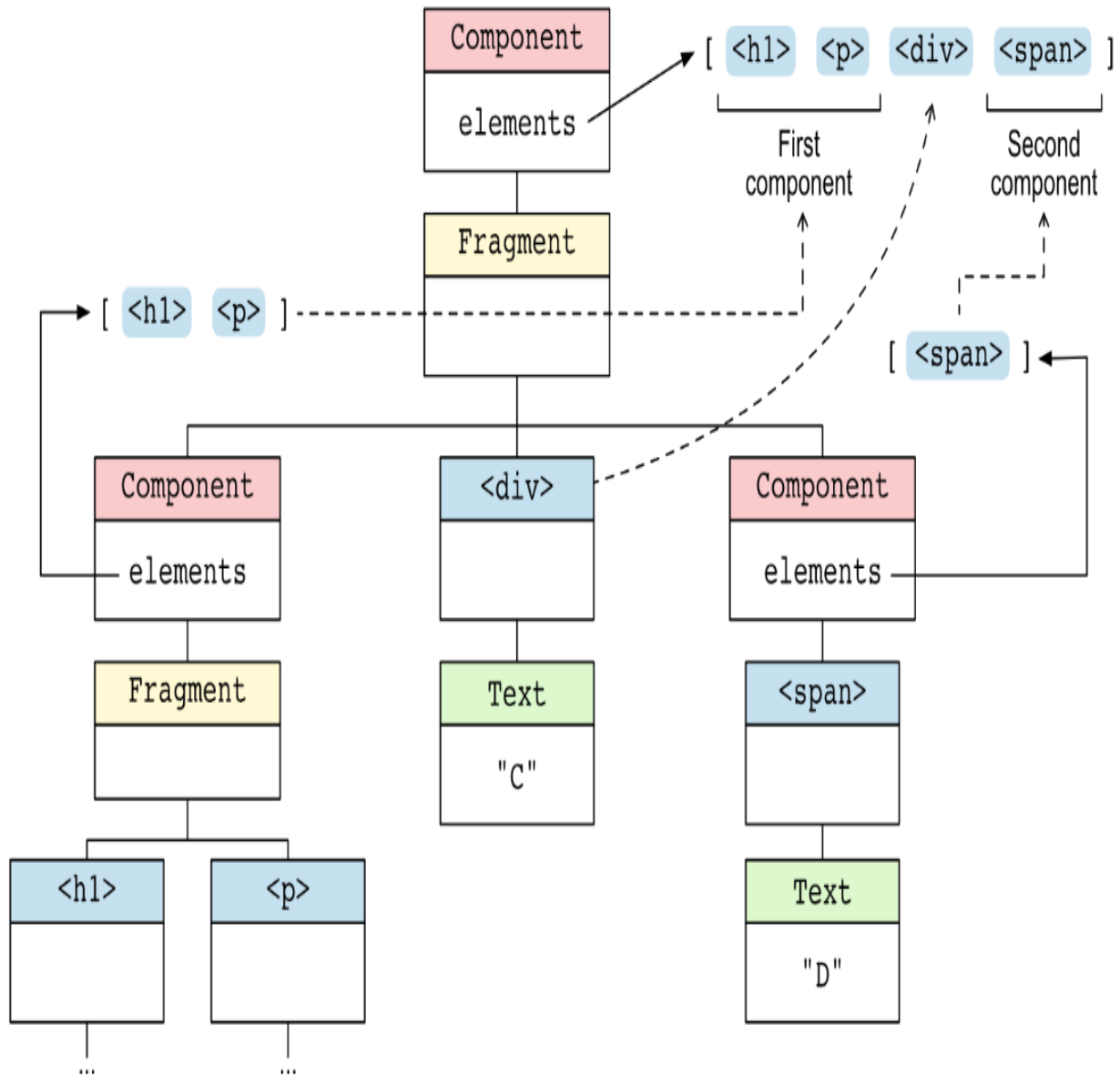
Now that virtual node trees can include component nodes, you want to update the `Component` class `elements` getter to return the elements of a component node.

11.1.1 Updating the `elements` getter

You'll recall that in chapter 9, we discussed the `elements` getter, which retrieves the top-level HTML elements of the virtual node tree representing the component's view. You used this getter to calculate the `offset` property of the component. This offset was crucial for accurately updating the component's DOM when the top-level node was a fragment.

You need to modify the case when the top-level node is a fragment to account for the subcomponents that might be inside the fragment. In this case you call the `elements` getter recursively, as illustrated in figure [11.3](#). You want to look at the figure from bottom to top. Each component in the hierarchy assembles an array of its view's first-level elements. If it finds a component, it calls the `elements` getter—hence the recursion. As you can see, the top-most component has a fragment at the root of its view, so it looks inside of it to find the first-level elements. First, it finds a component which has two elements in its first level (inside the fragment): an `<h1>` and a `<p>` element. Then, it finds a `<div>` element, which is the second element in the fragment. Finally, it finds another component, which has a single `<p>` element in its first level.

Figure 11.3 Calling the `elements` getter recursively to get the parent component's elements



The component's `elements` getter returns an array and not a single element, so you want to use the `flatMap()` method to flatten the array of arrays. Make these changes in the *component.js* file, as shown in bold font in listing [11.2](#).

Listing 11.2 Extracting the elements of a component with subcomponents (component.js)

```

export function defineComponent({ render, state, ...methods
}) {
  class Component {
    // --snip-- //
  }
}

```

```

    get elements() {
      if (this.#vdom == null) {
        return []
      }

      if (this.#vdom.type === DOM_TYPES.FRAGMENT) {
        return extractChildren(this.#vdom).map((child) =>
child.el)
return extractChildren(this.#vdom).flatMap((child) =>
{ #1
    if (child.type === DOM_TYPES.COMPONENT) { #2
      return child.component.elements #3
    }

    return [child.el] #4
  })
}

return [this.#vdom.el]
}

// --snip-- //
}

// --snip-- //

return Component
}

```

With this small—but necessary—change out of the way, let’s focus on mounting, destroying, and patching component nodes.

11.1.2 Mounting component virtual nodes

To mount component nodes, you first need to include the `DOM_TYPES.COMPONENT` case in the `mountDOM()` function’s `switch` statement. The code in bold font in listing [11.3](#) shows how to do this. Make this change in the *mount-dom.js* file.

Listing 11.3 Mounting component virtual nodes (mount-dom.js)

```

export function mountDOM(vdom, parentEl, index, hostComponent
= null) {
  switch (vdom.type) {

```

```

    case DOM_TYPES.TEXT: {
      createTextNode(vdom, parentEl, index)
      break
    }

    case DOM_TYPES.ELEMENT: {
      createElementNode(vdom, parentEl, index, hostComponent)
      break
    }

    case DOM_TYPES.FRAGMENT: {
      createFragmentNodes(vdom, parentEl, hostComponent)
      break
    }

    case DOM_TYPES.COMPONENT: { #1
      createComponentNode(vdom, parentEl, index,
hostComponent) #2
      break
    }

    default: {
      throw new Error(`Can't mount DOM of type:
${vdom.type}`)
    }
  }
}

```

The job of mounting the component is delegated to a function called `createComponentNode()`, which you need to implement next. Let's see what this function needs to do.

First of all, you want to extract the component's instance from the virtual node's `tag` key and instantiate it—something similar to the following:

```

const vnode = {
  type: 'component',
  tag: FooComponent,
  props: {},
}

const Component = vnode.tag
const component = new Component()

```

In chapter 9, you learned that a component class can be instantiated by passing it a `props` object. These props contain data received from the parent component. The virtual node you defined also includes a `props` object. Initially, you used it to pass attributes and event handlers to element nodes. You can use it to pass props and event handlers to component nodes as well.

Let's explore an example that demonstrates how this can be achieved. Say that you have defined a component called `Greetings` that receives a `name` prop and renders a paragraph with a greeting message:

```
const Greetings = defineComponent({
  render() {
    const { name } = this.props

    return h('p', {}, [`Hello, ${name}!`])
  }
})
```

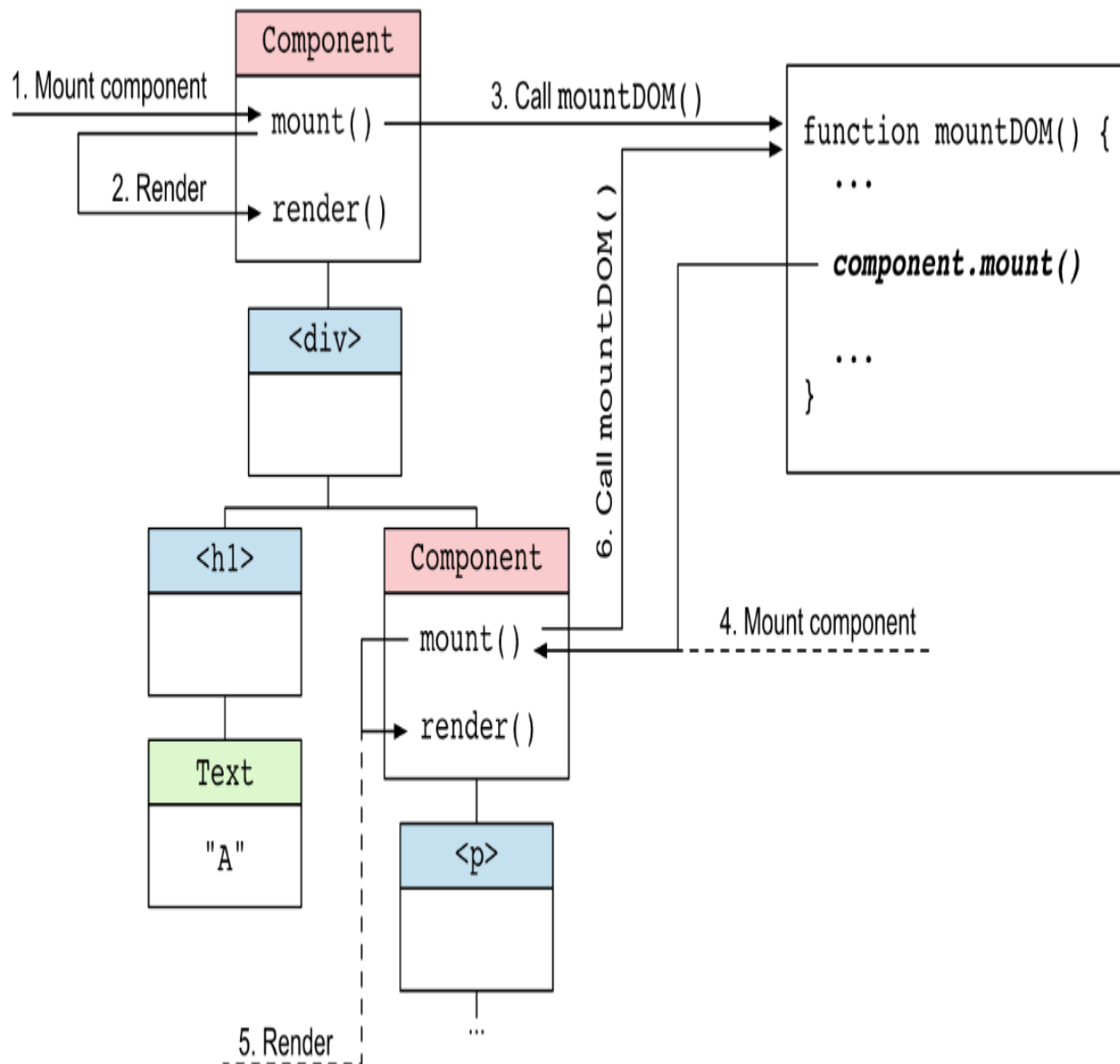
To instantiate this component including the `name` prop, you can do something like the following:

```
const vnode = {
  type: 'component',
  tag: Greetings,
  props: { name: 'Jenny' },
}

const Component = vnode.tag
const component = new Component(vnode.props)
```

Next, you want to mount the component. To mount the component, you need to call the component instance's `mount()` method. The component's `mount()` method renders its view via the `render()` method, and passes the resulting virtual DOM tree to the `mountDOM()` back to the function. Whenever another component is found in the virtual DOM tree, the process is repeated. This cycle is illustrated in figure [11.4](#).

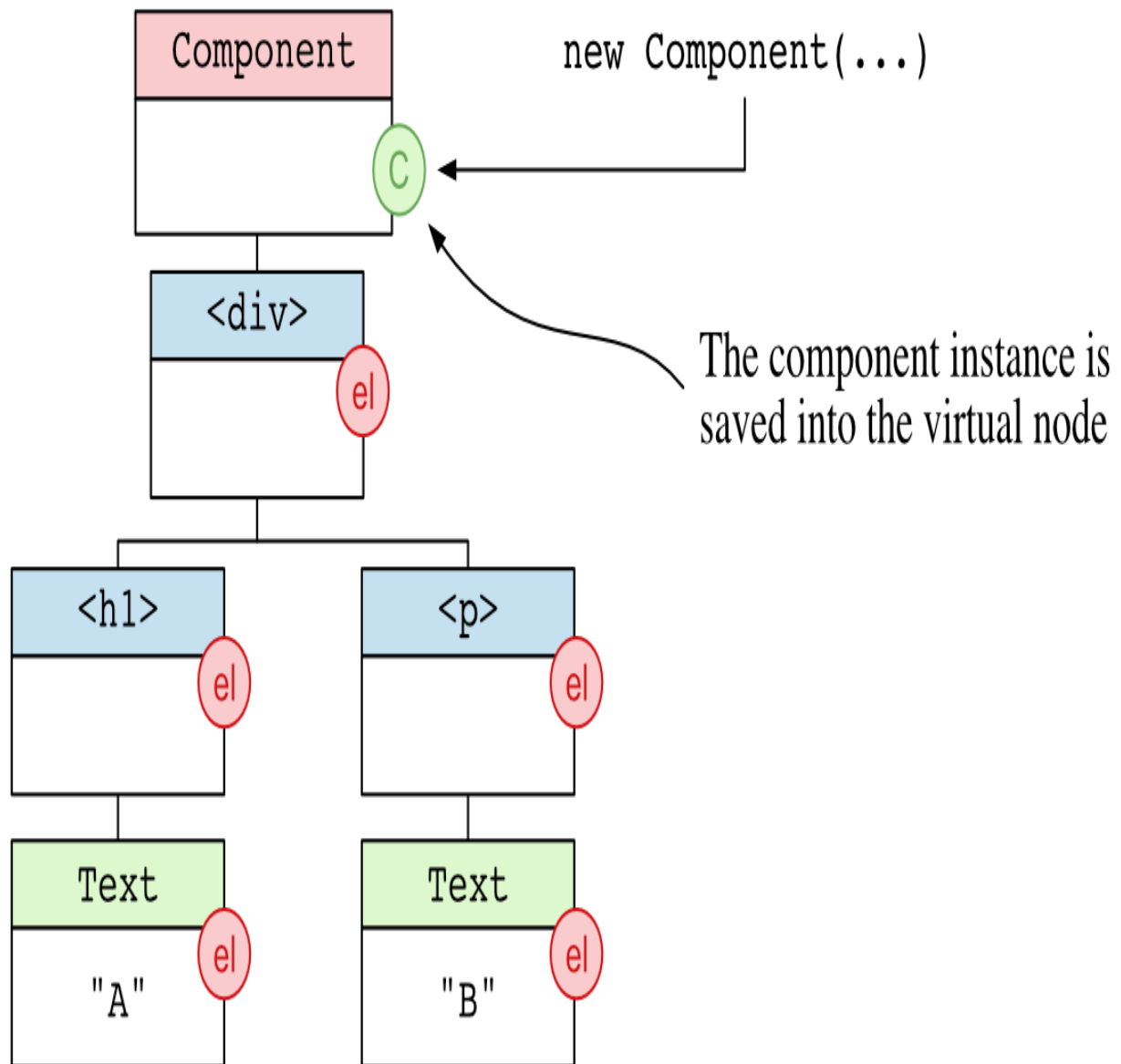
Figure 11.4 Mounting a component virtual node



There's one last—and very important—thing you have to keep in mind when mounting component nodes: you want to keep the instance of the component in the virtual node. Components have state, state that changes as the user interacts with its view. You want to keep this state "alive" throughout the application's lifecycle. If the component is instantiated every time the DOM is patched, the state is lost. This is the key difference between your new stateful components and the previous version, which were pure functions without internal state. By saving the component instance in the virtual node, you have access to it inside the `patchDOM()` function, and thus, don't need to instantiate

it again. Figure [11.5](#) illustrates how the component instance is saved in the virtual node.

Figure 11.5 A reference to the component instance is saved in the virtual node



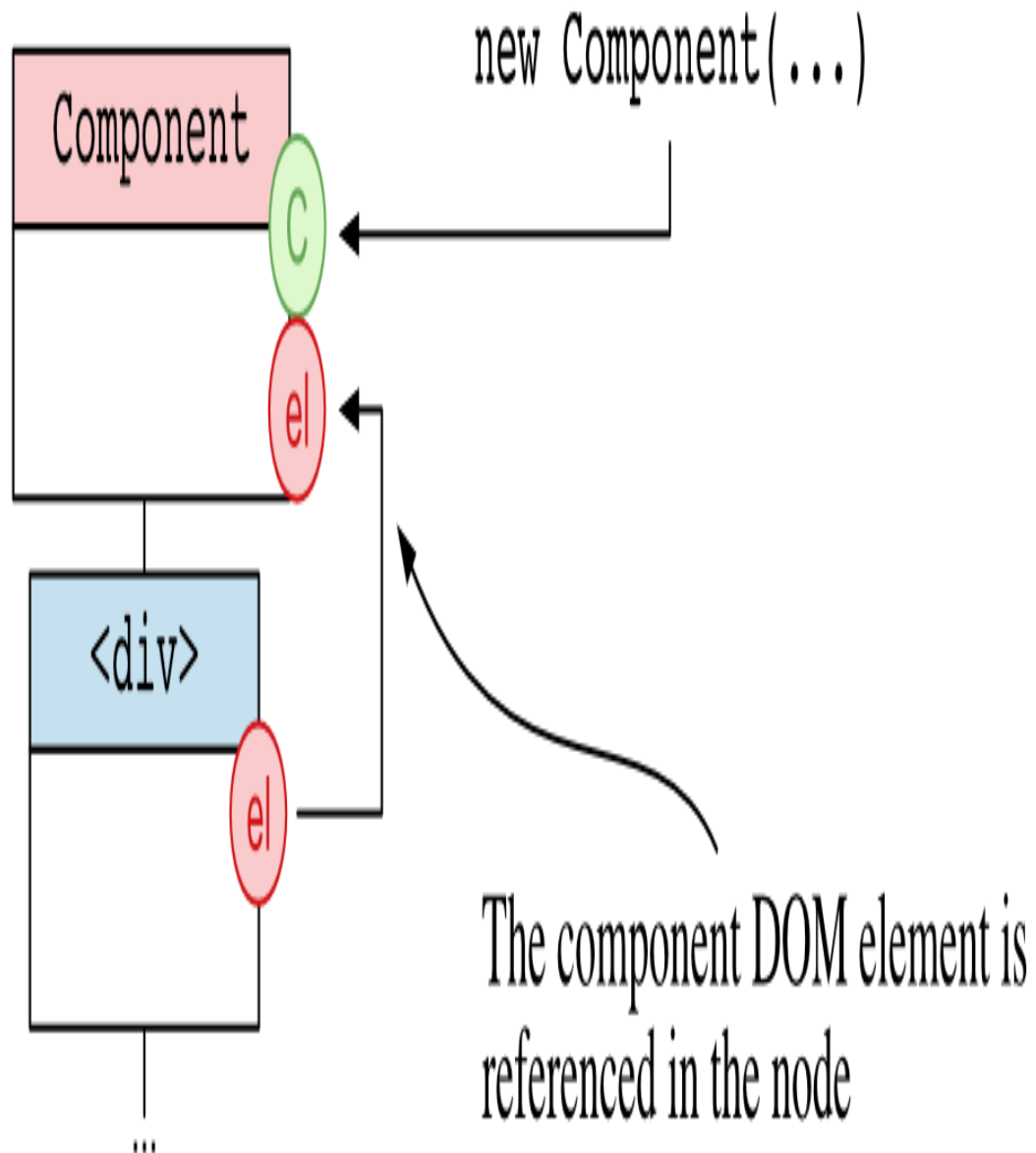
Following the preceding example, after you instantiate and mount the component, you save the instance in the virtual node, in a property called `component`, as follows:

```
const Component = vnode.tag
const component = new Component(vnode.props)
```

```
component.mount(parentEl)
vdom.component = component
```

And let's not forget to keep a reference to the component's DOM element in the virtual node's `el` property (figure [11.6](#)).

Figure 11.6 A reference to the component's DOM element is saved in the virtual node

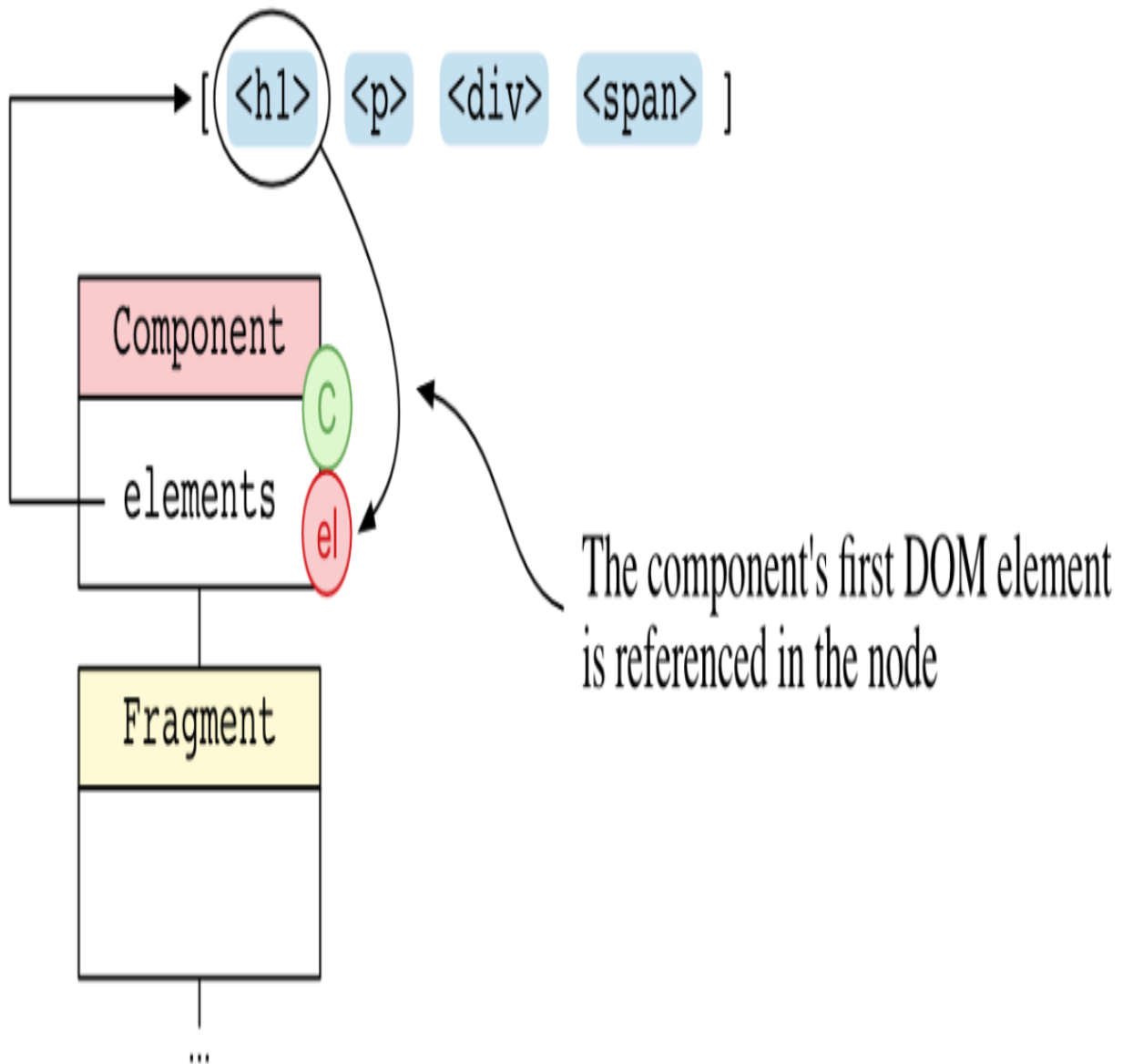


But wait! Again, you'll recall from chapter 9 that if a component's view is a fragment, it doesn't have one single HTML element as its root, but an array of

elements. What do we do in this case?

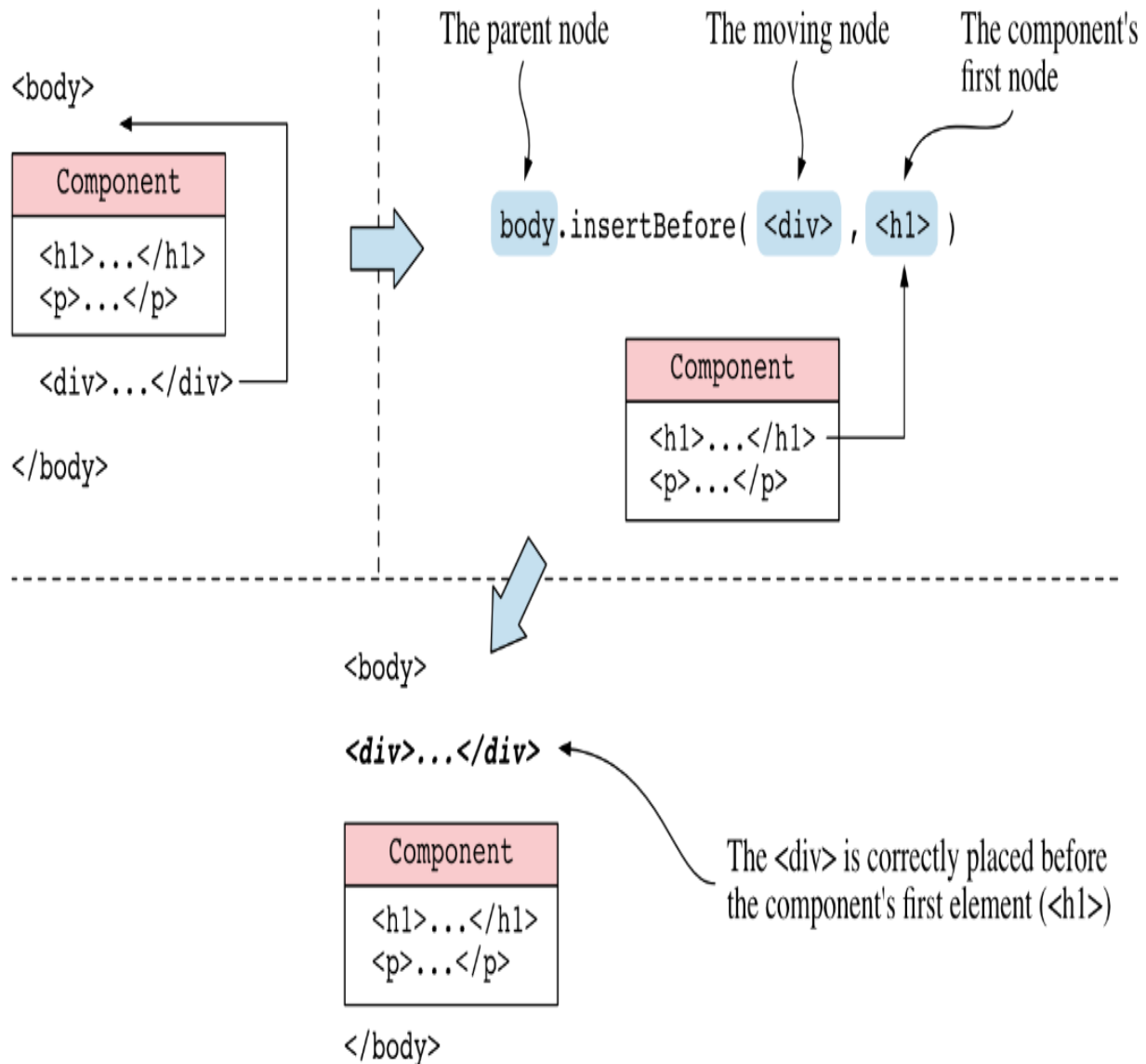
The use case for the `e1` reference is the reconciliation algorithm. In the case of a component, its sole use is to serve as a reference node to insert elements before the component. To move a node to a different position, the `insertBefore()` method is used, and it requires a reference node where the node will be inserted before. To insert an element before a component, the `e1` property should point at the component's first element. This is illustrated in figure [11.7](#).

Figure 11.7 The `e1` property points at the component's first element



By keeping the `el` reference of a component node pointing at the first element, you can move DOM nodes before the component. You can see an example of this in figure [11.8](#). In this case, the view of the component is made of a fragment containing an `<h1>` and a `<p>` element. A `<div>` outside the component is moved before the component, for which the component's first element, the `<h1>` element, is used as the reference node.

Figure 11.8 Moving a component to a different position in the DOM tree



You're now ready to implement the `createComponentNode()` function. Inside the `mount-dom.js` file, add the code in listing [11.4](#).

Listing 11.4 Mounting a component node (mount-dom.js)

```
function createComponentNode(vdom, parentEl, index,
hostComponent) {
  const Component = vdom.tag #1
  const props = vdom.props #2
  const component = new Component(props) #3

  component.mount(parentEl, index) #4
```

```
vdom.component = component #5
vdom.el = component.firstElement #6
}
```

Note

you passed the `hostComponent` as the fourth argument to the `mountDOM()` function, but you haven't used it yet. You'll use it in the next section, where you implement the event listeners. I thought it'd be convenient to have it in place already.

Hooraay! You can now mount component nodes, which allows you to have subcomponents inside your components. They can only be rendered once (when the component is mounted, because your patching algorithm doesn't handle components at the moment), but that's a good start.

Exercise 11.1

Write a component called `List`, that renders a list of items inside a `<ul>` element. These items are passed to the component as a prop called `items`, which is an array of strings. Each item should be rendered using another component called `ListItem`, which renders a `<li>` element with the item's text.

The result of mounting the component like so:

```
const items = ['foo', 'bar', 'baz']
const list = new List({ items })
list.mount(document.body)
```

Should be the following HTML:

```
<ul>
  <li>foo</li>
  <li>bar</li>
  <li>baz</li>
</ul>
```

Look at the answer at the end of the chapter if you get stuck. It's important to understand how to define subcomponents as you move forward with this chapter.

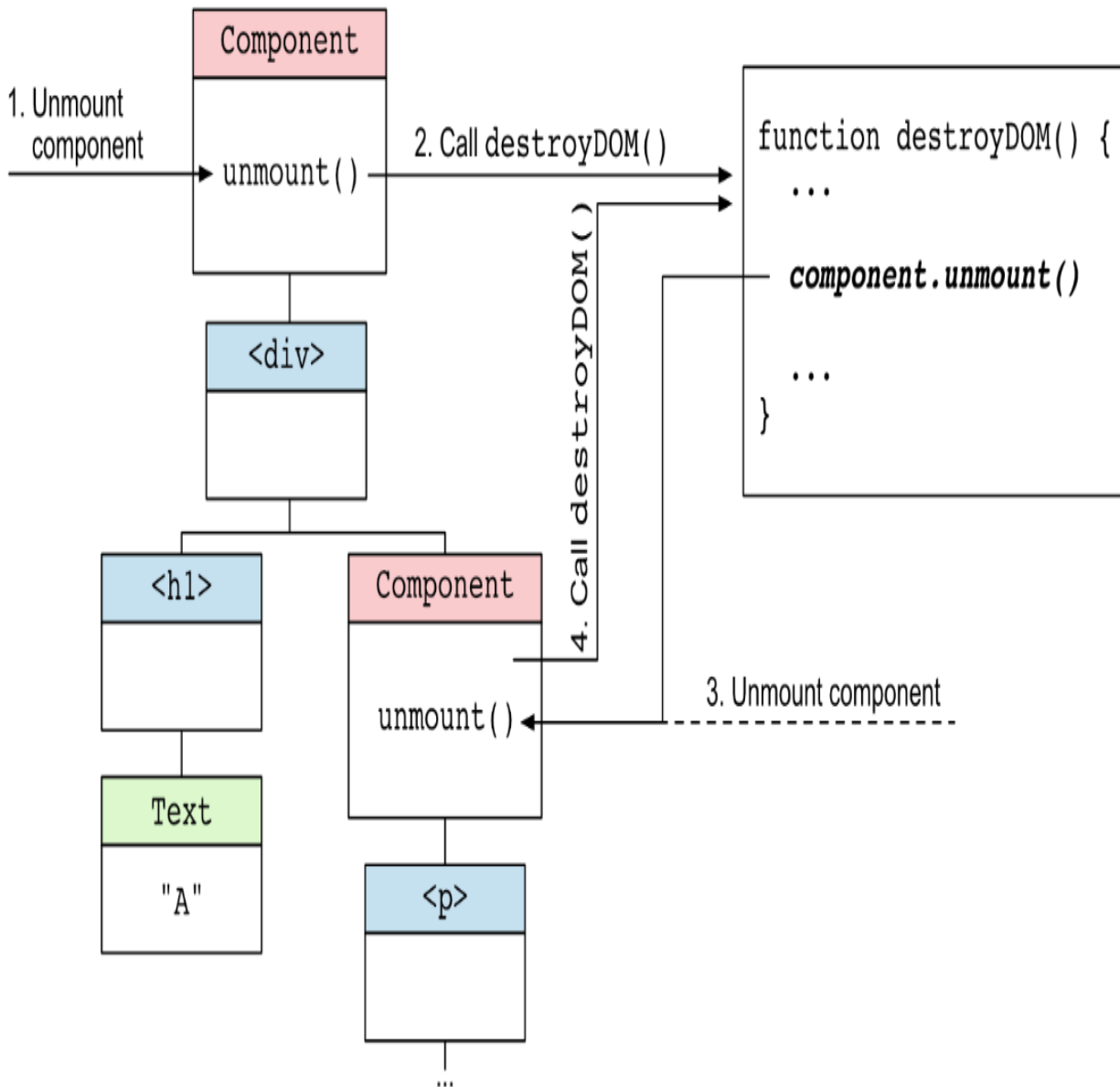
Find the solution at <http://mng.bz/W1rx>.

Let's now look at how you can destroy component nodes.

11.1.3 Destroying component virtual nodes

Once again, destroying is always easier than creating. In the case of a component, destroying it is as simple as calling its `unmount()` method—just that. You implemented the `unmount()` method in chapter 9, and if you recall, that method calls the `destroyDOM()` passing it the component's virtual DOM tree. If the component has subcomponents, the `destroyDOM()` method will be called recursively (figure [11.9](#)).

Figure 11.9 Destroying a component's DOM tree



Thanks to having saved the component instance in the virtual node, you can now access it inside the `destroyDOM()` function.

This is a straightforward addition to the `destroyDOM()` method. Open the *destroy-dom.js* file and add the code in bold font in listing [11.5](#).

Listing 11.5 Destroying the component (destroy-dom.js)

```
export function destroyDOM(vdom) {
  const { type } = vdom
```

```

switch (type) {
  case DOM_TYPES.TEXT: {
    removeTextNode(vdom)
    break
  }

  case DOM_TYPES.ELEMENT: {
    removeElementNode(vdom)
    break
  }

  case DOM_TYPES.FRAGMENT: {
    removeFragmentNodes(vdom)
    break
  }

  case DOM_TYPES.COMPONENT: {
    vdom.component.unmount() #1
    break
  }

  default: {
    throw new Error(`Can't destroy DOM of type: ${type}`)
  }
}

delete vdom.el
}

```

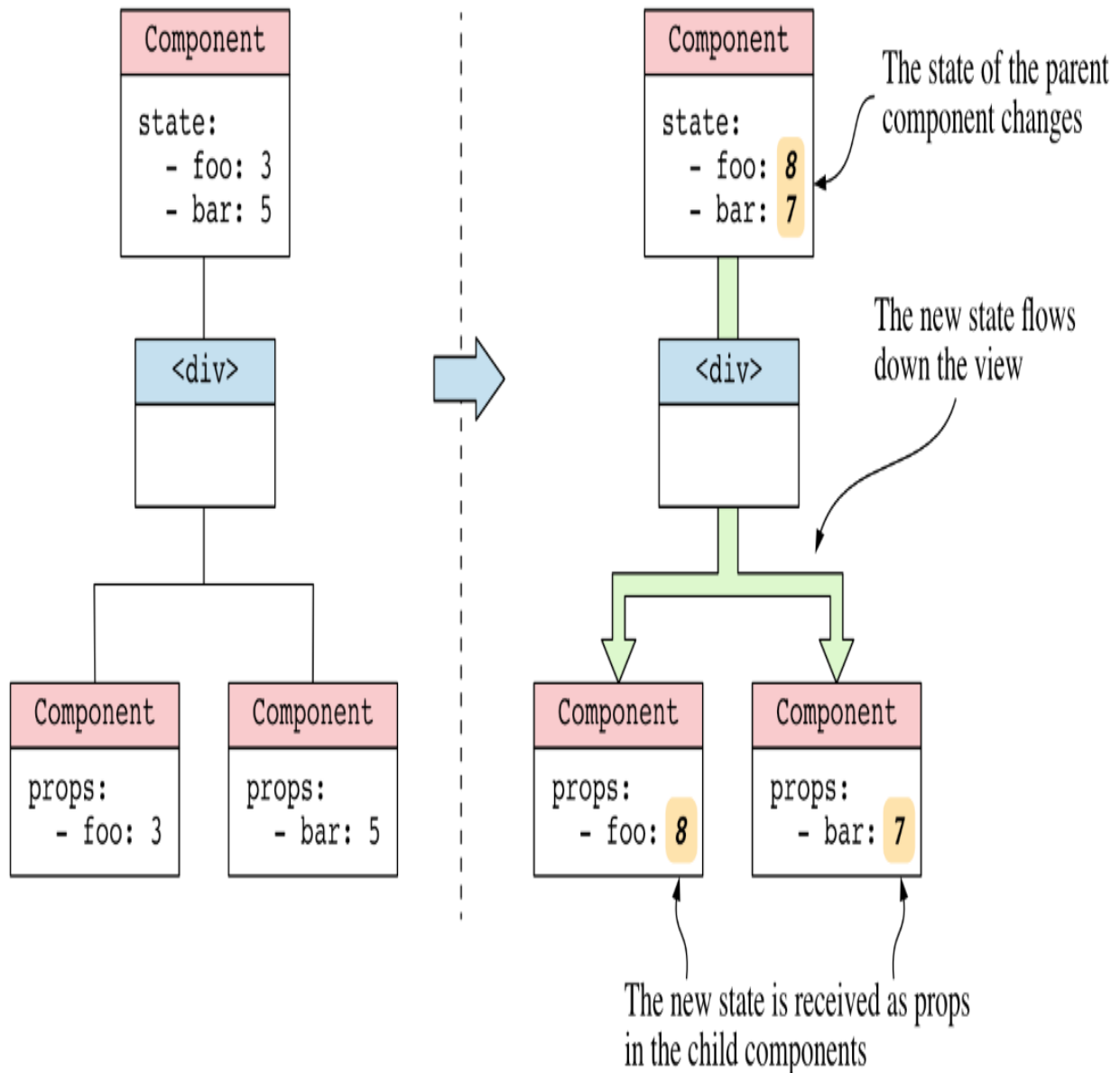
Let's now look at how you can patch component nodes.

11.1.4 Patching component virtual nodes

When the state of a component changes, or when it receives new props, this component patches its view, and instructs its subcomponents to do the same. The view of a component can only change if either its state or its props change. So, when the state or props of a component change, the component patches its view, and passes the new props to its subcomponents. The subcomponents state doesn't change, but their props might do.

Figure [11.10](#) illustrates how a component whose state changes notifies its subcomponents to patch their views.

Figure 11.10 The state of a component changes, and its subcomponents receive new props



Then, to patch the subcomponents inside the virtual DOM tree of a component, you need to extract the new props passed to them, and call their `updateProps()` method, which will internally call the `patch()` method of the component. Let's implement the `updateProps()` method in the `Component` class to update the props of the component and re-render it. Add the code in bold font in listing [11.6](#) inside the `component.js` file.

Listing 11.6 Updating the props of a component (`component.js`)

```

export function defineComponent({ render, state, ...methods
}) {
  class Component {
    // --snip-- //

    updateProps(props) {
      this.props = { ...this.props, ...props } #1
      this.#patch() #2
    }

    updateState(state) {
      this.state = { ...this.state, ...state }
      this.#patch()
    }

    // --snip-- //
  }

  // --snip-- //
}

```

Let's add the code to patch component nodes. Go ahead and add the code in bold font in listing [11.6](#), inside the *patch-dom.js* file. You add a `case` inside the `switch` statement that checks what type of node you're patching.

Listing 11.7 Adding the component case to the `patchDOM()` function (*patch-dom.js*)

```

export function patchDOM(
  oldVdom,
  newVdom,
  parentEl,
  hostComponent = null,
) {
  if (!areNodesEqual(oldVdom, newVdom)) {
    const index = findIndexInParent(parentEl, oldVdom.el)
    destroyDOM(oldVdom)
    mountDOM(newVdom, parentEl, index, hostComponent)

    return newVdom
  }

  newVdom.el = oldVdom.el

  switch (newVdom.type) {
    case DOM_TYPES.TEXT: {
      patchText(oldVdom, newVdom)
    }
  }
}

```



```

    return newVdom
  }

  case DOM_TYPES.ELEMENT: {
    patchElement(oldVdom, newVdom, hostComponent)
    break
  }

  case DOM_TYPES.COMPONENT: {
    patchComponent(oldVdom, newVdom) #1
    break
  }
}

patchChildren(oldVdom, newVdom, hostComponent)

return newVdom
}

// TODO: implement patchComponent()

```

Now, implement the `patchComponent()` function. This function is the one in charge of extracting the new props from the virtual node, and calling the component's `updateProps()` method. It should also save the component instance in the new virtual node, so that it can be accessed later on. And, as components can change their top-level element between renders, you want to save the component's first DOM element as in the node's `el` property. Write the code in listing [11.7](#) inside the *patch-dom.js* file.

Listing 11.8 Patching a component node (patch-dom.js)

```

function patchComponent(oldVdom, newVdom) {
  const { component } = oldVdom #1
  const { props } = newVdom #2

  component.updateProps(props) #3

  newVdom.component = component #4
  newVdom.el = component.firstElement #5
}

```

And just like that, you can now use child components inside your components, and they can be patched when the state of the parent component changes. If you're interested in optimizing the patching of components, avoiding

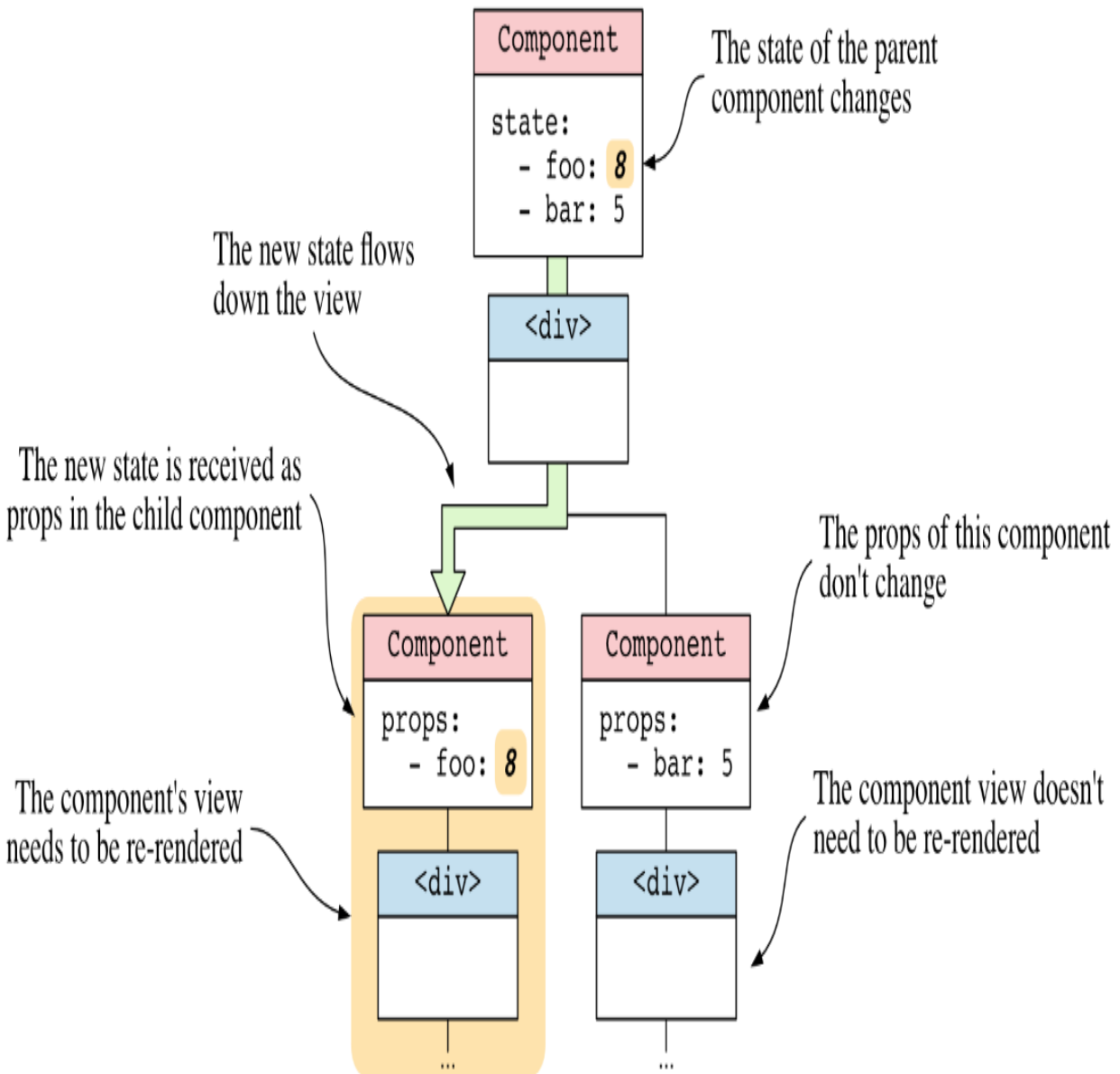
unnecessary re-renders, you can read the next optional section. Otherwise, you can jump to the next one, where we discuss how child components can communicate with their parent components via events.

11.1.5 (optional) A rendering optimization

You might have realized there's a good opportunity here to optimize the patching of components. Can you guess what it is? Exactly! You can check if the props of the component have changed, and only re-render it if they have. By doing a comparison between the old and new prop objects, you can avoid patching the component—and all of its subcomponents—if its props haven't changed, as illustrated in figure [11.11](#).

This can potentially avoid checking entire (and potentially long) branches of the virtual DOM tree. But it comes at a cost: you need to compare the old and new props objects, which can be expensive if the objects are large.

Figure 11.11 The state of a component changes, but not all the props of its child components change



Turns out comparing objects in JavaScript is not as straightforward as comparing primitive values. The `===` operator only returns `true` if both objects are the same object (reference equality). When the `render()` function of a component is called, it produces a fresh virtual DOM object, which means that the old and new props objects will never be the same object.

This isn't a big deal, because the community has satisfied this need by creating libraries that handle deep-object comparison for us. One such library is [fast-deep-equal](#), which is a very small library that does exactly what we need.

This is an optional optimization, so feel free to skip this section if you're not interested in it. To implement this optimization, you need to add the `fast-deep-equal` library to your *runtime* package (make sure you're in the *packages/runtime* folder):

```
npm install fast-deep-equal
```

Then, you need to import the library in the *component.js* file, and use it to compare the old and new props objects, inside the component's `updateProps()` method:

```
import equal from 'fast-deep-equal'

// --snip-- //

export function defineComponent({ render, state, ...methods
}) {
  class Component {
    // --snip-- //

    updateProps(props) {
      #this.props = { ...this.props, ...props }
      const newProps = { ...this.props, ...props } #1
      if (equal(this.props, newProps)) { #2
        return
      }

      this.props = newProps #3
      this.#patch()
    }

    // --snip-- //
  }
}
```

And with this simple change, your components will be patched only if their props have changed. But, to bundle the code in the `fast-deep-equal` library, you need to install two extra rollup plugins:

- [@rollup/plugin-commonjs](#): to transform the `fast-deep-equal` library code from CommonJS to ES syntax. This allows Rollup to bundle the library code.

- [@rollup/plugin-node-resolve](#): to resolve the `fast-deep-equal` library, which is a third-party module, and include it in the bundle. By default, Rollup resolves only modules that are part of your project. This plugin allows you to resolve third-party modules and include them in the bundle.

Install the libraries in the *packages/runtime* folder, as dev dependencies:

```
$ npm install --save-dev @rollup/plugin-commonjs
@rollup/plugin-node-resolve
```

Now, you need to edit rollup configuration file, *packages/runtime/rollup.config.js*, to add the plugins to the `plugins` array:

```
import commonjs from '@rollup/plugin-commonjs'
import { nodeResolve } from '@rollup/plugin-node-resolve'
import cleanup from 'rollup-plugin-cleanup'
import filesize from 'rollup-plugin-filesize'

export default {
  input: 'src/index.js',
  plugins: [commonjs(), nodeResolve(), cleanup()],
  output: [
    {
      file: 'dist/fe-fwk.js',
      format: 'esm',
      plugins: [filesize()],
    },
  ],
}
```

Run the `npm run build` script to make sure your configuration is correct and the output javascript file is generated correctly. If you open the bundled JavaScript file for your framework, under the *dist* folder, you should see the `fast-deep-equal` library code included in the bundle (look for a function called `fastDeepEqual()`):

```
var fastDeepEqual = function equal(a, b) {
  if (a === b) return true;
  if (a && b && typeof a == 'object' && typeof b == 'object')
  {
    if (a.constructor !== b.constructor) return false;
    // --snip-- //
  }
}
```

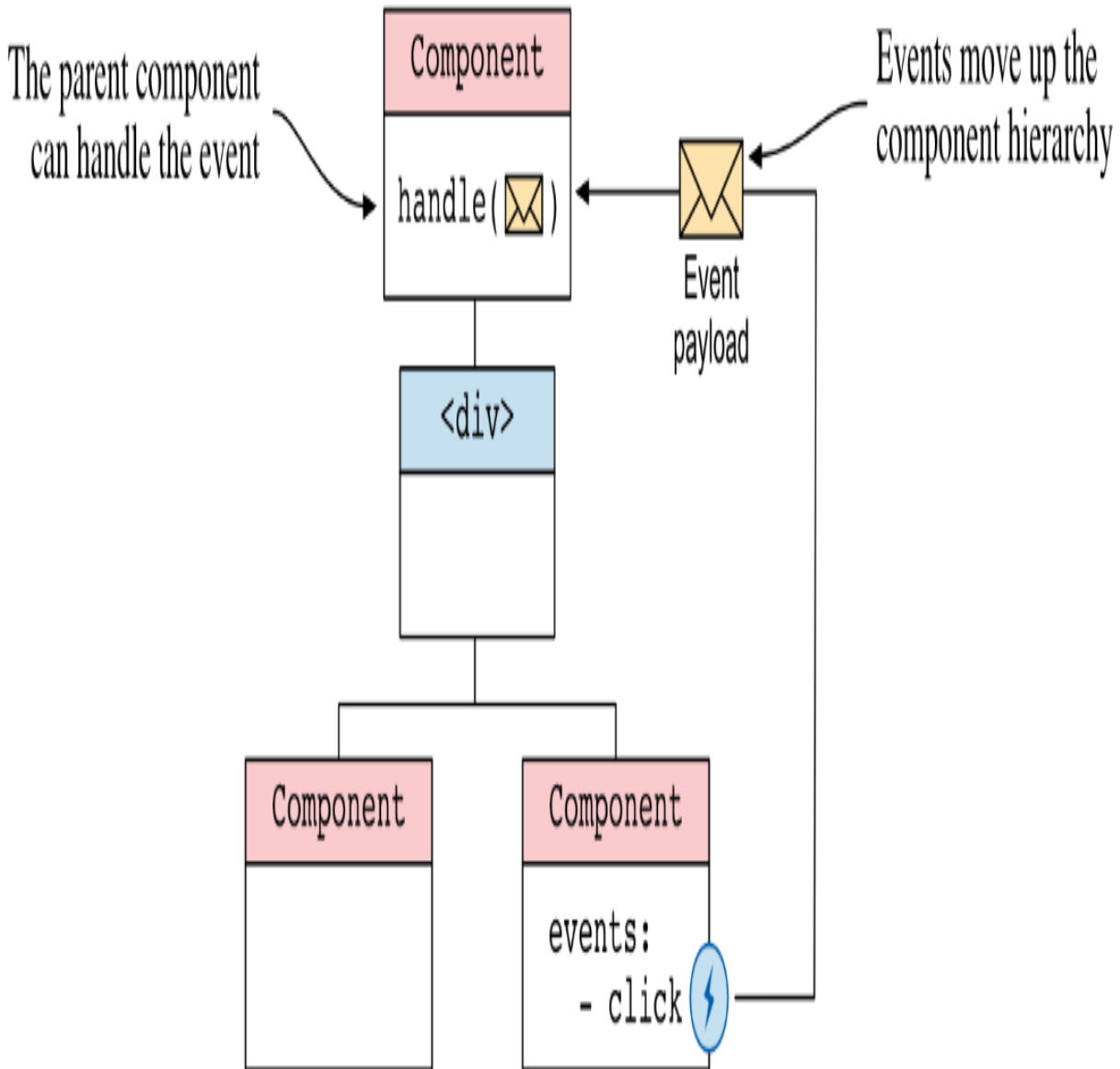
```
    // --snip--  
}
```

With this small change, you've added a very useful optimization to your framework.

11.2 Events

A component can pass information to its child components via props. This is how a parent component communicates with its children. But how can a child component communicate with its parent? Via events that the parent component can listen to, as shown in figure [11.12](#).

Figure 11.12 A child component communicates with its parent via events



The parent component defines the event handler function, and registers it to listen to events emitted by the child component. The events emitted by a child component aren't DOM events but custom events identified by a name and containing a payload. The child component receiving the event handler function from the parent component needs to save that function in a registry, where it can be called when the event is emitted. (If this reminds you a lot to your `Dispatcher` class, you're already in the right direction.)

Events vs. callbacks

If you've used React, you're probably used to passing callbacks from parent to child components:

```
function MyButton({ onClick }) {  
  return <button onClick={onClick}>Click me</button>  
}
```

The parent component then defines a function that will be called when the button is clicked, and passes it to the child component:

```
function App() {  
  function handleClick() {  
    console.log('Ooh la la!')  
  }  
  
  return <MyButton onClick={handleClick} />  
}
```

In other frameworks, such as Vue, you instead emit an event from the child component, but never receive the callback function from the parent component:

```
<template>  
  <button @click="$emit('click')">Click me</button>  
</template>
```

The `$emit()` method—very similar to what you'll implement in a moment—emits a custom (non-DOM) event. Then the parent component listens to that event:

```
<script setup>  
  import MyButton from './MyButton.vue'  
  
  function handleClick() {  
    console.log('Ooh la la!')  
  }  
</script>  
  
<template>  
  <MyButton @click="handleClick" />  
</template>
```

Although these seem like two different approaches, you'll see as you implement this feature yourself how in reality they're very similar. The

difference is mostly syntactical, in how a framework decides to expose the feature to the developer.

In the case of React, using JSX—which is a more convenient way of defining the virtual DOM tree for your view—you operate at the virtual DOM level, and thus require to explicitly pass the event handler function from the parent to the child component.

In the case of Vue, you work with a template. It feels natural to add event handlers inside the template of a component to handle the events that its child components emit. But under the hood, Vue compiles your template into a render function that operates at the virtual DOM level where the event handler is passed to the child component as a prop—the same as in React.

Your solution will look a lot like React in the beginning, as you still define the view of a component using a virtual DOM tree. But toward the end of the book, when you design a template system for your components, and a compiler that transforms the template into a render function, your solution will look more like Vue. This transition will illustrate how these two approaches are the same—or at least very similar—under the hood.

A parent component listening to an event from a child component fits very well with the way you’ve implemented the virtual DOM in your framework. If you recall, to add an event listener function to an element node, you include it inside the `on` object of the `props` parameter of the `h()` function:

```
h(  
  'button',  
  { on: { click: () => console.log('Ooh la la!') } },  
  ['Click me']  
)
```

You can make your components emit events. They won’t be instances of the [Event](#) interface, which represents events arising from the user interacting with the DOM. They’ll be simple JavaScript objects with the desired payload. Now, the key idea is that you can add event listeners to your components using exactly the same syntax as you use to add event listeners to DOM elements. For example:

```
h(
  MyFancyButton,
  { on: { btnClicked: () => console.log('Much fancy, too
cool, wow!') } },
  ['Click me']
)
```

Let's see how you can implement this feature in your `Component` prototype.

11.2.1 Saving the event handlers inside the component

You want to save those event listeners passed by the parent component inside the component that emits the event (the subcomponent or child component). If, for example, you have `MyFancyButton` component emitting a custom event called `btnClicked`, and its parent component renders it like so:

```
h(
  MyFancyButton,
  {
    on: {
      btnClicked: () => { ... }
    }
  },
  ['Click me']
)
```

You want to save the passed in `btnClicked` event handler inside the `MyFancyButton` component. The component can accept that event handlers object as a parameter in its constructor and save it in a private property called `#eventHandlers`:

```
class MyFancyButton {
  #eventHandlers = null

  constructor(eventHandlers = {}) {
    this.#eventHandlers = eventHandlers
  }
}
```

Then, when the child component (in this case `MyFancyButton`) emits an event, it can look up the event handler in the `#eventHandlers` object and call it.

Do you recall the `Dispatcher` you implemented early in the book? It's the perfect fit for this task — registering handlers for events and calling them when the event is emitted. You used it to dispatch and handle commands, but it can be used to dispatch and handle events as well. (Remember that the difference between commands and events is mostly in the nomenclature: events represent something that happened in the past, whereas commands represent something that should happen in the future.)

There is one important subtlety you need to take into account, though. When a parent component declares an event handler attached to a child component, it expects that the event handler is called with the parent component as the `this` value. A common use case is when the event handler modifies the state of the parent component:

```
const ParentComponent = defineComponent({
  state() {
    return { counter: 0 }
  },

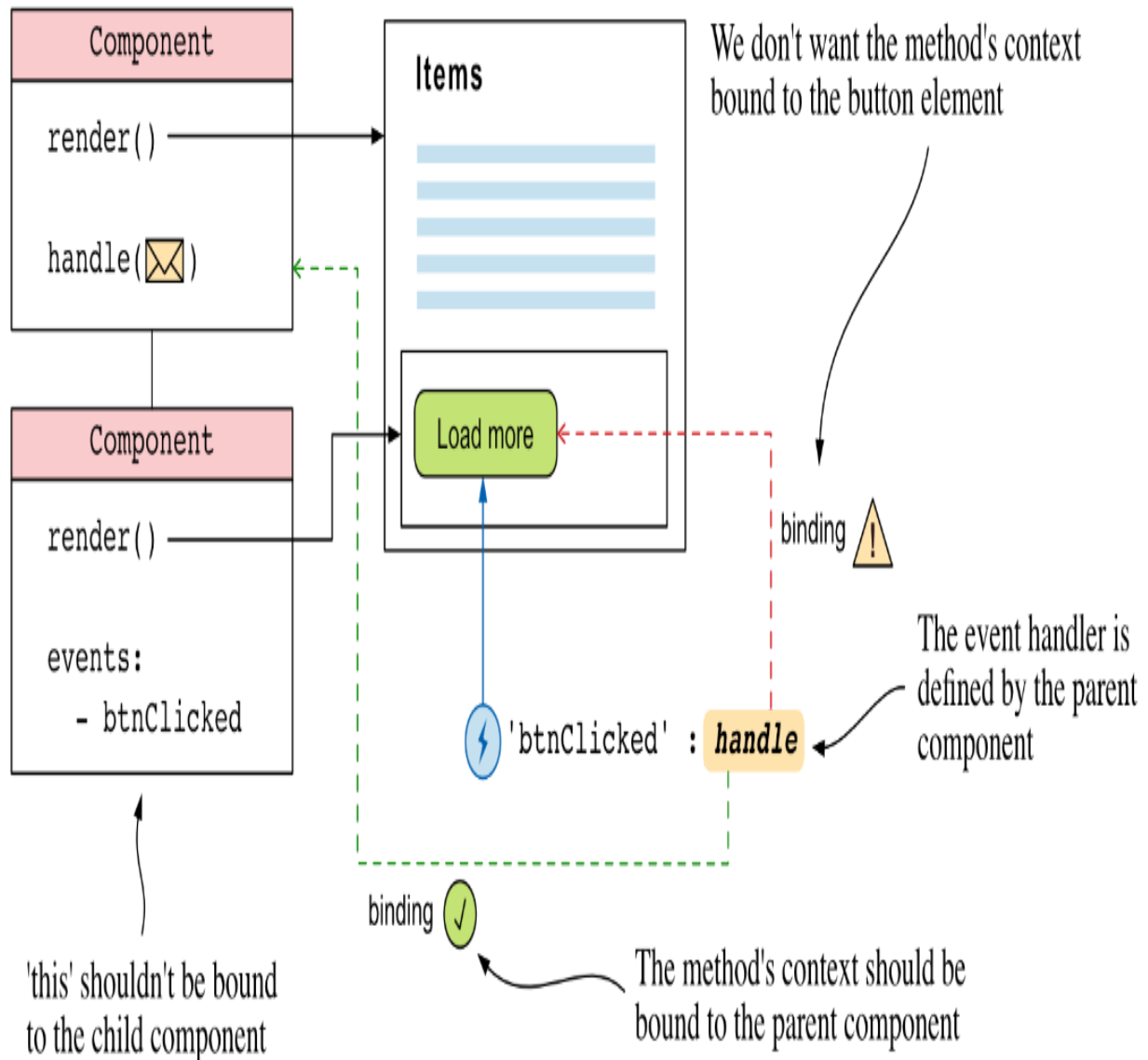
  render() {
    const { counter } = this.state

    return hFragment([
      h('p', {}, [The count is: ${counter}]),
      h(
        MyFancyButton,
        {
          on: {
            btnClicked: () => {
              this.updateState({ counter: counter + 1 }) #1
            },
          },
        },
        ['Increment']
      )
    ])
  }
})
```

You want to explicitly bind the `this` value of the event handler to the parent component, not the child component executing the event handler (figure [11.13](#)). This is very important, so keep it in mind: the `this` value of the event

handler should be the parent component, not the child component calling the event handler.

Figure 11.13 The `this` value of the event handler should be the parent component



Let's go ahead and add the `#eventHandlers` property to the `Component` prototype, as well as pass the reference to the parent component to the constructor, so it can be used to explicitly bind the event handler's context. In the `component.js` file, add the code in bold font in listing [11.9](#).

Listing 11.9 Passing event handlers to a component (component.js)

```
export function defineComponent({ render, state, ...methods
}) {
  class Component {
    #isMounted = false
    #vdom = null
    #hostEl = null
    #eventHandlers = null #1
    #parentComponent = null #2

    constructor(
      props = {},
      eventHandlers = {},
      parentComponent = null,
    ) {
      this.props = props
      this.state = state ? state(props) : {}
      this.#eventHandlers = eventHandlers #3
      this.#parentComponent = parentComponent #4
    }

    // --snip-- //
  }

  // --snip-- //

  return Component
}
```

Now, before you move forward implementing the wiring of the event handlers, you want to go back to the *mount-dom.js* and *patch-dom.js* files and pass the event handlers and the parent component reference to the component constructor.

11.2.2 Extracting the props and events for a component

As you probably remember from when you destructured the props and events for an element node (refer to the *mount-dom.js* file, inside the `addProps()` function), separating the props from the events is as simple as:

```
const { on: events, ...props } = vdom.props
```

You could simply add that same line both to the `createComponentNode()` (in the *mount-dom.js* file) and `patchComponent()` (in the *patch-dom.js* file) functions. However, you'll add some extra logic to the props and events extraction in the next chapter when you learn about keyed lists (namely, you want to remove the `key` prop, but let's not get ahead of ourselves). To avoid duplicating the same logic in two places, I prefer you to create a separate file inside the *utils/* directory, call it *props.js*, where you write a function that extracts the props and events for a component.

In that *utils/props.js* file, add the `extractPropsAndEvents()` function in listing [11.10](#).

Listing 11.10 Extracting the props and events for a component (utils/props.js)

```
export function extractPropsAndEvents(vdom) {
  const { on: events = {}, ...props } = vdom.props

  return { props, events }
}
```

Now, open the *mount-dom.js* file, import the `extractPropsAndEvents()` function, and use it inside the `createComponentNode()` function to extract the props and events. You then want to pass the props, event handlers, and parent component to the component constructor. The parent component is the fourth `hostComponent` parameter passed to the `createComponentNode()` function.

You added this parameter in listing [11.4](#), but you didn't use it then.

Implement these changes as shown in bold font in listing [11.11](#).

Listing 11.11 Extracting the props and events when creating a component (mount-dom.js)

```
import { extractPropsAndEvents } from './utils/props'

// --snip-- //

function createComponentNode(vdom, parentEl, index,
  hostComponent) {
  const Component = vdom.tag
  const props = vdom.props
```

```

    const component = new Component(props)
    const { props, events } = extractPropsAndEvents(vdom) #1
    const component = new Component(props, events,
hostComponent) #2

    component.mount(parentEl, index)
    vdom.component = component
    vdom.el = component.firstElement
}

```

Similarly, open the *patch-dom.js* file, import the `extractPropsAndEvents()` function, and use it inside the `patchComponent()` function, as shown in listing [11.12](#). In this case though, you pass only the props; we're expecting the event handlers to not change between patches.

Listing 11.12 Extracting the props and events when patching a component (*patch-dom.js*)

```

import { extractPropsAndEvents } from './utils/props'

// --snip--

function patchComponent(oldVdom, newVdom) {
  const { component } = oldVdom
  const { props } = newVdom
  const { props } = extractPropsAndEvents(newVdom)

  component.updateProps(props)

  newVdom.component = component
  newVdom.el = component.firstElement
}

```

With this little detour out of the way, you can now go back to the *component.js* file and implement the logic to wire the event handlers.

Note

After the child component is mounted, we don't allow the parent component to change the event handlers or register new ones. This is a deliberate design decision that simplifies a bit the implementation of the component patching logic. There might be a few good use cases for event handlers to change between renders, but they are not so common.

You can try to implement this logic as an exercise.

11.2.3 Wiring the event handlers

When the component is mounted, you want to iterate over the `#eventHandlers` object and register the event handlers in the component. How can you register event handlers in a component? Using the `Dispatcher` class you implemented earlier in the book. Each component instance has its own dispatcher where it can register event handlers and emit the events.

You want to save the functions that the `subscribe()` method returns, so you can call them later to unsubscribe the event handlers when the component is unmounted. (If you recall, the `subscribe()` method returns a function that, when called, unsubscribes the event handler.)

Inside the `Component` class, write a private method called `#wireEventHandler()` that takes the event name and the event handler as parameters, and subscribes the event handler to the component's dispatcher. Don't forget to bind the event handler's context to the parent component instance—the component that's defining the event handler. Then, for convenience, a second method called `#wireEventHandlers()` (in plural) can do the work of iterating over the `#eventHandlers` object and call the `#wireEventHandler()` method for each event handler, saving the unsubscribe functions in a `#subscriptions` private property.

In your *component.js* file, add the code in bold font in listing [11.13](#). Don't forget to import the `Dispatcher` class from the *dispatcher.js* file.

Listing 11.13 Wiring the event handlers from the parent component (component.js)

```
import equal from 'fast-deep-equal'
import { destroyDOM } from './destroy-dom'
import { Dispatcher } from './dispatcher'
import { DOM_TYPES, extractChildren } from './h'
import { mountDOM } from './mount-dom'
import { patchDOM } from './patch-dom'
import { hasOwnProperty } from './utils/objects'

export function defineComponent({ render, state, ...methods
}) {
```



```

class Component {
  #isMounted = false
  #vdom = null
  #hostEl = null
  #eventHandlers = null
  #parentComponent = null
  #dispatcher = new Dispatcher() #1
  #subscriptions = [] #2

  // --snip-- //

  #wireEventHandlers() {
    this.#subscriptions =
Object.entries(this.#eventHandlers).map( #3
    ([eventName, handler]) =>
      this.#wireEventHandler(eventName, handler)
    )
  }

  #wireEventHandler(eventName, handler) {
    return this.#dispatcher.subscribe(eventName, (payload)
=> {
      if (this.#parentComponent) {
        handler.call(this.#parentComponent, payload) #4
      } else {
        handler(payload) #5
      }
    })
  }

  // --snip-- //
}

// --snip-- //

return Component
}

```

You can now call the `#wireEventHandlers()` method from the `mount()` method. You also want to unsubscribe the event handlers when the component is unmounted. Add these changes as shown in bold font in listing [11.15](#).

Listing 11.14 Wiring the event handlers when the component is mounted, unsubscribing when unmounted (component.js)

```

export function defineComponent({ render, state, ...methods
}) {
  class Component {
    // --snip-- //

    mount(hostEl, index = null) {
      if (this.#isMounted) {
        throw new Error('Component is already mounted')
      }

      this.#vdom = this.render()
      mountDOM(this.#vdom, hostEl, index, this)
      this.#wireEventHandlers() #1

      this.#isMounted = true
      this.#hostEl = hostEl
    }

    unmount() {
      if (!this.#isMounted) {
        throw new Error('Component is not mounted')
      }

      destroyDOM(this.#vdom)
      this.#subscriptions.forEach((unsubscribe) =>
unsubscribe()) #2

      this.#vdom = null
      this.#isMounted = false
      this.#hostEl = null
      this.#subscriptions = [] #3
    }

    // --snip-- //
  }

  // --snip-- //

  return Component
}

```

You can now subscribe event handlers passed from the parent component in a child component, but you want a way of emitting these events from your component. As you can guess, that's as easy as calling the dispatcher's `emit()` method. Let's look at how you can do that.

Exercise 11.2

Can you explain why you need to bind the event handler's context to the parent component instance when you subscribe the event handler?

Find the solution at <http://mng.bz/84W2>.

11.2.4 Emitting events

You can emit events from the component by calling the `emit()` method on the component's dispatcher:

```
const MyFancyButton = defineComponent({
  render() {
    return h(
      'button',
      {
        on: { click: () => this.#dispatcher.dispatch('click')
      }
    ),
    ['Click me!']
  }
})
```

But that's a bit verbose. You want the users of your framework to use an API that's as simple as possible. Something like the following would be ideal:

```
const MyFancyButton = defineComponent({
  render() {
    return h(
      'button',
      {
        on: { click: () => this.emit('click') }
      },
      ['Click me!']
    )
  }
})
```

You can achieve that very easily, by adding an `emit()` method to the `Component` class. Let's do exactly that. Add the code in bold font in listing

[11.15](#) to your *component.js* file.

Listing 11.15 Emitting events from the component (component.js)

```
export function defineComponent({ render, state, ...methods
}) {
  class Component {
    // --snip-- //

    emit(eventName, payload) {
      this.#dispatcher.dispatch(eventName, payload)
    }
  }

  // --snip-- //

  return Component
}
```

And with this last change, your `Component` class is complete. In the next chapter you'll refactor the `createApp()` function to return an application instance made of stateful components. Then you'll look into a very interesting problem: what happens when stateful components move around the DOM? Does the reconciliation algorithm know how to handle that? After solving that problem, you'll publish the new version of your framework.

Exercise 11.3

Write the code for a component called `SearchField` which renders an input field and emits a `search` event when the user types in the field. You want to debounce the event by 500 milliseconds. (Assume you have a `debounce()` function that takes a function and a delay as parameters, and returns a debounced version of the function.)

Find the solution at <http://mng.bz/E9Vl>.

11.3 Summary

- A parent component can pass data to a child component by passing it as props.
- A child component can pass data to a parent component by emitting events that the parent component can subscribe to.
- The child component saves the registered event handlers in a `#eventHandlers` object, and subscribes them to its dispatcher when the component is mounted.
- When the state or props of a component change, the component re-renders its view. In this process, when a subcomponent appears in the view tree, the parent component passes the subcomponent's props to it, and tells it to re-render its view.

12 Keyed lists

This chapter covers

- Keyed lists of components
- Keyed lists of elements
- Updating the application instance
- Publishing the new version of the framework

Components have internal state. This state is hidden from the outside world—it isn't reflected in the virtual DOM tree that represents the view—and thus can't be used to compare two component virtual DOM nodes. But to correctly patch the view, it's paramount that, when there's a list of components, the reconciliation algorithm can distinguish between them. Let's look at an example that clarifies this.

Imagine you have a `Counter` component, with a `count` state property, which can be initialized with a different value for each component instance:

```
const Counter = defineComponent({
  state(props) {
    return { count: props.initialCount }
  },

  render() {
    return h('p', {}, [`Count: ${this.state.count}`])
  },
})
```

Then you render a list of three `Counter` components, each having a different value for the `count` state:

```
h('div', {}, [
  h(Counter, { initialCount: 0 }),
  h(Counter, { initialCount: 1 }),
  h(Counter, { initialCount: 2 }),
])
```

Note that, the value of each `count` state property isn't reflected in the virtual DOM tree. That information is part of the component's state, stored inside the `Counter` component instances. The rendered HTML would look like the following:

```
<div>
  <p>Count: 0</p>
  <p>Count: 1</p>
  <p>Count: 2</p>
</div>
```

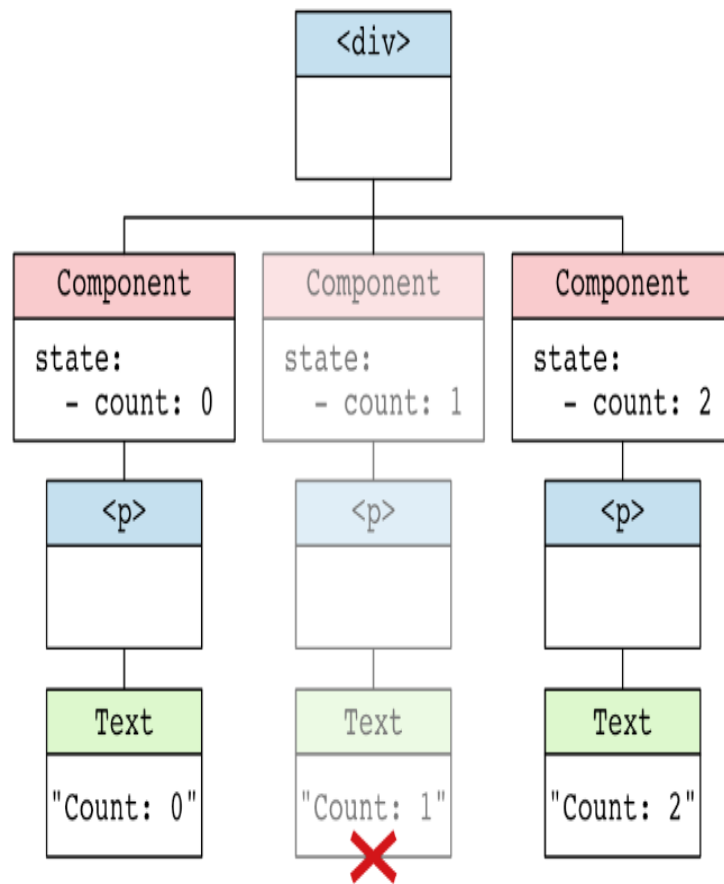
What happens now if the `Counter` component at the middle of the list is removed? The new virtual DOM tree would look like this:

```
h('div', {}, [
  h(Counter, { initialCount: 0 }),
  h(Counter, { initialCount: 2 }),
])
```

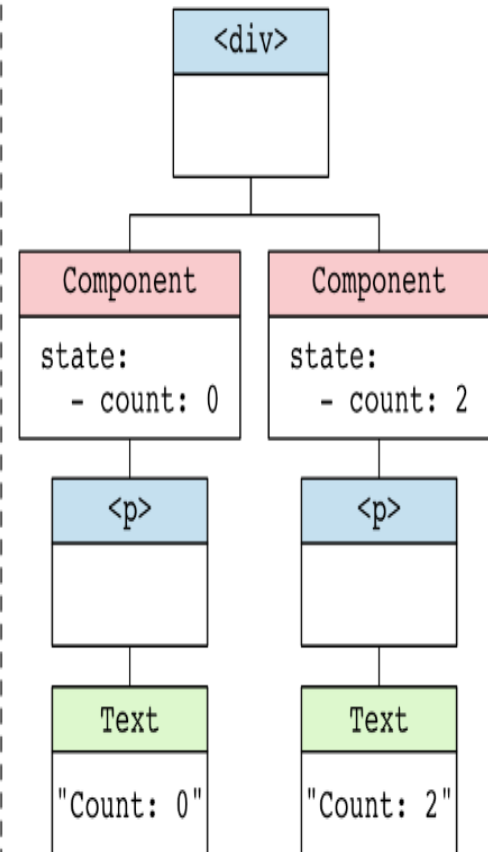
But the reconciliation algorithm you wrote just can't know which `Counter` component was removed, as illustrated in figure [12.1](#). In fact, the reconciliation algorithm doesn't know how to distinguish between the components at all; they all look the same to it.

Figure 12.1 All components look the same to the reconciliation algorithm

Old vdom



New vdom



The reconciliation algorithm can't access the components state, and thus can't know what component was removed. All components look equal to the reconciliation algorithm.

By comparing the two virtual DOM trees, the `arraysDiffSequence()` function would wrongly conclude that the last `Counter` component was removed. The resulting HTML would—incorrectly—be:

```
<div>
  <p>Count: 0</p>
  <p>Count: 1</p>
</div>
```

When in reality it should have been:

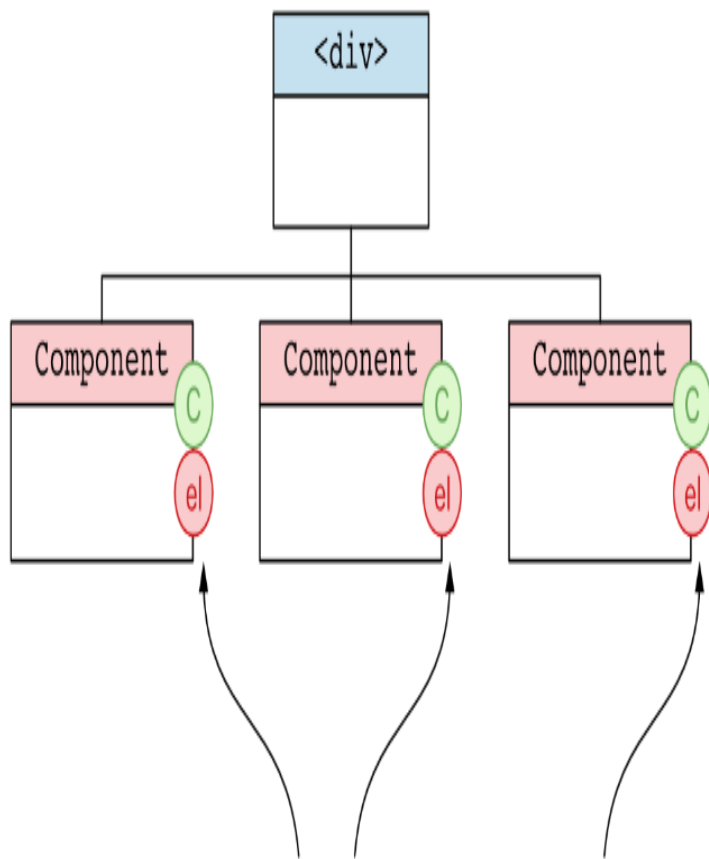

```
<div>
  <p>Count: 0</p>
  <p>Count: 2</p>
</div>
```

You might think the solution is as straightforward as making the `areNodesEqual()` compare the internal state and props of the components to determine if they are equal—a component is fully characterized by its internal data. Unfortunately, this is not possible. Before reading the next paragraph with the explanation, try to think why this is not possible yourself.

The reason this isn't possible is because the `areNodesEqual()` function receives one node from the old virtual DOM tree, which contains a reference to the component instance, and another node from the new virtual DOM tree, which doesn't. Without access to the instantiated component in the new vdom, you just can't access its internal state and props, as illustrated in figure [12.2](#).

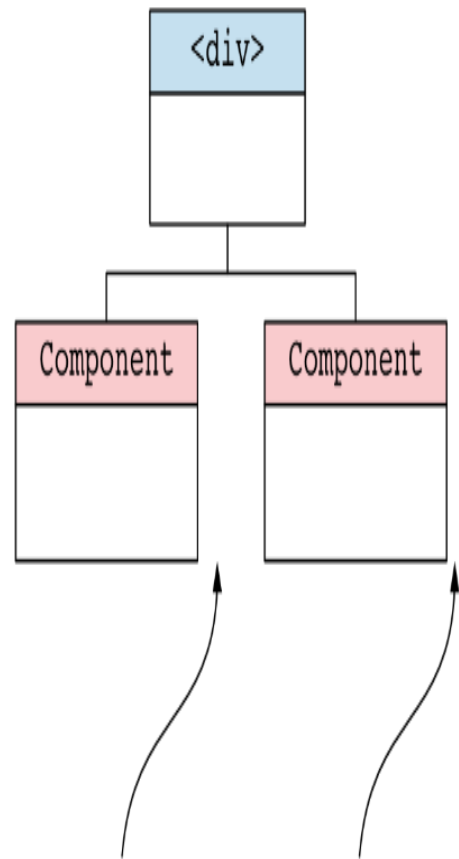
Figure 12.2 The new virtual DOM tree doesn't contain a reference to the component instance

Old vdom



Mounted components have a reference to the component instance and the mounted DOM element.

New vdom



Components that haven't been mounted or patched don't have a reference to an existing instance.

Your `areNodesEqual()` function would receive the following old and new nodes:

```
const oldNode = {  
  type: 'component',  
  tag: Counter,  
  props: {},  
  component: <instance of Counter>,  
}
```

```
const newNode = {  
  type: 'component',
```

```
    tag: Counter,  
    props: {},  
  }
```

It's clear that you can't access the `count` property of the component instance from the `newNode` object, and thus you can't use it to compare whether the two nodes are equal. You clearly need another solution.

The goal of this chapter is to explore the solution to this problem. As you might have guessed by the title of the chapter, that's the `key` attribute that the developer can provide to each component. We'll also explore some poor practices that you should avoid when using the `key` attribute.

Finally, you'll publish a new version of the framework, version 3.0, that supports stateful components. In the next chapter, you'll use this new version of the framework to refactor the `TODOs` application.

Important

You can find all the listings in this chapter in the [listings/ch12 directory](#) of the book's repository. The code in this chapter can be checked out from [the ch12 label](#):

```
$ git switch --detach ch12
```

Code catch-up

In the last chapter, you added the component as a new type of virtual DOM node, and extended the `h()` function to support it. You modified the `mountDOM()` function to include a case for the component type—handled by the `createComponentNode()` function. You also modified the `destroyDOM()` function to include a case for the component, in this case simply calling the component's `unmount()` method.

Then, you worked on adapting the `patchDOM()` function to support the component type. For it, you had to include a new method in the `Component` class: `updateProps()`. This method is called inside the `patchComponent()` function, a function you wrote to handle the component case in the `patchDOM()` function.

You implemented a mechanism for components to handle events emitted by their child components. This involved adding two new private properties to the `Component` class: `#eventHandlers` and `#parentComponent`. You then wrote a function called `extractPropsAndEvents()` to extract the props and events from the `props` object of a virtual DOM node, and used it inside the `createComponentNode()` and `patchComponent()` functions. A component subscribes to the events using the `#wireEventHandlers()` private method, which you call inside `mount()`. You managed these subscriptions using an instance of the `Dispatcher` class, saved inside the component as the `#dispatcher` private property.

Last, you wrote a new method, `emit()`, used to emit events from a component to its parent component.

12.1 The key attribute

If your framework has no way of comparing two component nodes in virtual DOM trees to determine if they are the same, how can you solve the reconciliation problem? In this case, you let the developer provide a unique identifier for each component: a `key` attribute.

Following the `Counter` example from the introduction, the developer would provide a `key` attribute to each component as follows:

```
h('div', {}, [
  h(Counter, { key: 'counter-0', initialCount: 0 }),
  h(Counter, { key: 'counter-1', initialCount: 1 }),
  h(Counter, { key: 'counter-2', initialCount: 2 }),
])
```

So, when the `Counter` component at the middle of the list is removed, the new virtual DOM tree would look like this:

```
h('div', {}, [
  h(Counter, { key: 'counter-0', initialCount: 0 }),
  h(Counter, { key: 'counter-2', initialCount: 2 }),
])
```

The reconciliation algorithm would now be able to compare the `key` attribute of the `Counter` components to determine which one was removed. The only drawback is that you rely on the developer to provide a unique `key` attribute for each component. When each item in the list that produces the components doesn't have a unique identifier, developers (I've done this myself) resort to using the index of the item in the list as the `key` attribute, but this is a bad idea, as we'll see later in this chapter. Your framework won't be resilient to these cases—all the responsibility is on the developer.

Let's start by including the component case in the `areNodesEqual()` function. Your framework doesn't currently know how to compare component nodes (as of now, it deems all component pairs different), so this is a good place to start. You want this function to, at least, know when two components are instances of the same component prototype. When the function is passed two component nodes that are instances of a different prototype, it should return `false`. If they are instances of the same prototype, it should return `true`, for now.

12.1.1 Component nodes equality

How can you compare two component virtual DOM nodes to determine if they are equal? First, you need to check if their `type` properties are equal to `DOM_TYPES.COMPONENT`. Then, you want to know if they are the same component prototype. In JavaScript, a prototype (or class definition) is, unsurprisingly, always equal to itself, so you can compare the `tag` properties of the two nodes. You can test this yourself in the browser console:

```
class Component {}  
  
Component === Component // true
```

Open the *nodes-equal.js* file and add the code in bold font in listing [12.1](#) to include the component case in the `areNodesEqual()` function.

Listing 12.1 Comparing two component nodes for equality (nodes-equal.js)

```
export function areNodesEqual(nodeOne, nodeTwo) {  
  if (nodeOne.type !== nodeTwo.type) {  
    return false
```

```

    }

    if (nodeOne.type === DOM_TYPES.ELEMENT) {
      const { tag: tagOne } = nodeOne
      const { tag: tagTwo } = nodeTwo

      return tagOne === tagTwo
    }

    if (nodeOne.type === DOM_TYPES.COMPONENT) { #1
      const { tag: componentOne } = nodeOne #2
      const { tag: componentTwo } = nodeTwo #3

      return componentOne === componentTwo #4
    }

    return true
  }
}

```

This is a great start. Let's now support the `key` attribute in the `areNodesEqual()` function.

12.1.2 Using the `key` attribute

You want developers to include a `key` attribute in a component node when they are part of a dynamic list of components, where the nodes might be reordered, added or removed. As we saw in the initial example, the `key` attribute is passed as a prop to the component, so you need to extract it from the `props` object.

You want to extract the `key` attribute from both nodes' `props` objects and compare them. If they aren't provided, they'll be `undefined`, so you can still use the `===` operator to compare them. As you know, despite all JavaScript equality wilderness, `undefined` is equal to itself (yay!).

Add the code in bold font in listing [12.2](#).

Listing 12.2 Using the `key` attribute to compare component nodes (nodes-equal.js)

```

export function areNodesEqual(nodeOne, nodeTwo) {
  if (nodeOne.type !== nodeTwo.type) {
    return false

```

```

    }

    if (nodeOne.type === DOM_TYPES.ELEMENT) {
      const { tag: tagOne } = nodeOne
      const { tag: tagTwo } = nodeTwo

      return tagOne === tagTwo
    }

    if (nodeOne.type === DOM_TYPES.COMPONENT) {
      const {
        tag: componentOne,
        props: { key: keyOne }, #1
      } = nodeOne
      const {
        tag: componentTwo,
        props: { key: keyTwo }, #2
      } = nodeTwo

      return componentOne === componentTwo && keyOne === keyTwo
    } #3

    return true
  }
}

```

With that small change, your framework now supports the `key` attribute to compare component nodes. How easy was that? But wait, there's one more important change you need to make.

When the developer provides a `key` attribute to a component, you don't want to pass it to the component as a prop. You want to remove it from the `props` object before passing the props to the component; otherwise, the component will receive a `key` prop that it doesn't know what to do with.

12.1.3 Removing the key attribute from the props object

If you recall, in the previous chapter you implemented a function called `extractPropsAndEvents()`. Its job was to separate the props from the events in the `props` object of a virtual DOM node. I told you that, despite being this just a single line of code, it made sense to extract it into a separate function because you would need to modify it later. Later is now.

Open the *utils/props.js* file and add the code in bold font in listing [12.3](#) to remove the `key` attribute from the `props` object before passing it to the component.

Listing 12.3 Removing the `key` from the `props` object (*utils/props.js*)

```
export function extractPropsAndEvents(vdom) {  
  const { on: events = {}, ...props } = vdom.props  
  delete props.key #1  
  
  return { props, events }  
}
```

You might be wondering whether removing the `key` attribute from the `props` object in the virtual DOM tree will affect the reconciliation algorithm, because the `areNodesEquals()` function requires this attribute to compare component nodes. That is a valid observation. But you aren't modifying the original `props` object in the `vdom` node, you are creating a new one when you use the spread operator:

```
// props is a shallow copy of vdom.props  
const { on: events = {}, ...props } = vdom.props
```

The component instance gets a `props` object without the `key` attribute, but the `props` object in the virtual DOM tree still has it. You can convince yourself about this by running a quick experiment in the browser console.

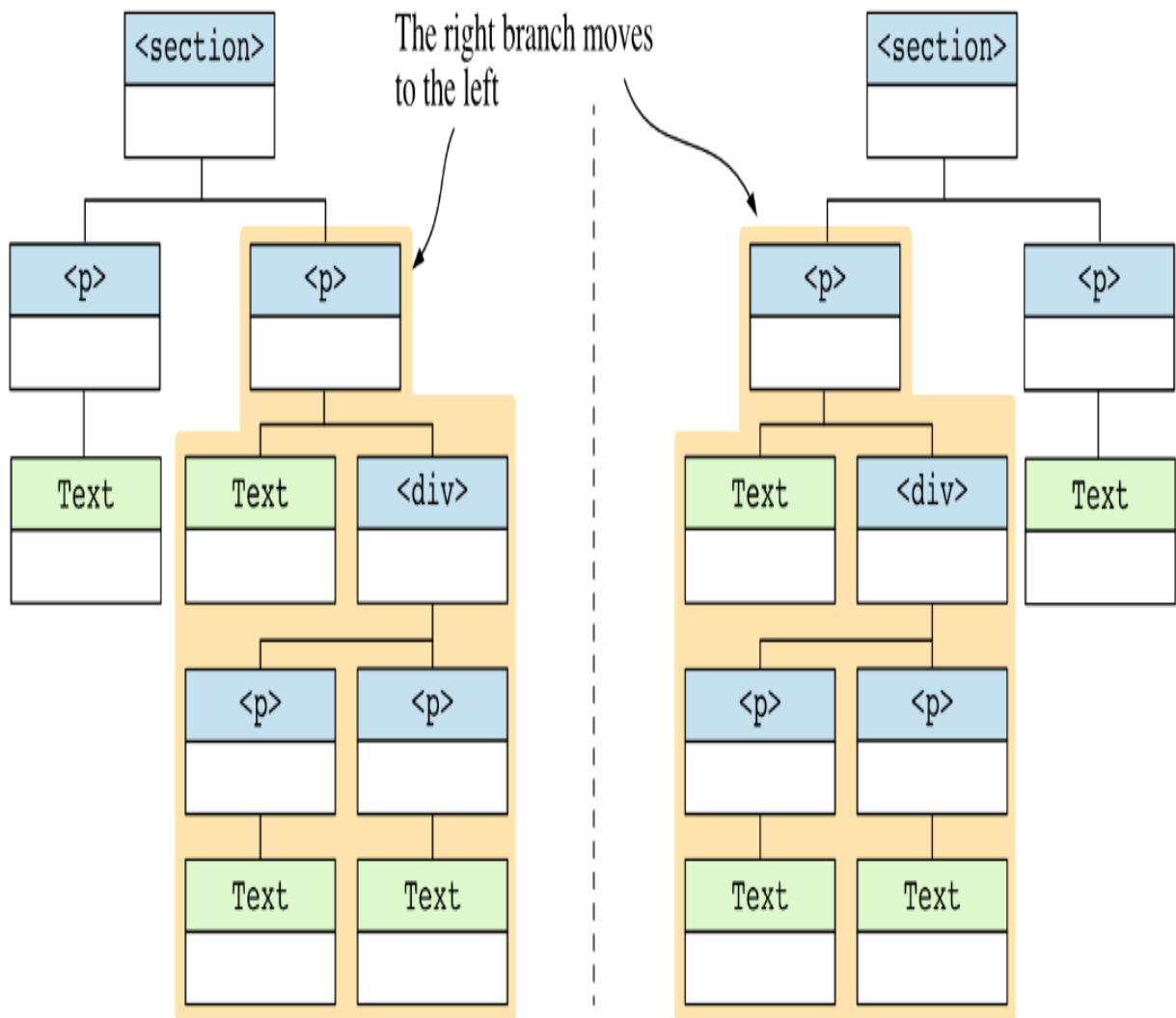
12.2 Extending the solution to element nodes

The reconciliation algorithm you wrote in chapters 7 and 8 is powerful enough to handle any kind of change in a list of elements. But it's also true that it might do much more work than necessary in some cases. Let's see an example of this illustrated in figure [12.3](#), where two `<p>` elements swap positions inside the same `<section>` element.

Figure 12.3 Two elements swapping positions

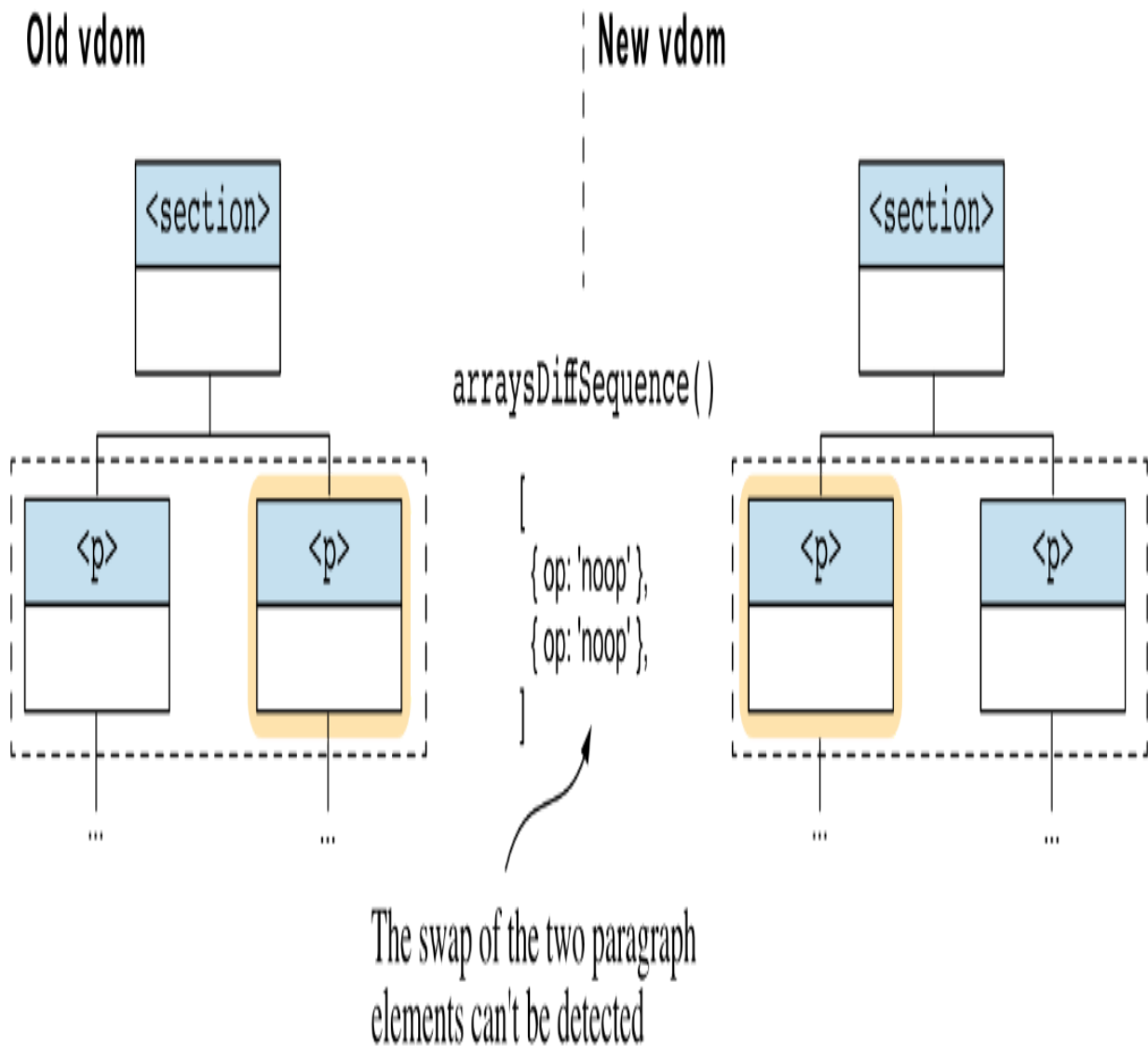
Old vdom

New vdom



In a list of elements, if all elements have the same tag, the reconciliation algorithm (the `arraysDiffSequence()` function, to be more precise) will never know that an element has been moved (figure [12.4](#)). Recall that we deem two elements to be equal if they have the same tag, no matter their contents.

Figure 12.4 Two elements swapping positions



Big portions of the DOM tree might be re-created unnecessarily, because the reconciliation algorithm is incapable of detecting that an element has been moved. In the preceding example, all the elements in the `<p>` elements will be re-created, even though they haven't changed at all; just moved together with their parent element.

To see an example of your reconciliation algorithm doing unnecessary extra work in action, you can work out the next exercise.

Exercise 12.1

Let's look at the problem of unnecessarily re-creating entire branches of the DOM tree in action. You want to copy/paste the code from your framework into the browser's console. You can use your framework's previous version from unpkg.com, or copy/paste it from the bundled file inside your *dist/* folder.

When you have in the console all of your framework's code available, you want to copy/paste the following virtual DOM tree:

```
const before = h('ul', {}, [
  h('li', {}, ['Item 1']),
  h('li', {}, ['Item 2']),
  h('li', {}, [
    h('div', {}, [
      h('div', {}, [
        h('div', {}, [
          h('p', {}, ['Item 3']),
        ]),
      ]),
    ]),
  ]),
])
```

It's a `<ul>` list with three `<li>` elements, the last one containing a nested tree of elements. Mount this view into the body of the current document:

```
mountDOM(before, document.body)
```

You should now see the list rendered in the browser. Now, paste the following virtual DOM tree, which is the same as that previous, but with the last item moved to the middle of the list (highlighted in bold font):

```
const after = h('ul', {}, [
  h('li', {}, ['Item 1']),
  h('li', {}, [
    h('div', {}, [
      h('div', {}, [
        h('div', {}, [
          h('p', {}, ['Item 3']),
        ],
      ],
    ],
  ],
  ],
])
```

```
h('li', {}, ['Item 2']),  
])
```

Your challenge is to set a breakpoint inside the `patchDOM()` function, to inspect the `diffSeq` calculated inside the `patchChildren()` subfunction. You want to check what set of operations the reconciliation algorithm is going to perform to transform the `before` tree into the `after` tree. When the code you want to debug is code you pasted into the browser's console, setting a breakpoint is a bit tricky. You might want to do a quick search to find how to do it in your browser of choice.

Then, execute the `patchDOM()` function to hit the breakpoint:

```
patchDOM(before, after, document.body)
```

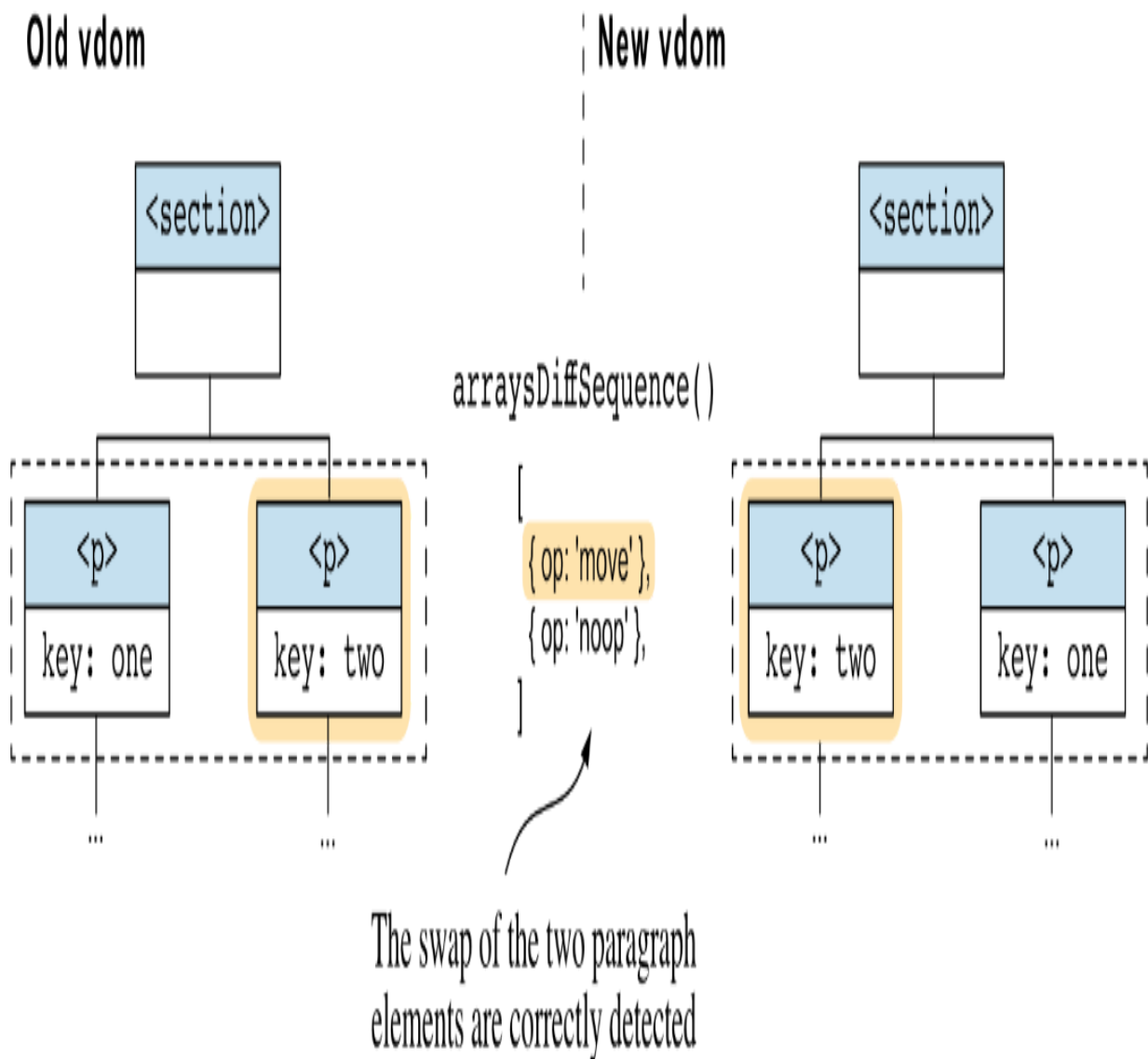
Explore the flow of execution of the `patchDOM()` function and try to answer the following questions:

- Did the reconciliation algorithm figure out that it could simply move the last `<li>` element to the middle of the list?
- Did it even detect there was any movement of elements inside the `<ul>` at all?
- Was the second `<li>` element re-created from scratch?

Find the solution at <http://mng.bz/NV82>.

Why not extend the `key` attribute to element nodes as well? With a bit of help from the developer, we can hint the reconciliation algorithm to help it detect when an element has been moved. Figure 12.5 shows the same example as before, but this time the developer has provided a `key` attribute to each element node and the reconciliation algorithm is able to detect that the element has been moved.

Figure 12.5 Two elements swapping positions using the `key` attribute



Let's modify the `areNodesEqual()` function to support the `key` attribute in element nodes. Open the *nodes-equal.js* file and add the code in bold font in listing [12.4](#).

Listing 12.4 Using the `key` attribute to compare element nodes (nodes-equal.js)

```
export function areNodesEqual(nodeOne, nodeTwo) {  
  if (nodeOne.type !== nodeTwo.type) {  
    return false  
  }  
}
```

```

if (nodeOne.type === DOM_TYPES.ELEMENT) {
  const {
    tag: tagOne,
    props: { key: keyOne }, #1
  } = nodeOne
  const {
    tag: tagTwo,
    props: { key: keyTwo }, #2
  } = nodeTwo

  return tagOne === tagTwo && keyOne === keyTwo #3
}

if (nodeOne.type === DOM_TYPES.COMPONENT) {
  const {
    tag: componentOne,
    props: { key: keyOne },
  } = nodeOne
  const {
    tag: componentTwo,
    props: { key: keyTwo },
  } = nodeTwo

  return componentOne === componentTwo && keyOne === keyTwo
}

return true
}

```

With this simple change, you can now use the `key` attribute to hint the reconciliation algorithm when an element has been moved. You now need to modify the `mountDOM()` function to use the `extractPropsAndEvents()` function, in charge of removing the `key` attribute from the `props` object before passing it to the element. If you didn't do this, your `addProps()` function would attempt to add a `key` attribute to the DOM node, which is not what you want. In fact, the `key` attribute doesn't exist for DOM elements.

Note

If the `key` attribute existed in the DOM standard, you'd want to name the prop differently, to avoid collisions. Some frameworks prepend their internal props with a dollar sign, like `$key`, or two dollar signs, like `$$key`. You could also look at Python for inspiration and use dunders (double underscores), like `__key__`.

For simplicity, we'll just use `key` as the prop name. The only drawback with this approach is that you won't be able to call "key" a prop in your components, because "key" is reserved for the framework.

Open the *mount-dom.js* file and modify the code in the `createElementNode()` function as in listing [12.5](#). You want to remove the part where you destructure the `props` object and pass it to the `addProps()` function. It's the `addProps()` function that'll be in charge of extracting the props and events from the `vdom` node.

Listing 12.5 Modifying the `createElementNode()` function (*mount-dom.js*)

```
function createElementNode(vdom, parentEl, index,
hostComponent) {
  const { tag, props, children } = vdom
  const { tag, children } = vdom #1

  const element = document.createElement(tag)
  addProps(element, props, vdom, hostComponent)
  addProps(element, vdom, hostComponent) #2
  vdom.el = element

  children.forEach((child) =>
    mountDOM(child, element, null, hostComponent)
  )
  insert(element, parentEl, index)
}
```

Now, inside the `addProps()` function, you need to extract the props and events from the `vdom` node. For this, you'll use the `extractPropsAndEvents()` function, which you just modified to remove the `key` attribute from the `props` object. Modify the `addProps()` function as in listing [12.6](#).

Listing 12.6 Modifying the `addProps()` function (*mount-dom.js*)

```
function addProps(el, props, vdom, hostComponent) {
function addProps(el, vdom, hostComponent) { #1
  const { on: events, ...attrs } = props
  const { props: attrs, events } =
extractPropsAndEvents(vdom) #2
```

```
vdom.listeners = addEventListeners(events, el,
hostComponent)
  setAttributes(el, attrs)
}
```

That's it! You can now use the `key` attribute to hint the reconciliation algorithm when an element has been moved.

Before you publish the new version of your framework, let's take a little detour to talk about some good practices when it comes to using the `key` attribute.

12.3 Using the key attribute

The `key` attribute is a powerful tool to help the reconciliation algorithm detect when an element has been moved. But it's also true that it's a tool that can be easily misused. In this section we'll look at two common mistakes when using the `key` attribute.

12.3.1 Mistake #1: Using the index as key

The typical example of misuse is when the developer uses the index of the element in the list as the value of the `key` attribute. Why is this a bad idea? In short, because it isn't guaranteed that the same elements or components will always be in the same position in the list, and hence, they won't always have the same index—a problem referred to as *unpredictable reordering*. The `key` attribute doesn't work unless a node consistently keeps the same `key` value.

This is a problem that affects lists of components only, and not lists of elements. The reconciliation algorithm, as you already saw, might need to do extra work by re-creating entire view trees, but it will always get the final result right. With components, this isn't the case, because components have their own state, invisible to the virtual DOM. So, components can't be re-created—the live instances need to be moved around in the DOM. This is why the `key` attribute is so important for components.

Imagine you have a list with three `<MyCounter>` components, and you decide to use their position in the list as the value of the `key` attribute:


```
<div>
  <MyCounter key="0" />
  <MyCounter key="1" />
  <MyCounter key="2" />
</div>
```

Now, imagine that the middle component is removed from the list, the one with the `key` attribute set to 1:

```
<div>
  <MyCounter key="0" />
  <MyCounter key="1" />
  <MyCounter key="2" />
</div>
```

The component with the `key` attribute set to 2 will be moved to the position of the component with the `key` attribute set to 1, and thus, its index changes from 2 to 1:

```
<div>
  <MyCounter key="0" />
  <MyCounter key="1" /> <!-- Wrong! was key="2" -->
</div>
```

By looking at the two virtual DOM trees, the reconciliation algorithm will incorrectly assume that the component with the `key="2"` was removed from the bottom of the list, when in reality, it was the middle component that was removed.

Lists of components or elements are typically generated inside a loop, and the index of the element in the list is the index of the loop. Thus, the index of the element might change as things are added, removed, or moved in the list. The developer must make sure that the `key` attribute is set to a value that doesn't change, even if the index of the element in the list changes.

Important

The `key` attribute attached to a component or element must be consistent across renders. The same element or component must always have the same `key` value, throughout the whole life cycle of the node.

To see this problem of using indices as keys in action, I recommend you work on the exercise below.

Exercise 12.2

Create a component called `MyCounter` that consists on a button with a counter. The button increments its counter when clicked. Next to the button, there should be another button with the label "Remove," which emits a `'remove'` event when clicked.

Create a second component (you can call it `App`) that has a count of counters as its state. Set its initial state to 3:

```
const App = defineComponent({
  state() {
    return { counters: 3 }
  },
  render() { ... }
})
```

Make the `App` component render a list of `MyCounter` components, one for each counter in the state. For their key, use the index of the counter in the list. When the `'remove'` event is emitted, update the state to remove one counter. You can do it like this:

```
const App = defineComponent({
  state() { ... },
  render() {
    const { counters } = this.state

    return h(
      'div',
      {},
      Array(counters)
        .fill()
        .map((_, index) => {
          return h(MyCounter, {
            key: index,
            on: {
              remove: () => {
                this.updateState({ counters: counters - 1 })
              },
            },
          )
        })
    )
  },
})
```

```

        })
    })
)
}
})

```

Mount the application into the DOM. Next, change the counter in the middle of the list to have a count of 2, and the last counter to have a count of 3. Click the "Remove" button of the counter in the middle (the second one, with a count of 2). Is the correct counter removed? (You'd expect to have two counters left, with counts 1 and 3.)

Find the solution at <http://mng.bz/D919>.

12.3.2 Mistake #2: Using the same key for different elements

Keys must be unique across the same list. Different lists can have elements with the same key, but not the same list. What happens when the same key is assigned to two elements or components in the same list? The reconciliation algorithm can't unequivocally determine which element was moved, and thus, might wrongly assume a node moved when it didn't.

Let's use a similar example to the previous case. Imagine you mistakenly assign the same `key` attribute to the two last components in the list, as follows:

```

<div>
  <MyCounter key="abc" />
  <MyCounter key="def" />
  <MyCounter key="def" /> <!-- Wrong! Same key as the
previous one -->
</div>

```

Now, if I remove one of the two last components from the list, would you be able to tell me which one I removed?

```

<div>
  <MyCounter key="abc" />
  <MyCounter key="def" />
</div>

```

You can't, and neither can the reconciliation algorithm.

Important

The `key` attribute attached to a component or element must be unique across the same list.

You can try the exercise below to see this problem in action.

Exercise 12.3

Use the same `MyCounter` component from the previous exercise. This time, modify the `App` render function to render all the counters with the same key (I'll use `'abc'` as the key):

```
render() {
  const { counters } = this.state

  return hFragment([
    h('h1', {}, ['Repeated key problem']),
    h('p', {}, [
      'Set a different count in each counter, then remove the
middle one.',
    ]),
    h(
      'div',
      {},
      Array(counters)
        .fill()
        .map(() => {
          return h(MyCounter, {
            key: 'abc',
            on: {
              remove: () => {
                this.updateState({ counters: counters - 1 })
              },
            },
          })
        })
    ),
  ])
}
```

Repeat the same process as in the previous exercise: set the count of the second counter to 2, and the count of the third counter to 3. Then, remove the middle counter. Does the correct counter get removed? Can you explain why?

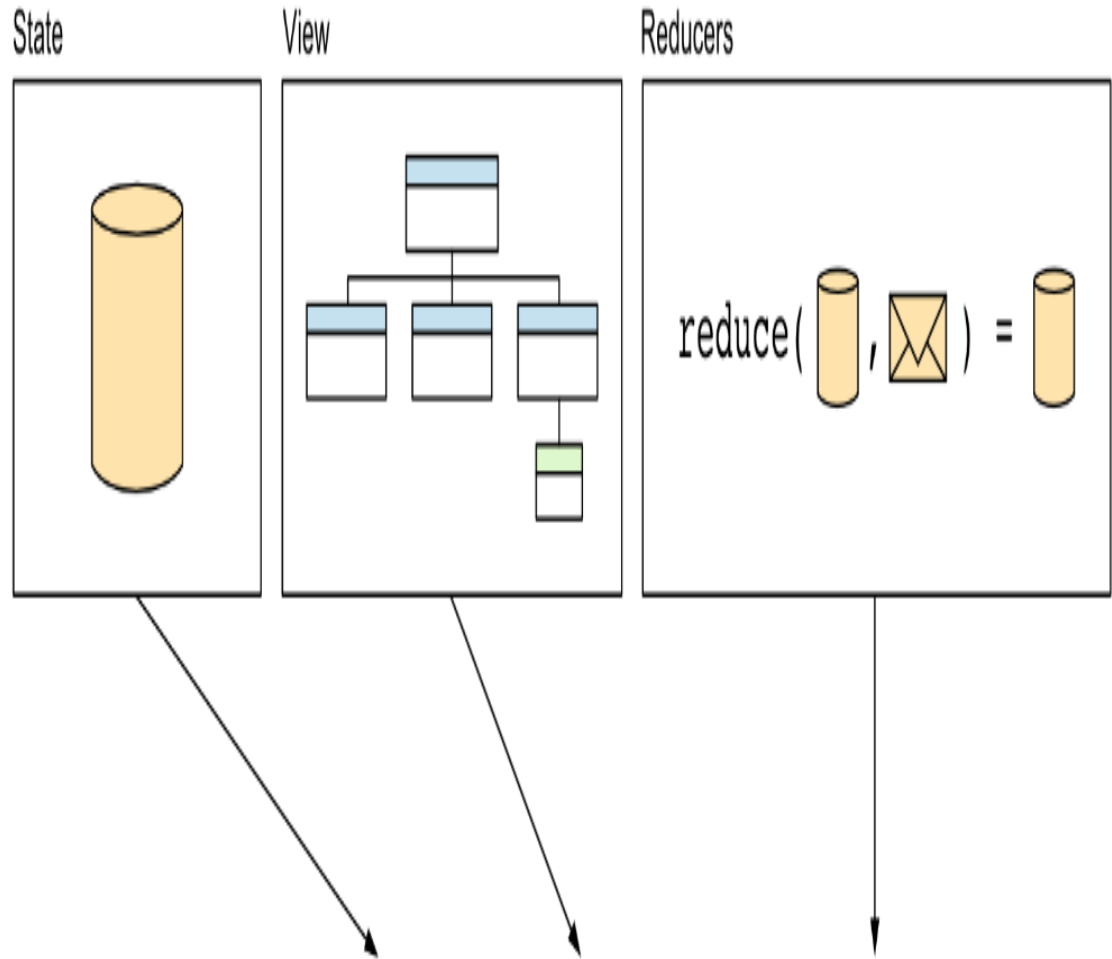
Find the solution at <http://mng.bz/1V2M>.

Let's now move on to modifying the `createApp()` function to use the new stateful component system.

12.4 The application instance

The application instance doesn't need to handle the entire application's state anymore; each component handles its own now. Thus, there's no need to pass the state object and reducers to the `createApp()` function anymore, as in figure [12.6](#).

Figure 12.6 The current framework's application instance requires the global state, reducers, and the root component

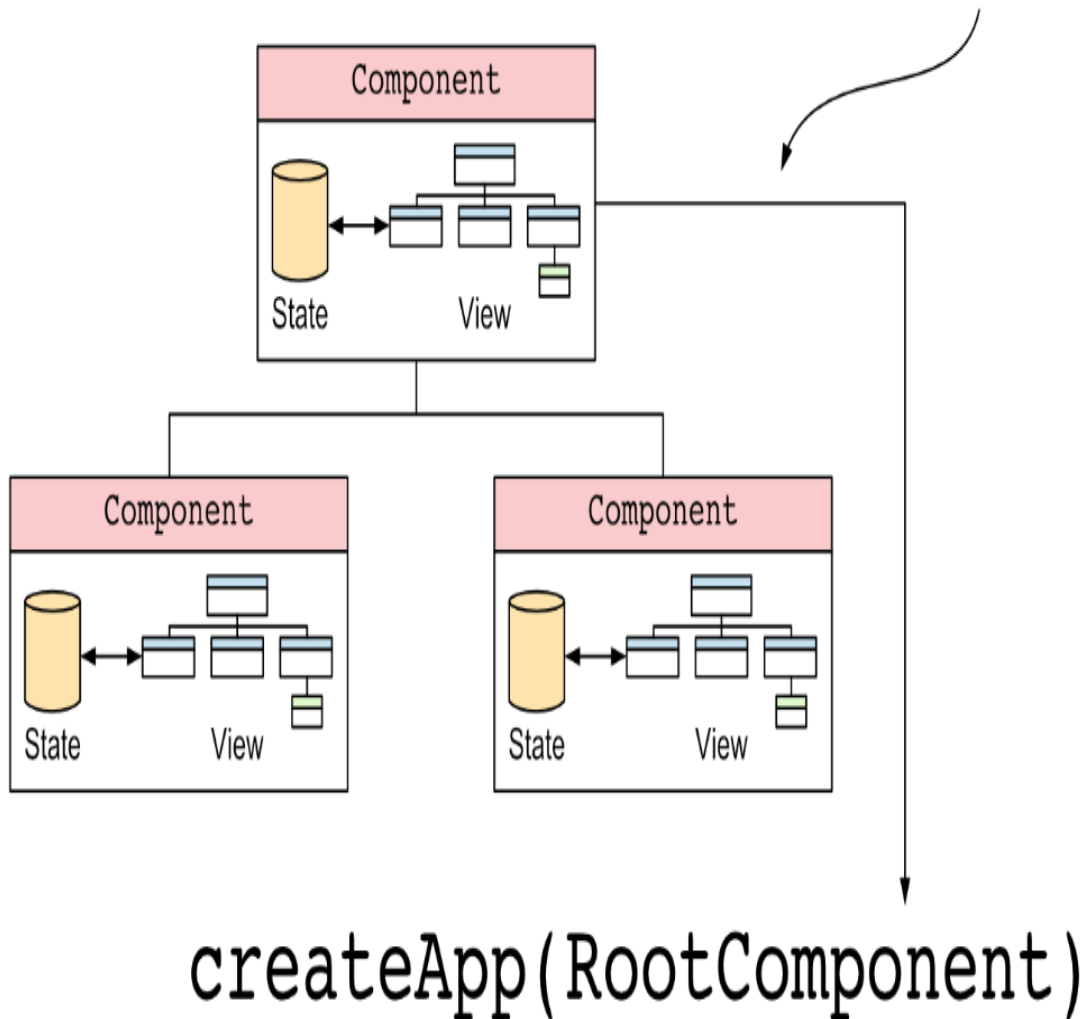


```
createApp({ state, view, reducers })
```

You just need to pass it the root component, the component that contains the whole application's view split in subcomponents, as depicted in [figure 12.7](#).

Figure 12.7 The new version of the framework's application instance requires the root component only

The root component, the one on top of the hierarchy is passed to `createApp()`



This root component might accept props, so you want to pass the props to the `createApp()` function as well:

```
function createApp(RootComponent, props = {}) {  
  // --snip--  
}
```

Open the `app.js` file and start by removing all the code you had written; things are going to change a lot. The idea is still the same, though: you want to return an object with two methods: `mount()` and `unmount()`. This time the code is

much simpler, because each component in the view hierarchy handles its own state.

In the *app.js* file, add the code in listing [12.7](#).

Listing 12.7 The application instance (app.js)

```
import { mountDOM } from './mount-dom'
import { destroyDOM } from './destroy-dom'
import { h } from './h'

export function createApp(RootComponent, props = {}) { #1
  let parentEl = null
  let isMounted = false
  let vdom = null

  function reset() { #2
    parentEl = null
    isMounted = false
    vdom = null
  }

  return {
    mount(_parentEl) {
      if (isMounted) { #3
        throw new Error('The application is already mounted')
      }

      parentEl = _parentEl #4
      vdom = h(RootComponent, props) #5
      mountDOM(vdom, parentEl) #6

      isMounted = true
    },

    unmount() {
      if (!isMounted) { #7
        throw new Error('The application is not mounted')
      }

      destroyDOM(vdom) #8
      reset() #9
    },
  }
}
```


Much simpler, right? To get the framework ready for publication, you want to export the `defineComponent()` function from your framework's public API. That is as simple as exporting it from the main barrel file: *src/index.js*. Open this file and add the line in bold font in listing [12.8](#).

Listing 12.8 The framework's updated API (index.js)

```
export { createApp } from './app.js'  
export { defineComponent } from './component.js'  
export { DOM_TYPES, h, hFragment, hString } from './h.js'
```

Now that the framework's API is ready, let's publish your new version of the framework to NPM.

12.5 Publishing the framework

To publish version 3.0 of your framework, you need to update the version number in the *package.json* file. Open this file (remember, that's inside the *packages/runtime* folder, not the top level one) and change the version number to 3.0.0:

```
{  
  "name": "your-fwk-name",  
  "version": "2.0.0",  
  "version": "3.0.0",  
  ...  
}
```

Now, you can publish the new version of the framework to NPM. Make sure your terminal is in the *packages/runtime* folder and run the following command:

```
$ npm publish
```

You—and the rest of the world—can now install and use the version of your framework that includes stateful components. You'll use this new version in the following exercise to refactor the TODOs application.

Exercise 12.4 — Challenge

Using the new version of your framework, refactor the TODOs application to use stateful components.

Find the solution at <http://mng.bz/BAKg>.

12.6 Summary

- In a list of components, a unique key must be provided to each component so that the reconciliation algorithm can differentiate them.
- In a list of elements, the key is not mandatory, but it's recommended to use it to improve performance. Without a `key` attribute, the reconciliation algorithm will do more work than necessary.
- The key must be unique and invariant in the list of elements or components.
- Never use the index of the element of component as key, as when an element is removed, added, or moved, the index of the rest of the elements will change.

13 The component lifecycle hooks and the scheduler

This chapter covers

- What a lifecycle hook is
- Executing code when a component is mounted
- Executing code when a component is unmounted
- Publishing version 4 of the framework

It's not uncommon for a component to execute some code when it's mounted into or unmounted from the DOM. The classic example is fetching data from a server right after the component is mounted. In that case, the component is already part of the DOM, thus can display a loading indicator while the data is being fetched. When the data is ready, the component updates its view by removing the loading indicator and rendering the newly fetched data.

Your existing framework runs code only when a user-originated event happens, like clicking a button. (Technically, it could also run code as part of a `setTimeout()` or `setInterval()` callback, but let's omit this possibility for the sake of simplicity in the explanation.)

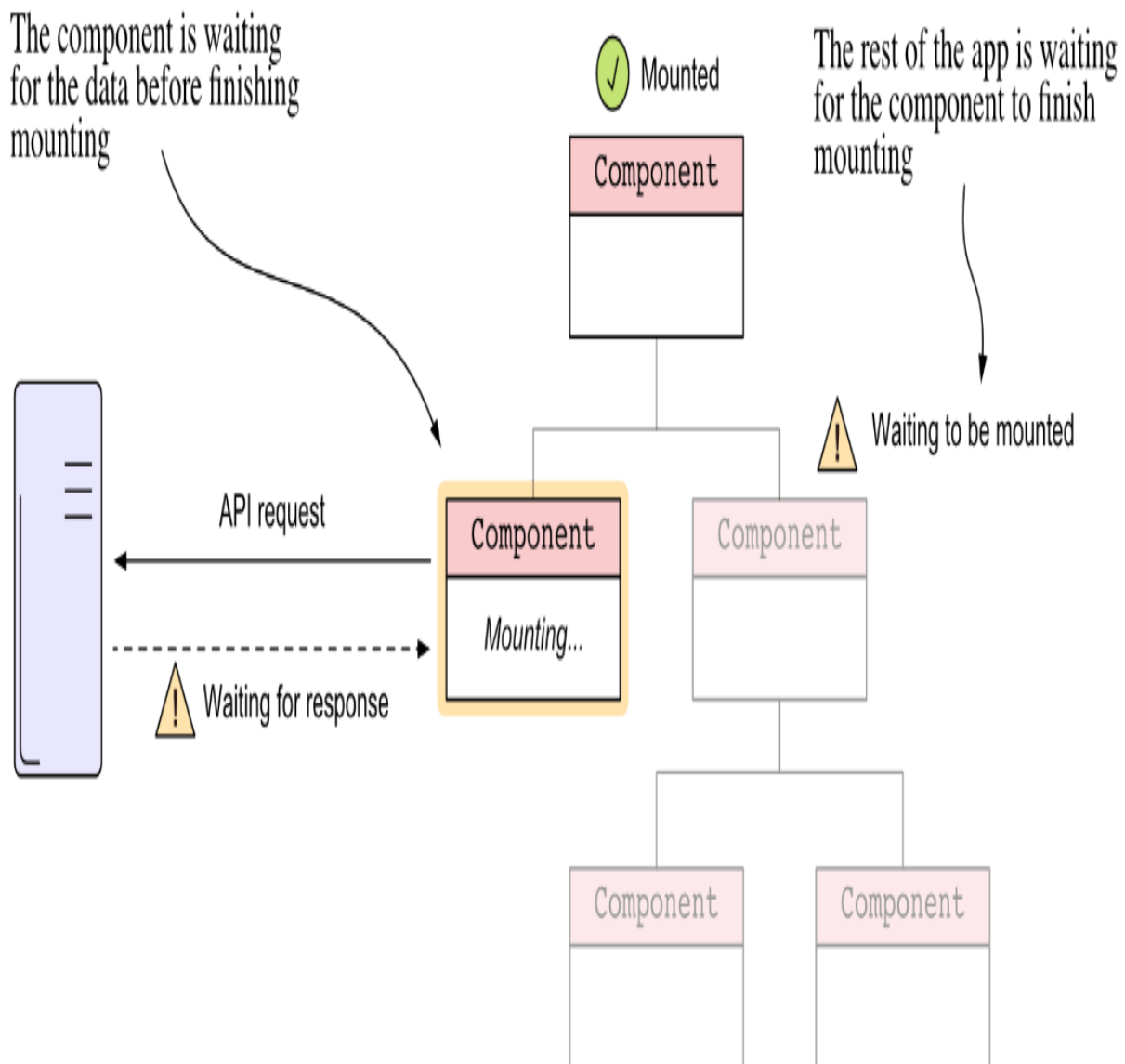
It can't fetch data when a part of the webpage first shows up. This is a limitation because developers can't make code run before the user interacts with the page.

A *lifecycle hook* is a user-defined function that is executed when a component goes through a lifecycle event. Lifecycle events include the creation, mounting, updating, and unmounting of a component. In this chapter, you'll explore how lifecycle hooks fix the aforementioned limitation, and how to implement them in your framework.

Lifecycle hooks are often asynchronous, like for example when the user uses them to fetch data from a server once a component is mounted. Your mounting

process has been synchronous so far, but as soon as you run asynchronous code as part of it, an interesting problem arises. If you `await` the component's hook that executes after the component is mounted, you block the first render of the application: while a component is waiting for the hook to finish, the remaining components have to wait in line to be rendered. That's not what you want; it yields a bad user experience and the sensation that the application is slow (when in reality is just waiting to finish mounting a component before it moves on to the next), as depicted in figure [13.1](#).

Figure 13.1 While a component is fetching data from a server, the rest of the application needs to wait to be mounted.



To avoid blocking the first render of the application you shouldn't use `await` for the `mounted` hook to finish. You'll learn about *the scheduler*, a mechanism that allows the execution of code to be delayed until the current render cycle is finished. With it you can schedule asynchronous hooks to run after the application is fully mounted, thus not blocking the first render.

In this chapter, you'll publish version 4 of your framework so that you can put lifecycle hooks to use.

Important

You can find all the listings in this chapter in the [listings/ch13 directory](#) of the book's repository. The code in this chapter can be checked out from [the ch14 label](#):

```
$ git switch --detach ch14
```

Code catch-up

In the last chapter, you updated the `areNodesEqual()` function to include the case of two component nodes. It checks that the component types are the same and that their `key` props—if they have one—are equal. You did a quick modification to the `extractPropsAndEvents()` function to remove the `key` property from the props object so that it doesn't reach the component and it isn't mistaken for a prop.

You also included the `key` prop comparison for the case of two element nodes inside the `areNodesEqual()` function. This change required you to modify the `createElementNode()` and `addProps()` functions to use the `extractPropsAndEvents()` function so the `key` prop is correctly removed from the props object before it reaches the element creation and is mistaken for an HTML attribute.

Last, you rewrote the `createApp()` function and exported it from the *runtime* package by exporting it from the *index.js* file.

13.1 The component lifecycle

The *lifecycle* of a component is the sequence of events that it goes through from the moment it's created until it gets destroyed. Let's take a look at the lifecycle of a component in your framework (table [13.1](#)).

Table 13.1 The lifecycle of a component

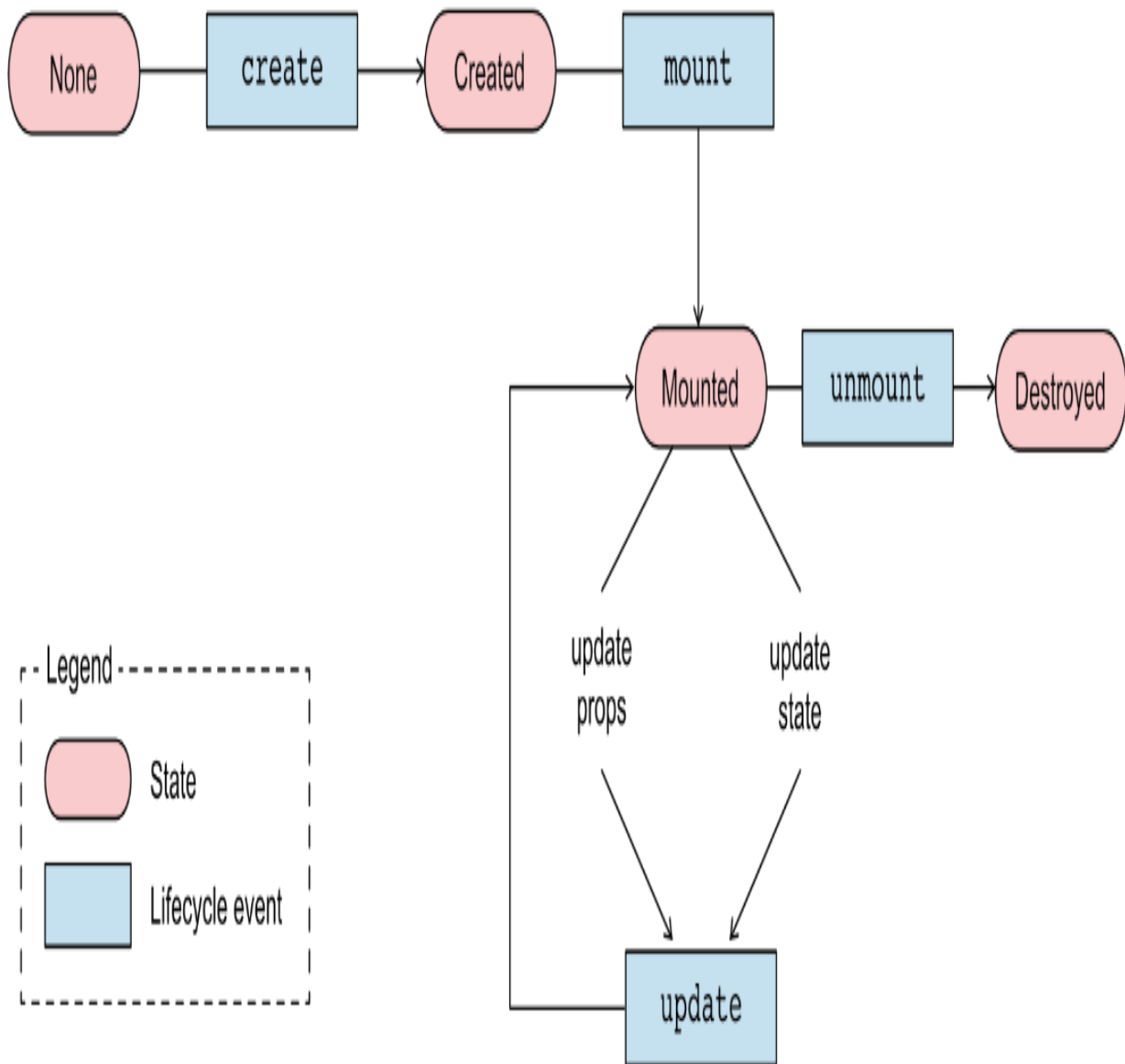
Lifecycle event	Code	Explanation
<i>Creation</i>	<code>new Component(...)</code>	When the <code>mountDOM()</code> function is

		passed a component, it first instantiates it.
<i>Mounting</i>	<code>component.mount (...)</code>	Then, the <code>mountDOM()</code> function calls the component's <code>mount()</code> method. This is when the component is mounted into the DOM and its view first appears in the screen.
<i>Updating</i>	<code>#patch() { ... }</code>	The component remains in the screen until it's removed from the DOM, but it can render itself multiple times when the internal state or the external props change. This is handled by the component's private <code>#patch()</code> method.
<i>Unmounting</i>	<code>component.unmount()</code>	Finally, when the component is removed from the DOM, the <code>destroyDOM()</code> function calls the component's <code>unmount()</code> method. This is when the component is unmounted from the DOM and its view disappears from the screen.

Figure [13.2](#) shows the lifecycle of a component in a diagram, which includes the lifecycle events discussed in the previous table. Starting from the left, the component starts in the *none* state—it's not instantiated yet. Then, after the component is created, it goes to the *created* state. This is when you have an instance of your component class, but it's not mounted into the DOM yet. Next is the mount lifecycle event, after which the component is in the *mounted* state. From there, it can take two paths: it can either be updated or unmounted. If it's updated, it goes back to the *mounted* state, and the cycle repeats. If it's

unmounted, it goes to the *destroyed* state; the instance of the component still exists, but it's detached from the DOM.

Figure 13.2 The lifecycle of a component



Now that you're familiar with how components go through different lifecycle events, let's dive into why it's useful. This feature lets developers working with your framework run their own code at these specific events. This way, they can customize how their components behave at each stage, tailoring the behavior to suit their application's needs.

13.1.1 Lifecycle hooks

Different frameworks have different lifecycle hooks, but *creation*, *mounting*, and *unmounting* are the three most common and can be found in almost all (if not all) frameworks. Table [13.2](#) is a quick comparison of the lifecycle hooks in some popular frameworks.

Table 13.2 Lifecycle hooks in popular frameworks

Framework	Hooks	Comments
Svelte (v4)	• <code>onMount()</code> — executed when the component is mounted into the DOM, • <code>beforeUpdate()</code> — executed before the component is updated, • <code>afterUpdate()</code> — executed after the component is updated, and • <code>onDestroy()</code> — executed when the component is unmounted from the DOM.	Svelte defines four main lifecycle hooks . Svelte also defines a special lifecycle hook called <code>tick()</code> that returns a promise that resolves once the "pending changes have been applied."
Vue (v3)	• <code>onBeforeMount()</code> — executed before the component is mounted into the DOM, • <code>onMounted()</code> — executed when the component is mounted into the DOM, • <code>onBeforeUpdate()</code> — executed before the component's DOM is updated, • <code>onUpdated()</code> — executed after the component's DOM is updated,	Vue defines as many as twelve lifecycle hooks , but these are the most common ones.

	• <code>onBeforeUnmount()</code> — executed before the component is unmounted from the DOM, and • <code>onUnmounted()</code> — executed when the component is unmounted from the DOM.	
Angular (v16)	• <code>ngOnInit()</code> — executed when the component is initialized, • <code>ngOnChanges()</code> — executed when the component's input properties change, • <code>ngAfterViewInit()</code> — executed after the component's view has been initialized, • <code>ngOnDestroy()</code> — executed before the component is destroyed.	Angular defines eight lifecycle hooks , but these are the most common ones.
React (v18)	To execute code when the component is mounted, call the <code>useEffect()</code> hook passing an empty array as the second argument: <pre>useEffect(() => { // code to execute when the component is mounted }, [])</pre> You can specify the dependencies of the hook by passing them in the array. In that case, the effect would run when the component is mounted and every time one of the dependencies changes. To execute code when the component is	The case of React is a bit different. React doesn't define lifecycle hooks, but it does define side effects that are executed when the component is mounted, updated, or unmounted.

unmounted, return a function from the `useEffect()` hook that has an empty array as the second argument:

```
useEffect(() => {  
  // code to execute when  
  the component is mounted  
  
  return () => {  
    // code to execute when  
    the component is unmounted  
  }  
}, [])
```

To execute code when the component is updated (including after being mounted), don't pass the dependencies array as the second argument to the `useEffect()` hook:

```
useEffect(() => {  
  // code to execute every  
  time the component is  
  updated  
})
```

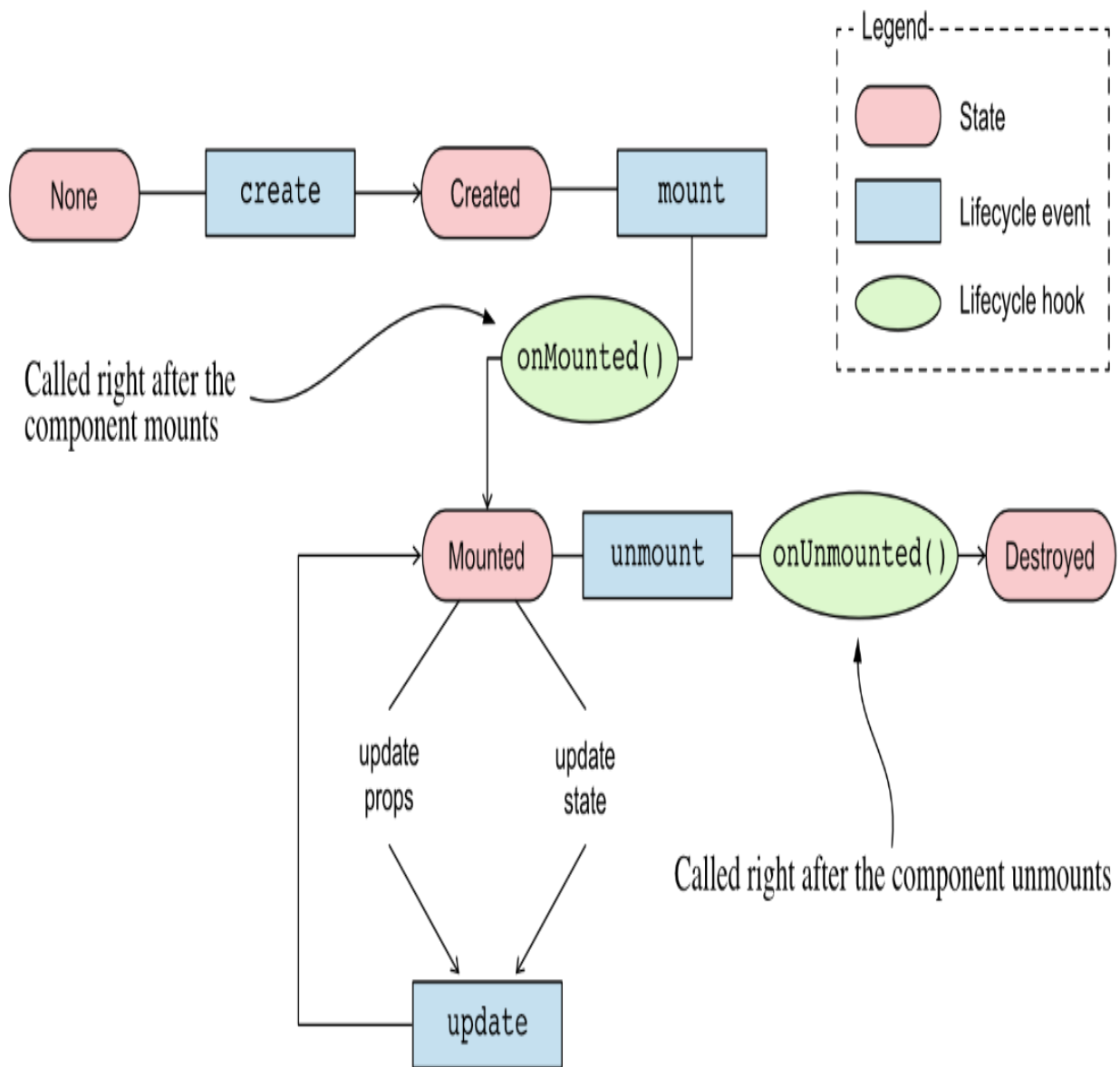
In this case, the effect would run when the component is mounted and every time the component is updated, that is, when the component's state or props change.

You'll implement two lifecycle hooks in your framework:

- `onMounted()` — allows the developer to run code just after the component is mounted into the DOM
- `onUnmounted()` — allows the developer to run code just after the component is unmounted from the DOM.

These two hooks are depicted in the diagram in figure [13.3](#), which is an updated version of the lifecycle diagram in figure [13.2](#).

Figure 13.3 The `onMounted()` and `onUnmounted()` lifecycle hooks



You'll see that the process is so straightforward that you might want to—taking inspiration from other frameworks—implement more lifecycle hooks on your own. The `onMounted()` and `onUnmounted()` lifecycle hooks are the most used by developers, so they are a great starting point.

No more talking, let's get to work!

13.2 Implementing the mounted and unmounted lifecycle hooks

The first step to implement the `onMounted()` and `onUnmounted()` lifecycle hooks is to define them in the `Component` class. You want to allow developers to define these lifecycle hooks when they create a new component using the `defineComponent()` function:

```
defineComponent({
  onMounted() { ... },
  onUnmounted() { ... },

  render() { ... }
})
```

To do that, you need to accept those two new functions in the object that's passed as argument to the `defineComponent()` function. You want to give them an empty function as their default value so that the developer can omit them. Open the `component.js` file and add the code in bold font in listing [13.1](#).

Listing 13.1 Defining a component with mounted and unmounted lifecycle hooks (`component.js`)

```
const emptyFn = () => {}

export function defineComponent({
  render,
  state,
  onMounted = emptyFn,
  onUnmounted = emptyFn,
  ...methods
}) {
  class Component {
    // --snip--
  }
}
```

There are two important things you want to take into account regarding the passed in `onMounted()` and `onUnmounted()` functions:

- Hooks asynchronicity
- Hooks execution context

Let's discuss these two concerns in order.

13.2.1 Hooks asynchronicity

The functions passed by the developer are quite possibly asynchronous and return a `Promise`. The typical use of the mounted hook is to fetch data from an API, so it's very likely that the function awaits for a `Promise` to be resolved:

```
async onMounted() {  
  const data = await fetch('https://api.example.com/data')  
  this.updateState({ data })  
}
```

It also might be the case that the hooks are synchronous, but wrapping their result inside a `Promise` doesn't hurt and has the benefit of allowing you to treat all hooks as if they were asynchronous:

```
Promise.resolve(onMounted())
```

The second concern is the execution context; let's see what that means.

13.2.2 Hooks execution context

As with `state()` and `render()`, the developer expects the context of the `onMounted()` and `onUnmounted()` hooks to be the component instance. Inside the hook functions, the developer should be able to access the component's state and methods:

```
async onMounted() {  
  const data = await fetch('https://api.example.com/data')  
  this.updateState({ data })  
}
```

You already know how to solve this: explicitly bind the functions to the component instance:

```
Promise.resolve(onMounted.call(this))
```

Here's what you want to do: create two new methods in the `Component` class called the same way as the lifecycle hooks, which wrap the hooks inside a promise and bind them to the component instance.

13.2.3 Dealing with asynchronicity and execution context

In the *component.js* file, add the code in bold font in listing [13.2](#), just below the constructor.

Listing 13.2 Wrapping the mounted and unmounted lifecycle hooks inside a Promise (component.js)

```
export function defineComponent({
  render,
  state,
  onMounted = emptyFn,
  onUnmounted = emptyFn,
  ...methods
}) {
  class Component {
    // --snip-- //

    constructor(props = {}, eventHandlers = {},
parentComponent = null) {
      // --snip-- //
    }

    onMounted() {
      return Promise.resolve(onMounted.call(this))
    }

    onUnmounted() {
      return Promise.resolve(onUnmounted.call(this))
    }

    // --snip-- //
  }

  // --snip-- //
}
```

Now that you have the `onMounted()` and `onUnmounted()` lifecycle hooks defined, you need to call them at the right time. But, what is the right time?

13.3 The scheduler

Mounting the application is (at least until now) a synchronous and fast process. That’s exactly what you want: the application’s first render should be as fast as possible. But lifecycle hooks might not be as fast; they might take some time to finish while they fetch data from a server, for example. What would happen if you `await` for the `onMounted()` hook to finish before moving on to the next component inside the `mountDOM()` function?

```
async function mountDOM(...) {
  switch (vdom.type) {
    // --snip --//

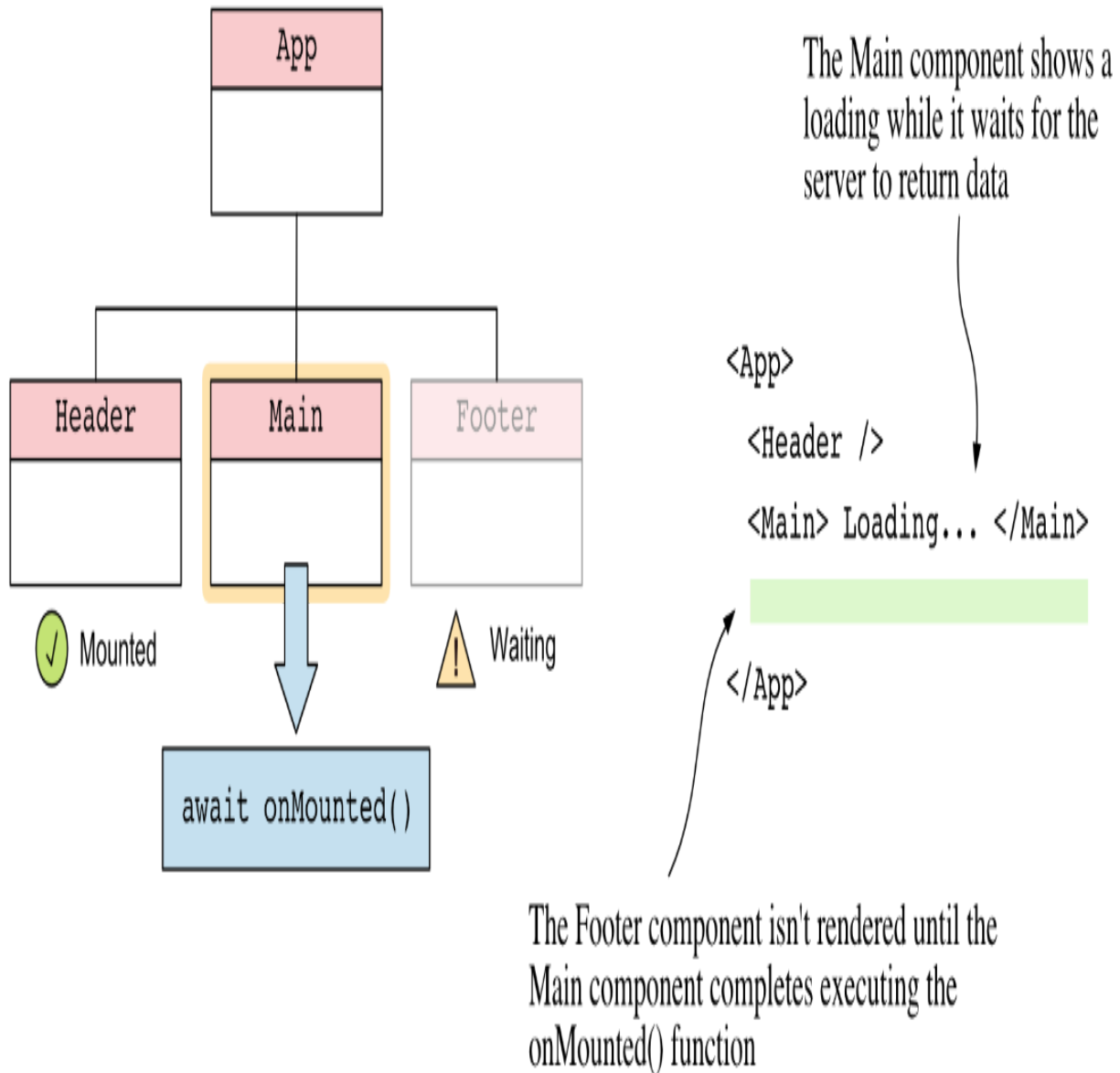
    case DOM_TYPES.COMPONENT: {
      createComponentNode(vdom, parentEl, index,
hostComponent)
      await vdom.component.onMounted() #1
      break
    }

    // --snip --//
  }
}
```

This would block the first render of the application, because the `mountDOM()` function wouldn’t move on to the next component until the `onMounted()` hook is finished—and this might take some time. Let’s look at an example.

The application whose virtual DOM tree is depicted in figure [13.4](#) has three components: `Header`, `Main`, and `Footer`. The `Main` component has an `onMounted()` hook that fetches data from a server. When the application mounts, the `mountDOM()` function first mounts the `Header` component, then the `Main` component, and then it calls the `onMounted()` hook of the `Main` component, which blocks the rendering of the `Footer` component until the hook is finished.

Figure 13.4 The `Main` component blocks the rendering of the `Footer` component



Let's look at a naive solution to this problem.

13.3.1 A simple solution... that doesn't quite work

A simple solution would be simply to not `await` for the `onMounted()` hook to finish:

```

function mountDOM(...) {
  switch (vdom.type) {
    // --snip --//
  }
}

```

```

    case DOM_TYPES.COMPONENT: {
      createComponentNode(vdom, parentEl, index,
hostComponent)
      vdom.component.onMounted()
      break
    }

    // --snip --//
  }
}

```

Let's explain why this solution might be problematic.

The code inside `onMounted()` would execute until the first `await` is encountered. At this point a callback would be added to the [microtask queue](#) (which I'll explain shortly), and the mounting process would continue. The promise's callback would be executed later, when JavaScript's execution call stack is empty—that is, after the `mountDOM()` function is finished.

For example, if the `onMounted()` hook of the `Main` component from the example above is defined as follows:

```

async onMounted() {
  this.setState({ loading: true })
  const data = await fetch('https://api.example.com/data')
  this.setState({ data, loading: false })
}

```

The first line of the hook, which sets the `loading` state to `true`, would be executed immediately. Then, the `fetch()` function would be called, which would return a `Promise` whose callback—the rest of the `onMounted()` function's body—would be added to the microtask queue, and the mounting process would continue.

Note

Recall that using `async/await` can be seen as syntactic sugar to handling promises using `then()` and `catch()`. Therefore, the preceding code can be rewritten as follows:

```

onMounted() {
  this.setState({ loading: true })
}

```

```
fetch('https://api.example.com/data').then((data) => {
  this.setState({ data, loading: false })
})
}
```

That might help you understand why the first line is called synchronously, whereas the rest of the code (the callback passed to `then()`) is moved to the microtask queue. The callback is made of the rest of the `onMounted()` function's body: the last line that sets the `loading` state to `false` and the data received from the server.

This might work well in most cases, but it leaves unhandled promises in your code, which exposes your mounting process to two problems:

- You have no control over the sequencing of the code in the mounting process. An `await` is a blocking operation which prevents some part of the code from executing until a promise is resolved. It ensures that the part of the code that requires the promise to have resolved doesn't run before the promise is resolved. If you don't handle the components' `onMounted()` promises, the user might interact with the component while the `onMounted()` callback is waiting in the microtask queue. This interaction might cause a re-render of the not yet fully-mounted component. Then, when the `onMounted()` callback is finally executed, it likely renders the component again, which might cause a race condition. It's best if a component isn't rendered while it's being mounted.
- If an error happens while executing the `onMounted()` hook, the promise is rejected, but you have no way to handle the error. If the developer isn't notified of the error, they might not even know that something went wrong. You want to, at least, print the error to the console so that the developer can analyze it and fix the problem.

Note

Typescript-eslint has a rule called "no-floating-promises" that warns you when you have unhandled promises in your code. Similarly, there is a plugin for ESLint called "eslint-plugin-no-floating-promise" that does the same. Unhandled (or floating) promises expose your code to sequencing and error handling problems that are complex to track down. As a general rule, avoid unhandled promises in your code.

What you want to do instead is mount the entire application synchronously, then execute all the `onMounted()` hooks in the order they were encountered, ensuring they complete and their errors are handled, and only then allow the framework to execute any other code. To do that, we need to learn about how asynchronous code is executed in the browser, so bear with me. If you're already familiar with the event loop, the task, and microtask queues, you can skip the next section. In particular, you should know about the order in which the event loop processes the tasks and microtasks.

13.3.2 Tasks, microtasks, and the event loop

When a website loads in a browser, the synchronous JavaScript code executes immediately. The application reaches a point in which there's no JavaScript code left to execute, something that's necessary for the website to be responsive to user interactions—the perks of single-threaded JavaScript. At this point we say the *execution stack* is empty; the JavaScript runtime doesn't have any code pending to execute.

Execution stack

The execution stack (or *call stack*) is a [stack data structure](#) that stores the functions that are currently being executed by the JavaScript runtime. When a function is called, it's added to the top of the stack, and when it returns, it's popped from the top of the stack.

The rest of the JavaScript code that runs is triggered by user interactions, such as clicking a button or typing in an input field, or scheduled tasks, such as those scheduled using `setTimeout()` or `setInterval()`. If nothing is scheduled to run, or the user doesn't interact with the application, the JavaScript runtime's execution stack is empty.

When the execution stack is empty the *event loop* reads from two queues (the task and microtask queues) and executes the callbacks that are in them. An example of tasks are the callbacks of the event listeners. When an event is triggered, the callback (the function registered as its event handler) is added to the task queue. Microtasks are mainly the callbacks of promises.

Event loop

The [event loop](#) is a loop that runs continuously in the browser's JavaScript runtime and decides what code to execute next. The event loop is responsible for reading from the task and microtask queues and pushing their callbacks to the execution stack.

Why does the browser need two queues and not just one? What's relevant for us now is that, the event loop handles the tasks and microtasks differently, and understanding these differences is key to understanding how to schedule asynchronous hooks to run after the application is fully mounted.

Let's open the hood of the event loop and see how it reads from the task and microtask queues and pushes their callbacks to the execution stack.

13.3.3 The event loop cycle

Understanding how the event loop works is key to understanding how to schedule asynchronous hooks to run after the application is fully mounted. By learning the steps of an event loop cycle, when it reads from the task and microtask queues, and how it pushes the callbacks to the execution stack, you'll be able to reason about what's the best way to schedule the execution of the `onMounted()` and `onUnmounted()` hooks.

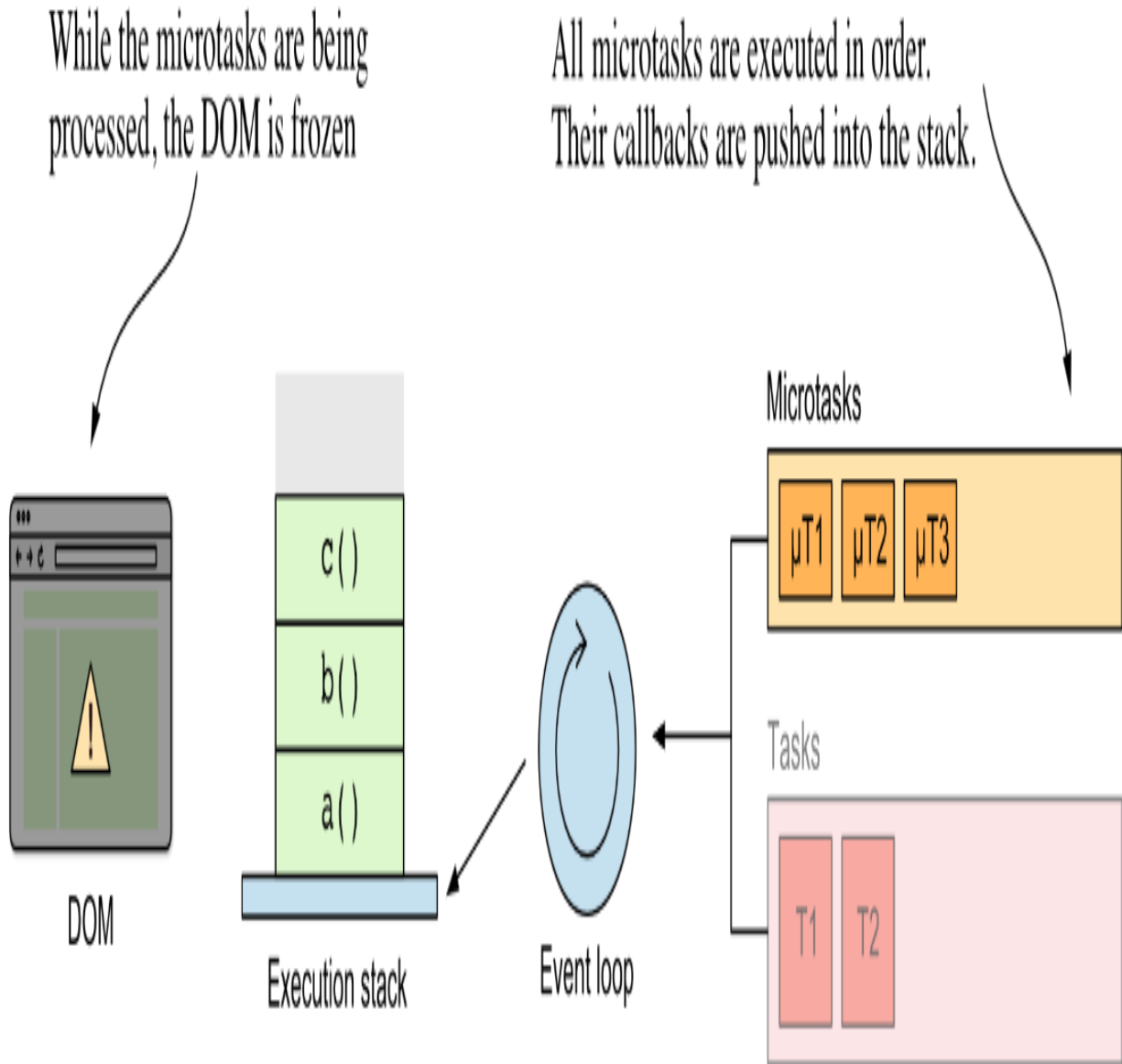
Note

You can read the execution steps with more detail in the HTML specification: <https://html.spec.whatwg.org/multipage/webappapis.html#event-loop-processing-model>. There is also a great article explaining the event loop in <https://javascript.info/event-loop#event-loop>.

The following are the simplified steps of an event loop cycle.

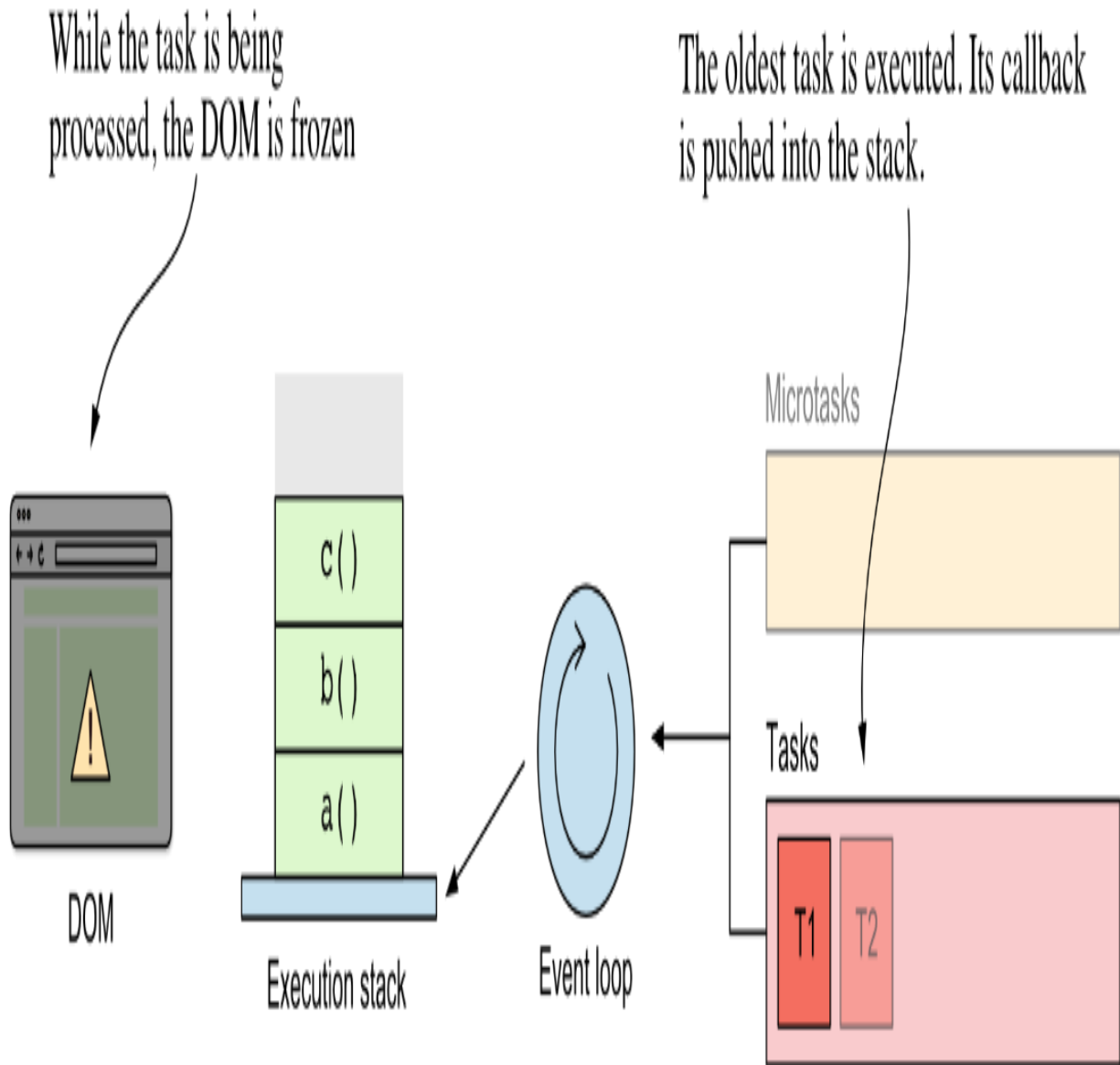
Step 1. Execute all the microtasks in the microtask queue in order, even those that were added while executing other microtasks in this same cycle (figure [13.5](#)). While executing the microtasks, JavaScript's thread is blocked, that is, the browser can't respond to user interactions. (Note that this isn't any different from what happens when you execute synchronous JavaScript code.)

Figure 13.5 Execute all the microtasks in the microtask queue



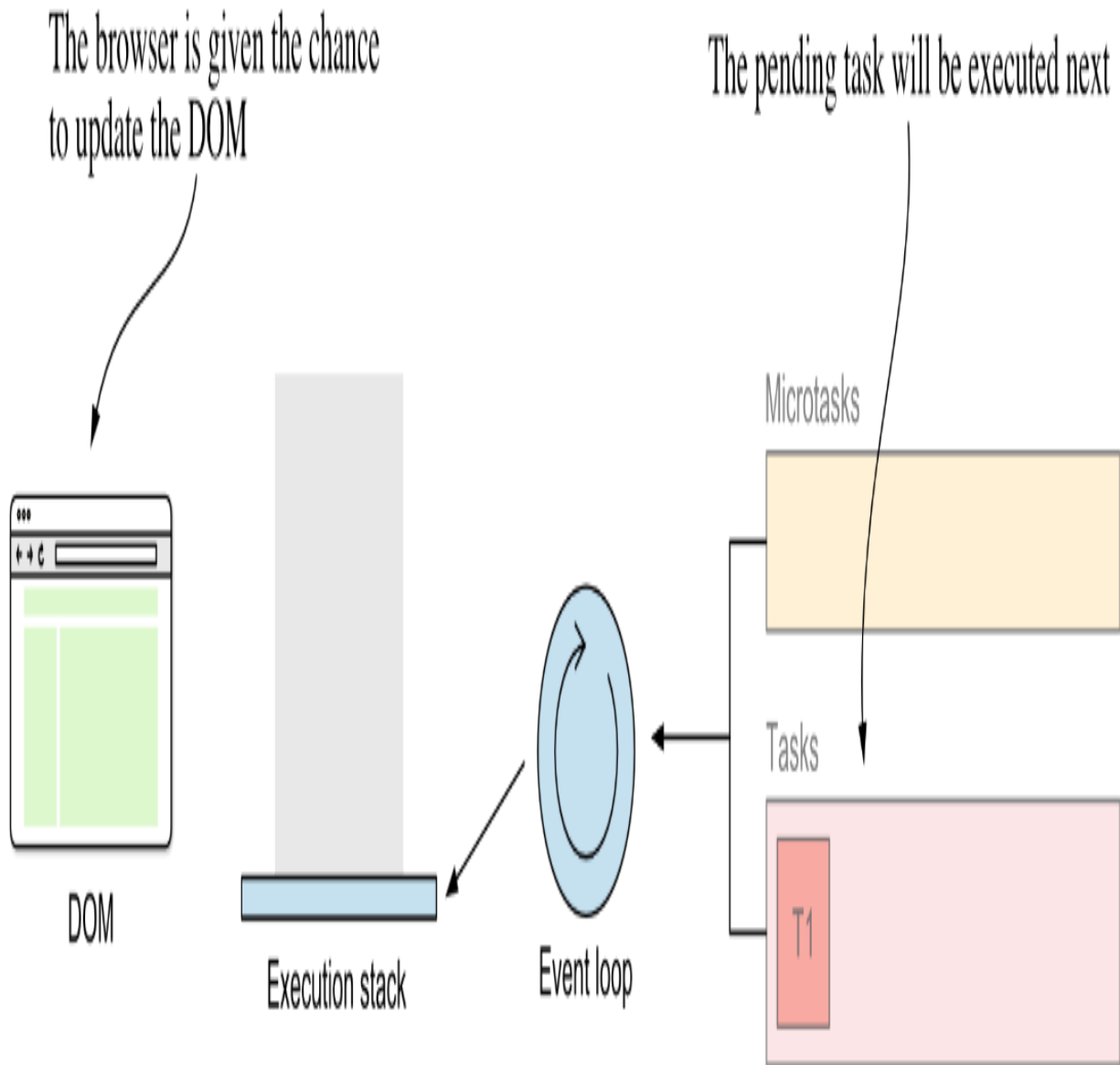
Step 2. Execute the oldest task in the task queue. Only those tasks that were added to the task queue before the start of this cycle are considered (figure [13.6](#)). While executing the task, JavaScript's thread is still blocked.

Figure 13.6 Execute the oldest task in the task queue



Step 3. Give the browser the opportunity to render the DOM if there are pending changes (figure [13.7](#)).

Figure 13.7 Give the browser the opportunity to render the DOM



Step 4. Go back to step 1 until there are no more tasks in the task queue.

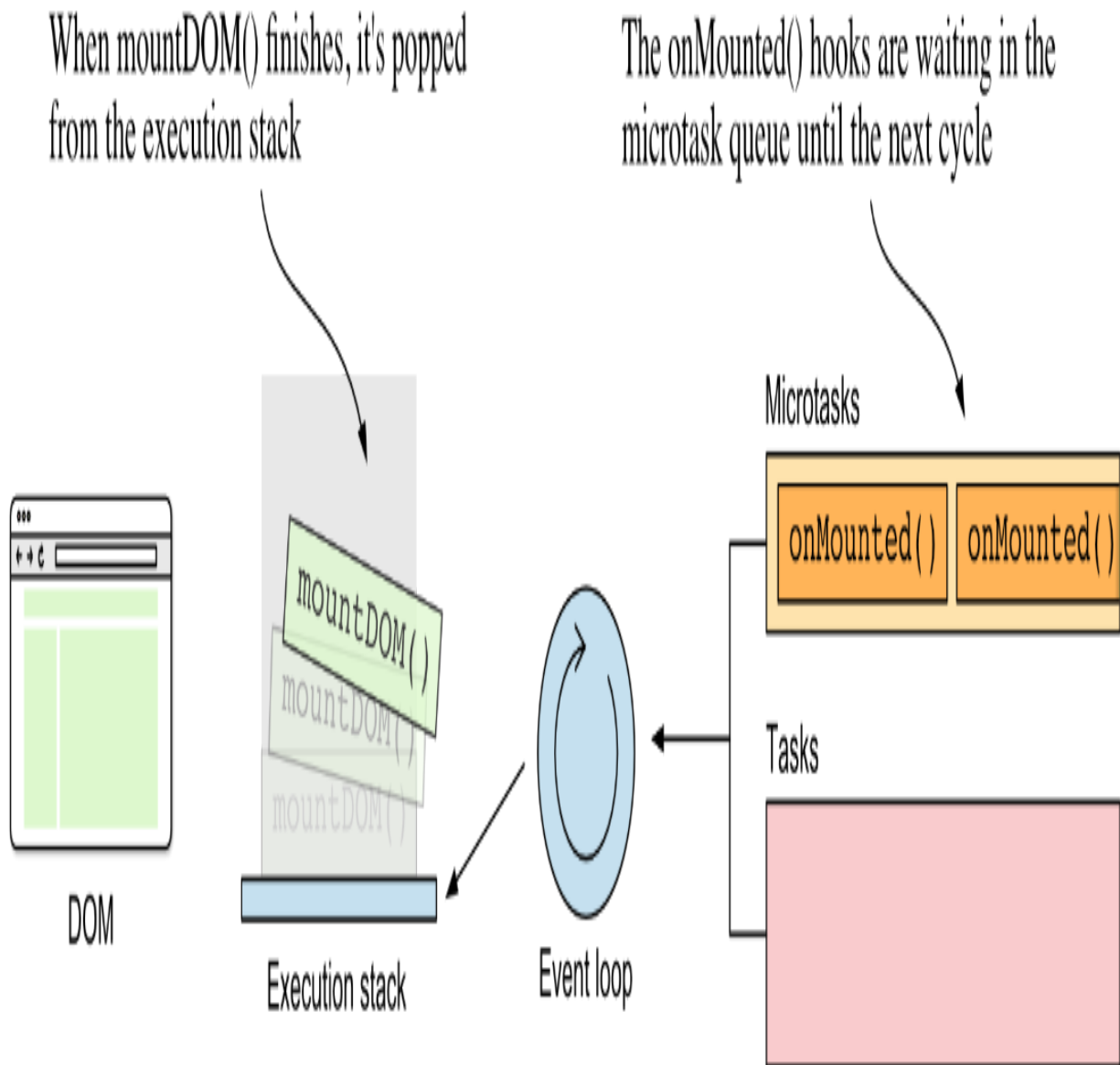
With this knowledge, let's think about how can you design a scheduler that schedules the `onMounted()` hooks to run after the application is fully mounted.

13.3.4 The fundamentals of the scheduler

The `mountDOM()` function executes synchronously when the application is loaded into the browser. When this code finishes executing, and assuming

there is no more synchronous JavaScript code, the execution stack becomes empty. It's at this point that you want the `onMounted()` hooks to execute in the same order they are scheduled, and for this, the microtask queue is perfect (figure [13.8](#)).

Figure 13.8 The `onMounted()` hooks are executed in the same order they were scheduled



The execution order is guaranteed for microtasks, and if those enqueue more microtasks, all of them will be executed in the same cycle. Let's think about how to implement a scheduler based on this idea.

You want to collect all the `onMounted()` lifecycle hooks in a queue of pending jobs:

```
const jobs = []

function enqueueJob(job) {
  jobs.push(job)
}
```

Another function processes the jobs, and it's as simple as dequeuing the jobs and executing them in order:

```
function processJobs() {
  while (jobs.length > 0) {
    const job = jobs.shift()
    job()
  }
}
```

To schedule the `processJobs()` function into the microtask queue so that it runs when the execution stack is empty, you can use the [`queueMicrotask\(\)` function](#), present in the global `Window` object. You have to be careful to not schedule the `processJobs()` function multiple times. For this, you can have a `isScheduled` boolean variable that keeps track of whether the function is already scheduled or not:

```
let isScheduled = false

function scheduleUpdate() {
  if (isScheduled) return

  isScheduled = true
  queueMicrotask(processJobs)
}
```

You want to call the `scheduleUpdate()` function every time a job is enqueued:

```
function enqueueJob(job) {
  jobs.push(job)
  scheduleUpdate()
}
```

Once the first job is enqueued, the `processJobs()` function is scheduled to run. All the jobs that are subsequently added before `processJobs()` is executed don't schedule the function again, because the `isScheduled` flag is set to `true`.

And with this, you have all the fundamentals of the scheduler. It's time to implement it.

13.3.5 Implementing a scheduler

You want to implement the scheduling logic in a separate file. Create a new file called *scheduler.js* inside the *src/* folder. In it, write the code in listing [13.3](#).

Listing 13.3 The scheduler (scheduler.js)

```
let isScheduled = false #1
const jobs = [] #2

export function enqueueJob(job) {
  jobs.push(job) #3
  scheduleUpdate() #4
}

function scheduleUpdate() {
  if (isScheduled) return

  isScheduled = true
  queueMicrotask(processJobs) #5
}

function processJobs() {
  while (jobs.length > 0) { #6
    const job = jobs.shift()
    job()
  }

  isScheduled = false #7
}
```

The jobs might be asynchronous, as it's the case for the `onMounted()` and `onUnmounted()` hooks. Let's do one more thing to avoid having [floating promises](#) in the application: wrap the result of calling each `job()` function

inside a `Promise`, and add callbacks to handle the result or rejection of the promise. When the promise resolves, there is nothing else you want to do, so you can keep that callback empty. When the promise rejects, you want to log the error to the console so that the developer knows that something went wrong.

Write the code in bold font in listing [13.4](#) in the *scheduler.js* file.

Listing 13.4 Handling promises in the scheduler (scheduler.js)

```
function processJobs() {
  while (jobs.length > 0) {
    const job = jobs.shift()
    const result = job() #1

    Promise.resolve(result).then( #2
      () => { #3
        // Job completed successfully
      },
      (error) => {
        console.error(`[scheduler]: ${error}`) #4
      }
    )
  }

  isScheduled = false
}
```

Now that you have a scheduler, let's use it to schedule the execution of the lifecycle hooks.

Exercise 13.1

What is the order of the `console.log()` resulting from running the following code:

```
console.log('Start')
setTimeout(() => console.log('Timeout'))
queueMicrotask(() => console.log('Microtask 1'))
enqueueJob(() => console.log('Job'))
queueMicrotask(() => console.log('Microtask 2'))
console.log('End')
```

Try to reason about it before running the code. Draw a diagram with the task and microtask queues if that helps.

Find the solution at <http://mng.bz/ddmX>.

13.3.6 Scheduling the lifecycle hooks execution

The `mountDOM()` function is the right place to schedule the execution of the `onMounted()` hooks. Right after a component virtual DOM node is created, you want to enqueue the `onMounted()` hook in the scheduler.

Open the *mount-dom.js* file and add the code in bold font in listing [13.5](#).

Listing 13.5 Scheduling the `onMounted()` method of a component (mount-dom.js)

```
import { setAttributes } from './attributes'
import { addEventListeners } from './events'
import { DOM_TYPES } from './h'
import { enqueueJob } from './scheduler' #1
import { extractPropsAndEvents } from './utils/props'

export function mountDOM(vdom, parentEl, index, hostComponent
= null) {
  switch (vdom.type) {
    // --snip-- //

    case DOM_TYPES.COMPONENT: {
      createComponentNode(vdom, parentEl, index,
hostComponent)
      enqueueJob(() => vdom.component.onMounted()) #2
      break
    }

    default: {
      throw new Error(`Can't mount DOM of type:
${vdom.type}`)
    }
  }
}

// --snip-- //
```

Let's now do the same with the `onUnmounted()` hook. This time, the `destroyDOM()` function is the right place to schedule the execution of the `onUnmounted()` hooks, right after the component is removed from the DOM.

Open the *destroy-dom.js* file and add the code in bold font in listing [13.6](#).

Listing 13.6 Scheduling the `onUnmounted()` method of a component (destroy-dom.js)

```
import { removeEventListeners } from './events'
import { DOM_TYPES } from './h'
import { enqueueJob } from './scheduler' #1
import { assert } from './utils/assert'

export function destroyDOM(vdom) {
  const { type } = vdom

  switch (type) {
    // --snip--

    case DOM_TYPES.COMPONENT: {
      vdom.component.unmount()
      enqueueJob(() => vdom.component.onUnmounted()) #2
      break
    }

    default: {
      throw new Error(`Can't destroy DOM of type: ${type}`)
    }
  }

  delete vdom.el
}
```

That's it! The `onMounted()` and `onUnmounted()` hooks are scheduled to run after the application is fully mounted and unmounted (respectively), and they are executed in the same order they were scheduled. It's nevertheless a good idea to debug the execution of an application using your framework to see how the scheduler works and how the hooks are executed at the right time. The next exercise is about that.

Exercise 13.2

Create a component that receives a name as a prop and renders that name. The component should also have an `onMounted()` hook that logs to the console that the component was mounted, including the name prop. Your component should look something like this:

```
const NameComponent = defineComponent({
  onMounted() {
    console.log(`Component mounted with name:
${this.props.name}`)
  },
  render() {
    return h('p', {}, [this.props.name])
  }
})
```

Now, create an App top level component including five `NameComponent` components, each with a different name. Mount the application into the DOM, like so:

```
const App = defineComponent({
  render() {
    return hFragment([
      h(NameComponent, { name: 'Alice' }),
      h(NameComponent, { name: 'Bob' }),
      h(NameComponent, { name: 'Charlie' }),
      h(NameComponent, { name: 'Diana' }),
      h(NameComponent, { name: 'Eve' })
    ])
  }
})

createApp(App).mount(document.body)
```

Instrument the application in the browser, adding a "log break point" in the `mountDOM()` call, inside the application's instance `mount()` method. Set another log breakpoint in the line immediately after the `mountDOM()` call, to know when the application has finished mounting.

What is the order in which the operations are logged to the console? Debug the application, setting breakpoints when the `onMounted()` hooks are scheduled and when they are executed. Did the scheduler do what you expected (enqueue the hooks and execute them in order just after `mountDOM()` finished executing)?

Find the solution at <http://mng.bz/rjmZ>.

Let's publish the new version of the framework.

13.4 Publishing version 4 of the framework

In the *package.json* file, update the version number to 4.0.0:

```
"version": "3.0.0",  
"version": "4.0.0",
```

Remember to move your terminal to the root or the project and run `npm install` so that the package version number in the *package-lock.json* file is updated:

```
$ npm install
```

Finally, place your terminal inside the *packages/runtime* folder and run:

```
$ npm publish
```

Congratulations! — The new version of your framework is now published.

Exercise 13.3 — Challenge

Refactor the to-dos application so that it saves the to-dos in the browser's local storage. Load the to-dos from the local storage when the application is mounted, and save them to the local storage when they are added, removed, or updated.

Find the solution at <http://mng.bz/VRGO>.

13.5 Summary

- The *lifecycle* of a component is the sequence of events that it goes through from the moment it's created until it's removed from the DOM.

- A *lifecycle hook* is a function that is called at a specific moment in the component's lifecycle.
- The *mounted* lifecycle hook is scheduled to run after the component is mounted into the DOM.
- The *unmounted* lifecycle hook is scheduled to run after the component is removed from the DOM.
- The scheduler is in charge of running jobs in the same order they were scheduled, and it's based on the microtask queue.
- The event loop is a loop that runs continuously in the browser's JavaScript runtime and decides what code to execute next. In every cycle, it first executes all pending microtasks, then the oldest task in the task queue, and finally it gives the browser the opportunity to render the DOM.

14 Testing asynchronous components

This chapter covers

- Testing synchronous components
- Testing components with asynchronous behavior
- Implementing the `nextTick()` function

Testing components whose behavior is purely synchronous is straightforward. But components can have asynchronous hooks or event handlers, so testing them becomes a bit more complicated (and interesting) in these cases.

The main question the tester must face is, how do you know when all the asynchronous jobs have finished executing and the component has re-rendered? You'll implement a `nextTick()` function that returns a `Promise` that resolves when all the pending jobs in the scheduler have finished executing. This way, by awaiting this `Promise`, you can be sure that the component has re-rendered and you can check what's in the DOM.

Important

You can find all the listings in this chapter in the [listings/ch14 directory](#) of the book's repository. The code in this chapter can be checked out from [the ch14 label](#):

```
$ git switch --detach ch14
```

Code catch-up

In the last chapter, you defined two new functions that can be passed as arguments to the `defineComponent()` function: `onMounted()` and `onUnmounted()`. You implemented two new methods in the `Component` class,

also called `onMounted()` and `onUnmounted()`, that wrap those two functions inside a `Promise` and bind them to the component instance.

You then implemented a simple scheduler to run the hooks asynchronously. The scheduler's API consists of just one function:

- `enqueueJob()` — queues a job to be executed asynchronously.

The scheduler internally uses two more functions:

- `scheduleUpdate()` — queues the `processJobs()` function to be executed as a microtask.
- `processJobs()` — runs all the jobs in the scheduler.

Last, you modified the `mountDOM()` and `destroyDOM()` functions to enqueue the `onMounted()` and `onUnmounted()` hooks, respectively, as jobs in the scheduler.

14.1 Testing components with asynchronous behavior — `nextTick()`

If you build an application with your own framework, you'll want to unit test it—as you normally do (because you do test your applications, right?). Before you introduced asynchronous hooks, testing components was straightforward. Let's look at an example.

Note

I'm using the [vite](#) testing library for the examples—the same I used in the repository of the project. Its API is very similar to [Jest](#), so if you're familiar with Jest you'll feel right at home.

Say you have a component called `Counter` consisting of a button that increments a counter when clicked. If you render the component into the DOM, you get something like this:

```
<span data-qa="counter">0</span>
<button data-qa="increment">Increment</button>
```

After the button is clicked, the counter is incremented, and the DOM is updated:

```
<span data-qa="counter">1</span>
<button data-qa="increment">Increment</button>
```

You can test this simple component as follows (note that I'm using `data-qa` attributes to select the elements in the DOM):

```
import { test, expect, beforeEach, afterEach } from 'vitest'
import { createApp } from 'fe-fwk'
import { Counter } from './counter'

let app = null

beforeEach(() => { #1
  app = createApp(Counter)
  app.mount(document.body)
})

afterEach(() => { #2
  app.unmount()
})

test('the counter starts at 0', () => {
  const counter = document.querySelector('[data-qa="counter"]') #3
  expect(counter.textContent).toBe('0') #4
})

test('the counter increments when the button is clicked', ()
=> {
  const button = document.querySelector('[data-qa="increment"]')
  const counter = document.querySelector('[data-qa="counter"]')

  button.click() #5

  expect(counter.textContent).toBe('1') #6
})
```

But what if the component has an `onMounted()` hook that fetches data from a server asynchronously? Then you can't assume that right after mounting the

application the components have already updated their state or re-rendered their view.

In this case, your framework needs to provide the developer with a function, typically called `nextTick()`, that returns a `Promise` that resolves when all the pending jobs in the scheduler have finished executing. This function would be used to wait for all components to run their `onMounted()` hooks, as follows:

```
test('the component loads data from a server', async () => {
  const app = createApp(MyComponent)
  app.mount(document.body)

  // At this point the component has gone through its first
  render, but
  // the onMounted() hooks hasn't started executing.

  await nextTick()

  // At this point MyComponent's onMounted() hook has
  finished executing.
  // You can safely check what is rendered in the DOM.
})
```

Let's look at a realistic example with a loading state so that this is clearer. For the example, we'll assume that the `nextTick()` function already exists in your framework, but it won't be until the next section that you'll implement it.

14.1.1 Testing a component with an asynchronous `onMounted()` hook

Let's say you have a component called `TodosList` that loads a list of to-dos from a server when it's mounted. While the data is being loaded, the component renders a loading indicator. This is how you might implement such a component:

```
export const TodosList = defineComponent({
  state() {
    return {
      isLoading: true,
      todos: [],
    }
  }
})
```

```

    },

    async onMounted() {
      const todos = await
fetch('https://api.example.com/todos')
      this.setState({ isLoading: false, todos })
    },

    render() {
      const { isLoading, todos } = this.state

      if (isLoading) {
        return h('p', {}, ['Loading...'])
      }

      return h('ul', {}, todos.map((todo) => h('li', {},
[todo])))
    }
  })
}

```

The first thing you'd naturally do is mock the `fetch()` function so that it returns a promise that resolves with the data you want to test:

```

import { vi } from 'vitest'

// Mock the fetch API
global.fetch = vi.fn().mockResolvedValue({
  json: vi.fn().mockResolvedValue([
    'Feed the cat',
    'Mow the lawn',
  ]),
})

```

That had a little trick in it because `fetch()` returns a promise that resolves with a response object that has a `json()` method that returns a promise that resolves with the data. That's a mouthful, but it's not that complicated. In any case, you've already mocked the call to the server.

To test that the loading indicator is rendered while the data is being fetched, you can do the following:

```

test('Shows the loading indicator while the todos are
fetched', () => {
  // If nextTick() isn't awaited for, the onMounted() hooks
haven't run yet

```

```
// so the loading should be displayed.  
  
expect(document.body.innerHTML).toContain('Loading...')  
}))
```

It's that easy! If you don't await for the `nextTick()` function, then the `onMounted()` hook hasn't run yet, so the loading indicator is displayed. But this means that at some point after the test has finished, the DOM will be updated and the loading indicator will disappear. You want to clean up the DOM after each test, so that the next test starts with a clean DOM.

The only thing you need to do is await for the `nextTick()` function before unmounting the application, as shown in bold font in the following listing:

```
afterEach(async () => {  
  await nextTick() #1  
  app.unmount()  
})
```

Note that, if there were `onUnmounted()` asynchronous hooks, you'd want to also await for the `nextTick()` function after unmounting the application:

```
afterEach(async () => {  
  await nextTick()  
  app.unmount()  
  await nextTick()  
})
```

And to test that the list of to-dos is rendered after the data is fetched, you can do the following:

```
test('Shows the list of todos after the data is fetched',  
  async () => {  
    // The nextTick() function is awaited for, so the  
    onMounted() hooks  
    // finished running and the list of todos is displayed.  
  
    await nextTick() #1  
  
    expect(document.body.innerHTML).toContain('Feed the cat')  
    expect(document.body.innerHTML).toContain('Mow the lawn')  
  })
```

It'd be much better if you added `data-qa` attributes to the elements in the DOM so that you can select them easily, then check their text content, but this was a simplified example to illustrate how to test components with asynchronous hooks.

The same `nextTick()` function would work to test components that have asynchronous event handlers, such as a button that loads more data from a server when clicked. So, you might be wondering how you implement the `nextTick()` function. I'm glad you asked! — Let's implement it. Be sure you have a clear understanding of the event loop, the task and microtask queues, because it is key to understanding what comes next. Buckle up!

14.1.2 The fundamentals behind the `nextTick()` function

The `nextTick()` function might appear magical, but it's simply based on a solid understanding of the event loop's mechanics. There is a similar function implemented in some of the big frameworks, and they're widely used when testing components with asynchronous behavior, so I'm positive that implementing your own will help you understand how it works, and when to use it.

The `nextTick()` function in some libraries

The `nextTick()` function defined in Vue can be used both in production code and in tests. It's usefulness resides in the fact that the code that you run after awaiting for the `nextTick()` function is executed after the component has re-rendered, when you know the DOM has been updated. This function is documented at <https://vuejs.org/api/general.html#nexttick>. You can also check how it's implemented in the *runtime-core* package, inside the *scheduler.ts* file: <http://mng.bz/7va4>.

Node JS also implements a `process.nextTick()` function, used to schedule a callback to be executed on the next iteration of the event loop. Here's the documentation for it: <https://nodejs.org/en/learn/asynchronous-work/understanding-processnexttick>. I recommend you give it a read, because it's short and very well explained.

Svelte defines a `tick()` function that returns a `Promise` that resolves when the component has re-rendered. It's usage is very similar to the `nextTick()` function in Vue. This function is documented at <https://svelte.dev/docs/svelte#tick>. You can check it's implementation in the `runtime.js` file, inside the `internal/client/` folder: <http://mng.bz/mjP0>.

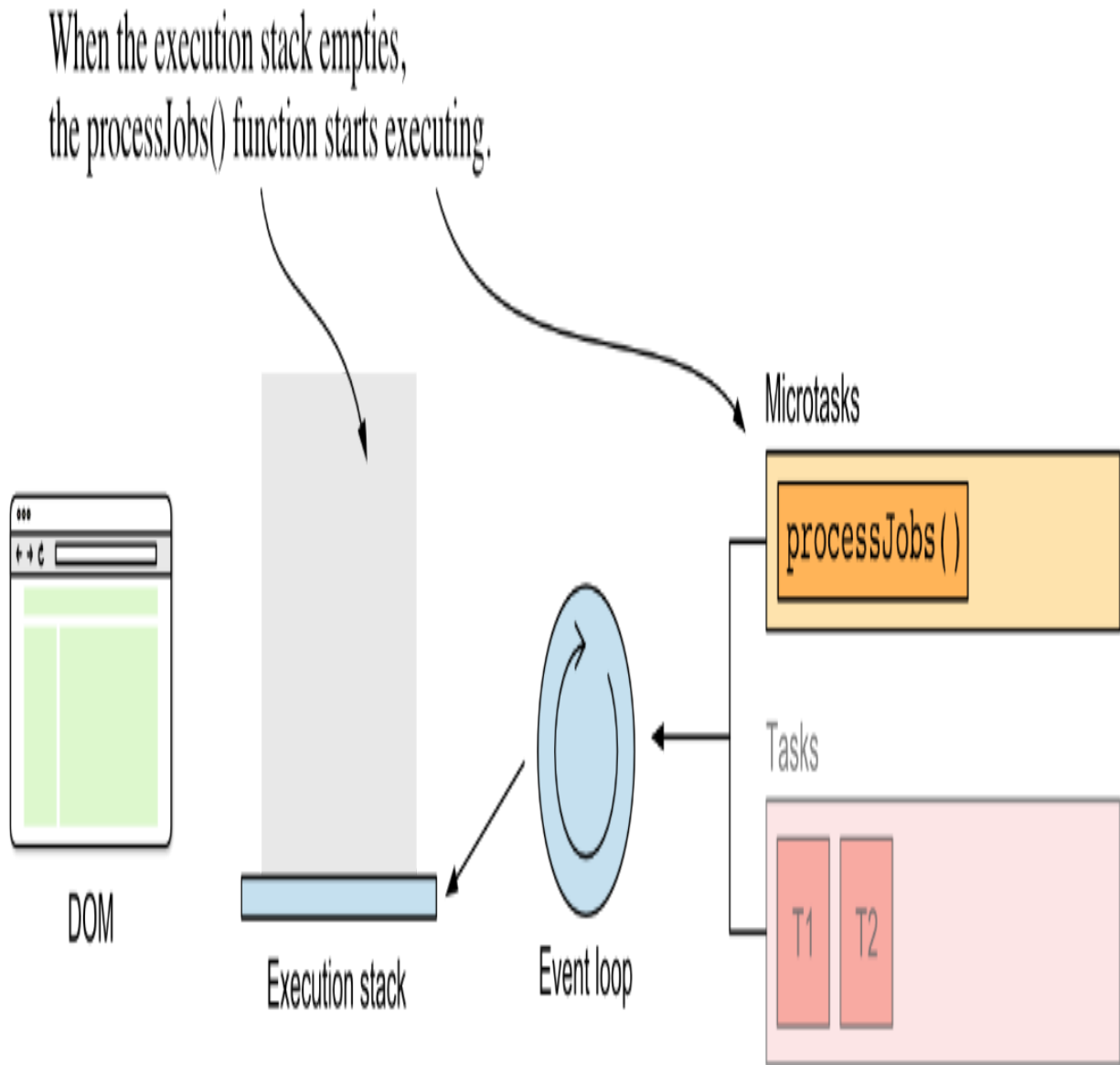
The `nextTick()` function returns a `Promise` that resolves when all the pending jobs in the scheduler have finished executing. So the question here is, how do you know when all the pending jobs in the scheduler have finished executing?

First, you want to call the `scheduleUpdate()` function to make sure that when the `nextTick()` function is called, the `processJobs()` function is scheduled to run:

```
export function nextTick() {  
  scheduleUpdate()  
  // ...  
}
```

If you remember, when the execution stack empties, the event loop first executes all the microtasks, then it executes the oldest task in the task queue, lets the DOM render, and then it starts over. You've just queued the `scheduleUpdate()` function to run as a microtask, so you know that as soon as the execution stack is empty, the `processJobs()` function will be executed (figure [14.1](#)).

Figure 14.1 The `processJobs()` function is scheduled to run as a microtask

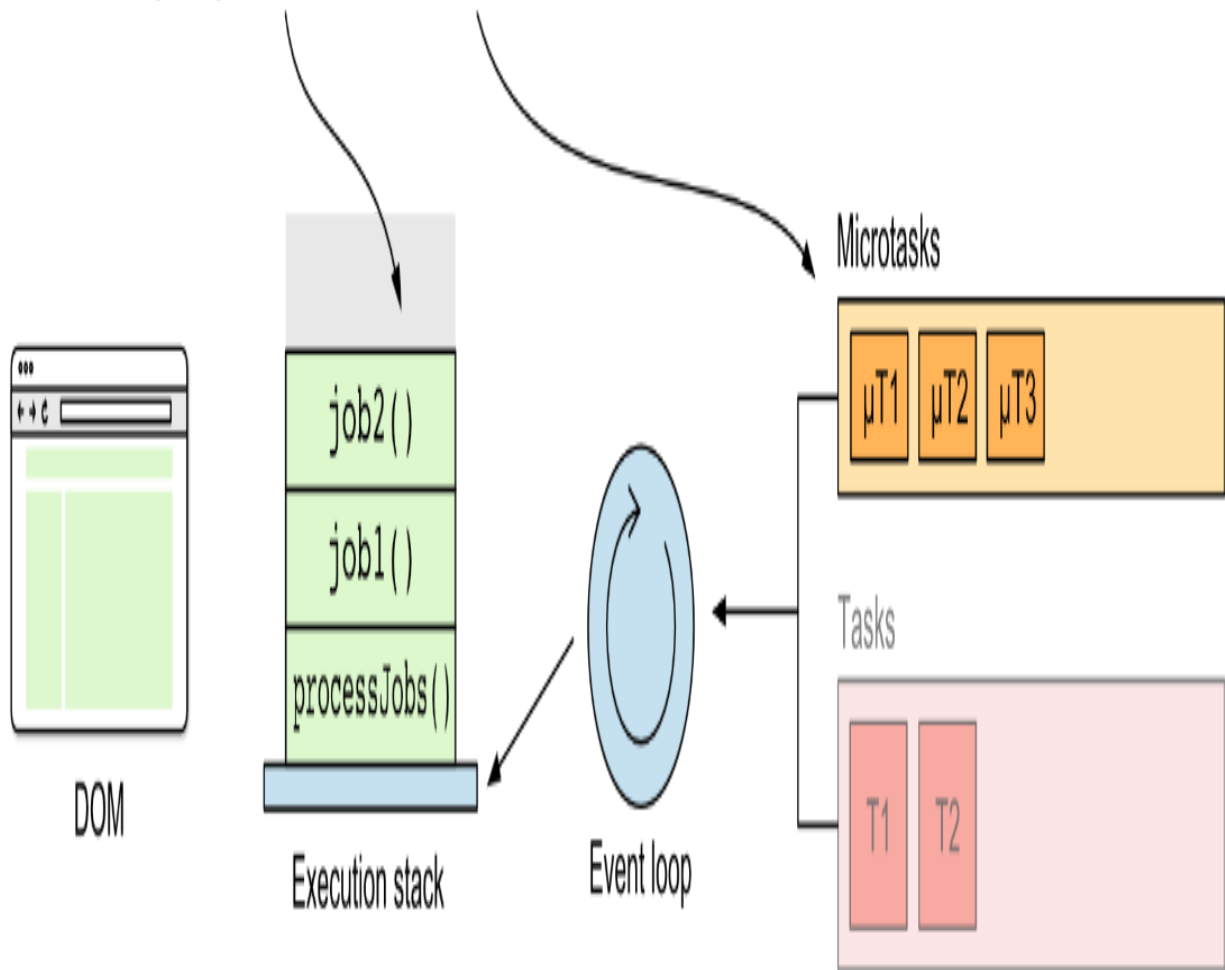


All the jobs in the scheduler are asynchronous, because even if they weren't originally, you've wrapped them inside a `Promise` and set up a `then()` callback to handle the result or rejection of the promise.

This means that the `processJobs()` function will queue more microtasks as it runs jobs (figure [14.2](#)).

Figure 14.2 The `processJobs()` function queues more microtasks

The processJobs() function schedules the microtasks created by the jobs it executes



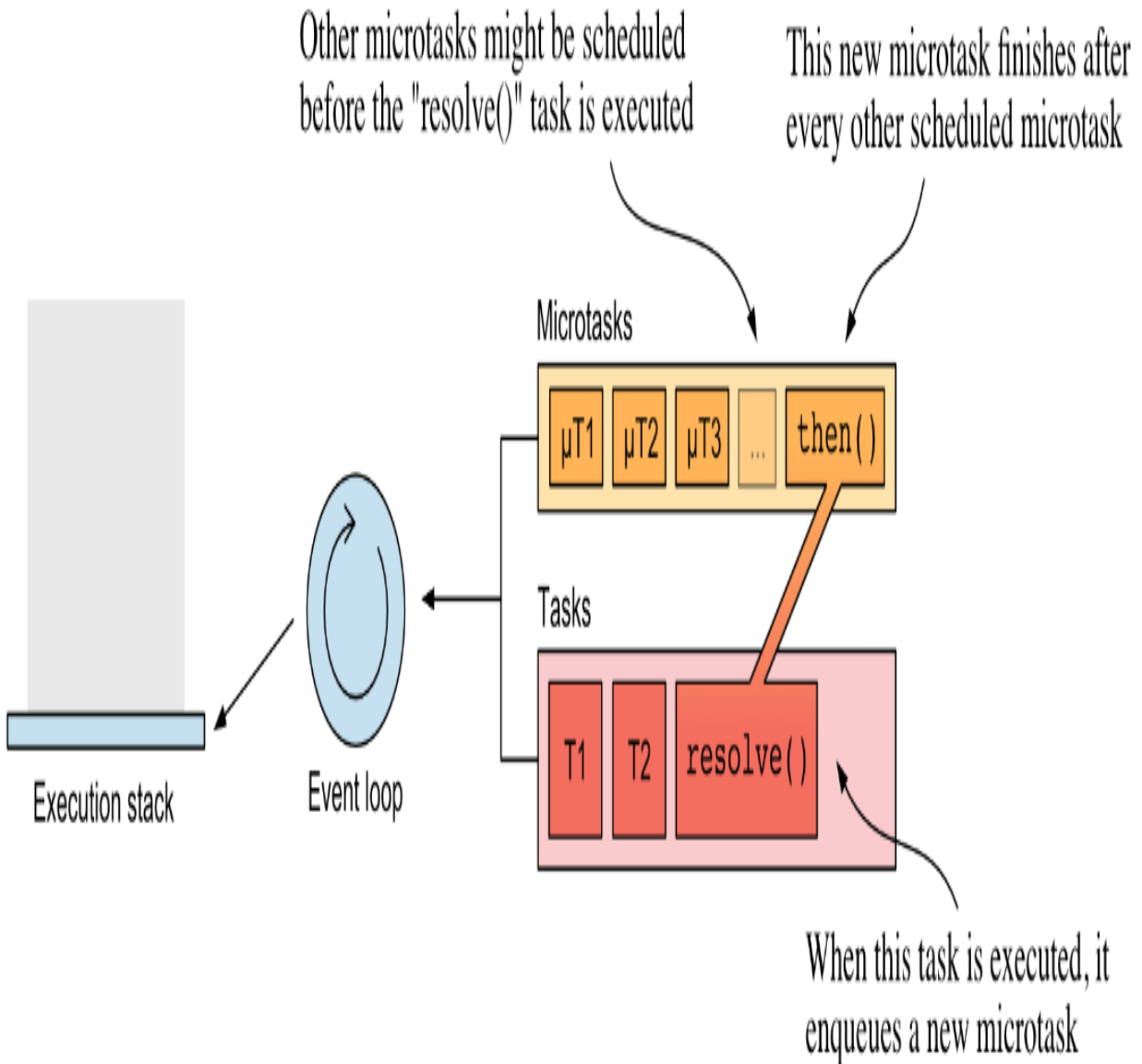
The event loop won't take the next task until all microtasks have been executed, including the microtasks that are queued by the `processJobs()` function. So, to make sure you resolve only the `Promise` returned by the `nextTick()` function when all the pending jobs in the scheduler finish executing, you need to schedule a task whose callback resolves the `Promise` returned by the `nextTick()` function. You can use `setTimeout()` to schedule a new task:

```
new Promise((resolve) => setTimeout(resolve))
```

By scheduling the created `Promise` in the task callback, you're forcing the event loop to go through (at least) another cycle before resolving the `Promise`.

This is because, the `resolve()` function inside a `Promise` is scheduled as a microtask. So if the microtask is queued only when a task is executed, the event loop needs to process all pending microtasks, process the task that resolves the `Promise`, and then process all the pending microtasks until that `resolve()` is executed. It's only then that awaiting for the `Promise` returned by the `nextTick()` function resolves (figure [14.3](#)).

Figure 14.3 The `Promise` returned by the `nextTick()` function resolves when all the pending jobs in the scheduler have finished executing



This process of scheduling a microtask via a task is commonly referred to as *flushing promises*. Many frontend and testing frameworks include it, mostly for testing purposes, so that you can easily write your unit tests without having to worry about the asynchronous behavior of the components.

I know this might be hard to wrap your head around, but it's key to understand how the event loop works to understand how the `nextTick()` function works. Take your time to process this information, and if you need to, read it again, draw the diagrams in this chapter yourself and experiment with the browser's console. In the exercise below, I'll ask you to do just that.

14.1.3 Implementing the nextTick() function

You should have a good understanding of how the `nextTick()` function works, so let's implement it. Inside the *scheduler.js* file, add the code in listing [14.1](#).

Listing 14.1 `nextTick()` and `flushPromises()` (*scheduler.js*)

```
export function nextTick() {
  scheduleUpdate()
  return flushPromises()
}

function flushPromises() {
  return new Promise((resolve) => setTimeout(resolve))
}
```

Now, before publishing the framework's next version, version 4.1 including the `nextTick()` function, I want you to do a little exercise. In it I'll challenge you to guess what the order of some `console.log()` calls will be—a classic JavaScript interview question. It'll help you understand the principle behind the `flushPromises()` function.

Exercise 14.1

Open the browser's console and write this slightly modified version of the `flushPromises()` function:

```
function flushPromises() {
  return new Promise((resolve) => {
    console.log('setTimeout() with resolve() as
callback...')
    setTimeout(() => {
      console.log('About to resolve the promise...')
      resolve(42)
    })
  })
}
```

Note how there are two console logs:

- One inside the `Promise` body, before the `setTimeout()` function is called, and
- One inside the `setTimeout()` callback, right before the `resolve()` function is called.

Then, write this `doWork()` function that schedules both microtasks and tasks, then awaits for the `Promise` returned by the `flushPromises()` function:

```
async function doWork() {
  queueMicrotask(() => console.log('Microtask 1'))
  setTimeout(() => console.log('Task 1'))
  queueMicrotask(() => console.log('Microtask 2'))
  setTimeout(() => console.log('Task 2'))

  const p = flushPromises()

  queueMicrotask(() => {
    console.log('Microtask 44')
    queueMicrotask(() => console.log('Microtask 45'))
  })
  setTimeout(() => console.log('Task 22'))

  const value = await p
  console.log('Result of flush:', value)
}
```

Can you guess the order in which all these `console.log()` calls will be executed when you execute the `doWork()` function? Draw a diagram with the task and microtask queues to help you figure it out. Then, run the `doWork()` function and compare the result with your expectations.

Find the solution at <http://mng.bz/xjGY>.

Let's now publish the new version of the framework.

14.2 Publishing version 4.1 of the framework

First, you want to include the `nextTick()` function as part of the public API of the framework. This allows developers to use the function inside their unit tests, as you saw in the previous section.

In the *index.js* file, add the import statement in bold font:

```
export { createApp } from './app.js'
export { defineComponent } from './component.js'
export { DOM_TYPES, h, hFragment, hString } from './h.js'
export { nextTick } from './scheduler.js'
```

Then, in the *package.json* file, update the version number to 4.1.0:

```
"version": "4.0.0",
"version": "4.1.0",
```

Remember to move your terminal to the root of the project and run `npm install` so that the package version number in the *package-lock.json* file is updated:

```
$ npm install
```

Finally, place your terminal inside the *packages/runtime* folder and run:

```
$ npm publish
```

The new version of your framework, which includes the `nextTick()` function, is now published.

14.3 Summary

- To test components with asynchronous hooks or event handlers, you need to await for the `nextTick()` function before checking what's in the DOM.
- The `nextTick()` function returns a `Promise` that resolves when all the pending jobs in the scheduler have finished executing.
- To implement the `nextTick()` function, you need to schedule a task whose callback resolves the `Promise` returned by the `nextTick()` function.

14.4 Where to go from here

Congratulations on reaching the final section of the book! By now, you've achieved an impressive learning milestone. Your grasp of how frontend frameworks work surpasses that of 99% of developers in this field. What's even more remarkable is that you've not only delved into the theory but have also mastered the practical aspect of frontends by building your own framework.

You might be eager to continue enhancing your skills. Consider adding a router to enable seamless transitions between different pages and incorporating a template compiler into your build process. Perhaps you're even contemplating the creation of a Chrome/Firefox extension to debug applications written using your framework. These are common features found in most frontend frameworks, and understanding how to implement them can further elevate your expertise. Originally, I intended to cover these topics in the book. Unfortunately, due to the book's length constraints, they had to be omitted.

I nevertheless wanted to cover this material, so I had two options: write a second part to this book, or publish the material online. The second option seemed like a better idea, and so I decided to publish some extra chapters in the Wiki section of the book's code repository:

<https://github.com/angelsolaorbaiceta/fe-fwk-book/wiki>

My suggestion is that you continue your learning journey by reading the chapters in the Wiki. They follow a similar structure to the chapters in this book, so you should feel right at home. The material covered in the Wiki is more advanced, so be ready to embrace the challenge.

Once you've finished reading the advanced extra chapters in the Wiki, you'll have a nice frontend framework of your own, one that you can try out in your next project. Build your portfolio with it, or that side project you've been thinking about for a while. It's then, when used in a medium-sized project, that you'll see its shortcomings; maybe it isn't as convenient to use as the other big frameworks, or maybe it's missing some features. Reflect on what features you could add to fix those shortcomings, and then try to implement them. It'll be challenging (and fun). You'll want to open the source code of other

frameworks, read their documentation, and debug their code—this is how you really learn!

I've set the GitHub's repository discussions section as a forum for you to propose new features to the framework, ask questions, and get help from the community so that we all learn from each other. If you have a great idea for a feature, but you don't know how to implement it, or have several options but want help from the community to evaluate its trade-offs, this is the place to go. You can find it at:

<https://github.com/angelsolaorbaiceta/fe-fwk-book/discussions>

This goes without saying, but as with any other public forum where people from all levels of experience participate and ask for directions, please be respectful and kind to others. I know you are a kind person with a big heart and willingness to help, so I don't think you need to be reminded of this point. Also, value the time of others, and don't expect them to do your work for you. Take the time to do your research and pose your questions in a way that makes it easy for others to help you. You can read more about this in the [How to ask](#) section of Stack Overflow.

Then, after having completed the book, the advanced topics in the Wiki, and having built a project with your framework, you can go a step further and try the following:

- Innovate. Do you have an idea for a new feature that would make your framework stand out from the rest? What's stopping you from implementing it?
- Collaborate in the development of a well-established framework such as Vue, React, Angular, or Svelte. These projects are open source, and they're always looking for new contributors. You'll first need to get acquainted with the codebase, but you have many of the core concepts already covered.
- Improve the performance of your framework. Do you need inspiration? You should definitely check out about [Inferno JS](#). Start by measuring the performance of the different operations in your framework, for which you might need to look for some tooling. Then, find the bottlenecks and think about how you can get rid of them.

- Write another framework from scratch, making all the decisions yourself, weighing the trade-offs, and learning from your mistakes. You can always come back to the book for reference, but this time you'll be the one making the decisions.

I hope you've enjoyed reading this book as much as I've enjoyed writing it. This is a goodbye from me, but I hope to see you around in the discussions section of the repository, or in the issues section if you find a bug in the code. I wish you all the best in your learning journey, in your professional career, and above all, in your life.

Appendix A. Setting up the project

Before you write any code, you need to have an NPM project set up. In this appendix, I'll help you create and configure a project to write the code for your framework.

I understand you might not typically configure an NPM project from scratch with a bundler, a linter, and a test library yourself, as most frameworks come with a CLI tool (like, for example *create-react-app* or *angular-cli*) which does the scaffolding and generating the boilerplate code structure for you. So I'll give you two options to create your project:

1. Use the CLI tool I created for this book. With a single command, you can create a project where you can start writing your code right away.
2. Configure the project yourself, from scratch. It's more laborious, but you get to learn how to configure the project.

If you want to get started quickly with the book and can't wait to write your awesome frontend framework, I recommend you use the CLI tool. In this case, you need to read section [A.8](#). But if you are the kind of developer who enjoys configuring everything from scratch, you can follow the steps to configure the project yourself in section [A.9](#).

Before you start configuring your project, let's first cover some basics about where you can find the code for the book and the technologies you'll use. These sections are optional, but I recommend you read them before you start configuring your project. If you can't wait to start writing code, you can skip them and move on to sections [A.8](#) and [A.9](#). You can always come back and read the rest of the appendix later.

A.1 Where to find the source code

The source code for the book is publicly available at:

<https://github.com/angelsolaorbaiceta/fe-fwk-book>

I recommend you clone the repository or download the ZIP archive of the repository, to follow along with the book. It contains all the listings that appear in the book (in the *listings* folder), sorted by chapter. I'm assuming you're familiar with git and know how to clone a repository and checkout tags or branches. If the previous sentence sounded like nonsense to you, don't worry, you can still follow along with the book. Download the project as a ZIP archive by clicking the `<> Code` button in the repository and then clicking the *Download ZIP* button.

Warning

The code you'll find in the repository is the final version of the framework, the one you'll have written by the end of the book. If you want to check the code for each chapter, you need to checkout the corresponding Git tag as explained below. You can also refer to the *listings/* directory in the repository, which contains all the listings from the book, sorted by chapter.

Let's take a look at how you can find the code for each chapter.

A.1.1 Checking out the code for each chapter

I tagged the source code with the number of each chapter where a new version of the framework is available. The tag name follows the pattern *chX*, where *X* is the number of the chapter. For example, the first working version of the framework appears in chapter 6, so the corresponding code can be checked out with the following git command:

```
$ git checkout ch6
```

Then, in chapters 7 and 8, you implement the reconciliation algorithm, resulting in a new, more performant version of the framework. The corresponding code can be checked out as follows:

```
$ git checkout ch8
```

By checking out the code for each chapter, you can see how the framework evolves as we add new features. You can also compare your code by the end of each chapter with the code in the repository. Note that not all chapters

have a tag in the repository, only those that have a working version of the framework. I also wrote unit tests for the framework that I won't cover in the book, but you can look at them to find ideas on how to test your code.

Note

If you're not familiar with the concept of git tags, you can learn about them in <https://git-scm.com/book/en/v2/Git-Basics-Tagging>. Basic git knowledge is assumed for this book.

I recommend you avoid copy/pasting the code from the book and instead type it yourself. If you get stuck because your code doesn't seem to work, look at the code in the repository, try to figure out the differences, and then fix it yourself. If you write a unit test to reproduce the problem, much better. I know this is more cumbersome, but it's the best way to learn, and it's actually how you'd probably want to tackle a real-world bug in your code.

You can also refer to the *listings/* directory in the repository, which contains all the listings from the book, sorted by chapter.

You can find instructions on how to run and work with the code in the [README file](#) of the repository. This file contains up-to-date documentation on everything you need to know about the code. Make sure to quickly read it before you start working with the code. (When you get started with the code of an open-source repository, reading the README file is the first thing you should do.)

A.1.2 A note on the code

The emphasis of the code I wrote wasn't to be the most performant and safest code possible. There are a lot of places in the code that could use better error handling, or where I could have done things in a more performant way by applying some optimizations. But I didn't, because I put the emphasis on writing code that's easy to understand, that's clean and concise. Code that you read and immediately understand what it does.

I pruned the code of all the unnecessary details that would've made it harder to understand, and left just the essence of the concept which I want to teach. I've gone to great lengths to simplify the code as much as I could, and I hope you'll find it easy to follow and make sense of.

So, just bear in mind that the code you'll find in the repository and the book has the purpose of teaching efficiently, not being production-ready.

A.1.3 Reporting issues in the code

As many tests as I wrote and as many times as I reviewed the code, I'm sure there are still bugs in it. Frontend frameworks are complex beasts, and it's hard to get everything right. If you find a bug in the code, please report it in the repository by opening a new issue.

To open a new issue, go to the issues tab of the repository:

<https://github.com/angelsolaorbaiceta/fe-fwk-book/issues>

Then, click the *New issue* button. In the row that reads *Bug report*, click the *Get started* button. Fill in the form with the details of the bug you found, and click the *Submit new issue* button.

You'll need to give enough information for me to easily reproduce and fix the bug. I know it takes time to file a bug report with these many details, but it's considered good etiquette in the open source community. It shows respect from your side towards the maintainers of a project, who use their free time to create and maintain it for everyone to use free of charge. It's good that we get used to being respectful citizens of the open source community.

A.1.4 Fixing a bug yourself

If you find a bug in the code, and you know how to fix it, you can open a pull request with the fix. Even better than opening an issue is to offer a solution to it yourself—that's how open source projects work. You might also want to take a look at bugs that other people reported and try to fix them yourself.

This is a great exercise to learn how to contribute to open source projects, and I'll be forever grateful if you do so.

If you don't know what GitHub pull requests are, I recommend you read about them at <https://docs.github.com/en/pull-requests>. Pull requests are the way to contribute to open source projects on GitHub (and also how many software companies add changes to their codebase), so it's good to know how they work.

Let's discuss where you can find the answers for the exercises in the book.

A.2 Solutions to the exercises

In the chapters of the book I'll present you with exercises to test your understanding of the concepts I've explained. Some of them might feel straightforward, but there are some challenging ones. If you want to really absorb the concepts in the book, I recommend you do the exercises yourself. Only if you get stuck or when you think you've found the solution, go ahead and check the solutions.

I've included the answers to all the exercises in the wiki of the repository:

<https://github.com/angelsolaorbaiceta/fe-fwk-book/wiki>

In the wiki you'll find a page for the solutions to the exercises in the book by chapter:

<https://github.com/angelsolaorbaiceta/fe-fwk-book/wiki/Solution-to-exercises>

In the wiki you'll see that there are some extra chapters—apart from those in this book—covering advanced topics. These chapters couldn't make it into the book, because it would've made it too long.

A.3 Advanced topics

When you finish with this book, you'll have a pretty decent frontend framework. One that you wouldn't use in large production applications, but that'd work fantastic for small projects. The key is for you to have learned how frontend frameworks work, and to have had fun in the process. But you might feel like you want to continue learning and improving your framework.

I have written some extra chapters that cover advanced topics, which you can find in the repository's wiki:

<https://github.com/angelsolaorbaiceta/fe-fwk-book/wiki>

In these chapters you'll learn how to:

- Add Typescript types to your framework bundle.
- Add support for server-side rendering.
- Write the templates in HTML and compile them to render functions.
- Include external content inside your components (slots).
- Create a browser extension to inspect the components in your application.

And much more. I hope you enjoy them! But you have to learn the basics first, so make sure you finish this book with a solid understanding of the topics we'll cover.

Moving on. Before you create your project, let's take a look at the technologies you'll use and how the project will be structured.

A.4 Note on the technologies used

There are heated discussions in the frontend ecosystem about what are the best tools. You'll hear things like "you should use Bun, because it's much faster than Node JS," or "Webpack is old now, you should start using Vite for all your projects." Some people argue over which bundler is the best, or which linter and transpiler you should be using. Something that I find especially harmful are the "top 10 reasons why you should stop using X"-kind of blog posts. Apart from being obvious clickbait, they don't usually

help the frontend community, let alone junior developers that have a hard time understanding what set of tools they "should be using."

A cautionary tale on getting overly attached to tools

I once worked for a start-up that grew quickly but wasn't doing great. We had very few customers using our app—if any at all—and every time we added something in the code, we'd introduce new bugs. (Code quality wasn't a priority; speed was, and automated testing was nowhere to be found.) Surprisingly, we blamed it on the tools we were using. We were convinced that, when we migrated the code to the newest version of the framework we were using, we would get more customers and things would work great from then onwards. Yes, I know how ridiculous this sounds now. But we made it to "modernize" the tooling, and guess what? the code was still the same: a hard-to-maintain mess that would break if you stared at it for too long. Turns out that using more modern tools doesn't make your code better if the code is poorly written in the first place.

What do I want to say with this? That I believe **your time is better used writing quality code than arguing about what tools will make your application successful. Well architected and modularized code** with a great suite of automated tests that, apart from making sure the code works and can be refactored safely, also serves as documentation, **beats any tooling**. That is not to say the tools don't matter, because they obviously do. Ask a carpenter if they can be productive using a rusty hammer or a blunt saw.

For this book, I've tried to choose tools that are mature and that most frontend developers might be familiar with. Choosing the newest, shiniest or trendiest tool wasn't my goal.

I want to teach you how frontend frameworks work, and the truth is, that most tools work perfectly fine for this purpose. The knowledge that I'll teach you in this book transcends the tools we choose to use. So, having said this, if you have a preference for a tool and some experience with it, feel free to use it instead of the ones I'll recommend here.

A.4.1 Package manager—NPM

We'll use the [NPM package manager](#) to create the project and run the scripts. If you're more comfortable with *yarn* or *pnpm*, you can use them instead. They work very similarly, so you shouldn't have any problems using your preferred one.

We'll use NPM [workspaces](#), which were introduced in version 7, so you want to make sure you have at least version 7 installed:

```
$ npm --version
8.19.2
```

Both *yarn* and *pnpm* also support workspaces, so you can use them as well. (In fact, they introduced workspaces before NPM did.)

A.4.2 Bundler—Rollup

To bundle the JavaScript code together into a single file, we'll use [Rollup](#). Rollup is very popular among JavaScript libraries, and it's very simple to use. If you prefer Webpack, Parcel or Vite, you can use them instead. You'll have to make sure you configure them to output a single ESM file (as we'll see later).

A.4.3 Linter—ESLint

To lint the code, we'll use [ESLint](#). It's a very popular linter, and it's very easy to configure. I'm a firm believer that static analysis tools are a must for any serious project where the quality of the code is important (as it always is the case).

ESLint will prevent us from having unused variables declared, imports that are not used, and many other things that can go wrong in our code. ESLint is super configurable, but most developers are happy with the default configuration: it's a good starting point. We'll use the default configuration for this book as well, but you can always change it to your liking. If there's a linting rule you deem important, you can add it to your configuration.

A.4.4 (Optional) Testing—Vitest

I won't be showing unit tests for the code which you'll write in this book to keep its size reasonable, but if you check the source code of the framework I wrote for the book, you'll see lots of them. (In fact, you can use them to better understand how the framework works. Tests are—when well written—a great source of documentation.) You might want to write tests yourself as well, to make sure the framework works as expected and you don't break it when you make new changes. Every serious project should have tests accompanying it and serving as a safety net as well as documentation.

I've worked a lot with [Jest](#); it's been my go-to testing framework for a long time, but I recently started using [Vitest](#) and decided to stick with it as it's orders of magnitude faster. The API is very similar to Jest, so you won't have any problems if you do decide to use it as well. If you want to use Jest, that'll do just fine.

A.4.5 The language—JavaScript

Saying we'll use JavaScript might seem obvious, but if I was writing the framework for myself, I'd use TypeScript without any hesitation. TypeScript is fantastic for large projects: types tell you a lot about the code, and the compiler will help you catch bugs before you even run the code (how many times have you accessed a property that didn't exist in an object using JavaScript? Does `TypeError` send shivers down your spine?). Non-typed languages are great for scripting, but for large projects, I'd recommend having a compiler as your companion.

So why am I using JavaScript for this book? Because the code tends to be shorter without types, and what I want to do here is teach you the principles of how frontend frameworks work, principles that apply equally well to both JavaScript and TypeScript. I prefer to use JavaScript because the code listings will be shorter and I can get to the point faster, and thus teach you more efficiently.

As with the previous tools, if you feel strongly about using TypeScript, you can use it instead of JavaScript. You'll need to figure out the types yourself, but that's part of the fun, right? Also, don't forget to setup the compilation step in your build process.

A.5 Read the docs

Explaining how to configure the tools you'll use in detail, and how they work, is out of the scope of this book. I want to encourage you to go to their websites or repository pages and read the documentation. One great thing about frontend tools is that they tend to be extremely well documented. In fact, I'm convinced that they compete against each other to see which one has the best documentation.

If you're not familiar with any of the tools you'll be using, take some time to read the documentation. That'll save you a lot of time in the long run. I'm a firm believer that developers should strive to understand the tools they use, and not just use them blindly (that's what got me into understanding how frontend frameworks work in the first place—which explains why I'm writing this book).

One of the most important lessons I've learned over the years is that taking the time to read the documentation is a great investment of your time. Reading the docs before you start using a new tool or library saves you the time you'd spend trying to figure out how to do something that, somewhere in the documentation, someone has taken the time and care to explain in detail. The time that you end up spending trying to figure things out on your own, and searching StackOverflow for answers, is—in my personal experience—usually greater than the time you'd have spent should you read the documentation upfront.

6 hours of debugging can save you 5 minutes of reading documentation

— Jakob (@jcsrb)
Twitter

Be a good developer—read the docs. (Oh well, or at least ask ChatGPT a well-structured question.)

A.6 Structure of the project

Let's briefly go over the structure of the project where you'll write the code for your framework. Your framework you'll write in this book will consist of a single package ([NPM workspace](#)): *runtime*. This package is the framework itself.

Note

The advanced topics you can read in the repository's wiki add more packages to the project. This is the reason you want to structure your project using NPM workspaces. It might seem silly to use workspaces when the project only consists of a single package, but it'll make sense when you add more packages to the project.

Note

If you're not familiar with the concept of NPM workspaces, you can read more about at <https://docs.npmjs.com/cli/v9/using-npm/workspaces>. It's important that you understand how they work, so the structure of the project makes sense to you and you can follow along the instructions that follow.

The folder structure of the project will be as follows:

```
examples/  
packages/  
└─ runtime/
```

When you add more packages (as part of the advanced topics, outside this book), they'll be inside the *packages* directory as well:

```
examples/  
packages/  
├─ compiler/  
├─ loader/  
└─ runtime/
```

Each package has its own *package.json* file, with its dependencies, and it's bundled separately. It's effectively three separate projects, but they are all part of the same repository. This project structure is very common these days, and it's called a *monorepo*.

The packages you include will define the same scripts:

- *build* — builds the package, bundling all the JavaScript files into a single file, which is published to NPM.
- *test* — runs the automated tests in watch mode.
- *test:run* — runs the automated tests once.
- *lint* — runs the linter to detect issues in your code.
- *lint:fix* — runs the linter and automatically fixes the issues it can.
- *prepack* — a special [life cycle script](#) that runs before the package is published to NPM. It's used to make sure the package is built before being published.

Caution

The aforementioned scripts are defined in each of the three packages' *package.json* files (*packages/runtime*, for example), not in the root *package.json* file. Bear this in mind in case you decide to configure the project yourself, because there will be two *package.json* files in total. (And one more per package you add to the project.)

A.7 Finding a name for your framework

Before you create the project or write any code, you need a name for your framework. Be creative! It needs to be a name that no one else is using in NPM (you will be publishing your framework to NPM), so you need to be original. If you're not sure what to call it, you can simply call it *<your name>-fe-fwk* (your name followed by *-fe-fwk*). You want to make sure the name is unique, so check if it's available on npmjs.com, by adding it to the URL:

www.npmjs.com/package/<your framework name>

If the URL displays the *404—not found* page, that means nobody is using that name for a Node JS package yet, so you're good to go; you can use that name.

Important

I will use the `<fwk-name>` placeholder to refer to the name of your framework in the sections that follow. Whenever you see `<fwk-name>` in a command, you should replace it with the name you chose for your framework.

Let's now create the project. Remember that you have two options. If you want to configure things yourself, you want to skip the next section and jump to section [A.9](#). If you want to use the CLI tool to get started quickly, read the next section.

A.8 Option A—Using the CLI tool

Using the CLI tool I created for the book is the fastest way to get started. It'll save you the time it takes to create and configure the project from scratch, so you can start writing code right away. You just need to open your terminal, move to the directory where you want the project to be created, and run the following command:

```
$ npx fe-fwk-cli init <fwk-name>
```

With `npx` you don't even need to install the CLI tool locally, it will be downloaded and executed automatically.

When the command finishes executing, it'll instruct you to `cd` in to the project directory and run `npm install` to install the dependencies:

```
$ cd <fwk-name>
$ npm install
```

That's it! You can now jump to the [A.10](#) section to learn how to publish your framework to NPM.

A.9 Option B—Configuring the project from scratch

Note

If you created your project using the CLI tool, you can skip this section.

To create the project yourself, you first want to create a directory for it. Using the command line, you can do that by running the following command:

```
$ mkdir <fwk-name>
```

Then, you want to initialize an NPM project in that directory:

```
$ cd <fwk-name>
$ npm init -y
```

This command creates a *package.json* file in the directory. There are a few edits you need to make to the file. You want to edit the `description` field to something like:

```
"description": "A project to learn how to write a frontend
framework"
```

Then, you want to make this package private, so you don't accidentally publish it to NPM. It's the workspace packages you'll create in a minute that you'll publish to NPM, not the parent project itself:

```
"private": true
```

You also want to change the name of this package to *<fwk-name>-project* to avoid conflicts with the name of your framework (which you'll use to name the *runtime* package you create in the next section). Every NPM package in the repository requires a unique name, and you want to reserve the name you chose for your framework for the *runtime* package.

Append *-project* to the `name` field:

```
"name": "<fwk-name>-project"
```

Finally, you want to add a `workspaces` field to the file, with an array of the directories where you'll create the packages your project will consist of (so far, only the *runtime* package):

```
"workspaces": [
  "packages/runtime"
```

```
]
```

Your *package.json* file should look similar to this:

```
{
  "name": "<fwk-name>-project",
  "version": "1.0.0",
  "description": "A project to learn how to write a frontend
framework",
  "private": true,
  "workspaces": [
    "packages/runtime"
  ]
}
```

You might have other fields, such as `author`, `license`, etc., but the ones in the previous snippet are the ones you need to make sure are there. Let's now create a directory in your project where you can add example applications to test your framework.

A.9.1 The examples folder

Throughout the book, you'll be improving your framework and adding new features to it. Each time you add a new feature, you'll want to test it by using it in example applications. Here, you'll configure a directory where you can add these applications, and a script to serve them locally.

Create an *examples* directory at the root of your project:

```
$ mkdir examples
```

Tip

While the examples folder remains empty, that is, before you write any example application, you might want to add a *.gitkeep* file to it, so that git picks up the directory and includes it in the repository. (Git doesn't track empty directories.) As soon as you put a file in the directory, you can remove the *.gitkeep* file, but keeping it doesn't hurt anyway.

To keep the *examples* directory tidy, you want to create a subdirectory for each chapter: *examples/ch02*, *examples/ch03*, etc. Each subdirectory will contain the example applications using the framework resulting from the chapter, which allows you to see the progress in how the framework becomes more and more powerful, and easier to use. You don't need to create the subdirectories now, you'll create them as you need them in the book.

Now you need a script to serve the example applications. Your applications will consist of an entry HTML file, which loads other files, such as JavaScript files, CSS files, and images. For the browser to be able to load these files, you need to serve them from a web server. The `http-server` package is a very simple web server that you can use to serve the example applications.

In the *package.json* file, add the following script:

```
"scripts": {  
  "serve:examples": "npx http-server . -o ./examples/"  
}
```

Important

This is the only script that you need to add to the project root *package.json* file. All the other scripts you'll add to the project will be added to the *package.json* files of the packages you'll create in the next sections. Bear this in mind. You won't add any other script to the root *package.json* file.

The project doesn't require the `http-server` package installed, as you're using `npx` to run it. The `-o` flag tells the server to open the browser and navigate to the specified directory, in the case of the previous command, the *examples* directory.

Your project should have the following structure so far:

```
<fwk-name>/  
├─ package.json  
├─ examples/  
│   └─ .gitkeep
```

Great—let’s now create the three packages that will make up your framework code.

A.9.2 Creating the runtime packages

As we’ve already discussed, your framework will consist of a single package: *runtime*. In the advanced topics published separately in the repository’s wiki, you’ll add more packages to the project, but for now, you’ll only create the *runtime* package.

Before you create the package, you need to create a directory where you’ll put it; the *packages* directory you specified in the `workspaces` field of the *package.json* file.

Make sure your terminal is inside the project’s root directory by running the `pwd` command:

```
$ pwd
```

The output should be similar to this (where *path/to/your/framework* is the path to the directory where you created the project):

```
path/to/your/framework/<fwk-name>
```

If your terminal is not in the project’s root directory, use the `cd` command to navigate to it. Then, create a *packages* directory:

```
$ mkdir packages
```

Let’s create the *runtime* package.

The runtime package

First, make sure you `cd` into the *packages* folder:

```
$ cd packages
```

Then create the folder for the *runtime* package and `cd` into it:

```
$ mkdir runtime
$ cd runtime
```

Next, initialize an NPM project in the *runtime* folder:

```
$ npm init -y
```

Open the *package.json* file that was created for you and make sure the following fields have the right values (you can leave the other fields as they are):

```
{
  "name": "<fwk-name>",
  "version": "1.0.0",
  "main": "dist/<fwk-name>.js",
  "files": [
    "dist/<fwk-name>.js"
  ]
}
```

Remember to replace *<fwk-name>* with the name of your framework. The *main* field specifies the entry point of the package, which is the file that will be loaded when you *import* the package. This means that, when you import code from this package like so:

```
import { something } from '<fwk-name>'
```

JavaScript resolves the path to the file specified in the *main* field. You've told NPM that the file is the *dist/<fwk-name>.js* file, which is the bundled file containing all the code for the *runtime* package. This file will be generated by Rollup, by running the *build* script you'll add to the *package.json* file in the next section.

The *files* field specifies the files that will be included in the package when you publish it to the NPM repository. The only file that you want to include is the bundled file, so you've specified that in the *files* field. Files like the README, the LICENSE, and the *package.json* file are automatically included in the package, so you don't need to include them here.

Your project should have the following structure (in bold font is what you've just created):

```
<fwk-name>/
├── package.json
├── examples/
│   └── .gitkeep
└── packages/
    ├── runtime/
    │   └── package.json
```

Let's now add Rollup to bundle the framework code.

Installing and configuring Rollup

[Rollup](#) is the bundler that you'll use to bundle your framework code. It's very simple to configure and use.

Let's install Rollup. Make sure your terminal is inside the *runtime* package directory by running the `pwd` command:

```
$ pwd
```

The output should be similar to this:

```
path/to/your/framework/<fwk-name>/packages/runtime
```

If your terminal is not inside the *packages/runtime* directory, use the `cd` command to navigate to it. Install the `rollup` package by running the following command:

```
$ npm install --save-dev rollup
```

You also want to install two plugins for Rollup:

- `rollup-plugin-cleaner`: to remove comments from the generated bundle.
- `rollup-plugin-filesize`: to display the size of the generated bundle.

Install them with the following command:

```
$ npm install --save-dev rollup-plugin-cleaner rollup-plugin-filesize
```

Now, you need to configure Rollup. Create a *rollup.config.mjs* file (note the *.mjs* extension, to tell Node JS this file should be treated as an ES module and thus can use the `import` syntax) in the *runtime* folder and add the following code:

```
import cleanup from 'rollup-plugin-cleanup'
import filesize from 'rollup-plugin-filesize'

export default {
  input: 'src/index.js', #1
  plugins: [cleanup()], #2
  output: [
    {
      file: 'dist/<fwk-name>.js', #3
      format: 'esm', #4
      plugins: [filesize()], #5
    },
  ],
}
```

The configuration, explained in plain english, means the following:

- The entry point of the framework code is the *src/index.js* file: starting from this file, Rollup will bundle all the code that is imported from it.
- The comments in the source code should be removed from the generated bundle (they occupy space and are not needed for the code to execute). This is what the `plugin-rollup-cleanup` plugin does.
- The generated bundle should be an ES module (using `import/export` syntax, supported by all major browsers) that is saved in the *dist* folder, as *<fwk-name>.js*.
- We want the size of the generated bundle to be displayed in the terminal to keep an eye on it and make sure it doesn't grow too much. This is what the `plugin-rollup-filesize` plugin does.

Now add a script to the *package.json* file (the one inside the *runtime* package) to run Rollup and bundle all the code, and another one to run it automatically before publishing the package:

```
"scripts": {
  "prepack": "npm run build",
  "build": "rollup -c"
}
```

The `prepack` script is a special script that is run automatically by NPM before publishing the package, which you do by running the `npm publish` command (more on this later). This makes sure that, whenever you publish a new version of the package, the bundle is generated. The `prepack` script simply runs the `build` script, which in turn runs Rollup.

To run Rollup, you simply call the `rollup` command and pass the `-c` flag to tell it to use the `rollup.config.mjs` file as configuration. (If you don't pass a specific file to the `-c` flag, Rollup will look for a `rollup.config.js` or `rollup.config.mjs` file.)

Before you test the `build` command, you need to create the `src` folder and the `src/index.js` file. You can do that with the following commands:

```
$ mkdir src
$ touch src/index.js
```

(In windows shell `touch` will not work use `call > filename` instead.)

Inside the `src/index.js` file, add the following code:

```
console.log('This will soon be a frontend framework!')
```

You can now run the `build` command to bundle the code:

```
$ npm run build
```

This command processes all the files imported from the `src/index.js` file (none at the moment) and bundles them into the `dist` folder. The output in your terminal should look similar to the following:

```
src/index.js → dist/<fwk-name>.js...
```

Destination: dist/<fwk-name>.js
Bundle Size: 56 B
Minified Size: 55 B
Gzipped Size: 73 B

```
created dist/<fwk-name>.js in 62ms
```


Recall that, instead of `<fwk-name>`, you should use the name of your framework. That rectangle in the middle of the output is the `rollup-plugin-filesize` plugin in action. It's reporting the size of the generated bundle (56 bytes in this case), and also the size it'd have if the file was minified and gzipped. We won't be minifying the code for this book, as the framework is going to be small anyway, but for a production-ready framework, you should definitely do it. A minified JavaScript can be loaded faster by the browser, and thus improve the [TTI \(Time to Interactive\)](#) of the web application using it.

A new file, `dist/<fwk-name>.js`, has been created in the `dist` folder. If you open it, you'll see it just contains the `console.log()` statement you added to the `src/index.js` file.

Great! — Let's now install and configure ESLint.

Installing and configuring ESLint

[ESLint](#) is a linter that will help you write better code. It analyzes the code you write and reports any errors or potential problems.

To install it, run the following command:

```
$ npm install --save-dev eslint
```

You'll use the ESLint recommended configuration, which is a set of rules that ESLint will enforce in your code. Create a `.eslintrc.js` file inside the `packages/runtime` directory with the following content:

```
module.exports = {
  env: {
    browser: true,
    es2021: true,
  },
  extends: 'eslint:recommended',
  overrides: [],
  parserOptions: {
    ecmaVersion: 'latest',
    sourceType: 'module',
  },
},
```

```
    rules: {},  
  }
```

And lastly, add the following two scripts to the *package.json* file:

```
"scripts": {  
  "prepack": "npm run build",  
  "build": "rollup -c",  
  "lint": "eslint src",  
  "lint:fix": "eslint src --fix"  
}
```

You can run the `lint` script to check the code for errors, and the `lint:fix` script to automatically fix some of them, those that ESLint knows the fix for. I won't be showing the output of the `lint` script in the book, but you can run it yourself to see what it reports as you write your code.

Let's now install Vitest to run the automated tests.

Installing Vitest (optional)

As I've mentioned, I used a lot of tests while developing the code for this book, but I won't be showing them in the book. You can take a look at them in the source code of the framework and try to write your own as you follow along. Having tests in your code will help you make sure things work as expected as you move forward with the development of your framework.

We'll use [Vitest](#) to run the tests. To install Vitest, run the following command:

```
$ npm install --save-dev vitest
```

Tests can run in different environments, such as Node JS, JSDOM, or a real browser. Because we will use the Document API to create DOM elements, we'll use JSDOM as the environment. (If you want to know more about JSDOM, please refer to their repository: <https://github.com/jsdom/jsdom/>.)

To install JSDOM, run the following command:

```
$ npm install --save-dev jsdom
```

With Vitest and JSDOM installed, you can now create the *vitest.config.js* file with the following content:

```
import { defineConfig } from 'vitest/config'

export default defineConfig({
  test: {
    reporters: 'verbose',
    environment: 'jsdom',
  },
})
```

This configuration tells Vitest to use the JSDOM environment and to use the *verbose* reporter, which outputs a very descriptive report of the tests.

You should place your test files inside the *src/__tests__* folder, and name them **.test.js* for Vitest to find them. Go ahead and create the *src/__tests__* folder:

```
$ mkdir src/__tests__
```

Inside, you can create a *sample.test.js* file with the following content:

```
import { expect, test } from 'vitest'

test('sample test', () => {
  expect(1).toBe(1)
})
```

Now, add the following two scripts to the *package.json* file:

```
"scripts": {
  "prepack": "npm run build",
  "build": "rollup -c",
  "lint": "eslint src",
  "lint:fix": "eslint src --fix",
  "test": "vitest",
  "test:run": "vitest run"
}
```

The `test` script runs the tests in watch mode, and the `test:run` script runs the tests once and exits. Tests in watch mode are handy when you're working

with TDD (Test Driven Development), as they run again every time you make changes in the test file or the code being tested.

Give a try to the tests by running the following command:

```
$ npm run test:run
```

You should see the following output:

```
✓ src/__tests__/sample.test.js (1)
  ✓ sample test

Test Files   1 passed (1)
Tests       1 passed (1)
Start at    15:50:11
Duration    1.47s (transform 436ms, setup 1ms, collect 20ms,
tests 3ms)
```

You can run individual tests passing the name or path of the test file as an argument to the `test:run` and `test` scripts. All the tests matching the name or path will be run. For example, to run the *sample.test.js* test, you can run the following command:

```
$ npm run test:run sample
```

This will run only the *sample.test.js* test and produce the following output:

```
✓ src/__tests__/sample.test.js (1)
  ✓ sample test

Test Files   1 passed (1)
Tests       1 passed (1)
Start at    15:55:42
Duration    1.28s (transform 429ms, setup 0ms, collect 32ms,
tests 3ms)
```

Your project structure should look like the following at this point (in bold font are the files and folders that you've created):

```
<fwk-name>/
├── package.json
├── examples/
│   └── .gitkeep
```

```
└─ packages/
   └─ runtime/
      ├── package.json
      ├── rollup.config.mjs
      ├── .eslintrc.js
      ├── vitest.config.js
      └─ src/
         ├── index.js
         └─ tests/
            └─ sample.test.js
```

Your *runtime* package is ready for you to start writing code. Let's see how you can publish it to NPM.

A.10 Publishing your framework to NPM

As exciting as it is to develop your own framework, it's even more exciting to share it with the world. NPM allows us to ship our package with just one simple command:

```
$ npm publish
```

But before you can do that, you need to create an account on NPM and log in to it from the command line. Let's do that now. If you don't plan to publish your framework, you can skip this section.

A.10.1 Creating an NPM account

To create an NPM account, go to <https://www.npmjs.com/signup> and fill out the form. It's that simple, and it's free.

A.10.2 Logging in to NPM

To log in to NPM in your terminal, run the following command:

```
$ npm login
```

and follow the prompts. To make sure you're logged in, run the following command:

```
$ npm whoami
```

You should see your username printed in the terminal.

A.10.3 Publishing your framework

To publish your framework, you need to make sure your terminal is in the right directory: inside *packages/runtime*:

```
$ cd runtime
```

Your terminal's working directory should now be *packages/runtime*, highlighted in bold font in this listing:

```
<fwk-name>/
├── package.json
├── examples/
│   └── .gitkeep
└── packages/
    ├── runtime/ #1
    ├── compiler/
    └── loader/
```

Warning

Make sure you're not in the root folder of the project. The root folder's *package.json* file has the *private* field set to *true*, which means that it's not meant to be published to NPM. It's the *runtime* package that you want to publish to NPM.

Remember that the *runtime* package is the code for the framework (what you want to publish), and the root of the project is just the monorepo containing the *runtime* and other packages (not to be published to NPM).

With your terminal in the *runtime* directory, run the following command:

```
$ npm publish
```

You might be wondering what gets published to NPM. There are some files in your project that always get published, and these are the *package.json* file,

the *README.md* file, and the *LICENSE* file. Apart from these, you can specify which files to publish in the *files* field of the *package.json* file. If you recall from a previous section, the *files* field in the *package.json* file of the *runtime* package looks like this:

```
"files": [  
  "dist/<fwk-name>.js"  
],
```

So, you're only publishing the *dist/<fwk-name>.js* file, the bundled version of your framework. The source files inside the *src/* folder are not published.

The package is published with the version specified in the *package.json* file's *version* field. Bear in mind that you can't publish a package with the same version twice. Throughout the book, we'll increment the version number every time we publish a new version of the framework.

A.11 Using a CDN to import the framework

Once you've published your framework to NPM, you can create a Node JS project and install it as a dependency:

```
$ npm install <fwk-name>
```

This is great if you plan to set up an NPM project with a bundler (such as Rollup or Webpack) configured to bundle your application and the framework together. This is what you typically do when you're building a production application with a frontend framework. But for small projects or quick experiments that use only a handful of files—as those you'll work on in this book—it's simpler to import the framework directly from the *dist/* directory in your project, or from a *Content Delivery Network* (CDN).

One free CDN that you can use is unpkg.com. Everything that's published to NPM is also available on unpkg.com, so once you've published your framework to NPM, you can import it from there, like so:

```
import { createApp, h } from 'https://unpkg.com/<fwk-name>'
```

In fact, if you browse to <https://unpkg.com/fwk-name>, you'll see the *dist/<fwk-name>.js* file you published in NPM. This is what the CDN serves to your browser when you import the framework from there. If you don't specify a version, it'll serve the latest version of the package. If you want to use a specific version of your framework, you can specify it in the URL, like so:

```
import { createApp, h } from 'https://unpkg.com/<fwk-name>@1.0.0'
```

Or, if you want to use the last minor version of the framework, you can just request a major version, like so:

```
import { createApp, h } from 'https://unpkg.com/<fwk-name>@1'
```

I recommend you use unpkg using the versioned URL, so that you don't have to worry about breaking changes in the framework that might break your examples as you publish new versions of it. If you import the framework directly from the *dist/* folder in your project, every time you bundle a new version that's not backwards compatible, your example applications will break. You could overcome this by instructing Rollup to include the framework version in the bundled file name, but I'll be using unpkg in this book.

You're all set! Time to start developing your framework.

FROM
SCRATCH

BUILD A

Frontend Web Framework

Angel Sola Orbaiceta

MEAP



MANNING